

Foundations of Modern Algorithmic Trading

Volume II: Machine Learning for Trading (Chapters 11 to 15)

Alejandro Reynoso

Chief Scientist, DeFi Capital Research;
External Lecturer, Judge Business School, Cambridge

January 5, 2026

Preface to Volume II

Volume I built the foundations that make any claim in systematic trading credible. It began with the real nature of market data: timestamps, calendars, corporate actions, and the subtle difference between an observed price and a tradable one. It then constructed the quantitative toolbox in a deliberately transparent way, emphasizing first-principles implementation and time-aware computation. From there, it formalized the basic objects of returns and risk, showing how uncertainty becomes measurable (variance, covariance, drawdown) and how multi-asset holdings become portfolio-level exposures. Finally, it moved from theory to disciplined simulation: feature engineering as a causal pipeline, backtesting as a calibrated instrument rather than a performance trophy, and classic strategy families—trend following, mean reversion, cross-sectional factors, and volatility targeting—as controlled experiments that teach what can and cannot survive realistic assumptions. In short, Volume I established the non-negotiables: preserve the timeline, charge yourself for reality, and log what you did so that results can be reproduced and audited.

Volume II introduces machine learning, but it does so under the constraints that Volume I makes unavoidable. In many domains, machine learning is primarily a pattern recognition problem: shuffle the data, fit a model, and report accuracy. Trading is different. The data are non-stationary, labels are noisy, and evaluation can be accidentally generous in countless ways. The objective of Volume II is therefore not to treat machine learning as a shortcut to alpha, but as a disciplined extension of signal research: a systematic way to propose, regularize, select, and evaluate predictive mappings from information available at decision time to outcomes realized only in the future. The central question is not “Can the model fit?” but “Can the model generalize out of time, across regimes, net of realistic frictions, and against strong baselines?”

Chapter 11 starts with supervised learning framed in the language of returns forecasting: targets, horizons, losses, and the operational meaning of a prediction once it becomes a position. It emphasizes that most of the work is not the algorithm, but the experiment design: how you split time, how you avoid leakage, how you select features without peeking, and how you translate probabilistic forecasts into trading rules. Chapter 12 then treats regularization and model selection as governance tools as much as statistical techniques. Regularization is not only about reducing variance; it is about constraining models to behave in ways that are stable enough to survive regime shifts. Hyperparameters are not merely knobs; they are research decisions that must be tracked, justified, and evaluated in walk-forward protocols that respect market chronology. The chapter makes model selection explicit: you are selecting not only performance, but a bias-variance posture that reflects the uncertainty of the market environment.

Chapter 13 brings deep learning into the same disciplined framework. Neural networks are presented as function approximators that can be useful when feature interactions are complex,

but they are not treated as inherently superior. The focus is on why and when deep models help, what they implicitly assume, and how to prevent them from exploiting accidental structure in the dataset rather than real tradable information. The chapter is careful about capacity control, stability, and the operational constraints that make many high-variance models fragile in finance. Chapter 14 extends the conversation from prediction to structure by introducing regime detection and Hidden Markov Models. The point is not to create an oracle for market states, but to provide a formal language for non-stationarity: if markets behave differently in different regimes, then both forecasting and risk management must become conditional. Finally, Chapter 15 introduces event-driven modeling and news/sentiment as a special case of machine learning in trading: information arrives irregularly, the market reacts quickly, and the risks of spurious correlation are elevated. The goal is to show how to integrate event features without collapsing time discipline, and how to evaluate such models without confusing attention with information.

By the end of Volume II, you should be able to build forecasting experiments that are rigorous enough to be believed and modular enough to be improved. You should have a toolkit for supervised prediction, a governance posture for model selection, and a sober understanding of what deep learning can add—and what it cannot guarantee. Most importantly, you should have internalized that machine learning does not replace the need for careful simulation; it intensifies it. A better model is still embedded in a trading system, and that system must survive costs, constraints, and operational realities.

Volume III completes the arc by taking these signals and models into the world where trading becomes real: portfolios, leverage, and risk budgets; execution, slippage, and microstructure; sequential decision-making under costs; multi-strategy systems that allocate capital across horizons; and, finally, the governance and infrastructure that separate research code from deployable platforms. Volume III is where the question changes from “Can we predict?” to “Can we allocate, execute, monitor, and explain decisions in a way that is robust, auditable, and scalable?” If Volume I is about truthful measurement and Volume II is about disciplined prediction, Volume III is about responsible deployment.

Contents

Preface to Volume II	1
11 Supervised Learning for Forecasting Returns	6
11.1 Position in the Book	8
11.2 Why Forecasting Returns Is a Special ML Problem	12
11.3 Problem Setup and Notation	17
11.4 Target Engineering and Label Design	24
11.5 Features: Connecting Chapter 05 to Predictive Modeling	30
11.6 Baseline Models You Must Beat	37
11.7 Loss Functions and Objectives Aligned With Trading	41
11.8 Evaluation Under Time: Walk-Forward Research Protocol	45
11.9 From Forecast to Trade: A Minimal Signal-to-Position Mapping	50
11.10 Failure Modes and Diagnostic Playbook	54
11.11 Governance-Native Checklist for Supervised Trading Models	58
11.12 Conclusion	64
12 Regularization, Hyperparameters, Model Selection	67
12.1 Position in the Book	69
12.2 Regularization from First Principles	72
12.3 Canonical Penalties and Their Trading Interpretations	78
12.4 Hyperparameters Are Part of the Model	79
12.5 Validation Under Time: Designing Splits That Preserve Causality	85
12.6 Model Selection Procedures	91
12.7 Selection Bias: When the Research Process Overfits	96
12.8 Implementation Notes (Within Chapter 12 Scope)	101
12.9 Governance-Native Research Checklist for Regularization and Model Selection . .	107
12.10 Conclusion	114
13 Neural Nets & Deep Learning in Trading	119
13.1 Position in the Book	121
13.2 Why Neural Nets in Trading, and Why This Is Hard	123
13.3 Neural Networks as Composed Functions	127
13.4 Training Dynamics: Optimization, Gradients, and Stability	137
13.5 Architectures for Market Data (and Their Inductive Biases)	146

13.6	Representing Market Information for Neural Nets	154
13.7	Time-Aware Validation and Model Selection for Deep Nets	162
13.8	Regularization for Neural Nets	171
13.9	Diagnostics: Understanding What the Network Learned	177
13.10	Governance-Native Checklist for Deep Learning in Trading	186
13.11	Practical Patterns and Anti-Patterns (Trading Reality Check)	195
13.12	Conclusion	200
14	Regime Detection & HMMs	204
14.1	Position in the Book	206
14.2	Motivation: Why “Regimes” Exist in Trading Systems	208
14.3	A Minimal Formal Definition of a Regime Model	213
14.4	Hidden Markov Models: Model Specification	219
14.5	Inference I: Filtering, Prediction, and the Forward Algorithm	227
14.6	Inference II: Smoothing and the Backward Algorithm	234
14.7	Learning HMM Parameters	240
14.8	Choosing the Number of Regimes	248
14.9	Feature Design for Regime Detection (Within Ch. 14 Scope)	253
14.10	From Regime Beliefs to Trading Decisions	258
14.11	Evaluation: What Counts as Evidence a Regime Model Helps?	265
14.12	Implementation Notes (Within Chapter 14 Scope)	270
14.13	Governance-Native Checklist for Regime Detection Pipelines	276
14.14	Conclusion	284
15	Event-Driven & News/Sentiment	288
15.1	Position in the Book	290
15.2	Motivation: Why Events and Text Are Different From Prices	293
15.3	A Minimal Formalism for Event-Driven Trading	297
15.4	Taxonomy of Events (Within Book Scope)	306
15.5	Data Sources and Their Hidden Biases	312
15.6	Building an Event Log: The Governance-Native Starting Point	318
15.7	From Text to Numbers: Feature Engineering Without Magic	327
15.8	Event Study Mechanics for Trading	336
15.9	Backtesting Event-Driven Signals (Within Chapter 06 Discipline)	342
15.10	Risk and Portfolio Integration of Event Signals (Preview Only)	350
15.11	Failure Modes and Anti-Patterns	355
15.12	Governance-Native Checklist for Event-Driven & Sentiment Pipelines	361
15.13	Conclusion	369

Machine Learning

Chapter 11

Supervised Learning for Forecasting Returns

Abstract

Supervised learning enters algorithmic trading with an uncomfortable truth: forecasting returns is rarely difficult to do *computationally*, but it is notoriously difficult to do *honestly*. This chapter introduces supervised learning as a governed forecasting layer inside a trading system, not as a standalone statistical exercise. We formalize the causal prediction problem $X_t \mapsto \hat{y}_{t+h}$, where features X_t must be available at decision time and labels y_{t+h} are defined at a future horizon. We then treat label design as an explicit engineering choice—horizon selection, return definitions, excess vs raw targets, and continuous (regression) versus directional (classification) formulations—and show how these choices determine what a model can learn and what a strategy can plausibly harvest.

The chapter establishes a “must beat” baseline discipline, beginning with null predictors and simple linear models, and introduces a time-ordered evaluation protocol built around walk-forward training and testing. Forecast quality is assessed with both statistical metrics (error, correlation, directional accuracy) and economically relevant diagnostics (stability across subperiods, turnover pressure, and sensitivity to risk targeting). Throughout, the primary failure mode is treated as leakage: we enumerate common pathways, impose causality gates as hard invariants, and specify a minimal artifact bundle (data fingerprints, feature/label manifests, split protocols, model specs, and lineage hashes) required for reproducible claims. The chapter concludes by mapping forecasts to positions via minimal, transparent rules, while postponing regularization and selection (Chapter 12), deep learning (Chapter 13), regime models (Chapter 14), event-driven features (Chapter 15), portfolio optimization (Chapters 16–17), and microstructure costs (Chapter 18).

11.1 Position in the Book

11.1.1 Prerequisites from Chapters 01–10

Chapter 11 is the first point in the book where we allow ourselves to say, explicitly and without apology: “we will fit a model.” Everything in Chapters 01–10 has been preparing the reader to do that without confusing a clean-looking regression output for a reliable trading edge. The prerequisite material is not “nice to have”; it is the scaffolding that prevents supervised learning from degenerating into (i) accidental look-ahead, (ii) unpriced risk exposure, or (iii) a backtest that is merely an overfit story about the past.

Chapter 01 establishes the logic of algorithmic trading as an information-to-decision pipeline. We treated markets as noisy information engines rather than stationary statistical generators, and we decomposed a trading system into signal, execution, risk, and governance components. That decomposition matters here because supervised learning sits inside the signal engine, not above the entire system. A return-forecasting model can only be evaluated within a decision protocol that respects when information becomes available, how trades are placed, and how exposures are constrained. Put differently: Chapter 01 supplies the conceptual “wiring diagram” that tells us where a predictor belongs and what it is not allowed to claim.

Chapter 02 provides the practical foundation: a Python quantitative toolbox built from first principles, without leaning on high-level abstractions that hide time alignment and rolling-window mechanics. Chapter 11 assumes you can compute returns, lags, rolling statistics, and simple transforms causally; you can construct train/test splits that preserve time ordering; and you can reproduce results deterministically with seeds, parameter registries, and run manifests. The prohibition against pandas is not aesthetic; it is pedagogical. It forces the reader to see that supervised learning for trading is not about calling `fit()` on a matrix, but about constructing that matrix correctly—with features that exist at decision time, labels that exist at horizon time, and splits that never allow future information to leak backward.

Chapter 03 contributes the core mathematics of returns, risk, and portfolio logic. Even though Chapter 11 focuses on prediction rather than optimization, it presupposes that the reader understands the difference between predicting a return and harvesting a return. A forecast that is weakly informative in a statistical sense can still be economically meaningful if it is translated into positions under sensible risk control; conversely, a statistically impressive model can be economically useless if it simply proxies for unpriced risk or if it produces excessive turnover. The language of expected return, variance, covariance, drawdowns, and risk-adjusted performance is the vocabulary we will use to interpret what a predictive model is actually doing.

Chapter 04 provides time-series anatomy: stationarity as an assumption to be tested rather than granted, serial dependence patterns, volatility clustering, heavy tails, and the difference between “predictable” and “tradable.” Supervised learning for returns is not a generic i.i.d. regression problem. It is a prediction problem where the data generating process may drift, where noise can dominate signal, and where error distributions are rarely Gaussian. Chapter 04 teaches the reader to expect regime changes, to distrust stable parameters, and to interpret performance decay over time as normal rather than surprising. This is crucial because supervised learning will often appear to “work” in-sample and then fail under drift; Chapter 04 gives the

conceptual permission to treat that failure as information about the environment rather than as a mere coding bug.

Chapter 05 supplies the engineering discipline for data pipelines and feature engineering. Chapter 11 depends on the idea that features are not a list of clever indicators but a governed artifact: defined, timed, computed causally, logged, and reproducible. Feature engineering is where most trading ML fails in practice, because leakage is easiest to introduce at this stage and hardest to detect later. Chapter 05 teaches you to think in terms of data lineage, manifests, and causal transforms; Chapter 11 will reuse that discipline to construct the supervised learning dataset $\{(X_t, y_{t+h})\}$ without contaminating X_t with information that only exists after time t .

Chapter 06 establishes backtesting and simulation. Chapter 11 does not re-teach backtesting mechanics; instead, it assumes the reader can conduct a walk-forward research protocol, can distinguish between in-sample and out-of-sample evaluation, and can interpret strategy metrics responsibly. The key prerequisite from Chapter 06 is the notion of event timing: signal time versus trade time. Without that, supervised learning becomes an attractive nuisance: the model appears accurate because the evaluation accidentally grants it access to the future. Chapter 06 also establishes the minimum standard for claiming that something “worked”: you must show that a process generalizes under time-ordered evaluation and that it survives reasonable perturbations.

Chapters 07–10 contribute the canonical strategy families and risk targeting logic that will quietly serve as baselines and as interpretive tools in Chapter 11. Trend following (Chapter 07) and mean reversion/pairs (Chapter 08) are not merely strategies; they are hypotheses about autocorrelation structure and mean dynamics that can be encoded as features or used as benchmark comparators. Factor and cross-sectional thinking (Chapter 09) introduces the idea that prediction can be about ranking and relative value rather than absolute return forecasts, even if the full portfolio construction for that ranking is postponed. Volatility modeling and risk targeting (Chapter 10) provides the essential wrapper: if a forecasting model outputs signals of varying strength, the system still needs a stable way to translate those signals into risk-managed exposures. In short: Chapters 07–10 supply both the “classical” baselines that supervised learning must beat and the risk discipline that prevents a predictor from becoming a leverage amplifier.

11.1.2 What This Chapter Adds

Chapter 11 introduces supervised learning as a disciplined forecasting layer within an algorithmic trading system. The central object is the causal mapping

$$X_t \mapsto \hat{y}_{t+h}, \quad (11.1)$$

where X_t is a feature vector that must be computable using only information available at decision time t , and y_{t+h} is a target defined at a future horizon $t + h$ (for example, a forward return, an excess return, or a directional label). This chapter adds a precise language for defining the prediction problem, a taxonomy of modeling tasks, and a research protocol that treats time as non-negotiable.

First, Chapter 11 formalizes the target-design problem. In trading, labels are not given by nature; they are constructed. Choosing h , choosing the return definition (simple vs log, raw vs excess, hedged vs unhedged), and choosing whether the target is continuous (regression), discrete (classification), or ordinal (ranking) are not minor details. They determine what the model can learn and how the signal will later be translated into a position. Chapter 11 therefore treats label engineering as a first-class design decision, with sanity checks and governance artifacts attached.

Second, Chapter 11 introduces the three supervised learning problem types that recur throughout the remainder of the book: regression (forecasting magnitude), classification (forecasting direction or threshold exceedance), and ranking (forecasting relative ordering across assets or opportunities). The ranking view is introduced carefully and conceptually, consistent with the book’s arc: we will not yet build a full cross-sectional portfolio optimizer here, but we will establish why ranking is often the natural object in finance—because absolute return levels are hard to forecast, while relative attractiveness can be more stable.

Third, the chapter emphasizes baselines and the “must beat” principle. Before trying sophisticated models, the reader learns to construct and evaluate null predictors, linear baselines, and simple heuristic comparators. This is not conservatism for its own sake; it is how we measure incremental value. If a machine learning model cannot beat a properly implemented baseline under the same walk-forward protocol, then it has not earned complexity.

Fourth, Chapter 11 establishes walk-forward evaluation as the default scientific method for supervised trading research. We operationalize expanding-window and rolling-window schemes, define what counts as training, validation, and test in a time series context, and treat performance stability as a core diagnostic. The chapter insists that you do not get to choose your splits arbitrarily; you must respect chronology, and you must state the split protocol as part of the experiment specification. This chapter thereby introduces the research hygiene that will be deepened in Chapter 12: the idea that model choice is itself a source of bias and must be governed.

Finally, Chapter 11 adds the conceptual bridge from forecast to trade. It introduces minimal mapping rules from $\hat{y}_{t+h}$ to a position w_t (thresholding, scaling, simple risk targeting), while explicitly labeling this bridge as a handshake with prior chapters (risk targeting from Chapter 10, backtest timing from Chapter 06). The goal is not to complete the entire execution and cost model here, but to ensure that the reader never evaluates forecasts in a vacuum: a forecast is only meaningful insofar as it can be embedded in a decision rule that respects constraints.

11.1.3 What This Chapter Postpones

To preserve the integrity of the 25-chapter arc, Chapter 11 draws a clear boundary around what it will *not* attempt to solve. The reader should leave this chapter with a reliable supervised learning research loop and an appreciation for the fragility of forecasting edges, but without the false confidence that comes from prematurely deploying a sophisticated ML stack.

The first and most important postponement is Chapter 12: regularization, hyperparameters, and model selection. Chapter 11 introduces supervised learning models as objects to be evaluated, but it does not yet perform a full treatment of complexity control. Questions such as: “How do

we choose between L1 and L2 penalties?” “How do we tune hyperparameters without leaking information?” “What is the correct analog of cross-validation under time ordering?” and “How do we account for multiple testing when we search over many model configurations?” are deferred to Chapter 12, where they can be treated with the seriousness they deserve. In the arc of the book, Chapter 11 teaches you to pose the problem correctly; Chapter 12 teaches you to avoid fooling yourself while solving it.

Second, deep learning is postponed to Chapter 13. While Chapter 11 may mention neural networks as part of the landscape, it does not introduce architectures, training dynamics, or representational learning. This is deliberate: deep learning adds both expressive power and a new set of failure modes (optimization instability, implicit leakage through preprocessing, excessive degrees of freedom). The book’s pedagogy requires that the reader first master causal datasets, baselines, and walk-forward evaluation before introducing models that can memorize patterns with ease.

Third, explicit regime modeling and Hidden Markov Models are postponed to Chapter 14. Chapter 11 will acknowledge non-stationarity and performance drift, but it will not yet formalize regimes or build regime-dependent predictors. That step requires a separate conceptual toolkit: latent state models, inference, and the interpretation of regime transitions. Chapter 11 simply motivates why regimes matter; Chapter 14 provides the machinery to model them.

Fourth, event-driven systems and news/sentiment are postponed to Chapter 15. Chapter 11 focuses on price/volume-derived features and the supervised learning mechanics that apply regardless of feature source. Incorporating text, events, and unstructured data introduces new timing issues (publication time, ingestion delays), new label definitions (event windows), and new evaluation pitfalls (selection bias through news availability). Those complexities are substantial enough to warrant their own chapter.

Fifth, portfolio construction and optimization are postponed to Chapters 16 and 17. Chapter 11 can map a forecast to a simple position, but it does not solve the question of how to allocate across many assets, how to manage factor exposures, how to enforce leverage and margin constraints, or how to optimize portfolios under estimation error. The book’s structure is intentional: forecasting is one component; portfolio construction is a distinct discipline that integrates multiple forecasts, constraints, and risk models.

Finally, the detailed treatment of transaction costs, slippage, and market microstructure is postponed to Chapter 18. Chapter 11 will warn that forecast performance can evaporate once costs are included, but it will not yet model those costs realistically. Cost modeling requires microstructure concepts, execution logic, and a careful simulation interface. Introducing them too early would obscure the core lesson of Chapter 11: a supervised learning model is only as credible as the causal dataset and the time-ordered evaluation protocol that produced it.

In summary, Chapter 11 is the gateway from handcrafted strategies to model-based forecasting, but it is deliberately *not* the gateway to “anything goes” machine learning. It inherits discipline from Chapters 01–10, contributes a supervised learning research loop that respects time and governance, and postpones the more powerful (and more dangerous) tools until the reader is equipped to use them without self-deception.

11.2 Why Forecasting Returns Is a Special ML Problem

11.2.1 Forecasting vs Explaining: predictive stance and operational constraints

Supervised learning in finance often begins with a misunderstanding about what the model is being asked to do. In many applied domains, the goal is primarily *explanation*: uncovering stable relationships that remain meaningful under interventions, or identifying features whose coefficients can be interpreted causally. In trading, the goal is almost always *prediction under constraints*. The system must map information available at a precise decision time to an actionable decision, and it must do so repeatedly in an environment where the mapping itself may drift. A forecasting model is not judged by how well it tells a plausible story about the past; it is judged by whether it can generate decisions that hold up under time-ordered evaluation, survive costs, and remain robust when the market changes its mind.

This predictive stance imposes an operational discipline that is alien to many standard machine learning pipelines. First, *information is timestamped*. A feature is not simply a function of a dataset; it is a function of what is known at time t . The difference between a feature that uses the close-to-close return for day t and one that uses intraday information that is only available after the close is not semantic; it is the difference between a tradable signal and a look-ahead artifact. Second, *predictions must be translated into trades*. Even if the model produces a scalar forecast $\hat{y}_{t+h}$, the system ultimately chooses a position w_t (size, direction, risk budget), and that position interacts with volatility, leverage limits, and portfolio constraints. A pure prediction metric can be misleading if it ignores how the forecast would be consumed operationally.

Third, *the unit of evaluation is not the observation; it is the research protocol*. In i.i.d. supervised learning, one can often shuffle, cross-validate, and treat each sample as exchangeable. In trading, the data is time-ordered, overlapping, and serially dependent. The relevant question is not whether the model fits well on a pooled dataset, but whether a model trained on the past can make predictions that remain useful in the future, across multiple contiguous subperiods. This is why walk-forward evaluation is not a detail but a scientific method: it operationalizes the only claim that matters in trading, namely that the predictor generalizes forward in time.

Finally, the predictive stance implies a subtle but important shift in how we interpret model “success.” A small predictive edge can be economically meaningful because trading systems compound weak signals over many decisions, but a small edge is also easily fabricated by accidental leakage, by implicit parameter tuning, or by exploiting transient artifacts. In other words, the profit opportunity is often too small to provide a loud signal of truth. This is why the operational constraints and the governance constraints become inseparable: we do not merely need models that forecast; we need models whose forecasting performance can be trusted.

11.2.2 Stylized facts that break naive ML assumptions

The temptation in return forecasting is to treat price data as just another dataset and to apply standard machine learning recipes: create features, split into train and test, fit a model, tune hyperparameters, and report accuracy. The problem is that the foundational assumptions behind

those recipes are systematically violated by financial time series. The violations are not minor. They are precisely the reasons why return forecasting is difficult and why trading edges are fragile.

The phrase “stylized facts” refers to empirical regularities observed across many markets and instruments. They do not constitute a full theory of markets, but they provide constraints on what a reasonable statistical model should expect to see. For supervised learning, these facts matter because they imply that the distribution of inputs and outputs is not stable, the noise is not well-behaved, and the signal is often near the limits of detectability. A naive ML pipeline that assumes stationarity, light tails, and i.i.d. samples is not merely suboptimal; it is structurally vulnerable to producing false discoveries.

Non-stationarity and regime drift

If there is a single fact that dominates financial ML, it is non-stationarity. Markets evolve. The mapping from information to returns changes as market structure changes, as participants adapt, as regulations shift, as macro regimes rotate, and as the distribution of risk premia re-prices. In practice, this means that the conditional distribution of returns given features, $p(y_{t+h} | X_t)$, is not constant over time. A model trained on one decade can easily fail in the next, even if the features are “reasonable” and the statistical fit looks strong in sample.

Non-stationarity appears in several operationally relevant forms. The most obvious is *level shifts* and *variance shifts*: volatility changes over time, correlations spike in crises, and liquidity regimes alternate. But the deeper issue is *concept drift*: the relationship between features and returns changes. A momentum signal that worked in one period may attenuate as it becomes crowded; a mean reversion pattern may strengthen when liquidity providers are active and weaken when risk limits tighten; a macro variable may flip sign when policy regimes change. Supervised learning models, especially flexible ones, will happily fit patterns that were true in the training period without any guarantee that those patterns will persist.

The consequence is that the classical idea of a single, global model estimated on the full history can be misguided. In trading, we often prefer models that are (i) trained on a recent window, (ii) updated frequently, and (iii) evaluated in a walk-forward manner to see how quickly performance decays. Yet this introduces a tradeoff: shorter windows adapt faster but reduce sample size, increasing estimation noise; longer windows provide more data but average across regimes, potentially learning relationships that are no longer relevant. Chapter 11 does not solve this tradeoff completely, but it insists that the tradeoff must be explicit. You cannot claim a forecasting edge without specifying the training window logic, the retraining cadence, and the stability of performance across time.

There is also a second, often overlooked, manifestation of non-stationarity: *changes in the data itself*. Corporate actions, index reconstitutions, survivorship in universes, changes in trading hours, and improvements in data quality alter the meaning of the inputs. Even if the market’s economic structure were stable, the measurement pipeline might not be. This is why governance artifacts such as data fingerprints, corporate-action policies, and feature manifests are not bureaucracy; they are the only way to interpret whether a model is learning economics or learning quirks of a changing dataset.

Heavy tails, outliers, and unstable moments

Many machine learning models implicitly rely on well-behaved noise. Even when Gaussianity is not assumed explicitly, the practical performance of least squares, standard errors, and many optimization routines can degrade in the presence of heavy tails and outliers. Financial returns routinely exhibit heavy tails: extreme events occur more frequently than a Gaussian model would predict, and those events often dominate both risk and performance.

From the perspective of supervised learning, heavy tails introduce three problems. First, they make naive loss functions brittle. If we train a regression model by minimizing mean squared error, then rare large moves can dominate the objective and pull the model toward fitting tail events that are not predictable in any stable way. Second, heavy tails destabilize empirical moments. Sample means and variances can change dramatically with the inclusion or exclusion of a handful of observations. This is not a minor statistical annoyance; it means that “signal” can be an artifact of a particular crisis or a particular outlier that happened to land in the training set. Third, heavy tails make model evaluation noisy. A model’s backtested P&L or forecast correlation can be heavily influenced by a small number of days, creating a false sense of precision.

Outliers are not merely statistical anomalies; they are often the events that matter most in trading. Crashes, gap moves, policy surprises, and liquidity dislocations produce returns that overwhelm months of ordinary drift. A model that performs well on average but fails catastrophically in these periods may be unacceptable; conversely, a model that is robust in crises may look mediocre on average but be valuable as a stabilizer. This tension forces the practitioner to think beyond a single metric. Chapter 11 therefore frames forecasting as part of a system: the predictor must be interpreted in relation to risk controls and to the distribution of outcomes, not just the average error.

Moreover, heavy tails connect directly to governance. When outcomes are heavy-tailed, the temptation to cherry-pick evaluation windows becomes stronger, because a few extreme observations can make or break a result. This makes the research protocol itself a target for unconscious manipulation: shifting the start date by a month, adjusting a filter, or changing a threshold can change the evaluation materially. A governance-native approach treats such degrees of freedom as risks that must be controlled through pre-specified protocols and comprehensive logging.

Low signal-to-noise: small edges, large uncertainty

Return forecasting is not hard because we lack models; it is hard because the predictable component of returns is often small relative to the unpredictable component. Even in markets where certain effects exist (momentum, carry, value), the out-of-sample predictability at short horizons is typically weak. For many liquid assets, the one-day-ahead expected return is tiny compared to daily volatility. This implies that a forecast model is often trying to detect a faint signal buried in noise that is orders of magnitude larger.

Low signal-to-noise has three consequences that shape everything in this chapter. First, it explains why simple baselines are so difficult to beat. A naive predictor of zero return is surprisingly competitive in mean squared error terms because the variance dominates. A

model can have a statistically significant relationship with returns and still fail to generate an economically meaningful edge once costs are included. Second, it implies that *overfitting is incredibly easy*. When the true signal is weak, many different models can appear to fit the training data well simply by capturing noise. Without rigorous time-ordered evaluation and model discipline, an ML pipeline will produce impressive-looking in-sample results almost by default.

Third, low signal-to-noise forces us to interpret performance through the lens of aggregation and stability. A model does not need to be “highly accurate” on individual predictions to be useful; it needs to produce a weak but persistent edge that can be harvested across many trades, ideally with controlled turnover. This is why metrics such as the information coefficient (correlation between forecast and realized return), stability across subperiods, and performance decay curves become central. A small but stable IC can be more valuable than a high but unstable accuracy metric that collapses out of sample.

Low signal-to-noise also explains the cultural difference between financial ML and many other ML applications. In some domains, a model that improves accuracy from 90% to 95% is clearly better and clearly deployable. In trading, improvements are often measured in basis points, and the confidence intervals around those improvements are wide. The result is that the practitioner must be both modest and meticulous: modest about what can be claimed, and meticulous about eliminating avoidable sources of error in the research process.

11.2.3 The governance constraint: leakage is the primary failure mode

If non-stationarity, heavy tails, and low signal-to-noise are the statistical reasons forecasting is hard, leakage is the operational reason why forecasting research is so often wrong. Leakage occurs when information from the future, or from the evaluation period, enters the model training or feature construction process in a way that would not be available in live trading. Leakage can be blatant (using tomorrow’s return as a feature) or subtle (scaling using full-sample statistics, selecting features based on test performance). Because predictive edges are small, even tiny leaks can manufacture apparently strong results. This makes leakage the primary failure mode: the most common reason a return forecasting model looks good in research and fails in reality is not that the market changed overnight, but that the research process accidentally cheated.

A governance-native stance treats leakage not as a mistake to be avoided informally, but as a risk to be managed with explicit gates, manifests, and invariants. In practice, this means that every feature must come with a definition of when it becomes available, every label must come with a definition of what time it refers to, and every split protocol must be stated and enforced. The goal is to build a research pipeline where leakage is hard to introduce and easy to detect.

Common leakage pathways in financial ML

Leakage pathways in financial ML are numerous precisely because time series pipelines have many stages: data cleaning, resampling, feature engineering, scaling, target construction, splitting, model fitting, and evaluation. The following pathways are among the most common and pernicious.

Full-sample normalization. A classic error is to compute mean and standard deviation using the entire dataset and then standardize features before splitting into train and test. This uses information from the future to scale the past. The effect can be subtle, but it is real: if volatility regimes change, full-sample scaling effectively leaks knowledge of future volatility into earlier periods. The correct approach is to fit scalars on the training window only (or to use expanding-window statistics) and apply them forward.

Rolling computations with incorrect alignment. Many features are rolling-window statistics: moving averages, rolling volatilities, z-scores. If the implementation accidentally includes the current or future observation in the window when the feature is supposed to be known at the start of the period, the feature becomes contaminated. The fix is not merely “use a lag”; it is to define precisely what decision time is. If trades are placed at the open, then any feature using the day’s close is unavailable. If trades are placed at the close, then intraday features might be available. The alignment must be stated and enforced.

Label leakage through overlapping horizons. If the target is a multi-day forward return, then adjacent labels overlap in time. This is not leakage by itself, but it creates dependence structures that can corrupt validation if the split boundaries are not handled carefully. If you train on observations whose labels overlap with the test period, you are effectively training on future returns that belong to the test window. Remedies include embargo periods and purging, which are treated conceptually here and more formally in Chapter 12.

Feature selection on the test set. Even if the model itself is trained only on the training data, the human can leak information by repeatedly trying features, models, and hyperparameters and choosing what looks best on the test window. The test set becomes part of the training loop through researcher behavior. This is one reason why Chapter 11 emphasizes baselines and protocol discipline and why Chapter 12 will formalize selection and validation more rigorously.

Universe and survivorship leakage. In cross-sectional contexts, constructing a dataset with only assets that survive to the end of the sample introduces information about the future. Even in a single-asset context, using revised data, restated fundamentals, or ex-post corrected datasets can leak future knowledge. While Chapter 11 focuses primarily on price-derived features, the general principle is that the data must reflect what would have been known at the time.

Backtest integration leakage. A subtle leakage occurs when the backtest uses the realized return for the same period to decide a trade in that period due to an ordering bug. For example, if the simulation updates portfolio value using today’s return and then uses the updated value to size today’s position, it implicitly uses knowledge of today’s return. This is why Chapter 06 timing discipline remains a prerequisite and why Chapter 11 insists on explicit decision-time ordering.

The unifying theme is that leakage usually arises from *implicit assumptions*. The pipeline assumes that all data is available, that scaling is harmless, that windows are aligned, that splits are independent. Governance requires replacing implicit assumptions with explicit contracts.

Causal feature timing: decision time vs label time

The most operationally important concept in supervised trading ML is the separation between decision time and label time. A supervised learning dataset in trading is not just a matrix X

and a vector y ; it is a collection of time-indexed objects with availability constraints. We must specify:

- the *decision time* t at which we choose an action (e.g., end of day, next open),
- the *information set* $\mathcal{I}_t$ that is available at that decision time,
- the *feature map* $X_t = f(\mathcal{I}_t)$ that must be computable from that information set,
- the *label time* $t + h$ defining the horizon at which the outcome is measured,
- the *target definition* y_{t+h} , such as a forward return over $(t, t + h]$.

A causal pipeline enforces that X_t depends only on $\mathcal{I}_t$, and that y_{t+h} depends on future prices that are not in $\mathcal{I}_t$.

This seems obvious until one writes actual code. Many data fields have ambiguous timing. “Close price” may refer to the last traded price, the official settlement, or a consolidated close that is only finalized later. Corporate actions can be recorded ex-post. Even within a day, the question “is the day’s volume known?” depends on whether you trade at the close or at the open. Chapter 11 therefore treats timing as part of the model specification: the feature manifest must record what time each feature becomes available and how it is lagged to match the decision schedule.

The causal timing discipline also forces clarity about what it means to forecast returns. A model that predicts the close-to-close return using features that include the close price is not forecasting; it is re-describing. A model that predicts next day’s return using features computed up to today’s close can be causal if the trade occurs after the close (or at the next open), but it is non-causal if the trade is assumed to occur at today’s close. These distinctions are not philosophical. They decide whether the model is tradable.

Finally, causal timing reveals why forecasting performance must be interpreted jointly with the execution protocol. If you decide at the close and trade at the next open, you must consider overnight gaps. If you decide at the open and trade immediately, you must ensure features do not use intraday information that arrives later. The model, the dataset, and the backtest must share a single clock. Governance is the act of making that clock explicit, auditable, and consistent across the entire pipeline.

In summary, forecasting returns is a special ML problem because the environment drifts, the noise is heavy-tailed, the signal is weak, and the easiest way to obtain a “good” result is to accidentally cheat. The chapter’s core stance is therefore not merely “build better models,” but “build a disciplined forecasting pipeline where any measured edge is more likely to be real than to be an artifact of timing errors, selection bias, or leakage.”

11.3 Problem Setup and Notation

11.3.1 Time index, information set, and decision-time features

We fix a discrete time index $t \in \{1, 2, \dots, T\}$ that corresponds to the *decision clock* of the strategy. In a daily system, t may represent trading days; in an intraday system, t may represent

bars of fixed duration. The central requirement is that the clock is consistent across data, features, labels, and evaluation. We therefore treat t not as an abstract counter but as an operational timestamp: it indicates when a decision is made, and it determines what information is admissible at that moment.

Let $\mathcal{I}_t$ denote the *information set* available at decision time t . This is the set of all observable variables, events, and metadata that are known (and recorded) by the strategy when it must choose an action. The definition of $\mathcal{I}_t$ is not purely statistical; it depends on the trading protocol. If the strategy forms signals at the close and trades at the next open, then the close price at time t belongs to $\mathcal{I}_t$ but the open price at time $t + 1$ does not. If the strategy trades at the close, then the close price may or may not be admissible depending on whether we assume the decision occurs strictly before the close print or after it; in practice, this is why we must be explicit about decision time versus execution time.

From $\mathcal{I}_t$ we construct a feature vector

$$X_t = f(\mathcal{I}_t) \in \mathbb{R}^d, \quad (11.2)$$

where f is a feature map that may include lagged returns, rolling volatilities, trend indicators, mean-reversion statistics, volume proxies, and any other engineered transforms permitted by the pipeline. The causality constraint is:

$$X_t \text{ is measurable with respect to } \mathcal{I}_t, \quad (11.3)$$

meaning that X_t may not depend on any data that arrives after decision time t . In an implementation, this is enforced via explicit lags and alignment rules; conceptually, it is a statement that features must be *decision-time features*.

We will often write the dataset of features as a matrix

$$X = \begin{bmatrix} X_1^\top \\ X_2^\top \\ \vdots \\ X_T^\top \end{bmatrix} \in \mathbb{R}^{T \times d}, \quad (11.4)$$

with the understanding that the rows are time-ordered and must remain so. No shuffling is permitted unless a very specific argument is made and logged (and for forecasting returns, shuffling is almost never justified).

Finally, we define an *action* or *position* decision w_t that the strategy takes at time t . Chapter 11 is about forecasting, not full portfolio construction, so we keep w_t abstract here; however, it is useful to remember that the forecast will ultimately feed a decision rule:

$$w_t = g(\hat{y}_{t+h}, \mathcal{I}_t; \theta_g), \quad (11.5)$$

where g may implement thresholding, scaling, or risk targeting. The existence of w_t in the notation is a reminder that the prediction problem is not the final objective; the model is a component of a trading system.

11.3.2 Targets for supervised trading models

A supervised trading model aims to learn a mapping from decision-time features X_t to a future outcome y_{t+h} , where $h \geq 1$ is the forecast horizon measured in decision steps. The canonical supervised learning object is therefore the pair

$$(X_t, y_{t+h}), \quad (11.6)$$

and the model produces a forecast $\hat{y}_{t+h} = m(X_t; \theta)$ for some parametric or non-parametric model class m with parameters θ . The crucial point is that y_{t+h} must be defined in a way that corresponds to an economically meaningful outcome *and* respects timing.

We will use P_t to denote a price at time t (with the understanding that “price” could be close, mid, or settlement depending on the data policy). For a single instrument, the simplest one-step return is

$$r_{t+1} = \frac{P_{t+1}}{P_t} - 1, \quad (11.7)$$

or its log-return analog $\ell_{t+1} = \log(P_{t+1}/P_t)$. In multi-asset settings we index by asset i , writing $P_{i,t}$ and $r_{i,t+1}$. In this chapter we mainly discuss the target engineering options; the full cross-sectional machinery is postponed.

A target definition is not merely a formula; it is a *contract* that specifies: (i) the horizon, (ii) the return type, (iii) whether returns are raw or excess, (iv) whether the target is hedged, and (v) whether the supervised task is regression, classification, or ranking. The following subsections enumerate the most common target families.

Next-period return and multi-horizon returns

The most direct forecasting target is the next-period return:

$$y_{t+1} = r_{t+1}. \quad (11.8)$$

Here the horizon is $h = 1$ and the model attempts to forecast the immediate next step. This formulation is conceptually clean and aligns with the decision clock, but it is often noisy, especially at high frequency. The shorter the horizon, the more microstructure effects, bid-ask bounce, and execution frictions dominate; these issues are treated later (Chapter 18), but they motivate why a one-step return can be a difficult label to learn.

Multi-horizon targets aggregate returns across $h > 1$ steps:

$$y_{t+h} = R_{t \rightarrow t+h} = \frac{P_{t+h}}{P_t} - 1, \quad (11.9)$$

or, equivalently, a cumulative log-return

$$y_{t+h} = \sum_{k=1}^h \ell_{t+k}. \quad (11.10)$$

A multi-horizon label can be interpreted as smoothing: it reduces the influence of single-period noise, but it introduces overlap between adjacent labels. Overlap is not automatically a problem,

but it affects evaluation. If h is large, then the label for time t shares most of its realized return window with the label for time $t + 1$. This creates serial dependence in y even if r_{t+1} were independent, and it can cause leakage-like effects in model selection if splits are not handled carefully. We therefore treat overlap as a first-class concern, revisiting it below when discussing purging and embargo logic.

In practice, multiple horizons may be used simultaneously. One can either train distinct models for each h or train a multi-output model. Within Chapter 11 scope, the key is to treat horizon as part of the experimental specification and to report how performance changes with h . A model that “predicts” at $h = 20$ may simply be capturing slow-moving exposure rather than short-term timing; that interpretation matters for how the signal will be used.

Excess returns, benchmark-relative returns, and hedged targets

Raw returns mix two components: systematic exposure (market beta, factor exposures) and idiosyncratic variation. Often, the objective of a forecasting model is not to predict raw returns but to predict returns relative to a benchmark or to a hedged baseline. This creates targets that better align with how many strategies are managed in practice: as alpha overlays rather than outright directional bets.

Let $r_{t+1}^{(b)}$ denote the return of a benchmark (e.g., a market index) and r_{t+1} denote the asset return. An *excess return* target is:

$$y_{t+1} = r_{t+1} - r_{t+1}^{(b)}. \quad (11.11)$$

This can be extended to multi-horizon excess returns by cumulating both series over the same horizon.

A more general benchmark-relative target uses a hedge ratio β :

$$y_{t+1} = r_{t+1} - \beta r_{t+1}^{(b)}. \quad (11.12)$$

If β is fixed, the target defines a static hedge. If β is estimated, the timing of that estimation becomes critical. A common mistake is to estimate β using the full sample (or using future data) and then apply it historically, which leaks future correlation structure into the past. The governance-correct approach is to estimate β using only historical information available at time t (e.g., an expanding or rolling regression up to t) and to log the hedge-ratio path as an artifact. Conceptually, we can write β_t to emphasize time dependence:

$$y_{t+1} = r_{t+1} - \beta_t r_{t+1}^{(b)}, \quad \beta_t \in \mathcal{I}_t. \quad (11.13)$$

The statement “ $\beta_t \in \mathcal{I}_t$ ” is precisely a causality constraint.

Hedged targets generalize beyond a single benchmark. One might hedge by multiple factors (market, sector, size, value) or by a related instrument (pairs trading). The same timing principle applies: any hedge parameter must be computed causally. Hedged targets can improve learnability by removing slow, dominant components, but they also shift the economic interpretation: a model that predicts hedged returns is a model for *relative* performance, not absolute direction.

That distinction will matter later when we integrate forecasts into portfolios (Chapters 16–17).

Direction, sign, and thresholded classification labels

Sometimes the objective is not to predict the magnitude of returns but to predict their sign or whether they exceed a threshold. This defines classification problems, which are attractive because they can be more robust to heavy tails and because some decision rules only require direction.

The simplest directional label is:

$$y_{t+1} = \text{sign}(r_{t+1}) \in \{-1, +1\}, \quad (11.14)$$

with a convention for $r_{t+1} = 0$ (often treated as 0 or assigned to a class). A related formulation uses a binary label:

$$y_{t+1} = \mathbb{I}\{r_{t+1} > 0\} \in \{0, 1\}. \quad (11.15)$$

Directional labels are easy to define but can be deceptive: an apparent improvement in classification accuracy may not translate into economic value if the model is only correct on small-return days and wrong on large moves. This again motivates multi-metric evaluation.

Thresholded labels introduce an “ignore zone” around zero:

$$y_{t+1} = \begin{cases} +1, & r_{t+1} \geq \tau, \\ 0, & |r_{t+1}| < \tau, \\ -1, & r_{t+1} \leq -\tau, \end{cases} \quad (11.16)$$

for some threshold $\tau > 0$. This formulation explicitly encodes the idea that very small moves may not be economically actionable after costs, while larger moves are. The choice of τ is therefore not merely statistical; it is linked to transaction costs and slippage (Chapter 18). Within Chapter 11 scope, the key is to treat τ as a parameter that must be declared, not tuned silently on the test set, and to report how results change across plausible values.

Classification labels also raise the issue of class imbalance. If returns have a slight drift or if the threshold is large, the “actionable” classes may be rare. This affects both training and evaluation. The notation here is simply a reminder: once we discretize labels, we must be careful about what metric we use (accuracy, precision/recall, balanced accuracy), and we must interpret the result in relation to the trading decision rule.

Ranking targets for cross-sectional prediction (conceptual only; full in Ch09/Ch16)

When the strategy trades a universe of assets, the economically meaningful question is often not “what will asset i return?” but “which assets are better relative opportunities at time t ?” This motivates ranking targets. Let $r_{i,t+1}$ be the next-period return for asset i in a universe of N assets. A ranking target can be defined by sorting returns:

$$\pi_t = \text{rank}\left(\{r_{i,t+1}\}_{i=1}^N\right), \quad (11.17)$$

where π_t is a permutation (or a vector of ranks) describing the cross-sectional ordering. A supervised model may aim to produce scores $s_{i,t}$ such that the ordering of $s_{i,t}$ matches the ordering of future returns. This is the conceptual core of many factor and cross-sectional ML systems.

In this chapter we keep ranking targets at the conceptual level, for two reasons aligned with the book arc. First, the construction of cross-sectional portfolios from rankings—including constraints, risk budgeting, and factor neutrality—belongs properly to Chapters 16 and 17. Second, ranking introduces additional data issues: the universe composition may change over time, missing data is pervasive, and survivorship biases are easier to introduce. Those issues are foreshadowed here but treated more completely later.

Nonetheless, the notation is useful now because it clarifies what supervised learning is doing in multi-asset contexts. A cross-sectional system can be framed as learning a scoring function $s_{i,t} = m(X_{i,t}; \theta)$, with evaluation based on rank correlation, top-minus-bottom spreads, or information coefficients. Even when we later optimize portfolios, the underlying predictive object remains a score or rank. Chapter 11 therefore introduces ranking as a target family to prepare the reader for the transition from single-asset forecasting to cross-sectional allocation.

11.3.3 Train/validation/test under time ordering

Having defined features and targets, we must define how the model is trained and evaluated. In standard supervised learning, random train/test splits and k -fold cross-validation are common. In return forecasting, such approaches typically violate the temporal structure and create optimistic bias. We therefore impose a time-ordering constraint: training data must occur strictly before validation and test data in calendar time.

Let $\mathcal{T}_{\text{train}}$, $\mathcal{T}_{\text{val}}$, and $\mathcal{T}_{\text{test}}$ denote disjoint sets of time indices used for training, validation, and testing. The time-ordering requirement is:

$$\max(\mathcal{T}_{\text{train}}) < \min(\mathcal{T}_{\text{val}}) < \min(\mathcal{T}_{\text{test}}), \quad (11.18)$$

in the simplest one-shot split. More generally, in walk-forward evaluation these sets change over multiple folds, but the same ordering logic applies in every fold.

The validation set plays a specific role: it is the period used for model selection, hyperparameter tuning, and threshold choices. The test set is reserved for final evaluation. In practice, many research failures occur because this separation collapses: the test set is used repeatedly as a feedback mechanism, and the reported result is effectively in-sample to the research process. Chapter 12 will treat model selection rigorously; Chapter 11 establishes the disciplined separation in notation and protocol.

Walk-forward splits and embargo concepts

A walk-forward split is an evaluation scheme that mimics the operational reality of trading: at each evaluation step, the model is trained on past data and tested on future data. A canonical

expanding-window walk-forward protocol is defined as follows. Choose a sequence of split points

$$1 < t_1 < t_2 < \dots < t_K < T, \quad (11.19)$$

and define for fold k :

$$\mathcal{T}_{\text{train}}^{(k)} = \{1, 2, \dots, t_k\}, \quad \mathcal{T}_{\text{test}}^{(k)} = \{t_k + 1, \dots, t_{k+1}\}, \quad (11.20)$$

with $t_{K+1} = T$ by convention. A rolling-window variant uses only the most recent L observations for training:

$$\mathcal{T}_{\text{train}}^{(k)} = \{t_k - L + 1, \dots, t_k\}, \quad (11.21)$$

when $t_k \geq L$. The expanding-window approach emphasizes data accumulation; the rolling-window approach emphasizes adaptation to drift. Both are valid, and the choice is part of the experimental specification.

The key advantage of walk-forward evaluation is that it produces a *time series of out-of-sample performance*, not a single number. This allows us to study stability: does performance persist, decay, or concentrate in a few subperiods? In low signal-to-noise settings, stability is often more informative than average performance.

Embargo concepts arise because in financial data, even when time ordering is respected, subtle dependence can persist across the train-test boundary. For example, if labels overlap across time (as in multi-horizon targets) or if features use rolling windows, then observations near the boundary may share information in a way that creates optimistic bias. An embargo is a gap inserted between training and testing:

$$\mathcal{T}_{\text{test}}^{(k)} = \{t_k + e + 1, \dots, t_{k+1}\}, \quad (11.22)$$

where e is the embargo length in time steps. The idea is that we avoid training on observations whose features or labels might overlap with the test period. In this chapter we treat embargo as a conceptual safeguard; Chapter 12 will formalize when it is required and how to choose e relative to horizons and feature windows.

Purging and overlap concerns (conceptual; mechanics in Ch12)

Purging refers to removing from the training set those observations whose label windows overlap with the test set. This issue is most salient when the target is a forward return over h steps. If the test set begins at time t^* , then any training observation at time t with $t + h \geq t^*$ has a label that depends on returns that occur during the test period. Training on such observations is tantamount to training on future outcomes.

Conceptually, we can define the label window for an observation at time t as the interval $(t, t + h]$. If the test period starts at t^* , then we should exclude from training any t satisfying:

$$(t, t + h] \cap [t^*, \infty) \neq \emptyset, \quad (11.23)$$

which simplifies to $t \geq t^* - h$. Thus, a simple purging rule is: when testing begins at t^* , remove

from training all observations with indices $t^* - h, \dots, t^* - 1$. This is closely related to an embargo of length h , but purging is more precise because it depends on the label definition rather than an arbitrary buffer.

Overlap concerns extend beyond labels. If features use rolling windows of length L , then features near the boundary may share input data with the test period if alignment is incorrect. In a causal implementation, features at time t should only use data up to time t , so a properly aligned feature window does not directly overlap into the future. However, scaling, normalization, or filtering steps can introduce boundary coupling if they are computed using both past and future values (for example, a centered moving average). Governance discipline therefore requires that every transform be declared as causal and that any non-causal transform be forbidden unless explicitly justified (and non-causal transforms are almost always invalid for live trading signals).

The reason Chapter 11 keeps purging “conceptual” is that the detailed mechanics depend on the full model selection framework. Once we tune hyperparameters, choose thresholds, and compare multiple models, the risk of subtle leakage through repeated evaluation increases. Chapter 12 will therefore treat purging and embargo as part of a broader selection-governance architecture. Here, the objective is to establish the notation and the intuition: in time-series supervised learning for trading, it is not enough to separate past from future; you must also separate training labels and preprocessing statistics from test-period outcomes.

Taken together, the notation of this section defines the supervised trading research object as a time-ordered collection of decision-time features and future outcomes, evaluated under protocols that respect chronology and guard against overlap. The remainder of Chapter 11 builds on this setup: we will propose baseline models, loss functions, and evaluation metrics, but every result is only meaningful insofar as it conforms to the timing and splitting contracts defined here.

11.4 Target Engineering and Label Design

Target engineering is the moment where supervised learning in trading becomes either a disciplined scientific program or an accidental exercise in self-deception. Unlike many machine learning applications where labels are given exogenously (a diagnosis, a category, a click), in trading we *construct* the label. That construction encodes what we mean by “success,” what horizon we are trying to monetize, what risks we want to neutralize, and what trading frictions we are implicitly willing to pay. A label is therefore not just a dependent variable; it is a contract that links the predictive task to an executable decision rule.

The consequences of label choices are profound. A model trained to predict raw one-day returns is fundamentally different from a model trained to predict a five-day hedged excess return; the latter may be learnable by capturing slow-moving exposures, while the former is dominated by microstructure and noise. Directional labels can make the learning task easier but can also hide economically relevant information if a model is “right” on small moves and “wrong” on large ones. De-noising labels by smoothing can create the illusion of predictability by leaking future information into the label or by shifting the objective from forecasting to trend

reconstruction. Because of these issues, Chapter 11 treats target engineering as a first-class research object, with explicit parameter registries, causal timing rules, and sanity checks that must precede any model comparison.

11.4.1 Return definitions: simple, log, and net-of-carry (conceptual)

We begin with the most basic question: what does it mean to define “return” for the purpose of a supervised target? Let P_t denote a price at time t on the strategy’s decision clock. The simplest definition is the *simple return*

$$r_{t+1}^{\text{simp}} = \frac{P_{t+1}}{P_t} - 1. \quad (11.24)$$

Simple returns are intuitive, map directly to P&L approximations for small moves, and are commonly used in portfolio calculations. However, simple returns do not add linearly across time: the h -period simple return is not the sum of one-period simple returns. This matters for label construction when horizons are longer than one step.

The alternative is the *log return*

$$r_{t+1}^{\text{log}} = \log\left(\frac{P_{t+1}}{P_t}\right), \quad (11.25)$$

which has the convenient property of additivity across time:

$$r_{t \rightarrow t+h}^{\text{log}} = \sum_{k=1}^h r_{t+k}^{\text{log}}. \quad (11.26)$$

For multi-horizon targets, log returns provide a clean mathematical object. They are also more symmetric for large positive and negative moves (though still not immune to heavy-tail effects). In many forecasting settings, the difference between log and simple returns is negligible for daily horizons, but the choice should be recorded because it affects aggregation and interpretation.

A third concept is *net-of-carry* returns. In many asset classes, the realized return includes a carry component: interest rate differentials in FX, funding and dividends in equities, roll yield and convenience yield in futures, and financing rates in leveraged products. If the trading objective is to forecast *price changes* rather than *total returns*, then the target may subtract a carry proxy. Conceptually, we can write a net-of-carry return as

$$r_{t+1}^{\text{nocarry}} = r_{t+1}^{\text{total}} - c_{t \rightarrow t+1}, \quad (11.27)$$

where $c_{t \rightarrow t+1}$ is the carry accrued over the holding period. The precise definition of c depends on the instrument and is often nontrivial; this book treats it conceptually here, because detailed instrument modeling can become a chapter in itself. The key lesson is governance: if net-of-carry is used, then the carry model must be documented, computed causally, and treated as part of the label manifest.

Return definitions also depend on *which price* is used. A close-to-close return differs from an open-to-open return, and a mid-to-mid return differs from a last-trade return. These choices are

not mere conventions; they represent different executable realities. A label based on close-to-close returns implicitly assumes you can hold a position through the close and mark it at the close; a label based on open-to-open returns aligns with trading at the open. If a strategy decides at the close and executes at the next open, then an overnight return becomes part of the realized outcome. The label definition must match the decision and execution clock, or else the predictive task is misaligned with what the strategy can actually implement.

11.4.2 Horizon choice and the bias–variance tradeoff

The forecast horizon h is the single most important knob in label design. A horizon is not only a temporal distance; it is a statement about which economic mechanisms you believe are exploitable. Very short horizons are dominated by microstructure, order flow, and execution details; longer horizons may reflect slower-moving behavioral effects, macro drift, or risk premia. The horizon also controls the statistical difficulty of the learning task via a bias–variance tradeoff.

At $h = 1$ (next-period returns), labels are extremely noisy. The conditional expectation $\mathbb{E}[r_{t+1} \mid X_t]$ may exist and may contain a small edge, but the variance of r_{t+1} is typically large. This produces high-variance estimates of predictive performance: the model may appear to work in some subperiods and fail in others simply due to noise. The temptation then is to increase h so that the label aggregates multiple periods:

$$y_{t+h} = r_{t \rightarrow t+h}. \quad (11.28)$$

Aggregation can reduce label variance relative to signal if the predictive effect is persistent across the horizon. However, aggregation also introduces *bias* in the sense that the target may no longer correspond to the decision rule you can execute. If a strategy trades daily but the label is a 20-day forward return, then the model may learn slow exposures rather than timing signals, and the resulting decisions may lead to high turnover relative to the horizon the label represents. In other words, a long-horizon label can accidentally train the model to approximate a moving average of returns, which is not the same as generating tradable daily timing.

There is also a subtle coupling between horizon and costs. If the model forecasts a long-horizon return, then a natural trading rule is to hold longer, which reduces turnover and costs. But if the implementation trades frequently while the label is long-horizon, the model’s signal is being harvested in a way that was not optimized for costs. Conversely, if the model forecasts a short-horizon return, the strategy must trade frequently to monetize it, increasing sensitivity to transaction costs (Chapter 18). The horizon is therefore a joint design choice that ties label design to the eventual execution layer, even if that layer is not yet fully modeled in Chapter 11.

A practical way to think about horizon is as a filter on the return-generating mechanisms you want to capture. Trend-following effects (Chapter 07) often manifest over intermediate horizons; mean-reversion effects (Chapter 08) may manifest over shorter horizons; volatility effects (Chapter 10) may influence position sizing more than direction. Choosing h is not a purely statistical tuning exercise; it is a hypothesis about what kinds of dynamics are stable enough to forecast.

Finally, horizon interacts with evaluation due to *overlapping labels*. If y_{t+h} is the return

over $(t, t + h]$, then y_{t+1+h} shares most of its realized return window with y_{t+h} . Overlap inflates the apparent effective sample size and complicates statistical inference. More importantly for governance, overlap creates dependence across train/test boundaries unless purging or embargo logic is applied. Thus, horizon choice increases not only modeling difficulty but also evaluation complexity. The horizon must be recorded and the corresponding overlap safeguards must be activated.

11.4.3 De-noising labels: smoothing, aggregation, and why it can mislead

Because returns are noisy, it is natural to consider de-noising the label. The most common de-noising approaches are smoothing, aggregation, and transformation. While these can make the learning problem numerically easier, they can also mislead by changing the task into something that is not forecastable in the way you think or by introducing leakage through non-causal operations.

The most benign form of de-noising is *causal aggregation* through horizon choice, already discussed. Summing returns over h periods is not inherently non-causal; it simply defines a different target. However, even causal aggregation can mislead if it changes the economic meaning of the task. A model that predicts a 20-day return may look more “accurate” than a model predicting a 1-day return, but the improvement may be a reflection of increased label persistence rather than genuine predictability at the decision frequency. If the strategy trades daily, you should be cautious about interpreting long-horizon predictability as a daily timing edge.

Smoothing becomes problematic when it uses *future information*. A centered moving average of returns, for instance,

$$\tilde{r}_t = \frac{1}{2L+1} \sum_{k=-L}^L r_{t+k}, \quad (11.29)$$

is non-causal because it includes future returns. If this smoothed series is used as a label, the model is implicitly being trained on a target that cannot exist in real time. Even if the smoothing is applied only to the label, not the features, it leaks future information into the supervised task. The model will then appear to have predictive power because the label itself is partially constructed from the future. Governance therefore demands a simple rule: any label de-noising must be causal with respect to the label time definition. If you must smooth, you should do so only with past information and you must record the smoothing operator explicitly.

Another seductive form of label de-noising is to predict *realized volatility* or *signed volatility* rather than raw returns, under the intuition that volatility is more predictable. This can be legitimate, but it changes the problem: you are now predicting risk, not direction, and the natural use of such a model is position sizing (Chapter 10) rather than directional alpha. If you conflate volatility predictability with return predictability, you will produce a strategy that looks “accurate” in forecasting a smoother label but does not generate alpha.

Transformations can also mislead. Consider predicting a clipped return:

$$y_{t+1} = \text{clip}(r_{t+1}, -c, c), \quad (11.30)$$

which reduces the influence of tail events. This may stabilize training under squared loss, but it effectively declares that tail events do not matter, which may or may not be acceptable depending on the strategy. Alternatively, predicting a rank-transformed return (mapping returns to percentiles) changes the objective to relative ordering, which may be appropriate for cross-sectional models but is a different task than forecasting magnitudes. The lesson is not that transformations are forbidden; it is that each transformation changes the meaning of the prediction task and therefore must be justified and documented.

Why can de-noising mislead even when it is causal? Because it can create labels that are easier to predict for reasons unrelated to tradable predictability. Smoothing can introduce persistence; aggregation can average out noise; clipping can remove difficult tail events. A model trained on such labels may show strong metrics, but those metrics may not translate to improved trading decisions. The correct stance is to treat label de-noising as a hypothesis: it should be evaluated not only by forecast metrics but also by how it impacts the end-to-end decision rule under realistic constraints.

11.4.4 Class imbalance in directional labels

Directional and thresholded labels often suffer from class imbalance. In a simple sign label, the class proportions may be close to 50/50 for many assets, but once thresholds are introduced (to ignore small moves), the “actionable” classes can become rare. Moreover, in some markets with drift or strong carry, the sign can be biased. Class imbalance matters because it can create deceptive performance: a naive classifier that always predicts the majority class can achieve high accuracy while providing no economic value.

Formally, suppose we define a ternary label with threshold τ :

$$y_{t+1} \in \{-1, 0, +1\}, \quad (11.31)$$

where 0 denotes “no action.” If τ is large, then most observations may fall into the 0 class. A classifier can then achieve high accuracy simply by predicting 0 always. But such a model produces no trades, hence no P&L. This is a reminder that classification metrics must be aligned with decision objectives. In trading, we care about performance *conditional on acting* and about the economic outcome of those actions. Thus, metrics such as precision/recall on the nonzero classes, or the expected return conditional on predicted action, may be more informative than overall accuracy.

Class imbalance also interacts with the training objective. Standard logistic loss weights each observation equally; if classes are imbalanced, the model will be optimized primarily on the majority class unless reweighting is used. Reweighting, however, is itself a modeling choice that must be documented, because it changes the implied objective: you are telling the model that rare events matter more. In trading, that may be appropriate if rare large moves drive profitability, but it also risks overfitting to a small set of observations. The governance principle is again consistency: any class weighting scheme must be part of the experiment configuration, not an ad hoc adjustment discovered during backtest iteration.

Finally, imbalance can reflect *economic structure*. For example, if the target is “return

exceeds transaction cost threshold,” and costs are substantial, then actionable moves may indeed be rare. In that case, imbalance is not a data problem; it is a reality check about the feasibility of the strategy at that frequency. A label design that yields extremely imbalanced actionable classes may be telling you that the horizon is too short or that the asset is too costly to trade for the proposed signal type. Label design thus becomes a diagnostic tool: imbalance can reveal the mismatch between intended trading frequency and economic frictions.

11.4.5 Practical label sanity checks

Before fitting any model, labels must pass sanity checks that test whether the supervised task is well-posed and whether the label design is likely to produce meaningful results. These checks are not “nice to have” diagnostics; they are hard gates that reduce the probability of publishing a leakage artifact or an overfit claim.

Distribution diagnostics. The first check is to inspect the empirical distribution of the label: mean, median, dispersion, tail behavior, and outliers. For regression targets, plot histograms and compute quantiles to see whether a small number of extreme values dominates. For classification targets, compute class proportions and verify that the task is not degenerate. In a governed pipeline, these summaries become part of the label manifest: they are recorded as artifacts so that future reruns can be compared and drift can be detected.

Time stability and subperiod summaries. Because markets drift, a label that is well-behaved in one period can behave differently in another. Compute label summary statistics in rolling subperiods: changes in volatility, skewness, and tail frequency. If the label distribution changes dramatically, then the model trained on early data may face a different problem later. This does not forbid forecasting; it simply informs window choice and retraining cadence. Crucially, if such changes are present, then any preprocessing applied to labels (such as scaling) must be causal and time-aware.

Autocorrelation and overlap diagnostics. Compute the autocorrelation function of the label series. For one-period returns, autocorrelation is often near zero (though not always). For multi-horizon aggregated labels, autocorrelation can be mechanically induced by overlap. The presence of strong autocorrelation is not necessarily evidence of predictability; it can be an artifact of label construction. The point of this diagnostic is interpretive: if your label has strong persistence, then a model may achieve good forecast metrics simply by exploiting that persistence, which may or may not translate into tradable edge depending on how positions are held.

Alignment checks: decision-time feasibility. Verify that the label is defined strictly in the future relative to the features. This sounds trivial, but in practice it requires explicit indexing checks: ensure that X_t uses only information up to t while y_{t+h} uses information strictly after t . In code, this is often enforced with assertions comparing timestamps and verifying that any rolling computation is properly lagged. In documentation, the check is recorded in the label manifest as a timing proof: the definition of y includes the price points used and their timestamps.

Economic plausibility checks. Labels should be evaluated against basic economic quantities: typical transaction costs, typical volatility, and realistic holding periods. For example, if you

define a threshold τ to create actionable labels, you should compare τ to plausible round-trip cost estimates. If τ is far below costs, the model will be trained on moves that are not monetizable; if τ is far above typical volatility, actionable labels will be rare and the strategy may have low capacity. Even though detailed cost modeling is deferred to Chapter 18, label design should not ignore economics entirely.

Baseline predictability checks. Before complex modeling, compute trivial baseline forecasts of the label: predict zero (for returns), predict the mean of the training window, or predict the majority class (for classification). If a naive baseline is already strong, it may indicate that the label contains mechanical persistence (e.g., due to overlap) or that the objective is mis-specified. Conversely, if baselines are strong only in some subperiods, that suggests regime dependence, which will later motivate Chapter 14.

Sensitivity checks. Finally, vary key label parameters (horizon h , threshold τ , hedge definition, carry adjustments) within reasonable ranges and observe how label properties change. A label design that produces radically different distributions under tiny parameter tweaks is brittle; any model trained on such labels is likely to be unstable. Sensitivity analysis should be logged as part of the experiment record, because it exposes degrees of freedom that could otherwise become avenues for p-hacking.

The common thread across these sanity checks is that labels are governed artifacts. They must be defined clearly, constructed causally, and validated before modeling begins. In a low signal-to-noise domain, the most valuable outcome of label engineering is often not a perfect target but a *credible* target: one whose definition aligns with decision timing and whose properties are stable enough that any predictive improvement is more likely to represent a genuine edge than an artifact of construction. Once labels pass these checks, we can proceed to model building and evaluation with a cleaner separation between signal discovery and research noise.

11.5 Features: Connecting Chapter 05 to Predictive Modeling

Features are the bridge between the governed pipelines of Chapter 05 and the forecasting models of Chapter 11. They are also the point where a research process most easily breaks: the same line of code that “improves” a signal can quietly introduce look-ahead, shift the effective decision time, or make a feature depend on statistics computed using future data. The objective of this section is therefore practical and disciplined: define a clear causal feature contract, enumerate canonical feature families that remain within Chapters 04–10 scope, explain how to scale features without leaking, and propose stability diagnostics that tell you when features have drifted enough that any learned mapping $X_t \mapsto y_{t+h}$ is unlikely to hold.

Throughout, we assume a decision clock $t = 1, \dots, T$ and a feature vector $X_t \in \mathbb{R}^d$ that must be computable using only information available at decision time t . All features are treated as *decision-time features*: by construction they must not use any prices, volumes, or statistics from $t + 1$ onward. We will keep the technical level consistent with the rest of Chapter 11: precise enough to enforce causality and reproducibility, but not so abstract that the reader loses the operational meaning.

11.5.1 Causality-first feature construction

A feature is valid for forecasting only if it respects the timing contract of the trading system. The contract has three parts.

(i) Define the decision time. Every feature must be associated with a decision time (e.g., “signal computed after the close of day t and traded at the next open”). If this is not stated, feature validity cannot be judged. The same numeric series can be causal or non-causal depending on when you assume the decision happens.

(ii) Enforce one-sided computations. Rolling means, rolling volatilities, and any windowed statistic must be computed using past observations only. Centered windows and symmetric filters are disallowed for live trading signals because they use future information. Even when a window is one-sided, you must align it correctly: the statistic computed using data up to time t must not be accidentally paired with the label that includes time t itself in a way that changes the meaning of the decision.

(iii) Make alignment explicit. A common safe convention is to compute an intermediate series at time t (e.g., rolling volatility using returns up to t) and then *shift it forward by one decision step* so that the feature used at decision time t is definitely based on data up to $t - 1$. This convention is conservative and avoids many subtle timing mistakes. It is especially useful when the precise “available-at” timestamp of a field is ambiguous (for example, when close prices might be revised or when a dataset reports end-of-day values that are only finalized later).

In practice, causality-first feature construction means the pipeline produces a feature manifest that includes, for each feature: (a) a definition, (b) the lookback window, (c) the exact alignment rule (including any lag), and (d) a statement of what raw fields it depends on. The feature manifest is part of governance: it is the only way to reproduce the design and audit whether the model was trained on information it could have had.

11.5.2 Canonical feature families (within prior scope)

The feature families below are intentionally conservative: they are derived from the diagnostics and strategies already developed in Chapters 04–10. The point is not that these are the only useful features, but that they form a well-understood baseline feature set whose behavior is interpretable and whose failure modes are familiar. Once these are implemented causally and evaluated rigorously, the reader can expand.

Return/volatility/realized variation features (from Ch04/Ch10)

The foundational features for forecasting returns are built from returns and realized variation. Let P_t be the relevant price (as defined by the data policy), and let r_t be a one-step return (simple or log). Common return-based features include:

Lagged returns. For $k \geq 1$,

$$f_{k,t}^{(r)} = r_{t-k}. \quad (11.32)$$

A stack of lagged returns is the simplest autoregressive feature set. It captures short-memory effects and provides a baseline that any more sophisticated model should outperform out of sample.

Aggregated returns. Over a lookback L ,

$$f_{L,t}^{(\text{cum})} = \sum_{j=1}^L r_{t-j}, \quad (11.33)$$

or the corresponding simple-return aggregation. This is a building block for momentum and for mean-reversion signals.

Realized volatility and realized variance. Over a lookback L ,

$$\hat{\sigma}_t(L) = \sqrt{\frac{1}{L} \sum_{j=1}^L r_{t-j}^2}, \quad \hat{v}_t(L) = \frac{1}{L} \sum_{j=1}^L r_{t-j}^2. \quad (11.34)$$

These are central to Chapter 10. They are not necessarily alpha features on their own, but they are critical control features: volatility regimes influence the mapping from predictors to positions, and they often explain performance instability.

Volatility-of-volatility and dispersion. Over a lookback L on the volatility estimate itself:

$$f_t^{(\text{volvol})} = \text{std}(\hat{\sigma}_{t-1}(L), \hat{\sigma}_{t-2}(L), \dots), \quad (11.35)$$

computed causally. Such features can help a model detect changes in risk regimes (without yet formalizing regime models, which is Chapter 14).

Standardized returns. A frequent building block is the return scaled by recent volatility:

$$f_t^{(\text{sr})} = \frac{r_{t-1}}{\hat{\sigma}_{t-1}(L) + \epsilon}, \quad (11.36)$$

with a small ϵ for numerical stability. This makes returns more comparable across volatility regimes, but it must be implemented causally and with careful alignment (the volatility in the denominator must be available before the return being standardized is used).

Trend features (from Ch07)

Trend features encode the idea that returns may exhibit persistence over certain horizons. The simplest is the moving-average difference or moving-average crossover style feature, computed on prices or returns.

Moving average (MA) levels. Define a one-sided moving average of price:

$$\text{MA}_t(L) = \frac{1}{L} \sum_{j=1}^L P_{t-j}. \quad (11.37)$$

A trend feature might be the deviation of price from its moving average:

$$f_t^{(\text{trend})} = \frac{P_{t-1} - \text{MA}_{t-1}(L)}{\text{MA}_{t-1}(L)}. \quad (11.38)$$

This is causal if it uses prices up to $t - 1$ and is evaluated at decision time t .

Multi-horizon momentum. Using cumulative returns over multiple lookbacks:

$$f_t^{(\text{mom})}(L_1, L_2) = \sum_{j=1}^{L_1} r_{t-j} - \sum_{j=1}^{L_2} r_{t-j}, \quad (11.39)$$

with $L_1 > L_2$ to capture intermediate momentum net of very short-term reversal.

Trend strength proxies. A simple and causal proxy is the ratio of absolute cumulative return to realized volatility over the same lookback:

$$f_t^{(\text{strength})}(L) = \frac{\left| \sum_{j=1}^L r_{t-j} \right|}{\sqrt{\sum_{j=1}^L r_{t-j}^2 + \epsilon}}. \quad (11.40)$$

This provides a normalized measure of directional persistence and can be used as an interaction feature (e.g., trend features may behave differently under high vs low trend strength).

Mean-reversion and spread features (from Ch08)

Mean-reversion features encode the hypothesis that prices (or spreads) revert toward a reference level. Even in a single-asset system, mean-reversion can be expressed through z-scores and deviations from rolling means.

Price or return z-scores. For a series z_t (often price changes, returns, or spreads), define a rolling mean and standard deviation:

$$\mu_t(L) = \frac{1}{L} \sum_{j=1}^L z_{t-j}, \quad s_t(L) = \sqrt{\frac{1}{L} \sum_{j=1}^L (z_{t-j} - \mu_t(L))^2}. \quad (11.41)$$

Then the z-score feature is:

$$f_t^{(z)}(L) = \frac{z_{t-1} - \mu_{t-1}(L)}{s_{t-1}(L) + \epsilon}. \quad (11.42)$$

Mean-reversion strategies often act on extreme z-scores; in ML, the z-score becomes a continuous feature.

Spread-based features (pairs concept). If two assets A and B are considered, a simple spread is $S_t = \log P_t^{(A)} - \log P_t^{(B)}$ (or a hedge-ratio-adjusted spread). Features are then z-scores or lagged changes of S_t computed causally. In Chapter 11 we treat this as a feature template; full pairs mechanics and hedge estimation governance were discussed conceptually in Chapter 08 and revisited in label design.

Reversal proxies. A short-horizon reversal feature is simply the negative of a short cumulative return:

$$f_t^{(\text{rev})}(L) = - \sum_{j=1}^L r_{t-j}. \quad (11.43)$$

This is a minimal representation of “recent move tends to revert.” In practice, reversal effects are often horizon- and volatility-dependent, which motivates interaction features such as $f_t^{(\text{rev})}(L) \times \hat{\sigma}_t(L')$.

Liquidity proxies and volume features (conceptual; microstructure deferred to Ch18)

Liquidity and volume features can be informative even at daily frequency, but they must be handled cautiously because their meaning is tied to microstructure and execution. Since Chapter 18 will treat microstructure in detail, we restrict ourselves here to conceptual proxies that can be computed from basic volume and price data and used as conditioning variables rather than as precise execution predictors.

Volume changes and volume surprises. Let V_t be traded volume over period t . A simple feature is log-volume change:

$$f_t^{(\Delta V)} = \log(V_{t-1} + 1) - \log(V_{t-2} + 1). \quad (11.44)$$

A “volume surprise” can be defined as a deviation from a rolling mean:

$$f_t^{(\text{vsurp})}(L) = \frac{\log(V_{t-1} + 1) - \mu_{t-1}^{(V)}(L)}{s_{t-1}^{(V)}(L) + \epsilon}, \quad (11.45)$$

with $\mu^{(V)}$ and $s^{(V)}$ computed causally.

Range-based proxies. Using daily high/low/close, one can form a simple range proxy for intraday movement:

$$f_t^{(\text{range})} = \frac{H_{t-1} - L_{t-1}}{P_{t-1}}, \quad (11.46)$$

where H_{t-1} and L_{t-1} are high and low. This is not a microstructure model, but it can help condition the forecast on “stress” or “activity”.

Dollar volume. A crude liquidity proxy is dollar volume:

$$f_t^{(\$V)} = P_{t-1} V_{t-1}, \quad (11.47)$$

often log-transformed. Again, this is best used to gate signals (e.g., avoid illiquid days) rather than to claim direct predictability without execution modeling.

The key governance stance is that liquidity proxies are often correlated with volatility and with regime changes. They can improve robustness by allowing the model to condition on “market state,” but they can also create spurious relationships if volume reporting conventions change. This is why stability diagnostics (below) matter.

11.5.3 Scaling and normalization under time ordering

Scaling features is often necessary because different features have different magnitudes, and many models behave better when inputs are comparably scaled. In trading ML, scaling is also a major leakage risk. The rule is simple: *any statistic used to scale features must be computed using only past data relative to the evaluation point*. This rule has two common implementations: expanding-window standardization and rolling-window standardization. Both can be valid; both can be misused.

Let x_t be a scalar feature time series (one component of X_t). A standardization uses:

$$\tilde{x}_t = \frac{x_t - \mu_t}{s_t + \epsilon}, \quad (11.48)$$

where μ_t and s_t must be computed causally. The difference between methods lies in how μ_t and s_t are defined.

Expanding-window standardization

Expanding-window standardization uses all available history up to time $t - 1$:

$$\mu_t^{\text{exp}} = \frac{1}{t-1} \sum_{j=1}^{t-1} x_j, \quad (s_t^{\text{exp}})^2 = \frac{1}{t-1} \sum_{j=1}^{t-1} (x_j - \mu_t^{\text{exp}})^2. \quad (11.49)$$

Then:

$$\tilde{x}_t = \frac{x_t - \mu_t^{\text{exp}}}{s_t^{\text{exp}} + \epsilon}. \quad (11.50)$$

This method is strongly causal and typically stable. It avoids the problem of the scaling parameters jumping dramatically from day to day. It is also aligned with a “learn progressively” worldview: as time passes, the system accumulates more information about the typical scale of the feature.

However, expanding-window scaling adapts slowly to regime changes. If volatility shifts upward permanently, then s_t^{exp} may lag behind, and standardized features may become systematically large in magnitude during the new regime. That can be good (it reflects a real change) or bad (it destabilizes the model if it was trained under the old scale). The point is that expanding-window scaling trades adaptivity for stability.

In a governed pipeline, expanding-window scaling is easy to audit: the scaling rule is deterministic, and the only degrees of freedom are initial conditions (e.g., how many observations are required before standardization begins). These should be in the parameter registry.

Rolling-window standardization and its leakage traps

Rolling-window standardization uses a fixed lookback L :

$$\mu_t^{\text{roll}}(L) = \frac{1}{L} \sum_{j=1}^L x_{t-j}, \quad (s_t^{\text{roll}}(L))^2 = \frac{1}{L} \sum_{j=1}^L (x_{t-j} - \mu_t^{\text{roll}}(L))^2. \quad (11.51)$$

Then:

$$\tilde{x}_t = \frac{x_t - \mu_t^{\text{roll}}(L)}{s_t^{\text{roll}}(L) + \epsilon}. \quad (11.52)$$

This method adapts more quickly to regime changes, which can be desirable in markets with time-varying volatility and liquidity. But it introduces two classes of traps.

The first trap is **boundary leakage**. In a walk-forward protocol, scaling must be performed inside each training window using only that window. If you compute rolling statistics on the full dataset and then split, the rolling means and variances near the split boundary may incorporate

observations from the validation/test period (depending on how the computation is implemented). This is especially easy to do by accident if the code computes rolling statistics once globally.

The second trap is **label-coupled leakage through preprocessing**. Even if rolling statistics are one-sided, if you use the *entire* training+validation+test period to choose the lookback L for scaling (because it improves final performance), you have effectively used the test set to design the preprocessing. The scaling window becomes a tuned hyperparameter. This is not forbidden, but it must then be treated as a model selection step, with the test set held out until the end. Chapter 12 will formalize this; the warning here is that preprocessing is part of the model, and therefore it is subject to the same selection governance.

A practical governance rule that reduces these traps is: *scalers are fit and applied as part of the walk-forward fold*. That is, for each fold, compute scaling parameters on the training portion only, transform both training and test using those parameters (or using causal rolling formulas restricted to the training history), and log the transformation. This makes the scaling step auditable and prevents silent cross-contamination.

11.5.4 Feature stability: drift diagnostics and monitoring

Even if features are constructed causally and scaled correctly, their meaning can drift. A feature that represented “trend strength” in one decade may represent “crowding” in another; volume proxies may change with market structure; volatility regimes shift; correlations rewire. Because supervised learning models learn a mapping from features to labels, feature drift can break the model even if the label distribution is stable.

Feature stability is therefore not an optional monitoring task; it is part of the research protocol. Before trusting any forecasting result, we should understand whether the feature distribution is stable enough that a model trained in one period is being asked to operate on similar inputs later.

A practical stability toolkit includes:

(i) Rolling distribution summaries. For each feature component x_t , compute rolling or expanding summaries: mean, standard deviation, quantiles, and tail frequency. Large shifts indicate drift. These summaries should be plotted over time and stored as artifacts.

(ii) Correlation drift. Compute rolling correlations between key features and between features and realized volatility. If correlations change sign or magnitude, the learned relationships can become unreliable. Correlation drift is especially important for feature sets that include multiple related transforms (e.g., lagged returns and moving average deviations), where multicollinearity patterns can change.

(iii) Feature–label relationship drift (diagnostic, not proof). Compute a simple rolling association measure between each feature and the label (e.g., a rolling correlation for regression labels, or a difference in mean label conditional on feature quantiles). This is not a reliable estimate of predictability, but it is a diagnostic for instability: if a feature’s relationship with the label flips sign across subperiods, any global model is likely to struggle.

(iv) Missingness and quality drift. Features derived from volume, high/low, or multi-asset spreads can suffer from missingness or reporting changes. Track the fraction of missing values, the frequency of zero volumes, and any data anomalies. A feature that is stable statistically but

unstable in availability is operationally fragile.

(v) Simple invariants. Maintain hard tests: no NaNs after feature construction (or explicit policies for handling them), monotonic time index, and consistent alignment between features and labels. These are not “drift” tests, but they prevent silent pipeline degradation, which is often mistaken for a market regime change.

The connection to governance is immediate: stability diagnostics should be part of the standard artifact bundle for every run. If a model “stops working,” the first question is not “is the market efficient now?” but “did the feature distribution shift, did scaling drift, or did the pipeline change?” Without stored diagnostics, you cannot answer these questions, and you cannot distinguish between economic decay and engineering failure.

In summary, features connect Chapter 05 to Chapter 11 by carrying forward the same discipline: causal construction, explicit definitions, controlled preprocessing, and audited stability. The canonical feature families provide a conservative baseline rooted in Chapters 04–10. Scaling methods are treated as part of the model and must be time-aware. Stability diagnostics are the mechanism by which we decide whether learned mappings are likely to generalize or whether they are being asked to operate out of distribution. With these elements in place, supervised learning becomes a controlled experiment rather than a search for accidental leakage.

11.6 Baseline Models You Must Beat

In return forecasting, the most common failure mode is not that a sophisticated model is “wrong” in some subtle way; it is that the model appears to work only because the evaluation was too forgiving, the feature engineering leaked, or the model is simply rediscovering a trivial baseline. For this reason, a disciplined supervised trading workflow treats baselines as *hard competitors*. Any proposed model must demonstrate incremental value over these baselines under the same walk-forward protocol, with the same feature timing rules, and under comparable turnover and risk constraints. Baselines are not an afterthought; they are the minimum standard for claiming that modeling complexity is justified.

This chapter focuses on baseline models that are (i) easy to implement causally, (ii) interpretable, and (iii) representative of what “no real modeling” looks like in trading. We emphasize four categories: null predictors, linear regression baselines, logistic regression for direction, and simple tree-based models as a conceptual introduction. Model selection and regularization are postponed to Chapter 12; here we keep baselines deliberately simple and transparent.

11.6.1 The null: predict zero, predict mean, or random sign

The null model answers a blunt question: “What happens if we assume the market is unpredictable at the chosen horizon?” In low signal-to-noise settings, the null is often surprisingly competitive on statistical metrics such as mean squared error, and it is an essential reference point for economic interpretation.

Predict zero. For return regression targets, the simplest null predictor is

$$\hat{y}_{t+h} = 0. \quad (11.53)$$

If y_{t+h} is a raw return, this corresponds to forecasting no movement; if y_{t+h} is an excess return, it corresponds to forecasting no alpha. Under squared error, predicting zero is near-optimal when the conditional mean is small relative to variance. If a complex model cannot beat this baseline out of sample, then it has not demonstrated usable predictability.

Predict the historical mean. A slightly less trivial baseline is to forecast the mean estimated from the training window:

$$\hat{y}_{t+h} = \bar{y}^{(\text{train})}, \quad \bar{y}^{(\text{train})} = \frac{1}{|\mathcal{T}_{\text{train}}|} \sum_{t \in \mathcal{T}_{\text{train}}} y_{t+h}. \quad (11.54)$$

In a walk-forward protocol, $\bar{y}^{(\text{train})}$ is recomputed for each fold using only past data. This baseline captures drift in the unconditional mean (to the extent it exists) but otherwise remains conservative. For many assets at short horizons, the mean is close to zero; in that case, the mean baseline and the zero baseline are nearly identical, which is itself an informative diagnostic about signal scarcity.

Random sign and majority class. For directional classification labels $y_{t+h} \in \{-1, +1\}$ or $\{0, 1\}$, the null is a random guess:

$$\hat{y}_{t+h} \sim \text{Bernoulli}(1/2), \quad (11.55)$$

or, in sign form, random ± 1 . But in practice, the more relevant null is the *majority-class predictor*:

$$\hat{y}_{t+h} = \arg \max_{c \in \mathcal{C}} \mathbb{P}_{\text{train}}(y = c), \quad (11.56)$$

where $\mathcal{C}$ is the set of classes. If directional labels are imbalanced (for instance, when thresholds introduce a large “no action” class), the majority-class predictor can achieve high accuracy while producing no tradable value. This is why classification evaluation must go beyond accuracy and include metrics aligned with trading decisions (precision/recall on actionable classes, conditional returns, and turnover pressure).

The broader point is that null baselines give you a reality check. If your model beats the null only marginally, you may still have something economically meaningful, but you must then demonstrate stability and cost robustness. If your model does not beat the null, complexity is not justified.

11.6.2 Linear regression baselines (OLS) and their interpretation

Linear regression is the workhorse baseline for return forecasting because it is transparent, fast, and easy to audit. It provides a minimal test of whether your features contain linear predictive information about the target. Let $X_t \in \mathbb{R}^d$ be the feature vector and $y_{t+h} \in \mathbb{R}$ the regression target. The OLS model assumes:

$$y_{t+h} = \beta_0 + \beta^\top X_t + \varepsilon_{t+h}, \quad (11.57)$$

and estimates (β_0, β) by minimizing the training-window squared error:

$$\min_{\beta_0, \beta} \sum_{t \in \mathcal{T}_{\text{train}}} \left(y_{t+h} - \beta_0 - \beta^\top X_t \right)^2. \quad (11.58)$$

In a walk-forward protocol, OLS is re-fit on each training fold and evaluated on the subsequent test fold. This re-fitting is not a detail; it is the mechanism that respects non-stationarity. A single OLS fit on the full sample is not a valid baseline for forecasting because it uses information from the future to estimate coefficients.

Interpretation in trading. OLS coefficients are often tempting to interpret as economic “drivers.” In trading, they should be interpreted more modestly: as a directional summary of association within the training window. Because features can be correlated and because the environment drifts, coefficient stability over time matters more than coefficient magnitude in a single fit. A useful baseline diagnostic is therefore the *coefficient path*: the sequence of $\hat{\beta}$ estimates over walk-forward folds. If signs flip frequently or magnitudes explode, the linear relationships are unstable, and more complex models are unlikely to be robust.

OLS as a baseline signal. Even if OLS has weak predictive power on traditional statistical metrics, it can still provide an economically meaningful baseline when translated into a trading signal. One simple mapping is to treat the forecast $\hat{y}_{t+h}$ as a score and apply thresholding or scaling. But Chapter 11 insists on separating the forecasting question from the portfolio question: the OLS baseline is first evaluated as a forecaster under time-ordered metrics (error, correlation, directional accuracy), and only then translated into a minimal signal. This avoids the trap of declaring victory because a particular position-sizing rule happened to work.

Common pitfalls. Two pitfalls are particularly relevant. First, scaling must be causal: if features are standardized using full-sample statistics, the OLS baseline becomes contaminated. Second, overlapping labels at horizon $h > 1$ can create optimistic evaluation if purging/embargo is neglected near fold boundaries. OLS is simple enough that these pitfalls are easy to detect and fix, which is another reason it is valuable as a baseline.

11.6.3 Logistic regression for direction

For directional targets, logistic regression is the natural baseline because it is the simplest probabilistic classifier with interpretable structure. Let $y_{t+h} \in \{0, 1\}$ denote an “up” indicator. Logistic regression models the conditional probability:

$$\mathbb{P}(y_{t+h} = 1 \mid X_t) = \sigma(\beta_0 + \beta^\top X_t), \quad \sigma(z) = \frac{1}{1 + e^{-z}}. \quad (11.59)$$

Parameters are estimated by maximizing the training-window log-likelihood (equivalently minimizing log-loss):

$$\min_{\beta_0, \beta} - \sum_{t \in \mathcal{T}_{\text{train}}} [y_{t+h} \log p_t + (1 - y_{t+h}) \log(1 - p_t)], \quad (11.60)$$

where $p_t = \sigma(\beta_0 + \beta^\top X_t)$.

What logistic regression provides. The key output is not only a class prediction but a probability estimate. This is operationally useful because trading decisions are rarely binary:

you may choose to trade only when the probability exceeds a threshold, or to scale position size by confidence. Even before we implement such rules, logistic regression provides a baseline for the informational content of features in a directional setting.

Evaluation that matters. Accuracy alone is often misleading. A better baseline evaluation includes: (i) directional accuracy, (ii) a calibration diagnostic (do predicted probabilities correspond to realized frequencies), and (iii) economic conditional performance (what is the average realized return when the model predicts “up” with high confidence). Importantly, all of these must be computed out-of-sample in walk-forward fashion.

Class imbalance and thresholds. If labels are imbalanced, logistic regression may default to predicting the majority class. In that case, the baseline must include balanced metrics and, if a thresholded trading rule is used, the threshold must be selected using validation data only. The message is consistent with the chapter’s governance stance: even for a baseline model, the degrees of freedom in converting probabilities to trades must be controlled.

11.6.4 Simple tree-based baselines (conceptual introduction)

Tree-based models (decision trees and their ensembles) are popular in trading because they capture nonlinear interactions and regime-like conditional behavior. However, they also introduce complexity and hyperparameters, which is why a rigorous treatment belongs in Chapter 12. In Chapter 11 we introduce them only as a conceptual baseline: a way to test whether modest nonlinearity provides value beyond linear models.

A simple decision tree partitions feature space into regions and assigns a constant prediction (regression) or class probability (classification) within each region. Conceptually, it learns rules of the form:

$$\text{if } f_1(X_t) \leq c_1 \text{ and } f_3(X_t) > c_2 \text{ then predict } \hat{y}_{t+h} = a, \quad (11.61)$$

where the thresholds c_1, c_2 and output a are fit from training data. Such rule-based structure is attractive in finance because it resembles conditional strategies: “trend works when volatility is low,” “mean reversion works when the z-score is extreme,” and so on.

Why trees are a useful baseline. If a tree improves out-of-sample performance over OLS/logistic regression in a stable way, it suggests that there are nonlinear interactions in the features that matter. If it does not, then the additional complexity of nonlinear modeling may not be justified. Either outcome is informative.

Why we keep it minimal here. Trees can easily overfit, especially in low signal-to-noise environments. Depth limits, minimum leaf sizes, and pruning rules materially change behavior. Proper tuning of these choices is model selection, which is Chapter 12 territory. Therefore, in Chapter 11 the tree baseline should be used with conservative settings (shallow depth, large minimum leaf size) and evaluated strictly walk-forward. The point is not to “win” with a tree; it is to test whether simple nonlinearity adds signal without opening the floodgates to p-hacking.

Interpretation. A tree can also serve as a diagnostic tool: its splits can reveal which features matter conditionally and can suggest interactions worth encoding explicitly or monitoring for drift. But interpretability must be handled carefully: a tree fitted in one regime may split differently in another, and the structure itself can drift.

In summary, baseline models serve as a scientific anchor. The null models tell you how hard the problem is. Linear regression and logistic regression test whether your features contain stable linear information. Simple trees test whether modest nonlinearity adds value. If your proposed model cannot beat these baselines out of sample under the same timing and governance constraints, it has not earned its complexity. Chapter 12 will then extend this baseline discipline into a full model selection and regularization framework that reduces false discoveries when you begin to search over richer model classes.

11.7 Loss Functions and Objectives Aligned With Trading

A supervised model is only as meaningful as the objective it is trained to optimize. In many machine learning applications, the loss function is chosen for statistical convenience and aligns naturally with the task: classification accuracy for labels, squared error for regression, log-loss for probabilities. In trading, the alignment is weaker. We ultimately care about economic outcomes (P&L, drawdowns, turnover, risk-adjusted performance), but we train models on statistical objectives because they are stable, differentiable, and easier to optimize under limited data. This creates a fundamental tension: a model can minimize prediction error and still be economically useless, and a model can be economically useful even if its point forecasts are noisy. The correct stance in Chapter 11 is therefore pragmatic: use statistical losses for training and comparison, but interpret them through decision-oriented diagnostics and mapping rules that respect trading constraints.

11.7.1 Statistical losses: MSE, MAE, log-loss

We begin with the standard losses that define the baseline supervised learning problems.

Mean squared error (MSE). For regression targets $y_{t+h} \in \mathbb{R}$ and forecasts $\hat{y}_{t+h}$, the MSE loss is:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} (y_{t+h} - \hat{y}_{t+h})^2. \quad (11.62)$$

MSE penalizes large errors heavily. This is both a feature and a bug in finance. It is attractive when large moves are economically important and predictable; it is problematic when large moves are mostly unpredictable tail events that dominate the objective and drive the model to chase noise. In low signal-to-noise domains, MSE can encourage overfitting because it rewards fitting rare extreme observations.

Mean absolute error (MAE). MAE is:

$$\mathcal{L}_{\text{MAE}} = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} |y_{t+h} - \hat{y}_{t+h}|. \quad (11.63)$$

MAE is more robust to outliers than MSE because errors grow linearly rather than quadratically. For heavy-tailed returns, MAE is often a more stable training and evaluation objective. The tradeoff is that MAE does not emphasize tail errors as strongly, which may be undesirable for strategies whose primary risk is rare large losses. A useful practical approach is to evaluate with both MSE and MAE: if a model looks good on MSE but not on MAE (or vice versa), that often

reveals sensitivity to extremes.

Log-loss (cross-entropy). For probabilistic classification with labels $y_{t+h} \in \{0, 1\}$ and predicted probabilities $p_t = \mathbb{P}(y_{t+h} = 1 \mid X_t)$, log-loss is:

$$\mathcal{L}_{\log} = -\frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} [y_{t+h} \log(p_t) + (1 - y_{t+h}) \log(1 - p_t)]. \quad (11.64)$$

Log-loss rewards calibrated probability estimates and strongly penalizes confident wrong predictions. This property is important for trading, because a system that places large bets based on overconfident probabilities is fragile. Accuracy alone is indifferent to confidence; log-loss is not. As a result, log-loss is a better training objective when probabilities will be used to scale positions or to trigger trades with confidence thresholds.

All three losses share a governance requirement: they must be evaluated out of sample under time ordering. A low training loss is meaningless in finance; the entire point is whether the loss remains low in the next period.

11.7.2 Decision-oriented objectives (conceptual)

Even when we train on statistical losses, we must remember that the forecast is not the product; the trade is the product. This motivates decision-oriented objectives: criteria that measure how forecasts perform when translated into positions. Chapter 11 treats these objectives conceptually because full backtesting mechanics were established in Chapter 06 and full portfolio and execution modeling appears later. Here, the goal is to connect the supervised learning output to a minimal, transparent trading rule and to explain why statistical improvement may or may not translate into economic improvement.

Predictive models as signal generators: mapping forecasts to positions

Let $\hat{y}_{t+h}$ be the model forecast. In a single-asset setting, a minimal position rule is:

$$w_t = \text{clip}(\kappa \hat{y}_{t+h}, -w_{\max}, w_{\max}), \quad (11.65)$$

where κ is a scaling constant (possibly chosen to target risk) and $w_{\max}$ is a position limit. Alternatively, a thresholded rule is:

$$w_t = \begin{cases} +w_0, & \hat{y}_{t+h} \geq \tau, \\ 0, & |\hat{y}_{t+h}| < \tau, \\ -w_0, & \hat{y}_{t+h} \leq -\tau, \end{cases} \quad (11.66)$$

where τ is a confidence threshold. In classification, with predicted probability p_t , the analogous rule is:

$$w_t = \begin{cases} +w_0, & p_t \geq p^*, \\ 0, & p_-^* < p_t < p^*, \\ -w_0, & p_t \leq p_-^*, \end{cases} \quad (11.67)$$

with thresholds chosen on validation data only.

The key idea is that the model output becomes a *signal* that is post-processed into a trade. This post-processing is where turnover, costs, and risk constraints enter. A model that reduces MSE slightly but produces very noisy forecasts may lead to frequent trading and high costs. A model that is less accurate statistically but produces stable sign predictions above a threshold may trade less and be more profitable net of costs. Thus, the mapping function is part of the system, and evaluating the model without specifying the mapping can be misleading.

Linking loss to P&L under simplifying assumptions

It is tempting to train directly on P&L, but P&L is noisy, non-convex, and depends on execution. Still, it is useful to understand how statistical losses relate to P&L under simple assumptions, because this clarifies what a loss is encouraging the model to do.

Consider a one-step holding return r_{t+1} and a position w_t chosen at time t . A simplified P&L contribution is:

$$\Pi_{t+1} = w_t r_{t+1}. \quad (11.68)$$

Suppose we choose $w_t = \kappa \hat{r}_{t+1}$, i.e., proportional to the forecast of next return. Then expected P&L is:

$$\mathbb{E}[\Pi_{t+1}] = \kappa \mathbb{E}[\hat{r}_{t+1} r_{t+1}]. \quad (11.69)$$

If we decompose $r_{t+1} = \hat{r}_{t+1} + e_{t+1}$ where e_{t+1} is the forecast error, then:

$$\mathbb{E}[\Pi_{t+1}] = \kappa \mathbb{E}[\hat{r}_{t+1}^2] + \kappa \mathbb{E}[\hat{r}_{t+1} e_{t+1}]. \quad (11.70)$$

Under idealized conditions where the forecast error is uncorrelated with the forecast, the second term vanishes and expected P&L scales with forecast variance. In that stylized case, improving forecast quality (reducing error variance) tends to improve P&L for a fixed scaling rule. But in realistic settings, the correlation structure can be complex, and costs matter. Moreover, a forecast can have low MSE simply by shrinking toward zero; such a forecast may reduce error but also reduce $\mathbb{E}[\hat{r}^2]$, leading to low trading activity and low P&L. This illustrates why pure error minimization does not automatically produce a good trading signal.

A more robust conceptual bridge uses correlation. The information coefficient (IC) is $\text{corr}(\hat{y}, y)$. If w_t is monotone in $\hat{y}$, then P&L often increases with IC, but the translation depends on scaling, turnover, and constraints. The takeaway is modest: statistical losses are proxies; they must be interpreted through the decision rule and the economic environment.

11.7.3 Asymmetric losses and tail risk considerations

Trading losses are not symmetric in practice. A model that is wrong during tail events can destroy a strategy even if it is usually right. Conversely, a model that sacrifices small average accuracy to avoid catastrophic errors can be valuable. This motivates asymmetric losses and tail-aware evaluation.

An asymmetric regression loss can weight positive and negative errors differently:

$$\mathcal{L}_{\text{asym}} = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \left[\lambda_+ \mathbb{I}\{e_t \geq 0\} e_t^2 + \lambda_- \mathbb{I}\{e_t < 0\} e_t^2 \right], \quad (11.71)$$

where $e_t = y_{t+h} - \hat{y}_{t+h}$ and $\lambda_+, \lambda_- > 0$. This is a conceptual tool for encoding that one type of error is more costly. For example, in a long-only mandate, overestimating returns (leading to larger positions) may be more damaging than underestimating them. Similarly, if the strategy is particularly vulnerable to downside tail moves, errors that coincide with large negative realized returns may deserve higher weight.

Tail risk can also be handled by focusing evaluation on extreme regimes: compute loss conditional on high realized volatility, or on large absolute returns. This does not require changing the training loss; it can be a diagnostic that reveals whether a model fails precisely when it matters most.

The governance caution is that asymmetric losses introduce additional degrees of freedom. If λ_+ and λ_- are chosen after observing test performance, the research process is contaminated. Thus, asymmetric objectives should be treated as part of model specification and selected using validation data only.

11.7.4 Calibration vs discrimination: what matters for trading

In classification problems, two distinct qualities matter: *discrimination* and *calibration*. Discrimination is the ability to rank outcomes correctly: assign higher probabilities to events that occur. Calibration is the ability to produce probabilities that match empirical frequencies: among predictions of 70%, the event happens about 70% of the time. Trading uses both, but in different ways.

Discrimination is essential when forecasts are used for ranking or thresholding. If you only trade the top decile of predicted returns or only act when p_t exceeds a threshold, you mainly need the model to order opportunities correctly. Metrics such as AUC (for binary labels) or rank correlation (for continuous targets) reflect discrimination.

Calibration matters when probabilities are used to size positions or to manage risk. If you interpret p_t as confidence and scale the bet accordingly, overconfident but miscalibrated probabilities can lead to excessive leverage in precisely the wrong moments. Log-loss is sensitive to calibration; accuracy is not. A model can have good discrimination but poor calibration, which might still be useful if you only use it for ranking and apply conservative position rules. Conversely, a model with good calibration but weak discrimination might produce stable but uninformative probabilities.

For trading, the pragmatic view is:

- If your decision rule is *threshold-based* (trade only when confidence is high), discrimination is primary, but calibration affects how you interpret thresholds.
- If your decision rule is *size-based* (position size scales with predicted probability or expected return), calibration becomes critical, because sizing magnifies errors.

In both cases, calibration and discrimination should be evaluated out of sample in walk-forward fashion, and the mapping from forecast to trade must be declared and governed.

In summary, statistical losses (MSE, MAE, log-loss) are the workhorses of supervised learning because they are stable and comparable across models. But trading is a decision problem, and the model output becomes a signal that must pass through thresholds, scaling, constraints, and risk controls. This is why Chapter 11 emphasizes alignment: train on statistical losses, evaluate with both statistical and decision-oriented diagnostics, treat tail risk explicitly, and distinguish calibration from discrimination depending on how the forecast will be used. The deeper question of how to select among losses, tune objectives, and avoid selection bias is postponed to Chapter 12, where regularization and model selection are treated as part of a governed research process.

11.8 Evaluation Under Time: Walk-Forward Research Protocol

In trading, evaluation is not a reporting step at the end of modeling; it is the core scientific method. Because markets drift, because returns are heavy-tailed, and because edges are small, the difference between a real signal and an artifact is almost always an evaluation question. A model that looks excellent under a single convenient split can collapse under a different split; a model that looks impressive on pooled data can fail when tested sequentially; a model that appears profitable can be the product of subtle leakage or iterative tuning. For these reasons, Chapter 11 treats evaluation as a protocol: a set of rules that must be declared in advance, executed deterministically, and logged as part of the experiment manifest.

The defining feature of a proper protocol is that it respects time. Training data must precede test data, preprocessing must be computed without future information, and the evaluation should reflect the operational reality of periodically retraining and deploying the model. Walk-forward evaluation is the natural mechanism: it mimics how a model would be used live and produces a time series of out-of-sample performance rather than a single number.

11.8.1 One-shot split vs rolling evaluation

A *one-shot split* partitions the time index into a single training set and a single test set, for example:

$$\mathcal{T}_{\text{train}} = \{1, \dots, t_0\}, \quad \mathcal{T}_{\text{test}} = \{t_0 + 1, \dots, T\}. \quad (11.72)$$

This is the simplest time-respecting evaluation. It is useful as an initial sanity check: it forces chronological separation and gives a first view of whether the learned relationship generalizes at all. However, one-shot splits have two major limitations.

First, the result depends heavily on the choice of t_0 . If the training period covers a calm regime and the test period contains a crisis (or vice versa), performance can change dramatically. In non-stationary markets, there is no reason to believe that one boundary is representative. Second, one-shot splits produce a single out-of-sample number, which is a fragile summary when outcomes are heavy-tailed. A few extreme observations can dominate the test period, creating a misleading impression of either success or failure.

A *rolling evaluation* (walk-forward evaluation) addresses these weaknesses by repeatedly

training on past data and testing on subsequent data across multiple contiguous segments. Instead of asking, “does this model work in one specific future window?”, we ask, “does this modeling approach repeatedly work as time advances?” This shift is crucial because it allows us to study stability and decay, and it reduces dependence on one arbitrary split boundary.

In practice, one-shot evaluation is acceptable for preliminary debugging and for demonstrating the basic feasibility of a pipeline. For any serious claim, especially in a low signal-to-noise domain, rolling evaluation is the default.

11.8.2 Walk-forward procedure: expanding vs rolling training windows

Walk-forward evaluation specifies a sequence of folds indexed by $k = 1, \dots, K$. Each fold consists of a training window followed by a test window. The model is fit on the training window and evaluated on the test window; then the clock moves forward and the process repeats. Two canonical schemes are used: expanding-window training and rolling-window training.

Expanding-window walk-forward. Choose cut points

$$1 < t_1 < t_2 < \dots < t_K < T, \quad (11.73)$$

and define, for fold k :

$$\mathcal{T}_{\text{train}}^{(k)} = \{1, \dots, t_k\}, \quad \mathcal{T}_{\text{test}}^{(k)} = \{t_k + 1, \dots, t_{k+1}\}, \quad (11.74)$$

with $t_{K+1} = T$. Training data grows over time; the model benefits from more history, but it may adapt slowly to regime changes because early data remains in the training set. Expanding windows are appropriate when you believe the signal is relatively stable and that more data improves estimation.

Rolling-window walk-forward. Choose a fixed training length L and cut points t_k . Then:

$$\mathcal{T}_{\text{train}}^{(k)} = \{t_k - L + 1, \dots, t_k\}, \quad \mathcal{T}_{\text{test}}^{(k)} = \{t_k + 1, \dots, t_{k+1}\}. \quad (11.75)$$

The training set length is constant; old data is dropped. Rolling windows adapt faster to drift but increase estimation variance because fewer samples are used. Rolling windows are appropriate when you suspect strong non-stationarity or when the feature-label relationship is known to change.

Fold granularity and test length. The test segment length can be a single step (one-step-ahead repeated) or a block (e.g., one month). One-step-ahead testing more closely resembles daily forecasting but can make evaluation noisy. Block testing smooths the evaluation but can hide intra-block decay. The choice should be logged as part of the protocol.

Embargo and purging (conceptual). When labels overlap (multi-horizon returns) or when there is dependence near boundaries, a buffer can be inserted between train and test (embargo) or training observations whose label windows overlap the test can be removed (purging). Chapter 11 treats these safeguards conceptually; Chapter 12 formalizes them as part of model selection discipline. Here, the key point is that walk-forward evaluation is only valid if the training labels do not contain returns that belong to the test period.

Operational realism. Walk-forward evaluation is valuable because it mirrors live usage: at regular intervals you retrain using available history and then deploy until the next retrain. A protocol should therefore specify a retraining cadence (daily, weekly, monthly) and a deployment horizon. Even if the model is technically refit at every step in research, reporting should communicate what deployment frequency is assumed.

11.8.3 Metrics for forecast quality

Forecast quality metrics evaluate the predictive relationship between $\hat{y}_{t+h}$ and y_{t+h} without committing to a full trading rule. They are necessary but not sufficient. The goal is to characterize whether the forecasts contain information, whether that information is stable, and whether it is expressed in a way that can plausibly be harvested.

Correlation (IC), directional accuracy, and stability

Information coefficient (IC). A central metric in finance is the correlation between forecasts and realized outcomes:

$$\text{IC} = \text{corr}(\hat{y}_{t+h}, y_{t+h}), \quad (11.76)$$

computed on the test window (or within each fold). IC measures whether the model ranks outcomes correctly in a linear sense. In cross-sectional contexts it is often computed across assets at a fixed time; in time-series contexts it is computed across time. A small but persistent IC can be economically meaningful.

Directional accuracy. For sign or binary labels, directional accuracy is:

$$\text{DA} = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \mathbb{I}\{\text{sign}(\hat{y}_{t+h}) = \text{sign}(y_{t+h})\}, \quad (11.77)$$

or the analogous indicator for $\{0, 1\}$ labels. Directional accuracy is intuitive but can be misleading: being correct on small moves and wrong on big moves can still yield high accuracy while losing money.

Stability across folds. Because single-period metrics are noisy, we care about the *distribution* of IC and DA across folds:

$$\{\text{IC}^{(k)}\}_{k=1}^K, \quad \{\text{DA}^{(k)}\}_{k=1}^K. \quad (11.78)$$

A model whose average IC is positive but whose IC is negative in half the folds is unstable; it may be regime-dependent or overfit. Reporting should therefore include fold-level metrics and not only pooled metrics. A simple stability summary is the fraction of folds with positive IC (or above a minimum threshold) and the dispersion of IC across time.

Error metrics and why low error may not mean tradable

MSE/MAE. Error metrics quantify closeness of forecasts to outcomes:

$$\text{MSE} = \frac{1}{|\mathcal{T}|} \sum_t (y_{t+h} - \hat{y}_{t+h})^2, \quad \text{MAE} = \frac{1}{|\mathcal{T}|} \sum_t |y_{t+h} - \hat{y}_{t+h}|. \quad (11.79)$$

They are appropriate for comparing models with the same target definition, but they can obscure economic relevance. The core reason is that returns have low predictability in magnitude. A model can achieve low MSE by shrinking forecasts toward zero, which reduces error but also reduces actionability. A model can also reduce MSE by fitting conditional mean structure that is too weak to overcome costs.

Why low error can be economically meaningless. Consider a target with near-zero mean and high variance. The best MSE predictor under many conditions is close to the conditional mean, which may be extremely small. Improving MSE by a few percent may correspond to extremely small improvements in conditional mean estimates. If those improvements do not translate into more correct ranking or more robust sign decisions at the tails, they may not change P&L materially. For this reason, error metrics should be paired with ranking metrics (IC) and with decision-conditioned diagnostics (e.g., performance when forecasts are in the top/bottom quantiles).

Conditional evaluation. A practical bridge between forecast metrics and tradability is to evaluate realized outcomes conditional on forecast strength. For example, compute average realized return when $\hat{y}$ is in the top decile versus bottom decile, or compute realized hit-rate when $|\hat{y}|$ exceeds a threshold. These diagnostics help determine whether forecasts become more informative when the model is more confident.

11.8.4 Metrics for strategy performance (conceptual; full mechanics in Ch06/Ch16–18)

Ultimately, forecasting is valuable only insofar as it improves strategy outcomes. Chapter 11 therefore includes strategy performance metrics conceptually, while deferring full mechanics to Chapter 06 (backtesting) and Chapters 16–18 (portfolio construction and execution). The key is that even a simple mapping from forecasts to positions produces a time series of strategy returns, and that time series can be evaluated using standard risk and performance measures.

Sharpe, drawdowns, hit-rate, turnover

Sharpe ratio. For strategy returns R_t (net or gross), the Sharpe ratio is:

$$\text{Sharpe} = \frac{\mathbb{E}[R_t]}{\text{std}(R_t)}, \quad (11.80)$$

typically annualized with an appropriate factor based on frequency. Sharpe is a compact risk-adjusted measure but can be unstable under heavy tails and non-stationarity. In walk-forward evaluation, it is more informative to compute Sharpe per fold and study its distribution.

Drawdowns. Maximum drawdown summarizes worst peak-to-trough loss. Because many strategies fail not by low average returns but by intolerable drawdowns, reporting drawdowns is essential even at the conceptual stage. Drawdown behavior can also reveal whether a model fails in particular regimes.

Hit-rate. The hit-rate is the fraction of periods with positive strategy return. Like directional accuracy, hit-rate can be misleading if gains are small and losses are large; nonetheless, it provides a simple view of consistency.

Turnover. Turnover measures how much the strategy trades. High turnover implies high cost sensitivity. Even without explicit cost modeling, turnover is a governance metric: if a forecasting model improves gross performance but doubles turnover, its net value is doubtful. Turnover should be reported alongside performance, because it indicates whether the forecasting edge is being “bought” by trading more.

Capacity intuition and transaction-cost sensitivity (deferred)

Capacity and cost sensitivity are where many apparent forecasting edges die. A signal that looks profitable on paper can vanish once realistic slippage and market impact are applied. While detailed modeling is deferred to Chapter 18, evaluation in Chapter 11 should at least include sensitivity checks: performance under simple cost assumptions, and performance as a function of turnover thresholds (e.g., requiring stronger forecast confidence before trading). If the model’s advantage disappears under small hypothetical costs, it is likely not robust.

Capacity intuition also matters in interpreting forecast metrics. A model that generates many small trades may not scale; a model that generates fewer, higher-conviction signals may have better capacity. Even at this stage, the reader should learn to ask: “Is the model’s edge concentrated in a high-turnover regime, or does it persist under stricter trading filters?”

11.8.5 Statistical hygiene: multiple testing and p-hacking warnings

Walk-forward evaluation reduces but does not eliminate the risk of false discoveries. In trading, the search space is large: many features, horizons, models, and thresholds can be tried. If you try enough variations, some will look good by chance, especially with heavy-tailed outcomes. This is the essence of multiple testing and p-hacking risk.

Multiple testing. Every time you evaluate a new model configuration on the same dataset, you are effectively running another hypothesis test. The probability that at least one configuration looks impressive by chance increases rapidly with the number of tries. This is why a single final backtest result is not sufficient evidence of a real edge unless the search process is controlled and documented.

P-hacking in time series. In finance, p-hacking often takes the form of adjusting start/end dates, changing thresholds slightly, switching horizons, adding or removing a feature, or selecting among similar assets and universes. Because results are noisy, these small degrees of freedom can produce large apparent improvements. The antidote is governance: pre-register the evaluation protocol, log every run and configuration, and separate validation (used for tuning) from the final test period.

What Chapter 11 enforces. At minimum, Chapter 11 enforces: (i) time-ordered splits, (ii) walk-forward reporting with fold-level stability, (iii) baseline comparisons under identical protocols, and (iv) explicit logging of choices that change turnover or label definitions. The deeper discipline of model selection, regularization, and robust validation is postponed to Chapter 12, where the book will treat the research process itself as an object that must be optimized for truth rather than for backtest performance.

In summary, evaluation under time is the defining discipline of supervised learning in trading.

One-shot splits are useful for initial sanity checks but are fragile in non-stationary markets. Walk-forward evaluation, with explicit expanding or rolling training windows, produces a stability-aware picture of out-of-sample performance. Forecast metrics (IC, directional accuracy, errors) must be interpreted carefully, because low error does not guarantee tradability. Strategy metrics (Sharpe, drawdowns, hit-rate, turnover) provide the economic lens, even when costs are treated only conceptually. Finally, statistical hygiene is essential: without governance against multiple testing and iterative tuning, even a time-respecting evaluation protocol can become a machine for manufacturing false edges.

11.9 From Forecast to Trade: A Minimal Signal-to-Position Mapping

A supervised model produces a forecast; a trading system takes a position. The gap between those two statements is where many “good” forecasting projects fail. A predictor can show a small but real statistical edge and still lose money because the mapping from forecast to trade induces excessive turnover, concentrates risk unintentionally, or violates basic exposure constraints. Conversely, a modest predictor can become economically useful if the mapping is disciplined: trade only when the forecast is strong, scale positions to a stable risk budget, and enforce hard limits that prevent blow-ups.

This section provides a minimal, transparent mapping from forecast to position that is consistent with the book’s arc. We stay within Chapter 11 scope: we do not introduce advanced portfolio optimization, sophisticated execution modeling, or detailed transaction cost mechanics. Instead, we define a *handshake* between forecasting and the backtesting/sizing machinery developed earlier (Chapter 06 for simulation discipline and Chapter 10 for risk targeting). The objective is to show how to translate $\hat{y}_{t+h}$ into a position w_t in a way that (i) is causal, (ii) is auditable, and (iii) controls the two main sources of fragility at this stage: turnover and risk.

11.9.1 Thresholding forecasts to control turnover

Forecasts are rarely smooth. Even when a model is genuinely informative, day-to-day predictions often fluctuate around zero. If we translate such forecasts mechanically into continuous positions, the strategy may trade frequently, generating high turnover and creating vulnerability to costs and slippage. Because detailed cost modeling is deferred (Chapter 18), Chapter 11 uses a simpler and more robust idea: *trade only when the forecast is sufficiently strong*.

Let $\hat{y}_{t+h}$ be the model forecast at decision time t for the target y_{t+h} . A thresholding rule defines a raw (pre-risk) signal s_t :

$$s_t = \begin{cases} +1, & \hat{y}_{t+h} \geq \tau, \\ 0, & |\hat{y}_{t+h}| < \tau, \\ -1, & \hat{y}_{t+h} \leq -\tau, \end{cases} \quad (11.81)$$

where $\tau \geq 0$ is a threshold. When $\tau = 0$, every nonzero forecast produces a trade direction; turnover is typically high. As τ increases, fewer trades occur, but the average “quality” of trades

(conditional on acting) may improve.

There are several practical variants of this same idea.

Quantile thresholding. Instead of a fixed numeric τ , we can trade only when the forecast is in the extreme quantiles of its recent distribution. Define a rolling window of past forecasts of length L and compute the empirical quantiles $q_{\alpha,t}$ and $q_{1-\alpha,t}$. Then:

$$s_t = \begin{cases} +1, & \hat{y}_{t+h} \geq q_{1-\alpha,t}, \\ 0, & q_{\alpha,t} < \hat{y}_{t+h} < q_{1-\alpha,t}, \\ -1, & \hat{y}_{t+h} \leq q_{\alpha,t}. \end{cases} \quad (11.82)$$

This is useful when the scale of forecasts drifts over time or changes with volatility. The governance constraint is that quantiles must be computed causally: $q_{\alpha,t}$ must use only forecasts from times $< t$.

Hysteresis to reduce churn. A common turnover problem is rapid flipping around the threshold. A minimal fix is hysteresis: use a higher threshold to enter a position and a lower threshold to exit. Let $\tau_{\text{in}} > \tau_{\text{out}} \geq 0$. Then maintain the prior position unless the forecast crosses the exit threshold, and only flip when the entry threshold is crossed in the opposite direction. Conceptually, this reduces whipsaw trading without introducing additional modeling complexity. Because hysteresis introduces state dependence, it should be logged and tested in backtesting carefully (Chapter 06 mechanics).

Magnitude-preserving signals. Instead of mapping to $\{-1, 0, +1\}$, one can retain magnitude but still threshold:

$$s_t = \begin{cases} 0, & |\hat{y}_{t+h}| < \tau, \\ \hat{y}_{t+h}, & |\hat{y}_{t+h}| \geq \tau. \end{cases} \quad (11.83)$$

This allows stronger forecasts to produce stronger positions, while still suppressing small, noisy forecasts that create churn.

Governance note on threshold choice. Thresholds (or quantile levels α) are not innocuous. They change turnover, which in turn changes the strategy's sensitivity to costs and to market impact. Therefore, thresholds must be treated as parameters: chosen using validation windows, logged in the run manifest, and stress-tested for sensitivity. Choosing τ after inspecting test-period P&L is a classic p-hacking pathway.

At this stage, the most important conceptual point is simple: thresholding is a mechanism to trade less, but with higher conviction. It is one of the cleanest ways to keep a forecasting model from producing an untradeable strategy.

11.9.2 Risk targeting wrapper (connect to Ch10)

Thresholding controls how often you trade; risk targeting controls how large you trade. Chapter 10 established the central idea: in many strategies, managing volatility is as important as predicting direction. A minimal signal-to-position mapping should therefore wrap any raw forecast signal with a risk targeting layer so that the strategy's ex-ante risk is stable across time.

Let s_t be the raw signal produced by thresholding or by a continuous mapping. Let $\hat{\sigma}_t$ be an estimate of future (or near-term) volatility available at decision time t , typically computed from realized variation using only past returns (Chapter 10). The basic risk-targeted position is:

$$w_t^{\text{rt}} = \frac{\kappa}{\hat{\sigma}_t + \epsilon} s_t, \quad (11.84)$$

where $\kappa > 0$ scales the overall risk budget (e.g., to target a desired volatility for the strategy), and ϵ is a small constant for numerical stability.

Equation (11.84) encodes a simple principle: hold smaller positions when volatility is high and larger positions when volatility is low, for the same signal strength. This creates a more stable risk profile and often improves the interpretability of performance: you are less likely to confuse “the model worked” with “volatility fell so leverage rose” or with the opposite.

Several practical details matter, all consistent with Chapter 10 scope:

Choice of volatility estimate. $\hat{\sigma}_t$ can be a rolling standard deviation of returns, an EWMA volatility, or any realized-volatility proxy. The key is causality: it must be computed using returns up to time $t - 1$ (or up to t if the decision is after the close and that is explicitly declared). The volatility estimator and its parameters (window length L , decay λ) must appear in the estimator manifest.

Interaction with thresholds. Risk targeting and thresholding complement each other. Thresholding reduces trading when signal is weak; risk targeting controls exposure when signal is active. Without thresholding, risk targeting can still lead to high turnover if s_t changes sign frequently. Without risk targeting, thresholding can lead to unstable risk because position size is fixed while volatility shifts.

Forecast-as-expected-return mapping. If $\hat{y}_{t+h}$ is interpreted as an expected return proxy, a common mapping is:

$$w_t = \text{clip} \left(\frac{\kappa}{\hat{\sigma}_t^2 + \epsilon} \hat{y}_{t+h}, -w_{\max}, w_{\max} \right), \quad (11.85)$$

motivated by mean-variance intuition (expected return divided by variance). In Chapter 11 we keep this as an optional conceptual mapping because the economic meaning of $\hat{y}$ can vary with label design; still, it shows how forecasting magnitude could influence position size in a controlled way.

Governance warning. Risk targeting can hide problems. A weak signal can appear stable because volatility scaling dampens exposure during turbulent periods. This is not necessarily bad, but it means that evaluation must separately report (i) forecast quality metrics and (ii) strategy metrics. Otherwise, one can mistakenly attribute risk-targeting benefits to predictive skill.

The overarching objective is to keep the forecasting model’s contribution clean: risk targeting should stabilize exposure so that changes in performance are more attributable to predictive content than to changes in volatility.

11.9.3 Position limits and exposure caps (conceptual; full in Ch17)

Even a minimal mapping needs hard limits. Without limits, extreme forecasts or volatility estimator failures can produce unbounded positions. In live trading, limits are not optional; they are the core of risk governance. Chapter 17 will formalize sizing, leverage, and risk management. Here we introduce the minimal conceptual layer: clipping and caps.

Position clipping. The simplest rule is:

$$w_t = \text{clip}\left(w_t^{\text{rt}}, -w_{\max}, w_{\max}\right), \quad (11.86)$$

where $w_{\max}$ is a maximum position size (in notional units, leverage units, or portfolio weight units depending on the system). This prevents runaway exposures due to small $\hat{\sigma}_t$ or extreme s_t .

Exposure caps and gross limits. In a multi-asset system, one would cap gross exposure $\sum_i |w_{i,t}|$ and possibly net exposure $\sum_i w_{i,t}$. Even in a single-asset system, gross exposure corresponds to leverage. We treat these conceptually here: the mapping from forecast to position must respect the system's risk budget and must enforce limits deterministically.

Staleness and fail-safe rules. A minimal governance policy also includes fail-safes:

- If a feature or volatility estimate is missing at time t , set $w_t = 0$ (or hold prior position) according to a declared policy.
- If $\hat{\sigma}_t$ is below a floor (possibly due to data issues), replace it with a floor value to avoid extreme leverage.
- If a forecast is extreme relative to its historical distribution (beyond a sanity threshold), clip it or flag it.

These are not advanced risk management; they are basic engineering sanity checks that prevent numerical accidents from becoming economic disasters. They also improve reproducibility: missing-data behavior must be deterministic and logged.

The conceptual point is that mapping forecasts to positions is not just a function; it is a controlled system with constraints. Even the minimal mapping must encode these constraints explicitly.

11.9.4 A minimal backtest handshake (conceptual; full in Ch06)

The final step is to connect this mapping to a backtesting engine. Chapter 06 provided the mechanics of simulation: event timing, signal time vs trade time, and no-look-ahead invariants. Here we describe a minimal handshake that makes the forecasting model evaluable as a trading rule without reintroducing hidden timing errors.

A minimal handshake requires four aligned objects:

(i) **A decision schedule.** Specify when the signal is computed and when the trade is executed (e.g., “compute features after the close at t , place trade at next open, hold until next decision time”). This schedule defines what price points are used for P&L.

(ii) **A forecast time index.** At each decision time t , compute X_t and produce $\hat{y}_{t+h}$. The model may be trained on a rolling window and refit at declared intervals.

(iii) **A position series.** Apply the mapping:

$$\hat{y}_{t+h} \rightarrow s_t \rightarrow w_t^{\text{rt}} \rightarrow w_t \quad (11.87)$$

with thresholding, risk targeting, and caps.

(iv) **A realized return series that matches the holding period.** If the strategy holds from t to $t + 1$ on the decision clock, then realized return is r_{t+1} ; if it holds for h periods, the realized return should match that horizon. The critical requirement is that the label horizon and the holding period must not be confused. A model trained to predict h -step returns does not automatically justify holding only one step.

With these pieces, a minimal backtest P&L update is:

$$\Pi_{t+1} = w_t r_{t+1}, \quad (11.88)$$

or, for multi-step holding, the corresponding aggregated return. Chapter 06 adds transaction timing, cash accounting, and event traces; here we keep it conceptual to emphasize alignment.

No-leakage invariants. The handshake should enforce two hard invariants (conceptually, and as assertions in code):

- The position w_t is computed using only information available at decision time t .
- The realized return used to compute Π_{t+1} is strictly after decision time t .

These invariants are the minimum guarantee that the backtest is not inadvertently using future information.

Why this handshake matters in Chapter 11. It allows us to evaluate a forecasting model as a trading component without prematurely introducing full portfolio optimization or microstructure modeling. It also clarifies what can be claimed: at this stage, we can claim that a forecasting approach produces signals that improve simple, risk-targeted trading performance under a governed walk-forward protocol. We cannot yet claim capacity, we cannot claim robust net performance under realistic costs, and we cannot claim optimal sizing. Those claims belong later in the book arc.

In summary, the minimal signal-to-position mapping has a clear structure: (i) suppress weak forecasts via thresholding to control turnover, (ii) stabilize exposure via a risk targeting wrapper from Chapter 10, (iii) enforce hard caps to prevent runaway leverage and to respect conceptual risk governance, and (iv) connect to the backtesting engine via a minimal handshake that preserves timing integrity. This is enough to translate forecasting research into a disciplined trading experiment, while postponing the deeper sizing, portfolio, and execution questions to Chapters 17, 16, and 18 respectively.

11.10 Failure Modes and Diagnostic Playbook

A supervised trading model rarely fails in a dramatic, obvious way. More often it fails quietly: performance looks strong in-sample, acceptable on a convenient split, and then degrades when

moved forward in time, when traded at realistic frequency, or when exposed to a different volatility regime. The purpose of this section is to provide a practical diagnostic playbook that is consistent with the governance stance of this book: treat failure modes as predictable engineering and statistical risks, build tests that catch them early, and log enough artifacts to distinguish “the market changed” from “the research process cheated.”

The playbook is organized around the most common failure modes in financial ML: leakage (the primary catastrophic failure), non-stationarity, overfitting under time dependence, feature instability, and post-modeling pitfalls related to costs and execution. The emphasis is on actionable diagnostics: what to test, what to plot, what to log, and how to interpret what you see.

11.10.1 Leakage triage: tests and invariants

Leakage is the failure mode that invalidates an entire result. If future information enters features, scaling, labels, or model selection, the reported out-of-sample performance is not evidence of predictability; it is evidence of contaminated evaluation. Because leakage pathways are numerous and often subtle, the correct approach is triage: maintain a small set of invariants and tests that must always pass before any performance number is trusted.

Invariant 1: strict time ordering. For every observation used for prediction at decision time t , verify that all feature inputs are timestamped $\leq t$ and that the label is timestamped $> t$. In practice, this becomes an assertion on indices after alignment: the feature row indexed by t must not be paired with a label that uses prices at time $\leq t$. This sounds trivial, but it catches common off-by-one mistakes.

Invariant 2: causal preprocessing. Any preprocessing step that estimates parameters from data (means, standard deviations, PCA-like transforms, winsorization thresholds, quantile cutoffs) must be fit using training data only within each walk-forward fold. A common leakage pathway is global standardization on the full sample. A minimal test is to re-run the pipeline with fold-wise fitting and confirm that results do not collapse; if they do, the original result likely depended on leakage.

Invariant 3: embargo/purging awareness for overlapping labels. If labels are multi-horizon returns, training observations whose label windows overlap the test window must be removed or the boundary must be embargoed. A diagnostic is to compute the horizon h and verify that the last h training points before the test start are excluded. If not, the model may have seen outcomes that belong to the test period.

Leakage “smell tests.” Leakage often produces telltale patterns:

- unrealistically high IC or directional accuracy that persists across many folds without degradation,
- very sharp improvements when adding a feature that should not be predictive (e.g., a “future” rolling statistic),
- large performance jumps when standardizing globally rather than fold-wise,

- performance that improves dramatically as you increase feature complexity without increasing sample size.

These are not proofs, but they are triggers for immediate triage: re-check alignment, disable suspicious features, enforce fold-wise preprocessing, and re-run.

Minimal leakage test suite. A practical playbook includes a small deterministic test suite that halts execution if violated: (i) monotonic time index, (ii) no NaNs after declared imputation policy, (iii) feature time index strictly precedes label index by at least one step, (iv) scaling parameters fit only on training data, and (v) walk-forward boundaries respect embargo/purge settings. These tests should be logged with pass/fail status in every run manifest.

11.10.2 Non-stationarity: rolling performance decay and retraining cadence

Even with perfect causality, the feature-label relationship can drift. A model may work in one period and fail in another because the market regime changes, participants adapt, or the risk environment shifts. The diagnostic question is not “did performance drop?” but *how* it dropped: abruptly, gradually, only in certain regimes, or only when volatility is high.

Rolling performance curves. The simplest diagnostic is to compute performance metrics as a function of time: fold-by-fold IC, fold-by-fold Sharpe (for a minimal mapping), and fold-by-fold turnover. A stable model shows modest variation; an unstable model shows sign flips, long flat periods, or concentrated performance in a small window. Plotting fold metrics is more informative than reporting a single pooled number.

Decay signatures. Performance decay can look like:

- *gradual decay*: the signal slowly weakens, suggesting crowding, adaptation, or drift in the underlying mechanism,
- *regime switches*: performance alternates between good and bad periods, suggesting regime dependence (formalized later in Chapter 14),
- *crisis failure*: performance is acceptable in calm periods but collapses in high volatility, indicating tail fragility or insufficient risk conditioning.

Retraining cadence as a control knob. In walk-forward evaluation you implicitly choose a retraining cadence (daily refit, weekly refit, monthly refit). A diagnostic approach is to test a small set of cadences and see whether performance stabilizes. If only extremely frequent retraining keeps performance positive, the model may be fitting noise rather than a durable relationship. If performance is insensitive to cadence, the signal may be more structural. The cadence choice must be treated as part of the protocol and should be selected via validation, not by optimizing the test-period outcome.

11.10.3 Overfitting symptoms in time series ML

Overfitting in time series ML does not always look like the textbook pattern of low training error and high test error. Because markets drift, training and test can differ structurally, so some gap is expected. The more informative question is whether the model’s complexity is being used to capture stable structure or ephemeral quirks of the training window.

Symptom 1: instability across folds. If a model’s out-of-sample metric distribution is wide and frequently negative, but the pooled metric is slightly positive, that is a classic sign of overfitting to specific subperiods. Real edges in finance are often small, but they typically show at least directional stability across many folds.

Symptom 2: sensitivity to small design changes. Overfit models often break when you slightly change the horizon h , the feature window lengths, or the train/test boundary. A robust model should not require a single knife-edge parameter choice. The playbook therefore includes sensitivity analysis: evaluate a small grid of plausible horizons and window lengths and check whether performance is qualitatively consistent.

Symptom 3: complexity outpaces data. If performance improves monotonically as you add more features, interactions, or nonlinear capacity without any regularization discipline, skepticism is warranted. In trading, the sample size is effectively smaller than the raw count because of dependence and overlap. Apparent “free” improvements are often selection artifacts.

Symptom 4: backtest feedback loops. If a research process involves repeatedly modifying features and thresholds based on test performance, then the test set becomes part of training. The symptom is not in the model; it is in the workflow. The diagnostic response is governance: freeze a test period, use validation for iteration, and log every run with configuration hashes so that the number of tries is visible.

11.10.4 Feature instability: regime sensitivity and proxy breakdown

Features are not immutable objects. A feature that represented “trend” may become a proxy for volatility; a liquidity proxy may change meaning when market structure changes; a spread feature may become unreliable when correlation regimes shift. Feature instability is therefore a first-class failure mode.

Distribution drift. Track rolling means, standard deviations, and quantiles of each feature. Large shifts suggest that the model is being fed out-of-distribution inputs. This is especially important after scaling: standardized features can still drift if the scaling window is too long or too short relative to regime changes.

Correlation drift and redundancy changes. Many feature sets include families of correlated transforms (lagged returns, moving-average deviations, volatility-normalized returns). If correlation structure changes, the model’s learned weights can become unstable. A practical diagnostic is to compute rolling correlation matrices (or at least correlations among a small subset of core features) and monitor for major changes.

Regime sensitivity. Compute performance conditional on simple regime proxies that are within prior scope, such as realized volatility buckets (low/medium/high) from Chapter 10 or trend strength proxies from Chapter 07. If a model works only in one bucket, it may still be useful, but it should then be treated explicitly as a conditional strategy rather than a universal forecaster.

Proxy breakdown. A proxy feature can fail when the data-generating process changes. Examples include volume reporting changes, corporate action handling differences, or instrument roll conventions. The playbook response is to log data quality artifacts (missingness rates, zero-volume frequencies, corporate action flags) and to build “feature health” checks: if a proxy

becomes unreliable, the system should reduce exposure or disable the feature until the issue is understood.

11.10.5 Post-modeling pitfalls: costs, slippage, and execution (deferred to Ch18)

A final category of failure modes arises even when forecasting is real: implementation frictions. A model can be predictive and still lose money net of costs because turnover is too high, because signals concentrate in illiquid periods, or because execution assumptions are unrealistic. Chapter 18 will treat microstructure, transaction costs, and slippage in detail. Here we state the minimal diagnostic stance.

Turnover as a warning light. Always report turnover alongside performance. If a model's advantage comes with much higher turnover than baselines, it is likely fragile. Thresholding and hysteresis (introduced earlier in Chapter 11) are basic tools to control this.

Cost sensitivity stress tests. Even before a full cost model, apply a simple per-trade or per-unit turnover penalty and observe whether performance survives. The exact numbers are placeholders, but the qualitative result matters: if the edge disappears under tiny assumed costs, the model is unlikely to survive realistic execution.

Timing realism. Ensure that the backtest uses price points consistent with decision time and execution time. Many apparent edges are artifacts of assuming fills at mid or at the close when the decision was made at the close. This is conceptually part of leakage discipline: you cannot assume you traded at a price you could not have obtained.

In summary, the diagnostic playbook treats failure as expected and manageable. Leakage triage is the first gate, enforced by invariants and fold-wise preprocessing discipline. Non-stationarity is addressed by fold-level performance time series and by evaluating retraining cadence as a controlled choice. Overfitting is detected through instability, sensitivity, and workflow governance rather than by naive train/test gaps. Feature instability is monitored via drift diagnostics and regime-conditional evaluation. Finally, post-modeling pitfalls related to costs and execution are acknowledged early through turnover reporting and cost sensitivity checks, while the full treatment is deferred to Chapter 18. This playbook is not a pessimistic add-on; it is the minimum standard for turning supervised learning results into credible trading research.

11.11 Governance-Native Checklist for Supervised Trading Models

Supervised learning for trading is a domain where the research process itself must be treated as a first-class object. The statistical problem is hard (low signal-to-noise, heavy tails, drift), and the operational stakes are high (a small methodological mistake can create a large economic illusion). This is why a governance-native approach is not bureaucracy; it is the minimum infrastructure required to distinguish signal from artifact. In Chapter 11, governance means three things: (i) *determinism* (so runs can be reproduced exactly), (ii) *lineage* (so every input

and transform can be traced), and (iii) *causality enforcement* (so the system cannot accidentally use the future). The checklist below is designed to be minimal but sufficient: if you implement these items, your supervised forecasting experiments become auditable scientific objects rather than fragile backtests.

The checklist is written as a set of operational commitments, not as generic advice. Each subsection culminates in concrete artifacts that should exist on disk after every run. These artifacts are deliberately redundant: they exist so that six months later you can answer simple questions quickly—“what data exactly did we use?” “what was the decision-time alignment?” “how were features scaled?” “what was the split protocol?” “which seed and configuration produced this figure?”—without re-running the world.

11.11.1 Determinism: seeds, configs, and run manifests

Determinism is the foundation of governed modeling. If you cannot reproduce a result exactly, you cannot audit it, and you cannot know whether a performance change is due to a methodological improvement or due to randomness. In trading ML, sources of nondeterminism include random initialization (in some models), stochastic optimization, randomized train/validation sampling (even when time-ordered, some procedures sample), and any simulation component (bootstrap, Monte Carlo diagnostics). Even when a model appears deterministic, numerical libraries can introduce variation if parallelism or non-fixed ordering is used. Therefore, the governance-native stance is simple: every run must be *fully specified* by a configuration file and a fixed set of seeds, and it must emit a run manifest that records the full provenance of the run.

Seed policy. A supervised trading experiment should define a seed registry with named seeds, for example:

```
seed_data, seed_split, seed_model, seed_eval. (11.89)
```

Even if some seeds are not used in a particular run (because the pipeline is deterministic end-to-end), the registry should exist and be logged, because it provides a consistent interface for future extensions. The important practice is that seeds are *not* left implicit. They are declared in config, and the code reads them from config, not from hidden global state.

Config as the single source of truth. All parameters that affect the result must live in a serialized config object: label horizon h , return definition (simple/log), feature windows, scaling rules, split schedule, model hyperparameters, thresholding rules, risk targeting parameters, and evaluation metrics. The config should be saved verbatim with the outputs. A common error is to tune parameters interactively and then forget which combination produced the reported result. A config-centric workflow prevents this: the run is the config.

Run manifest. Every execution emits a run manifest (e.g., `run.json`) that includes:

- run ID and timestamp,
- config file path (and ideally embedded config contents),
- seed registry,
- code version identifier (git commit hash or a local hash of source files),

- environment snapshot (Python version, library versions),
- list of produced artifacts and their hashes.

The manifest is what makes the run an auditable unit. It also enables comparisons: if two runs differ, the manifest tells you whether the difference is data, code, config, or randomness.

Determinism tests. A minimal test is to rerun the exact same config twice and verify that all output hashes match. If they do not, the pipeline is nondeterministic and must be fixed or at least documented. In high-stakes research, nondeterminism is not an acceptable default.

11.11.2 Data and feature lineage: fingerprints, hashes, and timing proofs

Lineage answers the question: “What exactly went into this model, and how did it become X_t and y_{t+h} ?” In supervised trading, lineage is not just about reproducibility; it is about credibility. Without lineage, you cannot assess survivorship bias, corporate action handling, missingness policies, or alignment decisions. The governance-native approach therefore treats data and features as *versioned artifacts*.

Data fingerprint. A data fingerprint is a compact summary of the dataset that is sufficient to identify it uniquely in practice. At minimum it includes:

- instruments/universe identifiers,
- sampling frequency and calendar policy,
- time span ($t_{\min}$ to $t_{\max}$),
- missingness summary (counts and percentages),
- corporate-action and adjustment policy (split/dividend adjustments, roll conventions if futures),
- any filtering applied (liquidity filters, trading-day filters).

The fingerprint should also include counts: number of rows, number of valid observations per instrument, and any known gaps. This makes it possible to detect silent changes in upstream data.

Hashes for raw and processed data. Fingerprints are descriptive; hashes are definitive. A governed pipeline hashes the raw input file(s) and the processed dataset used by modeling. The processed dataset hash is crucial because it reflects all preprocessing decisions (e.g., how missing data was filled, how returns were computed). Hashing can be done at the file level (hash of a serialized array) and at the artifact bundle level (hash of a directory). The key practice is that any published metric can be traced back to the hash of the exact data used.

Feature lineage and manifest. The feature manifest is the feature analog of the data fingerprint. It lists every feature:

- definition (formula or algorithmic description),
- lookback window(s) and parameters,

- scaling/normalization rule (expanding or rolling, fit policy),
- decision-time alignment rule (including lags/shifts),
- dependencies on raw fields (price, volume, high/low).

In addition to definitions, the manifest should include feature-level quality diagnostics: missingness per feature, basic distribution summaries, and any winsorization/clipping rules.

Timing proofs. Because timing errors are the most damaging silent failures, the pipeline must include an explicit “timing proof” for features and labels. In Chapter 11 terms, this means documenting:

$$X_t \text{ uses only information up to decision time } t, \quad y_{t+h} \text{ uses only information after } t. \quad (11.90)$$

The proof can be operational: e.g., “all rolling windows are one-sided and computed on indices $\leq t - 1$; all features are shifted by one step before being used.” The goal is not to write measure-theory; the goal is to provide an unambiguous statement that an auditor can check against code.

11.11.3 Causality gates: no-look-ahead assertions as hard stops

If determinism and lineage make experiments reproducible, causality gates make them valid. A causality gate is a test that enforces “no look-ahead” and halts the run if violated. These gates must be *hard stops*: a failed causality test invalidates the run and no performance metrics should be produced.

Alignment assertions. At minimum:

- *Index monotonicity*: time indices must be strictly increasing.
- *Feature/label offset*: for horizon $h \geq 1$, the label at index t must correspond to data strictly after t (e.g., price at $t + h$), never at t or earlier.
- *No future-dependent transforms*: forbid centered windows, symmetric filters, and any use of full-sample statistics in preprocessing.

These can be implemented as assertions on arrays and index relationships. What matters is that they exist and are always executed.

Fold-wise preprocessing gates. In walk-forward evaluation, preprocessing must be fit only on training data and then applied to test. A causality gate here is to detect whether any scaler or normalization uses test-period values. Practically, this can be enforced by structuring code so that preprocessing functions explicitly accept a training slice and return fitted parameters, and by asserting that those parameters were computed without accessing test indices.

Overlap and horizon gates. If the label horizon h creates overlapping label windows, the split protocol must either purge the overlapping training observations or impose an embargo. A causality gate checks that the last h training points before each test block are excluded (or that an embargo exists). If the gate fails, fold results are invalid.

Decision-time integrity gates. If the strategy assumes decisions at a particular time (e.g., after close), ensure features are not using fields that would only be available later (e.g., next-day open). This is where data policy meets model validity. The gate is not purely statistical; it encodes the operational contract.

11.11.4 Model cards for trading: assumptions, limits, and monitoring plan

A model card is a structured document that communicates what the model is, what it assumes, what it does not do, and how it will be monitored. In many ML contexts, model cards focus on fairness and user impact. In trading, model cards must also capture *market and execution assumptions* and the fragility points that matter economically.

A trading model card should include:

Problem definition. The target, horizon h , return definition, and the intended usage of the forecast (directional filter, magnitude scaling, ranking score). This prevents the common confusion of using a model trained for one objective as if it were trained for another.

Data scope and limits. Instruments, frequency, time span, missingness policy, corporate action handling, and known weaknesses (e.g., “volume data unreliable before year X”). This is where you document survivorship and data availability concerns.

Feature summary. A concise description of feature families used (returns/volatility, trend, mean reversion, liquidity proxies) and scaling rules. Include the decision-time alignment policy.

Training protocol. Walk-forward schedule, expanding vs rolling windows, retraining cadence, and any embargo/purge rules. Also record the loss function and objective.

Performance summary with stability. Not just averages: include fold-level stability and known failure regimes (e.g., “performance deteriorates in high volatility regime”), plus turnover proxies and any sensitivity to thresholds.

Assumptions and non-goals. This is crucial. For example:

- No explicit transaction cost model (deferred to Chapter 18),
- No portfolio optimization (deferred to Chapter 16),
- No leverage optimization beyond simple caps (deferred to Chapter 17),
- No regime model (deferred to Chapter 14).

Stating non-goals prevents over-claiming and keeps the chapter aligned with the book arc.

Monitoring plan. A minimal monitoring plan includes:

- feature drift monitoring (distribution shifts, missingness),
- forecast distribution monitoring (mean/variance shifts, extreme frequency),
- performance monitoring (rolling IC, rolling Sharpe under the minimal mapping),
- risk monitoring (realized volatility relative to target),
- alert thresholds and actions (reduce exposure, pause trading, retrain).

The plan should also specify how often retraining occurs and what triggers emergency retraining or shutdown.

11.11.5 Minimum reproducible artifact bundle

A governance-native workflow ends each run with a reproducible artifact bundle. The bundle is what makes research portable: you can hand it to a colleague (or to your future self) and they can understand, reproduce, and audit the result without reconstructing the entire environment.

A minimal artifact bundle for Chapter 11 supervised experiments includes the items in Table 11.1 plus a few practical additions:

Core artifacts (must-have).

- **Data fingerprint** and hashes for raw and processed data,
- **Feature manifest** with definitions, windows, scaling, and alignment proof,
- **Label manifest** with horizon, return definition, normalization, thresholds,
- **Split protocol** with walk-forward schedule and boundary rules,
- **Model specification** with algorithm, parameters, objective, seed,
- **Evaluation report** with forecast metrics and stability diagnostics,
- **Backtest handshake** describing forecast→position mapping and risk targeting,
- **Lineage hashes** linking inputs, code, config, and outputs.

Recommended supporting artifacts.

- **Plots** saved as files: rolling IC, fold Sharpe, turnover proxy over time, feature drift summaries,
- **Coefficient or model summary** when interpretable (e.g., OLS coefficients per fold),
- **Error and residual diagnostics** (distribution of errors by subperiod),
- **Run logs** capturing warnings, missing-data events, and gating results.

Artifact organization. A practical convention is to store each run in a directory named by run ID, containing:

```
config.json, run.json, data/, features/, labels/, eval/, plots/. (11.91)
```

The exact structure is less important than consistency. Consistency enables automation: scripts can aggregate results across runs, compare configs, and detect drift.

Why the bundle is “minimum.” The word “minimum” here is deliberate. The items listed are not everything one might want, but they are the smallest set that supports: (i) exact reruns, (ii) auditing for leakage, (iii) interpretation of what was modeled, and (iv) diagnosing why performance changed. If you omit any one of these pillars, supervised trading research becomes fragile. For example, without a feature manifest, you cannot audit timing; without split protocol records, you cannot compare studies; without hashes, you cannot be sure you used the same data; without a run manifest, you cannot replicate environment and config.

The governance-native checklist is therefore a research discipline encoded in artifacts. Determinism ensures results can be reproduced. Lineage ensures the chain from raw data to features to labels is visible and hashable. Causality gates ensure that the chain does not illegally jump forward in time. Model cards ensure that assumptions and limits are explicit and that monitoring is planned from the beginning. The reproducible artifact bundle ensures that the entire experiment is portable and auditable. This is the minimum standard for claiming a supervised trading result is credible in a domain where the easiest thing to predict is your own backtest overconfidence.

Artifact	Minimum content
Data fingerprint	instruments, frequency, span, missingness, corporate-action policy
Feature manifest	definitions, windows, scaling rules, decision-time alignment proof
Label manifest	horizon h , return definition, normalization, class thresholds
Split protocol	walk-forward schedule, embargo/purge rules (if any), period map
Model spec	algorithm, params, training objective, seed, convergence settings
Evaluation report	IC/accuracy, stability by subperiod, turnover proxies, diagnostics
Backtest handshake	mapping forecast→position, risk targeting, constraints (conceptual)
Lineage hashes	hashes for inputs, code version, config, and output bundle

Table 11.1: Minimal governance artifacts for Chapter 11 supervised forecasting experiments.

11.12 Conclusion

Supervised learning enters trading with an attractive promise: learn a mapping from information available at decision time to a future outcome, and use that mapping to tilt positions in your favor. Chapter 11 has treated that promise with the skepticism it deserves. We have framed forecasting as a time-ordered, governance-constrained problem, where the first objective is not to maximize a metric but to avoid self-inflicted errors—especially leakage, misalignment of decision time, and overfitting driven by iterative tuning. Within that disciplined framing, supervised learning can be a powerful component of a trading system. It can also be a remarkably efficient machine for producing false confidence if its limitations are ignored.

11.12.1 What supervised learning can reliably contribute to trading systems

Supervised learning can reliably contribute structure, consistency, and conditionality to trading decisions. Even when the predictive edge is small, a well-governed model can turn diffuse information into a measurable tilt: slightly better ranking of opportunities, slightly improved timing of exposure, or a modest filter that reduces participation in unfavorable states. In practice, the most robust contributions are often not dramatic point forecasts of returns, but *conditional signals* that help a system decide when to act and when to stand down.

First, supervised learning can combine many weak predictors into a single score in a way that is transparent and testable. Hand-crafted rules often encode one feature family at a time (trend, mean reversion, volatility), whereas supervised models can integrate these families and learn

interactions that are difficult to specify manually while remaining within a governed pipeline. Second, supervised learning can provide probabilistic outputs that support decision hygiene: confidence thresholds, abstention regions, and stable ranking. This is particularly valuable for turnover control and for reducing the impulse to trade continuously on weak signals. Third, supervised learning can help build adaptive systems when paired with time-aware evaluation: walk-forward training and monitoring allow models to update as the environment changes, with a clear record of when and why they were retrained.

Finally, supervised learning contributes a disciplined research interface. The requirement to define labels, features, loss functions, and evaluation protocols forces the practitioner to be explicit about objectives and timing. In a domain where ambiguity is the enemy of truth, this explicitness is itself a reliable contribution.

11.12.2 What supervised learning cannot guarantee in markets

Supervised learning cannot guarantee alpha, and it cannot guarantee stability of any discovered relationship. Markets are not stationary data-generating processes. The mapping from features to returns can change because participants adapt, because macro regimes shift, because microstructure evolves, or because a once-rare event becomes common. A model that was genuinely predictive in one window can become uninformative or even inversely informative later. This is not a flaw of the model class; it is a property of the environment.

Supervised learning also cannot guarantee implementability. A forecast that improves statistical metrics can fail economically if it induces high turnover, concentrates exposure in illiquid states, or relies on execution assumptions that are unrealistic. The model does not trade; the system trades. Without explicit transaction cost modeling and execution realism (Chapter 18), even a real forecasting edge can be illusory net of costs. Similarly, supervised learning cannot guarantee robustness to tail risk. Heavy tails and rare shocks mean that performance can be dominated by a small number of periods. A model can “work” on average while failing catastrophically in crises, and statistical training objectives do not automatically align with drawdown control.

Finally, supervised learning cannot guarantee that a positive backtest is not a selection artifact. In a large search space, something will look good. Even with time-ordered splits, repeated iteration on the same dataset can leak information through human choice: changing features, horizons, thresholds, or sample periods until the result improves. This is why governance is not optional and why Chapter 12 is the natural continuation of this chapter’s discipline.

11.12.3 A minimum standard for claiming a forecasting edge

Given these limitations, what is the minimum standard for claiming a forecasting edge in a supervised trading context?

First, the edge must be demonstrated *out of sample under time*. This means walk-forward evaluation with explicit expanding or rolling windows, and fold-level reporting of performance. A single split is not enough. Second, the model must beat *hard baselines* (predict zero/mean, linear or logistic regression) under the same protocol, with comparable turnover and the same feature timing rules. Third, the research must pass *causality gates*: no look-ahead in features,

scaling, labels, or split boundaries, with embargo/purging logic when labels overlap. These gates should be automated and enforced as hard stops.

Fourth, the result must show *stability*, not just an average. Stability means the sign of the information coefficient is consistently positive across folds more often than not, performance does not concentrate in a tiny subperiod, and the edge survives reasonable parameter perturbations (horizon, windows, thresholds). Fifth, the edge must be evaluated through a *minimal signal-to-position mapping* that controls risk (risk targeting) and controls turnover (thresholding), so that the claim is not purely statistical but also connected to a plausible trading mechanism. Finally, the entire experiment must be *reproducible*: configs, seeds, manifests, data fingerprints, feature and label manifests, and output hashes must exist as artifacts.

If these conditions are not met, the appropriate claim is not “we found alpha” but “we have a hypothesis that requires stronger governance and validation.”

11.12.4 Transition to Chapter 12: Regularization, Hyperparameters, Model Selection

Chapter 12 addresses the central vulnerability that remains after Chapter 11’s foundations: controlling complexity without manufacturing false discoveries. Once you move beyond simple baselines, you introduce hyperparameters, regularization choices, feature selection, and model class decisions. Each choice expands the search space and increases selection bias risk. Chapter 12 will formalize disciplined validation under time, discuss regularization as a governance tool (not just a statistical trick), and develop model selection workflows that respect temporal structure. We will treat robust validation as an algorithmic process with logged decisions, carefully handling nested evaluation ideas without violating time ordering. The objective is to make “trying more models” safer: to reduce p-hacking pathways, to improve generalization, and to build research workflows where complexity is earned rather than assumed.

Chapter 12

Regularization, Hyperparameters, Model Selection

Abstract

This chapter formalizes the moment where supervised learning transitions from “fit a model” to “run a disciplined research program.” In trading, targets are noisy, features are plentiful, regimes drift, and many choices that look like harmless implementation details are, in fact, degrees of freedom that can overfit. Regularization is introduced as a first-principles control on effective model complexity, trading a small and explicit bias for potentially large gains in stability and out-of-sample credibility. Hyperparameters are treated as part of the model: penalty weights, window lengths, decays, thresholds, and even split rules encode discretionary choices that must be tuned under time and then audited. We develop time-aware validation protocols that respect causality, explain why random cross-validation is typically invalid, and describe walk-forward evaluation with purging and embargo to prevent overlap contamination. The chapter then addresses selection bias as an endogenous research failure mode: repeated trials and adaptive decisions can manufacture apparent performance even with simple models. We conclude with a governance-native workflow—seeds, run manifests, parameter registries, trial ledgers, and minimum artifact bundles—that makes model selection defensible and prepares the reader for higher-capacity models in Chapter 13 without mistaking sophistication for evidence.

12.1 Position in the Book

12.1.1 Prerequisites (what you must already know)

Chapter 12 sits at a very specific inflection point in the arc of this book. Up to this point, we have been building the conceptual and computational infrastructure required to even *state* what it means for an algorithmic trading model to be credible. The prerequisites are not merely a list of topics that were covered earlier; they are a set of habits of thought and implementation rules that must already be internalized before the words “regularization” and “model selection” can mean anything operational rather than rhetorical.

From Chapter 01 you must already carry the decomposition of an algorithmic trading system into its functional engines: signal formation, execution, risk, and governance. This matters here because model selection is not a choice among abstract statistical objects; it is a choice among end-to-end systems that will be deployed under market constraints. Chapter 01 also established the core asymmetry that defines trading research: markets supply abundant data points, but not abundant *independent* information. A decade of daily returns is thousands of observations but only a small number of effective regimes. That asymmetry is the seed of why overfitting is the default and why selection discipline is not optional.

From Chapter 02 you must be able to implement the entire workflow in transparent numerical code. This is not a stylistic preference; it is the governance foundation of the book. When you build rolling computations, resampling, splits, scalars, and estimators from first principles in NumPy, you are forced to confront timing, indexing, and the subtle ways leakage occurs. Chapter 12 assumes you can write those operations without outsourcing the critical logic to opaque abstractions. If you cannot trace exactly how a feature at time t was computed, you cannot claim that your validation protocol respects causality, and model selection becomes an exercise in self-deception.

Chapter 03 provided the language of returns, risk, and portfolio math that anchors evaluation. Model selection is only meaningful relative to a target objective, and in trading the objective is rarely “minimize squared error” in isolation. Even when we use regression losses as proxies, we are implicitly optimizing a tradeable outcome: expected return per unit risk, stability of forecasts, or downstream performance under constraints. You must already understand how returns aggregate, how volatility and covariance behave, and why a model that improves a statistical metric can still fail at the portfolio level. Chapter 12 does not yet enter portfolio optimization (that belongs to Part IV), but it presumes you can interpret performance in risk-aware terms and can distinguish between statistical significance and economic significance.

Chapter 04 contributed a deeper prerequisite: the anatomy of trading time series. Markets are not i.i.d. draws from a stationary distribution; they exhibit autocorrelation structures that can be regime-dependent, volatility clustering, heavy tails, microstructure noise, and time-varying relationships between predictors and outcomes. In Chapter 12, this anatomy is not background color; it is the reason the chapter exists. Regularization is a response to instability under correlated predictors and heavy-tailed residuals. Time-aware validation is a response to dependence and non-stationarity. If you have not internalized that “more data” does not automatically mean “more generalization,” you will misread what regularization and

hyperparameter tuning can and cannot accomplish.

Chapter 05 is arguably the most direct technical prerequisite, because Chapter 12 is fundamentally about selection *over pipelines*. Feature engineering, scaling, normalization, and the alignment of signals with tradable decision times are the typical sites where leakage enters. Chapter 05 established the causal contract: every transform must be computable from information available at or before t , and any parameter fit on data (means, variances, quantiles, PCA directions, thresholds) must be fit only on a training window and then applied forward. Chapter 12 assumes you can enforce these rules mechanically. Without them, the most sophisticated validation protocol collapses, because the model is being evaluated on a dataset that already encodes future information through preprocessing.

From Part II, especially Chapter 06, you must be able to interpret backtests as simulations with explicit event timing. Model selection that ignores decision time is not model selection; it is pattern matching after the fact. Chapter 06 taught that a strategy is defined by what information is known when a trade is placed, how positions evolve, and what happens when the system is stressed. Chapter 12 sits upstream of execution costs and microstructure (Chapter 18), but it inherits the key conceptual discipline: you must define a forecast horizon, a signal time, a trade time, and a holding or evaluation window. Your validation splits and your purging/embargo rules only make sense once those timing conventions are explicit.

Finally, Chapter 11 is the immediate prerequisite in the narrow sense: it introduced supervised learning for forecasting returns, the baseline philosophy, and the core evaluation vocabulary. Chapter 12 assumes you can construct naive baselines that are hard to beat, define targets cleanly, and interpret forecasting performance in relation to a downstream decision. Most importantly, Chapter 11 introduced the principle that evaluation must match how the model will be used. Chapter 12 operationalizes that principle in the presence of hyperparameters, adaptive search, and multiple comparisons. In other words, if Chapter 11 taught you to ask “does my model predict?” Chapter 12 teaches you to ask “did my research process cheat without noticing?”

12.1.2 What this chapter adds (the new capability)

The contribution of Chapter 12 is to turn supervised learning into a disciplined research program. In practice, the difference between a fragile result and a defensible claim is almost never that the researcher knew a more exotic model. The difference is that the researcher constrained flexibility, measured stability, and treated the selection process as a first-class object subject to governance.

The first capability added is a first-principles understanding of regularization as controlled complexity. In trading data, predictors are often correlated, samples are not independent, and residuals are heavy-tailed. Under these conditions, unconstrained estimation tends to be unstable: coefficient vectors change dramatically when the training window shifts by a small amount, and forecasts become sensitive to tiny perturbations in features. Regularization introduces explicit constraints that trade some bias for a reduction in variance and sensitivity. Chapter 12 frames this not as a textbook bias–variance cartoon but as an operational question: does the model behave consistently across time folds, and does it avoid extreme parameter settings that appear predictive only within a narrow historical slice? Ridge, lasso, and elastic net are treated as archetypes not because they are fashionable, but because they embody different stability and

interpretability tradeoffs that recur later in the book.

The second capability is the elevation of hyperparameters to first-class status. Trading researchers often speak of hyperparameters as “settings,” but in finance those settings are frequently the real model: window lengths define the memory of the system, decays define the effective sample, thresholds define when the system acts, and penalty strengths define how much instability is tolerated. Tuning these quantities is not free; it consumes credibility. The chapter therefore teaches the reader to treat hyperparameter tuning as an optimization problem under constraints: a declared search space, a declared budget, a declared metric, and a declared selection rule. This is where Chapter 12 becomes governance-native: it does not merely recommend good practice, it insists that the tuning process be recorded in a trial ledger and that the research artifact include evidence of sensitivity rather than only a single best configuration.

The third capability is time-aware model selection. Many standard ML validation procedures assume exchangeability or near-i.i.d. sampling, and therefore they fail in market settings. Chapter 12 provides protocols that treat validation as a simulation of deployment: holdouts that respect time, rolling and expanding windows, walk-forward evaluation, and the conceptual rules of purging and embargo to address overlap contamination. The key capability is not the vocabulary; it is the ability to design a split scheme that matches decision time and horizon, and to interpret performance as a distribution across folds rather than a single statistic. This shift forces a different standard of evidence: a model that performs acceptably across many folds and does not collapse in the worst fold is more credible than a model with a brilliant average driven by a single lucky regime.

The final capability is the integration of all of the above into an auditable governance workflow. Chapter 12 introduces the minimum artifact bundle that makes selection claims defensible: deterministic seeds, run manifests, parameter registries, split specifications, causality gates, trial ledgers, and stability reports. The objective is reproducibility with accountability: another researcher should be able to reconstruct not only the final chosen model, but also the path by which it was chosen and the alternatives that were rejected. This is the chapter where “research hygiene” becomes a formal component of the trading system itself.

12.1.3 What this chapter postpones (explicitly out of scope)

A disciplined scope boundary is essential here because model selection is where projects often become uncontrolled. Chapter 12 explicitly does *not* introduce new model families beyond the supervised-learning toolkit established in Chapter 11. The chapter is about credibility under selection, not about expanding the hypothesis class. Deep neural networks, representation learning, and specialized regularization techniques such as dropout, batch normalization effects, and architecture-level priors belong to Chapter 13, where they will be treated as higher-capacity extensions that demand even stricter Chapter 12 discipline.

Similarly, regime detection and state models are postponed to Chapter 14. Chapter 12 acknowledges non-stationarity, but it does not yet formalize the market as switching among latent states, nor does it develop selection rules that are conditional on regime classification. That would require introducing additional modeling objects and evaluation protocols that distinguish between within-regime and cross-regime generalization. The book postpones that complexity

deliberately: first we learn to control selection bias and leakage in the simpler setting, then we generalize.

Event-driven and news/sentiment pipelines are postponed to Chapter 15. The reason is not that they are unimportant; it is that they multiply degrees of freedom dramatically. Feature definition becomes less standardized, labeling becomes more ambiguous, and timing becomes more subtle. If Chapter 12 is not mastered, event-driven research becomes a factory for false discoveries. Therefore, the book treats Chapter 12 as a gatekeeper: only after you can audit tuning and validation for structured numerical features do you move into noisier and more discretionary data sources.

Portfolio-level optimization, position sizing, leverage control, and execution frictions are deferred to Part IV (Chapters 16–18). Chapter 12 remains focused on the forecasting and selection layer. It does not claim that a better predictor automatically yields a better strategy once costs, constraints, and portfolio interactions are introduced. Those interactions will later become the central object of study. Here, the goal is narrower: ensure that the predictor you bring into Part IV is not a mirage produced by a flexible and ungoverned research process.

Finally, Chapter 12 explicitly avoids guaranteeing profitability. Regularization can stabilize estimates, validation protocols can reduce leakage, and governance can make claims auditable, but none of these mechanisms can manufacture an edge in a market that does not offer one. What the chapter provides are standards for credibility: conditions under which a claim is at least falsifiable, reproducible, and less likely to be a product of accidental p-hacking. In the logic of the book, that is the correct promise at this stage: not “we will win,” but “we will know when we are likely fooling ourselves.”

12.2 Regularization from First Principles

12.2.1 Restricting hypothesis space as a robustness instrument

Regularization is the deliberate restriction of the set of functions a model can represent, but it is more useful to think of it as restricting the set of *stories* a model is allowed to tell about a market. Without regularization, a flexible estimator can always construct a narrative that fits a particular slice of history: a precise combination of indicators, a narrow set of lags, a specific volatility filter, a convenient transformation of volume. The danger is not that the model is “complex” in some aesthetic sense, but that it is *adaptable* to noise. In trading, adaptivity is costly because market relationships drift, because returns are weakly predictable, and because the research process itself tends to be iterative and opportunistic. Regularization is therefore a robustness instrument: it is the mechanism that forces a model to behave in a constrained, stable manner when the data invites it to chase randomness.

The most concrete way to describe hypothesis space restriction without elevating mathematical complexity is to describe what changes in the implementation. Consider a supervised forecasting setup from Chapter 11: we construct a feature vector x_t (causally, using only information up to time t), define a target y_{t+h} (a forward return or a label derived from it), and fit a mapping $f_\theta(x_t) \approx y_{t+h}$. If we allow θ to take any values that minimize the training loss, the fitted mapping can become sensitive to tiny quirks in the training window. Regularization modifies

the fitting objective so that the algorithm is *penalized* for choosing unstable parameterizations, or it modifies the training protocol so the algorithm is not given enough freedom to memorize patterns. The result is not just “smaller coefficients” or “simpler models”; the result is a system whose forecasts do not swing violently as we slide the training window or perturb the feature set.

There are two broad channels for restricting the hypothesis space: explicit penalties and implicit constraints. Explicit penalties change the objective function by adding a term that discourages certain parameter configurations. This is the canonical ridge/lasso/elastic net family, but the chapter-level point is broader: the penalty is a formal expression of what behaviors you consider fragile. For example, a penalty can discourage large coefficients (reducing sensitivity to feature noise), discourage too many nonzero coefficients (reducing dependence on many weak predictors), or discourage rapid parameter changes over time (encouraging smoothness across retraining dates). Implicit constraints achieve the same restriction without adding a penalty term: you can reduce the number of features, impose monotone relationships, cap exposures, restrict interaction terms, or apply early stopping in iterative fitting so the estimator is not allowed to fully adapt to the training sample. In practice, many robust trading systems combine both channels: they apply explicit regularization in the model fit and implicit regularization in the feature set and training protocol.

In trading, the need for regularization is amplified by the kinds of features we build. Many feature engineering recipes deliberately create redundant views of the same underlying concept: multiple moving averages to measure trend; multiple z-scores to measure deviation from mean; multiple volatility estimates to measure risk; multiple volume ratios to measure activity. Redundancy is not automatically bad; it can increase robustness if the model learns stable combinations. But without regularization, redundancy creates a wide space of equivalent explanations, and the estimator can pick an explanation that is accidentally aligned with noise in the training window. A classic symptom is the “model that re-derives itself” every time you refit: coefficients change sign, the chosen subset of features changes, and performance looks inconsistent across folds even when the average metric seems acceptable. Regularization is the mechanism that narrows the set of permissible explanations so that the model is forced to choose a stable representation.

A second reason regularization is a robustness instrument is the unavoidable mismatch between research conditions and live conditions. In research, you often know the full dataset, you can compute global diagnostics, and you can iterate. In live trading, you only have the past. Regularization is a way to encode humility: it admits that the data may not justify a highly tuned, highly specific mapping, and it prefers a model that performs slightly worse in-sample but does not collapse out-of-sample. This is the key cultural shift the chapter wants to instill: in trading, the objective is not to maximize backtest metrics; it is to maximize the probability that the model behaves similarly under new conditions. Regularization is the main tool for increasing that probability without assuming more stationarity than markets actually provide.

12.2.2 Stability and estimation error under correlated predictors

The practical motivation for regularization in financial forecasting can be summarized in one sentence: *your features are almost never independent, and your effective sample size is almost never as large as you think*. Both facts lead to unstable estimation, and regularization is the

simplest way to manage that instability.

Start with correlated predictors. In engineered trading datasets, correlation appears for structural reasons. A 20-day moving average and a 30-day moving average are correlated because they are both smoothed versions of the same underlying price path. A 5-day momentum and a 10-day momentum are correlated because they share overlapping return contributions. Volatility estimates computed via rolling windows are correlated because adjacent windows share most of their observations. Even features that look conceptually different can be correlated because they respond to the same latent regime changes: trend and volatility often move together in crises; volume-based measures and volatility measures often spike together; cross-asset features can become correlated during macro-driven episodes. The consequence is that there are many near-equivalent ways to fit the training data. Without regularization, the estimator is free to choose among these near-equivalent solutions, and small perturbations in the sample can push it toward a different solution.

The second issue is effective sample size. Even if you have T daily observations, the number of independent “market situations” may be far smaller. Regime shifts, volatility clustering, and structural breaks mean that your training data is not a collection of identically distributed events. This matters for estimation error: parameter estimates can look precise if you treat observations as independent, but they are not. In such settings, an unconstrained fit can become too confident in patterns that are specific to one regime. This is why models can appear stable in one period and then fail abruptly: they were tuned to a regime-specific correlation structure that did not persist.

Stability is the operational concept that connects these issues. In this chapter, stability means several related things:

- **Parameter stability:** if you refit the model on adjacent training windows, do the coefficients remain similar in magnitude and sign?
- **Forecast stability:** if the feature vector x_t changes slightly due to noise or if the scaling parameters drift, do forecasts change smoothly or do they jump?
- **Selection stability:** if the model involves feature selection (explicitly or implicitly), does it repeatedly choose similar features across folds, or does it behave like a lottery among correlated proxies?
- **Performance stability:** across time folds, does performance remain within a reasonable band, or is the average driven by a small number of exceptional periods?

Regularization improves stability by shrinking and constraining the solution so that it is less sensitive to sample-specific quirks. In plain terms, it reduces the ability of the model to exploit the degrees of freedom created by correlated predictors.

Estimation error under correlation can be described without heavy math by focusing on the geometry of fitting. When predictors are correlated, many coefficient vectors produce similar training predictions because changing one coefficient can be offset by changing another. The loss surface becomes “flat” along certain directions: the training data cannot distinguish between many combinations. An unconstrained optimizer can drift along these flat directions and land

on a solution that looks good in-sample but is brittle out-of-sample. A penalty term acts like a gentle force that pulls the solution toward a region of the parameter space that is smaller in magnitude or simpler in structure. This is why ridge tends to shrink coefficients and stabilize them; why lasso tends to choose a sparse representation; and why elastic net can reduce the instability of lasso when predictors are strongly correlated.

In trading applications, correlated predictors are not only a statistical issue; they are a governance issue. When a model relies on delicate cancellations among many correlated features, it becomes hard to audit and hard to maintain. Small data changes (a revised corporate action adjustment, a single missing value in volume, a vendor change in bid-ask data) can alter the cancellations and therefore alter forecasts. Regularization reduces the reliance on such delicate cancellations. This is also why practitioners often prefer models with fewer active features and smaller exposures: not because they dislike sophistication, but because they have lived through the operational consequences of unstable estimators.

It is also important to emphasize that stability is not a synonym for “better average performance.” Regularization often reduces in-sample fit, and it can reduce out-of-sample performance in periods where the signal is strong and stable. The chapter’s criterion is deployability: a model that occasionally outperforms spectacularly but frequently underperforms and changes character is difficult to use in a live system. A model that performs moderately but consistently is often more useful because it can be combined, risk-managed, and monitored. Regularization is one of the few tools that systematically pushes models toward the latter behavior.

12.2.3 Penalized risk minimization: the canonical template

The chapter uses penalized empirical risk minimization as its canonical template because it is simple, transparent, and widely applicable across model classes. The fitting problem is written as

$$\hat{\theta} \in \arg \min_{\theta \in \Theta} \left(\mathcal{L}(\theta; \mathcal{D}) + \lambda \Omega(\theta) \right), \quad (12.1)$$

but the most important content is the interpretation of each component in a trading pipeline.

The dataset $\mathcal{D}$ is not a generic table; it is a *causally constructed* object. Each row corresponds to a time t , features x_t are computed using information available at or before t , and the target y_{t+h} is defined using an evaluation horizon h that is consistent with the decision you want to simulate. This matters because regularization does not rescue you from causal errors. If $\mathcal{D}$ leaks future information, the penalty simply stabilizes a cheating model. Therefore, the first technical detail is that penalized risk minimization must be embedded in time-aware data construction: fold-specific scalars, fold-specific feature computation when parameters are data-dependent, and split protocols that prevent overlap contamination.

The loss $\mathcal{L}(\theta; \mathcal{D})$ encodes what the model is trying to match. In forecasting returns, it could be squared error, absolute error, logistic loss for a directional label, or a custom loss that weights errors differently across regimes. Chapter 12 keeps the mathematical level modest, but it insists on a practical detail: the loss must match the evaluation objective used in selection. If you fit with squared error but select based on Sharpe-like downstream metrics, you must at

least understand that you are fitting one objective and selecting on another, which can induce instability and overfitting in the selection step. The chapter therefore encourages consistency: choose a loss that aligns with the decision metric or, if not possible, treat the mismatch as a risk factor and audit it.

The penalty $\Omega(\theta)$ expresses what forms of complexity you want to discourage. In this chapter, the penalty is interpreted as a stability preference. A penalty that discourages large coefficients discourages extreme sensitivity to feature noise. A penalty that encourages sparsity discourages dependence on many weak predictors and can improve interpretability. In practice, penalties can also encode structural preferences: group structure, smoothness across time, or constraints that reflect known relationships. The key technical detail is that the penalty changes not only the fitted parameters but also the entire selection landscape: different penalties create different sensitivities to hyperparameters and different stability properties across folds.

The scalar λ controls the strength of the penalty, and Chapter 12 makes a specific interpretive move: λ is not a nuisance constant; it is a governance knob. This is because λ controls how much you allow the model to adapt to the data. Low λ means the model is allowed to fit aggressively; high λ means the model is forced toward simpler, more stable solutions. In research practice, tuning λ is therefore an act of choosing a credibility level. If a result requires λ to be extremely low and is highly sensitive to small changes in λ , that is evidence of fragility. If performance is reasonably stable across a wide band of λ values, that is evidence of robustness. This is why the chapter insists on sensitivity plots: you do not only report the best λ ; you report how performance behaves as λ changes.

There is also an engineering interpretation of penalized risk minimization that matters in trading pipelines: penalties act as guardrails against ill-posedness. When you have more features than effective samples in a training window, or when features are nearly collinear, unregularized fitting can be numerically unstable. Regularization can improve numerical conditioning and produce more reliable solutions. This is a technical but non-mathematical point: the penalty is not only about statistics, it is also about computation. A model that sometimes fails to converge or produces wildly varying coefficients because of conditioning issues is not deployable; regularization reduces that risk.

Finally, penalized risk minimization integrates naturally with the chapter’s governance artifacts. The penalty strength and penalty type are hyperparameters that must appear in the parameter registry. The selection rule that chooses λ must be declared (e.g., maximize mean validation metric, choose the simplest model within one standard deviation of the best, minimize a stability-penalized score). Each trial must log λ , the fold metrics, and stability diagnostics. The result is that the selection process becomes auditable: you can see whether the chosen λ was a lucky spike or part of a stable region.

12.2.4 Bias–variance as an operational diagnostic

The phrase “bias–variance tradeoff” is often taught as a theoretical slogan, but in trading research its value lies in how it guides diagnostics. The central operational question is: *how much does the model change when the data changes in ways that plausibly occur in live deployment?* Regularization is valuable when the answer is “a lot.”

The first diagnostic is training-window sensitivity. You refit the model on adjacent windows (for example, rolling by one month) and examine how coefficients and performance move. If coefficients flip sign or if performance swings widely, the model is high variance in the deployment sense. Regularization should reduce this effect. Importantly, this diagnostic is not about proving a theorem; it is about measuring the system’s stability under realistic retraining schedules.

The second diagnostic is split-boundary sensitivity. In time-series validation, small changes in where you cut the train/validation boundary can change the sample composition near the boundary, which is precisely where leakage and overlap problems live. A robust model and protocol should not collapse when the boundary shifts slightly. Regularization contributes by reducing the model’s dependence on boundary-specific coincidences, but the protocol (purge/embargo) is equally important. Chapter 12 treats these as complementary: regularization stabilizes the estimator; split design stabilizes the evaluation.

The third diagnostic is feature-set perturbation. In practice, feature sets evolve: a feature may be removed because of data quality issues, a new feature may be added, or a vendor change may slightly alter the computation. A high-variance model is one whose behavior changes dramatically when you make modest feature-set perturbations. Regularization can reduce sensitivity by shrinking coefficients and discouraging reliance on many marginal features. The diagnostic here is simple: refit with a subset of features removed (or with small noise added to features) and check whether performance and forecasts degrade smoothly rather than catastrophically.

The fourth diagnostic is hyperparameter sensitivity. Instead of reporting only the best hyperparameter configuration, you examine how performance varies across a grid or a local neighborhood of λ and other settings (window lengths, decays). A robust model exhibits a “plateau” region: many nearby configurations perform similarly. A fragile model exhibits sharp spikes: one configuration looks excellent, and nearby ones collapse. Chapter 12 treats sharp spikes as the practical signature of selection risk. Regularization often converts spikes into plateaus by smoothing the solution space.

Fold dispersion is the core summary statistic that makes these diagnostics concrete. If you run walk-forward evaluation, you obtain a set of fold performances. The mean is not enough; you want the dispersion and the tail. A model that produces a high mean but a disastrous worst-fold is a model that likely learned regime-specific behavior. Regularization often improves worst-fold behavior at the cost of reducing peak performance, which is usually a desirable trade in deployment. This is the operational bias–variance story: accept some bias (reduced peak fit) to reduce variance (reduced dispersion and tail risk across folds).

It is also important to connect bias–variance diagnostics to governance. If you do not log the diagnostics systematically, you will remember only the best run. This is why Chapter 12 insists on a stability report as part of the artifact bundle: coefficient dispersion across folds, selection frequency for features, sensitivity curves, and fold-by-fold performance. The point is not to be bureaucratic; the point is to prevent the researcher’s cognitive selection bias from becoming part of the model selection mechanism.

Finally, the chapter emphasizes that bias–variance is not only a property of the model class; it is a property of the entire pipeline. Feature engineering choices, scaling schemes, target definitions,

and validation protocols all affect the effective variance of the final system. Regularization is a central tool, but it is not a magic fix. The operational stance is therefore integrated: if you observe high sensitivity, you can respond by increasing penalty strength, simplifying features, increasing embargo, changing the evaluation horizon, or reducing the hyperparameter search space. Chapter 12 teaches you to treat these responses as controlled interventions in a research process, each of which must be logged and justified.

In summary, “regularization from first principles” in this book means something very concrete. It means you treat the model as a constrained instrument designed to be stable under time, correlation, and drift. You interpret penalties as explicit preferences for robustness. You treat λ and other hyperparameters as governance knobs that control how much the model is allowed to adapt. And you measure the tradeoffs with operational diagnostics that reflect deployment reality: window sensitivity, split sensitivity, feature perturbation sensitivity, hyperparameter sensitivity, and fold dispersion. That is the practical content of regularization in an algorithmic trading context, and it is the discipline that makes the subsequent chapters on higher-capacity models intellectually and operationally defensible.

12.3 Canonical Penalties and Their Trading Interpretations

12.3.1 ℓ_2 (Ridge): shrinkage, multicollinearity, and smooth exposure

Ridge regularization shrinks coefficients continuously and is particularly effective when predictors are correlated, which is common in technical indicators and engineered features. In trading terms, ridge often produces smoother exposure profiles: it discourages extreme bets driven by collinear features and reduces the probability that the model “locks onto” a spurious linear combination that happens to correlate with the target in a particular historical slice.

12.3.2 ℓ_1 (Lasso): sparsity pressure and selection instability under time

Lasso induces sparsity by pushing many coefficients to zero, producing a compact model and an implicit feature selector. However, under time variation and collinearity, the identity of selected features can be unstable: lasso may switch between correlated proxies across folds. This chapter treats such switching as a diagnostic of fragility, motivating stability reporting and the idea that selection behavior itself is an empirical object to be audited.

12.3.3 Elastic Net: correlated predictors with stable sparsity

Elastic net combines ℓ_1 and ℓ_2 effects, often delivering a compromise: fewer active predictors than ridge, but less erratic selection than pure lasso when predictors are strongly correlated. For trading pipelines with many overlapping indicators, elastic net can be a practical default, especially when interpretability and stability are valued alongside predictive performance.

12.3.4 Structured penalties (conceptual): blocks, groups, and hierarchies

Trading features often come in blocks: trend family, mean-reversion family, volatility family, microstructure family. Structured penalties treat these blocks as coherent units, encouraging

selection at the block level or imposing hierarchical constraints such as selecting a family before selecting its variants. We keep this idea conceptual here and treat it as a bridge to better feature governance: the structure of the feature set can serve as a regularizer without introducing new model classes.

12.4 Hyperparameters Are Part of the Model

12.4.1 Definition: hyperparameters in a trading pipeline

A hyperparameter is any quantity chosen by the researcher that materially affects forecasts but is not estimated by the core fitting procedure on a fixed dataset. That definition sounds simple until you confront how trading systems are actually built. In practice, a trading “model” is not only a fitted mapping from features to targets. It is an end-to-end object whose behavior is determined by dozens of upstream and downstream choices that are often treated as mundane implementation details. Chapter 12 insists on a conceptual correction: if a choice changes the forecasts you generate or the trades you would place, then it is part of the model in the only sense that matters for deployment.

The reason this correction is essential is that discretionary choices can overfit as powerfully as estimated parameters. A linear regression with ten coefficients can be made to “work” through sufficiently adaptive feature engineering, window selection, scaling rules, label definition, and repeated re-evaluation. In that case the coefficients are not the main source of flexibility; the hyperparameters and protocol choices are. This is why the chapter adopts a governance-native stance: hyperparameters must be treated as first-class objects, logged, versioned, and audited, because they encode decisions the researcher made after seeing data, whether consciously or unconsciously.

In this book’s workflow, hyperparameters appear at four layers: (i) the feature layer, (ii) the model-fitting layer, (iii) the decision layer, and (iv) the evaluation layer. At the feature layer, hyperparameters include window lengths for rolling statistics, half-lives for exponential decays, thresholds for clipping or winsorization, and any parameter that defines how raw prices and volumes are transformed into predictors. At the model-fitting layer, hyperparameters include regularization strengths, choices of loss function variants, stopping criteria, and any algorithmic setting that changes how the estimator is fit. At the decision layer, hyperparameters include thresholds that convert forecasts into actions, holding period rules, and filtering conditions such as “only trade when volatility is below a cap.” At the evaluation layer, hyperparameters include split design parameters such as the length of validation blocks, purge widths, embargo lengths, and any rule that changes how training and testing are separated in time. A critical point for Chapter 12 is that these evaluation-layer hyperparameters are not neutral: they determine what information can flow from one period to another. If they are tuned adaptively, they can produce selection bias even when the predictive model is fixed.

This expanded notion of hyperparameters is not theoretical pedantry; it is the practical explanation for why many trading studies fail to replicate. Two researchers can “use the same model” but produce different results because they made different choices about scaling, missing data handling, horizons, windows, and split design. The model parameters are the same type of

object in both studies, but the hyperparameters define different pipelines and therefore different models in practice. Once you accept that hyperparameters are part of the model, it follows that the model-selection problem is not merely “choose λ ” or “choose L ”; it is choose a pipeline specification under explicit constraints and then defend that choice with time-aware evidence.

A useful operational test is the counterfactual question: if you changed this setting, would the strategy behave differently in a way that matters economically? If the answer is yes, you must treat the setting as a hyperparameter. For example, changing the volatility lookback from 20 to 60 days changes position sizing or filter rules; changing the forward-return horizon from 1 day to 5 days changes both the target and the evaluation overlap structure; changing the normalization scheme changes the relative scale of features and therefore the learned coefficients; changing the embargo length changes the effective independence of folds. Each of these choices can be used to improve apparent performance in-sample, and therefore each must be governed.

Chapter 12 also treats hyperparameters as the interface between statistical modeling and market microstructure realities, even before we formally study microstructure (Chapter 18). Consider a simple example: you forecast next-day return using features computed from close-to-close data. If you assume the signal is tradable at the close, you are embedding an implicit execution assumption that may be unrealistic; if you assume you trade at next open, you are shifting the decision time and thus the target alignment. That “timing choice” is not a parameter in your regression, but it is a hyperparameter in the pipeline because it changes what would be tradable. Many apparent forecasting edges are in fact timing edges created by inconsistent definitions of when information becomes available. Treating timing rules as hyperparameters forces you to put them in the open.

The governance point can be stated bluntly: if hyperparameters are not treated as part of the model, you are likely to understate your effective degrees of freedom. You will believe you fit a simple model, when in fact you searched across many pipeline variants. This undercounting is one of the dominant sources of false discoveries in algorithmic trading research. Therefore, Chapter 12 requires that any serious study maintain a parameter registry that includes not only model penalties but also feature windows, horizons, scaling choices, thresholds, and validation protocol parameters. The registry is the object that makes hyperparameter choices auditable.

12.4.2 Search spaces are mixed and constrained

Once hyperparameters are treated as first-class objects, the next practical reality emerges: the search space is rarely a clean continuous box where standard optimization intuitions apply. Trading hyperparameter spaces are mixed, discrete, conditional, and constrained, and this structure is a major reason why naive tuning procedures can mislead.

Start with discreteness. Many economically meaningful hyperparameters are discrete by nature: a lookback of 20, 60, or 120 days corresponds to widely used horizons (one month, one quarter, half a year). A holding period might be 1 day, 5 days, or 20 days. A rebalancing frequency might be daily, weekly, or monthly. These are not continuous knobs where a small change from 20 to 21 is qualitatively similar; sometimes it is, but often it is not. Discreteness means that hyperparameter tuning is frequently a model selection problem among qualitatively different behaviors. This is also why the search space can be deceptively large: if you allow

dozens of possible lookbacks for several features, the combinatorial explosion can create far more degrees of freedom than the dataset can support.

Now consider conditionality. Many hyperparameters only matter if certain features or transforms are enabled. A decay rate is irrelevant if you are not using an exponentially weighted feature. A threshold for a volatility filter is irrelevant if the filter is disabled. A penalty for a group of coefficients is irrelevant if that group is not present. Conditionality creates a tree-structured search space: you first choose a pipeline branch (which features are included, which transforms are used), and then you tune the settings within that branch. In trading research, many false discoveries are created by silently moving between branches without logging the branch choice. A disciplined approach treats branch choices as hyperparameters too and records them.

Constraints are even more central in trading than in many other ML applications because of time. A split protocol that is valid for one label horizon may be invalid for another. If your target is a 5-day forward return, then adjacent labels overlap by construction, and your embargo must exceed that overlap to avoid contamination across folds. If you compute rolling features with a 60-day window, then a fold boundary must include enough warm-up observations so that features in the test window do not depend on training data (or, if they do, you must treat that dependence explicitly). These constraints mean that hyperparameters cannot be tuned independently. A simple tuning loop that varies the horizon without adjusting the embargo is not just suboptimal; it is potentially invalid because it changes the leakage properties of the evaluation.

In addition to time constraints, there are economic and operational constraints. A model that uses a very short window might be too sensitive to microstructure noise; a model that uses an extremely long window might be too slow to adapt to structural changes. A model that selects too many features may be difficult to maintain, interpret, or compute in production. A model whose thresholds cause excessive turnover may be infeasible once transaction costs are considered. Chapter 12 does not yet model costs in detail, but it already recognizes that feasibility constraints should shape the hyperparameter space. This is part of disciplined research: you do not search over hyperparameters that you know would produce an untradeable strategy, because doing so inflates degrees of freedom and yields misleading research outcomes.

A practical implication is that the hyperparameter search space should be specified explicitly as an object. That object includes: (i) the list of tunable hyperparameters, (ii) their allowed values or ranges, (iii) conditional dependencies, and (iv) constraints that enforce validity (e.g., $\text{embargo} \geq \text{label overlap}$; $\text{training window length} \geq \text{maximum feature window}$; $\text{validation block length} \geq \text{minimum sample size for stable fitting}$). In code, this is the parameter registry. In governance, it is the document that allows an auditor to answer: what did you allow yourself to vary?

This explicit specification has two benefits. First, it reduces accidental adaptivity. If you define the search space before you run experiments, you are less likely to expand it opportunistically when results disappoint. Second, it allows you to compute a realistic notion of tuning budget and degrees of freedom. When the search space is a conditional tree, the number of possible pipelines can be much larger than the number of parameter settings within a single pipeline. Treating both as hyperparameters helps you understand where the real flexibility lies.

It is also important to recognize that mixed and constrained spaces alter what “optimization” should mean. In many ML contexts, practitioners imagine hyperparameter tuning as finding the global optimum. In trading, that mindset is dangerous. The objective surface is noisy, non-stationary, and often dominated by regime-specific luck. The search should be designed to discover robust regions and to quantify sensitivity, not to chase a single best configuration. Mixed spaces support that viewpoint naturally: instead of pretending the space is smooth, you treat it as a set of plausible design choices and you ask which designs remain credible under time-aware evaluation.

12.4.3 Sensitivity and fragility maps

The chapter’s central message about tuning can be summarized as follows: the goal is not to find the best hyperparameter configuration on a validation set; the goal is to map the region of hyperparameters where performance is stable, and to treat instability as evidence against deployability. This is why Chapter 12 emphasizes sensitivity and fragility maps.

A sensitivity map is any representation that shows how performance changes as hyperparameters change. The simplest map is one-dimensional: performance versus λ for a regularized model, or performance versus window length for a rolling feature. More complex maps can be two-dimensional: performance versus $(\lambda, \text{window length})$, or performance versus $(\text{decay rate, threshold})$. The precise plotting choice is less important than the discipline: you do not report a single number; you report a surface or curve that reveals whether the result is robust.

Fragility appears in several forms. The most obvious is the sharp optimum: one setting looks excellent, neighboring settings look mediocre. Sharp optima are dangerous in trading because they often signal that the model exploited a coincidence of the training window. In live deployment, you will not be able to hold the hyperparameter at exactly that optimum: data distributions shift, retraining windows change, and features drift. A sharp optimum therefore carries a built-in failure mode: small changes in conditions destroy performance. Regularization and disciplined selection aim to prefer plateau regions where many nearby settings behave similarly.

Another form of fragility is fold instability. Even if the average performance across time folds is good, the dispersion may be high. High dispersion indicates that the strategy depends on specific market conditions that occur in only some folds. Chapter 12 treats the worst-fold outcome as a crucial diagnostic because it approximates the question: what happens when the market enters a regime that is unfavorable to your learned relationships? A stable model is not necessarily one that never underperforms, but one whose underperformance is within a tolerable band and whose behavior does not change character.

Coefficient and selection stability provide another fragility lens. For ridge-like penalties, you can track the dispersion of coefficients across folds. For lasso-like penalties, you can track the frequency with which each feature is selected across folds. In trading, unstable selection among correlated features is common, and it should not be hidden. Instead, it should be reported as part of the model’s identity: “this predictor is one of several proxies, and the model alternates among them.” If the alternation is too chaotic, it suggests that the model is not learning a stable relationship, and regularization strength or feature design should be revisited.

A subtle but important fragility phenomenon is hyperparameter interaction. A penalty strength that looks reasonable for one feature set may be too weak or too strong when the feature set changes. A window length that works in one regime may not work in another. This creates interaction effects that can mislead if you tune hyperparameters sequentially without revisiting earlier choices. Chapter 12 addresses this by advocating for sensitivity analysis across multiple hyperparameters and by treating the pipeline as the object of study. You do not tune λ in a vacuum; you tune λ conditional on a feature and protocol specification, and you examine whether the tuned value remains sensible when reasonable perturbations occur.

To keep the mathematical level modest, the chapter frames sensitivity analysis as a series of concrete experiments rather than formal derivatives. You can run a grid over λ values and record fold-by-fold metrics. You can vary window lengths and record the stability of coefficients and forecasts. You can shift split boundaries slightly and observe performance changes. You can add small noise to features to emulate measurement error. These experiments do not require advanced mathematics; they require discipline and logging. The outputs become the fragility maps: not just a line plot, but a structured report of how the system responds to perturbations.

Sensitivity analysis also interacts with governance in a critical way. If you do not plan and log sensitivity experiments, you will tend to look only at the best-performing configuration. Human cognition is a selection mechanism, and in trading research it can become a hidden optimizer. Chapter 12 therefore treats sensitivity maps as mandatory artifacts: they are designed to prevent the researcher from concluding “it works” based on one peak. They force the researcher to confront whether performance is stable enough to justify moving forward.

Finally, sensitivity maps are the bridge to honest interpretation. Suppose you find that performance is good only for a very narrow range of hyperparameters. The correct conclusion is not necessarily “discard the strategy.” The correct conclusion is that the strategy is sensitive and therefore high-risk. That sensitivity should influence how you proceed: you might reduce model capacity further, simplify features, widen the evaluation horizon, or treat the strategy as one component in a diversified ensemble later (Chapter 20). But you cannot responsibly present the result as a robust edge. Chapter 12 teaches you to distinguish these interpretations, and sensitivity maps are the evidence that supports them.

12.4.4 Tuning budgets as credibility budgets

Hyperparameter tuning consumes compute, but in trading the more important scarcity is statistical credibility. Every additional configuration you try increases the probability that you will find a configuration that looks good by chance. This is the selection-bias version of “if you torture the data long enough, it will confess.” Chapter 12 therefore reframes tuning budgets as credibility budgets: the more you spend, the more skeptical you must be of the result, and the more stringent your evaluation must become.

The first governance move is to declare a tuning budget *ex ante*. This budget can be expressed as the number of configurations in a grid, the number of random trials, or the number of iterations in an adaptive search. The budget is part of the run manifest and the parameter registry. Declaring it *ex ante* is important because it prevents the common failure mode of expanding the search until something works. If you begin with a budget of 100 trials and you end

up running 2,000 because results were disappointing, you have changed the statistical meaning of your “best” result. Chapter 12 does not forbid expanding the budget, but it insists that doing so be logged and interpreted as increased selection risk.

The second governance move is to maintain a trial ledger. The ledger records every configuration tried, the split protocol used, the fold metrics obtained, and the outcome status (accepted, rejected, failed due to numerical issues, failed due to causality gate). The ledger is the key artifact that makes tuning auditable. Without it, the reported best configuration is not interpretable, because you do not know how many alternatives were explored and how adaptive the search was.

Adaptive tuning deserves special attention. In many settings, adaptivity is a strength: you learn from previous trials and move toward better regions. In trading research, adaptivity is also a risk because it allows information from the validation set to guide future choices. The more adaptive the search, the more the validation set becomes part of the training process. Chapter 12 addresses this by requiring that adaptive rules be disclosed. If you use an adaptive search, you must state the rule by which trials were chosen, and you must consider nested evaluation if you want an unbiased performance estimate.

The chapter also highlights that tuning budgets are not only about the number of trials, but about the breadth of degrees of freedom. Tuning five hyperparameters each with ten possible values is not “50 trials”; it can be up to 10^5 combinations if you allow them to interact. Even if you sample only a subset, the existence of a large combinatorial space affects the risk of cherry-picking. Therefore, credibility budgeting includes controlling the size of the space itself: limit the number of hyperparameters you tune simultaneously, constrain ranges to economically plausible values, and avoid expanding the feature set opportunistically.

A practical technique consistent with the chapter’s modest mathematical level is to adopt conservative selection rules that reduce the chance of choosing a lucky spike. For example, instead of choosing the configuration with the best mean fold metric, choose the simplest configuration whose performance is within a tolerance of the best, or choose the configuration that maximizes a stability-adjusted score (mean minus dispersion). The precise rule can vary, but the principle is the same: the selection criterion should incorporate stability, not only peak performance. This is another way tuning budgets relate to credibility: if you searched widely, you should be more willing to sacrifice a small amount of average performance to gain stability.

Tuning budgets also interact with reporting. A result that required extensive tuning is not automatically invalid, but it must be presented with the correct epistemic posture. If a strategy’s performance depends on delicate hyperparameter choices discovered after many iterations, it should be reported as fragile and treated as hypothesis-generating rather than deployment-ready. Conversely, if a strategy performs reasonably across a broad region with minimal tuning, it deserves more credibility. Chapter 12 teaches this asymmetry explicitly: robustness is evidence; luck is not.

Finally, tuning budgets must be understood in the context of time. Each time fold is effectively a different environment. If you tune on one set of folds and then evaluate on others, you reduce the chance of overfitting to a particular period. But if you repeatedly tune after observing performance on later folds, you are leaking future information into your choices.

Therefore, credibility budgeting includes strict separation between tuning and final evaluation. In practice, this often means nested selection or at least a final locked test window that is not touched until all tuning decisions are complete.

In summary, treating hyperparameters as part of the model changes the entire logic of trading research. It forces you to recognize that pipeline choices are degrees of freedom, that search spaces are mixed and constrained by time, that stability must be mapped rather than assumed, and that tuning budgets are credibility budgets that must be declared, logged, and interpreted. These principles do not require higher mathematics; they require higher discipline. Chapter 12 is where that discipline becomes formal, auditable, and inseparable from what it means to claim that a supervised trading model has any chance of generalizing beyond the past.

12.5 Validation Under Time: Designing Splits That Preserve Causality

12.5.1 Why random cross-validation is typically invalid

Random k -fold cross-validation is seductive because it is simple, widely taught, and often effective in domains where samples can be treated as approximately independent. In trading, that independence assumption is rarely defensible. The core problem is not philosophical; it is mechanical. A financial dataset is a time-ordered object in which the construction of each row typically involves rolling windows, overlapping labels, and transformations whose parameters are estimated from data. When you randomly shuffle rows into folds, you are not merely rearranging observations; you are changing what information is allowed to flow between training and testing. In a market setting, the result is almost always a form of leakage, and leakage in trading is often subtle enough to survive informal “sanity checks” while still inflating performance.

The most direct failure mode arises from overlapping labels. Suppose your target y_{t+h} is a h -day forward return. Then y_{t+h} and y_{t+1+h} share $h - 1$ daily returns. This overlap is not an exotic corner case; it is the default whenever you forecast multi-day outcomes. If you randomly assign these adjacent timestamps to different folds, you are effectively allowing the training set to include outcomes that share return components with the test set. The model may not literally see the test label, but it sees a closely related label that is mechanically coupled to it. This creates optimistic estimates that are not available in live deployment, where future returns are unknown and cannot be indirectly borrowed from nearby periods.

A second direct failure mode is overlap induced by rolling features. Many predictors in trading are rolling statistics: moving averages, rolling volatility, rolling correlations, rolling z-scores. These features are functions of past data, but they are also functions of *shared* past data across adjacent rows. When you randomly split rows, the training set and the test set share large portions of the same raw observations inside their feature computations. Again, this is not automatically invalid if the computation is strictly causal, but it becomes problematic because the model is evaluated on test features that are statistically dependent on training features in a way that does not resemble deployment. In live trading, the dependency exists too, but the key difference is that training and testing are separated by time. Random splitting removes that separation and therefore makes the evaluation unrealistically easy.

A third, and often underestimated, leakage channel is preprocessing leakage. Consider normalization and scaling: if you compute means, standard deviations, quantiles, clipping thresholds, or any other data-derived transform using the full dataset, you are encoding future distributional information into the training process. Many practitioners know this in principle, yet random cross-validation makes it far easier to commit this error accidentally because the folds interleave in time. Even if you compute scalers within each fold, the act of random mixing means each training fold contains samples from across the full historical period, so fold-specific scalers become implicitly informed by later regimes. In live deployment, you cannot know the distributional properties of future regimes when you set scaling rules today. Random cross-validation therefore grants the model an advantage it would not have.

Indirect leakage is equally damaging and often more difficult to detect. Markets evolve, and the structure of data-generating processes changes across time. Random folding mixes regimes: the training set contains samples from periods that occur after the test samples. Even if you prevent direct label overlap, the model can still benefit from learning relationships that only become true later. This is a form of “distributional hindsight.” The model is trained with knowledge of the future environment and then tested on the past environment, which is the reverse of deployment. The consequence is that the model appears more stable and more accurate than it would be when trained only on information available up to the test period.

There is also a practical engineering reason random cross-validation is misleading: it conflates the question of generalization across *samples* with the question of generalization across *time*. In trading, the relevant generalization problem is time generalization: can a model trained on past data produce useful forecasts in later periods, under the same decision-time constraints? Random cross-validation answers a different question: can a model interpolate among samples drawn from a mixture of time periods? That question is not useless, but it is not the one that governs live performance. When a model fails in production, it typically fails because the relationship it learned was not stable across time, not because it could not generalize to another random subset of the same mixed-regime dataset.

This is why Chapter 12 treats random cross-validation as typically invalid for trading research, not as a stylistic preference. The problem is not that random CV is always wrong in every conceivable financial dataset; the problem is that it is too easy to obtain flattering results that are artifacts of temporal mixing, overlap contamination, and distributional hindsight. In a governance-native workflow, a validation protocol must have a clear mapping to the deployment timeline. Random splitting breaks that mapping by construction, and therefore it cannot be the default.

12.5.2 Holdout and walk-forward as deployment simulations

Time-aware validation begins with a simple principle: evaluation should simulate how the model would actually be used. In live trading, you train on history that occurred before today, you fit any preprocessing rules on that history, you choose hyperparameters based on performance observed on a subsequent period, and then you deploy forward. A time-aware split design recreates this sequence explicitly. This is why the simplest valid baseline is a chronological holdout: train on an early period and evaluate on a later period.

A holdout split forces the first discipline that random CV avoids: you cannot use information from the future to fit the past. You select a cut date T_{cut} , fit your full pipeline (features, scalers, model) using data up to T_{cut} , and then evaluate strictly on the period after T_{cut} . If the model performs well, the result is at least consistent with deployment logic. If the model fails, the result is informative, because the failure occurred under a realistic temporal constraint rather than under a regime-mixed random split.

However, a single holdout is only a beginning. Markets are heterogeneous, and any one test period can be unusually favorable or unusually hostile. A model that performs well in a particular decade may fail in another decade; a model that thrives in trending regimes may fail in choppy regimes. Walk-forward evaluation addresses this by repeating the train–test exercise through time. The idea is simple: choose a sequence of evaluation dates, train the model on the window before each date, and test it on the subsequent block. The output is not a single performance number but a distribution of performance across market environments.

Walk-forward evaluation is best understood as a deployment simulation with a retraining schedule. In practice, trading systems are retrained periodically: daily, weekly, monthly, or when a threshold condition triggers retraining. Walk-forward evaluation mimics this by defining a cadence. For each fold, you train on a past window, optionally tune hyperparameters on a validation sub-window, and then test on the next out-of-sample block. You then roll forward and repeat. This structure makes it possible to answer practical questions that a single holdout cannot: does performance degrade when volatility rises? does the model become unstable around structural breaks? does the coefficient vector drift smoothly or jump?

Two common variants of walk-forward evaluation are rolling windows and expanding windows. In a rolling window scheme, the training set is a fixed-length lookback (for example, the most recent three years) that moves forward through time. This mimics a system that deliberately forgets older data to adapt to new regimes. In an expanding window scheme, the training set grows as time progresses (for example, from the start of the dataset to the current fold date). This mimics a system that accumulates information and assumes that older data remains partially relevant. Neither scheme is universally superior; they encode different beliefs about stationarity. The key is that the choice is explicit and consistent with how you would run the system live.

Technical details matter here, but they can be stated without heavy mathematics. Each fold must have enough history to compute features causally. If your features require a 60-day rolling window, then your training period must include at least 60 days before the first training label, and your test period must include enough warm-up to compute features without borrowing from future data. Similarly, if you use data-dependent preprocessing (scaling, clipping thresholds, normalization), those parameters must be fit on the training portion of each fold and then applied forward to validation and test. This fold-specific fitting is not an optional refinement; it is the mechanism that prevents the pipeline from learning future distributional properties.

Walk-forward evaluation also forces you to confront a second deployment reality: models are not static objects. Even if you keep the hyperparameters fixed, refitting the model on new data changes the fitted parameters. Walk-forward evaluation therefore produces a time series of fitted models, and this sequence can itself be audited. You can examine parameter drift, forecast

drift, and performance drift. In a governance-native workflow, this sequence is not discarded; it is an artifact. It tells you whether the model is learning a stable relationship or whether it is constantly reinventing itself.

Finally, walk-forward evaluation creates the natural setting for Chapter 12’s emphasis on stability. Instead of asking whether the model has a high average metric, you ask whether performance is consistent, whether the worst-fold outcomes are tolerable, and whether the model’s behavior changes character. This is the trading interpretation of generalization: not a single out-of-sample statistic, but a pattern of behavior across time that resembles what you might experience in deployment.

12.5.3 Purging overlap and embargo (protocol rules)

Time-aware splits are necessary but not sufficient. Even if you respect chronological order, you can still leak information across the train/test boundary through overlap structures that are intrinsic to how labels and features are constructed. Purging and embargo are protocol rules designed to neutralize this contamination. They are not cosmetic “cleanups”; they are the difference between a plausible evaluation and an optimistic artifact.

To see why, consider the simplest overlap: forward-return labels. If your target is the h -day forward return, then the label at time t depends on returns from $t + 1$ through $t + h$. Now imagine a fold boundary where the training period ends at time T_{train} and the test period begins at $T_{\text{test}} = T_{\text{train}} + 1$. The last training label, at T_{train} , depends on returns that extend into the test period (from $T_{\text{train}} + 1$ onward). That means the training set contains information that is literally constructed from test-period returns. Even if the model does not see the test labels, the training labels embed test-period returns. This is leakage.

Purging addresses this by removing training observations near the boundary whose labels overlap the test period. In the example above, you would remove the last $h - 1$ training observations (or more precisely, remove all training timestamps whose label window reaches into the test window). The principle is simple: if the label construction for an observation uses returns that occur in the test period, that observation cannot be used for training. Purging therefore reduces the size of the training set near boundaries, but it restores the causal separation that deployment requires.

Features can create overlap too, especially when they are defined using rolling windows that extend up to time t . At first glance, rolling features are causal, so why do they matter for overlap? They matter because adjacent feature vectors share many of the same raw observations. If you place the boundary too close, you create a situation where training and test samples are highly dependent through shared raw history. In deployment, this dependence exists across adjacent times, but the training and test periods are separated by time in a way that reflects how the model would be trained and then deployed. If the boundary is immediate and the overlap is extreme, the evaluation can behave more like interpolation than genuine forward prediction. This is particularly problematic when the model includes strong regularization or when you tune hyperparameters aggressively, because the model can effectively exploit the near-duplication of information across boundary-adjacent samples.

An embargo adds a buffer zone after the training period during which test observations are

not used for evaluation. The embargo is designed to prevent contamination from boundary effects that are not fully removed by purging alone. In practice, the embargo is often chosen to be at least as long as the overlap horizon in labels and the effective memory of the features. The exact rule can be stated operationally without formal derivations: you choose an embargo length E such that observations within E steps after the boundary are excluded from testing, ensuring that the test set is sufficiently separated from the training set that overlap dependencies do not dominate.

A useful way to think about purging and embargo is that they are time-series analogues of “don’t let the test set touch the training set.” In i.i.d. settings, we separate sets by sampling without replacement. In time-series settings with overlapping constructions, separation must be created by removing boundary-adjacent observations. Purging removes observations whose labels contaminate the other side; embargo removes observations that are too close for independence in practice. Together, they create a cleaner separation between what was known when the model was trained and what is being predicted out of sample.

These rules also interact with feature computation warm-up. If the first test observation requires a rolling window of length L , you must ensure that those L observations are available without borrowing from training in a way that violates your intended separation. There are two consistent approaches: either allow the rolling window to use pre-test history (which is realistic, because in deployment you have past history), or recompute features within the fold using only permitted history. The key is to be explicit. If you allow pre-test history, you must still ensure that your label and evaluation contamination is addressed; if you recompute features within the fold, you must ensure you have sufficient warm-up data. The protocol must specify which approach is used.

Purging and embargo also have an uncomfortable but essential consequence: they reduce the effective sample size. This is not a defect; it is the cost of honesty. Many trading studies look impressive because they implicitly count overlapping observations as independent and therefore report overly precise estimates. Chapter 12 treats reduced sample size as a warning that you are operating in a low-information environment, which is exactly when regularization and conservative selection become critical. A disciplined protocol accepts smaller effective samples in exchange for validity.

In governance terms, purging and embargo are not ad hoc choices made after seeing results. They are protocol parameters that must be selected ex ante, logged in the run manifest, and applied consistently across all candidate models and baselines. If you change embargo length after observing performance, you are tuning the evaluation protocol itself, which is a direct path to selection bias. Therefore, the rule is simple: if purging and embargo exist in the protocol, they must be part of the parameter registry, and any change must be treated as a hyperparameter change with explicit disclosure.

12.5.4 Metric choice must match the decision

Validation under time is not only about how you split data; it is also about what you measure. Metrics are part of the model because they define the objective that guides selection. If you choose a metric that does not correspond to the trading decision, you may select a model that

is statistically attractive but economically irrelevant, or you may encourage hyperparameter tuning that exploits artifacts of the metric rather than learning a robust relationship.

The simplest example is mean squared error (MSE). MSE is a reasonable loss for symmetric continuous prediction problems, but in trading the decision is often asymmetric. A forecast that is slightly wrong in magnitude but correct in sign may be more valuable than a forecast that is accurate in magnitude but frequently wrong in sign, depending on how the forecast is used. If the downstream strategy is directional (long if forecast positive, short if forecast negative), then sign accuracy, hit rate, or a utility-weighted score that penalizes wrong-way predictions more heavily may be a better proxy for economic value. Conversely, if the downstream decision is allocation or ranking (tilt exposure toward assets with higher predicted returns), then correlation-based metrics, rank correlation, or information coefficient (IC) style measures can be more relevant because they capture whether the ordering is useful even when magnitudes are noisy.

Metric choice also interacts with non-stationarity. In many market periods, returns have low variance; in others, variance is high. A metric that weights all errors equally across time may overemphasize high-volatility periods or may underemphasize the stability of forecasts in calm periods. Similarly, if you use classification labels derived from returns (e.g., top/bottom quantile), class balance can change across regimes. In such settings, metrics like accuracy can be misleading because they can be dominated by the majority class. Time-aware validation must therefore consider whether the metric behaves consistently across folds and whether it is robust to regime shifts.

Another technical detail is calibration versus ranking. Some models produce forecasts that are useful for ranking but poorly calibrated in level. If you intend to use forecasts to size positions (later chapters), calibration matters; if you intend to use forecasts only as signals to rank opportunities, ranking quality matters more. Chapter 12 does not yet model sizing mechanics, but it already insists that you be explicit about which role the forecast is meant to play. The metric should reflect that role. A common failure mode is to optimize a calibration-based loss and then evaluate as if ranking is the objective, or vice versa. This mismatch can create instability in hyperparameter tuning because the selection criterion is effectively optimizing a different object than the fitting procedure.

Time-aware evaluation also benefits from reporting multiple metrics, but with discipline. Reporting many metrics after the fact can become a form of metric shopping: you highlight the metric that looks best. The governance-native approach is to select a primary metric *ex ante* (the one that matches the intended decision) and then report secondary diagnostics that help interpret stability (dispersion across folds, worst-fold outcomes, sensitivity curves). The primary metric guides selection; the secondary metrics guide interpretation and risk assessment. This separation reduces the chance that you will redefine success after seeing results.

Metrics must also be applied consistently across baselines and candidates. If a candidate model uses a metric that favors its particular behavior while a baseline is evaluated under a different metric, the comparison is invalid. Chapter 12 insists that baselines from Chapter 11 be carried forward under identical split protocols, identical purging/embargo rules, and identical metric definitions. Only then can you claim that an improvement reflects a real increase in predictive or decision usefulness rather than a change in evaluation conventions.

Finally, metric choice must be logged and auditable. The run manifest should record the metric definition, any transformations applied (e.g., normalization of targets, clipping), and the exact aggregation method across folds (mean, median, stability-adjusted score). Without this, model selection becomes irreproducible and subject to hidden degrees of freedom. In a governance-native workflow, a metric is a formal object: it is defined, versioned, and applied in a repeatable way.

In summary, validation under time is a complete protocol: split design, purging and embargo rules, fold-specific preprocessing, and metric choice that matches the decision. Random cross-validation fails because it breaks the deployment mapping and invites leakage through overlap and regime mixing. Holdout and walk-forward evaluation restore the deployment sequence and produce stability evidence across environments. Purging and embargo protect the boundary from overlap contamination and force honest effective sample sizes. Metric discipline ensures that selection optimizes what matters for the trading decision rather than what flatters the backtest. Together, these elements turn validation from a convenience into an auditable simulation of how the model would actually live in the market.

12.6 Model Selection Procedures

12.6.1 Single split: the minimal defensible protocol

The single-split protocol is the smallest unit of model selection that can be defended in a trading context without resorting to hand-waving. Its value is not that it is statistically optimal; its value is that it forces you to separate what you *learn* from what you *claim*. In markets, where research is iterative and the temptation to adapt is constant, that separation is the core governance goal. The minimal protocol can be stated in one sentence: define the full pipeline, split time into train/validation/test in chronological order, tune hyperparameters on validation only, and report final performance on the held-out test exactly once.

The phrase “define the full pipeline” is doing more work than it seems. In this book, the pipeline includes every step that affects the forecast: feature construction (windows, decays, transforms), preprocessing (scaling, clipping, normalization), target definition (horizon, labeling rules), model class (e.g., linear model with a penalty), training procedure (loss function, optimizer, stopping rule), and the decision-time alignment that makes features tradable. The minimal protocol assumes you commit to this specification before you look at test results, because otherwise the test set becomes part of your design loop. In practice, this means you freeze a configuration object: a parameter registry that lists all tunable hyperparameters and all fixed choices, and a split specification that defines the boundaries and any purge/embargo rules.

Once the pipeline is defined, you choose the chronological split. The training period is the historical window used to fit model parameters and any data-derived preprocessing parameters. The validation period is the period used to tune hyperparameters and to decide among competing pipeline variants. The test period is held back as a proxy for the future and is not touched until all selection decisions are complete. The key is that each period must be separated in time to preserve causality. If the horizon is multi-day or if rolling features create overlap, you incorporate the Chapter 12 protocol rules (purging and embargo) at the boundaries, so the

training set does not contain labels that reach into validation and the validation set does not contaminate the test.

Hyperparameter tuning then occurs only on the validation set. This means that you fit the model (and fold-specific preprocessors, if needed) on the training set for each hyperparameter configuration, evaluate on validation, and select the configuration according to a declared selection rule. The simplest selection rule is to choose the configuration that maximizes a validation metric, but Chapter 12 encourages more conservative rules that incorporate stability: for example, choose the simplest configuration within a tolerance of the best or choose the configuration with the best stability-adjusted score. The point is not which rule you choose; the point is that you choose it *before* looking at the test.

Finally, you evaluate once on the held-out test. This “evaluate once” rule is the hardest discipline in practice because it is psychologically unnatural: you want to peek, adjust, and retry. But the moment you peek and adjust, the test set ceases to be a credible proxy for the future. The right mental model is that the test set is a one-time experiment. You are allowed to learn from it, but if you learn from it, you must then redefine it as part of training/validation and find a new test period. This is not bureaucratic purity; it is the only way to keep your claims honest.

The tradeoff of the single-split protocol is variance. A single test period may be unusually favorable or unfavorable, and therefore the reported performance estimate can be noisy. The benefit is clarity and reduced opportunities for silent adaptivity. In trading research, where degrees of freedom can explode through feature and hyperparameter choices, clarity is often the more valuable property. The single split provides a baseline: if you cannot demonstrate a plausible improvement under this discipline, more elaborate procedures are unlikely to rescue the result in a credible way.

There is also a governance benefit: the single split is easy to audit. An auditor can read the split boundaries, review the trial ledger of validation tuning, and verify that the test period was not used in selection. In environments where you want reproducible research artifacts (students, colleagues, institutional committees), this simplicity is a feature. Chapter 12 treats the single-split protocol as the minimal credible baseline, not as the final destination.

12.6.2 Time-series cross-validation variants

Time-series cross-validation generalizes the single-split idea by repeating it across multiple contiguous folds to obtain a stability picture. The motivation is straightforward: markets are not one environment. A model that works in one regime can fail in another, and a single held-out test window might not capture that variability. Time-series CV therefore trades some simplicity for a richer set of evidence: instead of one out-of-sample evaluation, you get many out-of-sample evaluations across different periods.

The defining constraint is that folds must preserve time order. Unlike random CV, you do not mix periods; you partition the timeline into blocks and simulate repeated training and forward testing. Two common variants are blocked CV and rolling CV. In blocked CV, you define disjoint blocks for training and testing, often using a sequence like: train on block 1, test on block 2; train on block 1–2, test on block 3; and so on. In rolling CV, you define a fixed-length training window that moves forward: train on years 1–3, test on year 4; train on years 2–4, test

on year 5; etc. The difference encodes a belief about relevance of older data: blocked/expanding CV assumes older data remains useful; rolling CV assumes the system should adapt and forget.

Both variants must integrate the Chapter 12 contamination controls: purging and embargo. If labels overlap across boundaries (e.g., multi-day forward returns), then the folds must remove boundary-adjacent observations whose labels depend on returns in the next block. If rolling features or preprocessing steps create dependency structures, embargo can be added to prevent near-boundary contamination from dominating. In code, this means that fold construction is not simply “slice arrays by indices.” It is “slice arrays by indices, then remove boundary regions according to overlap rules, then recompute fold-specific preprocessors.” This is where the “no pandas” first-principles mindset matters: you must be explicit about which indices are included in each fold and why.

The output of time-series CV is not just an average metric. In this chapter, the dispersion across folds is treated as first-class evidence. You want to know not only the mean, but also the variability and the tail. A model with a good average but a disastrous worst fold is a model that likely depends on a particular market environment. That may be acceptable if you have a regime filter (later chapters), but it is a fragility warning in Chapter 12. Conversely, a model with slightly lower average but tighter dispersion may be more deployable because it behaves consistently.

Fold-by-fold analysis also supports stability diagnostics beyond performance. You can track coefficient drift across folds, selection frequency for features, and sensitivity of forecasts to hyperparameters. For ridge-like penalties, you might examine whether coefficients remain similar across folds. For sparse models, you might examine whether the same features are selected. These diagnostics turn model selection into a robustness assessment: you are not only selecting the best-performing model, you are selecting the model whose behavior is stable enough to justify belief.

Time-series CV also forces a different view of hyperparameter tuning. In the single-split protocol, you tune hyperparameters using one validation period. In time-series CV, you can either (i) fix hyperparameters globally (chosen from an initial tuning step) and then evaluate across folds, or (ii) tune hyperparameters within each fold, mimicking a system that re-tunes as it retrain. The second approach is more realistic for deployment, but it introduces additional tuning variance and additional selection risk. Chapter 12 does not mandate one approach; it insists that whichever approach you choose, you must log it and interpret results accordingly. If hyperparameters are re-tuned in each fold, then the fold performance includes the effect of tuning, and the tuning budget must be counted as part of the research degrees of freedom.

A practical implementation detail is fold sizing. Training blocks must be large enough for stable estimation, especially when features are numerous and correlated. Testing blocks indicated as “out-of-sample” should be long enough to produce meaningful metrics rather than being dominated by a handful of events. Validation blocks must be long enough to tune hyperparameters credibly. If blocks are too small, fold metrics become extremely noisy and can mislead selection. This is one of the reasons Chapter 12 emphasizes stability and sensitivity: small folds can create sharp optima that are artifacts of sampling noise.

In short, time-series CV is valuable not because it produces a more impressive average metric,

but because it produces a richer evidence set. It tells you whether the model generalizes across time, whether performance collapses in specific environments, and whether parameter behavior is stable. In markets, those answers are often more important than a single number.

12.6.3 Nested selection: protecting evaluation from tuning bias

Nested selection is the protocol-level solution to a simple but profound problem: if you use the same data to tune hyperparameters and to estimate final performance, your performance estimate is biased upward. The bias arises because tuning is a search over configurations, and the best configuration on a given validation set is likely to be a lucky configuration that exploited noise. This is not an accusation; it is a statistical reality. The more you search, the more the validation set becomes part of the fitting process, and the less it can serve as an unbiased estimate of future performance.

Nested selection addresses this by separating “choose” from “evaluate” through two levels of splitting. The outer loop defines a sequence of out-of-sample evaluation folds. For each outer fold, you treat the outer test block as sacred: it is not used for tuning. Inside the outer training block, you define an inner split (or inner time-series CV) used for hyperparameter tuning. The algorithm is: within each outer fold, use the inner process to choose hyperparameters, refit the model on the outer training data using those chosen hyperparameters, and then evaluate on the outer test block. Repeat across outer folds. The final reported performance is aggregated across outer tests, not inner validations.

Conceptually, nested selection mirrors deployment. In a live system, you will tune hyperparameters using historical data available up to a certain date, then deploy into the future. The outer fold represents the future. The inner fold represents the tuning process using the past. By repeating this across time, you simulate multiple deployments and obtain a performance estimate that is less contaminated by tuning bias.

In trading research, nested selection is particularly important because tuning degrees of freedom are typically large. You may be tuning penalty strengths, feature windows, decays, thresholds, and even protocol parameters. Without nesting, it is easy to report a performance estimate that is essentially “the best we found after searching,” which is not the same as “what we should expect in the future.” Nested selection reintroduces honesty by making each outer test block untouched by the tuning process that selected hyperparameters.

There are practical considerations. Nested selection is compute-intensive because tuning occurs inside each outer fold. If your hyperparameter search is expensive, nesting multiplies that cost. Chapter 12 therefore treats nested selection as a governance choice tied to tuning budget: if you search widely, you should be more willing to pay the cost of nesting, because wide search increases bias. If your model is simple and your hyperparameter space is small, a simpler protocol may be acceptable. The key is to treat this tradeoff explicitly, not implicitly.

Nested selection also clarifies reporting. Under nesting, you can report two kinds of outputs: (i) the outer-fold performance distribution, which is your primary evidence about generalization, and (ii) the inner-fold tuning behavior, which is evidence about sensitivity. For example, if the inner loop selects wildly different hyperparameters across outer folds, that is evidence that the “best” hyperparameters are not stable and that the model is sensitive. Conversely, if selected

hyperparameters are similar across folds, that is evidence of robustness. This is a major benefit of nesting: it produces a record of how the tuning process behaves through time, not just a final chosen setting.

Nested selection also interacts with the governance artifacts emphasized in this chapter. The parameter registry defines the search space. The trial ledger records inner-loop trials. The run manifest records split boundaries for both inner and outer loops, including any purge/embargo rules. The stability report includes dispersion across outer folds and sensitivity of inner selections. Together, these artifacts make the entire selection process auditable: an external reviewer can see how many trials were run, what was chosen, and how performance was estimated without peeking.

Finally, nested selection is the protocol that makes it possible to be honest about hyperparameter tuning while still doing it. Chapter 12 does not claim that tuning is forbidden; it claims that tuning must be governed. Nested selection is the technical expression of that governance: it acknowledges that tuning uses information from data, and it ensures that performance estimation remains forward-looking and minimally biased.

12.6.4 Early stopping as regularization (conceptual)

Early stopping is often introduced in the context of deep learning, but the underlying idea is model-agnostic and fits naturally within Chapter 12 because it is a method of controlling effective complexity without changing the model class. The basic intuition is simple: many fitting procedures are iterative. As the optimizer runs, the model fits the training data progressively better. At some point, further improvement on training data begins to reflect fitting noise rather than learning stable structure. Early stopping halts the process before that overfitting occurs.

To keep the mathematical level modest, we treat early stopping as a protocol rule rather than a theoretical object. You choose a validation criterion, monitor it as training progresses, and stop when the criterion stops improving. The criterion can be a validation loss, a validation metric aligned with the trading decision, or a stability-adjusted score. The early stopping point is therefore a hyperparameter: it is chosen based on validation behavior. This is why early stopping belongs in the model-selection chapter. It is a way of selecting a model complexity level implicitly through training dynamics.

Early stopping can be understood as regularization because it limits the model’s ability to fit high-frequency noise. In many iterative algorithms, early iterations capture broad structure, while later iterations capture finer, noisier details. Stopping early prevents the model from chasing the latter. This is visible even in simple contexts: iterative solvers for regularized regression, boosting-like procedures, and any gradient-based optimizer. You do not need deep networks to see the effect. The chapter’s point is that training dynamics themselves create degrees of freedom, and early stopping is a governance mechanism that constrains them.

However, early stopping must be treated with the same discipline as other selection procedures. If you monitor the test set to decide when to stop, you have leaked test information. If you use random splits to decide when to stop, you may have mixed regimes and created optimistic stopping points. In a time-aware workflow, early stopping should be controlled using a validation period that is chronologically after the training period, and it should respect purge/embargo rules

when labels overlap. In other words, early stopping is not an excuse to weaken the validation protocol; it is another hyperparameter selection that must be embedded inside it.

Early stopping also interacts with stability goals. A model that requires extremely precise stopping to perform well is fragile, just like a model that requires extremely precise penalty tuning. Therefore, early stopping should be accompanied by sensitivity diagnostics: how does performance change if you stop a bit earlier or a bit later? Does the model exhibit a plateau region where many stopping points perform similarly? If performance is sharply peaked at one iteration count, the model is likely overfit to validation noise. Chapter 12’s emphasis on plateau regions and fragility maps applies directly.

Finally, early stopping is a conceptual bridge to Chapter 13. Deep learning introduces many additional degrees of freedom, and early stopping becomes an essential regularizer in that setting. But Chapter 12 uses early stopping to make a broader point: regularization is not only about adding penalties; it is about restricting adaptability in any form. Training dynamics are a form of adaptability. Therefore, early stopping belongs here as a model-agnostic regularization tool that reinforces the chapter’s governance-native philosophy.

Taken together, these model selection procedures form a hierarchy of discipline. The single split is the minimal defensible baseline that forces separation between tuning and evaluation. Time-series cross-validation adds stability evidence by repeating deployment simulations across folds. Nested selection protects performance estimates from tuning bias and makes the tuning process itself auditable through time. Early stopping provides an additional, model-agnostic regularization mechanism that must be embedded inside the same time-aware validation framework. The unifying principle is consistent: model selection is not a trick to maximize backtest performance; it is a protocol for producing claims that remain credible when the market changes and when the researcher is tempted to adapt after the fact.

12.7 Selection Bias: When the Research Process Overfits

12.7.1 Multiple comparisons and the inevitability of a lucky winner

Selection bias in trading research is not primarily a technical footnote; it is the default failure mode of the entire enterprise. The central mechanism is simple: if you try enough things, one of them will look good by chance. What makes this mechanism especially dangerous in algorithmic trading is that the space of “things you try” is usually far larger than you admit to yourself, and the data is noisy enough that chance can produce results that look impressively structured. The chapter frames this as an endogenous failure mode because it does not require bad intentions. It emerges naturally from the fact that markets invite experimentation while punishing overconfidence.

The multiple-comparisons problem is often explained using a toy example: if you test 100 independent hypotheses at a 5% significance level, you expect about five false positives. In trading, the situation is worse, not better. First, the hypotheses are rarely independent. Features are correlated, strategies are variations on the same themes, and market regimes create shared structure across time. This correlation means that the “effective” number of comparisons is hard to compute and easy to underestimate. Second, outcomes are heavy-tailed and regime-dependent,

so a lucky run can look like a genuine edge rather than a mild statistical fluctuation. Third, the researcher typically engages in adaptive experimentation: after seeing results, they change something and try again. Adaptivity turns the validation set into a training signal, and therefore increases the probability of finding a lucky configuration that is tuned to historical noise.

A critical practical point is that selection bias can occur even if each individual model is regularized. Regularization controls the flexibility of a single fit; it does not control the flexibility of the research process that chooses among many fits. If you run a lasso with a carefully tuned penalty, you may reduce overfitting within that model. But if you run 500 lasso variants with different feature sets, different horizons, different scaling rules, and different validation boundaries, then the selection among those variants can overfit. The “winner” is not necessarily the model that best captures signal; it is the model that best captured a coincidence in the particular historical sample you searched over.

This is why Chapter 12 insists on interpreting the research process itself as a model: it has degrees of freedom, it adapts, and it can overfit. In other words, the selection mechanism is an optimizer operating on noisy data. The more you let it search, the more it will discover patterns that are not stable. This is not a pessimistic philosophical stance; it is a practical warning about the behavior of search in a low signal-to-noise environment.

The inevitability of a lucky winner has implications for how you interpret out-of-sample results. A single strong test period is not necessarily evidence of robustness if the model was selected after extensive search. In many cases, the selection process can indirectly “optimize” for that test period through leakage or through the use of intermediate validation steps that overlap with it. Even without explicit leakage, the fact that you tried many models means the probability of selecting a model that performs well on the test by chance is higher than naive reasoning suggests. This is why the chapter repeatedly emphasizes the tuning budget and trial ledger: without them, you cannot interpret the epistemic weight of the winner.

The correct response to multiple comparisons is not to stop experimenting. Trading research is inherently exploratory. The correct response is to constrain and account for exploration. That means limiting the size of the search space to plausible designs, preferring stability over peak performance, using protocols like nested selection when search is wide, and reporting the extent of search honestly. In other words, you treat exploration as a controlled experimental process rather than an informal hunt for a good backtest.

12.7.2 Hidden degrees of freedom in preprocessing and feature engineering

When practitioners think about degrees of freedom, they often picture model parameters: coefficients in a regression, weights in a network, or thresholds in a classifier. In trading research, this is a dangerous undercount. The degrees of freedom that drive false discoveries are often upstream of the model fit, embedded in preprocessing and feature engineering choices that seem innocuous until you realize how much they can be adapted to make results look good.

Consider filtering and data cleaning. Decisions such as how you handle missing data, how you treat outliers, whether you adjust for corporate actions in a particular way, and whether you remove certain days (holiday effects, extreme events) can change results materially. Each decision can be justified post hoc: “that day was anomalous,” “that outlier was a data error,”

“that gap was due to illiquidity.” Sometimes these justifications are valid. The problem is that the researcher can choose among many plausible cleaning rules, and after seeing performance they can favor the rule that improves results. That is selection bias via preprocessing, and it is often invisible because it is not recorded as a tunable hyperparameter.

Scaling and normalization are another major hidden degree of freedom. If you normalize features globally, you can leak future distributional information. Even if you normalize correctly within each fold, you still have choices: standardization versus robust scaling; clipping thresholds; log transforms; volatility scaling; z-scores using different windows. These choices affect the relative influence of features and can alter which predictors appear important. In a regularized model, scaling choices can be decisive: a different scaling scheme can change which coefficients survive shrinkage, which in turn changes forecasts. Treating scaling as an “implementation detail” is therefore a common path to untracked adaptivity.

Feature engineering magnifies degrees of freedom even further. For a single conceptual idea like “trend,” you can create dozens of candidate features: moving-average crossovers with different windows, slope estimates, breakout indicators, momentum over different horizons, normalized variants, volatility-adjusted variants, and combinations with volume or volatility filters. The same is true for “mean reversion” or “risk.” Each feature family produces a combinatorial search space when combined. If you allow yourself to add, remove, or modify features in response to validation performance, you are effectively searching a vast hypothesis space, even if the final model is simple.

Target definition is itself a degree of freedom, and it is often overlooked. Forecasting a 1-day return is not the same as forecasting a 5-day return. Defining the target as a raw return is not the same as defining it as a sign label or a quantile label. Using open-to-close versus close-to-close returns changes the information set and the tradability assumptions. Each of these choices can improve apparent performance in certain periods. If you adjust the horizon or the label definition after observing results, you have tuned the problem statement to the data. That is selection bias at the deepest level: the model appears to solve a task that was chosen because it is solvable in the historical sample.

Even the validation protocol can become a hidden degree of freedom. Choices such as the exact split boundary, the length of validation blocks, the embargo length, and the retraining cadence can be adjusted after observing results. This is particularly dangerous because it affects the credibility of performance estimation directly. If you change the split boundary until performance looks good, you are no longer evaluating generalization; you are selecting a favorable historical slice.

The chapter’s governance-native response is to insist that preprocessing and feature engineering choices be tracked in manifests and treated as part of the selection space. This means the parameter registry includes not only model penalties but also feature windows, scaling schemes, clipping thresholds, target horizons, and protocol parameters. It means the trial ledger records which pipeline variant was used for each run. It means that changes to preprocessing are logged as changes to the model specification, not as silent tweaks. The purpose is not to bureaucratize research; it is to prevent self-deception by making degrees of freedom visible.

A practical implication is that you should distinguish between *design choices* and *tuning*

choices. Design choices are the high-level decisions you commit to before looking at results, motivated by economic reasoning or by prior chapters in the book. Tuning choices are the variations you allow yourself to search over under a controlled protocol. The boundary between these is not always sharp, but the act of declaring it is itself a governance mechanism. If you cannot explain why a choice belongs to design rather than tuning, you should probably treat it as tunable and log it as such.

12.7.3 Stability selection mindset

A stability selection mindset is the antidote to winner-takes-all research. Instead of asking “which configuration is best on average?” you ask “which configurations behave consistently under time, perturbations, and reasonable alternative specifications?” This shift matters because in trading, the cost of fragility is high. A fragile model can look excellent in backtests and then collapse in deployment; a stable model may look modest in backtests but remain usable across regimes. Stability selection is therefore not a statistical luxury; it is a practical preference for deployability.

Stability is operationalized through multiple complementary diagnostics. The first is fold-by-fold performance dispersion under time-aware cross-validation or walk-forward evaluation. A stable configuration is one whose performance does not depend on one lucky fold. You look at the distribution of metrics across folds: mean, median, dispersion, and worst-fold outcomes. You may prefer a configuration with slightly lower mean if it has substantially better worst-fold behavior. This is an explicit trade: you give up peak performance to reduce tail risk in out-of-sample behavior.

The second diagnostic is hyperparameter sensitivity. You do not only record the best hyperparameter setting; you examine performance as the hyperparameter varies. Stable regions produce plateaus, where many nearby settings perform similarly. Fragile results produce spikes, where one setting is uniquely good. In stability selection, spikes are treated as warnings. If the system requires precise tuning to perform well, it is likely exploiting noise. Plateaus are treated as evidence that the relationship is more robust to reasonable specification changes.

The third diagnostic is selection identity stability. In sparse models or feature selection procedures, you track how often each feature is selected across folds. If the selected set changes drastically across time, the model may not have identified a stable signal; it may be choosing among correlated proxies based on sample-specific noise. Some switching is expected when predictors are correlated, but extreme instability is a red flag. Stability selection therefore encourages reporting selection frequencies, not just the final chosen feature set. The model’s identity becomes probabilistic: “these features are reliably selected, these are occasional, these are unstable.”

The fourth diagnostic is coefficient dispersion for models that retain all features (such as ridge). You track the variability of coefficients across folds and retraining dates. Stable models produce coefficients that drift gradually; unstable models produce coefficients that jump, flip sign, or swing wildly in magnitude. Coefficient dispersion is a useful proxy for estimation variance and provides an interpretable stability report even when performance metrics are noisy.

The fifth diagnostic is robustness to perturbations. You test whether the model behaves

similarly when you slightly perturb the pipeline in ways that mimic real-world uncertainty: small changes in split boundaries, slight variations in feature windows, modest noise injected into features to mimic measurement error, or alternative scaling schemes. These perturbations are not adversarial tricks; they represent plausible changes that occur when data sources shift, when markets change, or when the system is maintained over time. A stable model should degrade gracefully, not collapse.

Stability selection does not mean you ignore performance. It means you treat performance as a distribution and you evaluate it under a family of reasonable perturbations rather than a single fixed specification. The output of this mindset is therefore a richer artifact: not just “we achieved metric X,” but “we achieved metric X with dispersion Y, sensitivity Z, and stable identity properties under these perturbations.” This is exactly the kind of evidence that supports credible claims in a noisy environment.

Stability selection also interacts with governance and reporting. Because it requires multiple folds and multiple perturbations, it naturally produces a larger set of results. Without discipline, this can itself become a selection opportunity: you can cherry-pick the perturbation that makes the model look best. The chapter therefore insists that stability experiments be planned and logged. You define the perturbation set *ex ante*, you run it, and you report the outcomes as a bundle. Stability selection is not a license to generate many numbers; it is a structured way to reduce the chance that a single lucky number dominates your conclusions.

12.7.4 Minimum disclosure for credible improvement claims

The final defense against selection bias is disclosure. In trading research, claims are often presented as if the reported model was the only model considered. In reality, the reported model is usually the survivor of a long, partially implicit search process. Without disclosure, the reader cannot assess how much selection pressure was applied and therefore cannot assess the credibility of the improvement. Chapter 12 therefore treats disclosure not as a bureaucratic obligation but as an epistemic necessity: without it, results are not falsifiable and cannot be compared across studies.

The minimum disclosure set is designed to answer a small number of concrete questions. How many trials were run? What were the degrees of freedom in the search space? What split protocol was used, including purge and embargo rules? What tuning budget was declared, and was it respected? What selection rule was used to choose the final configuration? How sensitive is performance to hyperparameters and split boundaries? Without answers to these questions, the reported improvement is ambiguous: it could be signal, or it could be a selection artifact.

Trial count and search space disclosure are the first elements. The trial ledger should record each configuration tried, including pipeline choices and hyperparameters, and should make it clear whether the search was grid-based, random, or adaptive. If the search was adaptive, the adaptation rule must be described. This disclosure is essential because adaptivity increases the risk that the validation set was used as a training signal.

Split protocol disclosure is the second element. You must state whether you used a single holdout, walk-forward, rolling CV, or nested selection. You must state how you handled overlap contamination: what purge width was used, what embargo length was applied, and why those

values were chosen relative to the label horizon and feature windows. These are not minor details; they define the causal validity of the evaluation. A model that performs well under an invalid split protocol is not evidence of generalization.

Selection rule disclosure is the third element. Did you choose the configuration with the best mean validation metric? Did you use a stability-adjusted criterion? Did you choose the simplest within a tolerance? Did you apply any human judgment beyond the formal rule? If human judgment played a role, that fact must be acknowledged because it is another selection mechanism. The goal is not to eliminate judgment; the goal is to make its influence explicit.

Sensitivity disclosure is the fourth element. You must show that the result is not a single spike. This can be done through sensitivity curves, fold dispersion tables, coefficient stability summaries, and robustness perturbation outcomes. The point is to provide evidence that the model's performance is not concentrated at a razor-thin specification that is unlikely to persist under live conditions.

Finally, disclosure must include baselines under identical protocols. Improvements are only meaningful relative to naive alternatives evaluated under the same split and preprocessing rules. If baselines are evaluated differently, the comparison is not credible. Chapter 12 therefore treats baseline bundles as part of minimum disclosure: you report baseline performance alongside candidate performance, with identical validation machinery.

The broader philosophy is that disclosure is the price of credibility in a field where search is unavoidable. Trading research cannot be pure confirmatory science; it is exploratory by necessity. The governance-native approach does not pretend otherwise. It says: explore, but log your exploration; tune, but separate tuning from evaluation; select, but report how selection was done; and claim improvement only when the evidence includes stability and auditability. Under that discipline, selection bias does not disappear, but it becomes measurable, discussable, and less likely to masquerade as an edge.

12.8 Implementation Notes (Within Chapter 12 Scope)

12.8.1 Decision-time logging as an engineering primitive

The practical contribution of Chapter 12 is not merely to recommend “time-aware validation” and “governance-native artifacts,” but to show how those principles become enforceable in code. The first implementation primitive is decision-time logging. In a trading setting, the central question is never “can we compute this feature?” but “when does this feature become knowable, and when can it be acted upon?” If you cannot answer those questions mechanically, the rest of Chapter 12 becomes prose. Decision-time logging is the mechanism that turns prose into constraints.

Implementation begins with explicit time ordering. This means your core data structures are indexed by an ordered time array, and that every operation preserves that ordering. In a NumPy-first workflow, this typically means you maintain arrays such as `t[0:T]`, `price[0:T]`, `volume[0:T]`, and a feature matrix `X[0:T,0:d]`. The key engineering stance is that the index is sacred: `i` corresponds to a time `t[i]`, and no computation may use values from indices greater than `i` when producing an output at `i`. This is not an aspirational guideline; it is a rule you can

test with assertions once you define how each feature is computed.

Decision-time logging extends this stance from “no future indices” to “no future information sets.” Many market objects are available only at certain times: the close is known only after the close; the next open is known only after the open; corporate actions are known with delays; some fundamental data is published with lags. Chapter 12 does not require a full market data clock (that is more relevant later), but it does require that you specify, for each feature, the timestamp at which it becomes available. For daily data, a simple convention is often enough: features computed from close-to-close returns are available at the close of day t (and therefore tradable only from $t + 1$ open in a conservative setup), while features computed from open prices are available at the open. The important part is not which convention you choose; it is that you choose one, document it, and implement it consistently.

A practical way to encode this is to maintain a “feature availability” log alongside the feature matrix. For each feature column j , you store metadata: a name, its parameter settings (window length, decay), and an availability rule (e.g., “available at end-of-day t ” or “available with one-day lag”). Then, for each time index i , you can compute a boolean mask indicating whether the feature is available for use at that index. When you produce a forecast $\hat{y}_i$, you log not only the forecast value but also the decision timestamp and the set of features that were valid at that time. This creates an audit trail: if later you discover that a feature was computed with a subtle look-ahead, you can locate exactly which forecasts were contaminated.

Decision-time logging also ties directly to Chapter 12’s validation rules. Purging and embargo are about preventing overlap contamination near fold boundaries. If you have an explicit mapping from timestamps to feature availability and label horizons, you can implement these rules as index manipulations with assertions. For example, if your label is a h -day forward return, you can mark any index i within h steps of a boundary as “unsafe” for training because its label window overlaps the test region. Similarly, if a feature uses a rolling window of length L , you can mark indices as “unsafe” until L observations have accumulated. These “unsafe” masks become part of fold construction rather than being remembered informally.

The chapter’s engineering message is that decision-time logging is not an extra layer you add later; it is the skeleton on which everything else hangs. When you run hyperparameter search, you are comparing models whose differences may change which features are available when, how labels overlap, and whether forecasts are tradable. If you do not log decision times, you risk selecting a model that is “best” only because it implicitly used information that would not have been available at the time the trade is placed. Decision-time logging makes that risk testable.

Finally, decision-time logging supports reproducibility in a deeper sense. A forecasting model is not only a set of parameters; it is a sequence of decisions. In a walk-forward procedure, you refit periodically, produce forecasts, and evaluate them. A decision-time log records the exact sequence: training window endpoints, chosen hyperparameters, forecast timestamps, and the moments when the model would have acted. This log is an artifact that allows you to reproduce not just the model fit, but the *deployment simulation* that defines Chapter 12’s credibility.

12.8.2 Leakage-safe scaling and normalization

Scaling and normalization are among the most common leakage vectors in applied trading research because they are easy to do “wrong” while still producing smooth-looking results. In many ML tutorials, scaling is treated as a harmless preprocessing step performed once on the dataset. In trading, that approach is often invalid because it allows future distributional information to influence the training process. Chapter 12 therefore treats scaling as a first-class component of the model selection protocol, requiring fold-specific fitting and explicit assertions.

The fundamental rule is simple: any transformation whose parameters are estimated from data must be fit using training data only, and then applied forward to validation and test. This includes standardization (mean and standard deviation), robust scaling (median and interquartile range), clipping thresholds computed from quantiles, and any normalization based on historical volatility or other statistics. The reason is not merely abstract “fairness”; the reason is that market distributions shift. If you compute a global mean and standard deviation using the entire dataset, you are using information about later regimes to scale earlier regimes. That can make the model appear more stable than it would be in live deployment, where you would not know how the distribution will evolve.

In practice, leakage-safe scaling must be implemented per fold. For each fold, you define a training index set. You compute scaling parameters on that set only. You store those parameters as part of the fold artifact. You then transform the validation and test feature rows using the stored parameters. Importantly, you do not recompute scaling parameters when transforming validation and test; you apply the training-derived parameters. This ensures that the model is evaluated under the same information constraints it would face when deployed.

Several technical details matter in trading contexts. First, scaling interacts with missingness. If a feature has missing values early in the sample (due to warm-up) or during illiquid periods, the computation of means and variances must be defined consistently. A leakage-safe approach typically computes scaling parameters using only the indices that are valid and non-missing within the training window, and then applies the same rule forward. The key is that missingness itself can carry information about time, so you must avoid procedures that implicitly use future patterns of missingness to make scaling decisions.

Second, scaling interacts with rolling features. Many features are already normalized by construction (e.g., z-scores computed with a rolling window). In that case, additional global scaling may be redundant or may distort interpretability. Chapter 12 does not prescribe a single scaling scheme; it prescribes that whatever scheme you use must be applied in a leakage-safe way. This is important because scaling is part of the hyperparameter space: you may compare standardization versus robust scaling, or choose whether to clip extremes. Those choices must be logged and treated as tunable, because they can be adapted to improve validation performance.

Third, scaling must be consistent across baselines and candidates. If you evaluate a baseline model without scaling and a candidate model with scaling, you have changed the pipeline in a way that may explain performance differences. This is not forbidden, but it must be explicit. The proper approach is to define scaling as part of the pipeline specification, and then compare pipelines as wholes. The run manifest should record which scaling method was used.

The chapter also emphasizes assertions as hard gates. A common failure in research code

is to accidentally fit a scaler on the full dataset because it is convenient. To prevent this, you implement tests that detect when validation or test indices were used in scaling computations. In a NumPy workflow, you can do this by passing explicit index arrays into scaling functions and by logging the index ranges used. You can then assert that the maximum index used for scaling is less than the minimum index in validation and test. For non-contiguous folds, you can assert set disjointness between scaling-fit indices and evaluation indices. The point is to convert “fit on training only” from a guideline into an enforced invariant.

Finally, scaling parameters are themselves artifacts. For each fold, you store the scaling method, the fitted parameters (means, standard deviations, medians, quantiles), and the training window boundaries used to compute them. If later you discover that a performance improvement was driven by a subtle scaling change, you can trace it. This is exactly the governance-native posture: scaling is not an invisible step; it is documented, reproducible, and auditable.

12.8.3 Reproducible hyperparameter search loops

Once decision-time logging and leakage-safe preprocessing are in place, hyperparameter tuning becomes an engineering problem rather than a mystical art. The key requirement is reproducibility: the search must be deterministic given a seed and a registry. Chapter 12 treats this as essential because hyperparameter search is a selection process over a noisy objective. If the search is not reproducible, you cannot audit it, and you cannot reliably interpret the best result.

The implementation pattern is a deterministic outer loop over configurations. You define a parameter registry object that enumerates the hyperparameters you will search, their allowed values or ranges, conditional dependencies, and constraints that enforce protocol validity. For example, the registry may specify $\lambda \in \{10^{-4}, 10^{-3}, \dots, 10^2\}$, window length $L \in \{20, 60, 120\}$, scaling method in a small set, and a constraint that embargo length E must be at least the label horizon. Conditional dependencies might include: only consider a decay rate if an EWMA feature is enabled; only consider a threshold if a filter is enabled. The registry is part of the run manifest.

Given the registry, you generate a sequence of trial configurations. This can be a grid, a random sample under a fixed seed, or an adaptive sequence. Chapter 12 does not require a particular search method; it requires that whatever method is used be logged. For random search, reproducibility means the seed must be stored and the sampling order must be deterministic. For adaptive search, reproducibility means the adaptation rule must be stored and the intermediate results that guided adaptation must be recorded. The core governance idea is that the search path is part of the artifact.

Each trial configuration is evaluated under the same time-aware split protocol. This means: construct fold index sets with purge/embargo rules; fit preprocessing parameters on training only; transform validation/test; fit model on training; evaluate on validation; compute metrics; compute stability diagnostics. The trial then writes a record to a ledger. The record includes: the configuration values, fold boundary definitions, computed metrics per fold, aggregated score, and a status indicator. Status is important because trading pipelines fail for reasons that can bias results: convergence failures, numerical instability, or causality-gate failures. If you silently drop failed trials, you may bias the search toward configurations that happen to be numerically

convenient rather than economically meaningful. Therefore, failures must be logged explicitly.

The ledger is not just bookkeeping; it is the primary defense against accidental p-hacking. It makes it possible to answer: how many trials were run, which ones failed, which ones were close contenders, and how sensitive was the winner to small changes. It also supports the chapter’s insistence that tuning budgets are credibility budgets: you cannot claim a result is robust if you cannot report how widely you searched and how much you relied on a single lucky configuration.

Reproducible search loops also require disciplined naming and hashing of configurations. A practical pattern is to create a canonical representation of each configuration (for example, a sorted list of key-value pairs), serialize it deterministically, and compute a hash. That hash becomes the trial identifier. All artifacts produced in the trial—scalers, fitted coefficients, fold metrics, and plots—can be stored under that identifier. This allows you to reproduce a particular trial exactly and to compare trials reliably.

Another key implementation detail is that the search loop must enforce time awareness systematically. It is not enough to “remember” not to leak. The trial evaluation function should accept explicit index sets and should assert that training indices precede validation indices. It should assert that scalers were fit only on training indices. It should assert that label overlap rules were applied at fold boundaries. These assertions act as hard stops. If a configuration violates the protocol, it should be recorded as a failure, not quietly accepted. This is how the chapter’s governance principles become code-level invariants.

Finally, reproducibility includes the environment: seeds, library versions, and any numerical settings that can affect results. Chapter 12’s scope does not require a full deployment stack, but it does require that runs be deterministic given seed and config. The run manifest therefore includes the seed and a summary of the environment. This is crucial when you later revisit results: you need to know whether an apparent change in performance is due to a real code change or a nondeterministic rerun.

12.8.4 Essential diagnostics and plots for the companion notebook

The companion notebook for Chapter 12 is not primarily about producing a single best model. Its objective is to produce evidence about robustness. Therefore, the diagnostics and plots are chosen to reveal stability, sensitivity, and fragility, and to support an audit trail that is consistent with the chapter’s governance-native requirements.

The first essential diagnostic is fold-by-fold performance reporting. For any time-aware validation protocol, you should produce a table or structured output that lists metrics per fold, including the primary metric aligned with the decision and secondary metrics that aid interpretation. The point is to expose dispersion. Alongside fold metrics, you should report aggregate summaries such as mean, median, dispersion, and worst-fold outcome. In a trading context, worst-fold performance is particularly informative because it approximates the risk that the model fails in an unfavorable regime. The notebook should treat this as a first-class output, not as a hidden detail.

The second essential diagnostic is hyperparameter sensitivity curves. For key hyperparameters such as regularization strength λ or window length L , the notebook should plot performance as the hyperparameter varies, ideally with fold dispersion information. The goal is to distinguish

plateau behavior from spike behavior. Plateaus indicate robustness: many nearby settings perform similarly. Spikes indicate fragility: performance depends on precise tuning. The notebook should also record the selected hyperparameter and justify it in terms of stability rather than purely in terms of peak performance.

The third essential diagnostic is coefficient or selection stability. For shrinkage models where all coefficients are present, you should track coefficient values across folds or retraining dates and summarize their dispersion. For sparse models, you should track selection frequency: how often each feature is selected across folds. These stability summaries are a direct response to the chapter’s claim that correlated predictors and noisy targets create unstable fits. If the model’s coefficients or selected features change dramatically across folds, the notebook should flag that as evidence of fragility even if average performance is acceptable.

The fourth essential diagnostic is baseline comparisons under identical split protocols. Chapter 12 is explicit that improvements are meaningful only relative to naive alternatives evaluated under the same validation machinery. Therefore, the notebook should implement Chapter 11 baselines and evaluate them using the same folds, the same purge/embargo rules, and the same metric definitions as the candidate models. The output should be a comparison table and, where helpful, a plot of fold-by-fold differences. This prevents the common research failure mode where a candidate model appears better only because it was evaluated under a more favorable protocol.

The fifth essential diagnostic is protocol auditing output. The notebook should explicitly print the fold boundaries, the purge width, the embargo length, and the sizes of training/validation/test sets after purging and embargo. It should also record and display the scaling parameters per fold (at least summary statistics) and the fact that they were fit only on training data. These are not glamorous plots, but they are the governance-native evidence that the evaluation is causal. Chapter 12’s philosophy is that such evidence is part of the scientific product, not a private implementation detail.

The sixth essential diagnostic is a trial ledger summary. Even if the notebook does not display every trial in full, it should summarize the search: how many configurations were tried, how many failed and why, what the top configurations were, and how close the contenders were. This summary is the bridge between tuning and credibility. If the best configuration is only marginally better than many others, that suggests robustness; if it is uniquely better and the search was large, that suggests potential overfitting to validation noise. The notebook should make this visible.

Finally, the notebook should produce a compact “artifact bundle” at the end of the run: the run manifest, the parameter registry, the split specification, the trial ledger, and the key plots. The book’s arc is governance-native: results are not just numbers, they are reproducible objects. Chapter 12 is the chapter where that philosophy becomes inseparable from model selection itself.

In summary, the implementation notes of Chapter 12 are not an appendix of conveniences; they are the engineering substrate that makes the chapter’s claims enforceable. Decision-time logging makes information sets explicit and tradability auditable. Leakage-safe scaling prevents distributional hindsight from contaminating results. Reproducible hyperparameter search loops turn tuning into an auditable experiment rather than an informal hunt. The notebook diagnostics emphasize stability, sensitivity, and baseline fairness, producing artifacts that answer the chapter’s

central question: is the result robust enough to justify escalating model capacity in Chapter 13, or is it still a fragile product of selection under noise?

12.9 Governance-Native Research Checklist for Regularization and Model Selection

12.9.1 Determinism: seeds, configs, and run manifests

A governance-native workflow begins with a simple admission: in trading research, many “discoveries” are not discoveries at all, but artifacts of rerunning code until the noise looks cooperative. This can happen even to careful researchers because modern pipelines contain more nondeterminism than we usually acknowledge. Random initializations, randomized sampling in hyperparameter search, floating-point non-associativity, and even different library versions can change results. When the researcher is iterating quickly, these changes can be misinterpreted as meaningful improvements or regressions. Determinism is therefore not a pedantic obsession; it is the barrier between disciplined experimentation and accidental p-hacking.

The first checklist item is that every run declares a seed policy. In a Chapter 12 notebook, the most obvious stochastic components are random hyperparameter sampling (if used), randomized train/validation split choices (if any), and any randomized optimization subroutine. Even if the model itself is deterministic (e.g., closed-form ridge regression), the research process often is not. The engineering rule is that a single master seed is declared at the start of the run and is then used to derive all other seeds in a reproducible way. The derivation rule must be explicit: for example, seed the hyperparameter sampler with `seed_hp`, seed any synthetic data generator with `seed_data`, and seed any perturbation experiments with `seed_perturb`. The details are less important than the principle that rerunning the notebook yields the same trial sequence and the same outputs.

The second checklist item is that every run stores a configuration object that fully specifies the pipeline. This configuration is not only the model hyperparameters. It includes feature engineering choices (which features, with what windows and decays), preprocessing choices (scaling method, clipping, handling of missingness), target definition (horizon, labeling rules), and validation protocol (split boundaries, fold structure, purge width, embargo length). The configuration also includes the selection rule: how you choose among trials, what metric is primary, and what stability criteria are used. The point is to prevent the researcher from reconstructing the pipeline from memory. If it is not in the config, it is not part of the run.

The third checklist item is the run manifest. The manifest is the minimal document that allows a future reader (including the future version of you) to answer: what exactly was run, when, and under what code identity? In a Chapter 12 context, the run manifest contains a run ID, the timestamp (start and end), the master seed, the hash or identifier of the configuration object, and a code identifier. The code identifier can be a git commit hash when available, or a hash of the notebook/script content if the environment is not under version control. The manifest may also record environment information such as Python version and NumPy version, because minor changes can matter when results are near the noise floor.

A subtle but important governance point is that determinism prevents a specific type of silent

adaptivity: the researcher interprets changes due to randomness as changes due to modeling choices. Suppose you change a window length and rerun, and the performance changes. If the hyperparameter search was stochastic and not seeded, the trial sequence also changed, and therefore your observed performance difference may be confounded by a different search path. Determinism isolates cause: the only differences between runs should be those declared in the configuration. That isolation is what makes debugging and scientific interpretation possible.

Determinism also has a psychological function. When you know the run is reproducible, you become less tempted to rerun until you get a pleasing outcome because you realize that rerunning without changing the config will not produce a different number. This is why the checklist treats determinism as an anti-p-hacking device: it removes the easy escape hatch of “maybe it was just a bad run” when the run was in fact a lucky or unlucky draw. In a governance-native workflow, luck must be measured via controlled perturbations and fold dispersion, not via repeated unlogged reruns.

Finally, the determinism checklist item includes an explicit discipline for handling refactors. In Chapter 12, you are often making small changes to code structure (modularizing feature functions, changing the way folds are constructed, reorganizing logging). These refactors can change outputs if they unintentionally alter alignment or preprocessing. The run manifest allows you to compare manifests across runs and identify whether a result change occurred because the configuration changed or because the code identity changed. This is essential because Chapter 12 is about selection credibility; you cannot claim a model is robust if you cannot even tell whether your pipeline is stable under routine engineering changes.

12.9.2 Data and feature lineage: fingerprints, hashes, and timing proofs

Regularization and model selection cannot be governed without governing the input data and the transformations that produce features. In trading research, it is common to focus on model choice while treating data as a given. This is precisely backward. Most selection bias enters through data and feature choices, and most irreproducibility enters through untracked differences in input datasets and preprocessing. Chapter 12 therefore requires data and feature lineage artifacts that make the pipeline traceable.

The first lineage artifact is the data fingerprint. A data fingerprint is not the dataset itself; it is a compact summary that uniquely characterizes the dataset enough to detect when it changes. In a Chapter 12 notebook, a data fingerprint includes: the instrument or instrument set, the frequency (daily, hourly, etc.), the time span (start and end timestamps), the number of observations, and a missingness summary. Missingness summary is critical because missing values can change feature computation and effective sample size. The fingerprint also includes simple distributional summaries of the raw series: mean and variance of returns, quantiles, volatility measures, and any other low-cost statistics that help detect whether data was revised. The goal is not to prove the dataset is correct; the goal is to make it hard for the dataset to change silently.

Where feasible, the fingerprint is supplemented with hashes for lineage. A hash can be computed on the raw arrays or on a canonical serialization of the dataset. The hash serves as a stable identifier: if the hash changes, the input changed. In a synthetic-data-first workflow, this

is especially powerful because the dataset should be fully determined by the seed and generator parameters; therefore, changes in the hash indicate a generator change or an unintended nondeterministic element. In real data settings, hashes reveal when vendors revise historical data or when corporate action adjustments change.

Feature lineage is the second lineage artifact. Feature lineage records each transform applied and its parameters. In Chapter 12, this includes rolling windows, decays, thresholds, clipping rules, and any derived quantities. Importantly, feature lineage also records the availability rule: when the feature is considered known and tradable. This ties lineage directly to decision-time logging. The lineage record can be as simple as a structured list: feature name, input fields, parameter settings, and output indexing rule. The point is to ensure that when you tune hyperparameters, you know which hyperparameters you tuned, and when you revisit results, you can reconstruct exactly how each feature was computed.

Timing proofs are the third, and most distinctive, component of lineage in this book. A timing proof is not a formal proof in a mathematical sense; it is an executable test suite that verifies causal correctness. For each feature, you want tests that validate that the output at time t depends only on inputs at times $\leq t$ under the declared availability rule. For rolling features, you want to verify that the window uses the correct endpoints and that no future values are included. For targets, you want to verify that the horizon is correctly aligned and that labels are computed from future returns only in the target construction, not leaked back into features. For scaling, you want to verify that scaling parameters are fit only on training indices. These tests are timing proofs because they are the mechanism by which you demonstrate that the pipeline respects time.

Timing proofs matter because of a specific failure mode: features are often correct in spirit but incorrect in indexing. Off-by-one errors, using inclusive endpoints incorrectly, or shifting the wrong array can produce a subtle look-ahead of one period, which in trading can turn noise into a fake edge. Without timing proofs, such errors can survive for months because the pipeline “looks reasonable” and the results look strong. The governance-native stance is that timing correctness must be asserted, not assumed.

The data and feature lineage checklist therefore has a strict logic: if you cannot fingerprint the data, you cannot know whether you are comparing like to like. If you cannot record feature lineage, you cannot know what the model actually used. If you cannot enforce timing proofs, you cannot claim that the validation protocol was causal. Regularization and model selection depend on these conditions. A perfectly executed nested selection protocol does not rescue a pipeline whose features leak.

A final detail is that lineage artifacts must be portable. The objective is that an independent reviewer could use the fingerprint and lineage records to reconstruct the dataset and features, at least in principle. In a synthetic workflow, this is straightforward: the generator spec and seed reproduce the data. In a real-data workflow, the fingerprint and hash allow the reviewer to confirm they have the same dataset, and the lineage record allows them to compute the same features. This is why the checklist treats lineage as a publication-quality requirement rather than an internal convenience.

12.9.3 Causality gates: assertions as hard stops

Governance becomes real when it can stop the run. A research checklist that only describes good behavior but does not enforce it is advice, not governance. Chapter 12 therefore introduces causality gates: executable assertions that halt the pipeline when core invariants are violated. These gates are the operational translation of the chapter’s central claim that time-aware validation must be a property of implementation, not a retrospective claim.

The first causality gate is time monotonicity. All core arrays must be indexed by a strictly increasing time vector, and no operation may reorder indices. This seems trivial, but many subtle bugs begin with accidental reindexing or sorting mismatches between features and targets. A time monotonicity gate asserts that $\mathbf{t}[i] < \mathbf{t}[i+1]$ for all i , and that any join or alignment procedure preserves that order. In a no-pandas environment, this is often managed by explicit index arrays and sanity checks after every merge-like operation.

The second gate is feature causality. For each feature function, you can implement a test that checks whether the feature at index i depends on any input indices greater than i . A practical technique is to construct a synthetic input array with a single nonzero spike at a future index and verify that the feature outputs at earlier indices do not change. This is a simple but powerful unit test: if a future spike changes earlier features, the feature computation is leaking. Such tests are easy to implement in NumPy and fit the book’s first-principles philosophy. They convert a conceptual rule into an empirical guarantee.

The third gate is label overlap control at fold boundaries. If you are using multi-step horizons, you must purge training observations whose labels overlap the validation or test region. A causality gate asserts that no training label window intersects any evaluation window. This can be implemented by checking index ranges: for each training index i , the label uses returns from $(i + 1)$ to $(i + h)$; you assert that these indices are strictly within the training region. Similarly, you assert that validation labels do not use returns from within the test region if the protocol intends strict separation. These checks make purging a property of fold construction rather than an optional step.

The fourth gate is leakage-safe scaling. Scaling is a common leakage vector precisely because it is easy to compute on the full dataset accidentally. A causality gate here is straightforward: you store the indices used to fit the scaler and assert that they are disjoint from validation and test indices. In chronological splits, you can assert that the maximum index used for scaling is less than the minimum evaluation index. In non-contiguous outer folds, you assert set disjointness. If the assertion fails, the run stops, and the trial is marked as invalid in the ledger.

The fifth gate is protocol consistency. If your run manifest declares a split protocol with specific purge and embargo lengths, the actual fold-construction logic must implement those lengths. A causality gate asserts that the post-purge training set excludes the correct number of boundary-adjacent observations and that the embargo region is excluded from evaluation. This prevents a common research error where the protocol is written in prose but not implemented correctly, leading to an unintended contamination.

Causality gates also apply to model selection logic itself. A common silent failure mode is evaluating on the test set multiple times while tuning. You can implement a gate that prevents the evaluation function from accessing test indices until a configuration has been frozen. This

can be done by structuring code so that test evaluation is a separate stage that requires an explicit flag indicating that tuning is complete. While this is more of a software design choice than a numerical assertion, it embodies the same principle: the process must make it hard to cheat accidentally.

The chapter’s insistence on hard stops has a deeper rationale. In a noisy domain, researchers are tempted to rationalize violations: “it is only a small leak,” “the boundary effect is minor,” “the scaler probably did not matter.” These rationalizations are precisely how fragile results become publishable narratives. Causality gates remove the possibility of rationalization by turning violations into errors. You cannot proceed until you fix the pipeline, and the ledger records the failure. This is governance in the strongest sense: it enforces standards by construction.

12.9.4 Tuning governance: budgets, ledgers, and adaptive-search disclosure

Hyperparameter tuning is where selection bias concentrates because it is where search interacts with noise. The governance checklist therefore treats tuning as a regulated activity. The core idea is that tuning must be declared, bounded, logged, and reconstructible. Without these properties, a “best model” claim is not interpretable because the reader cannot tell whether the result emerged from a small, principled search or from a large, adaptive hunt.

The first tuning governance item is the declared budget. Before tuning begins, the run manifest must state the search method and the budget: grid size, number of random trials, or number of adaptive iterations. This budget is not merely an engineering parameter; it is a credibility declaration. A larger budget implies more opportunities to find a lucky configuration. Chapter 12 does not forbid large budgets, but it requires that they be disclosed and that performance claims be interpreted in light of them.

The second item is the hyperparameter registry and constraint set. The registry defines the space that tuning is allowed to explore, including conditional dependencies and protocol constraints such as embargo length relative to label horizon. Declaring the registry *ex ante* prevents silent expansion of the search space when results disappoint. It also makes it possible to answer a key question: did you search over protocol parameters as well as model parameters? If yes, then your evaluation protocol is part of the selection space and must be treated as such in reporting.

The third item is the trial ledger. Every attempted configuration must be recorded, including those that performed poorly and those that failed. The ledger must include the configuration values, the fold definitions used, the metrics obtained per fold, and an aggregated selection score. It must also include a status field: success, failure due to causality gate, failure due to numerical instability, or failure due to missing data conditions. This is essential because failures are informative. If many configurations fail, the effective search space is smaller than declared, and the remaining configurations may be biased toward those that are numerically convenient rather than economically meaningful. Logging failures makes this visible.

The ledger also records stopping criteria. In a fixed-budget search, the stopping rule is simple: stop when the budget is exhausted. In adaptive searches, stopping criteria may include convergence of performance, lack of improvement, or a maximum number of iterations. These criteria must be logged because they determine how long the search continued and whether the

search was extended in response to observed results. Extending search after seeing performance is a form of adaptivity that increases selection bias. The ledger is the artifact that reveals whether that happened.

Adaptive-search disclosure is the fourth item. If the search is adaptive, the adaptation rule must be disclosed. This includes how new configurations are proposed based on prior outcomes, what objective is being optimized, and whether any randomness is involved. The purpose is not to discourage adaptivity; it is to make the procedure reconstructible. In a governance-native framework, a result should not depend on undocumented human intuition about where to search next. If human decisions guided the search, that fact must be recorded as part of the process, because human intuition is itself an adaptive search mechanism.

A practical tuning governance technique is to separate exploration from confirmation. You may allow yourself a broader exploratory search on an early portion of data, then freeze the search space and protocol, and then run a confirmatory evaluation on a later held-out period or via nested selection. This reduces the chance that the final performance estimate is inflated by exploration. Chapter 12 treats such separation as an advanced but highly desirable practice when the search space is large. Even when nested selection is not used, the tuning governance checklist encourages a “locked test” discipline: you do not touch the test set until selection is complete.

Finally, tuning governance requires that stability evidence accompany the chosen configuration. The ledger alone tells you how many trials were run and what won. It does not tell you whether the winner is robust. Therefore, the tuning artifact bundle must include sensitivity curves and fold dispersion for the chosen configuration and for near competitors. If the winner is a sharp spike, that should be visible. If the top ten configurations are similar, that should also be visible. This connects tuning governance to the chapter’s central notion that selection is about finding stable regions, not peak points.

12.9.5 Minimum artifact bundle for publication-quality selection claims

A “publication-quality” claim in the context of this book does not mean the result is necessarily profitable or that it will survive in live trading. It means the claim is interpretable, reproducible, and auditable under the time-aware standards of Chapter 12. The minimum artifact bundle is the set of objects that allows an independent reviewer to reconstruct the selection process and verify that reported metrics come from a genuinely held-out evaluation rather than from silent adaptivity.

The first artifact is the run manifest. The manifest includes the run ID, timestamps, code identifier, environment summary, and master seed. It is the entry point for reproducibility: without it, you cannot rerun the same experiment or even know which version of the pipeline produced the results.

The second artifact is the hyperparameter and pipeline registry. This includes not only model hyperparameters but also feature engineering settings, preprocessing settings, target definitions, and validation protocol parameters (fold structure, purge width, embargo length). The registry must distinguish what was fixed from what was tuned. It must also record any constraints and conditional dependencies. This artifact defines the degrees of freedom of the research process.

The third artifact is the split specification. For each fold (or for the single split), you must record the exact index ranges used for training, validation, and test, and the post-processing applied (purging and embargo). It is not enough to say “we used walk-forward CV.” The specific fold boundaries matter because regime composition and overlap contamination depend on them. The split specification should therefore be explicit and machine-readable.

The fourth artifact is the full trial ledger. The ledger contains every configuration tried, with metrics per fold, aggregated scores, and status. It also contains the declared budget and the realized number of trials, along with stopping criteria. This ledger is the explicit defense against selection bias because it reveals how much searching occurred. Without it, the reader cannot know whether the reported configuration is one among five attempts or one among five thousand.

The fifth artifact is stability diagnostics. This includes fold-by-fold performance distributions for the chosen configuration, hyperparameter sensitivity curves, and coefficient or selection stability summaries. These diagnostics answer the question: is the chosen configuration robust to time and to reasonable perturbations, or is it a narrow spike? The chapter treats this evidence as indispensable because trading results are often driven by regime-specific luck.

The sixth artifact is baseline comparisons under identical protocols. A credible improvement claim requires that baselines (from Chapter 11) be evaluated using the same split protocol, purge/embargo rules, preprocessing conventions, and metrics. The baseline comparisons must be included in the artifact bundle so that the improvement can be interpreted as a difference between pipelines rather than a difference in evaluation machinery.

The seventh artifact is the causality gate report. This can be as simple as a log that lists which assertions were enforced and confirms that they passed for the reported run. It may also include summary outputs from timing proofs for key features. The purpose is to make the time-aware validity claim concrete: the reader can see that the pipeline was subjected to causal checks and that those checks were not merely rhetorical.

A final, practical component of the artifact bundle is the linkage among artifacts. The run manifest should reference the registry hash and the ledger file. The ledger should reference configuration hashes. The split specification should be referenced by both. This linkage is what allows an independent reviewer to trace from a headline metric back to the exact pipeline and data conditions that produced it. In a governance-native workflow, such traceability is not an optional enhancement; it is the core of what makes a selection claim credible.

Taken together, the checklist is designed to make the research process legible. Determinism ensures reruns do not become hidden search. Data and feature lineage ensure you know what you trained on and what you computed. Causality gates ensure that time-aware claims are enforced, not asserted. Tuning governance ensures that search is declared and visible. The minimum artifact bundle ensures that the final claim can be reconstructed and audited. Chapter 12 does not promise that these practices will produce alpha. It promises something more fundamental: that when you claim improvement, the claim will mean what it appears to mean, and that your future self, your collaborators, and your reviewers will be able to trust the process that produced it.

12.10 Conclusion

12.10.1 What regularization can reliably contribute to trading models

Regularization is one of the few tools in quantitative modeling that reliably improves the *credibility* of a trading forecast even when it does not guarantee a large increase in headline performance. Markets are noisy, predictors are often redundant proxies for similar economic ideas, and the effective sample size is typically smaller than it looks because time dependence and overlap reduce independent information. In that environment, the naive estimator is rarely the right baseline. It may fit in-sample well, but it is usually unstable: small changes in the training window produce large changes in coefficients, forecasts, and ultimately trades. Regularization directly targets this instability by restricting the space of admissible solutions and forcing the model to prefer smoother, less reactive parameterizations.

The most reliable contribution of regularization is therefore a reduction in estimator variance and parameter sensitivity. When features are correlated, unregularized fits can arbitrarily allocate weight among redundant predictors, leading to coefficients that fluctuate in sign and magnitude across retraining dates. Regularization shrinks those coefficients toward more conservative solutions, so that the forecast behaves more like a stable summary of the feature set rather than a brittle selection among near-duplicates. In practical trading systems, this matters because a strategy that changes identity from one retraining date to the next is difficult to interpret, difficult to risk-manage, and often expensive to execute. A slightly less aggressive forecast that is consistent through time can be more valuable than a more reactive forecast that occasionally looks brilliant but frequently changes character.

Regularization also functions as an implicit defense against the research process itself. It does not eliminate selection bias—Chapter 12 is explicit that selection can overfit even when individual models are regularized—but it reduces one source of fragility: the ability of a model to fit idiosyncratic noise patterns within a particular window. In time-aware validation, this often appears as tighter fold dispersion: the model’s performance does not rely as heavily on one lucky fold because the fitted parameters are less sensitive to small differences in the training sample. This is one reason why the chapter treats stability as a first-class diagnostic. Regularization is not judged only by whether the mean metric improves, but by whether the model becomes more stable across folds, less sensitive to hyperparameters, and less dependent on a narrow region of the search space.

Another reliable contribution is that regularization forces the researcher to confront complexity as a governed resource. In many trading pipelines, complexity enters through feature engineering long before it enters through the model class. Regularization provides a mechanism to contain that complexity within a model that remains interpretable and auditable. It turns “we used many features” into “we used many features but constrained their combined influence.” That is not a guarantee of signal, but it is a disciplined response to the fact that correlated predictors can otherwise produce unstable estimates. In the book’s arc, this discipline is a prerequisite for escalating model capacity. If you cannot govern complexity in linear or modestly nonlinear settings, adding more capacity later will amplify fragility rather than reveal deeper structure.

Finally, regularization contributes operationally to the creation of deployable forecasts. A

deployable forecast is not only one that has a good backtest; it is one that can be generated under explicit decision-time constraints, with stable behavior under retraining, and with risk properties that can be monitored. Regularization helps by producing forecasts that are less extreme, less likely to swing abruptly when data shifts, and therefore easier to integrate into a broader system that includes risk controls and execution logic later in the book. In that sense, regularization reliably contributes to *system quality* even when its impact on point estimates of predictive accuracy is modest.

12.10.2 What hyperparameter tuning cannot guarantee in markets

Hyperparameter tuning is often described in machine learning as a technical step that improves performance by selecting the best configuration. In trading, that framing is dangerously incomplete. Hyperparameter tuning cannot guarantee a durable edge because the historical evidence used to tune is limited, non-stationary, and contaminated by the possibility of adaptive experimentation. Even in the best-case scenario, tuning is choosing among constrained candidates based on finite samples of market history. It can improve internal consistency, but it cannot force markets to remain in a regime that rewards the chosen configuration.

The central limitation is that markets change in ways that matter for the tuned choices. A window length that is optimal in a stable volatility environment may be suboptimal during volatility clustering. A regularization strength that balances bias and variance in one period may be too strong when signals weaken or too weak when correlations shift. A threshold that filters noise in one regime may filter true opportunities in another. Because these shifts occur without warning and often without clean statistical signatures, tuning based on one historical regime cannot guarantee performance in the next. The best tuning can do is select a configuration that appears robust across the regimes represented in the validation design, and even that depends on whether the validation folds actually resemble future conditions.

The second limitation is selection bias. The more you tune, the more you search, and the more likely you are to find a configuration that looks good by chance. This is not a hypothetical concern; it is an inevitable consequence of searching in a low signal-to-noise domain. Trading targets are weakly predictable at best, and heavy tails can produce seemingly impressive validation outcomes from rare events. Aggressive tuning can therefore exploit chance alignments with noise, especially when many hyperparameters are varied at once and when preprocessing and feature engineering choices are allowed to drift in response to results.

The third limitation is that tuning is often entangled with protocol choices. In a time-aware workflow, split design, embargo length, purge width, and decision-time definitions are part of the evaluation machinery that defines what “out-of-sample” means. If these protocol parameters are adjusted after observing results, tuning becomes inseparable from evaluation bias. Even when protocol parameters are fixed, tuning can inadvertently leak information if preprocessing steps (especially scaling and normalization) are not fit strictly on training data within each fold. The chapter emphasizes that these leakage channels can produce performance that evaporates in truly forward tests. Therefore, tuning cannot guarantee validity unless it is embedded in a strictly enforced protocol.

Because of these limitations, the chapter’s practical message is that tuning must be treated as

a controlled experiment rather than a performance maximization routine. That means declaring a tuning budget, recording a trial ledger, disclosing adaptivity, and using nested or time-aware selection designs when search is broad. It also means interpreting tuned results with humility: a tuned configuration is not proof of a persistent relationship; it is a hypothesis that survived a disciplined selection process under time. In markets, that is the most you can claim without overreaching.

12.10.3 A minimum standard for claiming model selection improved performance

In a domain where selection bias is the default and where protocol mistakes can quietly manufacture alpha, a minimum standard for improvement claims is not an academic nicety; it is the difference between evidence and storytelling. Chapter 12’s minimum standard can be summarized as a set of non-negotiable conditions that make an improvement interpretable.

First, baselines and candidates must be evaluated under identical time-aware splits. If a baseline is evaluated under one split and a candidate under another, the comparison is not meaningful. The split protocol must be stated explicitly: whether it is a single chronological holdout, walk-forward evaluation, blocked or rolling cross-validation, or nested selection. If labels overlap or rolling features create boundary dependence, the purge and embargo rules must be identical across all models. This ensures that differences in performance reflect differences in the model and pipeline, not differences in what information was implicitly allowed.

Second, there must be a clean separation between tuning and final evaluation. This means that hyperparameters are chosen using validation data only, and that the final test is held out until selection is complete. If the test set is used repeatedly during tuning, the reported performance is contaminated by adaptivity. The minimum standard therefore includes a discipline of “evaluate once” on the held-out test, or a nested selection protocol that estimates performance in an outer loop untouched by inner tuning.

Third, improvement must be supported by stability, not by a single lucky period. In trading, it is easy to find a model that performs well in one regime. The relevant question is whether performance persists across time folds and whether worst-fold outcomes remain within a tolerable range. Therefore, the minimum standard requires reporting fold dispersion and worst-fold outcomes alongside mean performance. A model that improves the average metric but collapses in a subset of folds should be treated as fragile rather than superior, unless the strategy explicitly includes a regime filter that justifies that behavior (which is postponed to later chapters).

Fourth, the improvement must not be concentrated at a razor-thin optimum. Sensitivity diagnostics are therefore required: performance as a function of key hyperparameters, with attention to whether there is a plateau region or a sharp spike. Sharp spikes indicate that tuning may have selected a lucky configuration. Plateaus indicate robustness. Similarly, coefficient or selection stability should be examined. If the chosen configuration relies on unstable feature selection identity or wildly varying coefficients across folds, the improvement is suspect even if the headline metric improves.

Fifth, the selection process must be auditable. This is where the trial ledger and artifact

bundle become part of the minimum standard rather than optional enhancements. The ledger records how many configurations were tried, which ones were tried, what their outcomes were, and why the process stopped. Without this, an improvement claim is not falsifiable: the reader cannot know whether the reported model is one among a few principled attempts or the result of a long adaptive search. The artifact bundle includes the run manifest, parameter registry, split specification, causality tests, stability report, and baseline bundle. The purpose is to allow an independent reviewer to rerun the selection process and verify that the reported metrics are produced under the stated protocol.

Taken together, these conditions do not guarantee a durable edge. They guarantee that the claim “model selection improved performance” has a clear meaning: it means that under a specified, time-aware, leakage-controlled validation protocol, a candidate pipeline outperformed baselines in a way that is stable, not overly sensitive, and reproducible. In the world of markets, that is the appropriate level of confidence for a research result.

12.10.4 Transition to Chapter 13: Neural Nets and Deep Learning in Trading

The transition to Chapter 13 is not a jump to novelty for its own sake. It is a principled escalation in capacity that is justified only after the Chapter 12 machinery is in place. Deep learning introduces model families with far more flexibility than the supervised tools of Chapter 11 and far more hyperparameters than the regularized linear baselines that Chapter 12 disciplines. This flexibility can be valuable, but it also amplifies every failure mode discussed in this chapter: tuning risk increases because there are more knobs; selection bias increases because there are more configurations to search; and leakage vulnerabilities increase because deep models can exploit subtle artifacts in preprocessing and alignment that simpler models might not.

Therefore, Chapter 13 will treat deep models as tools that must earn credibility through the same time-aware validation and governance-native artifacts established here. The discipline does not become less important as models become more powerful; it becomes more important. In practice, this means that the notebook and research workflow will carry forward the same run manifests, parameter registries, split specifications with purge/embargo rules, trial ledgers, and stability diagnostics. It also means that the researcher must be even more conservative about reporting: deep models can produce impressive in-sample fits easily, and therefore out-of-sample evidence must be both strict and stable.

The deeper message is that Chapter 12 redefines what it means to be “ready” for deep learning in trading. Readiness is not familiarity with architectures; it is the ability to evaluate models without self-deception. Once you can enforce decision-time constraints, prevent leakage in preprocessing, bound and log hyperparameter search, and interpret performance through stability rather than peak outcomes, you can responsibly explore higher-capacity models. Without that foundation, deep learning is likely to produce sophisticated overfitting that looks like insight. The arc is therefore deliberate: governance-native selection is not an appendix to modeling; it is the gateway to using complexity responsibly.

Artifact	Minimum content
Run manifest	seed(s), code identifier, timestamp, dataset fingerprint, train/valid/test spans
Parameter registry	search space, ranges/priors, constraints, selection rule, compute budget
Split specification	walk-forward or blocked/rolling design, purge/embargo rules, decision-time definition
Causality tests	monotone time, causal rolling windows, no future normalization, no overlap leakage at
Trial ledger	trial count, all configs tried, adaptation rule (if any), stopping criteria, failure reasons
Stability report	fold dispersion, worst-fold outcomes, sensitivity curves, coefficient/selection stability
Baseline bundle	Chapter 11 baselines, identical splits, identical preprocessing rules, comparable metrics

Table 12.1: Minimum governance artifacts for Chapter 12: regularization, hyperparameters, and model selection.

Chapter 13

Neural Nets & Deep Learning in Trading

Abstract

Neural networks are attractive in trading because they can represent nonlinear interactions among predictors, learn useful intermediate representations, and, in some cases, compress large feature sets into signals that generalize better than hand-crafted rules. Yet the trading setting is unusually hostile to deep learning: labels are noisy, distributions drift, samples are effectively small once we respect time ordering, and many apparent gains vanish once we enforce leakage-free validation. This chapter introduces neural nets for return forecasting as disciplined function classes rather than as “black boxes.” We build intuition from first principles (composition, activations, loss functions, capacity), then connect training dynamics to practical stability issues (optimization, learning rates, initialization, normalization, early stopping). We discuss common architectures used on market data—MLPs, 1D CNNs, recurrent models, and attention-based models—emphasizing the inductive biases each imposes and the specific leakage traps that arise when data are windowed into tensors. A central theme is time-aware model selection: walk-forward evaluation, embargo logic, robust metrics (including IC and rank IC), and sanity checks such as random-label canaries. The chapter closes with a governance-native checklist that makes deep-learning claims auditable: deterministic runs, data fingerprints, split manifests, normalization proofs, causality assertions, training traces, and model registries with reproducible inference. The objective is not to promise that deep learning “works,” but to specify when it can help, how to test it honestly, and what minimum evidence is required to claim an edge.

13.1 Position in the Book

This chapter sits in Part III (Machine Learning) and is intentionally positioned *after* Chapter 11 and Chapter 12, because deep learning in trading only becomes coherent once the reader has already internalized the discipline of supervised forecasting under time. Chapter 11 established the forecasting frame: how to define targets (returns, excess returns, directional labels, or ranking objectives), how to represent predictors, how to split data along the arrow of time, and how to evaluate performance without contaminating the future. Chapter 12 then hardened that frame into a research discipline: controlling complexity through regularization, tuning hyperparameters without implicitly selecting on noise, and enforcing model selection logic that respects temporal structure. This chapter assumes both of those foundations and treats them as non-negotiable prerequisites. In the book’s arc, neural networks are not presented as a shortcut; they are presented as a *more flexible* function class that magnifies every mistake a practitioner can make. Therefore, the role of Chapter 13 is to enlarge the hypothesis space while simultaneously tightening the experimental rules.

What Chapter 13 adds is a principled account of neural networks as explicit function classes for trading-related forecasting problems. The chapter begins by re-framing neural nets as compositions of simple maps: affine transformations and nonlinearities assembled into a parameterized function that can approximate complex relationships. This matters in finance because the temptation is to speak about “models” as opaque artifacts; here we insist on describing what is being fit, how many degrees of freedom it contains, and how that capacity interacts with noisy labels. The chapter also extends the model-selection logic of Chapter 12 into the deep learning setting, where complexity is not only about the number of features or coefficients, but also about architecture, depth, width, normalization layers, optimization schedules, and training duration. In other words, the chapter treats *training procedure* as part of the model. This is a key conceptual shift: two networks with the same architecture but different learning rates, initialization, or stopping rules can behave as entirely different estimators. If Chapter 12 made the reader careful about tuning a ridge penalty or choosing a LASSO path, Chapter 13 forces that same care onto learning rates, batch sizes, dropout rates, weight decay, early-stopping patience, and even the choice of activation functions.

A second contribution of the chapter is the emphasis on *training dynamics* as an object of financial relevance. Markets are non-stationary and labels are weak; as a result, unstable training is not merely a software annoyance but a signal that the estimator is too sensitive to noise, split choice, and random seeds. The chapter therefore treats optimization (stochastic gradient descent and its variants), initialization, and normalization as core conceptual material. Rather than presenting backpropagation as a mystical procedure, we treat it as chain rule bookkeeping over composed functions, and we link that bookkeeping to practical outcomes: exploding or vanishing gradients, brittle convergence, and models that appear “good” on a lucky fold and collapse on the next. The chapter’s purpose is not to turn the reader into a deep learning engineer in general, but to make the reader competent at diagnosing why deep nets succeed or fail in the particular statistical regime of trading.

A third contribution is the discussion of architecture choices *as inductive biases over market data*. The chapter considers standard classes of models used in practice—multi-layer perceptrons,

one-dimensional convolutional networks for local pattern extraction, recurrent models for sequential dependence, and attention-based models for longer context windows—but always with an emphasis on what each architecture assumes about the data-generating process. In finance, architectural choice is not primarily about beating a benchmark on a fixed dataset; it is about encoding a plausible bias (locality, smoothness, persistence, multi-scale effects) while remaining robust under drift. This chapter therefore frames architecture as a controlled bet: we choose a function class that can express certain patterns, then we demand that those patterns survive time-aware validation and governance checks. Put differently, the chapter is not about adopting the newest neural architecture; it is about choosing the simplest architecture consistent with the hypothesis, and then proving that the observed performance is not an artifact of leakage, normalization errors, or implicit look-ahead.

This leads to the most important boundary condition for Chapter 13: *strict causality and leakage resistance remain the governing law*. Deep learning introduces new leakage surfaces that do not exist in linear models. The moment we convert a time series into windowed tensors, it becomes easy to misalign labels and inputs by one step, to normalize using statistics computed with future data, or to batch examples in ways that allow subtle information sharing across the train/validation boundary. Therefore, Chapter 13 is deliberately governance-forward. The chapter insists on explicit split manifests, explicit normalization logs (fit on training only, then applied forward), and hard “no-look-ahead” assertions that fail loudly when any alignment invariant is violated. It also emphasizes sanity checks that are particularly necessary with deep nets: random-label tests, feature permutation tests, and other canaries that reveal whether the pipeline can manufacture spurious predictability. These ideas do not replace the discipline of Chapter 12; they instantiate it in a more failure-prone setting.

The chapter’s governance-native stance is also what defines what it postpones. Regime models (Chapter 14) are deferred because they introduce a different conceptual object: latent states that segment time and allow the analyst to treat non-stationarity as structure rather than noise. Chapter 13 may motivate regime thinking by showing that model performance varies across time, but it does not introduce HMMs or regime inference methods. Likewise, event-driven and news/sentiment pipelines (Chapter 15) are postponed because they require distinct data primitives (text, timestamps, publication delays), different leakage hazards (publication vs reaction times), and different evaluation designs. Keeping Chapter 13 focused prevents an uncontrolled expansion of scope; the chapter’s job is to establish deep learning as a disciplined forecasting tool on structured market data, not to conflate it with alternative data or event studies.

Portfolio construction and risk decisions are also postponed. Chapters 16–17 begin the transition from forecasting to allocation: turning predictions into portfolios, controlling leverage, sizing positions, and managing drawdowns and exposure constraints. Chapter 13 does not attempt to answer those allocation questions because doing so would mix model estimation with objective design and risk constraints in a way that hides where performance is coming from. Here, the output of the chapter is a forecasting signal whose properties are understood: stability across folds, sensitivity to hyperparameters, robustness under canaries, and auditable provenance. Only after that is established does the book permit the reader to connect forecasts

to portfolios in a way that can be evaluated responsibly.

Execution and microstructure (Chapter 18) are deferred for similar reasons. Deep models are often appealing precisely because they can be made large and complex, but that same complexity can be operationally incompatible with execution realities: latency constraints, slippage, queue position, and transaction cost modeling. Chapter 13 keeps the analysis at the forecasting level and does not claim that a predictive edge automatically survives execution. The book’s arc makes that separation explicit: first learn to estimate signals honestly; then learn how costs, fills, and market impact erode them; then learn to design systems that are robust to those frictions.

Finally, reinforcement learning is explicitly postponed to Chapter 19. This matters because deep learning is frequently introduced in finance through RL narratives, where a network is treated as a policy rather than a forecaster. That transition is real, but it is also dangerous if the reader has not yet mastered causality, validation under time, and governance artifacts. Chapter 13 therefore draws a hard line: we are still doing supervised learning, still measuring forecasting skill, and still requiring time-aware evaluation. Neural networks are introduced as estimators of conditional expectations (or conditional ranks) under strict temporal discipline. Policy optimization, exploration, and sequential decision learning remain out of scope until Chapter 19.

In summary, Chapter 13 advances the book by adding a powerful and flexible modeling family, but it does so by tightening, not loosening, standards. The chapter’s position is defined by what it requires from earlier chapters (forecasting discipline and model selection governance), what it contributes (neural function classes, training dynamics, architecture choices as inductive biases, and deep-learning-specific leakage defenses), and what it postpones (regimes, events/news, portfolio construction, execution, and reinforcement learning). The aim is to make the reader capable of building deep learning forecasting pipelines that are auditable, causality-safe, and intellectually honest about what performance can and cannot mean in markets, before the book expands toward regimes, events, and decision-making systems in subsequent chapters.

13.2 Why Neural Nets in Trading, and Why This Is Hard

Neural networks enter trading for the same reason they enter almost every other predictive domain: they offer a flexible family of functions that can represent nonlinear relationships and interactions among predictors without requiring the researcher to pre-specify the exact form of those interactions. In finance, that flexibility is both the attraction and the trap. It is the attraction because market data are rarely polite: relationships that look linear over one regime can become curved or even inverted in another; signals often depend on interactions (a momentum effect that appears only when volatility is low, a mean-reversion effect that appears only when liquidity is abundant, a carry effect that depends on macro state). It is the trap because the financial learning problem is typically underpowered relative to the degrees of freedom we can give a network: labels are weak, distributions drift, the effective sample size is small once we respect time ordering, and the researcher has many hidden “knobs” that can accidentally tune to noise. This section explains why neural nets can be useful *within the scope of this book*—that is, as supervised forecasters of returns under strict time-aware validation

(Chapter 11) and disciplined model selection (Chapter 12)—and why, precisely in trading, even technically correct deep learning practice can still lead to fragile results.

13.2.1 What deep learning can add beyond linear models (within this book’s scope)

The first legitimate contribution of deep learning is *nonlinear function approximation* under a controlled experimental discipline. Linear models in Chapters 11–12 are not “bad”; they are often the correct starting point because they are stable, interpretable, and relatively hard to overfit when regularized and validated properly. But linearity is a strong structural assumption: it asserts that the effect of each predictor on the target is additive and constant, independent of the level of other predictors. Many trading hypotheses are, implicitly, hypotheses about interactions and conditional effects. Consider a simple example that is still within the supervised forecasting frame: suppose a short-horizon reversal signal exists primarily when spreads are tight (high liquidity) and vanishes when spreads are wide (low liquidity). A linear model can include both “reversal indicator” and “spread proxy” but cannot, without explicitly engineered interaction terms, express the conditional statement “reversal matters only in liquid conditions.” A neural network can represent such interactions naturally through its layered composition: intermediate units can encode conditional gates and nonlinear blends, and the final output can reflect combinations of features that a linear model would require hand-designed cross-terms to approximate.

A second contribution is *representation learning* for structured inputs. In trading, predictors are often produced by feature engineering pipelines (Chapter 05), and the temptation is to keep expanding those pipelines: more indicators, more windows, more transformations, more cross-products. Deep learning offers a different approach: instead of enumerating many handcrafted transforms, we provide a disciplined input representation (for instance, a fixed-length tensor of recent returns, volumes, and volatility proxies) and allow the model to learn intermediate representations that summarize what matters. Convolutional architectures can learn local motifs (e.g., short bursts, oscillations, abrupt breaks) across multiple scales; recurrent or attention-based models can learn longer dependencies or context. Even in a strictly supervised forecasting setting, this can reduce the need for brittle feature engineering, provided that the representation is constructed causally and that its complexity is justified by out-of-sample performance under time-aware evaluation.

A third contribution is *capacity to absorb heterogeneous predictors* without forcing the researcher to decide ex ante which interactions “should” matter. In practice, many trading systems combine price-based features, volatility features, macro proxies, and sometimes cross-asset information. Linear models can incorporate all of these, but they often require careful scaling, interaction design, and explicit regularization to prevent unstable coefficient estimates in the presence of collinearity. Neural networks can still be sensitive to these issues, but their flexibility allows them to learn smoother mappings in high-dimensional spaces when trained with proper regularization and early stopping. Importantly, this book’s scope insists that such flexibility be exercised only under the protective discipline of Chapter 12: architecture choice, optimizer choice, learning-rate schedules, and training duration are *hyperparameters* and must

be treated as such. “Deep learning adds value” is only meaningful if the comparison is made against strong regularized linear baselines under the same time splits and the same governance constraints.

A fourth contribution is *multi-objective forecasting* within a supervised frame. In finance, we often care not only about point forecasts of returns but also about ranking (cross-sectional selection) or probabilistic statements (calibration of sign or tail-risk events). While linear models can be adapted to these objectives, neural networks can be naturally trained with loss functions aligned to them: classification losses for direction, pairwise or listwise losses for ranking, and quantile losses for conditional quantiles. The chapter does not claim that these objectives automatically improve trading performance; rather, it emphasizes that they can be expressed cleanly, and then tested honestly, within the same governance-native evaluation framework.

Finally, deep learning can add practical value through *model ensembling and averaging* as variance reduction tools. The key idea is not “bigger is better” but “unstable estimators can sometimes be stabilized by averaging.” In trading, where labels are noisy, two models with similar average performance may have very different stability properties across folds. A small ensemble of modest networks, each trained deterministically under controlled random seeds and validated properly, can sometimes deliver more stable signals than a single complex model. This is still within the supervised scope: we are not optimizing a policy; we are producing a forecast (or a rank) and analyzing it under time-aware folds.

13.2.2 What deep learning does not magically fix: noise, drift, and selection bias

The most important point, and the one that must be stated without euphemism, is that deep learning does not create signal where there is none. Financial returns are noisy, and in many liquid markets the conditional expectation of returns given commonly available predictors is close to zero at short horizons. A neural network can fit noise extremely well; the question is whether that fitted structure persists out of sample. Deep models can easily produce impressive in-sample metrics, and even impressive validation metrics if the validation design is imperfect. This is why the chapter treats leakage defenses as a first-order concern: if you allow the model to see the future through a normalization mistake, a label alignment error, or an improper split, it will happily convert that forbidden information into apparent predictability.

Non-stationarity is the second adversary. Even when a predictive relationship is real, it may be regime-dependent or time-varying. A network trained on one period may internalize patterns that cease to hold after a structural break: a change in monetary regime, a change in market microstructure, a new participant base, or a new volatility environment. Deep learning does not solve drift; it can, in fact, amplify its consequences by learning complex decision surfaces that are fragile under distribution shift. If the model’s success depends on subtle combinations of predictors that existed historically, then drift can destroy those combinations. This chapter therefore emphasizes training and evaluation designs that reveal drift sensitivity: walk-forward folds, stability analysis across time, and error analysis broken down by volatility and liquidity proxies (as a preview of the explicit regime framework introduced in Chapter 14). The lesson is not “never use deep learning”; it is “treat temporal instability as the default, and require

evidence of robustness.”

Selection bias is the third adversary, and it has a uniquely deep-learning flavor because the space of choices is enormous. In linear modeling, the researcher tunes a handful of hyperparameters. In deep learning, architecture, depth, width, optimizer, learning rate schedule, batch size, normalization layers, dropout rates, early stopping rules, and feature representations are all potential degrees of freedom. If the researcher explores these choices aggressively and reports only the best result, the reported performance is likely a statistical artifact. Chapter 12 introduced the logic of disciplined model selection; Chapter 13 applies it ruthlessly to deep learning. The chapter insists that deep-learning “research” must look like controlled experimentation: a declared search space, a limited budget, time-aware validation, and explicit accounting of how many variants were tried. Without this discipline, the probability that a neural network appears to “work” purely because of search is unacceptably high.

A related issue is *implicit look-ahead via preprocessing*. Deep learning workflows often normalize inputs, standardize features, or perform dimensionality reduction. If those transformations are fitted on the entire dataset, they leak future information into the training set. This is not a minor technicality; it is a primary failure mode. It is especially dangerous with windowed tensors: a subtle shift in indexing can align the label with information from the same day’s close, or can allow the model to consume returns that include the period being predicted. The chapter therefore treats preprocessing as part of the model and demands forward-only fitting for any transformation with estimated parameters. It also introduces canary tests specifically designed to catch these issues: random labels should produce no predictive power; permuting time order should destroy performance; replacing the last element of each window with a future value should artificially inflate performance and therefore serve as a detectable signature of leakage sensitivity.

Finally, deep learning does not resolve the economic question of *tradability*. A forecasting improvement measured by a statistical metric can be economically irrelevant if it does not survive transaction costs, slippage, or capacity constraints. Chapter 13 stays within the forecasting scope and does not yet model execution (Chapter 18) or portfolio construction (Chapters 16–17), but it explicitly cautions that predictive metrics are not the same as implementable P&L. A neural network can be very accurate about tiny return differences that are not monetizable after costs. Therefore, the chapter’s standard for “useful” is not only a metric improvement but also stability, robustness, and an explicit statement of limitations.

13.2.3 Scope boundary: forecasting returns (Ch. 11) with neural nets (Ch. 13), not RL (Ch. 19)

Because deep learning is often introduced alongside reinforcement learning in popular narratives, this book draws a clear boundary: Chapter 13 remains in the supervised learning paradigm of Chapter 11. The object of study is a forecast function (or a ranking function) that maps information available at decision time to a prediction about future returns, directions, or relative performance. The evaluation logic remains the same: time-aware splits, walk-forward testing, and disciplined hyperparameter tuning. The governance logic remains the same: determinism, data lineage, explicit manifests, and causality gates. Neural networks are introduced here as

estimators—powerful, flexible estimators, but still estimators—and the chapter’s intellectual work is to show how to control that flexibility so that the resulting evidence can be trusted.

Reinforcement learning, by contrast, changes the problem definition. Instead of fitting a conditional expectation, we fit a policy that chooses actions to maximize long-horizon reward under feedback loops and endogenous state transitions. That shift introduces new hazards: exploration, off-policy evaluation, reward shaping, and the interaction between the agent and market impact. It also changes the meaning of validation: we must evaluate decision policies, not forecasts, and we must confront the reality that the agent’s actions can affect the environment. Those topics are deferred to Chapter 19 precisely because they require the reader to already be fluent in causality under time and in governance-native experimental discipline. If one tries to jump to RL before mastering leakage-free supervised learning, the risk is that the entire system becomes a complicated machine for manufacturing overfit.

Within this boundary, Chapter 13 has a narrow and concrete aim: to make the reader capable of designing and auditing deep learning forecasting pipelines that are time-causal, properly validated, and honestly interpreted. The chapter encourages the reader to start from strong regularized linear baselines, to add neural networks only when the hypothesis demands nonlinearity or representation learning, and to treat every additional degree of freedom as a liability that must be justified by robust evidence. If the chapter succeeds, the reader will not merely know how to “train a network,” but will understand why training a network is easy and claiming an edge is hard, and will be able to tell the difference using the same disciplined language of splits, manifests, and causality gates that the book has insisted upon since Chapter 11.

13.3 Neural Networks as Composed Functions

A useful way to demystify neural networks for trading is to treat them as nothing more (and nothing less) than *composed functions*. In Chapters 11–12 we framed supervised learning as the search for a function f that maps information available at decision time to a target defined on a future horizon. Neural networks do not change that framing; they enlarge the family of functions we are willing to search over. The key pedagogical move is therefore to stop thinking of a network as a “machine” and start thinking of it as a structured mathematical object: a composition of simple maps (affine transforms) and simple nonlinearities (activations), trained by minimizing a loss function on time-ordered data.

This perspective is particularly valuable in trading because most confusion and most errors come from mixing up three distinct ideas: (i) the *function class* (what kinds of mappings can in principle be represented), (ii) the *training procedure* (how we choose a particular mapping from data), and (iii) the *evaluation design* (how we decide whether the chosen mapping generalizes under time). This section is about (i): the function class. We will still keep one eye on (ii) and (iii), because in practice capacity, activations, and losses interact with optimization and with governance constraints, but the objective here is to build a clean mental model of what the network is, what it can represent, and what “complexity” means in this setting.

13.3.1 Perceptron, multilayer perceptron, and universal approximation (conceptual)

Let $x \in \mathbb{R}^d$ denote the feature vector available at decision time t (for example, a vector built from lagged returns, volatility proxies, or engineered indicators constructed causally in Chapter 05). A *perceptron* is the simplest neural model: an affine map followed by a nonlinearity. In its most basic form, with an activation $\phi(\cdot)$, the perceptron is

$$f(x) = \phi(w^\top x + b), \quad (13.1)$$

where $w \in \mathbb{R}^d$ and $b \in \mathbb{R}$ are parameters. If ϕ is the identity, we recover linear regression (for real-valued targets) or a linear score (for classification). If ϕ is a step function $\phi(z) = \mathbb{I}\{z \geq 0\}$, we obtain the classic linear classifier that separates the input space by a hyperplane. In trading language, this is already meaningful: even a linear score can represent a weighted combination of signals, and a step can convert a score into a discrete decision. But the perceptron, in this one-layer form, cannot represent interactions or curved decision boundaries unless those interactions are already baked into x by feature engineering.

The next step is the *multilayer perceptron* (MLP). An MLP composes multiple affine maps and nonlinearities. For a network with L layers (counting hidden layers and an output layer), we can write

$$h^{(0)} = x, \quad (13.2)$$

$$z^{(\ell)} = W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}, \quad \ell = 1, \dots, L, \quad (13.3)$$

$$h^{(\ell)} = \phi^{(\ell)}(z^{(\ell)}), \quad \ell = 1, \dots, L-1, \quad (13.4)$$

$$f(x) = g(z^{(L)}), \quad (13.5)$$

where $W^{(\ell)}$ and $b^{(\ell)}$ are weight matrices and bias vectors, $\phi^{(\ell)}$ are hidden-layer activations, and g is an output activation appropriate to the task (for example, identity for regression, sigmoid for binary probability, or softmax for multi-class probabilities). The key is that each layer creates a new representation $h^{(\ell)}$ as a nonlinear transformation of the previous representation. Instead of forcing the researcher to enumerate all interactions in x , the network can learn intermediate features that behave like interaction terms, thresholds, and mixtures.

To see why composition matters, consider a simple illustrative case: suppose the forecasting relationship depends on the conjunction of two conditions, such as “short-term reversal is strong when volatility is low *and* trend strength is weak.” In a linear model, the conjunction requires an explicit interaction feature (a product or nonlinear transform). In an MLP, hidden units can learn threshold-like features of volatility and trend strength, and the next layer can combine them, effectively implementing a soft “AND” gate. The network does not guarantee that it will learn the desired structure; it merely makes such structures representable without manual enumeration.

Representational power increases with depth and width, but it is important to state what that means precisely. A common conceptual anchor is the *universal approximation* idea: under mild conditions on the activation function (for example, non-polynomial continuous activations),

an MLP with a single sufficiently wide hidden layer can approximate any continuous function on a compact set to arbitrary accuracy. A qualitative statement is:

For many standard activations, given a compact domain $\mathcal{X} \subset \mathbb{R}^d$ and any continuous function $f^* : \mathcal{X} \rightarrow \mathbb{R}$, there exists a one-hidden-layer network $\hat{f}$ such that $\sup_{x \in \mathcal{X}} |f^*(x) - \hat{f}(x)|$ is as small as desired.

This is conceptually important but easy to misuse in trading. It does *not* say that a network trained on finite, noisy, drifting market data will learn the true mapping. It says only that the function class is large enough to represent very complicated mappings. In finance, that largeness is a liability as much as an asset. A universal approximator can approximate the true signal, but it can also approximate noise artifacts, regime-specific quirks, and leakage-induced patterns if the pipeline is flawed. Therefore, in this book, universal approximation is treated as a warning label: if the function class is very expressive, then disciplined validation (Chapter 11) and disciplined selection (Chapter 12) become even more crucial.

A second nuance is that the theorem is an *existence* statement, not an *efficiency* statement. The width required to approximate an arbitrary function can be enormous, and the network found by gradient-based training may not be the one that provides the approximation. In practice, depth often offers a more parameter-efficient way to represent compositional structure: functions that can be written as compositions of simpler functions can be represented compactly by deeper networks. This is relevant for market data because many hypotheses are naturally hierarchical: micro-movements combine into short-horizon patterns; short-horizon patterns combine into regime-sensitive context; context combines with cross-asset structure. Chapter 13 does not claim that markets are “compositional” in a way that guarantees success, but it does argue that if a hypothesis has a compositional character, then networks that respect that structure may be sensible candidates.

Finally, it is worth emphasizing that even the MLP is a *static* mapping $x \mapsto y$. When we feed a window of past observations into the feature vector x , the network is still not “dynamic”; it is simply mapping a fixed-length summary of the past to a forecast. This is consistent with the scope boundary: we are forecasting returns, not optimizing policies. More specialized sequence models (RNNs, attention) can be interpreted similarly: they are still functions mapping past information to a forecast, but they impose different structural biases about how the past should be summarized. Those architectures are discussed elsewhere in the chapter; the foundational point here is that all these models are composed functions trained by loss minimization.

13.3.2 Activations: ReLU, tanh, sigmoid, and why they matter for optimization

If affine maps were the whole story, a network would collapse to a single affine map: the composition of linear transformations is linear. The nonlinear activation is therefore the ingredient that gives the network expressive power. But in practice, activations matter for a second, equally important reason: they strongly shape the *optimization landscape* and the *flow of gradients* during training.

Let $z = w^\top x + b$ denote a pre-activation scalar. Common activations include:

Sigmoid. The sigmoid (logistic) function is

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (13.6)$$

It maps $\mathbb{R}$ to $(0, 1)$ and is historically associated with probabilistic interpretation for binary outcomes. Its derivative is

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)), \quad (13.7)$$

which is bounded above by $1/4$ and becomes very small when $|z|$ is large (saturation). This has a crucial training implication: if many units operate in saturated regions, gradients shrink as they propagate backward, leading to slow learning or failure in deep networks. In trading, where scaling and normalization are delicate (and must be fitted on training only to avoid leakage), sigmoid saturation can occur if inputs are poorly scaled or if weights grow quickly early in training.

Hyperbolic tangent. The tanh activation is

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad (13.8)$$

mapping $\mathbb{R}$ to $(-1, 1)$ with derivative

$$\frac{d}{dz} \tanh(z) = 1 - \tanh^2(z). \quad (13.9)$$

Like sigmoid, tanh saturates for large $|z|$, shrinking gradients. Its advantage is that it is zero-centered, which can make optimization easier in some settings. In financial forecasting, zero-centered activations are conceptually attractive because many targets (returns) are roughly centered around zero. However, the saturation issue remains, which is why modern deep learning often prefers non-saturating activations in hidden layers.

ReLU. The rectified linear unit is

$$\text{ReLU}(z) = \max(0, z). \quad (13.10)$$

Its derivative is 1 for $z > 0$ and 0 for $z < 0$ (and undefined at 0, usually handled by a subgradient convention). ReLU does not saturate on the positive side, which typically improves gradient flow and enables deeper networks to train more reliably. It also introduces a sparsity-like effect: units with negative pre-activations output zero, effectively turning off. In a trading interpretation, this can behave like a learned gating mechanism, where certain intermediate features become active only in certain regions of the input space (for example, only when volatility is above a threshold). This gating can be useful for conditional effects. But ReLU introduces its own risk: if a unit's pre-activation becomes negative for all training samples, the gradient through that unit is zero and it may never recover (*dead ReLU*). Careful initialization and learning-rate choices mitigate this, but the point is that activation choice is inseparable from optimization stability.

Why activations matter for optimization. Training a network typically means minimizing an empirical loss over parameters θ (collecting all weights and biases):

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_{\theta}(x_i), y_i), \quad (13.11)$$

where (x_i, y_i) are training pairs ordered in time and selected by the fold logic of Chapter 11. Gradient-based methods update parameters in the direction of the negative gradient:

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} \left[\frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_{\theta}(x_i), y_i) \right], \quad (13.12)$$

with step size (learning rate) η . The gradient ∇_{θ} is computed by backpropagation, which repeatedly applies the chain rule through compositions. This is where activations matter: each layer contributes a factor involving the derivative of its activation. If those derivatives are mostly small (as in saturated sigmoid/tanh), gradients shrink exponentially with depth (vanishing gradients). If derivatives or weight matrices amplify too much, gradients can blow up (exploding gradients). ReLU-like activations reduce one major source of vanishing gradients, but do not eliminate all instability.

In trading, optimization stability matters for two reasons beyond convenience. First, unstable training makes results non-reproducible across seeds, which complicates governance and increases the risk of selection bias (picking the run that “worked”). Second, unstable training may indicate that the estimator is overly sensitive to the sample, which often correlates with poor out-of-sample robustness under drift. Therefore, activation choice is not merely an engineering detail; it is part of the model’s inductive bias and part of its effective complexity.

Output activations. While hidden layers often use ReLU (or similar variants), output activations are chosen to match the meaning of the output. For regression forecasting of returns, it is common to use identity output:

$$\hat{y} = f_{\theta}(x) \in \mathbb{R}. \quad (13.13)$$

For binary classification (e.g., predicting whether the next return is positive), sigmoid output is natural:

$$\hat{p} = \sigma(f_{\theta}(x)) \in (0, 1), \quad (13.14)$$

interpreted as $\mathbb{P}(y = 1 \mid x)$ under a probabilistic model. For multi-class classification, a softmax maps a vector of scores to a probability simplex. These choices link directly to the loss functions used for training, which we discuss next.

13.3.3 Loss functions for trading targets: regression vs classification vs ranking

A neural network becomes a practical estimator only when we specify what it is being trained to do. That specification is made through the *loss function*. In trading, this choice is subtle because the same economic objective can be expressed in multiple statistical ways, and because some losses are easier to optimize than others. Moreover, due to the scope boundary, we focus on losses that produce *forecasts* or *rankings*, not losses that directly optimize a trading policy with transaction costs (those belong later, once execution and portfolio constraints are introduced).

Regression targets. A canonical supervised target is a future return:

$$y_t = r_{t \rightarrow t+h}, \quad (13.15)$$

where h is a fixed forecast horizon (one day, five days, etc.), and the feature vector x_t is constructed only from information up to time t (strictly causal). The simplest regression loss is mean squared error (MSE):

$$\mathcal{L}_{\text{MSE}}(\hat{y}, y) = (\hat{y} - y)^2. \quad (13.16)$$

Minimizing MSE corresponds (under standard assumptions) to estimating the conditional expectation $\mathbb{E}[y \mid x]$. In trading, this is meaningful but can be fragile: returns have heavy tails and occasional jumps, and MSE places quadratic weight on large errors. This can cause the model to devote capacity to rare extremes in ways that are not robust. A common alternative is the Huber loss, which is quadratic near zero and linear in the tails:

$$\mathcal{L}_{\delta}(\hat{y}, y) = \begin{cases} \frac{1}{2}(\hat{y} - y)^2, & |\hat{y} - y| \leq \delta, \\ \delta \left(|\hat{y} - y| - \frac{1}{2}\delta \right), & |\hat{y} - y| > \delta. \end{cases} \quad (13.17)$$

The parameter δ controls where the loss transitions from quadratic to linear. Conceptually, Huber loss can yield estimators that are more robust to outliers. Within the governance discipline of Chapter 12, δ is a hyperparameter and must be tuned with time-aware validation.

Regression can also target transformed returns (for example, clipped returns or volatility-scaled returns) to stabilize training, but transformations must be chosen carefully and justified economically. A transformation that looks beneficial in-sample can encode regime-specific artifacts, so any such choice must be evaluated across folds and accompanied by explicit documentation in the run manifest.

Classification targets. Sometimes the trading hypothesis is about direction rather than magnitude. Define

$$y_t = \mathbb{I}\{r_{t \rightarrow t+h} > 0\} \in \{0, 1\}. \quad (13.18)$$

A standard loss is binary cross-entropy (log loss). If the network outputs a probability $\hat{p} \in (0, 1)$, then

$$\mathcal{L}_{\text{BCE}}(\hat{p}, y) = -[y \log(\hat{p}) + (1 - y) \log(1 - \hat{p})]. \quad (13.19)$$

Minimizing this loss corresponds to maximum likelihood under a Bernoulli model. The benefit is that the output can be interpreted probabilistically and can be calibrated. The risk is that direction labels discard magnitude information and can be noisy at short horizons where sign is close to random. Still, direction classification can be useful for certain decision rules or for risk-filtering, provided that performance is assessed honestly and not conflated with implementable profit.

For multi-class settings (e.g., “down, flat, up”), one can define thresholds and use softmax with categorical cross-entropy. But thresholding introduces additional hyperparameters and can increase selection bias unless handled carefully. In a book that emphasizes first-principles and governance, the recommendation is to keep label design simple unless there is a strong reason to complicate it.

Ranking targets. Many trading problems are naturally cross-sectional: at each date, we rank assets and form long/short portfolios based on ranks (Chapter 09 formalizes that world). Even within a time-series setting, one might want the model to produce a score whose *ordering* correlates with future returns. Ranking objectives are therefore relevant even before we formally move to cross-sectional factor models.

A simple ranking-oriented loss is pairwise: given two assets (or two candidates) i and j with future returns y_i and y_j at the same decision time, we want the model scores s_i and s_j to preserve the ordering: $s_i > s_j$ when $y_i > y_j$. A logistic pairwise loss is

$$\mathcal{L}_{\text{pair}}(s_i, s_j, y_i, y_j) = \log(1 + \exp(-\text{sign}(y_i - y_j)(s_i - s_j))). \quad (13.20)$$

This encourages correct ordering without requiring the model to match return magnitudes. Conceptually, this can be aligned with information coefficient ideas (correlation between scores and outcomes), though the evaluation metric is separate from the training loss. The key caution is that ranking losses require carefully constructed training examples: comparisons must be made within the same time slice to avoid inadvertently leaking time information. Moreover, in purely time-series single-asset forecasting, ranking is less natural; it becomes more natural when the “instance” is an asset-date pair and the training batches are constructed per date. That setup is beyond what we fully develop here, but the loss illustrates how deep learning can be aligned with ordinal objectives.

Loss choice as a statement of belief. Choosing MSE versus cross-entropy versus pairwise ranking is not simply a technical selection; it is a statement about what kind of predictability one believes exists. If one believes the model can estimate conditional mean returns, regression losses are natural. If one believes only directional edges exist, classification may be appropriate. If one believes the model can rank relative opportunities better than it can forecast levels,

ranking losses may be the right training target. Crucially, Chapter 12’s discipline applies: loss choice must be compared against baselines and validated under time, and the number of choices explored must be documented to avoid selection bias.

Losses are not profits. It is tempting to define a loss that looks like P&L and train the network to “maximize returns.” Within the scope of Chapter 13, we avoid that temptation for principled reasons. P&L depends on transaction costs, execution, constraints, and dynamic position sizing (Chapters 16–18). Training directly on P&L without modeling those components risks building a model that exploits backtest artifacts. Therefore, the losses emphasized here are forecasting-aligned and interpretable. The mapping from forecasts to trades is discussed later, once the book has introduced the necessary mechanics and governance for costs and sizing.

13.3.4 Parameter counting, capacity, and effective complexity

A deep learning model can be described in one sentence: “a lot of parameters fitted by minimizing a loss.” In trading, that sentence contains both the promise and the danger. To reason about overfitting, stability, and governance, we need to quantify *how many parameters* we are fitting and what that implies about the model’s capacity.

Counting parameters in an MLP. Consider a fully connected network with input dimension d , hidden layer widths $m_1, \dots, m_{L-1}$, and output dimension k (often $k = 1$ for regression or binary classification logits). The weight matrix at layer ℓ has shape $m_\ell \times m_{\ell-1}$ (with $m_0 = d$ and $m_L = k$), and the bias has length m_ℓ . The total parameter count is

$$P = \sum_{\ell=1}^L (m_\ell m_{\ell-1} + m_\ell). \quad (13.21)$$

Even modest widths can lead to large P . For example, with $d = 50$, two hidden layers of width $m_1 = m_2 = 128$, and $k = 1$, we have

$$P = (128 \cdot 50 + 128) + (128 \cdot 128 + 128) + (1 \cdot 128 + 1) \quad (13.22)$$

$$= (6400 + 128) + (16384 + 128) + (128 + 1) \quad (13.23)$$

$$= 6528 + 16512 + 129 = 23169. \quad (13.24)$$

Twenty-three thousand parameters may not sound large in modern deep learning, but in a financial forecasting setting it can be enormous relative to the effective information content of the data, especially once we enforce time splits and embargoes. The point is not that such a model is impossible; it is that its flexibility must be matched by validation discipline and by regularization, and its instability must be measured, not assumed away.

Capacity versus sample size under time. A naive comparison would match P to the number of training samples N . In iid settings, one sometimes uses $N \gg P$ as a rough comfort heuristic (though even that can be misleading). In trading, N itself is deceptive. If data are autocorrelated, if labels overlap, if the signal-to-noise ratio is tiny, and if the distribution

drifts, the *effective* sample size is much smaller than the raw count of observations. Moreover, Chapter 11 insists on holding out future periods, which reduces training data in each fold. Therefore, parameter counting is not a guarantee of anything, but it provides a necessary sanity check: if P is large and N_{train} in each fold is limited, the default expectation should be overfitting unless regularization and early stopping are strong and unless performance is stable across folds.

Effective complexity: more than parameter count. Parameter count is a crude proxy for complexity. Two networks with the same P can behave very differently depending on activations, weight norms, regularization, and training dynamics. In practice, what often matters is not just how many parameters exist, but *how large* the fitted weights become and how sharply the function can vary over the input space. A useful conceptual handle is that functions with smaller weight norms are often smoother and less sensitive to small perturbations. Weight decay (L2 regularization) explicitly penalizes large weights, shrinking norms and often improving generalization:

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_{\theta}(x_i), y_i) + \lambda \|\theta\|_2^2. \quad (13.25)$$

Within the book’s arc, this directly connects to Chapter 12: λ is a regularization hyperparameter, and its value trades bias and variance. For deep nets, weight decay can be viewed not only as “preventing overfit” but as defining which solutions among many interpolating solutions the optimizer prefers.

Early stopping as capacity control. Training time is also a form of complexity control. Even if P is fixed, a network trained for a very long time can overfit more than the same network trained briefly. Early stopping chooses a training iteration k^* based on validation performance:

$$k^* = \arg \min_{k \in \{1, \dots, K\}} \mathcal{L}_{\text{val}}(\theta_k), \quad (13.26)$$

where θ_k denotes parameters after k updates. Conceptually, early stopping constrains how far the optimizer can move in parameter space, often limiting weight growth and function complexity. In a trading setting, early stopping is particularly important because the model can quickly fit transient patterns that do not persist. However, early stopping itself is a hyperparameter-dependent procedure (patience, minimum improvement thresholds, learning-rate schedules), so it must be governed and audited like any other tuning choice.

Dropout and implicit ensembles. Dropout is another mechanism that affects effective complexity. In dropout training, hidden units are randomly set to zero with some probability p during training. This discourages reliance on any single pathway through the network and can act like training an ensemble of subnetworks that share weights. The economic interpretation is not “randomness makes it better,” but “the estimator is forced to be robust to missing intermediate features,” which can reduce co-adaptation and overfitting. Again, p is a hyperparameter. In finance, where reproducibility and governance matter, dropout’s randomness must be controlled by seeds and included in the run manifest.

The interpolation phenomenon and why it is dangerous to romanticize. Modern deep learning often observes that large networks can fit training data nearly perfectly (interpolation) and still generalize. This has led to discussions of “double descent” and the idea that over-parameterization is not necessarily harmful. In trading, it would be a mistake to romanticize this phenomenon. Markets differ from many benchmark ML settings because label noise is not just measurement noise; it is often fundamental unpredictability. A model that interpolates noisy returns can do so by fitting micro-artifacts that are not stable across time. Over-parameterization can therefore increase the temptation and ability to overfit, especially when combined with extensive hyperparameter search. The book’s governance stance is conservative: we accept that over-parameterized models can sometimes generalize, but we refuse to infer generalization without strong time-aware evidence and stability checks. “It trained well” is not evidence; “it was stable across walk-forward folds under limited, declared tuning budgets and passed leakage canaries” begins to be evidence.

Capacity as a portfolio of risks. In a forecasting pipeline, model capacity interacts with multiple risks:

- *Statistical risk*: fitting noise, instability across folds.
- *Selection risk*: many architectures and training settings tried, best result reported.
- *Leakage risk*: higher capacity exploits subtle leakage more effectively.
- *Operational risk*: more complex models are harder to reproduce, monitor, and debug.

Although we avoid bullet-heavy exposition elsewhere, this compact list is worth stating explicitly because it clarifies why this chapter repeatedly insists on governance artifacts. Effective complexity is not a single number; it is a bundle of choices, each of which can introduce fragility.

A practical complexity principle for this book. Given the above, a practical principle consistent with the book’s arc is: *increase capacity only as needed to express the hypothesis, and only after a strong baseline is established.* If a regularized linear model already captures most of the stable predictive structure in the data, adding a large network can easily reduce robustness. If the hypothesis genuinely requires nonlinear interactions or representation learning from structured inputs, then a network is justified, but the architecture should be the simplest that plausibly captures the needed structure. Parameter counts, training curves, and fold stability should then be treated as first-class outputs of the research process, not as afterthoughts.

Connecting back to trading meaning. In financial forecasting, the goal is not to win an abstract prediction contest; it is to produce a signal that can survive time, drift, and later, costs. Thinking in composed functions helps keep that goal honest. The network is a mapping from decision-time information to a forecast; activations determine what kinds of nonlinearities and gates are representable and how training behaves; losses define what it means to “fit”; and capacity measures how much freedom the model has to fit both signal and noise. When these pieces are understood as explicit objects, the researcher can ask the right questions: Is

the function class justified by the hypothesis? Is the activation choice consistent with stable optimization under our causal preprocessing constraints? Is the loss aligned with the target we care about (magnitude, direction, rank)? Is the capacity proportionate to the effective sample size and to the drift risk? Those questions are the bridge between deep learning as a generic tool and deep learning as a disciplined component of a trading research workflow.

In the chapters that follow, we will continue to treat these design choices as governed decisions. We will not assume that deep learning “works”; we will construct the evidence case for when it may help, and we will explicitly track the artifacts that make that evidence auditable. This section provides the mathematical language for that discipline: neural networks are composed functions, and in trading, the costs of compositional expressiveness must always be paid in the currency of careful validation and rigorous governance.

13.4 Training Dynamics: Optimization, Gradients, and Stability

Deep learning in trading rises or falls on a very practical question: *can we train the model in a way that is stable, reproducible, and meaningfully related to out-of-sample performance under time?* In textbooks, training is often presented as a clean minimization problem with neat convergence stories. In markets, training is the messy middle layer between a plausible hypothesis and an honest evaluation. The same architecture can look brilliant or useless depending on learning rate, initialization, normalization, and the interaction of these choices with noisy labels and non-stationary data.

This section explains training dynamics with minimal mathematics and maximum clarity. The goal is to make each ingredient of the training pipeline feel concrete: what gradient descent is doing, why we use mini-batches, how backpropagation computes gradients, why learning rates can make training explode or stall, why “just failing” is often predictable, and why early stopping is not a hack but a disciplined regularizer that fits naturally into the book’s Chapter 12 emphasis on controlling complexity.

Throughout, keep the supervised forecasting frame in mind (Chapter 11). We have observations indexed by time $t = 1, 2, \dots, T$. At each time t we have features x_t (constructed causally from information available up to time t) and a target y_t (defined on a future horizon, such as the return from t to $t + h$). We fit a neural network $f_\theta(\cdot)$ with parameters θ to produce predictions $\hat{y}_t = f_\theta(x_t)$, and we choose θ to minimize a loss. Training is simply the method by which we choose θ .

13.4.1 Gradient descent, mini-batches, and stochasticity (mechanics and intuition)

The basic objective. We start with a training set (for one fold) containing N pairs (x_i, y_i) , where each index i corresponds to some time point (and possibly some asset, but we focus on the single-instance perspective). The empirical risk (average loss) is

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_\theta(x_i), y_i), \quad (13.27)$$

where $\mathcal{L}$ is a loss function such as squared error $(\hat{y} - y)^2$ for regression or cross-entropy for classification. Training means: find θ that makes $J(\theta)$ small. In deep learning, we do not generally solve this exactly; we reduce it iteratively.

Gradient descent as “walk downhill.” The gradient $\nabla_{\theta}J(\theta)$ is a vector of partial derivatives:

$$\nabla_{\theta}J(\theta) = \left(\frac{\partial J}{\partial \theta_1}, \frac{\partial J}{\partial \theta_2}, \dots \right). \quad (13.28)$$

Each component tells us: if we increase θ_k slightly, how does J change? If $\frac{\partial J}{\partial \theta_k}$ is positive, increasing θ_k increases J , so to decrease J we should move θ_k down. If it is negative, we should move it up. Gradient descent does this simultaneously for all parameters:

$$\theta^{(m+1)} = \theta^{(m)} - \eta \nabla_{\theta}J(\theta^{(m)}), \quad (13.29)$$

where m is the iteration number and $\eta > 0$ is the learning rate (step size). This is the most important training equation in the chapter. Everything else is about making it practical and stable.

A good mental model is walking downhill in fog. $J(\theta)$ is the altitude, θ is your location, and the gradient tells you the direction of steepest ascent, so $-\nabla J$ is steepest descent. The learning rate η is how big a step you take. Steps that are too large overshoot valleys; steps that are too small waste time and may appear stuck.

Why not compute the full gradient every time? Computing $J(\theta)$ and $\nabla J(\theta)$ over all N examples can be expensive. More importantly, in trading settings N can be large if we use many time points or many assets. A standard trick is to use a *mini-batch*: a small subset B of size $|B| = b$:

$$J_B(\theta) = \frac{1}{b} \sum_{i \in B} \mathcal{L}(f_{\theta}(x_i), y_i), \quad (13.30)$$

and update

$$\theta^{(m+1)} = \theta^{(m)} - \eta \nabla_{\theta}J_B(\theta^{(m)}). \quad (13.31)$$

If B is chosen as a random subset, then ∇J_B is a noisy estimate of ∇J . This is *stochastic gradient descent* (SGD). The key point is that, on average, the mini-batch gradient points roughly in the right direction, but it has noise.

Why stochasticity can help. The noise is not only tolerated; it can be useful. Deep learning objectives are non-convex: they have many valleys and saddles. A deterministic method can get stuck in flat regions or poor local minima. Noise can help “shake” the parameters out of flat spots and can prevent the optimizer from over-committing early to a brittle solution. In trading, where labels are noisy and the landscape can be chaotic, this can improve practical training.

But stochasticity has a cost: results can vary across runs because mini-batch selection and

initial weights differ. That is why the book’s governance stance demands deterministic seeds and recorded run manifests: if a network “works” only on one seed and fails on others, that is evidence of instability.

Mini-batches and time ordering. A special caution in trading is time. In many ML settings, one shuffles data freely to build mini-batches. In time series forecasting, shuffling can break the dependence structure and can cause subtle leakage if batching interacts with normalization. In practice, many training pipelines do shuffle within the training period for efficiency, but the critical governance requirement is: *validation and test periods must remain strictly later than the training period*. Shuffling within training does not leak future information if the features and labels are properly defined causally, but it can obscure debugging because time-local regimes get mixed. An alternative is to use mini-batches that preserve local time blocks. The chapter’s core rule is that you must be able to state, in the split manifest, how batches were formed and confirm that no batch ever includes future-period information relative to the training boundary.

Batch size intuition. Batch size b trades off noise and computational efficiency:

- Small b (e.g., 16, 32) $\Rightarrow$ noisier gradients, more updates per epoch, potentially better exploration, but more variance and less stable training curves.
- Large b (e.g., 256, 1024) $\Rightarrow$ smoother gradients, more stable curves, but can converge to sharp solutions and may require careful learning-rate scaling.

In finance, the choice is not just about speed. If small batches produce wildly varying results across seeds and folds, that variance is a warning. If very large batches make training stable but reduce generalization, that is also a warning. The right choice depends on data size, signal-to-noise ratio, and the architecture.

Epochs. An epoch means one full pass through the training set. With mini-batches, an epoch consists of N/b updates (roughly). The training loop is therefore:

$$\text{for epoch } e = 1, \dots, E : \quad \text{for each batch } B : \theta \leftarrow \theta - \eta \nabla J_B(\theta). \quad (13.32)$$

This simple loop hides most of the training dynamics. To understand stability, we need to understand how gradients are computed.

13.4.2 Backpropagation as chain rule bookkeeping (high-level derivation)

Backpropagation is often described as a magical algorithm. It is not. It is the chain rule applied repeatedly to a composed function. The only reason it feels magical is that it automates derivative calculations for many parameters.

A two-layer example. Consider a simple network with one hidden layer:

$$z = Wx + b, \quad (13.33)$$

$$h = \phi(z), \quad (13.34)$$

$$\hat{y} = v^\top h + c, \quad (13.35)$$

where W is a matrix, b a vector, v a vector, and c a scalar. Suppose we use squared error loss:

$$\mathcal{L}(\hat{y}, y) = (\hat{y} - y)^2. \quad (13.36)$$

We want derivatives with respect to parameters v, c, W, b . Let

$$e = \hat{y} - y. \quad (13.37)$$

Then $\mathcal{L} = e^2$, so

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = 2e. \quad (13.38)$$

Now, $\hat{y} = v^\top h + c$, so

$$\frac{\partial \hat{y}}{\partial v} = h, \quad \frac{\partial \hat{y}}{\partial c} = 1, \quad \frac{\partial \hat{y}}{\partial h} = v. \quad (13.39)$$

By chain rule,

$$\frac{\partial \mathcal{L}}{\partial v} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial v} = (2e) h, \quad (13.40)$$

$$\frac{\partial \mathcal{L}}{\partial c} = (2e) \cdot 1 = 2e, \quad (13.41)$$

$$\frac{\partial \mathcal{L}}{\partial h} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h} = (2e) v. \quad (13.42)$$

So far, we have gradients for the output layer and a gradient signal flowing back to the hidden layer: $\partial \mathcal{L} / \partial h$.

Next, $h = \phi(z)$, applied elementwise. Let $\phi'(z)$ be the elementwise derivative. Then

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial h} \odot \phi'(z), \quad (13.43)$$

where $\odot$ denotes elementwise multiplication. Finally, $z = Wx + b$, so

$$\frac{\partial z}{\partial W} = (\text{depends on } x), \quad \frac{\partial z}{\partial b} = 1, \quad \frac{\partial z}{\partial x} = W^\top. \quad (13.44)$$

The parameter gradients become

$$\frac{\partial \mathcal{L}}{\partial W} = \left(\frac{\partial \mathcal{L}}{\partial z} \right) x^\top, \quad (13.45)$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial z}. \quad (13.46)$$

This is the whole story: compute the loss, compute an error signal at the output, propagate that signal backward through the derivatives of each layer, and accumulate gradients.

The general pattern. For a deep network, the same pattern repeats. Each layer receives an “upstream” gradient signal and transforms it via local derivatives. Backpropagation is simply an efficient way to avoid recomputing the same partial derivatives. In each layer ℓ we store intermediate values (such as $z^{(\ell)}$ and $h^{(\ell)}$) during the forward pass. During the backward pass, we compute a local gradient for that layer and pass a new gradient backward.

Why this matters for stability. The chain rule multiplies derivatives across layers. If derivatives are consistently small (e.g., saturated sigmoid/tanh), gradients shrink as they move backward. If derivatives or weight matrices amplify, gradients can explode. This explains why training stability is sensitive to activation choice, weight initialization, and normalization. Backpropagation itself is not unstable; the network and training setup can make the gradients unstable.

Backpropagation and trading data. The book’s time-aware stance matters here because gradient computation assumes that each training example (x_i, y_i) is valid and causally constructed. If x_i accidentally includes future information, backprop will exploit it extremely effectively, producing apparently excellent training and validation performance. That is why causality gates are not optional in deep learning pipelines: backprop will faithfully optimize whatever objective you gave it, including a leaked objective.

13.4.3 Learning rates, schedules, and failure modes (divergence, plateaus)

If there is one knob that most directly determines whether training succeeds, it is the learning rate η . The learning rate controls step size in parameter space. Because the loss surface is complicated, there is no single best value; but the qualitative failure modes are common and predictable.

Divergence: steps too large. If η is too large, updates overshoot minima and can cause the loss to increase. In severe cases, weights can blow up to very large values, producing numerical issues (NaNs) and completely unstable training. A simple intuition comes from a one-dimensional quadratic: $J(\theta) = a(\theta - \theta^*)^2$. Gradient descent update is

$$\theta_{m+1} = \theta_m - \eta \cdot 2a(\theta_m - \theta^*). \quad (13.47)$$

This converges only if η is not too large relative to a . In deep networks, local curvature changes across directions, so a learning rate that is safe in one region can be too large in another. This is why training can begin fine and then suddenly explode.

Plateaus: steps too small or gradients too small. If η is too small, training is painfully slow and may appear stuck. But plateaus can also occur when gradients are small due to saturation or due to poor initialization. In such cases, increasing η may help, but can also

destabilize training elsewhere. The model may also sit near a saddle point: a region where the gradient is close to zero but the point is not a minimum. Stochasticity from mini-batches can help escape saddles, but sometimes the system still crawls.

Learning-rate schedules. A learning-rate schedule changes η over time. A common intuition is: start with a larger learning rate to make rapid progress, then reduce it to refine. Formally, one can define η_m as a function of iteration m , such as:

$$\eta_m = \frac{\eta_0}{1 + \alpha m} \quad (\text{simple decay}), \quad (13.48)$$

or step-down schedules (reduce η by a factor when validation loss stops improving). In deep learning practice, schedules can improve both speed and stability. In a governance-native pipeline, the schedule is part of the model specification and must be recorded. In trading, schedules can also interact with non-stationarity: if training is done on a rolling window, the right schedule length may depend on the window length and the noise level.

Why adaptive optimizers exist (conceptual). Many practitioners use optimizers like Adam, which adapt learning rates per parameter based on gradient statistics. Without going deep into the algorithm, the conceptual reason is simple: different parameters experience different gradient magnitudes and curvatures. A single global η can be too small for some directions and too large for others. Adaptive methods try to normalize updates so that each parameter moves an appropriate amount. This can improve stability and reduce tuning burden. However, in trading research, adaptive methods can also increase the degrees of freedom and can sometimes overfit if not validated carefully. The chapter’s stance is: you may use them, but treat optimizer choice and its hyperparameters as part of the tuning budget and document them.

Failure modes that matter in finance. Training failure is not always dramatic. A model can “train” and produce decreasing training loss while still being useless. Common failure patterns include:

- *Overfit*: training loss decreases, validation loss worsens.
- *Underfit*: both losses remain high; model too small or learning rate too low.
- *Unstable*: loss oscillates wildly; learning rate too high or batch size too small.
- *Seed sensitivity*: one seed looks good, others fail; indicates instability or selection bias.

In trading, seed sensitivity is especially important because it is a practical proxy for how fragile the “edge” is. A real, stable signal should not depend critically on one lucky initialization.

13.4.4 Initialization and normalization: why training sometimes “just fails”

A beginner’s frustration with deep learning often sounds like: “I changed nothing important and it stopped working.” This is usually not random; it is the interaction of initialization and normalization with gradients.

Why initialization matters. At the start of training, weights are random (but deterministic given a seed). If weights are too large, pre-activations z become large, saturating sigmoid/tanh and killing gradients. If weights are too small, signals shrink across layers and gradients can vanish. Good initialization aims to keep activations in a reasonable range. Conceptually, one wants the variance of activations to be roughly stable across layers. This is why common schemes scale random weights by the inverse square root of layer width. You do not need the full derivation to understand the idea: we choose initial weights so that typical outputs neither explode nor vanish as they pass through many layers.

In trading, initialization matters even more because signal-to-noise is small. If the initial network produces outputs that are too large, early gradients may be dominated by noise and can push parameters into unstable regions. If outputs are too small, learning can be extremely slow. This is why training can appear to “do nothing” for many epochs in poorly initialized setups.

Normalization: making inputs and intermediate activations well-behaved. Normalization has two meanings in our context:

1. *Input normalization:* scaling the features x (e.g., subtract mean, divide by standard deviation).
2. *Internal normalization:* normalizing intermediate activations inside the network (e.g., batch normalization, layer normalization).

Input normalization is essential in many models because features can have different scales (returns, volumes, volatilities). But in trading, input normalization is a notorious leakage trap. The correct rule is: compute normalization statistics (means, standard deviations) using *training data only*, then apply them forward to validation and test. If you normalize using all data, you leak future distribution information, and a deep model can exploit that leak.

Internal normalization is used to stabilize training by keeping activation distributions consistent. Conceptually, if each layer receives inputs with wildly changing scale over training, optimization becomes harder. Normalization reduces this problem and can allow higher learning rates. However, it introduces additional computation and potential subtleties. In a governance-native pipeline, you must treat normalization choices as part of the model spec and ensure determinism. Also, because markets drift, the distribution of inputs can change across time; input normalization must therefore be handled carefully when retraining on rolling windows, and statistics must be logged as artifacts.

Why training “just fails.” Now we can list the most common reasons a deep model appears to fail suddenly:

- The learning rate is too high for the chosen initialization and batch size, causing divergence.
- The learning rate is too low, causing near-zero progress.
- Activations saturate (sigmoid/tanh) due to scaling, causing vanishing gradients.
- ReLU units die due to negative pre-activations and poor initialization, causing capacity loss.

- Input normalization is inconsistent or leaking, causing apparent success or instability across folds.
- The data window changed (rolling window), altering the scale and noise structure.

These are not mysteries. They are mechanical consequences of gradients and scaling.

In trading, we add one more important possibility: the relationship in the data has changed. A model that trained well last month may fail this month because the underlying distribution shifted. That is not a software failure but a market reality. This is why the chapter emphasizes stability diagnostics and why it previews regime analysis (Chapter 14) as a structured way to understand non-stationarity.

13.4.5 Early stopping as a time-aware regularizer (ties to Ch. 12)

Early stopping is often described as “stop training when validation loss stops improving.” That is correct, but too shallow. In this book’s arc, early stopping is a form of *regularization* and therefore belongs conceptually next to Chapter 12.

The basic procedure. We split data into training and validation sets along time. During training, after each epoch (or after a fixed number of updates), we compute validation loss:

$$J_{\text{val}}(\theta^{(m)}) = \frac{1}{N_{\text{val}}} \sum_{i \in \text{val}} \mathcal{L}(f_{\theta^{(m)}}(x_i), y_i). \quad (13.49)$$

We keep track of the best validation loss seen so far. If validation loss does not improve for a certain number of checks (“patience”), we stop and restore the best parameters. This yields a chosen iteration m^* :

$$m^* = \arg \min_m J_{\text{val}}(\theta^{(m)}). \quad (13.50)$$

Why this regularizes. A network with many parameters can keep reducing training loss by fitting finer and finer details of the training set, including noise. Early in training, the network tends to learn broad patterns that generalize; later, it can learn idiosyncrasies. Stopping early prevents the network from fitting those idiosyncrasies. That is regularization: it constrains the effective complexity of the fitted function by limiting how far parameters travel in training. The model family is large, but the chosen model is restricted by the stopping time.

Time-aware meaning. The phrase “time-aware” matters because, in trading, validation is not just a random sample; it is a future period relative to training. Early stopping therefore enforces a practical discipline: we stop at the point that best predicts a *later* block of time. This is exactly aligned with the logic of Chapter 11 and Chapter 12. It is also why early stopping must be handled carefully: if you monitor performance too frequently and tune stopping rules aggressively, you can overfit to the validation period. Patience, check frequency, and stopping criteria are hyperparameters that must be included in the selection discipline.

Early stopping and selection bias. Early stopping creates an additional degree of freedom: the iteration number is selected based on validation. If one also tunes learning rates, architectures, and other hyperparameters, selection bias can become severe. The correct governance stance is:

- declare the early stopping rule and patience;
- apply it consistently across candidates;
- record the chosen m^* and validation curves as artifacts;
- evaluate final performance on a strictly held-out test period that was not used for stopping.

This is precisely the model selection discipline of Chapter 12 applied to training time.

Why early stopping is often essential in finance. Because financial labels are noisy and drift is real, the “generalization window” during training can be narrow. A network can quickly begin fitting transient microstructure or historical quirks. Early stopping is therefore not optional; it is one of the primary defenses against fragile overfit. But it is not a magic defense. If the validation period is itself unrepresentative, early stopping can choose a model that generalizes poorly to the next period. That is why walk-forward evaluation (multiple folds) is crucial: the stopping behavior should be stable across folds, not lucky once.

What to look at in practice. A stable deep learning pipeline in trading should produce:

- smooth, reproducible training curves across seeds (within reasonable variation);
- validation curves that improve and then flatten or worsen (indicating a sensible early stop);
- fold-to-fold stability in both the chosen stopping times and the resulting out-of-sample metrics;
- failure detection when curves behave pathologically (divergence, oscillation, or no learning).

These are not aesthetic preferences; they are governance artifacts that let you argue that the model is not a one-off.

The big picture. Training dynamics are not a separate topic from trading realism; they are one of the primary interfaces between statistical learning theory and market data. Gradient descent, mini-batches, and backpropagation explain how parameters move. Learning rates, initialization, and normalization explain why those moves are stable or unstable. Early stopping explains how we control effective complexity and remain consistent with the book’s disciplined approach to model selection under time. If you understand these mechanics, you can usually diagnose training failures quickly and, more importantly, you can recognize when “success” is suspicious. In financial machine learning, being able to diagnose suspicious success is as valuable as being able to improve a model.

This is why Chapter 13 treats training as a governed procedure. In later chapters, when we discuss regimes (Chapter 14), event-driven signals (Chapter 15), portfolio construction (Chapters 16–17), and eventually decision-making agents (Chapter 19 and beyond), the systems become more complex. Training discipline established here becomes a foundational competence: without stable, auditable training, later sophistication is indistinguishable from elaborate overfit.

13.5 Architectures for Market Data (and Their Inductive Biases)

In trading, choosing a neural architecture is not mainly a question of fashion. It is a question of *inductive bias*: what kinds of patterns does the model assume are likely, and what kinds of patterns does it make cheap or expensive to represent? Because financial data are noisy and non-stationary, the wrong inductive bias does not merely reduce predictive accuracy; it can create an illusion of structure where none exists, or it can overfit to regime-specific quirks that do not persist. The practical objective of this section is therefore conservative: understand what common architectures are really doing, what they assume about market data, and when a simpler model is not only sufficient but preferable.

The thread connecting all architectures in this chapter is the supervised forecasting frame from Chapter 11. At each decision time t , we construct a feature object from information up to t and ask the model to predict a target defined on a future horizon $t \rightarrow t + h$. The architectures differ primarily in how they represent the past and how they share parameters across time. This parameter sharing is the central idea behind most sequence architectures: it reduces the number of free parameters compared to treating each time index as unrelated, and it encodes a belief that similar patterns can occur at different times. In markets, this is a reasonable belief (patterns repeat), but it is also a dangerous one (markets change). The inductive bias must therefore be matched to the horizon, the asset class, the stability of the microstructure, and the governance constraints of the research process.

13.5.1 MLPs as baselines: when they are sufficient

A multilayer perceptron (MLP) is the simplest general-purpose neural architecture: a feedforward network that maps a fixed-dimensional vector $x_t \in \mathbb{R}^d$ to a prediction $\hat{y}_t$. For time series, an MLP typically receives as input a *flattened window* of past values. For example, suppose we have a univariate series of returns r_t . We can construct a window of length L :

$$x_t = [r_{t-L+1}, r_{t-L+2}, \dots, r_t] \in \mathbb{R}^L. \quad (13.51)$$

If we have p features per time step (returns, volume proxy, volatility proxy, etc.), the input becomes a matrix $X_t \in \mathbb{R}^{L \times p}$, which we can flatten into a vector in $\mathbb{R}^{Lp}$. The MLP then computes:

$$\hat{y}_t = f_\theta(x_t), \quad (13.52)$$

where f_θ is a composition of affine maps and nonlinearities as discussed earlier.

What is the inductive bias of an MLP? An MLP, in its basic form, has almost no time-specific structure. It treats the input vector as a generic feature vector. The only time structure comes from how we *construct* x_t . If we order the window from oldest to newest, the MLP can in principle learn that later coordinates are more relevant than earlier ones, but it does not have built-in notions like locality (nearby times matter more) or translation invariance (a pattern shifted in time should be recognized similarly). In other words, the MLP’s inductive bias is: “whatever relationship exists between this d -dimensional vector and the target, learn it.”

This seems weak, but it can be a virtue in finance. When we do not have strong reason to assume a particular structure, a minimal inductive bias can reduce the chance of imposing a false pattern. The cost is that the MLP may require more data to learn certain structures that specialized architectures would learn more efficiently. But in trading, data are often effectively scarce and drifting, so the “data efficiency” advantage of complex architectures can disappear if the learned structure does not persist.

When MLPs are sufficient. MLPs can be sufficient in several common forecasting setups:

1. *Feature-engineered pipelines:* If Chapter 05-style feature engineering already produces informative summary statistics (moving averages, momentum scores, volatility estimates, cross-asset spreads), then the network’s job is mainly to combine these features nonlinearly. An MLP is often enough, and its simplicity reduces the number of degrees of freedom that can be accidentally tuned to noise.
2. *Short windows and low frequency:* For daily data with modest windows (say $L \in [20, 60]$), flattening a window into a vector is not absurd. The network can learn interactions among recent days without requiring heavy sequence modeling.
3. *Small datasets:* If the number of training examples per fold is limited (as it often is once we enforce time splits), an MLP with controlled width and strong regularization may outperform a more complex model simply because it is easier to train stably and less prone to overfit.
4. *Interpretability demands:* While MLPs are not fully interpretable, their simplicity makes them easier to probe with sensitivity tests and ablations. In governance-heavy environments, simpler models are often easier to audit.

MLPs and the “baseline discipline.” In this book, an MLP plays a similar role to a regularized linear model: it is a strong baseline. Before reaching for CNNs, RNNs, or Transformers, we should ask: does a well-regularized MLP, trained with strict time-aware evaluation and early stopping, already capture the stable signal? If yes, then the additional complexity of a specialized architecture is not justified. In trading, the goal is not maximum theoretical expressiveness; it is robust out-of-sample performance under drift and costs. Baselines are not a formality; they are an epistemic control.

13.5.2 1D CNNs for local patterns and multi-scale features

A one-dimensional convolutional neural network (1D CNN) introduces a strong and explicit inductive bias: *locality* and *translation invariance* along the time axis. The key idea is that the same local pattern (a short burst of returns, a volatility spike, a micro-trend) might appear at different times, and we want the model to recognize it with the same parameters.

Convolution on a sequence. Suppose our input is a sequence $x_t \in \mathbb{R}^{L \times p}$, a window of length L with p channels (features). A convolutional filter (kernel) of width k slides along the time axis. For simplicity, consider one channel and one filter. Let the kernel weights be $w \in \mathbb{R}^k$. The convolution output at position j is

$$s_j = \sum_{i=0}^{k-1} w_i x_{j+i}, \quad (13.53)$$

where x_{j+i} refers to the input value at time index $j+i$ within the window. This computes a weighted local sum. With multiple channels, the kernel has shape $k \times p$, and we sum over both time and channels. The same kernel weights are used at every position, which is parameter sharing.

Why this is an inductive bias. Parameter sharing means the model assumes that a pattern learned in one part of the window is relevant elsewhere. This is exactly what “translation invariance” means: a motif shifted in time should be detected similarly. In markets, this can be plausible if the motif corresponds to behavior that repeats (e.g., mean reversion bursts, volatility clustering). But it can also be dangerous if the motif is regime-specific or tied to a particular market era. The CNN will try to reuse the same detector everywhere; if patterns change, this reuse can become a mismatch.

Locality and noise. A second bias is locality: convolution filters look at small neighborhoods. This can be beneficial because it reduces the effective degrees of freedom: instead of connecting every input coordinate to every hidden unit (as in an MLP), the CNN restricts connections to local time neighborhoods. This can reduce overfitting when data are limited. It also matches a common trading belief: short-horizon signals often depend on recent local behavior (e.g., last few returns, last few volatility moves). However, locality can hurt if the signal depends on long-range structure that cannot be captured by stacking filters or using pooling.

Multi-scale features. CNNs become powerful when we stack layers. Early layers learn very local patterns; later layers combine them into larger-scale patterns. A simple way to increase receptive field is pooling or strided convolutions, which reduce resolution and allow deeper layers to aggregate information over longer horizons. Conceptually, this creates multi-scale representations: the model can learn to detect short-term bursts, medium-term drifts, and longer-term regimes (though explicit regimes are deferred to Chapter 14). In finance, multi-scale feature learning is attractive because markets exhibit structure at multiple horizons: intraday

microstructure, daily momentum, monthly carry. But the book’s caution applies: representing multi-scale structure is not the same as proving it is stable and tradable.

When 1D CNNs are a good idea. CNNs are often sensible when:

1. Inputs are *rawer* (e.g., sequences of returns/volatility rather than heavy feature engineering), and we want the model to learn local motifs.
2. The hypothesis involves *patterns that recur* across time windows: repeated oscillations, bursts, or technical shapes that are not easily captured by linear summaries.
3. We want a compromise between an MLP and an RNN/Transformer: CNNs can be easier to train, and their parameter sharing reduces overfitting risk relative to large MLPs on long windows.

CNN pitfalls in trading. CNNs can still fail in characteristic ways:

- They can learn “patterns” that are artifacts of the sampling frequency (e.g., end-of-day effects) that do not persist after market structure changes.
- If normalization is done incorrectly, the convolution can detect leaked distribution shifts.
- CNNs can be overconfident if they see repeated local motifs that are common but not predictive.

The governance stance is therefore the same: strong baselines, strict causality checks, walk-forward stability, and sanity tests.

13.5.3 RNNs and LSTMs (conceptual): sequence memory and limitations

Recurrent neural networks (RNNs) introduce a different inductive bias: *stateful memory*. Instead of treating a window as a fixed vector, an RNN processes the sequence step by step, maintaining a hidden state that summarizes what it has seen so far.

Basic RNN update. Let $x_1, x_2, \dots, x_L$ be inputs in a window (each $x_i \in \mathbb{R}^p$). An RNN maintains a hidden state $h_i \in \mathbb{R}^m$ and updates:

$$h_i = \phi(W_x x_i + W_h h_{i-1} + b), \quad (13.54)$$

with h_0 initialized (often to zero). After processing the window, we produce an output based on the final state:

$$\hat{y} = g(Vh_L + c). \quad (13.55)$$

The same weights W_x, W_h are reused at each time step. This reuse is parameter sharing across time, like convolution, but the mechanism is different: the state evolves.

Inductive bias of RNNs. The RNN assumes that the sequence can be summarized by a state that is updated incrementally. This is a natural bias for time series: the past matters through a compressed memory. In trading, this aligns with the idea that markets have “state” (volatility level, trend strength, liquidity conditions). However, a plain RNN has practical limitations: it can have difficulty learning long-range dependencies because gradients can vanish or explode through time. This is the time-version of the deep network gradient problem: backpropagation through time multiplies derivatives across many steps.

LSTMs and gates. Long Short-Term Memory (LSTM) networks were designed to address long-range dependency issues by introducing gating mechanisms that control what information is stored, forgotten, and output. While the full equations can look intimidating, the concept is simple: instead of forcing the hidden state to be overwritten at each step, an LSTM maintains a “cell state” that can carry information forward with less decay, and gates decide how much to update it. This can make training more stable and allow memory over longer windows.

Why RNNs/LSTMs can be appealing in finance. They can be appealing when:

1. The hypothesis involves *sequential dependencies* where order matters strongly and where simple window summaries may miss relevant temporal structure.
2. We want the model to learn an internal notion of “market state” as it processes the sequence.
3. The window length is large enough that flattening into an MLP is too parameter-heavy.

However, the book’s conservative stance remains: many financial forecasting tasks do not require long memory, and if they do, that memory may be unstable under drift. The elegance of an LSTM does not guarantee a stable edge.

RNN/LSTM limitations in trading practice. Several limitations matter:

- *Training complexity:* They can be harder to train than MLPs or CNNs, requiring careful learning rates and initialization.
- *Latency:* Sequential processing can be slower than convolutions, which can matter in high-frequency settings (though this chapter remains mostly conceptual about latency).
- *Interpretability:* Hidden states are harder to interpret and audit; governance requires more diagnostic work.
- *Non-stationarity:* The learned memory can encode regime-specific patterns that do not transfer.

For these reasons, RNNs are not the default in a governance-heavy trading pipeline; they are chosen only when the hypothesis truly demands sequential memory and when simpler baselines fail.

13.5.4 Transformers (conceptual): attention, context windows, and leakage traps

Transformers are often described as “attention models.” The conceptual leap is that instead of summarizing the past by a single state (as in an RNN), the model allows each position to *attend* to other positions, learning which parts of the past matter for each prediction.

Attention intuition. In a simplified form, attention computes a weighted combination of input representations. For each time position i , the model computes weights over positions j that measure relevance, then forms a context:

$$\text{context}_i = \sum_{j=1}^L a_{ij} v_j, \quad (13.56)$$

where a_{ij} are attention weights (nonnegative and summing to one over j) and v_j are value vectors derived from the input at position j . The weights are learned functions of query and key vectors. The mechanical details are less important than the meaning: the model can learn to focus on particular past events (spikes, breaks, transitions) regardless of their position in the window.

Inductive bias of Transformers. Transformers assume that relationships in sequences may be *non-local*: a pattern at the end of the window may depend on something that happened far earlier. Attention makes it easy to represent such dependencies. This can be useful for financial series where distant events matter (e.g., a large drawdown weeks ago changes risk behavior), but it can also be overkill for many short-horizon tasks.

Transformers also require positional information (because attention is otherwise permutation-invariant). This is handled by positional encodings, which inject information about order. The model then learns both “what happened” and “when it happened.”

The leakage trap: causal masking. The most important caution for Transformers in time series is leakage. Attention, by default, allows each position to attend to all other positions. In forecasting, we must enforce that the model at time t cannot use information from times after t . In language modeling, this is handled by a *causal mask* that prevents attention to future tokens. In market forecasting, a similar mask is mandatory if the model processes sequences that include multiple time steps and makes predictions at intermediate points.

If we construct each training example as a fixed window ending at t and predict y_t (future return after t), then attention within the window is not leakage because the window contains only past information. But leakage can occur in more complex setups:

- If we create sequences that span both training and validation periods and allow attention across the boundary, the model can leak future information.
- If normalization or feature construction uses statistics from beyond t , attention can amplify that leak.

- If we train a model to predict multiple time steps in one sequence and forget causal masking, the model will literally attend to the future.

Because Transformers are powerful, they can exploit small leakage and produce convincing but fake performance. This is why they require especially strict causality gates and careful split manifests.

Context windows and data size. Transformers typically benefit from large datasets because they have many parameters and because attention can model complex relationships. In trading, data size is often insufficient relative to typical Transformer capacity, especially once time-aware evaluation is enforced. This does not mean Transformers cannot work; it means that a Transformer must be sized conservatively and validated aggressively. A small Transformer might be reasonable; a large one may simply be a noise-fitting engine.

Transformers and interpretability. Attention weights can be visualized, which creates an impression of interpretability. In practice, attention maps are not a guarantee of meaningful explanation; they are one diagnostic tool. Still, in a governance context, attention visualizations can help debug whether the model is focusing on sensible parts of the window or whether it is responding to artifacts. The danger is to treat such visualizations as proof rather than as hints.

13.5.5 Choosing an architecture under constraints: data size, latency, interpretability

Given these options, how should a practitioner choose? In this book’s ethos, architecture selection is not a creative act; it is a governed decision constrained by data, objectives, and operational realities. The correct answer is often “simpler than you think.”

Constraint 1: data size and effective sample size. The first constraint is the amount of *reliable* training information. In markets, the effective sample size is reduced by autocorrelation, overlapping labels, and non-stationarity. If we have limited history or if the strategy horizon is long (reducing the number of independent examples), then large architectures are inappropriate. The baseline order is often:

$$\text{Regularized linear} \rightarrow \text{Small MLP} \rightarrow \text{1D CNN} \rightarrow \text{RNN/LSTM} \rightarrow \text{Small Transformer}, \quad (13.57)$$

where moving right requires stronger justification and stronger evidence. This ordering is not dogma; it reflects increasing degrees of freedom and increasing risk of overfit.

Constraint 2: latency and inference cost. Latency matters in execution, though the deep discussion is in Chapter 18. Still, architecture choice affects inference time. MLPs and CNNs can be very fast at inference, especially on fixed windows, because computations are parallel. RNNs process sequentially and can be slower for long windows. Transformers can be costly because attention scales roughly like L^2 in window length (each position attends to each

other). In high-frequency settings, this can be prohibitive. Even in daily systems, inference cost matters if we evaluate many assets or many strategies. A governance-native pipeline should include a simple inference cost estimate in the model registry so that the research result is not operationally impossible.

Constraint 3: interpretability and auditability. In institutional trading, you often need to explain what the model is doing, or at least demonstrate that it is not fragile. MLPs are simpler to probe with perturbation tests. CNNs can be probed by analyzing learned filters and receptive fields. RNNs and Transformers require more sophisticated diagnostics. If your governance environment demands strong audit artifacts and you cannot provide them, the model’s complexity is not justified. In this book, interpretability is not only about explanations; it is about the ability to detect leakage, instability, and regime sensitivity.

Constraint 4: stability under drift. The most subtle constraint is drift. Complex architectures can learn complex patterns that may not persist. A model that relies on delicate long-range dependencies might collapse when the market regime changes. In practice, many trading edges are local and modest. A CNN or even an MLP may capture what is stable without overcommitting to fragile structure. Therefore, the architecture should be chosen not by what maximizes in-sample accuracy, but by what is most likely to remain stable under plausible drift.

A disciplined selection workflow. A practical workflow consistent with Chapters 11–12 is:

1. Define the forecasting target and horizon, and define a causally constructed input representation.
2. Establish strong baselines: regularized linear and a small MLP.
3. If baselines fail and the hypothesis demands local pattern recognition, test a small 1D CNN.
4. If evidence suggests long-range dependencies and CNNs fail, consider an RNN/LSTM, sized conservatively.
5. Consider a small Transformer only if data support it and if causal masking and leakage defenses are explicit and tested.
6. For every step, use walk-forward evaluation, limited tuning budgets, seed robustness, and canary tests.

This workflow is not about preventing innovation; it is about preventing self-deception.

How inductive bias links back to trading hypotheses. Finally, the architecture choice should be legible as a hypothesis:

- An MLP says: “the engineered features already summarize the relevant past; learn a nonlinear combination.”

- A CNN says: “local motifs repeat; detect them with shared filters and build multi-scale features.”
- An RNN/LSTM says: “there is a latent market state that evolves; memory and gating matter.”
- A Transformer says: “non-local dependencies matter; the model must learn which past events are relevant at each point.”

In trading research, making this hypothesis explicit is part of governance. If you cannot state what inductive bias you are buying and why it is plausible in the market context, the architecture choice is not disciplined.

In summary, deep architectures are tools for embedding beliefs about structure in time series. Markets may or may not reward those beliefs. This chapter’s stance is to treat architecture as a governed bet: start simple, escalate only when justified, and demand time-aware evidence that the chosen inductive bias yields stable, auditable improvement over strong baselines.

13.6 Representing Market Information for Neural Nets

Neural networks do not “see” markets. They see arrays of numbers. What separates a disciplined forecasting pipeline from a fragile one is how those arrays are constructed, ordered, scaled, and aligned with labels under the arrow of time. In trading, representation is not a cosmetic choice; it is where most leakage enters and where most false edges are manufactured. This section explains how to represent market information for neural nets in a way that is simple, explicit, and causality-safe, while remaining flexible enough to support multiple architectures (MLPs, CNNs, RNNs, Transformers) without changing the underlying principles.

The guiding idea is continuity with earlier chapters. Chapter 03 established returns and basic risk math as the primitive objects. Chapter 04 emphasized time series anatomy: lags, windows, stationarity concerns, and the non-negotiable nature of time ordering. Chapter 05 treated data pipelines and feature engineering, emphasizing deterministic transforms and auditable lineage. Here, we translate those foundations into the objects neural networks consume: vectors, matrices, and tensors with well-defined shapes and strict alignment rules. The reader should come away with a practical, almost mechanical approach: you should be able to point to any element of your input tensor and say which market observation it corresponds to, and you should be able to point to your label and prove it uses information strictly after the input.

13.6.1 Returns and log-returns as primary inputs (ties to Ch. 03)

The most defensible raw input to a forecasting model is not price, but *return*. Price levels are non-stationary and scale-dependent: the S&P 500 at 5000 and at 2500 are not comparable in raw level, and many models will waste capacity learning that the same absolute move means different things at different levels. Returns, by contrast, are closer to stationary and naturally interpretable as percentage changes.

Let P_t denote the price (or adjusted close) at time t . The simple (arithmetic) return from $t - 1$ to t is

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}. \quad (13.58)$$

The log return is

$$\ell_t = \log\left(\frac{P_t}{P_{t-1}}\right) = \log(P_t) - \log(P_{t-1}). \quad (13.59)$$

For small moves, $\ell_t \approx r_t$, but log returns have convenient additive properties across time:

$$\log\left(\frac{P_{t+h}}{P_t}\right) = \sum_{j=1}^h \ell_{t+j}. \quad (13.60)$$

This additivity is useful when defining multi-step horizons. It also aligns with certain modeling assumptions (e.g., multiplicative price evolution).

Which should you use? Within this chapter’s scope, the choice is mostly pragmatic:

- Use simple returns when you want direct interpretability in percent terms and when moves are small.
- Use log returns when you want horizon additivity and when you may have larger moves or want symmetry for up/down moves.

Either is acceptable, but the key is consistency. Once chosen, it must be recorded in the data fingerprint and preprocessing manifest.

Returns as inputs versus returns as labels. Neural nets can use returns as both inputs and outputs (labels). A common pattern is:

$$x_t = \text{a window of past returns (and other features) up to time } t, \quad (13.61)$$

$$y_t = r_{t \rightarrow t+h} \text{ (a future return)}. \quad (13.62)$$

This sounds simple, but it already embeds two critical design choices:

1. What does “past” mean (how many lags, which frequency, which sampling convention)?
2. What does “future” mean (what horizon, what overlap, what alignment with decision time)?

The rest of this section is largely about making those choices explicit and safe.

Why not raw prices? It is not forbidden to include price-related information, but it should enter as *relative* information: returns, log prices (differences), normalized distances to moving averages, z-scores computed causally, or other stationary-ish transforms. If you do include raw prices, you must explain why (e.g., the model is learning level-dependent behavior), and you

must ensure scaling is handled correctly to avoid instability and leakage. In general, for this book’s pedagogy and governance focus, returns (and transforms derived from them) are the default primitive.

13.6.2 Windowed tensors: shape conventions and time ordering (ties to Ch. 04–05)

Neural nets operate on arrays. The most common representation for time series forecasting is a *windowed tensor*. We choose a window length L and, for each decision time t , we construct an input object containing the last L observations of each feature.

From sequences to tensors. Suppose we have p features at each time: returns, rolling volatility proxy, volume change, etc. Let $u_t \in \mathbb{R}^p$ denote the feature vector at time t (constructed causally). Then a windowed input at time t is

$$X_t = \begin{bmatrix} u_{t-L+1}^\top \\ u_{t-L+2}^\top \\ \vdots \\ u_t^\top \end{bmatrix} \in \mathbb{R}^{L \times p}. \quad (13.63)$$

This matrix is the canonical representation for CNNs, RNNs, and Transformers. For an MLP, one often flattens it:

$$x_t = \text{vec}(X_t) \in \mathbb{R}^{Lp}. \quad (13.64)$$

The most important convention: order. You must choose whether the first row corresponds to the oldest observation or the newest. The safest convention is:

- Row 1 is the oldest ($t - L + 1$), row L is the newest (t).

This matches the natural reading order and the typical input order for sequence models. The choice is not mathematically mandatory, but it must be consistent across training and inference. Many leakage bugs arise from off-by-one shifts combined with reversed ordering.

Batch dimension. In training, we rarely process one X_t at a time. We create batches. If we have N training examples (times), we stack them into a 3D tensor:

$$\mathcal{X} \in \mathbb{R}^{N \times L \times p}, \quad (13.65)$$

where $\mathcal{X}[i, :, :]$ corresponds to the window for the i -th training example. The corresponding labels form a vector (for regression) $\dagger \in \mathbb{R}^N$ or a matrix for multi-output tasks.

Explicit index mapping. A governance-native pipeline should keep an explicit mapping between example index i and time index t_i . That mapping is part of the split manifest. In code,

you should be able to assert:

the last row of X_t corresponds to time t and uses only information up to t . (13.66)

This is not rhetorical; it should be enforceable by deterministic construction and hard assertions.

Overlapping windows and dependence. Windowed inputs overlap heavily: X_t and X_{t+1} share $L - 1$ rows. This is not a bug; it is how sliding windows work. But it reduces effective sample size and creates dependence between training examples. In a purely iid worldview, this would be troublesome; in time series forecasting, it is expected. The key is to ensure that train/validation splits respect time so that overlapping windows do not cross the boundary. If a window in validation contains rows that were also used in training, that is not leakage by itself (the same past observation can appear in both) as long as the boundary is defined by time and the model is not seeing future labels. However, overlapping *labels* can be a more serious leakage risk, discussed below.

Multi-frequency features. Sometimes features come at different frequencies (daily prices with weekly macro indicators). Within this book's scope, we keep it simple: resample to a common decision grid (Chapter 05) and hold values forward if necessary, but always record the alignment rule. The rule is: at decision time t , only include macro values that were published by time t . If publication delays are ignored, leakage becomes likely. Event timing is treated more deeply in later chapters, but even here, the representation must respect availability.

13.6.3 Scaling and normalization: in-sample only, then apply forward (causality gate)

Normalization is simultaneously essential and dangerous. It is essential because neural networks train better when inputs have comparable scales. It is dangerous because computing normalization statistics on the full dataset leaks future information. In trading, that leak can be subtle: even if the model never sees future labels, it can see future *distributional information* through means and variances computed using future data. Deep models can exploit that.

A simple normalization scheme. A common approach is standardization:

$$\tilde{x} = \frac{x - \mu}{\sigma}, \quad (13.67)$$

where μ and σ are mean and standard deviation estimated from data.

The causality gate. The rule is strict:

Compute μ and σ using training data only, then apply the same μ and σ to transform validation and test data.

If you use μ and σ computed on all data, you leak. If you compute them separately on validation or test, you leak in a different way (you allow the model to see future distribution properties at inference). The only acceptable approach is: fit on training, apply forward.

In a walk-forward setting, this is done per fold: each fold has its own training period and therefore its own normalization statistics. Those statistics must be logged as artifacts (means, standard deviations, and the exact feature definitions they correspond to). This is not bureaucracy; it is the only way to prove causality compliance.

Normalization and windowed tensors. With windowed tensors, we must decide what we normalize:

1. Normalize each feature channel across all times in the training set (global per-feature normalization).
2. Normalize per window (e.g., subtract the window mean).

Global per-feature normalization is easier to govern because it clearly separates training-fitted parameters from forward application. Per-window normalization can be legitimate (for example, centering each window), but it must be handled carefully: the window contains only past information, so in principle it is causal. However, per-window normalization changes the meaning of the features and can remove level information that might be predictive. It also changes the distribution across windows in ways that can interact with drift. Within this chapter’s pedagogical scope, the recommended default is global per-feature normalization fitted on training only, because it is easier to audit and reason about.

Robust scaling. Because returns can have outliers, one might prefer robust scaling (median and interquartile range) or clipping. These are not forbidden. The governance requirement is that any such transformation must be:

- fitted on training only (if it estimates parameters),
- applied forward consistently,
- recorded in the preprocessing manifest,
- treated as a hyperparameter choice if multiple variants are tried.

Why normalization leakage is economically meaningful. A skeptical reader might ask: “How can knowing future mean and variance help predict future returns?” The answer is that in non-stationary environments, future means and variances contain information about the regime and volatility state. If the model sees normalization statistics computed with future data, it can infer whether volatility later increases, whether the distribution later shifts, and so on. That information is not available at decision time and therefore constitutes leakage. Even if the predictive gain seems small, it can produce a statistically significant improvement over many observations, creating a false edge.

A simple sanity test. A governance-native pipeline should include a test: if you intentionally compute normalization on full data and compare results, performance should improve suspiciously. If it does, that is proof your pipeline is sensitive to leakage and that your causality gates matter.

Conversely, if you correct the normalization and performance collapses, that is a warning that the earlier “edge” was likely leakage.

13.6.4 Label design: horizons, overlapping labels, and leakage risks

Label design is where supervised learning becomes trading-specific. The same input representation can be paired with many label definitions, and each definition implies a different economic and statistical interpretation. The key is to define labels so that they correspond to a decision problem and do not introduce leakage through overlap or timing confusion.

Forecast horizon. Let the horizon be h steps. For daily data, $h = 1$ is next-day return; $h = 5$ is one-week return. A simple label is

$$y_t = r_{t \rightarrow t+h} = \frac{P_{t+h} - P_t}{P_t}. \quad (13.68)$$

If using log returns, one might define

$$y_t = \log \left(\frac{P_{t+h}}{P_t} \right) = \sum_{j=1}^h \ell_{t+j}. \quad (13.69)$$

The crucial point is that y_t uses prices after t , while x_t uses information up to t .

Decision time versus label time. In a realistic trading pipeline, one must be explicit about when the decision is made. For example, if features are computed using the close price at day t , then the earliest you can trade based on them is after the close (or at next open), depending on your execution assumptions. Chapter 06 deals with simulation mechanics, but even here, label timing must be consistent: if you pretend you can trade at the same close you used to compute features, you have implicit look-ahead. Therefore, label definition should reflect an implementable decision rule. A safe convention is:

- Features use data up to close of t .
- Label is return from close of t to close of $t + h$ (or from open of $t + 1$ onward, depending on the chosen convention).

Whatever convention is chosen must be recorded and used consistently.

Overlapping labels. If $h > 1$, labels overlap. For $h = 5$, y_t uses returns from days $t + 1$ to $t + 5$, and y_{t+1} uses returns from $t + 2$ to $t + 6$. This overlap means the label series is strongly autocorrelated and adjacent labels share information. This has several implications:

1. Effective sample size decreases.
2. Validation can be contaminated if training labels overlap with validation labels across a boundary.

The second point is often overlooked. Suppose training ends at time τ and validation begins at $\tau + 1$. If $h = 5$, then the last few training labels $y_{\tau-3}, y_{\tau-2}, y_{\tau-1}, y_{\tau}$ use future prices that lie in the validation period. That is not necessarily leakage if the model is trained only on training data and evaluated on validation later, but it creates a dependence between training labels and validation outcomes that can inflate apparent stability and reduce the strictness of the separation. This is why Chapter 11 and Chapter 12 often mention *embargo* ideas: one can leave a gap between training and validation to avoid overlap. Within this chapter's scope, the key is to acknowledge the overlap and treat it as a design choice with consequences.

Label leakage through feature construction. A more direct leakage risk is accidental inclusion of future returns in the input. Common mistakes include:

- Using r_{t+1} inside x_t due to an off-by-one shift.
- Computing rolling statistics that include the current or future period when you intended a lagged window.
- Computing targets with the wrong anchor time (e.g., using P_{t+h} in features).

Because neural nets are powerful, they can exploit a single leaked component and produce dramatically better performance. This is why label alignment should be tested with assertions. For each example time t , you should be able to assert:

$$\max\{\text{time indices used in } x_t\} \leq t \quad \text{and} \quad \min\{\text{time indices used in } y_t\} \geq t + 1. \quad (13.70)$$

This is the causality gate in symbolic form.

Alternative labels: direction, quantiles, and volatility-scaled returns. Labels can be transformed. Direction labels are

$$y_t = \mathbb{I}\{r_{t \rightarrow t+h} > 0\}. \quad (13.71)$$

Quantile labels might be used for tail prediction. Volatility-scaled returns might be

$$y_t = \frac{r_{t \rightarrow t+h}}{\hat{\sigma}_t}, \quad (13.72)$$

where $\hat{\sigma}_t$ is a volatility estimate computed from past data only. This scaling can stabilize training and align the model with risk-adjusted forecasting. But the volatility estimate must be computed causally and logged, and the choice to scale or not scale is a modeling decision that should be validated.

Label choice is part of the claim. If the model predicts scaled returns, the claim is about risk-adjusted predictability. If it predicts direction, the claim is about sign predictability. If it predicts ranks, the claim is about cross-sectional ordering. The label is therefore part of the scientific statement, and it must be included in the estimator manifest.

13.6.5 Cross-sectional vs time-series setups (preview only; factors in Ch. 09)

Thus far we have described a time-series setup: for a given asset, at each time t we predict that asset's future return. Many trading problems, however, are cross-sectional: at each time t , we have many assets $a \in \{1, \dots, A\}$ and we want to rank them or allocate across them. Chapter 09 formally develops factor and cross-sectional trading. Here we only preview how representation changes.

Time-series setup (single asset). We have examples indexed by time:

$$(x_t, y_t), \quad t = L, L+1, \dots, T-h. \quad (13.73)$$

The model learns a mapping from the past of that asset to its future.

Cross-sectional setup (asset-time pairs). We have examples indexed by both asset and time:

$$(x_{a,t}, y_{a,t}), \quad a = 1, \dots, A, \quad t \in \mathcal{T}. \quad (13.74)$$

Here $x_{a,t}$ could include asset-specific features (past returns of asset a) and cross-asset or macro features shared across assets. The label $y_{a,t}$ is the future return of asset a over horizon h .

This changes representation in two ways:

1. We may want to include *cross-sectional normalization* at each time t (e.g., z-scoring signals across assets). This is a powerful tool but a potential leakage trap if done incorrectly. If the z-score uses information from assets whose data are missing or stale, or if it inadvertently uses future updates, the ranking can be contaminated. In a governance-native pipeline, cross-sectional scaling must be done at time t using only information available at t .
2. We may want the model to output *scores* that are comparable across assets so that ranks are meaningful. This can favor ranking losses or calibration strategies.

A simple cross-sectional tensor view. At a fixed time t , one can represent all assets' features as a matrix:

$$X_t^{\text{cs}} \in \mathbb{R}^{A \times d}, \quad (13.75)$$

where each row is the feature vector for an asset. Over time, this becomes a tensor $T \times A \times d$. Models can be trained on asset-time pairs, often with batches that group examples by time so that ranking losses can be computed within a time slice. This is a preview; Chapter 09 formalizes the cross-sectional setting and portfolio formation.

Why we do not fully develop it here. Cross-sectional modeling introduces additional design layers: missingness patterns across assets, survivorship bias, corporate actions, and portfolio

constraints. Those topics belong to the factor and portfolio chapters. Here, the key takeaway is that the same representation principles still apply: strict time ordering, causality-safe scaling, explicit shape conventions, and auditable alignment of inputs and labels.

The unifying principle. Whether time-series or cross-sectional, the representation discipline is the same:

- choose primitives that are as stationary as possible (returns/log returns);
- construct windowed tensors with explicit, documented shape and ordering;
- fit any scaling parameters on training only and apply forward;
- design labels with explicit horizons and decision-time conventions;
- enforce causality with hard checks, not hope.

If you adopt this discipline, neural architectures become interchangeable modules consuming well-defined tensors. If you do not adopt it, the most sophisticated architecture will become a machine for optimizing leakage and overfitting. In trading, representation is the first and most important line of defense against that outcome.

13.7 Time-Aware Validation and Model Selection for Deep Nets

Deep learning increases the size of the function class and the number of modeling decisions. In trading, that combination is dangerous unless evaluation and selection are designed to respect time, to limit researcher degrees of freedom, and to make leakage mechanically difficult. Chapter 11 established the time-aware forecasting frame and Chapter 12 established the model selection discipline under time. This section applies those same principles to deep nets, but with two key differences. First, the tuning surface in deep learning is much larger: architecture, optimizer, learning rates, batch sizes, regularization, and early stopping are all hyperparameters. Second, deep models can exploit tiny forms of leakage or preprocessing mistakes more aggressively than linear models, producing results that look plausible but are not tradable.

The objective is therefore not to present a menu of validation tricks. It is to specify an evaluation regime that is conservative, auditable, and robust against self-deception. A deep net may be worth deploying only if it survives (i) walk-forward evaluation, (ii) tuning discipline that limits selection bias, (iii) metrics that are appropriate to the trading claim, (iv) sanity checks that detect pipelines capable of manufacturing predictability, and (v) explicit defenses against deep-learning-specific leakage traps.

13.7.1 Walk-forward evaluation and embargo logic (conceptual; mechanics in notebooks)

Why random splits are wrong for time series. In iid machine learning, we often split data randomly into training and test sets. In trading, that breaks the arrow of time. If we train on future observations and test on earlier ones, we are implicitly allowing information from the

future to influence the model. Even if the target is defined correctly, random splitting mixes regimes and makes evaluation optimistic: the model can learn patterns tied to a later regime and appear to generalize, but the trader could not have learned those patterns at the earlier date. Therefore, evaluation must be chronological.

The walk-forward idea. Walk-forward evaluation (also called rolling or expanding window evaluation) is a simple concept: we choose a sequence of cutoffs, and for each cutoff we train on a past window and test on a future window. Let time indices run from 1 to T . Choose folds indexed by $k = 1, \dots, K$, each with:

$$\text{Train period: } [1, \tau_k] \quad (\text{expanding}) \quad \text{or} \quad [\tau_k - W + 1, \tau_k] \quad (\text{rolling}), \quad (13.76)$$

and

$$\text{Test period: } [\tau_k + 1, \tau_k + H], \quad (13.77)$$

where W is a fixed training window length (rolling case) and H is the test horizon length. The model is fitted using only training data and then evaluated on the immediately subsequent future block. Then we move forward and repeat. This produces multiple out-of-sample evaluations across different market periods.

The conceptual payoff is enormous: instead of claiming “the model works” based on one split, we observe how it behaves across time. In non-stationary markets, that behavior is often the most important result.

Embargo logic: why we sometimes need a gap. A subtle problem arises when labels overlap across time. Suppose the label at time t uses future returns over horizon h :

$$y_t = r_{t \rightarrow t+h}. \quad (13.78)$$

If we define a train/test boundary at τ , then training labels near the end of training (e.g., $y_{\tau-h+1}, \dots, y_\tau$) use prices in the test period. This does not mean the model explicitly saw test labels, but it creates dependency between training and test outcomes and can make the separation less strict. In addition, certain feature engineering steps (rolling volatility estimates, target scaling, etc.) can inadvertently couple training and test if the window straddles the boundary.

Embargo logic introduces a gap between training and test:

$$\text{Train: } [1, \tau] \quad \text{Embargo: } [\tau + 1, \tau + E] \quad \text{Test: } [\tau + E + 1, \tau + E + H], \quad (13.79)$$

where E is the embargo length, often chosen to be at least the label horizon h (or more if additional overlap exists). The embargo removes observations around the boundary, reducing leakage and dependence.

The word “embargo” can feel bureaucratic, but the intuition is simple: if the model’s training targets depend on future prices, and we want to evaluate on a future block, we should ensure

that the last training targets do not look into the evaluation block. Removing a small region is often worth the reduction in bias.

Expanding versus rolling training windows. An expanding window uses all history up to τ_k . A rolling window uses only the most recent W observations. Expanding windows have more data and can reduce estimation variance, but can also make the model learn outdated patterns that no longer apply. Rolling windows adapt faster to regime change but have fewer samples and can increase variance. In deep learning, rolling windows can be especially challenging because deep nets often require data volume to train stably. Therefore, the choice is a trade-off:

- Expanding window: more data, potentially more stable optimization, but higher risk of learning stale relationships.
- Rolling window: more adaptive, but can yield unstable training and seed sensitivity if W is small.

In a governance-native workflow, you should treat window choice as part of the experiment design, record it, and avoid changing it opportunistically.

Fixed test blocks and the meaning of “out-of-sample.” In walk-forward evaluation, each fold has its own out-of-sample test block. The model is retrained in each fold. This matches trading reality: in production, we would retrain periodically. The evaluation therefore tests the entire pipeline: feature construction, scaling, training, and inference. This is why walk-forward evaluation is more meaningful than a single holdout split.

What mechanics belong in notebooks. This chapter treats walk-forward and embargo conceptually; the notebook will implement:

- explicit fold construction with date indices;
- per-fold scaling fit on training only;
- per-fold training with deterministic seeds;
- per-fold early stopping on a validation subset inside training;
- per-fold out-of-sample evaluation and aggregation of metrics.

The conceptual requirement here is that the reader understands *why* those mechanics exist: they are defenses against time leakage and non-stationarity.

13.7.2 Nested tuning discipline: what we keep from Ch. 12 and what changes here

Chapter 12’s main lesson is that hyperparameter tuning is not free. Each choice is a statistical bet, and exploring many choices without proper accounting creates selection bias. Deep learning multiplies the number of choices, so the discipline must become stricter, not looser.

What we keep from Chapter 12. We keep three principles:

1. *Separation of roles:* training is used to fit parameters; validation is used to choose hyperparameters; test is used for final evaluation.
2. *Time-aware splits:* all splits preserve chronology.
3. *Budgeted search:* we declare a tuning budget and avoid infinite exploration.

These are non-negotiable.

What changes in deep learning. The tuning space changes shape. In linear models, hyperparameters might be a small set (e.g., λ for ridge/LASSO). In deep learning, hyperparameters include:

architecture (depth, width), activation, optimizer, η (learning rate), b (batch size),
(13.80)

weight decay, dropout, learning-rate schedule, early stopping patience, normalization choice.
(13.81)

The problem is not only that there are many hyperparameters; it is that they interact. A good learning rate depends on batch size and normalization; a good architecture depends on regularization and data size. Therefore, naive grid search becomes expensive and can create huge selection bias.

Nested evaluation in time. A conservative approach is nested tuning: within each walk-forward fold, we create an internal split of training into sub-training and validation. Hyperparameters are chosen on that internal validation, and only then the model is evaluated on the fold's test block. Conceptually:

Outer loop: walk-forward folds produce out-of-sample tests, (13.82)

Inner loop: time-aware validation inside training chooses hyperparameters. (13.83)

The outer test is never touched by hyperparameter decisions. This is exactly the logic of nested cross-validation, adapted to time.

Practical simplification: limited candidate sets. Because deep learning can become unmanageably complex, the book's practical recommendation is to restrict the search space sharply. Instead of searching over dozens of architectures and schedules, choose a small family of candidates grounded in hypothesis:

- one small MLP baseline (fixed depth and width);
- one modest CNN candidate (fixed filter widths);
- optionally one modest RNN or Transformer if justified.

Then tune only a few sensitive hyperparameters (learning rate, weight decay, dropout) within a narrow range. The goal is not to find the global best; it is to find a robust estimator that generalizes across time. In trading, robustness is often worth more than a small metric gain.

Seed discipline as part of tuning. In deep learning, randomness matters: initialization and mini-batch order can change outcomes. A fragile model might look good on one seed and bad on others. Therefore, seed robustness should be treated like a hyperparameter check. For each candidate, we should evaluate performance across multiple seeds and consider the mean and dispersion. A model that has slightly lower mean performance but much smaller variance may be preferable. Importantly, we must not select the best seed; we must average over seeds or fix a small set of seeds and report all results.

Early stopping is a tuned procedure. Early stopping introduces another choice: how long to train and when to stop. Because early stopping uses validation signals, it contributes to selection bias if tuned aggressively. Therefore, the early stopping rule (patience, improvement threshold) should be fixed across candidates or tuned only within the inner loop, not based on test performance.

What “model selection” means in trading. In many ML contexts, model selection means choose the best validation score. In trading, the correct meaning is: choose the model that demonstrates stable performance across time folds, with plausible behavior, and with minimal evidence of overfit or leakage. This is a stronger criterion than “best metric.”

13.7.3 Metrics: MSE, sign accuracy, IC, rank IC, and calibration

Metrics are how we translate model outputs into evidence. In trading, metric choice is not neutral. It determines what kind of predictability we are claiming. A disciplined pipeline often uses multiple metrics, but the key is to interpret them correctly.

Mean squared error (MSE). For regression forecasts of returns, MSE is:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2. \quad (13.84)$$

MSE measures the average squared deviation. Minimizing MSE corresponds to estimating conditional mean under standard assumptions. In trading, MSE can be dominated by rare large moves and may not correlate with economic value. A model can have slightly better MSE but no improvement in directional usefulness or ranking ability. Therefore, MSE is a basic statistical metric, not a trading verdict.

Sign accuracy. If the model outputs a real-valued forecast, we can assess whether it gets the sign right:

$$\text{SignAcc} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}\{\text{sign}(\hat{y}_i) = \text{sign}(y_i)\}. \quad (13.85)$$

Sign accuracy is intuitive for directional strategies. But it can be misleading: if returns are close to zero, sign labels are noisy and sign accuracy can be near 50% even for useful ranking signals. Moreover, sign accuracy ignores magnitude. In practice, sign accuracy is best used as a secondary diagnostic, not as the sole metric.

Information coefficient (IC). The information coefficient is the correlation between model scores and future returns. If $\hat{s}_i$ are model scores (they can be forecasts) and y_i are realized returns, then:

$$\text{IC} = \text{corr}(\hat{s}, y). \quad (13.86)$$

A positive IC indicates that higher scores tend to correspond to higher future returns. In cross-sectional finance, IC is a foundational metric because portfolios are built by ranking. In a time-series setting, IC can still be used as a measure of linear association, but it must be interpreted carefully because time dependence affects significance. The key idea is that IC captures *monotonic association* rather than point accuracy. A model with poor MSE can still have a useful IC if it ranks outcomes correctly.

Rank IC. Rank IC is correlation between ranks, often Spearman correlation:

$$\text{RankIC} = \text{corr}(\text{rank}(\hat{s}), \text{rank}(y)). \quad (13.87)$$

Rank IC is more robust to outliers and focuses on ordering. This is often closer to the economic objective of selecting winners and losers. In deep learning, rank IC can be an important diagnostic because the model may be learning a ranking signal rather than a precise magnitude signal.

Calibration. Calibration asks: do predicted probabilities correspond to realized frequencies? If the model outputs $\hat{p}_i$ as the probability of a positive return, then calibration means:

$$\mathbb{P}(y = 1 \mid \hat{p} = 0.7) \approx 0.7. \quad (13.88)$$

Calibration matters when we use probabilities to size positions or manage risk. A model can have good classification accuracy but poor calibration (overconfident predictions). Calibration can be assessed by binning predictions and comparing predicted versus realized frequencies, or by using calibration error measures. In trading, calibration is often more important than raw accuracy because position sizing depends on confidence.

Metric choice depends on output type. A regression model naturally yields MSE and correlation-based metrics. A classifier yields accuracy and calibration metrics. A ranking model

yields IC and rank IC. One of the most common deep learning mistakes in trading is to choose a loss aligned with one metric and then report a different metric without understanding the gap. For example, training with MSE but reporting sign accuracy can be fine, but you must recognize that the optimizer was not directly targeting sign. Conversely, training with cross-entropy but reporting IC requires understanding how probability scores relate to returns.

Fold-wise reporting and stability. In time-aware evaluation, metrics must be reported per fold and aggregated. The distribution across folds is itself an important result. A model that has a high average IC but wildly varying fold IC may be regime-sensitive. In trading, that volatility can matter more than the mean.

13.7.4 Sanity checks: random labels, permuted features, and “canary” tests

Deep learning pipelines can produce impressive results even when there is no signal, simply by exploiting leakage or by benefiting from selection bias. Sanity checks are designed to answer a blunt question: *can this pipeline manufacture predictability under conditions where no predictability should exist?* If the answer is yes, then any apparent edge is suspect.

Random labels test. Replace y_i with random noise (or shuffle labels across time) while keeping x_i unchanged. Train the model as usual. The out-of-sample performance should collapse to noise levels: MSE should be high, accuracy near 50% for balanced labels, IC near zero. If the model still performs well, something is wrong. Either leakage exists (the features contain information about the original labels or about time), or evaluation is flawed (train/test separation is violated). Random label tests are the simplest canary.

Permuted features test. Shuffle one feature across time (or across samples) while keeping labels unchanged. If that feature was not predictive, performance should not change much. If that feature was predictive, performance should degrade. This is a diagnostic tool: it tests whether the model relies on plausible features. It also detects pipelines that inadvertently encode the label in a feature (a shift error). If permuting a feature destroys performance completely, and that feature is something like “future return,” you have discovered leakage.

Time reversal test. Train the model on time-reversed inputs (reverse the order of each window). If the model’s performance remains unchanged, it may indicate that the model is not using temporal order meaningfully. Sometimes that is fine; sometimes it suggests the model is capturing distributional quirks rather than sequential structure. This test is not a universal rule, but it can reveal whether sequence architectures are actually leveraging order.

Leakage canary: intentional leak. A powerful sanity check is to intentionally introduce a small leak (for example, include r_{t+1} inside x_t) and observe performance. Performance should jump dramatically. This confirms that the architecture and pipeline are capable of exploiting leaked information. That is not a problem; it is a reality. The point is then to show that your causality gates prevent such leaks in the real pipeline. In governance terms, this is like a

penetration test: you verify that the system would be vulnerable if the guardrail were removed, and then you verify that the guardrail works.

Stability across seeds as a sanity check. Run the same configuration on multiple seeds. A legitimate signal should produce similar results, within noise. If results vary wildly, the model is unstable. Instability can arise from small data, high learning rate, or weak signal. In any case, instability is a warning against strong claims.

Why sanity checks are non-negotiable in deep learning. Because deep nets are universal approximators, they can fit almost anything. Without sanity checks, the risk of fooling oneself is high. The checks do not prove that a model is correct, but they can disprove that an apparent edge is trustworthy.

13.7.5 Common deep-learning traps: implicit look-ahead via normalization and batching

Most deep-learning failures in trading are not failures of the architecture; they are failures of the pipeline. The most common trap is implicit look-ahead introduced through preprocessing, batching, or training procedures that quietly violate time separation.

Normalization leakage. We discussed scaling earlier; here we emphasize the evaluation consequence. If you compute normalization statistics using the entire dataset, you leak future distribution information. Deep nets can use that to infer regime shifts. The performance gain can look modest but statistically significant. In a walk-forward setup, correct practice is: fit scaling on training only, apply to validation and test using the same parameters, per fold. Any deviation is a leak.

Batching across boundaries. In some implementations, one constructs a large dataset tensor and then uses a data loader that shuffles and batches samples. If train and validation indices are not separated correctly, batches can mix samples from different periods. Even if the model is not explicitly trained on validation labels, mixing can occur through normalization layers that compute batch statistics (for example, batch normalization). If a batch contains both training and validation samples, the batch statistics incorporate validation distribution information, leaking it into training. This is a subtle but real issue. The governance fix is simple: never batch across boundaries; ensure data loaders are constructed separately for train and validation; and if using batch-dependent normalization, understand its implications.

Overlapping windows crossing the split. If you construct windows naively, a window for a validation sample might include time points that were used in training windows. That is not necessarily leakage, but if the overlap crosses the boundary in a way that includes data after the training cutoff, it becomes leakage. The key is to define windows with explicit time indices and ensure that every element of x_t satisfies the causality rule. For each fold, the latest time index in training inputs must be $\leq \tau_k$; the earliest time index in test labels must be $\geq \tau_k + 1$ (or plus embargo). Hard assertions should enforce this.

Early stopping on the test set (forbidden). A common mistake is to use the test set as the validation set for early stopping. This contaminates the test evaluation because the model is indirectly tuned to test performance. In deep learning, this can produce large bias because early stopping is a powerful selection mechanism. The correct practice is to use a validation set inside training, and reserve test for final evaluation.

Hyperparameter tuning on test performance (forbidden). Similarly, it is tempting to try many learning rates and architectures and then report the one with the best test performance. This is selection bias. Chapter 12 warned against it; deep learning amplifies it. The correct practice is to tune on validation inside each fold or on a separate development period, then lock the configuration and evaluate on test.

Data snooping via repeated backtests. Even with correct splits, repeatedly running many experiments and selecting the best is a form of data snooping. In trading, this is common because researchers iterate. Governance requires logging experiments, limiting tuning budgets, and being honest about how many attempts were made. Deep learning makes iteration easy and therefore makes snooping more likely.

A practical checklist for avoiding traps. Within the chapter’s scope, the key defenses are:

1. Explicit fold construction by date, with recorded boundaries.
2. Optional embargo to reduce label overlap leakage.
3. Per-fold preprocessing: fit on train only, apply forward.
4. Separate data loaders for train and validation; no batching across boundaries.
5. Inner-loop validation for hyperparameters and early stopping; outer-loop test untouched.
6. Sanity checks: random labels, feature permutation, intentional leak tests.
7. Seed robustness reporting: mean and dispersion across seeds.

These steps are not “best practices” in the abstract; they are necessary conditions for making any credible deep-learning claim in trading.

The core message. Deep nets do not fail because they are too fancy. They fail because the research process becomes too permissive. Time-aware validation and disciplined model selection are the barriers that keep deep learning from becoming a sophisticated overfitting engine. If a model survives walk-forward evaluation with embargo where needed, if it is selected under nested tuning discipline with explicit budgets, if its performance is expressed in appropriate metrics and stable across folds and seeds, if it passes sanity checks designed to detect leakage, and if it avoids implicit look-ahead in normalization and batching, then we have a basis for cautious belief. Without those conditions, impressive results are likely to be mirages.

This section therefore establishes a minimum scientific standard for deep nets in trading: the standard is not “high accuracy” but “credible evidence under time and governance.” In the

remainder of the chapter, we build on this standard to discuss regularization mechanisms specific to deep learning, interpretability diagnostics, and the governance-native artifact set required to make the full pipeline auditable.

13.8 Regularization for Neural Nets

Regularization is the set of techniques that restrict a model’s effective complexity so that it generalizes beyond the training sample. Chapter 12 introduced regularization in the linear and generalized linear world (ridge, LASSO, elastic net) and emphasized a non-negotiable principle: in trading, where labels are noisy and regimes drift, regularization is not an optional performance enhancer; it is an epistemic safety device. Neural networks dramatically increase representational capacity, so the need for regularization becomes more acute. But regularization in deep learning also becomes more subtle. The same technique that stabilizes training and improves out-of-sample performance in one regime can degrade performance in another. Some techniques change not only how complex the fitted function is, but also how optimization behaves; this makes the tuning problem more intertwined.

The purpose of this section is to give a simple, operational view of deep regularization mechanisms and to emphasize two ideas. First, “regularization” is not one knob; it is a bundle of interacting design choices. Second, “more regularization” is not a monotone good: beyond a point it can underfit, distort calibrations, or remove the very signal we are trying to capture. The disciplined stance of Chapter 12 therefore carries forward: regularization choices must be validated under time, documented, and treated as part of a controlled tuning budget.

13.8.1 Weight decay (L2), dropout, and why “more regularization” is not monotone

Weight decay as L2 regularization. Weight decay is the deep-learning analogue of ridge regression. Suppose the training objective without regularization is

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_{\theta}(x_i), y_i). \quad (13.89)$$

Weight decay adds an L2 penalty:

$$J_{\lambda}(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_{\theta}(x_i), y_i) + \lambda \|\theta\|_2^2, \quad (13.90)$$

where $\lambda \geq 0$ controls penalty strength. The L2 norm $\|\theta\|_2^2$ is the sum of squares of parameters. In practice, one often applies weight decay to weights but not to biases, and sometimes not to normalization parameters, but conceptually it is an L2 shrinkage.

The intuition is simple: smaller weights usually imply a smoother function and reduce sensitivity to noise. In a trading context, where the signal is weak, this can prevent the network from fitting spurious micro-patterns. It can also improve stability across seeds and folds, which is often as important as improving the mean metric.

Weight decay changes optimization. Weight decay is sometimes described purely as a complexity penalty, but it also changes the optimization trajectory. In gradient descent terms, the update becomes:

$$\theta \leftarrow \theta - \eta (\nabla_{\theta} J(\theta) + 2\lambda\theta). \quad (13.91)$$

The additional $2\lambda\theta$ term pulls parameters toward zero at every step. This can stabilize training by preventing runaway weight growth, especially with high learning rates. But it can also slow learning if λ is too large. In deep nets, where optimization is delicate, this coupling matters.

Dropout as stochastic regularization. Dropout is a different idea. During training, we randomly zero out a fraction of hidden units (or inputs) with probability p . If h is a hidden activation vector, dropout produces:

$$\tilde{h} = m \odot h, \quad (13.92)$$

where m is a random mask with entries $m_j \in \{0, 1\}$, $\mathbb{P}(m_j = 0) = p$, and $\odot$ is elementwise multiplication. The network is forced to make predictions even when some intermediate features are missing. At inference time, we use the full network, often with a scaling factor so that expected activations match training.

The intuition is that dropout discourages fragile co-adaptations: the network cannot rely on a single pathway or a single hidden unit always being present, so it must distribute information across multiple pathways. This is often described as training an implicit ensemble of subnetworks. In finance, where patterns can be unstable, this robustness can reduce overfitting.

Why “more regularization” is not monotone. It is tempting to think: if some regularization helps, more must help more. In trading, this is rarely true. There are at least four reasons:

1. *Underfitting:* If λ is too large or dropout p is too high, the model cannot represent even the weak signal that exists. Training and validation performance both degrade.
2. *Signal distortion:* Some signals are small but systematic and may require the model to use consistent combinations of features. Excessive dropout can break those combinations during training, preventing the network from learning them.
3. *Optimization interference:* Strong regularization can change the training dynamics in ways that cause plateaus or convergence to poor solutions. For example, high dropout can increase gradient noise, requiring different learning rates and more epochs.
4. *Metric mismatch:* Regularization that improves one metric (e.g., MSE) can worsen another (e.g., calibration). In trading, calibration can matter for sizing, so “better loss” can still mean worse decision quality.

Therefore, regularization must be tuned and validated. The correct claim is not “regularization

improves performance,” but “there exists a regularization regime that yields the best out-of-sample stability under our time-aware evaluation.”

Weight decay versus dropout: complementary, not interchangeable. Weight decay shrinks weights continuously. Dropout injects discrete randomness. They often complement each other, but they can also overlap in effect: both can reduce effective capacity. In small datasets, using both strongly can underfit. In larger datasets, mild weight decay plus moderate dropout can be stable. The key is to treat them as hyperparameters and tune within a disciplined budget (Chapter 12 logic).

13.8.2 Batch normalization vs layer normalization (conceptual, pitfalls)

Normalization layers are often introduced as training accelerators, but they also act as regularizers and can influence generalization. In time series and trading, they come with additional pitfalls because they interact with batching and time-aware evaluation.

The problem normalization tries to solve. During training, the distribution of activations in each layer can change as weights update. This makes optimization harder because each layer must adapt to shifting inputs. Normalization tries to stabilize these distributions by re-centering and re-scaling activations.

Batch normalization (BatchNorm). Batch normalization normalizes activations using statistics computed over the mini-batch. For a given feature dimension, it computes batch mean and variance and transforms:

$$\hat{z} = \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad (13.93)$$

then applies learned scale and shift parameters:

$$\text{BN}(z) = \gamma \hat{z} + \beta. \quad (13.94)$$

During training, μ_B and σ_B^2 depend on the batch. During inference, BatchNorm typically uses running averages of these statistics accumulated during training.

Why BatchNorm can be dangerous in trading. BatchNorm’s dependence on batch composition creates two key pitfalls:

1. *Boundary leakage through mixed batches:* If a batch accidentally contains samples from different time periods (e.g., training and validation mixed due to a bug), BatchNorm statistics incorporate information from validation distribution and leak it into training. Even if labels are not leaked, distributional information is leaked.
2. *Non-stationarity mismatch at inference:* BatchNorm uses running statistics estimated from training. If the distribution shifts in the test period, those statistics may be wrong,

causing unpredictable behavior. In markets, distribution shift is the default, not an exception. This can make BatchNorm fragile unless retraining and statistic updates are frequent and carefully governed.

Additionally, BatchNorm interacts with batch size: small batches yield noisy statistics, which can destabilize training. In finance, where we might have small effective sample sizes, this can be problematic.

Layer normalization (LayerNorm). Layer normalization normalizes across features within a single sample rather than across a batch. For a vector $z \in \mathbb{R}^m$, it computes mean and variance across the m dimensions:

$$\mu = \frac{1}{m} \sum_{j=1}^m z_j, \quad \sigma^2 = \frac{1}{m} \sum_{j=1}^m (z_j - \mu)^2, \quad (13.95)$$

and normalizes:

$$\text{LN}(z) = \gamma \odot \frac{z - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta. \quad (13.96)$$

Because LayerNorm does not depend on batch composition, it avoids the cross-sample leakage pitfall. It is often used in Transformers for exactly this reason: attention models rely on stable per-sample normalization.

LayerNorm pitfalls in trading. LayerNorm is not free of issues. By normalizing within each sample, it can remove absolute scale information that might be predictive. For example, if the magnitude of volatility is an important signal, LayerNorm can reduce the model’s ability to represent that magnitude unless the representation is designed to preserve it. LayerNorm also changes the geometry of the optimization problem, which can help or hurt depending on architecture.

A conservative stance. Within this chapter’s governance ethos:

- If using normalization layers, prefer LayerNorm in settings where batch composition could create leakage risks (especially in sequence models with shuffling).
- If using BatchNorm, be explicit about batch construction, keep train/validation/test loaders separate, and log the running statistics artifacts.
- In all cases, treat normalization choice as a hyperparameter and validate under time.

The point is not that BatchNorm is “bad.” It is that in trading it requires extra care, and many of the naive pipelines that work in iid problems are unsafe when time ordering matters.

13.8.3 Data augmentation for time series (careful): jitter, masking, and invariances

Data augmentation is common in image and text tasks because it increases effective data size and encourages invariances (e.g., a cat is a cat even if the image is slightly rotated). In time series

forecasting for trading, augmentation is trickier. We must not create synthetic examples that violate the market’s causal structure or distort the economic meaning of the signal. Nevertheless, carefully designed augmentation can sometimes improve robustness by preventing the model from overfitting to brittle details.

The guiding question: what invariance do we believe? An augmentation is justified only if we believe the predictive relationship should be invariant (or approximately invariant) to that transformation. In finance, few invariances are truly safe. But some approximate invariances can exist:

- small measurement noise in features should not change the forecast;
- missing observations should not break the model completely;
- slight timing jitter within a window should not change the signal if the signal is coarse.

These are not guaranteed; they are hypotheses.

Jitter (add small noise). Jitter adds small random noise to inputs:

$$\tilde{x} = x + \epsilon, \quad \epsilon \sim \text{small noise distribution.} \quad (13.97)$$

The intent is to encourage smoothness: the model should not change drastically for tiny input changes. This is a form of regularization similar in spirit to weight decay. In trading, jitter can be legitimate when inputs are already standardized and when the noise scale is carefully chosen. However, jitter can also destroy delicate signals (for example, if the signal depends on precise thresholds) and can create unrealistic patterns if applied to raw returns (because returns have heavy tails and are not well approximated by Gaussian perturbations). A safer approach is to jitter standardized features rather than raw returns, and to keep noise magnitudes small.

Masking (randomly hide parts of the window). Masking sets some inputs to zero (or to a sentinel value) within the window. This resembles dropout at the input level:

$$\tilde{X}_t[j, k] = \begin{cases} 0, & \text{with probability } p_{\text{mask}}, \\ X_t[j, k], & \text{otherwise.} \end{cases} \quad (13.98)$$

The goal is robustness to missing data and to encourage the model to use distributed evidence rather than rely on one time step. In markets, missingness is real (stale quotes, holidays, data issues), so robustness is valuable. But masking can also harm if the signal genuinely depends on a particular recent observation (e.g., last return). Therefore, masking probability should be modest, and the effect should be validated under time.

Time jitter / small shifts. One might shift windows by a small amount, or randomly drop or duplicate a time step within the window. This is extremely risky in trading because timing matters and because small shifts can effectively change what the model “knows” relative to the

label. If augmentation changes which observations precede the label horizon, it can create subtle label misalignment or leakage. Therefore, within this book’s conservative approach, time shifts are generally discouraged unless the decision horizon is coarse and the augmentation is explicitly designed not to cross the boundary between past and future relative to the label. If used at all, it must be implemented with strict index checks.

Augmentation must preserve causality. A hard rule: augmentation must not use future information. This sounds obvious, but some seemingly benign augmentations violate it. For example, smoothing a series with a centered moving average uses future points. That is leakage. Any filtering-based augmentation must use causal filters only, or be applied entirely within the past window without borrowing from after t .

Augmentation as hypothesis testing. In trading, augmentation is less about “making the model better” and more about probing robustness. If a model’s performance collapses under small jitter, it may be overfitting to brittle artifacts. If performance is stable, that is evidence of smoothness. Thus, augmentation can be used as a diagnostic tool even when not used for final training.

13.8.4 Model averaging and ensembling (conceptual; multi-strategy in Ch. 20)

Ensembling is a powerful regularization idea: average multiple models to reduce variance. In finance, variance reduction is often more valuable than squeezing out the last bit of average performance, because unstable signals can be expensive in real trading.

Why averaging helps. Suppose we have M models with predictions $\hat{y}^{(1)}, \dots, \hat{y}^{(M)}$. A simple ensemble prediction is the average:

$$\hat{y}^{\text{ens}} = \frac{1}{M} \sum_{m=1}^M \hat{y}^{(m)}. \quad (13.99)$$

If individual models have errors that are not perfectly correlated, averaging reduces error variance. Even if each model is slightly biased, variance reduction can improve out-of-sample stability.

In deep learning, different initializations and mini-batch order can lead to different local solutions. An ensemble can therefore be seen as smoothing over the randomness of training. This can be particularly useful in trading where single runs can be seed-sensitive.

What kinds of ensembles make sense here. Within Chapter 13 scope (forecasting), sensible ensembles include:

- *Seed ensembles*: same architecture and hyperparameters, different random seeds.
- *Architecture ensembles*: small MLP + small CNN, for example, if they capture different aspects of the data.
- *Time ensembles*: models trained on slightly different training windows (careful; must respect time).

The ensemble must be constructed without peeking at test results. If you choose ensemble members based on test performance, you reintroduce selection bias. A disciplined approach is to predefine ensemble construction rules and apply them consistently.

Ensembling is not a free lunch. Ensembles increase computation and operational complexity. They can also reduce interpretability: if the final signal is an average of many models, it can be harder to attribute behavior. In a governance context, this must be weighed explicitly. However, in many institutional settings, an ensemble of modest models can be more acceptable than a single very large model because it can be more stable and easier to monitor (each component can be tracked).

Connection to multi-strategy systems. Chapter 20 addresses multi-strategy and multi-horizon systems, where the portfolio combines multiple signals. Model ensembling is conceptually similar: we combine multiple estimators of the same target. The difference is that multi-strategy systems often combine signals with different economic rationales, whereas ensembling often combines similar models to reduce estimator variance. Still, the underlying principle is the same: diversify sources of error and avoid brittle dependence on a single model.

A governance-native view of regularization. Regularization in deep nets is not just “add dropout.” It is a system of defenses:

1. Penalize complexity directly (weight decay).
2. Inject robustness via noise (dropout, masking).
3. Stabilize optimization (normalization layers, learning-rate schedules).
4. Reduce variance by averaging (ensembles).

Each defense has costs and failure modes, and none should be applied by superstition. The correct workflow is to begin with a simple, regularized baseline, add one mechanism at a time, validate under walk-forward folds, and record artifacts that show stability rather than relying on a single best-run metric.

The deep lesson is that regularization is not about making the model “smaller.” It is about making the model’s behavior more predictable under time and drift. In trading, that predictability is the foundation of any credible edge claim. A model that generalizes weakly but consistently is usually more valuable than one that occasionally looks brilliant and often fails. Regularization is how we bias our training process toward the former outcome.

13.9 Diagnostics: Understanding What the Network Learned

A deep network that produces a good number on a backtest is not yet a trustworthy component of a trading system. In trading, the core question is not only “does it predict?” but also “*what* did it learn, *how* fragile is that learning, and *under what conditions* does it break?” Diagnostics are the tools we use to answer these questions. They do not provide certainty—markets do not

offer that—but they provide evidence. They help us distinguish a model that learned a stable relationship from a model that learned a transient coincidence, an implementation bug, or a leak.

Diagnostics are especially important for deep nets because their flexibility allows them to fit many different functions that all appear to perform similarly in-sample. Two models can have the same validation metric but very different behavior under stress, drift, or small perturbations. In addition, deep nets can exploit subtle pipeline flaws: normalization leakage, alignment errors, and batch boundary mixing. Diagnostics are therefore both scientific and forensic. They help us understand the mapping learned by the network, and they help us detect whether the mapping is “too good to be true.”

This section is organized around four diagnostic families that are practical and governance-friendly. First, learning curves: the simplest window into training dynamics and overfitting. Second, feature sensitivity: tests that probe whether the model’s predictions are stable under small changes and whether the model relies on plausible information. Third, attribution methods: conceptual tools like saliency that attempt to explain which inputs mattered, along with a sober discussion of their limitations in finance. Fourth, error analysis by regime proxies: a preview of the regime lens that will be formalized in Chapter 14, used here only as a diagnostic stratification tool.

Throughout, we maintain the scope boundary: we are forecasting returns (or directions or rankings) with supervised learning. We are not yet evaluating execution, costs, or portfolio optimization. The diagnostics discussed here live at the forecasting layer. They are still essential, because weak or fragile forecasts will not become strong simply by being embedded in a later pipeline.

13.9.1 Learning curves and overfit signatures

What learning curves are. A learning curve is a plot of performance metrics as training progresses. The most common is training loss versus epoch and validation loss versus epoch:

$$J_{\text{train}}^{(e)} \quad \text{and} \quad J_{\text{val}}^{(e)}, \quad e = 1, 2, \dots, E, \quad (13.100)$$

where e denotes epoch number. We compute $J_{\text{train}}^{(e)}$ on the training set (or a subset) and $J_{\text{val}}^{(e)}$ on a validation set that is strictly later in time than the training set (Chapter 11 discipline). For classification, one might also track accuracy; for ranking, one might track IC or rank IC.

Learning curves are not merely plots; they are the simplest diagnostic evidence about whether training is stable, whether the model is learning anything, and whether it is beginning to overfit. In deep learning, where optimization can fail for many reasons (learning rate, initialization, normalization), the shape of curves often tells you quickly what is wrong.

The baseline patterns. There are a few canonical learning curve patterns worth internalizing:

1) Healthy learning with eventual overfit. Training loss decreases steadily. Validation loss decreases initially, then reaches a minimum, then increases. This is the classic overfit

signature:

$$J_{\text{train}}^{(e)} \downarrow \quad \text{while} \quad J_{\text{val}}^{(e)} \downarrow \quad \text{then} \quad \uparrow. \quad (13.101)$$

Interpretation: the model is learning generalizable patterns early and then fitting noise. Early stopping should select the epoch near the validation minimum. In finance, this pattern is common, and the location of the minimum can shift across folds because regimes differ.

2) Underfitting or optimization failure. Training loss decreases little or not at all; validation loss is flat. This can mean:

- the model is too small,
- the learning rate is too low,
- the inputs are poorly scaled,
- the target is too noisy for the chosen representation,
- or a bug exists (e.g., labels misaligned, gradients not flowing).

In trading, it can also mean the honest truth: there is no predictive signal at that horizon for that representation.

3) Divergence / instability. Training loss oscillates wildly or increases; sometimes it becomes NaN. Validation loss follows. This often indicates:

- learning rate too high,
- batch size too small relative to learning rate,
- numerical instability,
- exploding gradients,
- or bad normalization.

In a governance-native pipeline, divergence should trigger hard failure and automatic logging of the run as invalid rather than silent continuation.

4) Training improves but validation does not. Training loss decreases substantially; validation remains flat or worsens from the start. This is a common deep-learning trap in finance because the model can fit noise easily. Interpretation: the model has enough capacity to memorize training patterns but those patterns do not generalize. Remedies are not primarily architectural; they are evaluation and regularization changes: reduce capacity, increase regularization, shorten training (early stopping), or rethink the label/horizon.

5) Validation improves later than expected. Sometimes validation improves only after many epochs. This can happen with learning-rate schedules: a high learning rate explores, then a lower learning rate refines. It can also happen when the model must first learn a stable representation. In finance, one must be cautious: a late improvement can also be a sign of accidental alignment with a specific validation period. Walk-forward replication is crucial.

Overfit signatures beyond the loss. Overfitting is not only about loss divergence. In trading, additional overfit signatures include:

- *Metric inconsistency:* validation MSE improves but IC deteriorates, or vice versa. This can indicate the model is optimizing a metric that is not aligned with the economic objective, or it can indicate instability.
- *Calibration drift:* predicted probabilities become more extreme over epochs while accuracy does not improve. This indicates overconfidence and can be dangerous for sizing.
- *Seed sensitivity:* the epoch of best validation performance differs wildly across seeds, and the best validation performance itself varies. This indicates that the optimization is landing in different solutions, suggesting weak signal and high variance.

Learning curves across folds as diagnostic evidence. In a walk-forward evaluation, each fold has its own learning curves. A stable model should exhibit broadly similar patterns across folds. Not identical (regimes differ), but recognizable. If one fold shows clean learning and another shows complete failure, the model may be regime-sensitive. That is not necessarily disqualifying; it is information. It suggests that the model's predictive relationship depends on market conditions, which motivates regime analysis (Chapter 14).

A governance-native approach therefore stores learning curves as artifacts for each fold and seed. They are part of the evidence package. They should be examined at least once for each major experiment family, and summarized systematically (e.g., average and dispersion of best epoch, average train-val gap at best epoch).

What learning curves cannot prove. Learning curves cannot prove that a model is tradable or that it will remain stable. They cannot rule out subtle leakage. A model can have plausible curves and still be wrong. But implausible curves are strong evidence of failure. In trading, that asymmetry is valuable: diagnostics are often better at rejecting bad models than at certifying good ones.

13.9.2 Feature sensitivity: perturbation tests and stability across folds

Even when a model generalizes in validation, we need to know whether it is stable under small perturbations. Markets are noisy; data pipelines are imperfect; small changes in feature values are common. A robust forecasting model should not change its prediction radically when inputs change slightly, unless the input change corresponds to a meaningful state change. Feature sensitivity tests probe this.

Local perturbation tests. A basic perturbation test takes an input x and creates a perturbed version:

$$\tilde{x} = x + \epsilon, \quad (13.102)$$

where ϵ is small noise (often scaled relative to feature standard deviation). We then compare predictions:

$$\Delta = f_{\theta}(\tilde{x}) - f_{\theta}(x). \quad (13.103)$$

If ϵ is tiny, Δ should usually be small. If Δ is often large, the model is highly sensitive, suggesting it may be fitting noise or relying on brittle thresholds.

In finance, there is a nuance: some signals are genuinely threshold-like (e.g., breakout conditions). But deep models that behave like brittle threshold machines across many features are risky, because tiny data errors can flip decisions. Sensitivity analysis helps reveal whether the model is smooth or fragile.

One-feature perturbations. Instead of perturbing all features at once, we can perturb one feature channel at a time. Let $x = (x_1, \dots, x_d)$. Define $\tilde{x}^{(j)}$ by perturbing only coordinate j :

$$\tilde{x}_k^{(j)} = \begin{cases} x_k + \epsilon, & k = j, \\ x_k, & k \neq j. \end{cases} \quad (13.104)$$

Then measure the prediction change. This yields an empirical sensitivity score per feature. This is conceptually similar to estimating partial derivatives, but in a discrete and robust way.

In windowed tensors, we can perturb an entire channel across time in the window, or a particular time index. This can reveal whether the model relies mostly on the most recent observation (common), or whether it uses longer history, or whether it uses a particular engineered feature.

Permutation importance as a stronger test. A more aggressive test is permutation: shuffle a feature across samples while keeping other features intact. If the feature matters, shuffling should degrade performance. This can be done per fold. In finance, permutation tests are useful because they do not assume small perturbations; they remove the feature’s informative alignment entirely.

Permutation importance has two major uses:

1. *Interpretation*: which features are truly used by the model?
2. *Leakage detection*: if permuting a suspicious feature destroys performance, that feature might be leaking label information.

However, permutation importance is not perfect: if features are correlated, permuting one feature may not change performance much because the model can use correlated substitutes. Therefore, interpretation must consider feature redundancy.

Stability across folds. In trading, a feature’s importance should not fluctuate wildly across folds unless there is a regime reason. If the model relies heavily on volume in one period and ignores it in another, this could reflect changing microstructure. But it could also reflect instability and overfitting. Therefore, feature sensitivity and permutation importance should be computed per fold and compared. Consistency is evidence of stable structure; inconsistency is evidence of regime dependence or fragility.

Sensitivity to scaling and normalization. A special sensitivity test in finance is to slightly alter scaling parameters (within plausible estimation error) and observe predictions. If the model’s predictions change drastically, it may be overly dependent on exact scaling. This matters because scaling parameters estimated on training data may differ slightly when retraining on a new window. A robust model should tolerate such variation.

Sensitivity to missingness. Markets contain missing values and stale quotes. A practical perturbation test is to mask a small fraction of window entries (set to zero or forward-fill) and evaluate prediction stability. If the model collapses under small masking, it may be too reliant on specific data points and may fail operationally.

A diagnostic hierarchy: from local to structural. Feature sensitivity tests range from local (small perturbations) to structural (permutation, masking). The deeper lesson is that robustness should be established at multiple levels. A model that passes one test can fail another. In finance, we prefer models whose predictions are stable to small perturbations and whose performance does not rely on a single fragile feature.

13.9.3 Attribution methods (conceptual): saliency and limitations in finance

Attribution methods try to answer: “which parts of the input influenced the prediction?” In deep learning, this is often done through gradients. The simplest is saliency: compute the gradient of the output with respect to the input:

$$S(x) = \nabla_x f_\theta(x). \quad (13.105)$$

If $S_j(x)$ is large in magnitude, it suggests that changing feature j slightly would change the output, so that feature is influential for that example.

Saliency in windowed inputs. For windowed tensors $X \in \mathbb{R}^{L \times p}$, saliency yields a matrix of the same shape:

$$S(X) = \frac{\partial f_\theta(X)}{\partial X} \in \mathbb{R}^{L \times p}. \quad (13.106)$$

This can be visualized as a heatmap: which time steps and which channels mattered most. This is appealing for market data because it can reveal whether the model focuses on recent returns, volatility spikes, or specific engineered features.

Why saliency is not a proof. Despite its appeal, saliency has serious limitations in finance:

1) Gradient does not equal causality. A large gradient indicates sensitivity, not causal importance. In markets, features are correlated and can proxy for each other. A model may be sensitive to a feature because it correlates with the true driver, not because it is the true driver. Attribution cannot resolve this.

2) Gradient methods can be noisy and unstable. Small changes in input or in model parameters can change gradients significantly. In finance, where signals are weak, the attribution map can look like noise. Smooth attribution methods exist (averaging gradients over noise perturbations), but they remain heuristic.

3) Saturation and nonlinearity distort gradients. If activations saturate, gradients can be small even when a feature is important. Conversely, gradients can be large in regions where the model is uncertain. Thus, saliency can mislead.

4) Attribution can be manipulated by normalization. If inputs are scaled, gradients reflect the scaled units. Comparing attributions across features requires consistent scaling. In trading, scaling is fold-specific; comparing saliency maps across folds can therefore be tricky. If one fold has higher volatility, scaled returns differ, and gradients may not be comparable.

5) Finance-specific problem: regime dependence. A feature might matter in one regime and not in another. Saliency on a random set of examples may average away regime structure. Without stratification, attribution can look incoherent.

Integrated gradients and related ideas (conceptual). More sophisticated attribution methods, such as integrated gradients, attempt to reduce gradient noise by integrating along a path from a baseline input x_0 to the actual input x :

$$\text{IG}_j(x) = (x_j - x_{0,j}) \int_0^1 \frac{\partial f_\theta(x_0 + \alpha(x - x_0))}{\partial x_j} d\alpha. \quad (13.107)$$

The conceptual point is that instead of looking at the derivative at one point, we aggregate derivatives along a path. This can yield more stable attributions. But it introduces a new choice: the baseline x_0 . In finance, choosing a baseline is nontrivial. Is it zero returns? Average window? A calm regime? Different choices yield different narratives. Therefore, integrated gradients can be useful as a diagnostic, but the results must be presented with baseline assumptions explicitly stated.

Attribution should be paired with perturbation. A robust approach is to treat attribution as a hypothesis generator, then test hypotheses with perturbation. If saliency suggests that the last return is crucial, then permute or mask the last return and see whether predictions change. If attribution suggests that volatility channel matters, perturb volatility estimates and check

stability. Attribution without perturbation is storytelling. Perturbation without attribution is blind. Together, they form a disciplined diagnostic pair.

Governance and interpretability claims. In a governance-native setting, one should be careful not to oversell attribution. The correct language is:

“These attribution maps suggest that the model is sensitive to these inputs; perturbation tests confirm that removing or randomizing these inputs materially changes predictions.”

Not:

“The model uses this feature for this economic reason.”

The latter is rarely justifiable in finance without stronger causal designs.

13.9.4 Error analysis by regime proxies (preview; regimes in Ch. 14)

Markets are not stationary. A model’s errors are rarely uniform across time. A deep net might perform well in trending periods and fail in choppy periods, or perform well in low volatility and fail in high volatility. Understanding this structure is crucial both for scientific honesty and for practical deployment decisions (e.g., when to trust the model, when to reduce exposure). Chapter 14 formalizes regime detection using HMMs and related methods. Here we preview a simpler idea: use *regime proxies* to stratify error analysis.

What is a regime proxy? A regime proxy is a simple observable statistic that correlates with different market conditions. Examples include:

- realized volatility over a recent window,
- trend strength (e.g., moving average slope),
- liquidity proxies (volume, bid-ask proxies),
- drawdown from a recent peak,
- dispersion across assets (in cross-sectional settings).

These proxies are not regime models; they are simple partitions of the data.

Stratifying errors. Let $e_t = \hat{y}_t - y_t$ be the forecast error (for regression). We can compute error metrics within bins of the proxy. For example, define volatility proxy v_t computed causally from past returns, and define bins:

$$\text{Low-vol : } v_t \leq q_{33}, \quad \text{Mid-vol : } q_{33} < v_t \leq q_{66}, \quad \text{High-vol : } v_t > q_{66}, \quad (13.108)$$

where quantiles are computed within the training set (or within each fold, carefully). Then compute MSE or IC within each bin:

$$\text{MSE}_{\text{high-vol}} = \frac{1}{|\mathcal{T}_{\text{high}}|} \sum_{t \in \mathcal{T}_{\text{high}}} (\hat{y}_t - y_t)^2. \quad (13.109)$$

Similarly, for sign accuracy or rank IC. This yields a regime-conditional performance profile.

What we learn from regime-conditional errors. Several useful patterns can appear:

1) Asymmetric failure. The model may be accurate in low volatility but fail in high volatility. This can be economically important because high volatility periods often dominate risk and P&L distribution.

2) Calibration breakdown. A classifier may be well-calibrated in calm regimes and overconfident in turbulent regimes. This is a common failure and directly impacts sizing.

3) Architecture mismatch. A CNN that detects local patterns might work well in choppy mean-reverting regimes, while an MLP might work well in smoother regimes where engineered features dominate. Error stratification can reveal such differences.

4) Evidence for regime-aware modeling. If performance differences across regimes are strong and stable across folds, that is evidence that a regime-aware approach (Chapter 14) might be justified. Importantly, this is evidence, not a guarantee. But it motivates the next step in the book's arc.

Causality and proxy construction. The regime proxy must be computed causally. If you compute realized volatility using future returns relative to t , you leak. Therefore, even for diagnostics, the proxy must be defined using information available at time t . For example, realized volatility could be computed from the past L returns ending at t :

$$\hat{\sigma}_t = \sqrt{\frac{1}{L} \sum_{j=0}^{L-1} r_{t-j}^2}. \quad (13.110)$$

This uses only past information.

Avoiding post-hoc regime storytelling. A diagnostic risk is to define many proxies and then tell a story that fits the observed performance. That can become a new form of selection bias. The governance discipline is to predefine a small set of proxies (e.g., volatility and trend strength) and report results consistently. If additional proxies are explored, record that exploration.

Regime diagnostics as deployment inputs. Even before Chapter 14, regime-conditional diagnostics can inform how we would deploy a model. For example, if the model fails in high volatility, we might condition exposure on a volatility filter. That becomes a strategy decision

(later chapters), but diagnostics provide the evidence needed to justify such filters rather than adding them as ad hoc patches.

The diagnostic mindset. The overarching mindset is that deep learning in trading must be accompanied by an “evidence dossier.” Learning curves show training stability and overfit behavior. Sensitivity tests show robustness and plausibility of information use. Attribution methods offer localized explanations but must be treated skeptically and paired with perturbation. Regime-proxy stratification reveals conditional performance and motivates regime-aware modeling.

This diagnostic mindset is not optional. Without it, a deep net that appears to work is indistinguishable from a deep net that found a leak. With it, we can at least separate plausible models from obviously fragile ones and we can communicate, in a governance-native way, what we believe the model learned and where we expect it to fail. In markets, knowing where a model fails is often the most valuable knowledge the research process can produce.

13.10 Governance-Native Checklist for Deep Learning in Trading

Deep learning in trading is not primarily a question of model capacity. It is a question of *process credibility*. A neural network can fit almost any function, and in market data that flexibility creates a paradox: the easier it becomes to produce impressive backtest metrics, the harder it becomes to believe them. Governance is how we resolve this paradox. Governance does not mean paperwork. It means designing the pipeline so that (i) errors are detectable, (ii) leakage is mechanically difficult, (iii) results are reproducible, and (iv) claims are limited to what the evidence supports.

This checklist is the culmination of Chapter 13’s arc. Chapters 11 and 12 established time-aware supervised learning and disciplined model selection. Chapter 13 adds deep learning architectures, training dynamics, regularization, and diagnostics. The governance layer binds these components into something that can be trusted. In an institutional setting, governance artifacts are the difference between “a promising experiment” and “a result that can be discussed seriously.” In an academic setting, they are the difference between an anecdote and a publication-quality claim.

The deep-learning governance checklist has six pillars. Determinism ensures that results can be reproduced and that variance is measured rather than hidden. Data lineage ensures that we can trace every number from raw market data to input tensors and labels, and prove that splits and normalizations were time-safe. Causality gates ensure that no-look-ahead is enforced at the level where deep models actually operate: tensors and label alignment. Training logs ensure that the training process is visible, diagnosable, and hash-verifiable. A model registry ensures that trained models can be reproduced and used consistently in inference. Finally, a minimum audit artifact set provides the evidence pack needed to support claims without overreach.

The table below summarizes the minimal artifact set. The subsections explain what each item should contain and why it matters.

13.10.1 Determinism: seeds, configs, and run manifests

Why determinism is foundational. Deep learning is stochastic by default. Weight initialization is random. Mini-batch ordering is random. Some hardware kernels can be nondeterministic. In trading research, that stochasticity can be mistaken for alpha. A model that “works” on one run may fail on another. Without deterministic controls, we cannot tell whether performance differences are due to model design or due to randomness.

Determinism therefore has a practical meaning: you must be able to rerun the experiment with the same configuration and obtain the same results (or results within a documented and understood tolerance if certain operations cannot be forced deterministic). Even when full determinism is difficult, the governance goal is to make randomness explicit and controlled.

Seeds as first-class parameters. A seed is not an afterthought; it is part of the experiment configuration. At minimum, seeds should control:

- parameter initialization,
- mini-batch shuffling or batch sampling,
- any stochastic regularization (dropout masks),
- any stochastic data augmentation.

The seed must be written into the run manifest. If multiple seeds are used for robustness, the seed list must be declared before results are examined, and results must be reported for all seeds, not just the best.

Configs as immutable specifications. A configuration defines everything that affects the run: data inputs, preprocessing choices, window length, label horizon, model architecture, optimizer, learning-rate schedule, regularization settings, early stopping rules, and evaluation metrics. The config should be treated as immutable once the run begins. The governance pattern is:

1. write a config file or structured config object,
2. compute a hash of the config content,
3. log that hash in the run manifest,
4. store the config alongside outputs.

This prevents “silent” changes that invalidate comparisons.

Run manifests. A run manifest is a short record that answers: *what exactly ran, on what environment, with which randomness controls?* At minimum, it includes:

- the seed(s),
- a config hash,

- basic environment details: library versions, Python version, hardware note (CPU/GPU),
- the timestamp and a unique run ID.

The goal is reproducibility and traceability. When results are shown in a paper or internal memo, the run ID should allow retrieval of all artifacts.

Determinism does not mean pretending variance is zero. In finance, we want to measure variance across seeds and folds. Determinism lets us do that cleanly: we can run multiple controlled seeds and quantify dispersion. Without determinism, dispersion is contaminated by hidden run-to-run differences. The correct outcome is not “one deterministic run” but “a controlled set of deterministic runs whose variance is reported.”

13.10.2 Data lineage: fingerprints, split manifests, and normalization proofs

Why data lineage is the main defense against accidental fraud. Most false edges in deep learning for trading are caused by data issues: survivorship bias, misalignment, look-ahead in feature construction, or normalization leakage. Because deep models can exploit subtle leaks, we must be able to trace every step in the data pipeline and prove that it respects time.

Data fingerprints. A data fingerprint summarizes the dataset used in a run. It is not the data itself; it is a compact description sufficient to identify and reproduce the dataset. At minimum:

- instrument(s) or universe definition,
- frequency (daily, hourly, etc.),
- time span (start/end dates),
- missingness summary,
- corporate action handling note (if applicable, even if placeholder),
- a preprocessing hash (hash of the code or transform pipeline version).

The fingerprint is important because it prevents silent changes in data sources (e.g., switching from adjusted close to close) from invalidating comparisons.

Split manifests. A split manifest describes the time-aware evaluation design. For each fold, it records:

- training start/end dates,
- validation dates (if used),
- test dates,
- embargo rules (length and rationale),
- label horizon h and overlap policy.

This artifact is the proof that the evaluation respected time. Without it, a reported metric is just a number.

Normalization proofs. Normalization is one of the most common leakage channels. A normalization log must show:

- which window was used to fit normalization statistics (training only, per fold),
- the statistics themselves (means, variances, or robust equivalents),
- proof of forward application: a recorded statement or test that validation/test normalization used training-fitted parameters, not refitted parameters.

In a strict pipeline, the proof is a hard assertion: the normalization “fit” function is never called on validation/test, and any attempt triggers failure. The log should also record whether normalization was global per-feature, per-window, or per-sample, because that changes interpretation.

Lineage as hashes and versioning. Data lineage is strengthened by hashing intermediate artifacts:

- hash of raw input file(s) or download snapshot,
- hash of processed feature arrays,
- hash of label arrays,
- hash of the final tensor dataset.

The point is not cryptographic security; it is auditability. If someone questions a result, you can show exactly which tensor was used.

13.10.3 Causality gates: no-look-ahead assertions for tensors and labels

Why causality must be enforced at the tensor level. In deep learning, the model consumes tensors and labels. It does not know dates. It does not know that a feature came from the future. If the tensor contains leaked information, the model will exploit it. Therefore, causality must be enforced where it matters: in the construction and alignment of tensors and labels.

Tensor alignment assertions. A causal tensor construction should satisfy a simple invariant: for each example time t , every element of the input window must correspond to information available at or before t . If we index the window as $X_t[j, :] = u_{t-L+j}$, then we require:

$$\max\{\text{time index in } X_t\} = t. \quad (13.111)$$

This becomes a practical assertion in code: the final row of the window corresponds to the decision time, and no row extends beyond it.

In a governance-native pipeline, each dataset example carries metadata: the decision time index t and the time indices used in the window. We then assert mechanically that they are consistent. This may feel heavy, but it is the most reliable way to prevent off-by-one errors.

Label shift assertions. Labels must be strictly future relative to the decision time. For a horizon h :

$$y_t = r_{t \rightarrow t+h}, \quad (13.112)$$

the label uses data from $t + 1$ onward. The causality invariant is:

$$\min\{\text{time indices used in } y_t\} \geq t + 1. \quad (13.113)$$

Again, this is enforceable by construction and can be tested for every example.

Split boundary assertions. Even if each example is causal, splits can still leak if boundaries are mishandled. We require:

- training examples have decision times $\leq \tau$,
- validation/test examples have decision times $> \tau$ (plus embargo rules),
- no normalization fit includes decision times beyond training.

These are simple checks, but they catch a surprising number of bugs.

Leakage canaries as explicit tests. Causality gates should be complemented with canary tests:

1. *Random label canary*: shuffle labels; performance must collapse.
2. *Intentional leak canary*: add a known leaked feature (e.g., a shifted future return); performance should jump. This proves that the model would exploit leaks and therefore that guarding against leaks is essential.
3. *Normalization canary*: compute normalization on full data and observe metric inflation; then show that proper per-fold normalization removes that inflation.

These canaries are not about showing off; they are a way of demonstrating that the pipeline has been stress-tested against common failure modes.

13.10.4 Training logs: losses, metrics, gradients, and checkpoint hashes

Training is part of the evidence. In deep learning, the training process itself contains information about model credibility. If training is unstable, if loss diverges, if validation improves only on a single seed, those are warnings. Training logs make these phenomena visible and auditable.

Loss and metric curves. At minimum, logs should include:

- training loss per epoch (or per update),
- validation loss per epoch,

- selected evaluation metrics (MSE, sign accuracy, IC, rank IC) per epoch,
- early stopping events (best epoch, patience, stopping time).

These curves are essential to diagnose overfit and to justify early stopping decisions.

Gradient diagnostics (lightweight). Full gradient logging is expensive, but lightweight gradient diagnostics can be highly informative:

- gradient norm statistics per epoch (mean, max),
- detection of NaNs or infs in gradients,
- optionally, parameter norm statistics (to monitor weight growth).

In finance, exploding gradients can produce catastrophic instability and can also be a symptom of poor scaling or label noise. Logging gradient norms helps identify whether training is stable or merely lucky.

Checkpoint hashes. A trained model is an artifact. In governance, artifacts should be hash-verifiable. Each saved checkpoint (especially the best checkpoint chosen by early stopping) should have:

- a file hash,
- associated config hash,
- associated data fingerprint hash,
- associated fold and seed identifiers.

This ensures that when you claim “these results came from this checkpoint,” the statement is verifiable.

Why checkpoint discipline matters in trading. It is common to rerun training and accidentally evaluate a different checkpoint than the one intended. It is also common to save the final epoch rather than the best epoch and then misreport performance. Hashes and explicit checkpoint IDs prevent such errors. They also enable reproducible inference: you can load exactly the checkpoint used for reported results.

13.10.5 Model registry: architecture spec, parameter counts, and reproducible inference

A model is more than weights. A deep model is defined by:

- architecture (layers, widths, activations, normalization layers),
- initialization rules,
- training procedure (optimizer, learning rate schedule, regularization),

- preprocessing pipeline (scaling, feature construction),
- inference procedure (how inputs are shaped and normalized).

If any of these change, the model changes. A model registry records these elements so that the model can be reconstructed and used consistently.

Architecture specification. The model spec should include:

- input shape conventions (e.g., $L \times p$ window),
- layer-by-layer description,
- activation functions,
- normalization layers used (BatchNorm, LayerNorm),
- dropout rates,
- output type (regression, classification, ranking score).

This is the minimal description needed to reimplement the model in a new environment.

Parameter counts and capacity. Parameter count is not a perfect measure of complexity, but it is a useful sanity indicator. In trading, extremely large models relative to data size are suspicious. Therefore, the registry should include:

- total parameter count,
- trainable parameter count (if some parameters are frozen),
- rough ratio of samples to parameters (as a warning signal, not a theorem).

The goal is to make the scale of the model explicit.

Reproducible inference: the end-to-end contract. A model cannot be deployed or audited if inference is not reproducible. The registry therefore must include:

- exact preprocessing steps applied at inference (including normalization parameters),
- tensor shape and ordering conventions,
- handling of missing values at inference,
- decision-time convention (what data up to time t is included).

The key is an “inference contract”: given raw inputs and a timestamp, the pipeline produces the same tensor as was used in training. If this contract is broken, the model will behave unpredictably.

Versioning. Each model in the registry should have:

- a unique model ID,
- the run ID that produced it,
- the config hash,
- the data fingerprint,
- the checkpoint hash.

This ensures that deployed models can be traced back to experiments and that results can be reproduced.

13.10.6 Minimum audit artifacts for publication-quality claims

What “publication-quality” means here. In this book, a publication-quality claim is not necessarily a claim of large economic profitability. It is a claim that the forecasting result is:

- reproducible,
- time-aware and causality-safe,
- evaluated under disciplined selection,
- accompanied by diagnostics that reveal limitations,
- presented with appropriate skepticism about non-stationarity.

The audit artifact set is the evidence needed to support such a claim.

The minimal artifact pack. The table in the prompt summarizes the minimal artifacts. Here we explain what makes them sufficient:

1) Run manifest. This anchors the experiment in a reproducible environment. Without it, one cannot even rerun the same code with confidence. It also supports comparability: if two runs differ, the manifest reveals whether they differ due to config or environment.

2) Data fingerprint. This anchors the dataset. Many research disputes in finance are ultimately data disputes. The fingerprint ensures the dataset is identifiable and that preprocessing is tracked.

3) Split manifest. This anchors the evaluation design. It is the proof that the experiment respected time and that any embargo policy was applied consistently. It also communicates the label horizon and overlap policy, which are essential to interpret metrics.

4) Normalization log. This is the proof against one of the most common forms of leakage. It should be explicit enough that an auditor can confirm that the model could not have accessed future distributional information through scaling.

5) Model spec. This is the intellectual content of the deep learning claim: the architecture and training choices. It must be recorded in a way that is not dependent on memory or hand-waving.

6) Training trace. This is evidence that the model trained stably and that early stopping and selection rules were applied as declared. It also supports diagnoses: if a model performs poorly later, the trace can reveal whether training was unstable.

7) Causality tests. Assertions and canaries are the mechanical enforcement of no-look-ahead. In deep learning, this is the central guardrail. Publication-quality claims require showing that these gates existed and passed.

8) Result pack. This is the summary of evidence: fold-wise metrics (not just averages), plots, error stratification, and a limitations statement. The limitations statement is part of governance: it prevents over-claiming and clarifies what the results do and do not establish.

The role of limitations statements. In markets, even a clean out-of-sample result can be a temporary regime artifact. Therefore, publication-quality reporting must include limitations such as:

- sensitivity to regime proxies,
- sensitivity to horizon choice,
- dependence on specific instruments or sample periods,
- potential impact of transaction costs (not modeled here, but acknowledged),
- uncertainty due to finite sample size and autocorrelation.

Limitations are not admissions of defeat; they are signs of scientific maturity.

From checklist to culture. A checklist is only useful if it is followed consistently. The deeper goal is a research culture where every deep learning experiment automatically produces the artifact pack. That culture reduces friction: instead of arguing after the fact about whether leakage might exist, the pipeline makes leakage difficult and produces proofs. It also accelerates learning: stable artifacts make it easier to compare models, understand failures, and iterate responsibly.

Why this matters for the rest of the book. Chapter 13 is the point where model complexity begins to escalate. Chapter 14 will introduce explicit regime structure, Chapter 15 will integrate event-driven information, Chapters 16–18 will link forecasts to portfolios and execution, and Chapter 19 will shift from forecasting to decision-making under reinforcement learning. If governance is not established here, later chapters become ungrounded. Governance is the spine that keeps sophistication from collapsing into overfit.

In summary, deep learning in trading can be credible, but only under a governance-native workflow. Determinism makes randomness measurable. Data lineage makes every number traceable. Causality gates make look-ahead mechanically difficult. Training logs make optimization visible. A model registry makes inference reproducible. And the audit artifact pack makes claims defensible. Without these elements, deep learning results in trading are too easy to produce and too hard to trust. With them, we can make cautious, evidence-backed statements about what the network learned and how it might contribute to a trading system.

13.11 Practical Patterns and Anti-Patterns (Trading Reality Check)

Deep learning in trading is attractive because it promises to learn structure automatically. It is also dangerous because it makes it easy to confuse structure with coincidence. The purpose of this section is to translate the chapter’s technical material into a reality check: what tends to work in practice, what tends to fail, and why. The tone here is not cynical. It is disciplined. Markets are adversarial in the sense that they punish overconfident inference. The best research workflows therefore adopt habits that reduce degrees of freedom, establish strong baselines, and treat deep nets as *incremental tools* rather than as a magic upgrade.

A useful way to think about practical patterns is as a sequence of gates. First, define a forecasting problem that is implementable and time-safe. Second, establish baselines that are hard to beat and easy to audit. Third, only escalate to deep nets if there is a reason—an explicit hypothesis about nonlinearity, interaction structure, or representation learning that the baselines cannot express. Fourth, when deep nets do help, insist that the improvement is stable across folds and seeds and survives sanity checks. Fifth, when considering deployment, assume drift and define triggers for retraining and monitoring. This section focuses on these gates.

13.11.1 Start with strong baselines (linear and tree-free baselines within scope)

The most common deep-learning anti-pattern in trading is to begin with a large neural model, tune it until it looks good, and only then ask what the result means. This is backwards. In a governance-native research process, we begin with strong baselines because they establish what “easy” performance looks like and because they force us to justify complexity.

Within this book’s constraints and scope, the baseline family is deliberately “tree-free” and simple. That is not because trees are unimportant in practice, but because Chapter 13 is in the deep-learning arc, and we want baselines that are conceptually continuous with Chapters 11–12 and that do not introduce an entirely different modeling paradigm. The baselines should be:

Regularized linear models. Ridge and LASSO (Chapter 12) are still the first line. They are strong precisely because they are hard to fool: they impose smoothness and sparsity, they train quickly, and their failure modes are interpretable. If a neural net cannot beat a well-tuned ridge model under walk-forward evaluation, it is not earning its complexity. In trading, ridge

is often surprisingly competitive because many signals are approximately linear in engineered features and because the true signal-to-noise ratio is low.

Simple nonlinear baselines without representation learning. A small MLP with one hidden layer can serve as the minimal nonlinear baseline. It introduces nonlinearity and interactions but remains relatively easy to train and audit. Importantly, it forces you to test whether the benefit of “deepness” is real, or whether a shallow nonlinear function is sufficient.

Naive forecasters and invariance baselines. A serious research process also includes naive baselines: predicting zero return, predicting the mean return, predicting the last observed return, or predicting sign persistence. These are not jokes; they are anchors. If a deep net improves over ridge but fails to improve over a naive volatility-scaled heuristic, that matters. Trading systems must beat naive heuristics net of costs.

Baseline discipline is about degrees of freedom. Baselines reduce degrees of freedom because they limit the ways we can overfit. In deep learning, there are many ways to get a “better” validation score: change architecture, tweak learning rates, adjust normalization, adjust early stopping. Baseline discipline prevents this from becoming an unbounded search. It also provides a sanity check: if the deep model’s improvement is tiny relative to the baseline and unstable across folds, it is likely noise.

A practical rule: the baseline must be hard to beat. If your baseline is weak, your deep net’s improvement means nothing. Therefore, baseline construction must be competent: correct time-aware splits, correct normalization, disciplined hyperparameter selection. The goal is not to make the deep net look good; it is to make the baseline a fair and strong competitor. Only then does “beating the baseline” become evidence rather than theater.

13.11.2 When deep nets help: nonlinearity, interactions, and representation learning

Deep nets can help in trading, but not for the reasons most people hope. They rarely help because they “discover secret patterns” in prices. They help when the forecasting problem has structural properties that deep networks are well-suited to express and when the data pipeline provides enough signal for those properties to matter.

Nonlinearity beyond engineered features. Many financial relationships are nonlinear in raw variables. For example, risk effects can be nonlinear (volatility thresholds), and microstructure effects can be nonlinear (liquidity regimes). If we provide features that contain such information but the mapping to returns is nonlinear, a neural model can capture it. The key is that the nonlinearity must be stable across time. If the nonlinearity exists only in one regime, the model may overfit.

Feature interactions. Interactions are often the practical case. A feature may be predictive only when another feature is in a certain range. For example, a short-term momentum signal might work only when volatility is moderate, not when volatility is extreme. Linear models can represent interactions only if we manually include interaction terms. Neural nets can learn interactions automatically. In this sense, deep learning can reduce feature engineering burden: instead of explicitly creating every pairwise interaction, we allow the network to discover a subset of them. The discipline remains: the discovered interactions must generalize.

Representation learning on rawer inputs. Deep nets are most distinctive when they learn representations rather than merely combining hand-engineered features. A 1D CNN can learn local motifs in return sequences; an RNN can learn a state representation; a Transformer can learn non-local dependencies. In trading, representation learning can help when:

- we have a reasonably large dataset (relative to model capacity),
- the raw input contains structure that feature engineering did not capture,
- the structure is stable enough to survive walk-forward evaluation.

This is more plausible in settings with rich data: intraday microstructure, multi-asset panels, or alternative data streams. It is less plausible in sparse, low-frequency settings with small sample sizes.

Learning conditional behavior. A deep net can be viewed as learning a conditional mapping:

$$\hat{y}_t = f_\theta(X_t), \quad (13.114)$$

where X_t includes both signal features and context features. The model can learn conditional behavior: “in this context, trust this signal; in that context, ignore it.” This is often what practitioners mean when they say deep nets help with non-stationarity. The important caveat is that the conditioning variables must themselves be observable and stable. Otherwise, the model is simply fitting regime noise.

Where the improvement tends to show up in metrics. In many practical cases, deep nets do not dramatically improve MSE. Instead, they improve ranking metrics (IC, rank IC) or improve calibration of probabilities. This is consistent with the idea that deep nets learn better *ordering* rather than precise magnitude forecasts. In trading, ordering can be economically valuable. Therefore, the deep net’s contribution should be evaluated with metrics aligned to the trading decision.

The “small-but-stable” improvement is the realistic target. A realistic expectation in many trading domains is not a large uplift, but a small improvement that is stable across time and seeds. Deep learning is valuable when it makes the forecast signal more consistent, less noisy, or more robust across regimes. The moment you find yourself chasing large in-sample gains, you are likely tuning into noise.

13.11.3 When deep nets hurt: small data, unstable regimes, and tuning degrees of freedom

Deep nets often hurt in trading because the problem violates the conditions under which deep learning thrives. The model class is too rich relative to the data and relative to the stability of the environment. The result is a model that appears to work until it meets new conditions, at which point it fails.

Small data and effective sample size. Even if you have thousands of daily observations, the effective sample size can be much smaller due to autocorrelation and overlapping labels. If you enforce time-aware splits, the training set in each fold may not be large. Deep nets trained in this setting often become high-variance estimators: they fit one particular realization of noise. Ridge regression, by contrast, is designed to be stable in precisely this low-signal regime.

A telling symptom is seed sensitivity: one seed produces good results, another seed produces poor results. If the signal were strong, initialization would matter less. In small data, seed sensitivity is a warning that the model is not learning a robust relationship.

Unstable regimes. Markets change. A model trained on one regime may not transfer. Deep nets can internalize regime-specific patterns in subtle ways. For example, a CNN might learn a pattern that is predictive in one volatility environment but not in another. If regime shifts are frequent, the net becomes a regime detector rather than a return predictor, and its out-of-sample performance becomes fragile.

This is why Chapter 14 exists: if regime structure is central, we should model it explicitly rather than hoping a deep net will implicitly handle it. In Chapter 13, we treat regime dependence as a diagnostic: if performance varies strongly by regime proxies, that is evidence that a regime-aware approach may be necessary.

Tuning degrees of freedom and selection bias. Deep learning encourages experimentation: change architecture, change learning rate, change dropout, change schedule, change normalization. Each change is a statistical test. If you do enough tests, one will look good by luck. This is not a moral failure; it is a statistical inevitability.

The practical anti-pattern is to tune until the results look good and then stop. A governance-native workflow combats this by limiting tuning budgets, using nested validation, and reporting results across folds and seeds. If you cannot describe how many configurations you tried, you cannot quantify selection bias. In that case, your “best” model is likely an artifact of search.

Implicit leakage amplified by deep nets. Deep models are extremely good at exploiting leakage. A tiny misalignment or a normalization mistake can produce large apparent predictability. In finance, the most common deep-learning “success” stories that fail replication are leakage stories. This is why Chapter 13 insists on causality gates and canary tests. If your pipeline does not include these, you should assume that the result might be leakage until proven otherwise.

Overconfident predictions and sizing risk. Even when a deep model improves accuracy, it can produce poorly calibrated outputs: probabilities near 0 or 1 that are not justified. If those outputs are used for sizing, the strategy can take excessive risk. This is a uniquely dangerous failure mode: performance metrics can look good in sample while risk behavior is unstable. Therefore, calibration diagnostics belong in the evaluation, and conservative clipping or calibration methods may be needed later (still within forecasting scope).

13.11.4 Deployment cautions: drift monitoring and retraining triggers (preview; Ch. 23)

A model that passes walk-forward evaluation is still not ready for deployment. Deployment introduces new constraints: inference must be reproducible, data must arrive reliably, and the model must be monitored for drift. Chapter 23 will treat infrastructure and implementation pathways; here we preview the essential deployment cautions that follow from the deep-learning reality check.

Assume drift; design monitoring from the start. In markets, drift is normal. Therefore, a deployed model should be accompanied by drift monitoring that tracks:

- input distribution drift (feature means/variances, quantiles),
- prediction distribution drift (score histograms, entropy for classifiers),
- performance drift (realized IC, hit rates, calibration),
- regime proxy drift (volatility regimes, trend regimes).

Monitoring does not prevent drift; it detects it early. The governance implication is that the pipeline must store enough artifacts in production to compute these monitoring statistics.

Retraining triggers should be defined ex ante. A common anti-pattern is to retrain when performance “feels bad.” A disciplined approach defines retraining triggers in advance, such as:

- feature drift exceeding thresholds,
- degradation in IC or rank IC beyond confidence bounds,
- calibration error exceeding a threshold,
- detection of a regime shift according to a proxy.

These triggers must be tied to the evaluation logic used in research so that retraining decisions are consistent with the evidence base.

Beware of feedback loops. Once a model is deployed, its signals can influence trading decisions, which can influence execution, which can influence observed returns and the data used for retraining. This creates feedback loops. While Chapter 13 does not model execution, the governance stance is to log decision-time inputs, predictions, and realized outcomes so that post-hoc analysis can separate model drift from market drift and from execution effects.

Reproducible inference is non-negotiable. In deployment, the most common failure is not that the model is wrong, but that the inference pipeline differs from training: different scaling, different windowing, different missingness handling. Therefore, the model registry and preprocessing contract (from the governance checklist) are deployment prerequisites. A retrained model must be comparable to the previous model, and that comparability depends on consistent inference.

Small models often deploy better. A practical observation is that smaller, well-regularized models often deploy more reliably than large deep nets. They are easier to monitor, less sensitive to small pipeline changes, and faster to retrain. In high-frequency settings, latency and reliability dominate. Even in daily settings, operational simplicity matters. This is not an argument against deep learning; it is an argument for humility: choose the simplest model that meets the performance objective with stability.

The realistic deployment posture. The correct posture for deploying deep nets in trading is cautious:

- deploy as a component, not as a sole decision-maker;
- monitor drift continuously;
- retrain on a schedule plus triggers;
- keep baselines running in parallel as canaries;
- maintain rollback capability.

This posture reflects the reality that markets can change faster than modeling cycles, and that governance is about resilience as much as about performance.

Closing reality check. The practical patterns and anti-patterns can be summarized as a philosophy: deep learning is most useful when it is treated as a controlled, incremental extension of a strong supervised forecasting discipline. Start with baselines that are hard to beat. Escalate only when you have a plausible hypothesis about nonlinearity, interactions, or representation learning. Expect small, stable improvements rather than dramatic wins. Assume that tuning degrees of freedom and leakage are your main adversaries. And if you move toward deployment, treat drift monitoring and retraining as part of the model design, not as an afterthought.

This reality check is not pessimism. It is what allows deep learning to be used responsibly in trading. Without it, deep nets become a machine for producing seductive but fragile numbers. With it, they can become a genuinely useful tool: a stable, auditable forecasting component that earns its place by surviving time-aware evaluation, diagnostics, and governance scrutiny.

13.12 Conclusion

Deep learning enters this book at a moment of deliberate escalation. Chapters 11 and 12 established the supervised forecasting frame and the discipline of time-aware selection. Chapter 13 then introduced neural function classes, training dynamics, architectures, regularization,

diagnostics, and governance-native evidence standards. The overall message is not that neural nets are superior, nor that they are doomed. The message is that deep learning is a powerful tool whose value in trading depends far more on *process* than on *hype*. When the process is weak, deep learning is a remarkably efficient engine for manufacturing false confidence. When the process is strong—time-aware validation, strict causality gates, conservative tuning budgets, robust diagnostics, and auditable artifacts—deep learning can make a measurable, incremental contribution to forecasting signals and to the stability of a multi-component system.

13.12.1 What deep learning can reliably contribute to trading systems

Within the scope of this book’s supervised-learning arc, deep learning can reliably contribute in three related ways. First, it can express nonlinear mappings and interactions without requiring an explosion of manual feature engineering. Many trading signals are conditional: a feature matters only under certain volatility conditions, only when trend strength is present, or only when multiple indicators align. A well-regularized network can represent these interactions compactly and can turn a set of reasonably defined, causality-safe inputs into a smoother scoring function than a linear model can produce.

Second, deep learning can sometimes improve *representation* when the input is closer to raw market structure—windowed returns, multi-scale patterns, or multi-asset panels—rather than being fully pre-digested into a small set of handcrafted factors. CNN-style architectures can learn local motifs across time; sequence models can learn state summaries that behave like soft filters. In practice, the reliable value is not mystical pattern discovery; it is the ability to compress a window of market history into a stable, low-dimensional internal state that supports prediction more consistently across nearby time periods.

Third, deep learning can contribute to *robustness* when used correctly. Regularization (weight decay, dropout, careful normalization), model averaging across seeds, and conservative early stopping can yield forecasts that are less sensitive to small input noise and less unstable across retraining cycles. In trading, where the signal-to-noise ratio is low, robustness is not a luxury; it is often the main economic value of a forecasting model. A forecast that is slightly less accurate in a point metric but far more stable across folds and seeds can be more useful than one that occasionally looks brilliant and often fails.

All of these contributions are conditional. They require that the data pipeline is correct, that scaling is fitted on training only, that splits respect time, and that the evaluation does not smuggle future information through normalization, batching, or label overlap. Deep learning’s reliable contribution is therefore best described as: *given a disciplined forecasting setup, deep nets can expand the hypothesis class in a controlled way and sometimes deliver small, stable improvements in ranking, calibration, or robustness.*

13.12.2 What deep learning cannot guarantee in markets

Deep learning cannot guarantee an edge because markets do not guarantee stationarity. A neural network trained on historical data learns a relationship that held *in that data-generating environment*. In markets, the environment changes. Participants adapt, policy shifts, volatility regimes change, and microstructure evolves. A deep net can be more flexible than a linear model,

but flexibility does not equal foresight. If the underlying relationship disappears, the network cannot conjure it back.

Deep learning also cannot guarantee protection against selection bias. The tuning surface is large: architectures, optimizers, learning rates, schedules, regularizers, and preprocessing variants. Without strict budgets and nested validation discipline, it is easy to search until something looks good. The resulting model may be the winner of a search process rather than a robust estimator. This is not a deep-learning-specific sin; it is a general statistical fact. Deep learning simply makes it easier to commit because it offers more knobs.

Furthermore, deep learning cannot guarantee immunity to leakage. On the contrary, deep models are excellent at exploiting small leaks. A subtle misalignment, an accidental use of future data in normalization statistics, a batch boundary that mixes periods, or a label horizon overlap that contaminates the evaluation boundary can generate impressive performance that disappears in production. Therefore, the absence of explicit causality gates and leakage canaries means that even visually plausible results should be treated skeptically.

Finally, deep learning cannot guarantee economic profitability even if it improves forecasting metrics. Metrics are proxies for decisions. A small improvement in IC may or may not survive transaction costs, slippage, and portfolio constraints (later chapters). A model can be accurate in sign but poorly calibrated, leading to poor sizing. A model can rank well but concentrate risk in fragile tails. Deep learning operates at the forecasting layer; converting forecasts into robust P&L requires the portfolio, sizing, and execution layers that follow in the book’s arc.

13.12.3 A minimum standard for claiming a neural net “edge”

Because deep learning can impress by accident, a minimum standard for an “edge” claim must be conservative and evidence-based. At minimum, the claim should satisfy the following conditions.

First, the edge must be demonstrated under time-aware evaluation: walk-forward folds with explicit date boundaries, and embargo logic where label overlap or boundary dependence could inflate results. Fold-wise metrics must be reported, not only averages, and performance must be reasonably stable across folds rather than driven by a single favorable period.

Second, model selection must be disciplined: hyperparameters chosen only using training and time-consistent validation within each fold (or an equivalent development period), with a declared tuning budget and without using the out-of-sample test block for early stopping decisions or for iterative tuning. If multiple configurations were tried, that fact must be reported, and the selection process must be documented.

Third, causality gates must be enforced mechanically. This means explicit tensor alignment assertions, label shift assertions, and normalization proofs that scaling parameters were fitted on training only and applied forward. In addition, at least one leakage canary should be run (random labels, feature permutation, or an intentional leak test) to demonstrate that the pipeline would detect or reveal suspicious performance inflation.

Fourth, robustness must be demonstrated across randomness. Deep nets are sensitive to seeds. Therefore, the claim should include results across a small set of predetermined seeds, reporting the mean and dispersion. Selecting a single lucky seed is not an edge; it is an anecdote.

Fifth, the evidence package must include diagnostics: learning curves that justify early

stopping, error stratification by simple regime proxies, and at least one sensitivity analysis showing that predictions are not wildly unstable under small perturbations or missingness. These diagnostics do not prove the edge, but they reduce the probability that the model is exploiting brittle artifacts.

Finally, the claim must be bounded. A publication-quality statement should specify exactly what was shown (e.g., stable positive IC over specified folds and instruments under specified horizons) and explicitly state limitations (non-stationarity, potential costs not modeled, regime dependence). In trading, credibility often comes from restraint: claiming less, but proving it more carefully.

13.12.4 Transition to Chapter 14: Regime Detection and HMMs

Chapter 14 introduces explicit latent-state structure: instead of asking a single model to fit all times, we model markets as switching between regimes. This will let us interpret instability in deep-learning performance as a data-generating feature rather than a nuisance, and it will provide a disciplined way to segment evaluation, features, and risk controls. Concretely, where Chapter 13 treated regime variation as a diagnostic (performance differs by volatility or trend proxies), Chapter 14 will formalize regimes as latent states with transition dynamics, so that the research workflow can condition forecasts, training windows, and risk rules on an inferred market state. The conceptual shift is from “one flexible predictor for all periods” to “a structured view of market time,” in which deep models become components that operate within and across regimes rather than pretending the market is a single stationary environment.

Chapter 14

Regime Detection & HMMs

Abstract

Financial time series are rarely stable long enough for a single forecasting model, a single set of risk parameters, or a single strategy narrative to remain valid. This chapter introduces the regime perspective: markets are modeled as switching among a small number of latent states, each with its own return distribution, volatility scale, and persistence. Hidden Markov Models (HMMs) provide a disciplined and interpretable way to operationalize that idea. We present the HMM as a two-layer generative model: a Markov chain for the unobserved regime and an emission model for observed data such as returns, volatility proxies, drawdowns, or spreads. The chapter emphasizes the distinction between *filtering* (online regime beliefs using information available up to time t) and *smoothing* (retrospective inference using the full sample), and treats that distinction as a central governance issue: a regime detector is only tradable if its signal is filtered and causality-safe. We develop the forward–backward logic for inference, the EM (Baum–Welch) procedure for parameter learning, and practical design choices for the number of regimes and emission families. Finally, we connect regime probabilities to trading decisions through risk overlays, strategy switching, and uncertainty-aware position sizing, and we specify evaluation protocols that test whether regimes add value beyond simpler baselines while avoiding leakage, overfitting, and narrative fallacies.

14.1 Position in the Book

14.1.1 Prerequisites from earlier chapters

Chapter 14 arrives at a moment in the book where we have already learned to be suspicious of stability. In Part I (Chapters 1–5) we established the basic logic of algorithmic trading: markets generate data through a mixture of structure and noise, our job is to build decision rules that map information available *at decision time* into trades, and our responsibility is to preserve time order so that we never confuse what was knowable with what is only obvious in hindsight. Those chapters also gave us the practical grammar of trading data: how prices become returns, how sampling frequency changes the meaning of volatility, why missing values and corporate actions matter, and how features must be engineered with causal rolling windows rather than “whole-sample” computations. If you cannot state precisely what information set is available at time t and how your feature vector is constructed from that information set, you are not ready for regimes, because regime inference magnifies leakage risks: it is extremely easy to infer “states” that are really just future knowledge sneaking into the present.

In Part II (Chapters 6–10) we learned how to evaluate strategies with simulation discipline. The backtesting chapter (Ch. 06) is the immediate operational prerequisite: regime models are not merely statistical objects; they are *pipelines* that must be refit, updated, and traded according to an explicit event clock. Trend following (Ch. 07) and mean reversion (Ch. 08) trained our intuitions about conditional performance: trend often “works” during persistent moves and fails during range-bound churn; mean reversion often thrives in stable microstructure and fails during breaks and crashes. Factor and cross-sectional ideas (Ch. 09) reinforced that relative-value signals can be regime-dependent too (dispersion, correlation, and crowding change over time). Volatility modeling and risk targeting (Ch. 10) added a critical piece: volatility is not just an input; it is a *state variable* that shapes exposures, drawdowns, and survivability. If you already think of “high vol” and “low vol” as qualitatively different trading environments, you are already thinking in regimes, even if you have not yet formalized it.

Finally, Part III up to Chapter 13 (Chapters 11–13) explains why Chapter 14 is necessary rather than decorative. Supervised learning for returns (Ch. 11) showed that predictive relationships are weak, unstable, and easy to overfit; regularization and model selection (Ch. 12) made instability explicit by forcing us to confront the difference between in-sample fit and out-of-sample credibility; and deep learning (Ch. 13) pushed model capacity higher while simultaneously highlighting a harsh reality: even sophisticated models can fail not because they are poorly optimized, but because the data-generating process itself changes. That is the conceptual bridge. Regimes are not a trick to make forecasting magically easier. They are a way to *name* and *model* the time-variation that earlier chapters taught us to fear. When a model’s performance drifts, when residuals change character, when risk explodes, or when correlations reorganize, the regime perspective says: perhaps the market is not one process; perhaps it is several processes that we visit over time.

14.1.2 What this chapter adds

What Chapter 14 adds is an explicit latent-state layer to our trading worldview, along with a disciplined method—Hidden Markov Models (HMMs)—for turning that worldview into a reproducible, auditable, and tradable component of a system. The central idea is simple but powerful: instead of assuming that returns (or features) are generated from a single stationary distribution, we assume there exists an unobserved regime variable $s_t \in \{1, \dots, K\}$ that evolves over time with persistence, and that the observable data are generated conditionally on that regime. This is the smallest mathematical step that allows “market mood” to be an object we can estimate rather than a story we tell after the fact. It also forces clarity: a regime model is only useful to traders if it produces a *belief* about the current state that is updated online using information available up to time t . This is why Chapter 14 elevates the distinction between *filtering* and *smoothing* from a technical detail into a governance principle. Filtering corresponds to $p(s_t | x_{1:t})$, the probability of the regime given data up to time t . Smoothing corresponds to $p(s_t | x_{1:T})$, the probability given the entire sample. Smoothing is often visually appealing and diagnostically helpful, but it is not a tradable signal unless you can reproduce it without future data. A surprisingly large fraction of “regime” papers and dashboards fail precisely here: they present beautifully segmented histories that cannot be achieved in real time.

Chapter 14 also adds the mechanics required to implement HMMs from first principles in a trading pipeline. We specify the model components (transition probabilities and emission distributions), explain the forward algorithm for online inference, and develop the backward pass for smoothing as a research tool. We introduce EM (Baum–Welch) parameter learning in a way that emphasizes deterministic initialization, convergence diagnostics, and failure modes. We discuss how to choose the number of regimes without turning model selection into a narrative machine, and we show how regime probabilities can be used in three practical ways that are common in professional systems: (i) *risk overlays* that change exposure as uncertainty and volatility shift, (ii) *strategy switching* that tilts between families such as trend and mean reversion depending on the inferred environment, and (iii) *regime-conditioned evaluation* that attributes performance and drawdowns to states rather than averaging everything into a single number. The goal is not to build a “regime oracle.” The goal is to add a transparent, testable module that can reduce the cost of non-stationarity by making it explicit in the system’s logic.

Equally important, Chapter 14 adds a new standard of experimental discipline. Because regime models sit between raw data and trading actions, they are a prime location for subtle leakage and overfitting. The chapter therefore formalizes causality gates (filtered-only invariants), refit cadences (what gets re-estimated when), and audit artifacts (posteriors, transition matrices, likelihood traces) as part of the baseline deliverable. In other words: by the end of Chapter 14, “I found regimes” is not a conclusion; it is a claim that must come with evidence that the regimes were inferred in a time-aware way, that they are stable enough to be useful, and that they improve decisions relative to simpler baselines.

14.1.3 What this chapter deliberately postpones

Chapter 14 is deliberately scoped to regime detection and the translation of regime beliefs into *simple* decision overlays. It does not attempt to solve the entire portfolio and execution problem,

and it does not introduce reinforcement learning, agentic systems, or deployment architectures. This boundary is not arbitrary; it is pedagogical and methodological. Regime inference is already a high-leverage tool, but it is also a high-risk source of self-deception. Before we allow regimes to drive sophisticated capital allocation, we must first learn to estimate them causally, validate them honestly, and interpret them conservatively.

For that reason, we postpone the deeper portfolio consequences to Chapters 16 and 17. Portfolio construction and optimization (Ch. 16) will treat regime probabilities as inputs into covariance and expected return estimates, and will address constraints, diversification, and objective design. Position sizing, leverage, and risk management (Ch. 17) will formalize how regime uncertainty should affect gross and net exposures, drawdown controls, and survival constraints. We also postpone the gritty realities of execution to Chapter 18. Even a perfectly estimated regime belief can be monetarily irrelevant if the strategy requires frequent switching and the transaction costs dominate. Microstructure, slippage, and market impact (Ch. 18) are the place where “regime churn” becomes a P&L problem rather than a charting curiosity.

We also postpone reinforcement learning (Ch. 19) and multi-strategy systems (Ch. 20). Regimes are often used as a state representation for RL agents, but using them responsibly requires first understanding the difference between filtered and smoothed states, the stability of state definitions under refits, and the governance of online updates. If we introduce RL too early, we risk building agents that learn to exploit leakage or that overfit to regime labels that are not stable out of sample. Likewise, agentic architectures (Ch. 24) and the capstone system (Ch. 25) will integrate regime detection as one module among many—data governance, feature pipelines, forecasting models, risk overlays, execution logic, monitoring, and human oversight. Chapter 14’s job is to make regimes *real* in a disciplined sense: defined mathematically, inferred causally, evaluated honestly, and connected to decisions without pretending that they are a universal solution.

In short, this chapter is a hinge in the book’s narrative. Earlier chapters taught us how to build and evaluate models under time. Chapter 14 teaches us how to acknowledge that time itself changes the problem by introducing latent state. Later chapters will scale this latent-state awareness into portfolio design, execution-aware implementation, and ultimately AI-native systems. But the sequence matters: we first learn to see regimes without hallucinating them, and only then do we let regimes steer capital.

14.2 Motivation: Why “Regimes” Exist in Trading Systems

14.2.1 Non-stationarity as the default, not the exception

A useful starting point for regime thinking is not a sophisticated statistical argument; it is a trader’s basic lived constraint: the market you trade this year is not the same market you traded two years ago, even if the ticker symbols are unchanged. The vocabulary that academics often use is *non-stationarity*, but the phenomenon is broader than a drifting mean or a changing variance. What changes in markets is the full data-generating context: who the marginal buyer is, which constraints bind, what information arrives and when, what instruments are dominant in hedging flows, and what leverage is cheap or expensive. Even at the level of a single return

series, you can see this in a way that does not require any advanced methods: volatility clusters, correlations jump, drawdowns concentrate into episodes, and “quiet” markets punctuate into sudden discontinuities. The act of writing a trading rule is an implicit bet that some relationship is sufficiently stable to exploit; the act of running a trading business is the recognition that relationships degrade and must be monitored, adapted, or replaced.

In earlier chapters we treated this instability as a practical nuisance that our research process must respect. We learned to build time-aware features, to avoid leakage, to validate with walk-forward splits, and to test robustness across time folds. Regime thinking turns the perspective around: instability is not merely an evaluation concern; it can be treated as part of the model. This is an important philosophical shift. If markets were truly stationary, then we could pool data indefinitely and our estimates would become more precise over time. In practice, adding more history often makes the model *worse* because old observations come from an environment that no longer exists. The central motivation for regimes is therefore not that we enjoy complexity; it is that the simplest stationary models systematically misrepresent the world. They average across qualitatively different market conditions and, by doing so, they produce parameter estimates that are “correct” for no period in particular.

One way to see regimes without any mathematics is to think in terms of conditional behavior. Many strategies that are defensible in principle have a conditional performance profile: they do well in some environments and poorly in others. Trend following is the classic example: it tends to benefit from persistent directional moves and can bleed in choppy ranges. Mean reversion has the opposite temperament: it can harvest oscillations in stable microstructure and can be annihilated by breakouts and crash dynamics. Volatility targeting assumes that estimated risk is informative about future realized risk; that assumption is not uniformly true across all periods and often fails at regime boundaries. Even “buy and hold” behaves differently when inflation is anchored and rates are stable than when inflation surprises repeatedly and discount rates reprice violently. If conditional behavior is the norm, then it is natural to ask whether we can *infer* the environment and adapt exposures accordingly. That question is the seed of regime detection.

The important caution is that regimes are not metaphysical entities waiting to be discovered. Markets do not publish a label saying “Regime 1: risk-on, Regime 2: risk-off.” Regimes are a modeling device: a small set of latent states that we choose to summarize complex, shifting conditions. The goal is to compress a high-dimensional environment into an interpretable state variable that is useful for decisions. That compression is valuable because trading systems must act under uncertainty, on a schedule, with finite complexity. A good regime model does not “explain everything.” It explains enough of the instability that our decisions become more robust.

14.2.2 When a single model fails: instability signatures from Chapter 13

Chapter 13, on neural nets and deep learning in trading, is a convenient reference point for regime motivation because deep models are powerful enough that their failures are often diagnostic. When a simple linear model underperforms, we can always argue that it lacked capacity. When a well-tuned deep model underperforms, it becomes harder to blame the architecture and easier to suspect the environment. This is not a claim that deep learning is useless; it is a claim that, in markets, model capacity is rarely the binding constraint. The binding constraint is that the

mapping from features to returns is weak, noisy, and time-varying.

There are several instability signatures that appear repeatedly in practice and that should trigger regime thinking. The first is *performance drift*: a model that looks decent over one window slowly loses its edge, then occasionally recovers, then fails again. If you plot rolling information coefficients, rolling Sharpe, or rolling calibration curves, you often see long stretches of near-zero predictive power punctuated by short bursts of apparent signal. A stationary model interprets this as randomness around a stable mean. A regime view interprets it as the system visiting states where the relationship exists and states where it does not. In other words, the “average” effect is not what you trade; you trade the conditional effect.

The second signature is *residual structure*: errors are not i.i.d.; they come in patterns. A forecasting model’s residuals may show volatility clustering, sign asymmetry (systematically missing negative moves), or periods where the model is consistently too confident. In the deep-learning context, this often manifests as miscalibration: predicted probabilities or score magnitudes cease to match realized outcomes. You can also see the model becoming sensitive to small feature perturbations in certain periods but not others, a symptom that the model’s internal representation is struggling with a shifting data manifold.

The third signature is *feature instability*. In Chapter 12 we learned to fear multicollinearity and selection bias; in Chapter 13 we add another fear: in non-stationary data, feature importance is not stable. A feature that correlates with returns in one period may flip sign later, not because the feature became “wrong,” but because the economic mechanism behind it changed. For example, momentum may work when trend persistence is supported by macro flows and underperform when markets mean revert due to range-bound policy signals. Value spreads may matter when dispersion is high and underperform when crowding compresses opportunities. Volatility itself may be a risk premium proxy in one regime and a crash-warning variable in another. Deep models can sometimes learn these nonlinearities, but only if you give them enough data from each regime and only if the regime transitions do not destroy their learned mapping faster than training can adapt.

A fourth signature is *catastrophic breakdown*: long periods of small gains interrupted by rare, large losses that dominate the distribution. This is not unique to deep learning; it is a general property of many trading strategies. But deep learning can make the pattern more dangerous because the model may appear stable right up to the break, and the break may correspond to a structural change in volatility and correlation. Many strategies “work” in a quiet regime and then fail during liquidity stress. That is not an argument to avoid the strategy; it is an argument to explicitly model the possibility of switching into a stress regime and to control exposure as the probability of stress increases.

These signatures do not prove that regimes exist as discrete states. They prove something weaker but sufficient: the one-model assumption is fragile. Regime modeling is one way to address that fragility without pretending that we can perfectly forecast returns. The point is not “predict more.” The point is “fail better”: detect when your assumptions are breaking and adapt exposures before the P&L forces adaptation.

14.2.3 Regimes as a modeling choice (not an oracle)

The language of regimes can seduce us into thinking we are discovering hidden truths about markets. A healthier stance is to treat regimes as a design choice in the same family as feature choice or strategy choice: we define a small set of states and we evaluate whether that state variable improves decisions. In that sense, regime detection is closer to representation learning than to prophecy. We are choosing a representation of time-varying conditions that is compact enough to use and rich enough to matter.

A key implication is that there is no single “correct” regime definition. The right regimes depend on the decision you are trying to improve. If you are managing risk, a two-state “high vol vs low vol” model might already be valuable. If you are switching between trend and mean reversion, you might want states that track persistence versus choppiness. If you are trading carry, you might want states that capture crash risk versus calm accumulation. If you are operating across asset classes, you might want states that reflect correlation regimes or funding liquidity. Different regime definitions can be simultaneously valid because they summarize different aspects of the environment.

This viewpoint also helps resolve a common confusion: why use discrete regimes at all? Why not let parameters drift continuously? Continuous-time parameter drift models exist, and in some contexts they are preferable. But discrete regimes offer two pragmatic advantages that matter in trading systems. First, they are interpretable: you can communicate “we are likely in a stress regime” to a risk committee more easily than “our latent volatility parameter increased by 0.37 in a diffusion process.” Second, they impose a regularization structure: by forcing the world into a small set of states, you reduce the degrees of freedom and can sometimes generalize better with limited data. Discrete regimes are an explicit bias: we accept some misspecification in exchange for robustness and usability.

The “not an oracle” principle must be stated more sharply: regime models can be wrong in systematic ways, and their wrongness can be expensive. A model may invent regimes to fit noise, especially if you allow too many states. It may confuse volatility clustering with a true switch in underlying dynamics, leading to excessive switching. It may become unstable under refits, producing regimes that cannot be compared across time. It may detect a regime boundary only after the P&L damage is already done, because the evidence for switching accumulates slowly. It may also produce a false sense of control: the chart of colored regimes looks like understanding, but in real time the posterior probabilities may be ambiguous and noisy.

Therefore, treating regimes as a modeling choice implies a specific evaluation posture. We do not ask “are the regimes true?” We ask “do regime beliefs improve decisions under a causality-safe pipeline?” This changes the burden of proof. The deliverable is not a pretty segmentation of history; it is a time-aware signal $p(s_t \mid x_{1:t})$ with documented refit rules, and evidence that using that signal improves risk-adjusted outcomes relative to simpler baselines. If the regime model does not pass that test, we should not deploy it, regardless of how plausible the narrative sounds.

14.2.4 Trading use-cases: hedging, risk-on/risk-off, volatility states, carry vs crash

The motivation for regimes becomes concrete when we tie it to specific decisions that traders and risk managers actually make. The first and most common use-case is *risk scaling and hedging*. Many trading books describe “risk-on/risk-off” as a narrative, but a regime detector can turn it into a probability that modulates exposure. When the probability of a stress regime rises, the system can reduce gross leverage, tighten drawdown controls, widen stop thresholds, or increase hedges. Importantly, this does not require perfect classification. Even a noisy probability can be useful if it shifts exposures in a direction that improves tail behavior. The point is not to predict the crash day; it is to reduce vulnerability as conditions deteriorate.

A second use-case is *volatility states*. This is the simplest regime concept and often the easiest to validate. If realized volatility is clustered, then a latent state model that distinguishes high and low volatility can produce a smoother, more robust estimate of “current risk” than a single rolling window. This can improve volatility targeting, option hedging schedules, and margin-aware exposure management. It can also improve the interpretation of strategy performance: a strategy may have modest average returns but very different drawdown characteristics across volatility regimes. Knowing this can change how you allocate capital and how you set risk limits.

A third use-case is *strategy switching*. This is where regimes can add alpha-like value, but also where they can do the most harm if mishandled. The idea is to allocate more to trend strategies when the market is persistent and to mean reversion when the market is range-bound, or to allocate differently across factors depending on dispersion and correlation regimes. The challenge is that switching costs are real and classification errors are inevitable. A regime model that churns between states will destroy performance through costs and whipsaws. Therefore, effective switching use-cases often rely on probabilistic blending rather than hard switches: allocate weights as smooth functions of regime probabilities, impose hysteresis (require sustained evidence to switch), and cap turnover.

A fourth use-case is *carry versus crash*, which is a particularly instructive regime archetype. Carry strategies—selling volatility, harvesting term premia, earning credit spread—often produce steady small gains but are vulnerable to rare, severe losses. This payoff profile is naturally regime-like: a calm regime where carry accrues and a crash regime where carry is punished. A regime detector can serve as a crash-risk proxy that reduces exposure when crash probability rises. Again, the detector does not need to predict the exact crash; it needs to reduce the fraction of time you are maximally exposed when the environment is shifting toward stress.

Finally, regime thinking helps with *communication and governance*. Trading systems fail not only because of bad models but because of bad decision processes under pressure. If a system can say, in probabilistic terms, “the market looks more like state A than state B,” then a risk manager can tie pre-agreed actions to that belief: tighten limits, reduce leverage, increase hedges, or pause certain signals. This is one of the understated benefits of regimes: they create an interface between statistical inference and operational control. In a professional setting, that interface is as valuable as any incremental Sharpe.

To summarize the motivation: regimes exist in trading systems because markets exhibit persistent changes in behavior that make the single-model assumption fragile; because the failures

of high-capacity models reveal that instability is often environmental rather than architectural; because a small set of latent states can serve as a useful representation of time-varying conditions; and because many practical trading decisions—risk scaling, hedging, switching, and crash protection—are naturally conditional. The rest of this chapter is about making that intuition precise, tradable, and governable without pretending that regimes are a magic lens into the future.

14.3 A Minimal Formal Definition of a Regime Model

14.3.1 Observed process and latent state process

A regime model begins with an admission: what we observe in markets is not the thing we most want to model. We observe prices, returns, spreads, volumes, and a long list of engineered features. What we would like to model is the *environment* that generates those observables: the mix of participants, constraints, risk appetite, funding conditions, policy stance, and liquidity. We cannot measure that environment directly, so we represent it with a latent variable. The regime model is therefore a two-layer probabilistic story: a latent state process that evolves over time, and an observed process whose distribution depends on the current latent state.

Let $\{x_t\}_{t=1}^T$ denote the observed time series used for regime inference. In a minimal presentation, x_t can be a scalar return; in practical systems, it is often a vector of features, for example

$$x_t = (r_t, |r_t|, \hat{\sigma}_t, \text{drawdown}_t, \text{spread}_t, \dots),$$

where every component must be constructed causally (only using information up to time t). The goal of a regime model is not necessarily to predict x_t with high accuracy; it is to infer a low-dimensional summary s_t that captures persistent changes in the distribution of x_t .

Let $\{s_t\}_{t=1}^T$ denote the latent regime sequence, with $s_t \in \{1, 2, \dots, K\}$ for some small K . We interpret s_t as the regime at time t . The regime model specifies:

1. A *state evolution* model for s_t (how regimes persist and switch).
2. An *observation* (emission) model for x_t conditional on s_t (how observables behave within a regime).

This is intentionally minimal. We are not yet imposing a particular parametric form (Gaussian, Student- t , etc.), and we are not yet saying whether x_t is univariate or multivariate. The minimum structure we need is the factorization of the joint distribution into a state component and an observation component.

A widely used baseline is a first-order Markov evolution for s_t :

$$\mathbb{P}(s_t = j \mid s_{t-1} = i, s_{t-2}, \dots) = \mathbb{P}(s_t = j \mid s_{t-1} = i) \equiv A_{ij},$$

where $A \in \mathbb{R}^{K \times K}$ is a row-stochastic transition matrix (each row sums to one). We also specify an initial state distribution $\pi \in \mathbb{R}^K$:

$$\mathbb{P}(s_1 = i) = \pi_i, \quad i = 1, \dots, K.$$

The observation model specifies a family of conditional distributions

$$p(x_t \mid s_t = k; \theta_k),$$

where θ_k denotes parameters for regime k (e.g., mean and covariance in a Gaussian emission model). Collectively, the parameters are $\Theta = (\pi, A, \theta_1, \dots, \theta_K)$.

The regime model therefore defines a joint distribution for $(s_{1:T}, x_{1:T})$:

$$p(s_{1:T}, x_{1:T}; \Theta) = p(s_1; \pi) \prod_{t=2}^T p(s_t \mid s_{t-1}; A) \prod_{t=1}^T p(x_t \mid s_t; \theta_{s_t}) \quad (14.1)$$

$$= \pi_{s_1} \prod_{t=2}^T A_{s_{t-1}, s_t} \prod_{t=1}^T p(x_t \mid s_t; \theta_{s_t}). \quad (14.2)$$

This factorization is the “minimal formal definition” in the sense that once you accept (i) a discrete latent state and (ii) state-dependent observation distributions, you have already committed to the essential structure. Everything else—the exact emission family, the number of regimes, constraints, covariates, higher-order dynamics—is a refinement.

From the trading point of view, the object of interest is rarely the latent state itself (which we cannot observe), but rather the *belief state* or posterior distribution over regimes:

$$\gamma_t(k) \equiv \mathbb{P}(s_t = k \mid \mathcal{I}_t),$$

where $\mathcal{I}_t$ denotes the information set available at decision time t . This posterior is what we will feed into risk overlays, switching logic, or regime-conditioned evaluation.

14.3.2 Conditional independence assumptions

The apparent simplicity of the factorization above hides a set of strong conditional independence assumptions. These assumptions are not cosmetic; they define what the model is allowed to attribute to regimes versus what it is forced to treat as noise. They also define how we can compute inference efficiently.

The first assumption is the *Markov property* for the latent states:

$$s_t \perp\!\!\!\perp (s_{1:t-2}) \mid s_{t-1}.$$

Economically, this says that regime persistence and switching depend only on the most recent regime, not on the full history. This is a convenience assumption that often works as a good approximation because it captures persistence via large diagonal elements of A and captures switching behavior via off-diagonal elements.

The second assumption is *conditional independence of observations given the states*:

$$x_t \perp\!\!\!\perp (x_{1:t-1}, x_{t+1:T}) \mid s_t,$$

and more specifically, the model implies

$$p(x_{1:T} \mid s_{1:T}) = \prod_{t=1}^T p(x_t \mid s_t).$$

This says that, once we know the regime sequence, the observations are independent across time. In market terms, this is rarely literally true: returns have residual autocorrelation, volatility clusters, and microstructure effects create dependencies. The point is not that the assumption is perfectly accurate; the point is that the latent state is meant to capture the dominant persistent variation, leaving residual dependencies small enough that the model is useful.

There is also an implicit assumption about the direction of causality: the latent state influences the distribution of x_t , not the other way around. In the generative story, we first draw the state sequence according to π and A , and then we draw observations conditional on states. Of course, in the real world, our “state” is a proxy for many interacting forces, and the observed variables may feed back into those forces. The HMM framework abstracts away feedback and treats the state as exogenous to observation conditional on time. This is a modeling idealization. In practice, it can still be useful if the resulting posterior γ_t tracks meaningful changes in market behavior.

A further conditional independence assumption arises when x_t is multivariate. Suppose $x_t \in \mathbb{R}^d$ and we use, for example, a multivariate Gaussian emission

$$x_t \mid (s_t = k) \sim \mathcal{N}(\mu_k, \Sigma_k).$$

Then we are not assuming that components of x_t are independent; we allow correlation through Σ_k . However, we are assuming that *within* a regime the distribution is time-invariant (the same μ_k and Σ_k apply for all times when $s_t = k$). This is where regimes become a regularizer: instead of allowing parameters to drift at every time step, we require parameters to be constant within each state, and we allow changes only when the state switches.

Why do these assumptions matter? First, they make inference tractable. The forward–backward recursions and EM learning rely on the ability to factor the joint distribution into local terms. Second, they force interpretability. If we allow the observation model to depend on the entire past, or if we allow state transitions to depend on everything, we can fit almost anything but understand nothing. Third, they define what we consider a “regime.” A regime is a period over which the conditional distribution of observations is stable enough that it can be summarized with a small parameter set.

In trading systems, we must also translate these assumptions into implementation rules. For instance, if we engineer features that use future information (explicitly or implicitly), then even a perfectly implemented HMM will produce posterior probabilities that are contaminated by leakage. The conditional independence assumption does not protect you from bad features. Similarly, if we use smoothed probabilities (conditioning on the full sample) as if they were online signals, we violate the temporal direction of the model and import future observations into present decisions.

14.3.3 What a regime model can and cannot identify

The word “identify” is dangerous in trading because it carries a promise of truth. A minimal regime model can identify certain things in a limited, model-relative sense, but it cannot identify others, and misunderstanding this is one of the fastest routes to over-claiming. The first limitation is fundamental: the latent state is not observed. Therefore, any inference about s_t is necessarily probabilistic and model-dependent.

At best, an HMM identifies *clusters of observations in time* that are consistent with different emission distributions and connected by a persistent transition structure. If the data genuinely contain periods where volatility is higher, tails are fatter, correlations are larger, or mean behavior shifts, then a regime model can represent those differences as separate states. In such cases, the posterior γ_t can act as a useful summary of “which cluster we seem to be in now.” This is the operational meaning of identification for a regime model: it identifies a low-dimensional state that captures persistent distributional differences.

However, several non-identifiabilities arise immediately.

Label switching. If the model has K regimes, the labels $1, \dots, K$ are arbitrary. If we permute labels consistently (swap state 1 and state 2 everywhere), the likelihood is unchanged. Therefore, the model cannot identify which regime is “state 1” in any absolute sense. Interpretation requires you to impose a labeling convention (e.g., “state with higher volatility is called stress”), and stability across refits requires careful alignment of labels (often by sorting states by a statistic such as volatility).

Multiple explanations for the same data. A regime model can fit the same sequence of observations using different combinations of transition probabilities and emission parameters. For example, you can represent a long high-volatility episode as either (i) a highly persistent stress state (large diagonal transition probability) or (ii) a less persistent stress state that is revisited frequently (higher switching). These two explanations can produce similar likelihood but very different implications for online detection and for turnover in switching logic. This is why maximum likelihood is not enough; you must evaluate the decision consequences.

Regimes are not causes. Even if the posterior regime sequence lines up with macro events, it does not follow that the regime model has discovered the causal driver. The model is clustering the *observations*. If a central bank event coincides with a volatility spike, the model may assign a new state, but it is not attributing causality; it is responding to changed distributional features. In practice, this is acceptable as long as you treat regimes as signals rather than explanations.

Regime boundaries are uncertain. In real time, regime changes are rarely clean. The evidence that a switch has occurred accumulates gradually, and different models may place the boundary at slightly different times. If you look at smoothed posteriors, regime boundaries will look sharp because future data retroactively clarifies the boundary. In filtered posteriors, boundaries are often fuzzy and delayed. Therefore, a regime model cannot identify the “true

switch date” in the way a narrative might suggest. It can only provide a time-varying belief about the state.

Discrete regimes are approximations. The market environment is high-dimensional and often changes continuously. Discrete regimes approximate that environment with a small set of prototypes. This approximation can be valuable, but it cannot capture fine-grained drift without increasing K , which increases overfitting risk. Thus, a regime model cannot identify all subtle non-stationarities; it can only identify a coarse partition that trades off fidelity and robustness.

These limitations imply a practical discipline: we should not evaluate a regime model by how compelling its historical segmentation looks. We should evaluate it by (i) whether its filtered posteriors behave sensibly, (ii) whether regimes are stable enough across refits to be operationally meaningful, and (iii) whether decisions that use regime beliefs improve outcomes relative to baselines.

14.3.4 Time causality and the “information set”

In algorithmic trading, the single most important formal object is not the model; it is the information set. Everything else is downstream. If we do not define what is known at time t , we cannot define what it means to trade at time t without cheating. Regime models make this issue more acute because they involve inference over sequences: it is easy to inadvertently let the future influence your present belief.

Let $\mathcal{I}_t$ denote the information set available at time t when a decision is made. In daily trading, a common convention is: at the close of day t , we observe closing prices and compute features; we then place trades for execution at the open (or close) of day $t + 1$. Under this convention, a causality-safe regime belief used to decide at time t must be measurable with respect to $\mathcal{I}_t$, meaning it can depend only on observations up to time t . In regime notation, the tradable belief is

$$\gamma_t(k) = \mathbb{P}(s_t = k \mid x_{1:t}),$$

assuming x_t itself is observable at decision time. If the trade is executed at $t + 1$, then the decision uses γ_t and never γ_{t+1} .

This is the formal meaning of *filtering*. Filtering is the operation that maps past and present observations into a posterior belief about the current state:

$$(x_{1:t}) \mapsto \gamma_t(\cdot).$$

By contrast, *smoothing* maps the entire sample into a posterior belief:

$$(x_{1:T}) \mapsto \mathbb{P}(s_t = \cdot \mid x_{1:T}).$$

Smoothing is an intrinsically retrospective object. It is useful for understanding history, for diagnosing whether the model is capturing structure, and for training or calibrating parameters. But it is not a valid live trading signal unless you can reproduce it without future data, which you cannot by definition.

The information-set view also clarifies a subtle but common leakage mechanism: using features that are not measurable with respect to $\mathcal{I}_t$. Suppose you compute a rolling volatility estimate $\hat{\sigma}_t$ using returns from $t - L + 1$ to t . That is causal. But if you standardize returns by a volatility computed using a centered window (e.g., using returns from both before and after t), or if you z-score using the full sample mean and variance, you have introduced future information. The HMM will then infer regimes partly based on future data hidden inside the features. The resulting posterior will look excellent historically and will fail live.

Another frequent issue is *refit leakage*. Regime models often require parameter estimation (transition matrix and emission parameters). If you estimate parameters using the full sample and then compute filtered posteriors as if you were trading from the beginning, you are mixing two time axes: the inference step is filtered, but the parameters were learned with future information. This does not necessarily invalidate research if you are explicit about it, but it is not a faithful simulation of a live system. In a live system, parameters must be estimated using data available up to the refit time. Therefore, you must define a refit schedule: for example, every month, refit the HMM parameters using the past N years of data, then run filtering forward until the next refit. This schedule becomes part of the information set: parameters at time t are themselves random variables determined by past data.

Formally, we can represent this by including the parameter estimate $\hat{\Theta}_t$ in the information set:

$$\mathcal{I}_t = \sigma(x_{1:t}, \hat{\Theta}_t),$$

where $\hat{\Theta}_t$ is computed from a training set that is a subset of $\{x_1, \dots, x_t\}$. Then the tradable belief is

$$\gamma_t(k) = \mathbb{P}(s_t = k \mid \mathcal{I}_t),$$

which emphasizes that our belief depends on both the observed data and the current model parameters. This is a governance-native way of thinking: your regime signal is not merely a sequence; it is an artifact produced by a pipeline whose rules must be recorded.

Causality constraints also reshape how we interpret regime outputs. In smoothed plots, it is tempting to draw a line at time t^* and declare “the regime changed here.” In a causality-safe system, what matters is when the filtered posterior crossed some threshold:

$$\gamma_t(\text{stress}) > \tau.$$

That crossing time is typically later than t^* because evidence accumulates. This delay is not a bug; it is the price of not cheating. Therefore, any trading use of regimes must incorporate the fact that regime detection is *lagged*. The appropriate response is not to pretend the lag does not exist; it is to design decisions that remain valuable under lag, for example by using regimes primarily for risk scaling rather than for precise timing, by smoothing the posterior itself (causally, using only past posteriors), or by requiring persistence (hysteresis) before switching exposures.

To conclude this minimal formal section: a regime model posits a latent state process s_t and an observed process x_t whose distribution depends on s_t . The tractability and interpretability of this model come from conditional independence assumptions that must be respected in

implementation. The model identifies a useful representation only in a probabilistic and model-relative sense; labels are arbitrary, boundaries are uncertain, and discrete states approximate a continuous reality. Above all, regime modeling is inseparable from the information set. The only regime beliefs that count in trading are those that are measurable at decision time, computed with causal features, and produced under a refit schedule that respects time. The rest of the chapter builds on this foundation by specifying Hidden Markov Models as a concrete regime family and by developing inference and learning procedures that keep these causality principles intact.

14.4 Hidden Markov Models: Model Specification

14.4.1 Latent Markov chain: transition matrix and stationarity

A Hidden Markov Model (HMM) becomes a regime model the moment we commit to two ideas: the latent regime s_t is a discrete state that evolves as a Markov chain, and the observed data x_t are generated from a state-conditional emission distribution. In this section we specify that model carefully, because the specification is not a formality. In trading, what you specify determines what you can learn, what you will confuse, and what you will accidentally leak.

Let $s_t \in \{1, \dots, K\}$ denote the latent state at time t . The latent process is a first-order Markov chain:

$$\mathbb{P}(s_t = j \mid s_{t-1} = i, s_{t-2}, \dots) = \mathbb{P}(s_t = j \mid s_{t-1} = i) \equiv A_{ij}, \quad (14.3)$$

where A is the transition matrix with entries $A_{ij} \geq 0$ and row sums $\sum_{j=1}^K A_{ij} = 1$ for every i . We also specify an initial distribution π :

$$\mathbb{P}(s_1 = i) = \pi_i, \quad \pi_i \geq 0, \quad \sum_{i=1}^K \pi_i = 1. \quad (14.4)$$

The pair (π, A) defines the state process completely.

In regime detection, the transition matrix is not just a technical object; it is the mathematical encoding of *persistence*. When practitioners say “volatility regimes persist” or “stress lasts,” they are implicitly describing the magnitude of diagonal elements A_{ii} . A regime model with A_{ii} close to one will produce states that, once entered, are likely to persist; a model with smaller diagonals will switch frequently. This has immediate trading consequences because frequent switching can translate into turnover and cost, even if the posterior is used smoothly rather than as a hard classifier.

A useful quantity derived from A is the expected duration of state i under the Markov chain. If the chain is in state i at time t , the probability it remains in i at the next step is A_{ii} , and the probability of leaving is $1 - A_{ii}$. Under the Markov assumption, the duration in state i is geometrically distributed, so the expected duration is

$$\mathbb{E}[\text{duration in state } i] = \frac{1}{1 - A_{ii}}. \quad (14.5)$$

This formula is deceptively important. It connects a parameter estimate to an operational

interpretation: if your fitted $A_{ii} = 0.98$ on daily data, the implied expected duration is $1/(1 - 0.98) = 50$ trading days. If $A_{ii} = 0.90$, the expected duration is about 10 days. If your fitted model implies durations of 2–3 days, then either the market truly switches that fast in your feature space or, more likely, your model is overfitting noise and will produce unstable signals.

Another important concept is *stationarity* of the Markov chain. A Markov chain has a stationary distribution $\mu \in \mathbb{R}^K$ if μ satisfies

$$\mu^\top = \mu^\top A, \quad \sum_{i=1}^K \mu_i = 1, \quad \mu_i \geq 0. \quad (14.6)$$

If the chain is irreducible and aperiodic, this stationary distribution exists and is unique, and s_t converges in distribution to μ regardless of the initial distribution. In regime terms, μ_i is the long-run fraction of time the chain spends in state i . Practitioners sometimes interpret μ as the unconditional regime frequency. That interpretation is valid in the model, but with a caution: the HMM is fit to data under a particular estimation window, and real markets are not stationary in that strong sense. Still, μ is a useful summary because it tells you whether the model is using one state as a “default” and another as a rare exception, and whether those frequencies are plausible.

Stationarity also matters for initialization. If you do not have a strong prior about the starting regime, a common modeling choice is to set $\pi = \mu$, effectively starting the chain in its stationary distribution. This can reduce sensitivity to arbitrary initialization when filtering. But again, in trading pipelines, initialization is a technical detail only at the beginning of the sample. Once you have filtered forward for a while, the posterior tends to become insensitive to the initial prior, provided the model is not degenerate.

The Markov chain specification raises two modeling questions that are particularly relevant for finance.

First, should A be constant over time? In the standard HMM, yes. But in markets, regime switching intensity itself can change: the market can be “calm and stable” for months and then “twitchy” for a period. Time-varying transition probabilities are possible (non-homogeneous HMMs), but they add complexity and weaken identifiability. In this chapter we keep A constant for interpretability and because a constant A already captures persistence via diagonal dominance.

Second, should the chain allow direct transitions between all states? Again, in the standard HMM, yes: A_{ij} can be nonzero for any pair (i, j) . But in some trading contexts it is sensible to impose structure. For example, you might believe the market transitions from low-vol to medium-vol to high-vol and rarely jumps directly from low to high. This can be encoded by constraining A to be banded or to permit only certain transitions. Imposing such constraints can improve stability and interpretability at the cost of model flexibility. Whether constraints help depends on data quantity and on whether the constraint matches reality sufficiently well.

14.4.2 Emission distributions: Gaussian, Student- t , and simple alternatives

The emission distribution links the latent regime to observable data. The emission is where we decide what “different regimes” mean mathematically. In trading, we do not typically need a

perfect generative model of returns; we need an emission family that is expressive enough to distinguish meaningful environments and stable enough to estimate reliably.

Let $x_t \in \mathbb{R}^d$ denote the observation at time t . The HMM assumes

$$p(x_t \mid s_t = k) = f_k(x_t; \theta_k), \quad (14.7)$$

where f_k is the emission density for state k with parameters θ_k . Common choices include Gaussian, Student- t , and mixtures or discretized alternatives. We discuss these choices with a trading mindset: how they relate to stylized facts (fat tails, volatility clustering, asymmetry), and how they affect regime inference stability.

Gaussian emissions. The simplest and most common choice is Gaussian. In the univariate case ($d = 1$),

$$x_t \mid (s_t = k) \sim \mathcal{N}(\mu_k, \sigma_k^2), \quad (14.8)$$

with density

$$f_k(x) = \frac{1}{\sqrt{2\pi\sigma_k^2}} \exp\left(-\frac{(x - \mu_k)^2}{2\sigma_k^2}\right). \quad (14.9)$$

In the multivariate case ($d > 1$),

$$x_t \mid (s_t = k) \sim \mathcal{N}(\mu_k, \Sigma_k), \quad (14.10)$$

where $\mu_k \in \mathbb{R}^d$ and $\Sigma_k \in \mathbb{R}^{d \times d}$ is symmetric positive definite. Gaussian emissions are attractive because they lead to simple maximum-likelihood updates in EM and because they provide clear interpretations: μ_k is the regime-conditional mean vector, and Σ_k captures regime-conditional covariance.

However, finance returns are famously non-Gaussian: they exhibit fat tails and often skewness. The key question is not “is Gaussian true?” (it is not), but “is Gaussian adequate for regime inference?” In many trading applications, the dominant difference between regimes is volatility scale and correlation structure, and Gaussian emissions can capture that surprisingly well. If your aim is to detect high-volatility stress versus low-volatility calm, a Gaussian HMM often suffices, because the tail events are partly absorbed by the high- σ state. The model becomes an approximation in which fat tails are represented as a mixture of Gaussians with different variances. This is one of the practical virtues of regimes: even if each regime is Gaussian, the overall unconditional distribution is a mixture, which can be fat-tailed.

Still, Gaussian emissions can fail in two common ways. First, extreme outliers can dominate parameter estimation, causing the model to create a dedicated “outlier state” rather than a meaningful regime. Second, if skewness and tail asymmetry are crucial (e.g., crash risk), Gaussian emissions may mischaracterize the likelihood of negative moves relative to positive moves, leading to misleading posteriors.

Student- t emissions. A natural alternative is the Student- t distribution, which explicitly models heavier tails. In the univariate case,

$$x_t \mid (s_t = k) \sim t_{\nu_k}(\mu_k, \sigma_k^2), \quad (14.11)$$

with degrees of freedom $\nu_k > 2$ (often) controlling tail thickness. The density has the form

$$f_k(x) \propto \left(1 + \frac{(x - \mu_k)^2}{\nu_k \sigma_k^2}\right)^{-(\nu_k+1)/2}. \quad (14.12)$$

In the multivariate case, a multivariate t distribution with mean μ_k , scatter matrix Σ_k , and degrees of freedom ν_k plays the analogous role.

The trading motivation for t emissions is straightforward: returns have fat tails, and stress regimes are particularly fat-tailed. With t emissions, a single regime can accommodate occasional large moves without forcing the model to switch states purely because of an outlier. This can stabilize regime inference and reduce spurious switching. It can also produce regimes that differ not only in variance but also in tail behavior (through ν_k), which can be valuable if your goal is to distinguish “high vol but symmetric” from “crash-prone” environments.

The cost is complexity. Estimating ν_k reliably can be hard in limited samples, and EM updates for t emissions are more involved than Gaussian. There is also a subtle identification issue: a Gaussian mixture of variance states can mimic heavy tails, and a t distribution can mimic a mixture in some ranges. In practice, a disciplined approach is to decide what you want the emission model to capture. If you mainly care about volatility regimes, Gaussian may be sufficient and more stable. If you care about tail risk and crash protection, t emissions can be justified, but you must validate that the inferred states are stable and not artifacts of estimation noise.

Simple alternatives: discrete and robust emissions. Not all regime models need continuous parametric densities. In some trading systems, it is useful to define x_t as a small set of discrete indicators, for example the sign of returns, an exceedance of a volatility threshold, or a discretized drawdown bucket. Then the emission model becomes categorical:

$$\mathbb{P}(x_t = m \mid s_t = k) = B_{km}, \quad (14.13)$$

where m indexes discrete observation categories. This is a legitimate HMM and can be surprisingly robust because it avoids heavy reliance on parametric tail behavior. The downside is information loss: discretization throws away magnitude and correlation detail.

Another practical alternative is to use Gaussian emissions but with robust preprocessing: winsorize extreme returns, standardize using causal volatility estimates, or use log-volatility as an observed variable instead of raw returns. These are not separate emission families, but they are often more important than the choice between Gaussian and t . In trading, model misspecification is inevitable, and robust feature engineering can yield more stability than switching to a theoretically “more correct” density.

What should x_t be? The emission model choice cannot be separated from the choice of observation vector x_t . If x_t is just returns, regimes will mainly reflect mean/variance/tail differences in returns. If x_t includes realized volatility proxies, regimes may reflect volatility states more cleanly. If x_t includes spreads or liquidity proxies, regimes may pick up liquidity stress. If x_t includes cross-asset features, regimes may reflect correlation and risk-on/risk-off behavior. The model specification is therefore a joint design: you choose what signals the state is allowed to “see,” and you choose how the state translates into likelihoods.

In this chapter’s scope, the guiding principle is: x_t should be causality-safe, economically interpretable, and low-dimensional enough to estimate reliably. If you include too many features, you may create regimes that are unstable or that reflect noise in some dimensions.

14.4.3 Parameterization choices: 2-state vs K -state models

Once we specify a Markov chain and an emission family, we must choose the number of states K and the parameterization details within each state. This is where regime modeling can become either a disciplined representation or a narrative generator. The temptation is to increase K until the historical segmentation looks “right.” The disciplined approach is to choose K based on decision usefulness and stability.

Two-state models. The two-state HMM is the workhorse of regime detection because it is interpretable and often stable. The states are typically understood as “calm” versus “stress,” “low vol” versus “high vol,” or “risk-on” versus “risk-off.” With Gaussian emissions, the two-state model often allocates one state to low variance and one state to high variance. With t emissions, one state may be both higher variance and heavier-tailed.

Two-state models have several practical advantages:

1. *Interpretability:* it is easy to map states to risk actions (de-risk or not).
2. *Estimation stability:* fewer parameters means less overfitting and more robust EM convergence.
3. *Operational alignment:* many real decisions are binary or nearly binary (full risk budget vs reduced).

They also have a limitation: markets often exhibit more than two distinct behaviors. A two-state model may lump together “moderately volatile but trending” with “moderately volatile but mean-reverting,” even though those environments matter for switching between strategies.

K -state models. Increasing K allows richer distinctions: low vol, medium vol, high vol; or trend regime, range regime, crash regime; or correlation regimes in multi-asset systems. But increasing K increases parameter count substantially.

To see this, consider a Gaussian emission in d dimensions. Each state has a mean vector μ_k (with d parameters) and a covariance matrix Σ_k (with $d(d+1)/2$ free parameters). Thus, emission parameters per state scale like $O(d^2)$. The transition matrix adds $K(K-1)$ free

parameters (each row sums to one). Therefore, complexity grows rapidly with both K and d . This is why many practical regime systems keep d small and use K sparingly.

The real trade-off is not statistical elegance; it is robustness versus granularity. More states can:

- improve fit (higher likelihood),
- provide more nuanced risk actions,
- allow state-specific strategy weights,

but they can also:

- create unstable state definitions across refits,
- produce regimes that have too few observations to estimate reliably,
- increase switching (churn) because the model can explain noise by switching states.

In trading, these pathologies are expensive. An unstable regime label is not just a statistical inconvenience; it is an operational hazard. If state 3 means “trend” this quarter and “medium vol” next quarter, you cannot build policy around it.

Parameter tying and constraints. A middle ground is to use $K > 2$ but constrain parameters to improve stability. Common constraints include:

1. *Shared means, different variances:* set $\mu_k = 0$ (or a common mean) and allow σ_k to differ, focusing regimes on volatility.
2. *Diagonal covariance:* in multivariate settings, restrict Σ_k to be diagonal to reduce parameter count (at the cost of ignoring correlations).
3. *Shared covariance structure with scaling:* assume $\Sigma_k = c_k \Sigma_0$, meaning regimes differ only by a scale factor.
4. *Structured transitions:* restrict A to be near-diagonal or banded to encode gradual regime shifts.

These choices are not merely computational tricks; they encode what you believe regimes represent. If regimes are primarily volatility states, tying means and focusing on variance is coherent and often robust. If regimes are about correlation structure, you must allow covariance differences, but then you must accept the increased estimation burden.

Decision-aligned selection of K . A practical way to choose K is to map each additional state to a distinct decision policy. If adding a third state does not change what you would do, it may not be worth it. For example, if your risk policy is “normal leverage in calm, reduced leverage in stress,” then a two-state model aligns naturally. If you also want a “transition” state where you reduce leverage moderately, then three states might be justified. But if the third

state merely creates a slightly different shade of calm without changing decisions, it is likely to add noise rather than value.

This decision-aligned perspective also helps with validation. You can evaluate whether state-specific policies improve outcomes relative to a baseline policy. The regime model is not judged by likelihood alone; it is judged by whether it improves the trading objective when used causally.

14.4.4 Identifiability, label switching, and practical interpretation

Even with a fully specified HMM, the meaning of regimes is not automatically stable or interpretable. The model’s mathematical structure implies several identifiability and interpretation issues that must be addressed explicitly in any professional deployment.

Label switching and invariance. As noted earlier, HMM likelihood is invariant under permutation of state labels. If we apply a permutation φ to labels, replacing each state k with $\varphi(k)$, and permute π , rows/columns of A , and emission parameters accordingly, the joint likelihood of observations is unchanged. Therefore, there is no absolute meaning to “state 1.” Practical interpretation requires a labeling convention, such as sorting states by volatility (the state with the smallest σ_k is labeled “calm”). This is not optional. Without such conventions, state identities can flip between refits, making regime-based decisions incoherent.

Local maxima and EM sensitivity. HMM parameter estimation via EM is not guaranteed to find the global maximum likelihood. It often converges to a local maximum that depends on initialization. This matters because different local maxima can imply different regime interpretations and different switching behavior. In trading pipelines, we therefore treat initialization as part of governance: we fix seeds, document initialization rules, and often run multiple initializations and choose the best by a criterion that includes stability, not just likelihood.

Degenerate solutions. HMMs can exhibit degenerate fits, especially when K is large relative to data. A common degeneracy is a state that captures only a handful of outliers with extremely small variance (or a state with near-zero occupancy). Another is a transition matrix that becomes nearly absorbing (one state with $A_{ii} \approx 1$ and others rarely visited), effectively reducing the model to fewer states. These degeneracies may maximize likelihood but are not useful for trading. Detecting and preventing them requires constraints, regularization, or post-fit diagnostics that reject pathological solutions.

Regimes as statistical clusters, not narratives. Interpretation in finance is tempting because regimes often align with known historical events (crises, policy shifts). But the correct interpretation is statistical: regimes are clusters of observation patterns with persistence. When a regime aligns with a known event, that is a comforting sanity check, not a proof of causality. It is equally possible for a regime to be “real” in the model sense and yet not map cleanly onto a macro label. For example, in equity indices, one regime might reflect “high correlation, moderate vol” (crowded de-risking) and another might reflect “high vol, low correlation” (idiosyncratic

dispersion), which are not standard newspaper categories. The regime model is allowed to discover patterns that do not fit simple narratives, but the trading system must still translate them into coherent actions.

Hard classification versus probabilistic belief. A major practical interpretation choice is whether to treat s_t as a hard label or to treat $\gamma_t(k) = \mathbb{P}(s_t = k \mid \mathcal{I}_t)$ as the output. In most trading systems, it is safer to use probabilistic beliefs. Hard classification amplifies errors near boundaries: when the posterior is 0.51 versus 0.49, flipping the label can trigger large changes in exposure. Using probabilities allows smooth changes: you can blend strategy weights or scale risk gradually. This is not merely a comfort feature; it is a defense against the reality that filtered posteriors are uncertain, especially at regime boundaries.

Choosing interpretability anchors. To make regimes practically interpretable, it is useful to define “anchors”: statistics computed within each regime that help name it. Examples include regime-conditional volatility, average drawdown, tail quantiles, correlation measures, or average trend strength. Once you compute these anchors, you can label regimes consistently (e.g., the state with the largest 5% left-tail loss is “crash-prone”), and you can monitor whether those anchors remain stable across refits. If the anchors drift, the meaning of the regime is changing, and the decision policy should be revisited.

The stationarity myth and the stability objective. Finally, practical interpretation must respect that markets are not stationary, and so regime definitions themselves can drift. This creates a tension: the HMM assumes stationary parameters within each state and stationary transitions between states, but the real world violates both assumptions. The correct response is not to abandon the model, but to treat stability as an empirical objective. You want regimes that are sufficiently stable over a window to be useful, even if they drift over longer horizons. This is why refit cadence and training window selection matter. If you fit on too long a window, you force the model to reconcile incompatible eras. If you fit on too short a window, you overfit noise and lose persistence structure.

A disciplined regime modeling practice therefore includes:

1. a clear model specification for A and emissions,
2. a justified choice of K and parameter constraints,
3. a labeling convention and stability diagnostics across refits,
4. an explicit commitment to filtered (causal) posteriors for trading.

These elements make the HMM more than a charting device. They turn it into a component that can be integrated into a trading system without violating the book’s core principles: time awareness, no leakage, and governance-native reproducibility.

To conclude the specification section, we emphasize a pragmatic viewpoint. The HMM is not a claim that the world truly consists of K discrete regimes with stationary transitions and Gaussian emissions. It is a compact, interpretable approximation that is often useful precisely

because it is structured. The transition matrix encodes persistence; the emission distributions encode what differs across environments; K encodes how fine-grained your approximation is; and identifiability issues remind you that regimes are not “discovered facts” but model-dependent representations. The next sections build on this specification to derive the inference procedures (forward filtering and backward smoothing) and to establish how these beliefs become tradable signals under strict causality constraints.

14.5 Inference I: Filtering, Prediction, and the Forward Algorithm

14.5.1 Filtering as “online” regime belief: $p(s_t \mid x_{1:t})$

The practical value of an HMM in trading is not the existence of latent states in an abstract sense. The value is that the model gives you an *online* belief about which state you are likely in right now, computed using only information available up to the present. This is the filtered regime posterior

$$p(s_t \mid x_{1:t}),$$

and it is the only regime object that can be used as a live trading signal without violating causality. Everything else—smoothed posteriors, retrospective segmentations, “true regime timelines”—is either a research diagnostic or, if misused, a hidden leak.

To derive filtering, we begin with the HMM specification from the previous section. We have a latent Markov chain $s_t \in \{1, \dots, K\}$ with initial distribution π and transition matrix A , and we have emission densities

$$p(x_t \mid s_t = k) = f_k(x_t; \theta_k),$$

where x_t is the observed vector at time t and θ_k are regime-specific emission parameters. The joint distribution factorizes as

$$p(s_{1:T}, x_{1:T}) = \pi_{s_1} \prod_{t=2}^T A_{s_{t-1}, s_t} \prod_{t=1}^T f_{s_t}(x_t).$$

Filtering asks: given the observations up to time t , how do we compute the posterior distribution over s_t ? A useful intermediate is the *forward variable*

$$\alpha_t(k) \equiv p(x_{1:t}, s_t = k), \tag{14.14}$$

the joint probability of seeing the data up to time t and being in state k at time t . Once we have $\alpha_t(k)$, the filtered posterior is obtained by normalization:

$$p(s_t = k \mid x_{1:t}) = \frac{\alpha_t(k)}{\sum_{j=1}^K \alpha_t(j)}. \tag{14.15}$$

This definition makes the logic transparent: $\alpha_t(k)$ carries two kinds of information at once—(i) how likely state k is given transitions and priors, and (ii) how likely the observed data are under state k .

The forward recursion arises from marginalizing over the previous state:

$$\alpha_t(k) = p(x_{1:t}, s_t = k) \quad (14.16)$$

$$= p(x_t \mid s_t = k) p(x_{1:t-1}, s_t = k) \quad (14.17)$$

$$= f_k(x_t) \sum_{i=1}^K p(x_{1:t-1}, s_{t-1} = i, s_t = k). \quad (14.18)$$

Using the Markov property and conditional independence,

$$p(x_{1:t-1}, s_{t-1} = i, s_t = k) = p(x_{1:t-1}, s_{t-1} = i) p(s_t = k \mid s_{t-1} = i) = \alpha_{t-1}(i) A_{ik}.$$

Therefore,

$$\alpha_t(k) = f_k(x_t) \sum_{i=1}^K \alpha_{t-1}(i) A_{ik}. \quad (14.19)$$

The initialization at $t = 1$ follows directly from the definition:

$$\alpha_1(k) = p(x_1, s_1 = k) = p(s_1 = k) p(x_1 \mid s_1 = k) = \pi_k f_k(x_1). \quad (14.20)$$

Equations (14.19) and the initialization are the forward algorithm in its simplest form.

There is an alternative representation that is often more intuitive for trading: separate the update into a *prediction step* and an *measurement update*. Define the filtered posterior at time $t - 1$:

$$\gamma_{t-1}(i) \equiv p(s_{t-1} = i \mid x_{1:t-1}).$$

Then the one-step predicted distribution over s_t before seeing x_t is

$$p(s_t = k \mid x_{1:t-1}) = \sum_{i=1}^K p(s_t = k \mid s_{t-1} = i) p(s_{t-1} = i \mid x_{1:t-1}) = \sum_{i=1}^K A_{ik} \gamma_{t-1}(i). \quad (14.21)$$

After observing x_t , Bayes' rule updates this predicted distribution:

$$\gamma_t(k) \equiv p(s_t = k \mid x_{1:t}) = \frac{p(x_t \mid s_t = k) p(s_t = k \mid x_{1:t-1})}{\sum_{j=1}^K p(x_t \mid s_t = j) p(s_t = j \mid x_{1:t-1})} = \frac{f_k(x_t) \sum_{i=1}^K A_{ik} \gamma_{t-1}(i)}{\sum_{j=1}^K f_j(x_t) \sum_{i=1}^K A_{ij} \gamma_{t-1}(i)}. \quad (14.22)$$

This form is highly operational: it tells you exactly what the “regime detector” is doing each day. It carries forward yesterday’s regime probabilities, mixes them through the transition matrix (persistence and switching), and then reweights them by how plausible today’s observed features are under each regime’s emission distribution.

The filtered posterior $\gamma_t(k)$ is the primary output used in trading. You can use it in at least three ways:

1. *Soft classification*: declare the regime as $\arg \max_k \gamma_t(k)$, but only for reporting or visualization.
2. *Risk scaling*: define leverage or exposure as a smooth function of γ_t , for example exposure $\propto 1 - \gamma_t(\text{stress})$.

3. *Strategy blending*: allocate weights across strategies as convex combinations driven by regime probabilities.

In all cases, the virtue of filtering is that it is measurable with respect to the information set at time t . If x_t is known at decision time and your features are causality-safe, then γ_t is a valid live signal.

14.5.2 One-step-ahead predictive distributions

Filtering provides $p(s_t | x_{1:t})$. In trading, we also care about predictive distributions: what is the distribution of the next observation x_{t+1} given what we know at time t ? This matters for forecasting, for risk estimation, and for constructing regime-conditioned forecasts.

The HMM makes this prediction natural because the observation distribution at $t+1$ depends on s_{t+1} , which depends on s_t . We want

$$p(x_{t+1} | x_{1:t}).$$

We compute it by marginalizing over s_{t+1} :

$$p(x_{t+1} | x_{1:t}) = \sum_{k=1}^K p(x_{t+1} | s_{t+1} = k) p(s_{t+1} = k | x_{1:t}). \quad (14.23)$$

Thus, the one-step predictive distribution is a mixture of the emission distributions, weighted by the predicted state probabilities.

We therefore need $p(s_{t+1} = k | x_{1:t})$. This is the state prediction step analogous to (14.21):

$$p(s_{t+1} = k | x_{1:t}) = \sum_{i=1}^K p(s_{t+1} = k | s_t = i) p(s_t = i | x_{1:t}) = \sum_{i=1}^K A_{ik} \gamma_t(i). \quad (14.24)$$

Plugging (14.24) into (14.23) gives the predictive distribution as a regime-weighted mixture.

This mixture form is both intuitive and useful. If one regime dominates the posterior (say $\gamma_t(k^*) \approx 1$), then the predictive distribution is close to the emission distribution of that regime. If there is uncertainty (multiple regimes have nontrivial posterior mass), the predictive distribution becomes a mixture with fatter tails and wider variance than any single regime. That is precisely what we want in risk management: uncertainty about regime membership is itself a source of risk, and the predictive distribution correctly reflects that by widening.

To make this concrete, consider the univariate Gaussian emission case:

$$x_{t+1} | (s_{t+1} = k) \sim \mathcal{N}(\mu_k, \sigma_k^2).$$

Then the predictive distribution is a mixture of Gaussians:

$$p(x_{t+1} | x_{1:t}) = \sum_{k=1}^K w_{t+1}(k) \mathcal{N}(\mu_k, \sigma_k^2), \quad w_{t+1}(k) \equiv p(s_{t+1} = k | x_{1:t}).$$

From this mixture we can compute predictive moments. The predictive mean is

$$\mathbb{E}[x_{t+1} \mid x_{1:t}] = \sum_{k=1}^K w_{t+1}(k) \mu_k. \quad (14.25)$$

The predictive variance uses the law of total variance:

$$\text{Var}(x_{t+1} \mid x_{1:t}) = \mathbb{E}[\text{Var}(x_{t+1} \mid s_{t+1}, x_{1:t}) \mid x_{1:t}] + \text{Var}(\mathbb{E}[x_{t+1} \mid s_{t+1}, x_{1:t}] \mid x_{1:t}) \quad (14.26)$$

$$= \sum_{k=1}^K w_{t+1}(k) \sigma_k^2 + \sum_{k=1}^K w_{t+1}(k) \mu_k^2 - \left(\sum_{k=1}^K w_{t+1}(k) \mu_k \right)^2. \quad (14.27)$$

The first term is the within-regime variance averaged by weights; the second and third terms form the variance of the regime means (between-regime variance). This decomposition is economically meaningful: your predictive risk comes from uncertainty within a regime and from uncertainty about which regime you will be in. In many applications, especially with low signal-to-noise in returns, the between-regime term can be small if μ_k are near zero. But with volatility regimes, the within-regime term changes substantially across states, so the mixture variance can increase sharply when the posterior shifts toward high-vol states.

The predictive distribution is not only about x_{t+1} itself. In trading, we often want the predictive distribution of returns to compute risk measures such as Value-at-Risk (VaR) or expected shortfall, or to forecast the distribution of P&L under a strategy. With regime mixtures, these risk measures inherit regime uncertainty naturally. For example, VaR at level α is the α -quantile of the mixture distribution, which can be computed numerically. A practical advantage of the regime mixture is that it tends to produce heavier tails than a single Gaussian, which is closer to reality.

14.5.3 Numerical stability: scaling / log-domain computations

The forward recursion is conceptually simple, but naive implementation is numerically fragile. The problem is underflow: probabilities shrink exponentially fast in t because $\alpha_t(k)$ is a joint probability of t observations. Even with double precision, $\alpha_t(k)$ quickly becomes zero for moderate t if computed directly. In finance, where T can be thousands or tens of thousands, naive forward recursion will fail.

There are two standard remedies: scaling and log-domain computation. Both preserve the normalized posterior while avoiding numerical collapse.

Scaling (normalized forward variables). The scaling approach rescales α_t at each time step. Define the unscaled forward vector

$$\tilde{\alpha}_t(k) = f_k(x_t) \sum_{i=1}^K \alpha_{t-1}(i) A_{ik},$$

then define a scaling constant

$$c_t \equiv \sum_{k=1}^K \tilde{\alpha}_t(k). \quad (14.28)$$

We then set

$$\alpha_t(k) \leftarrow \frac{\tilde{\alpha}_t(k)}{c_t}. \quad (14.29)$$

After scaling, $\sum_k \alpha_t(k) = 1$, so the scaled forward vector is effectively the filtered posterior itself. Indeed, if we scale at each time step by the sum, we are directly computing $\gamma_t(k)$.

The scaling constants c_t are not arbitrary; they carry meaning. The product of scaling factors relates to the likelihood of the observed sequence:

$$p(x_{1:T}) = \prod_{t=1}^T c_t, \quad \log p(x_{1:T}) = \sum_{t=1}^T \log c_t. \quad (14.30)$$

This is extremely useful in practice because it gives a stable way to compute the log-likelihood for EM convergence diagnostics. In trading pipelines, tracking $\log c_t$ over time is also an audit artifact: it tells you whether the model is encountering observations that are systematically surprising (very small c_t).

Scaling is easy and typically sufficient for filtering and for EM. But you must implement it carefully. If you use scaled α_t in the recursion, you must adjust accordingly (the recursion becomes stable because you renormalize at every step). The essential point is that scaling does not change the posterior γ_t because posterior depends on ratios, and scaling multiplies all $\alpha_t(k)$ by the same constant.

Log-domain computation. An alternative is to work with log probabilities. Define

$$\ell_t(k) \equiv \log \alpha_t(k).$$

Then the recursion becomes

$$\ell_t(k) = \log f_k(x_t) + \log \left(\sum_{i=1}^K \exp(\ell_{t-1}(i) + \log A_{ik}) \right).$$

The key is the term $\log \sum_i \exp(\cdot)$, which must be computed with the log-sum-exp trick to avoid overflow/underflow. For any values z_i ,

$$\log \left(\sum_i e^{z_i} \right) = m + \log \left(\sum_i e^{z_i - m} \right), \quad m \equiv \max_i z_i.$$

Using this identity, we can compute the log recursion stably. Once we have $\ell_t(k)$, we recover the posterior by normalization in log space:

$$\gamma_t(k) = \frac{\exp(\ell_t(k))}{\sum_j \exp(\ell_t(j))}.$$

Which approach should you use? Scaling is generally simpler and faster, and it integrates naturally with EM. Log-domain computation is conceptually robust and sometimes easier to reason about when you implement custom emission densities that can produce very small likelihoods. In both cases, the objective is the same: do not let floating-point limitations

determine your regime beliefs.

There is a broader numerical stability issue that is not purely about underflow: emission likelihoods can become extreme if your feature scaling is inconsistent. For instance, if you feed returns in percent sometimes and in decimals other times, the Gaussian likelihood can collapse. Or if you feed a volatility proxy that occasionally hits zero, you can create singularities. A governance-native pipeline therefore records feature scaling conventions and performs sanity checks. Numerical stability is not only an algorithmic detail; it is a data discipline.

14.5.4 Causality gates: preventing accidental smoothing in live signals

Filtering is only meaningful as a trading signal if it is implemented causally. In regime modeling, the most common failures are not mathematical errors; they are *pipeline errors* where the model is correct but the way it is used leaks future information. Because regime outputs are visually compelling, these failures can persist unnoticed. A robust implementation therefore includes explicit causality gates: assertions and design rules that prevent accidental smoothing and other forms of look-ahead.

We can organize causality gates into four layers: (i) feature construction, (ii) parameter estimation and refitting, (iii) inference mode (filtering vs smoothing), and (iv) signal-to-trade alignment.

Gate 1: Feature construction must be measurable at time t . This is the first and most fundamental gate. Every component of x_t must be computable from data available at time t under your trading convention. If x_t includes rolling statistics, they must be computed with trailing windows. If x_t includes standardization, the mean and variance used for standardization must be estimated causally (e.g., rolling or expanding). The most dangerous leakage is “full-sample normalization”: computing a z-score using the global sample mean and variance and then feeding those standardized values into the HMM. This makes the entire feature vector depend on future data, which then contaminates the filtered posterior even if the forward algorithm itself is causal.

A practical discipline is to tag each feature with a *time index* that indicates the latest raw datum it uses. Then you can assert that this index is $\leq t$ for features used at time t . In code, this becomes an invariant checked at runtime.

Gate 2: Parameter estimation must respect refit boundaries. Even if filtering is causal given fixed parameters, the act of estimating parameters can introduce leakage. If you estimate (A, θ_k) using the full sample and then filter from the start, your posterior is “filtered” only in form, not in substance. A live system would not have access to future data to estimate parameters at the beginning. Therefore, you must define a refit schedule. At each refit date τ , you estimate parameters using only data up to τ (or up to τ minus a buffer if needed for execution timing), and then you filter forward until the next refit. This ensures that both parameters and posteriors are measurable with respect to the information set.

In research, you might still fit parameters on the full sample to explore whether regimes exist. That is acceptable as exploratory analysis, but it must be labeled as such and must not be

confused with a tradable pipeline. The causality gate is therefore not “never fit on full sample,” but “never call the result a live signal if parameters used future data.”

Gate 3: Filtering only for signals; smoothing only for diagnostics. The forward algorithm produces filtered posteriors. The forward-backward algorithm produces smoothed posteriors. A common mistake is to compute smoothed posteriors because they look cleaner, then treat them as regime labels for strategy evaluation. This creates an optimistic bias because smoothed labels know the future. In fact, the bias can be severe: smoothed regimes often align perfectly with volatility spikes and recoveries, giving the impression that the model “detected” the regime change early. In reality, filtering would detect it later.

Therefore, a causality gate in a professional pipeline is: *the object exported to the trading system must be filtered posterior γ_t and must be computed without backward passes or future data*. Smoothing may still be computed, but it should be separated into a diagnostics artifact folder and should never feed back into signal generation. This is not a stylistic recommendation; it is a structural separation in code and data flows.

Gate 4: Align signal time and trade time explicitly. Even if your posterior γ_t is causal, you can still leak by trading at the wrong time. Suppose you compute x_t using close-to-close returns and produce γ_t at the close of day t . If you then assume you could have traded at the close of day t based on γ_t , you have a timing inconsistency (unless you truly trade at the close with that data). The correct simulation is: compute γ_t after observing the close, then trade at $t + 1$ open (or close, depending on your convention). This is the same discipline we enforced in backtesting (Ch. 06), but regime modeling makes it even more important because inference is sequential and easy to misalign.

A robust pipeline therefore stamps each posterior value with the decision time at which it becomes available and the execution time at which it is applied. It also includes tests that verify you never use γ_{t+1} to decide at time t and that you never use smoothed quantities to simulate online decisions.

A practical “no accidental smoothing” checklist. To make these gates concrete, it helps to state a small checklist that can be implemented as asserts:

1. The code path that produces trading signals calls only the forward filter, never backward smoothing.
2. The regime posterior used for day $t + 1$ trades is computed from data up to day t (or up to the close of day t under the convention).
3. Any normalization constants (means, variances) used in features at time t are estimated from data up to t only.
4. Any model parameters used to compute γ_t are estimated from a training window that ends at or before t (according to the refit schedule).
5. Any evaluation that uses regime labels distinguishes between filtered labels (tradable) and smoothed labels (diagnostic), and reports both if needed.

These checks prevent the two most common regime modeling sins: accidental look-ahead through preprocessing and accidental smoothing through the backward pass.

Why this matters economically. It is tempting to treat causality gates as bureaucratic obstacles. They are not. In markets, the most expensive errors are those that create false confidence. A smoothed regime timeline can make you believe you have an early-warning system when you actually have a retrospective explanatory device. If you build position sizing rules or hedges on that false belief, you can take on more risk than you can afford. In contrast, a causality-safe filtered posterior forces humility: it will be uncertain, it will lag, and it will sometimes be wrong. But it is honest. And honest signals can be engineered into robust decision systems. Dishonest signals cannot.

Filtering as the core artifact. For the rest of this chapter, we treat the filtered posterior γ_t and the one-step predictive mixture as the primary inference outputs. They are the objects that connect the HMM specification to trading decisions. The forward algorithm computes them efficiently, scaling or log-domain implementations make them numerically stable, and causality gates make them legitimate. In the next section we will introduce smoothing and the backward algorithm, but we will do so with a strict separation of roles: smoothing is a research instrument; filtering is the live signal.

14.6 Inference II: Smoothing and the Backward Algorithm

14.6.1 Smoothing definition: $p(s_t \mid x_{1:T})$ and why it is tempting

Filtering is the HMM’s answer to the trading question: “what do I believe about the regime *now*, given what I have seen so far?” Smoothing is the HMM’s answer to a different question: “what do I believe about the regime at time t , given *everything* I saw in the entire sample?” Formally, smoothing is the posterior

$$p(s_t \mid x_{1:T}),$$

where T is the final time in your dataset. This is a retrospective object: it uses future information to infer the past. In research, that is often exactly what we want, because it yields a cleaner, more confident reconstruction of the latent regime sequence. In live trading, smoothing is exactly what we must *not* use, because it is a form of look-ahead.

To define smoothing precisely, recall the forward variables from the previous section:

$$\alpha_t(k) \equiv p(x_{1:t}, s_t = k).$$

Smoothing requires combining forward information (evidence from the past up to t) with backward information (evidence from the future after t). We define the backward variables:

$$\beta_t(k) \equiv p(x_{t+1:T} \mid s_t = k), \tag{14.31}$$

the probability of seeing future observations from $t + 1$ to T conditional on being in state k at

time t . With these definitions, the smoothed posterior is proportional to the product of forward and backward messages:

$$p(s_t = k \mid x_{1:T}) = \frac{p(x_{1:T}, s_t = k)}{p(x_{1:T})} \quad (14.32)$$

$$= \frac{p(x_{1:t}, s_t = k) p(x_{t+1:T} \mid x_{1:t}, s_t = k)}{p(x_{1:T})}. \quad (14.33)$$

By the conditional independence properties of the HMM, once we condition on s_t , the future observations are independent of the past observations:

$$p(x_{t+1:T} \mid x_{1:t}, s_t = k) = p(x_{t+1:T} \mid s_t = k) = \beta_t(k).$$

Therefore,

$$p(s_t = k \mid x_{1:T}) = \frac{\alpha_t(k) \beta_t(k)}{\sum_{j=1}^K \alpha_t(j) \beta_t(j)}. \quad (14.34)$$

Equation (14.34) is the central smoothing identity: smoothed regime belief equals the normalized product of a forward probability and a backward probability.

We now derive the backward recursion. By definition,

$$\beta_t(k) = p(x_{t+1:T} \mid s_t = k).$$

We can express this in terms of s_{t+1} :

$$\beta_t(k) = \sum_{j=1}^K p(x_{t+1:T}, s_{t+1} = j \mid s_t = k) \quad (14.35)$$

$$= \sum_{j=1}^K p(x_{t+2:T} \mid x_{t+1}, s_{t+1} = j, s_t = k) p(x_{t+1} \mid s_{t+1} = j, s_t = k) p(s_{t+1} = j \mid s_t = k). \quad (14.36)$$

Using the HMM conditional independence, x_{t+1} depends only on s_{t+1} , and future observations beyond $t+1$ depend on s_{t+1} through $\beta_{t+1}(j)$:

$$p(x_{t+2:T} \mid x_{t+1}, s_{t+1} = j, s_t = k) = p(x_{t+2:T} \mid s_{t+1} = j) = \beta_{t+1}(j),$$

and

$$p(x_{t+1} \mid s_{t+1} = j, s_t = k) = p(x_{t+1} \mid s_{t+1} = j) = f_j(x_{t+1}).$$

Also,

$$p(s_{t+1} = j \mid s_t = k) = A_{kj}.$$

Therefore, the backward recursion is

$$\beta_t(k) = \sum_{j=1}^K A_{kj} f_j(x_{t+1}) \beta_{t+1}(j). \quad (14.37)$$

The initialization is $\beta_T(k) = 1$ for all k because $x_{T+1:T}$ is an empty sequence with probability

one:

$$\beta_T(k) = p(\text{empty} \mid s_T = k) = 1.$$

Equations (14.37) and the initialization define the backward algorithm.

Why is smoothing so tempting? Because it often produces the picture we wish we had in real time: crisp state assignments with clean regime blocks. In filtered posteriors, uncertainty near boundaries is obvious. In smoothed posteriors, boundaries appear sharper because future data retroactively resolves ambiguity. This visual clarity is seductive. It makes the regime model look more accurate, more decisive, and more “predictive” than it truly is. It also makes it easier to tell stories: “the market entered stress here and exited there.” In truth, smoothing is not telling you when the market entered stress; it is telling you how, given the entire future path, the model would best segment the past. That is a valid statistical object. It is not a tradable one.

14.6.2 When smoothing is acceptable (research) and when it is leakage (signals)

Smoothing is not bad. Misusing smoothing is bad. The crucial discipline is to separate research roles from signal roles.

In research, smoothing is acceptable and often valuable for at least four reasons.

First, smoothing is a diagnostic of model fit. If the model is sensible, the smoothed posterior should align with obvious distributional changes: volatility spikes, drawdown episodes, correlation bursts, liquidity stress, or other structural features that motivated the regime model. If smoothing produces regimes that do not correspond to any interpretable changes in the data, the model may be misspecified or overfitting.

Second, smoothing helps interpret state characteristics. When you compute regime-conditional statistics (means, variances, tail measures, correlations), you often want a clean assignment of times to states. Using smoothed posteriors provides a more stable weighting of observations than filtered posteriors, especially early in the sample where filtered beliefs are more sensitive to priors. In other words, smoothing can provide better estimates of what “state k ” looks like under the model.

Third, smoothing is used inside the EM (Baum–Welch) learning algorithm. The E-step of EM requires expected sufficient statistics under the posterior distribution of states and transitions. Those expectations depend on smoothed state posteriors and smoothed transition posteriors. In this internal algorithmic role, smoothing is not leakage; it is the correct computation for maximum likelihood estimation on a fixed dataset.

Fourth, smoothing can be useful for historical attribution and narrative reporting, provided you do not confuse attribution with prediction. For example, a risk manager might want to explain why a strategy underperformed in 2020 versus 2017. A smoothed regime timeline can be a structured way to say: “under the model, the system spent more time in the high-volatility regime in 2020.” This is retrospective explanation, not signal generation.

In contrast, smoothing is leakage whenever it influences decisions that are evaluated as if they were made in real time. There are two common leakage modes.

The first mode is explicit: you compute $p(s_t \mid x_{1:T})$ and then use it as the regime label

for trading rules at time t . This directly uses future data in present decisions. The resulting backtest is invalid because the regime “knew” what happened after time t .

The second mode is subtle: you compute smoothed regimes to “train” a regime classifier or to design switching rules, and then you evaluate those rules on the same data without respecting refit boundaries. For example, you might smooth to get a regime label sequence, then train a supervised classifier to map features to those labels, then backtest the classifier as if it were a live regime detector. If the labels were smoothed, the classifier is trained on future-informed targets. Even if you later apply the classifier causally, you have baked in information about the future through the label construction. This is leakage-by-teacher: the teacher (smoothed label) is non-causal, so the student inherits a non-causal target.

Therefore, the rule is strict: smoothing can be used for understanding, for estimating parameters, and for designing hypotheses, but the moment you claim “this is a trading signal,” smoothing is forbidden. The only exception would be a contrived scenario where your trading decision truly has access to future information, which in realistic markets it does not.

A disciplined research workflow will therefore produce two parallel regime timelines:

1. A *filtered* posterior timeline $\gamma_t(k) = p(s_t = k \mid x_{1:t})$, which is tradable.
2. A *smoothed* posterior timeline $\tilde{\gamma}_t(k) = p(s_t = k \mid x_{1:T})$, which is retrospective.

Both can be reported, but they must never be conflated. The difference between them is not a minor technicality; it is the difference between signal and hindsight.

14.6.3 A disciplined workflow: fit with all data, trade with filtered beliefs

The phrase “fit with all data, trade with filtered beliefs” captures a pragmatic compromise that many practitioners use, and it is worth stating carefully because it can be either reasonable or misleading depending on what you mean by “fit.”

There are three distinct contexts in which fitting “with all data” might occur:

(1) Exploratory research on a historical dataset. In an exploratory phase, you may fit an HMM on the entire historical dataset to answer: do regimes appear to exist in the data at all? This is legitimate as long as you treat the result as descriptive. The fitted parameters and smoothed posteriors can help you see whether the model captures plausible segmentation. In this context, “fit with all data” is simply a way to maximize information for diagnosis.

(2) Production inference at the present time. In a live system, “all data” means all data available up to now. When you refit the model today, you use the full available history (or a chosen rolling window) up to today. That is not leakage; it is what you actually can do. Then you trade tomorrow using filtered beliefs computed from data up to today. In other words, in production, you always fit with all available past data, and you always trade with filtered beliefs because the future is not available.

(3) Backtesting and simulation of a live process. In backtesting, we want to simulate what would have happened if we ran the model historically. Here, “fit with all data” is dangerous because it can mean “fit using future data” at each historical time. The disciplined approach is to define refit times and training windows, and at each refit time τ fit the model using only $x_{1:\tau}$ (or a trailing subset), then filter forward to produce beliefs used for trades. This simulates a live process faithfully.

So what is the recommended workflow in this chapter? It is the following:

1. **Model specification.** Choose K , emission family, feature vector x_t , and constraints. Document these choices in a parameter registry.
2. **Exploratory fit (optional).** Fit once on a large historical window to check plausibility. Compute both smoothed and filtered posteriors, but treat smoothing as a diagnostic.
3. **Define a refit schedule.** Choose dates $\tau_1 < \tau_2 < \dots$ (e.g., monthly, quarterly) and a training window rule (expanding or rolling). This is part of the system design, not an afterthought.
4. **Walk-forward estimation.** At each refit date τ_m , estimate parameters $\hat{\Theta}_{\tau_m}$ using only training data up to τ_m . Store these parameters as artifacts.
5. **Forward filtering for signals.** Using $\hat{\Theta}_{\tau_m}$, run the forward filter to produce γ_t for $\tau_m \leq t < \tau_{m+1}$. These γ_t are the only regime beliefs allowed to influence trading actions.
6. **Trading decision.** Convert γ_t into exposures for $t + 1$ according to explicit decision rules (risk scaling, switching, hedging). Ensure signal-to-trade alignment.
7. **Diagnostics and reporting.** Separately compute smoothed posteriors on training windows if desired for interpretation, but never feed them into the signal path. Report filtered and smoothed timelines distinctly.

This workflow accomplishes two goals simultaneously. First, it lets you use smoothing where it belongs: to understand, to estimate, and to diagnose. Second, it ensures that the deployed signal is filtered and causally valid.

It is worth highlighting a nuance: even within this workflow, the *parameters* used for filtering are estimated using historical data. Therefore, parameter estimation error is part of the signal uncertainty. In production, parameters will drift as new data arrive. In backtests, you must simulate that drift by refitting. If you instead fit parameters once on the full sample and then filter as if parameters were known in advance, you overstate performance because you understate parameter uncertainty. This is not as glaring as smoothing leakage, but it is still an optimism bias. That is why refit cadence belongs in Chapter 14’s scope: regimes are not just inferred; they are maintained.

14.6.4 Diagnostics that reveal leakage-by-smoothing

Because smoothing produces attractive results, it is not enough to state “do not use smoothing.” A disciplined practitioner also needs diagnostics that detect whether smoothing has leaked into

the evaluation, intentionally or accidentally. The diagnostics should be simple, automated, and difficult to rationalize away.

We present four practical diagnostics.

Diagnostic 1: Filtered vs smoothed boundary lag. Compute both filtered and smoothed posteriors on the same fitted model and examine regime transitions. In a properly implemented system, the filtered posterior should change more slowly and often later than the smoothed posterior at regime boundaries. If your “live” regime label aligns almost perfectly with the smoothed boundary timings, that is suspicious. In reality, online inference lags. If it does not, you might be accidentally using future information or using features that embed future data.

A quantitative version of this diagnostic is to measure the time difference between the first crossing of a threshold in the smoothed posterior and the first crossing in the filtered posterior. If the “live” posterior consistently crosses at the smoothed time, you are likely leaking.

Diagnostic 2: Unrealistic Sharpe jump when using regimes. Run the backtest twice: once using filtered regimes and once using smoothed regimes (everything else identical). If the smoothed-regime backtest shows a dramatic improvement (especially in drawdown avoidance and crash timing), while the filtered-regime backtest shows modest or inconsistent improvement, that gap is a signature that smoothing is providing future information. This is not surprising; it is exactly what smoothing does. The diagnostic is to ensure you are not accidentally reporting the smoothed result as if it were tradable.

In an honest report, you might even include both curves to educate the reader: “here is the upper bound you get with hindsight (smoothed), and here is what is feasible online (filtered).”

Diagnostic 3: Causal permutation test on the future block. A particularly revealing test is to break the future while keeping the past intact. Suppose you compute your regime signal for times up to t and then for evaluation you continue through T . Now randomize or permute observations *after* time t (or replace them with noise) and recompute the regime beliefs for times *before* t . In a causal system, changing the future should not change beliefs about the past. If your regime beliefs before t change when you perturb data after t , you have used smoothing or you have used a preprocessing step that depends on the full sample. This diagnostic is powerful because it directly tests the causal separability of past and future.

Diagnostic 4: Feature lineage audit for global operations. Leakage-by-smoothing is often accompanied by leakage-by-preprocessing. The most common culprits are global mean/variance normalization, centered rolling windows, and any transform that uses the full sample (including certain forms of PCA fitted on the full dataset). A feature lineage audit is a structured check: for each feature, record whether its computation uses only past data. In practice, you can test this by recomputing features on prefixes of the dataset and verifying that feature values at time t are identical whether you compute them on the prefix $1 : t$ or on the full sample $1 : T$. If they differ, the feature depends on the future. This can be automated and should be part of the governance checklist.

Why these diagnostics matter. The deeper reason these diagnostics are necessary is that regime models can “look right” even when they are wrong. The human eye is excellent at seeing patterns and poor at detecting subtle information leaks. The discipline of algorithmic trading requires that we treat attractive plots as hypotheses, not as evidence. Smoothing is a legitimate inference procedure, but it is a procedure that uses the future. Therefore, any system that appears to detect regimes with no lag, or that seems to avoid crashes with uncanny timing, must be treated with suspicion until proven otherwise.

Smoothing as a pedagogical tool. Despite these dangers, smoothing has a valuable pedagogical role, which is why we include it in the chapter. It shows what the HMM believes *in hindsight*. By comparing smoothed and filtered outputs, you learn what information the model needs to become confident, how regime transitions manifest in data, and how uncertainty evolves. This comparison also teaches humility: if the filtered posterior is often uncertain and late, then your trading actions should be robust to uncertainty and lag. That is a more realistic design posture than building brittle switching rules that assume perfect regime detection.

To close this section, we restate the key operational principle. The forward algorithm gives you filtering, which is causal and tradable. The backward algorithm gives you smoothing, which is retrospective and diagnostic. In research, smoothing can help you understand regimes and estimate parameters; in signals, smoothing is leakage. A disciplined workflow separates these roles structurally, and robust diagnostics ensure that hindsight has not slipped into the live pipeline.

14.7 Learning HMM Parameters

14.7.1 Maximum likelihood via EM (Baum–Welch): intuition and structure

So far we have treated the transition matrix A and the emission parameters $\{\theta_k\}_{k=1}^K$ as if they were known. In practice they are not, and learning them is a central part of turning HMMs from a conceptual tool into an operational regime detector. The standard approach is maximum likelihood estimation (MLE): choose parameters $\Theta = (\pi, A, \theta_1, \dots, \theta_K)$ that maximize the marginal likelihood of the observed data,

$$\hat{\Theta} \in \arg \max_{\Theta} p(x_{1:T}; \Theta).$$

The difficulty is that $p(x_{1:T}; \Theta)$ involves a sum over all possible latent state sequences $s_{1:T}$, and there are K^T of them. The EM algorithm, specialized to HMMs and historically called the Baum–Welch algorithm, solves this by exploiting the model’s conditional independence structure. Conceptually, EM alternates between (i) inferring what the hidden states *probably were* under the current parameters (E-step), and (ii) updating parameters as if those inferred states were observed with fractional weights (M-step).

The intuition is easiest if we imagine that the state sequence were observed. If we knew $s_{1:T}$, then learning would be straightforward:

- The initial distribution π would be estimated by the empirical frequency of each state at

time 1.

- The transition matrix A would be estimated by counting transitions $i \rightarrow j$ and normalizing each row.
- The emission parameters θ_k would be estimated from the subset of observations assigned to state k (e.g., sample mean and covariance for Gaussian emissions).

The hidden-state nature of the problem prevents us from doing this directly. EM approximates it by replacing hard state assignments with posterior probabilities.

Formally, EM maximizes a lower bound on the log-likelihood. The observed-data log-likelihood is

$$\log p(x_{1:T}; \Theta) = \log \sum_{s_{1:T}} p(x_{1:T}, s_{1:T}; \Theta).$$

Let $q(s_{1:T})$ be any distribution over state sequences. By Jensen's inequality,

$$\log p(x_{1:T}; \Theta) = \log \sum_{s_{1:T}} q(s_{1:T}) \frac{p(x_{1:T}, s_{1:T}; \Theta)}{q(s_{1:T})} \geq \sum_{s_{1:T}} q(s_{1:T}) \log \frac{p(x_{1:T}, s_{1:T}; \Theta)}{q(s_{1:T})}.$$

EM chooses q to be the posterior under the current parameters Θ^{old} :

$$q(s_{1:T}) = p(s_{1:T} \mid x_{1:T}; \Theta^{\text{old}}),$$

which makes the bound tight at Θ^{old} , and then maximizes the bound with respect to Θ to produce Θ^{new} . This guarantees non-decrease of the likelihood at each iteration.

For HMMs, the E-step reduces to computing two types of posterior quantities:

$$\gamma_t(k) \equiv p(s_t = k \mid x_{1:T}; \Theta^{\text{old}}) \quad (\text{smoothed state posterior}), \quad (14.38)$$

$$\xi_t(i, j) \equiv p(s_t = i, s_{t+1} = j \mid x_{1:T}; \Theta^{\text{old}}) \quad (\text{smoothed transition posterior}). \quad (14.39)$$

These quantities can be computed efficiently using the forward-backward algorithm. Specifically, $\gamma_t(k)$ is obtained from $\alpha_t(k)$ and $\beta_t(k)$ as

$$\gamma_t(k) = \frac{\alpha_t(k) \beta_t(k)}{\sum_{j=1}^K \alpha_t(j) \beta_t(j)}.$$

The pairwise posterior $\xi_t(i, j)$ is also computed from forward and backward messages:

$$\xi_t(i, j) = \frac{\alpha_t(i) A_{ij} f_j(x_{t+1}) \beta_{t+1}(j)}{\sum_{a=1}^K \sum_{b=1}^K \alpha_t(a) A_{ab} f_b(x_{t+1}) \beta_{t+1}(b)}. \quad (14.40)$$

The denominator normalizes so that $\sum_{i,j} \xi_t(i, j) = 1$.

In the M-step, we update parameters by maximizing the expected complete-data log-likelihood, where expectations are taken under the posterior computed in the E-step. The complete-data

log-likelihood is

$$\log p(x_{1:T}, s_{1:T}; \Theta) = \log p(s_1; \pi) + \sum_{t=2}^T \log p(s_t | s_{t-1}; A) + \sum_{t=1}^T \log p(x_t | s_t; \theta_{s_t}) \quad (14.41)$$

$$= \log \pi_{s_1} + \sum_{t=2}^T \log A_{s_{t-1}, s_t} + \sum_{t=1}^T \log f_{s_t}(x_t; \theta_{s_t}). \quad (14.42)$$

Taking expectations with respect to the posterior, the terms involving π and A become weighted counts:

$$\pi_k^{\text{new}} = \gamma_1(k), \quad (14.43)$$

$$A_{ij}^{\text{new}} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)}. \quad (14.44)$$

Equation (14.44) is the fractional-count version of “count transitions and normalize.”

The emission parameters update depends on the emission family. For Gaussian emissions with $x_t \in \mathbb{R}^d$,

$$x_t | (s_t = k) \sim \mathcal{N}(\mu_k, \Sigma_k),$$

the M-step updates are weighted sample estimates:

$$\mu_k^{\text{new}} = \frac{\sum_{t=1}^T \gamma_t(k) x_t}{\sum_{t=1}^T \gamma_t(k)}, \quad (14.45)$$

$$\Sigma_k^{\text{new}} = \frac{\sum_{t=1}^T \gamma_t(k) (x_t - \mu_k^{\text{new}})(x_t - \mu_k^{\text{new}})^\top}{\sum_{t=1}^T \gamma_t(k)}. \quad (14.46)$$

In the univariate case, Σ_k reduces to σ_k^2 . For Student- t emissions, the M-step can be implemented via an augmented-variable representation (viewing the t as a scale mixture of Gaussians) which introduces additional latent scale variables; the updates then become weighted versions of robust mean/variance estimates along with an update for degrees of freedom ν_k (often requiring numerical solving). The key message is that EM replaces hard assignments with posterior weights and then performs weighted maximum-likelihood updates.

From the perspective of this book, the Baum–Welch algorithm is more than a recipe: it is a governance object. Each EM iteration produces measurable artifacts: the log-likelihood value, the updated transition matrix, emission parameters, and posterior occupancy of each state. These artifacts are exactly what you need to diagnose whether regime learning is stable or pathological.

14.7.2 Initialization: why it matters and how to do it deterministically

EM is monotone but not globally optimal: it increases likelihood each iteration but can converge to different local maxima depending on initialization. In trading, this sensitivity is not an academic footnote. Different local maxima can imply different regime meanings, different switching intensities, and different downstream trading behavior. If your regime detector changes character because you reran EM with a different random seed, you do not have a model; you

have a roulette wheel.

Initialization matters for two main reasons. First, the likelihood surface for HMMs is highly non-convex, especially as K increases or emissions become flexible. Second, regime models are prone to degenerate solutions: one state capturing outliers, states with near-zero occupancy, or near-absorbing transitions. A bad initialization can steer EM into these degeneracies.

A deterministic initialization strategy is therefore a core part of a production-grade pipeline. The goal is not to avoid all local maxima (impossible in general), but to ensure reproducibility and to bias the algorithm toward sensible solutions.

A practical deterministic approach can be organized into three layers: initialize emissions, initialize transitions, initialize priors.

Initialize emissions via clustering. If emissions are Gaussian, a common method is to initialize state means and covariances using a clustering procedure on the observations. For example, one might run K -means on $\{x_t\}$ to assign each x_t to a cluster, then compute the sample mean and covariance in each cluster to initialize (μ_k, Σ_k) . To make this deterministic, you fix the seed and/or use a deterministic clustering initialization (e.g., choose initial centroids as quantiles along a principal direction, or use a fixed subset rule). In a no-pandas, governance-native pipeline, the important part is that the rule is explicit and reproducible.

For univariate volatility regime detection, a particularly robust deterministic initialization is to initialize based on volatility quantiles. For example, compute a causal volatility proxy v_t (e.g., rolling standard deviation) and define initial state assignments by quantiles of v_t : lowest quantile is “low vol,” highest is “high vol,” and intermediate quantiles for additional states. Then initialize emission parameters from these assignments. This aligns initialization with the economic meaning of regimes.

For Student- t emissions, you can initialize (μ_k, Σ_k) as above and set ν_k to a moderate value (e.g., $\nu = 8$ or $\nu = 10$) for all states initially, letting EM adjust later if you implement ν estimation. Again, the specific value matters less than the fact that it is documented and deterministic.

Initialize transitions via persistence priors. In trading regimes, persistence is usually expected. Therefore, a sensible default is to initialize A as diagonally dominant. For example, set

$$A_{ii}^{(0)} = a, \quad A_{ij}^{(0)} = \frac{1-a}{K-1} \quad (i \neq j),$$

with a chosen to imply plausible expected durations. If daily data and you expect regimes to last weeks or months, choose a between 0.95 and 0.99 depending on how persistent you want the initial model. This is not “cheating”; it encodes a prior belief that regimes persist. EM can move away from it if the data disagree.

A more data-driven deterministic transition initialization is to use the initial state assignments from clustering and compute empirical transition frequencies, then shrink them toward a diagonally dominant matrix. Shrinkage is a form of regularization (discussed below), but even at initialization it can prevent zero transition probabilities that can trap EM.

Initialize priors. The initial distribution π can be set to the empirical occupancy of initial assignments (fraction of points assigned to each state), or simply to the stationary distribution implied by $A^{(0)}$ (if you compute it). In many cases, setting π to uniform is fine because filtering will quickly move away from it. Again, the main requirement is that the choice is explicit and reproducible.

Multiple deterministic restarts. Even with deterministic rules, it is often prudent to run a small number of restarts with different deterministic initializations (e.g., different quantile splits, different seeds recorded in the run manifest) and then choose the best model by a criterion that includes stability. In a governance-native workflow, you would store all candidates and the selection rule. This prevents hidden cherry-picking and makes the research auditable.

14.7.3 Convergence criteria and failure modes

A professional regime pipeline must treat EM convergence as a monitored process, not a black box. “It converged” is meaningless unless you specify what convergence means and what failure modes you checked for.

Convergence criteria. The simplest convergence criterion is change in log-likelihood:

$$\Delta\ell = \ell^{(m)} - \ell^{(m-1)}, \quad \ell^{(m)} \equiv \log p(x_{1:T}; \Theta^{(m)}),$$

and stop when $\Delta\ell$ is below a tolerance. Because log-likelihood scales with T , it is common to use per-observation log-likelihood or relative improvement:

$$\frac{\Delta\ell}{T} < \varepsilon \quad \text{or} \quad \frac{\Delta\ell}{|\ell^{(m-1)}|} < \varepsilon.$$

Another criterion is parameter change, e.g., maximum absolute change in A entries or emission parameters. In practice, log-likelihood improvement is the most reliable because parameters can change in ways that do not affect likelihood much, especially under label permutations.

In addition to convergence, you must impose a maximum number of iterations. EM can make tiny improvements for many iterations, and the incremental benefit may not justify compute time. A typical governance approach is: stop at max iterations or when improvement is small for several consecutive iterations.

Failure mode 1: state collapse (near-zero occupancy). A common failure is that one state receives almost no posterior mass: $\sum_t \gamma_t(k)$ is tiny. This can happen when K is too large or when initialization creates redundant states. State collapse is not always fatal (sometimes the model effectively chooses fewer than K states), but it undermines interpretability and can create numerical instability in parameter updates (division by small occupancy). A disciplined pipeline monitors effective sample size per state:

$$N_k^{\text{eff}} \equiv \sum_{t=1}^T \gamma_t(k),$$

and flags states with N_k^{eff} below a threshold.

Failure mode 2: outlier state. Another failure is a state that captures only a handful of extreme observations and develops extreme parameters (very large variance, very small variance, or extreme mean). In returns data, this often happens around crash days. Whether this is “bad” depends on your modeling intent. If you want a crash regime explicitly, an outlier state may be meaningful. But often it is an artifact: the model uses a state to mop up outliers because the emission family is too rigid. The diagnostic is to examine whether the state persists (high A_{ii}) and has substantial occupancy, or whether it is visited briefly for isolated points. A crash regime should have persistence and coherent characteristics; an outlier state often has near-zero duration and unstable emissions.

Failure mode 3: transition degeneracy (absorbing states). EM can produce transition matrices with entries near 0 or 1, including near-absorbing states where $A_{ii} \approx 1$ and $A_{ij} \approx 0$ for $j \neq i$. This can happen if the model overfits to a segmentation and then makes it extremely costly to leave a state. In trading, near-absorbing behavior can create posterior stickiness that may look stable but can also prevent timely adaptation. A diagnostic is to compute implied expected durations $1/(1 - A_{ii})$ and check whether they are plausible relative to the data frequency and the objectives.

Failure mode 4: oscillatory switching (churn). The opposite degeneracy is a model that switches too frequently. This often occurs when emissions are not well separated and the transition matrix is not diagonally dominant. The model explains noise by switching states. In trading, this is typically unacceptable because it creates high turnover if used for switching, and it produces unstable beliefs. A diagnostic is the posterior entropy and the number of state changes in the MAP path (most likely state sequence) under filtering. Excessive switching suggests either too many states, insufficient persistence constraints, or features that are too noisy.

Failure mode 5: local maxima with different meanings. Because EM can converge to different local maxima, you may find that different runs produce different regime interpretations with similar likelihood. This is not a numerical failure; it is an identifiability and optimization issue. The remedy is not to pick the prettiest segmentation. The remedy is to incorporate stability into model selection: prefer solutions that are consistent across nearby initializations and across time folds, and whose regime anchors (volatility, tail risk, correlation) are stable.

Failure mode 6: numerical underflow/overflow in likelihoods. Even with scaling or log-domain implementation, emission likelihoods can underflow if features are poorly scaled or if covariance matrices become ill-conditioned. In Gaussian emissions, an ill-conditioned Σ_k can yield unstable log-determinants and inverses. A disciplined pipeline enforces minimum variance floors, checks positive definiteness, and uses stable matrix decompositions (e.g., Cholesky) when implementing multivariate Gaussians. If you cannot guarantee numerical stability, your posterior beliefs become artifacts of floating-point errors rather than data.

The important message is that “EM converged” is not sufficient. You must verify that it converged to a *sensible* solution: states with meaningful occupancy, plausible persistence, stable emissions, and interpretable anchors.

14.7.4 Regularization and constraints (within scope; connect back to Ch. 12)

Chapter 12 taught us that regularization is not optional in finance; it is how we survive noisy targets and correlated predictors. HMM parameter learning has the same reality: maximum likelihood can overfit, especially with limited data or large K . Therefore, we need regularization and constraints that stabilize estimation and improve out-of-sample usefulness.

We emphasize four regularization mechanisms that fit naturally within Chapter 14 scope.

(1) Transition matrix smoothing (Dirichlet priors / pseudocounts). The MLE update for A in (14.44) uses fractional counts. If some transitions are rare, MLE can set them near zero, which can make the model brittle. A simple regularization is to add pseudocounts, equivalent to a Dirichlet prior on each row of A . Concretely, modify the update to

$$A_{ij}^{\text{new}} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j) + \kappa_{ij}}{\sum_{t=1}^{T-1} \gamma_t(i) + \sum_{j=1}^K \kappa_{ij}}, \quad (14.47)$$

where $\kappa_{ij} \geq 0$ are pseudocounts. A common choice is to set larger pseudocount on the diagonal to encourage persistence, and smaller equal pseudocounts off-diagonal. This is an explicit way to encode the belief that regimes persist, while still allowing data to override the prior if there is strong evidence.

This is directly analogous to ridge regularization in regression: we shrink estimates toward a structured baseline rather than letting noise push them to extremes.

(2) Emission parameter shrinkage and variance floors. In Gaussian emissions, covariance estimates can become unstable, especially in higher dimensions. A standard remedy is shrinkage:

$$\Sigma_k^{\text{shrunk}} = (1 - \lambda) \Sigma_k^{\text{MLE}} + \lambda D,$$

where D is a structured target (often diagonal or identity scaled), and $\lambda \in [0, 1]$ controls shrinkage. In univariate settings, a simple variance floor

$$\sigma_k^2 \leftarrow \max(\sigma_k^2, \sigma_{\min}^2)$$

prevents the model from collapsing variance and overfitting a small set of points.

These are not cosmetic. Without variance floors or shrinkage, EM can create ill-conditioned covariances that produce numerical instability and unrealistic state likelihoods. In trading, such models are fragile and fail exactly when you need them most.

(3) Parameter tying (structured emissions). As discussed in the specification section, you can constrain emissions to reduce degrees of freedom. For example, you may enforce $\mu_k = 0$ for all k in returns data, which reflects the belief that mean differences are small and difficult

to estimate, and you allow regimes to differ mainly in volatility. Alternatively, in multivariate settings you may enforce $\Sigma_k = c_k \Sigma_0$ to model regimes as volatility scaling of a common correlation structure. Parameter tying is a form of regularization because it reduces flexibility and forces the model to use regimes to represent the dominant variation you care about.

This connects directly to Chapter 12’s principle: choose a hypothesis class that is rich enough to represent the effect but constrained enough to generalize.

(4) Penalizing excessive switching (persistence constraints). Because the transition matrix controls switching frequency, you can regularize against churn by encouraging diagonal dominance. This can be done via priors (as above) or via explicit constraints such as $A_{ii} \geq a_{\min}$ for all i . Another approach is to add a penalty to the objective that increases with switching probability. While this is not standard in pure MLE, in trading it can be justified because frequent switching is often economically undesirable due to transaction costs and because regime labels become less meaningful.

A disciplined way to incorporate this within scope is to treat it as a model selection criterion: among models with similar likelihood, prefer those with higher persistence and lower churn, because they are more likely to be usable. This is analogous to preferring a simpler model with similar validation performance in supervised learning.

Regularization is also governance. In Chapter 12 we emphasized that regularization choices must be recorded and justified. The same applies here. If you impose a persistence prior, you must record its strength. If you impose covariance shrinkage, you must record λ and the target structure. If you impose variance floors, you must record the floor value and rationale. These are not “hyperparameters” in a casual sense; they are part of the model definition and must be treated as such.

Finally, regularization interacts with refit cadence. A model fitted on a short window needs stronger regularization because it has less data. A model fitted on a long window may need weaker regularization but faces a different risk: it may average across structural eras and produce regimes that are too broad. In a professional pipeline, you choose the training window and regularization jointly, guided by stability diagnostics and decision usefulness.

To close the parameter learning section, we highlight a central methodological message: learning an HMM is not merely running Baum–Welch. It is choosing a constrained, regularized model that can be estimated stably, defining a deterministic initialization and restart policy, monitoring convergence and failure modes, and embedding the entire procedure in a time-aware refit schedule. Done correctly, this turns the HMM from a retrospective segmentation tool into a reproducible component that produces online regime beliefs suitable for risk overlays and strategy adaptation.

14.8 Choosing the Number of Regimes

14.8.1 Model selection criteria: AIC/BIC and practical caveats

Choosing the number of regimes K is the point where regime detection can become either disciplined modeling or unconstrained storytelling. The danger is not merely that you overfit in-sample; the deeper danger is that you build a state space that is not stable enough to support decisions. In finance, the best K is rarely the one that maximizes likelihood. It is the one that produces a representation that is interpretable, persistent, robust under refits, and economically useful given trading frictions.

A traditional starting point is information criteria such as AIC and BIC, which formalize a penalty for model complexity. Let $\hat{\Theta}_K$ denote the maximum likelihood estimate for an HMM with K states and let

$$\ell_K \equiv \log p(x_{1:T}; \hat{\Theta}_K)$$

denote the maximized log-likelihood. Let p_K be the number of free parameters of the model. Then the Akaike Information Criterion (AIC) and Bayesian Information Criterion (BIC) are

$$\text{AIC}(K) = -2\ell_K + 2p_K, \quad (14.48)$$

$$\text{BIC}(K) = -2\ell_K + p_K \log T. \quad (14.49)$$

Lower is better. The difference is the penalty strength: BIC penalizes complexity more strongly as sample size grows because $\log T$ exceeds 2 for typical datasets.

To apply these criteria responsibly, you must count parameters correctly. For the transition matrix A with K states, each row has $K - 1$ free parameters (the row sum is 1), so A contributes $K(K - 1)$ free parameters. The initial distribution π contributes $K - 1$. Emission parameters depend on the emission family. For univariate Gaussian emissions, each state has (μ_k, σ_k^2) , so $2K$ parameters. For multivariate Gaussian with dimension d , each state has d mean parameters plus $d(d + 1)/2$ covariance parameters, so per state $d + d(d + 1)/2$, and total $K(d + d(d + 1)/2)$. If you estimate degrees of freedom for Student- t emissions, that adds K more parameters.

Already you can see why AIC/BIC can become blunt instruments in multivariate settings: parameter count rises quickly, and the criteria may favor very small K even if richer states are economically meaningful, or they may favor larger K if the likelihood gains dominate and the penalty is too mild relative to the true overfitting risk.

The practical caveats are therefore crucial:

Caveat 1: Likelihood improvements can be illusory in non-stationary data. Information criteria assume a stable data-generating process over the sample. Markets violate this. Adding more regimes can improve likelihood by fitting idiosyncratic subperiods that will never repeat. AIC/BIC will reward this fit even if it harms future usefulness. In other words, a model can win on information criteria and lose in live trading.

Caveat 2: EM local maxima distort comparisons. To compare ℓ_K across different K , you need to fit each model well. But EM can converge to different local maxima, and larger K

often makes the surface more non-convex. If you do not use consistent initialization and multiple restarts, your likelihood comparisons are contaminated by optimization artifacts. A model with smaller K may appear better simply because it is easier to optimize.

Caveat 3: Parameter constraints change the meaning of p_K . If you impose tying (e.g., common mean, scaled covariance, diagonal covariance), the effective number of parameters drops and the right penalty changes. Information criteria can still be used, but you must compute p_K for your constrained model, not for an unconstrained one. More importantly, constraints are often more important than K itself: a $K = 4$ model with strong tying can generalize better than a $K = 2$ model with overly flexible emissions.

Caveat 4: Decision usefulness is not in the objective. AIC/BIC optimize statistical fit, not trading outcomes. A regime model may be statistically optimal and economically irrelevant. For example, it might separate states by tiny mean differences that are not tradable. Or it might create a state that appears rarely but drives large likelihood improvements due to outliers, without offering actionable policies. Therefore, information criteria should be treated as a screening tool, not as the final arbiter.

In disciplined practice, you use AIC/BIC to generate a short list of plausible K values (often small: 2 to 5 in most trading applications), and then you evaluate those candidates by stability and economic criteria.

14.8.2 Stability under time folds: regime consistency tests

Because markets are time-varying, the most important selection principle is not “best fit,” but “stable enough to be used.” Stability must be assessed in a time-aware way, using folds that respect ordering. The goal is to test whether regimes learned on one period resemble regimes learned on another, and whether the inference behavior is consistent under refits.

A first stability requirement is *label consistency*. Since regime labels are arbitrary, you must align labels across folds using an anchor rule. For example, if your regimes are primarily volatility states, you can order states by their estimated volatility (or by a tail-risk statistic) and then map labels accordingly. Once you have a mapping, you can ask whether “state 1” means the same thing across time.

A second requirement is *parameter stability*. For each fold, you estimate parameters $\hat{\Theta}_K^{(m)}$ and examine how emission parameters and transition probabilities vary across folds. Some variation is expected (markets change), but extreme variation suggests that the state definition is not robust. For example, if the “high-vol” state has $\hat{\sigma}$ that ranges from moderate to extreme across folds, you may not have a coherent state. Or if the transition matrix flips from persistent to highly switching depending on the fold, the model is not stable enough to base turnover-sensitive policies on it.

A third requirement is *posterior stability*. Even if parameters vary, the real question is whether the filtered posterior behaves similarly. You can test this by running a walk-forward process: fit the model on an initial training window, filter forward on a test window, then refit and continue. For each candidate K , you record:

- posterior entropy (how confident the model is on average),
- switching frequency in the MAP filtered path,
- distribution of posterior mass (does it collapse to one state or remain diffuse),
- duration statistics implied by the posterior (how long the model stays in a state).

Models with too many states often produce high entropy (uncertainty) and high switching (churn), while models with too few states can produce overly confident but crude segmentation. The stability test is to see which candidate produces behavior that is consistent across folds and plausible given your decision use-case.

A fourth requirement is *regime consistency tests*. These tests evaluate whether states correspond to reproducible characteristics. For each state k , compute state-conditional summary statistics on each fold: volatility, tail quantiles, drawdown contribution, correlation measures (if multivariate), and any domain-specific anchors. Then test whether the ordering and separation of these statistics is preserved. For example, if state A is always higher volatility than state B across folds, that suggests a meaningful distinction. If sometimes state A is higher and sometimes lower, you may not have a stable regime definition.

The key point is that time-fold stability is a stronger and more relevant criterion than static AIC/BIC. In trading, you will refit repeatedly. If the regime meaning is not stable under refits, the model cannot be deployed coherently, regardless of its in-sample likelihood.

14.8.3 Overfitting regimes: “too many states” pathology

The “too many states” pathology is not subtle. It shows up in predictable ways that are worth naming explicitly, because once you know the symptoms you can diagnose them quickly.

Pathology 1: Redundant states (splitting a regime into clones). When K is larger than what the data support, the model often creates states with very similar emission parameters. For instance, two states might have nearly identical volatility and mean, differing only slightly. Why does the model do this? Because it can improve likelihood by allocating different subsegments of a broad regime to slightly different parameter values, even if those differences are not robust. In practice, these states are not economically distinct; they are statistical clones. They create unstable label assignments and increase switching without improving decisions.

Pathology 2: The outlier sponge. Another symptom is a state that exists primarily to capture outliers. It has very large variance or heavy tails (or, in degenerate cases, very small variance around an extreme mean), and it is visited only on a handful of days. This state can yield large likelihood improvements because outliers are expensive under Gaussian emissions. But it is usually not a “regime” in an operational sense. A regime should persist. Outliers are points. A crash regime can exist, but it should have coherent dynamics and duration beyond isolated spikes.

Pathology 3: Churn and indecision. Too many states often leads to frequent switching in the filtered posterior, or to persistent ambiguity where several states share probability mass. The model becomes indecisive because multiple states explain the data similarly well. This is not merely a cosmetic issue: if the posterior is diffuse, any decision rule that maps states to actions becomes noisy. The system will either churn (if you use hard labels) or become unstable (if you use probabilities that fluctuate).

Pathology 4: Unstable transitions (nonsense A). With too many states, the transition matrix can become ill-behaved: some states have near-zero outgoing probability except to one other state, creating an artificial two-state subchain; other states have transitions that do not reflect persistence; and small estimation noise can produce large changes in A . In trading, A is supposed to encode persistence. If A is unstable, your regime detector is unstable.

Pathology 5: Non-reproducibility under minor changes. Finally, the most important pathology is fragility. If adding or removing a small portion of data causes regimes to reorganize dramatically, your model is too complex relative to the signal. This shows up as high sensitivity to initialization, large variation across folds, and inconsistent regime anchors.

The remedy is not simply “choose smaller K .” The remedy is to align K with the information content of your features and with the decisions you intend to make. A model with $K = 5$ can be stable if features are informative and constraints are applied; a model with $K = 3$ can be unstable if features are noisy and unconstrained. But as a general rule in finance, starting small (often $K = 2$) is a disciplined baseline, and increasing K is justified only when you can show stable, decision-relevant differences.

14.8.4 Stress testing: crisis periods, low-volatility periods, structural breaks

Even if a choice of K looks good on average, regimes are most valuable and most dangerous at extremes: crises, unusually calm periods, and structural breaks. A regime model that fails in these contexts can cause precisely the failures it is meant to prevent. Therefore, K selection should include stress tests that focus on these regimes-of-regimes, so to speak.

Stress test 1: Crisis periods. Choose historically stressed intervals relevant to your asset class (e.g., liquidity crises, sudden repricings, high-volatility clusters). Evaluate whether the model assigns high probability to a coherent stress state during these periods under *filtered* inference. The key is filtered: smoothing will always look good in hindsight. You want to know whether the model detects stress with reasonable lag and whether the posterior remains elevated during sustained stress rather than spiking only on the worst days. For candidate K , measure:

- time-to-detection: how many days after a crisis onset does $\gamma_t(\text{stress})$ cross a threshold,
- persistence: does the stress state remain likely during the crisis,
- false negatives: does the model return to calm too quickly.

If increasing K creates multiple stress-like states that fragment the crisis episode, you may end up with ambiguity rather than a clear risk signal. In that case, smaller K or constrained emissions may be preferable.

Stress test 2: Low-volatility periods. Calm periods are where overfitting often hides. In low-vol environments, returns are small and noise dominates. A model with too many states can invent regimes that separate insignificant differences. Evaluate whether the model produces stable calm states or whether it churns. In calm periods, you should see high persistence and low switching. If the posterior is noisy in calm periods, the model is likely too complex.

Stress test 3: Structural breaks and new eras. Markets change structurally: policy regimes shift, inflation regimes change, microstructure evolves, participant composition changes. A regime model cannot prevent structural breaks, but it should not catastrophically misbehave when they occur. A useful stress test is a “holdout era” test: fit parameters on one era and evaluate filtered inference and decision rules on a later era with different characteristics. For each candidate K , ask:

- Do the inferred regimes remain interpretable under the new era?
- Do state occupancies collapse (model effectively fails)?
- Does switching become extreme (model cannot reconcile new patterns)?
- Do emission parameters become numerically unstable under refit?

A model with too many states often fails here because it learned era-specific micro-patterns rather than robust regime structure. A simpler model can generalize better because it represents only coarse differences.

Stress test 4: Regime boundary behavior. The most operationally important moments are boundaries: transitions into stress, transitions out, and oscillations near thresholds. Evaluate candidate K by how its filtered posterior behaves around boundaries. Is the posterior smooth enough to support risk scaling? Does it exhibit hysteresis-like behavior naturally (persistence) or does it flip rapidly? If you need to impose heavy smoothing on the posterior to make it usable, that suggests the model is too complex or the features are too noisy.

Stress test 5: Decision-level robustness with costs. Finally, because the purpose is trading, stress tests should include transaction costs and slippage assumptions consistent with Chapter 18’s eventual treatment (conceptually, without building the full microstructure model here). A regime model that yields small performance improvements before costs may be useless after costs if it causes more turnover. Increasing K often increases switching. Therefore, a cost-aware stress test frequently favors smaller K or more persistent transitions.

Putting it together: a disciplined selection protocol. A coherent protocol for choosing K within Chapter 14 scope is:

1. Choose a small candidate set, typically $K \in \{2, 3, 4, 5\}$, guided by the decision use-case.
2. For each K , fit the model with deterministic initialization and multiple controlled restarts; record best stable solution.
3. Compute AIC/BIC as a coarse screen, but do not decide solely on them.
4. Run walk-forward refit and filtering to assess posterior stability and switching behavior.
5. Evaluate regime anchor stability across time folds (volatility ordering, tail risk separation, etc.).
6. Stress test on crisis, calm, and structural-break intervals with filtered inference.
7. Choose the smallest K that delivers stable, interpretable regimes and improves decisions relative to baselines.

The last step is important: prefer parsimony. In finance, the price of complexity is not only estimation risk; it is operational fragility. A regime model exists to reduce the cost of non-stationarity, not to create a new source of instability.

In short, selecting K is an exercise in disciplined approximation. Information criteria can help, but time-fold stability and stress behavior matter more. “Too many states” manifests as redundancy, outlier sponges, churn, and fragility. The right K is the one whose regimes remain meaningful under refits, remain useful under costs, and remain honest under the causality constraints that define real trading.

14.9 Feature Design for Regime Detection (Within Ch. 14 Scope)

14.9.1 Inputs: returns, volatility proxies, spreads, drawdowns, volume (conceptual)

Regime detection is often introduced as if the latent state were a property of returns alone. In practice, regimes are easier to infer when you give the model observables that respond directly and persistently to the environmental changes you care about. The art is to include inputs that (i) have clear economic meaning, (ii) are available reliably in your data pipeline, (iii) can be constructed causally, and (iv) are low-dimensional enough to estimate robustly. The point is not to feed the HMM “everything.” The point is to feed it a small set of indicators that make regimes separable.

The first and most fundamental input is the return itself. Depending on the asset class and the regime question, you might use simple close-to-close returns r_t , log returns, or returns standardized by a volatility estimate. Returns carry directional information and they also carry variance information implicitly (through magnitude). However, raw returns are extremely noisy,

and regimes that differ mainly in volatility may be hard to separate using r_t alone unless the volatility difference is large. Therefore, even when returns are included, they are rarely sufficient as the sole observation in a robust regime detector.

A second class of inputs are volatility proxies. These are often the most regime-informative features because volatility clustering is one of the strongest stylized facts in financial time series. Examples include rolling standard deviation, exponentially weighted moving standard deviation, rolling average of absolute returns, realized volatility estimators (for higher-frequency data), or implied volatility proxies (e.g., an index such as VIX or instrument-specific implied vol). Volatility proxies act like thermometers: they do not tell you why conditions changed, but they tell you that conditions changed in a way that matters for risk and strategy performance.

A third class of inputs are spread and liquidity proxies. In many markets, stress regimes are not only about volatility; they are about liquidity and transaction costs. Even in daily data, simple proxies can capture this: bid-ask spreads (if available), high-low ranges, measures of intraday volatility, or deviations between related instruments (basis). In equities or ETFs, volume and turnover can serve as partial liquidity indicators, though volume can be ambiguous: high volume can signal both healthy participation and panic liquidation. The virtue of spreads is that they tend to widen in stress, making them helpful for identifying regimes where trading becomes expensive and risk limits should tighten.

A fourth class of inputs are drawdown and path-dependent risk measures. Regimes are often experienced not as “higher variance” in an abstract sense, but as sequences of negative outcomes and capital erosion. Simple drawdown measures can encode this. For example, a rolling maximum-to-current drawdown, rolling count of negative return days, or rolling cumulative return. These features are path-dependent and therefore can capture slow-moving deterioration that volatility proxies sometimes miss. They can also help distinguish regimes where volatility is high but returns are positive (e.g., trending bull markets) from regimes where volatility is high and returns are negative (crash-like states).

A fifth class of inputs are volume and flow proxies (conceptual). Volume can rise in both breakout trend regimes and stress regimes. Including volume-related features can therefore help distinguish “high vol with participation” from “high vol with liquidity stress,” but only if you have additional context (e.g., spread measures, correlation measures, or cross-asset indicators). Within Chapter 14 scope we keep this conceptual: the point is that volume can be informative, but it is rarely definitive on its own.

The key design principle is that features should reflect the regime concept you want to infer. If the goal is volatility regimes, include volatility proxies and possibly absolute returns. If the goal is risk-on/risk-off across assets, include cross-asset returns or correlations. If the goal is liquidity stress, include spread proxies and possibly volume. A regime model cannot infer what it cannot see. The latent state is a compression of the observation history; if your observations omit the dimension of change you care about, the regime detector will either fail or will infer regimes that correspond to something else.

14.9.2 Scaling and robustness: winsorization, volatility normalization

Once you choose inputs, scaling is the difference between a regime detector that learns stable states and one that chases outliers. HMM inference depends directly on emission likelihoods. If one feature has a wildly larger scale than others, it will dominate the likelihood and effectively define the regimes by itself. If outliers are not handled, the model may create regimes that exist primarily to absorb extreme points. Robust scaling is therefore not a cosmetic preprocessing step; it is part of the model specification.

A first robustness tool is winsorization (or more generally, outlier capping). The idea is to cap extreme values of a feature at some percentile (e.g., 1st and 99th) computed causally (rolling or expanding) so that a few extreme observations do not dominate parameter estimates. In returns, a single crash day can be several standard deviations away from typical variation, and if you fit a Gaussian emission model, the likelihood penalty for that day can be enormous. Winsorization can prevent the model from dedicating an entire state to one or two days unless that is your explicit intent (e.g., a crash regime). The discipline is to decide what you want: if you want the model to treat extreme moves as evidence of a stress regime, you may cap less or not at all; if you want the model to detect regimes based on persistent conditions rather than single-day spikes, capping can help.

A second tool is volatility normalization. Raw returns have time-varying scale. One common regime concept is precisely that scale changes (volatility regimes). But there are cases where you do not want scale to dominate because you want regimes related to other behavior (e.g., correlation regimes or trend persistence). In those cases, normalizing returns by an estimate of contemporaneous volatility can isolate directional dynamics from scale dynamics. For example, define

$$\tilde{r}_t = \frac{r_t}{\hat{\sigma}_t},$$

where $\hat{\sigma}_t$ is a causal volatility estimate computed from past data. Then $\tilde{r}_t$ is a dimensionless standardized return. If your emission model is Gaussian, $\tilde{r}_t$ can be more stable, and regimes may become more interpretable in terms of mean or tail asymmetry rather than variance.

However, volatility normalization is not free. If your regime model is meant to detect volatility regimes, normalizing returns can remove the very signal you want. Therefore, normalization is a choice tied to the regime concept. A disciplined workflow is to treat normalization as part of the feature set rather than a mandatory preprocessing step: include both r_t and $|r_t|$ or $\hat{\sigma}_t$ explicitly if volatility is the regime driver; normalize only when you want to suppress scale.

A third tool is feature standardization across dimensions. If you use a multivariate x_t , you often want features to have comparable magnitude so that one does not dominate. Standardization can be done by subtracting a rolling mean and dividing by a rolling standard deviation. But this immediately raises causality concerns: the mean and variance must be estimated using only past information. Full-sample standardization is a silent leak and will contaminate regime inference. The correct approach is rolling or expanding standardization with explicit window lengths recorded in the run manifest.

Finally, there is an important conceptual point: robustness does not mean “make the data look Gaussian.” It means “make estimation stable and decisions reliable.” Sometimes, keeping

heavy tails is exactly what you want because heavy tails are part of stress regimes. In such cases, a Student- t emission or a mixture model may be a better solution than aggressive winsorization. The right robustness tool depends on whether you want outliers to define regimes or merely to be tolerated within regimes.

14.9.3 Causality-safe rolling features (link to Ch. 05 without re-teaching it)

Chapter 05 established the machinery and discipline for feature pipelines: rolling windows, alignments, missing data handling, and most importantly, no look-ahead. Chapter 14 does not repeat those engineering details, but regime detection makes the causality requirement even more central because the model is sequential and because smoothing is always lurking as a temptation. Therefore, we restate the regime-specific causality rule in a compact way:

Causality rule for regime features: every component of x_t must be measurable with respect to the information set at time t . If your trading decision uses γ_t at the close of day t to trade at $t + 1$, then x_t may use prices up to the close of day t , but it may not use any price from day $t + 1$ onward.

Rolling features are the main place where causality violations occur. A causality-safe rolling feature uses a trailing window. For example, a rolling mean of returns with window L is

$$m_t = \frac{1}{L} \sum_{\ell=0}^{L-1} r_{t-\ell},$$

which uses returns from $t - L + 1$ through t . A causality-unsafe version would use a centered window (including future returns), which must never be used for regime inference if you plan to trade.

The same applies to volatility proxies. A trailing realized volatility estimator might be

$$\hat{\sigma}_t^2 = \frac{1}{L} \sum_{\ell=0}^{L-1} (r_{t-\ell} - m_t)^2,$$

or an exponentially weighted version with decay parameter λ , where the weights are applied to past squared returns. All such estimators must be computed using past data only. In a governance-native pipeline, you record L or λ as part of the parameter registry.

Drawdown features are also rolling/path-dependent. A simple drawdown relative to the trailing peak is

$$\text{DD}_t = \frac{P_t}{\max_{u \in \{t-L+1, \dots, t\}} P_u} - 1,$$

where P_t is price and the maximum is computed over a trailing window. Again, the window is trailing, not centered.

A subtle but common causality violation arises through *global operations*. These include:

- standardizing features using the full-sample mean and variance,
- computing PCA or other dimensionality reduction on the full sample and then applying it historically,

- choosing winsorization thresholds using full-sample quantiles.

All of these embed future data into the feature values at time t . The fix is always the same: compute any statistic used in preprocessing on a rolling or expanding basis that ends at t . If that is too expensive, use an expanding estimate with periodic updates (e.g., monthly) and treat the update schedule as part of the information set. The point is not that you must update every day; the point is that you must never update using the future relative to the time you are simulating.

The link back to Chapter 05 is therefore conceptual: regime detection does not require new feature engineering ideas beyond what you already built; it requires stricter enforcement and better bookkeeping because the regime posterior is a sequential belief and will reflect any leakage in features immediately. If you want one simple test to validate causality: recompute features on the prefix $1 : t$ and on the full sample $1 : T$ and verify that the feature value at time t is identical. If it is not, you have a leak.

14.9.4 Event timing: daily close vs intraday approximations (conceptual)

Feature design for regimes is inseparable from event timing. In trading, the model does not live in continuous time; it lives on a decision schedule. A regime detector can be built on daily data, intraday data, or some hybrid approximation. Within Chapter 14 scope we keep this conceptual but precise enough to avoid common timing errors.

In daily-close systems, a typical convention is:

1. Observe daily close prices and compute features at the close of day t .
2. Compute the filtered regime posterior γ_t using $x_{1:t}$.
3. Use γ_t to decide positions for day $t + 1$ (executed at open or close depending on the strategy).

Under this convention, x_t can include close-to-close returns, end-of-day volatility proxies, and rolling indicators computed using closes. This is the cleanest setting for pedagogy and for many medium-frequency strategies, and it aligns naturally with governance and backtesting discipline.

Intraday trading complicates timing because you have multiple decision points within a day and because “end of day” is no longer the relevant boundary. A true intraday regime detector would define t at intraday intervals (e.g., 5-minute bars) and compute features and filtering at that frequency. This is possible, but it changes microstructure considerations and data requirements (treated later in Chapter 18). Within Chapter 14 scope, the key is to avoid pretending that you have intraday regime detection when you only have daily data.

That said, practitioners often want *intraday approximations* while still operating a daily model. Examples include:

- using high-low range within the day as a volatility proxy available at the close,
- using realized volatility computed from intraday bars but summarized to a daily value,
- using volume and spread measures aggregated over the day.

These approximations are acceptable as long as you respect the information set: you can use intraday information up to the decision time, but you cannot use intraday information that would not have been observed yet. In a daily close model, you may use the full day’s intraday data because it is known at the close; you may not use tomorrow’s intraday data. This sounds obvious, but timing errors often occur when data vendors provide daily “features” that are computed with nontransparent conventions.

A further timing issue is *asynchronous data*. If you include cross-asset features (e.g., equities and FX) in a single x_t , their trading hours may differ. A regime detector that uses “daily close” must define what close means across assets, otherwise it can inadvertently include information from an asset that closes later as if it were available earlier. This is a classic leakage mechanism in multi-asset systems. The fix is to align on a common timestamp and, if needed, lag features from later-closing markets to ensure they are not used prematurely.

Finally, the choice of decision frequency interacts with what regimes you can meaningfully detect. Some regimes are slow (volatility regimes over weeks), some are fast (liquidity shocks over hours). A daily model can detect slow regimes reasonably; it will detect fast regimes late by construction. This is not a failure; it is a design choice. The correct question is: is the lag acceptable for the decision you want to improve? If your goal is to reduce exposure during prolonged stress, a daily regime detector may be sufficient. If your goal is to avoid intraday crash moves, a daily regime detector is structurally too slow, and the solution lies in execution and microstructure (later chapters), not in forcing daily regimes to do intraday work.

To conclude: feature design for regime detection is a deliberate selection of a small, interpretable observation vector that makes regimes separable. Robust scaling prevents outliers and scale disparities from dominating inference. Causality-safe rolling features must be constructed exactly as in Chapter 05, with even stricter enforcement because sequential inference magnifies leakage. And event timing must be explicit: daily close models are clean and governable; intraday approximations are acceptable only when their information availability is correctly aligned. If these principles are respected, the regime detector’s outputs will reflect genuine shifts in observed market behavior rather than artifacts of preprocessing or timing mistakes.

14.10 From Regime Beliefs to Trading Decisions

14.10.1 Regime-conditioned forecasting: mixture views and simple decision rules

An HMM becomes relevant to trading only when its regime beliefs change what you do. The filtered posterior

$$\gamma_t(k) \equiv p(s_t = k \mid x_{1:t})$$

is not the end product; it is an intermediate representation of uncertainty about the market environment. The trading question is: how do we map γ_t into forecasts, exposures, and risk controls in a way that respects causality, avoids brittle switching, and produces a system whose behavior is explainable?

A clean starting point is the mixture view of prediction. Under an HMM, a one-step-ahead

predictive distribution for a return-like quantity r_{t+1} can be written as a mixture of state-conditional forecasts. Conceptually, imagine that in regime k the next return has conditional distribution $p(r_{t+1} \mid s_{t+1} = k)$ with conditional mean μ_k and conditional variance σ_k^2 . Then the predicted probability of being in state k at $t + 1$, given information at t , is

$$w_{t+1}(k) \equiv p(s_{t+1} = k \mid x_{1:t}) = \sum_{j=1}^K \gamma_t(j) A_{jk}.$$

The predictive return distribution is the mixture

$$p(r_{t+1} \mid x_{1:t}) = \sum_{k=1}^K w_{t+1}(k) p(r_{t+1} \mid s_{t+1} = k),$$

and the predictive mean and variance follow (for simple moment-based decision rules) as

$$\hat{\mu}_{t+1} \equiv \mathbb{E}[r_{t+1} \mid x_{1:t}] = \sum_{k=1}^K w_{t+1}(k) \mu_k,$$

$$\hat{\sigma}_{t+1}^2 \equiv \text{Var}(r_{t+1} \mid x_{1:t}) = \sum_{k=1}^K w_{t+1}(k) \sigma_k^2 + \sum_{k=1}^K w_{t+1}(k) \mu_k^2 - \left(\sum_{k=1}^K w_{t+1}(k) \mu_k \right)^2.$$

This mixture view highlights something that is easy to miss: even if each regime is Gaussian, the predictive distribution can be heavy-tailed because it is a mixture. In decision terms, regime uncertainty widens predictive risk. The system should react to that.

In practice, most trading systems do not rely on HMMs to extract precise return predictability μ_k because mean forecasting is weak and unstable. Instead, they use regimes to adjust risk and to select strategies whose edge depends on the environment. Still, the mixture framework yields simple, transparent decision rules that can be used as baselines or as components:

Rule 1: sign-of-mixture-mean. If you have a regime-conditioned expected return μ_k (perhaps derived from a simple model within each regime), then a naive directional decision is

$$\text{position sign at } t + 1 \propto \text{sign}(\hat{\mu}_{t+1}).$$

This is pedagogically clean but often too noisy for real returns unless signals are strong.

Rule 2: thresholded belief. If one regime corresponds to “stress” and you have a trading policy that should be active only outside stress, define

$$\text{trade if } \gamma_t(\text{stress}) < \tau.$$

This is effectively a gating rule. Importantly, the gate is a probability threshold, not a hard state label.

Rule 3: convex blending of actions. If each regime corresponds to an action vector a_k (e.g., target exposures), then the probability-weighted action is

$$a_t = \sum_{k=1}^K \gamma_t(k) a_k.$$

This rule is the simplest way to avoid brittle switching. It respects uncertainty and creates smooth behavior.

These simple rules illustrate a broader principle: regime beliefs should typically be used as *weights* or *gates*, not as categorical labels. A regime model provides a distribution; a trading system should use that distribution.

14.10.2 Risk overlay: exposure caps, risk targeting, and volatility regimes [\(link to Ch. 10\)](#)

The most robust use of regimes in practice is risk overlay. This is because volatility and stress regimes are among the most reliable regime distinctions, and risk management is where even coarse regime signals can add value. This section connects directly to Chapter 10's theme: volatility modeling and risk targeting. The difference is that Chapter 10 treated volatility as a continuous estimate; here we treat volatility as regime-dependent and uncertain.

Suppose the regimes primarily reflect different volatility levels. For each regime k , let σ_k denote the regime-conditional volatility of returns (or a proxy). At time t , the regime belief is $\gamma_t(k)$ and the predicted regime distribution for $t + 1$ is $w_{t+1}(k)$ as above. A natural predictive volatility is the mixture variance's square root, but for risk targeting we can use the mixture second moment:

$$\hat{\sigma}_{t+1}^{\text{pred}} \approx \left(\sum_{k=1}^K w_{t+1}(k) \sigma_k^2 \right)^{1/2},$$

which ignores between-regime mean variation (usually small in returns). Then risk targeting is conceptually unchanged:

$$\text{leverage}_{t+1} = \frac{\sigma^*}{\hat{\sigma}_{t+1}^{\text{pred}}},$$

where σ^* is your target volatility. This is an HMM-informed version of volatility targeting: when the posterior shifts toward high-vol regimes, predicted volatility rises, and leverage is cut.

Regime beliefs can also implement exposure caps. Define a maximum leverage schedule by regime:

$$L_{\max}(k) \quad \text{for each } k,$$

then the effective cap under uncertainty can be a convex combination:

$$L_{\max,t} = \sum_{k=1}^K \gamma_t(k) L_{\max}(k),$$

or a conservative bound such as

$$L_{\max,t} = \min_{k: \gamma_t(k) > \epsilon} L_{\max}(k),$$

which caps leverage by the most restrictive regime that has nontrivial posterior mass. The convex combination produces smoother behavior; the conservative bound is safer but can be overly pessimistic. Which is appropriate depends on your tolerance for drawdowns versus opportunity cost.

A particularly useful overlay in crisis-aware systems is a *de-risking function* driven by stress probability. In a two-state model (calm vs stress), define $p_t \equiv \gamma_t(\text{stress})$. A smooth de-risking function might be

$$\text{exposure multiplier} = g(p_t),$$

where g is decreasing, with $g(0) = 1$ and $g(1) \approx 0$. For example, $g(p) = \max(0, 1 - p)$ is linear; $g(p) = \exp(-cp)$ is exponential; and a logistic curve yields a soft threshold. The key is not the exact functional form; it is that the mapping is smooth and documented. It makes the system robust to posterior noise near boundaries.

This is where regime uncertainty matters as a risk input. In continuous volatility targeting, uncertainty about volatility is often ignored. In regime models, uncertainty is visible: if p_t is near 0.5, the system is unsure whether it is in calm or stress. In such cases, a conservative risk manager will reduce exposure because uncertainty itself is a risk factor. One simple mechanism is to penalize posterior entropy:

$$H_t = - \sum_{k=1}^K \gamma_t(k) \log \gamma_t(k),$$

and reduce exposure as H_t increases. This can be interpreted as “when the regime detector is confused, reduce risk.” Again, this is a transparent rule that respects uncertainty rather than hiding it.

The governance benefit of regime-based overlays is that they are explainable. When exposure is reduced, you can attribute it to an increase in stress probability or to an increase in predicted volatility driven by regime beliefs. This is precisely the kind of traceability that we want in production systems.

14.10.3 Strategy switching: trend vs mean reversion conditioned on $p(s_t \mid x_{1:t})$

Risk overlays are often the first use of regimes, but the most conceptually exciting use is strategy switching: choosing between distinct alpha engines based on the inferred environment. This is where the book’s earlier chapters connect: Chapter 07 developed trend following; Chapter 08 developed mean reversion and pairs; and Chapter 13 discussed how single models fail under non-stationarity. Regime beliefs provide a disciplined mechanism to *condition* strategy usage without pretending we can perfectly forecast returns.

The design pattern is: each strategy has regime-dependent performance, and the regime model provides a probability distribution over regimes. We then allocate between strategies as a function of that distribution.

Consider two strategies:

- A trend strategy that tends to perform better in persistent directional markets (often associated with moderate volatility and sustained moves).

- A mean-reversion strategy that tends to perform better in range-bound or liquidity-providing environments, but can suffer in trending crashes or in sudden volatility expansions.

We want to use regimes to adjust weights between them.

The simplest structure is to define, for each regime k , a target weight for the trend strategy w_k^{trend} and for the mean-reversion strategy w_k^{mr} , with $w_k^{\text{trend}} + w_k^{\text{mr}} = 1$ (for a two-strategy blend). Then, given filtered regime beliefs,

$$w_t^{\text{trend}} = \sum_{k=1}^K \gamma_t(k) w_k^{\text{trend}}, \quad w_t^{\text{mr}} = 1 - w_t^{\text{trend}}. \quad (14.50)$$

This is convex blending. It avoids hard switching and makes strategy allocation a smooth function of beliefs.

Where do the regime-specific weights come from? Within Chapter 14 scope, the important point is conceptual: they must be estimated with time-aware evaluation (Chapter 06 mechanics) and disciplined model selection (Chapter 12 mindset). Practically, you would:

1. Run the regime model on a training window and compute filtered beliefs (not smoothed) to avoid leakage.
2. Partition time into regimes using a probabilistic weighting (e.g., weight each time t by $\gamma_t(k)$) rather than hard labels.
3. Estimate each strategy's performance metrics conditional on regimes: expected return, volatility, drawdown, tail risk.
4. Choose weights w_k^{trend} that reflect which strategy is preferable in regime k under your objectives and constraints.

The discipline is to treat regimes as soft assignments. When you compute “strategy performance in regime k ,” you compute weighted performance using posterior probabilities, not by chopping the sample into blocks based on smoothed labels.

A more sophisticated but still transparent approach is to compute a regime-conditioned *expected utility* for each strategy. For regime k , define a simple utility measure

$$U_k(\text{strategy}) = \hat{\mu}_k - \lambda \hat{\sigma}_k^2,$$

or a drawdown-penalized proxy. Then the regime-belief-weighted utility at time t is

$$U_t(\text{strategy}) = \sum_{k=1}^K \gamma_t(k) U_k(\text{strategy}),$$

and you allocate to the strategy with higher U_t , or you blend weights using a softmax:

$$w_t(\text{trend}) = \frac{\exp(\eta U_t(\text{trend}))}{\exp(\eta U_t(\text{trend})) + \exp(\eta U_t(\text{mr}))},$$

where η controls how sharply you switch. This approach still avoids hard classification because it maps uncertainty into probabilistic allocation.

The key trading insight is that strategy switching is not about predicting returns; it is about avoiding environments where a strategy's known failure mode dominates. Mean reversion often fails in strongly trending markets and in volatility expansions; trend often underperforms in choppy, mean-reverting markets with frequent reversals. Regimes give you a structured, auditable way to reduce exposure to a strategy in its known bad environments, without requiring perfect timing. Even if regime detection lags, reducing exposure during sustained stress can materially improve drawdowns.

Two cautions are essential.

First, switching itself has costs. Even if you blend weights smoothly, changing weights can create turnover. Therefore, strategy switching must be evaluated with transaction costs in mind and should often include inertia (e.g., do not change weights too aggressively in response to small posterior fluctuations). This can be implemented by smoothing the weights over time causally (e.g., an exponential moving average of w_t) or by imposing a minimum holding period for allocation changes.

Second, regimes can drift. If you learned that regime k favors trend in one era, that relationship may weaken in another. Therefore, regime-conditioned performance estimates must be updated under a refit schedule, just like model parameters. Strategy switching is not “fit once and forget.” It is a living policy.

14.10.4 Regime uncertainty as a risk input (avoid hard classification)

The most common mistake in regime-based trading is to treat s_t as a known label. The entire point of inference is that s_t is unknown and must be represented probabilistically. Hard classification discards uncertainty and amplifies noise near regime boundaries. It also encourages narrative overconfidence: “we are in regime 2,” as if that were a fact. In reality, you have a belief distribution that may be diffuse, and diffuse beliefs should lead to cautious actions.

There are three practical reasons to avoid hard classification:

Reason 1: boundary instability. Near a regime transition, the posterior can be close to 50/50 between two states. A hard rule will flip labels back and forth if the posterior oscillates around the threshold. This creates churn in decisions. A probabilistic rule yields small adjustments rather than flips.

Reason 2: model error and misspecification. Even if the posterior is confident, the model may be wrong because emissions are misspecified or because the market has changed. Treating a model output as a fact compounds model risk. Treating it as a probability invites risk controls.

Reason 3: uncertainty is informative. High posterior entropy signals that the current observation is not clearly explained by any learned regime. This can happen during transitions, during structural breaks, or when the market enters a new environment not seen in training. In

such cases, uncertainty is itself a warning signal. A hard classifier hides that warning by forcing a label.

Therefore, a disciplined regime-based trading system treats regime uncertainty as part of risk management. There are several straightforward mechanisms:

Entropy-based de-risking. As above, compute posterior entropy H_t . Define an exposure multiplier that decreases with H_t :

$$m_t = 1 - \alpha \frac{H_t}{\log K},$$

where $H_t / \log K$ is a normalized entropy between 0 and 1, and $\alpha \in [0, 1]$ controls the strength. This rule reduces exposure when the model is uncertain.

Worst-case mixing. If certain regimes are dangerous (e.g., stress), you can act on the worst-case probability mass. For example, if stress corresponds to a set of regimes $\mathcal{S}$, define

$$p_t^{\text{stress}} = \sum_{k \in \mathcal{S}} \gamma_t(k),$$

and then apply risk limits as a function of p_t^{stress} . This uses uncertainty appropriately: even if stress is not the most likely single state, a high cumulative stress probability leads to de-risking.

Mixture risk measures. Instead of computing VaR or expected shortfall under a single regime, compute it under the predictive mixture distribution. Because mixtures widen tails, this tends to produce more conservative risk estimates when regime uncertainty is high. This approach is consistent with the probabilistic nature of the model and avoids the false precision of selecting one regime and ignoring others.

Hysteresis in decision rules. Uncertainty near boundaries can be managed by requiring persistence before major decision changes. For example, reduce risk only if p_t^{stress} stays above a threshold for M consecutive days, and restore risk only if it stays below a lower threshold for M' days. This is not hard classification; it is a temporal stability rule. It reduces churn and aligns with the economic idea that regimes persist.

In summary, the mapping from regime beliefs to trading decisions should be governed by three principles. First, use regimes primarily where they are strongest: risk overlays and environment-dependent strategy weighting rather than precise return prediction. Second, implement switching and scaling as smooth functions of posterior beliefs to avoid brittle behavior. Third, treat regime uncertainty as a first-class risk input. A regime model does not remove uncertainty; it makes uncertainty explicit. A good trading system respects that and turns it into disciplined, auditable decisions rather than into overconfident regime narratives.

14.11 Evaluation: What Counts as Evidence a Regime Model Helps?

14.11.1 Out-of-sample protocol: walk-forward, refit cadence, and audit trails

A regime model is easy to “validate” if you allow yourself to look at the whole sample, smooth the states, and tell a compelling story about how the model “identified” stress and calm. None of that is evidence. Evidence, in the sense that matters for trading systems, is performance and behavior under a protocol that mirrors how the model would be used live: parameters refit using only past data, regime beliefs filtered (not smoothed), trades executed with correct timing, and results logged with enough detail that an independent reviewer could reproduce them and diagnose failures. This section defines what counts as an evaluation protocol and why looser protocols produce false confidence.

The baseline out-of-sample protocol for regime models is a walk-forward process. We choose a training window rule (expanding or rolling), a refit cadence, and a trade horizon. Let t index decision times under a clearly defined market calendar. Choose refit times

$$\tau_1 < \tau_2 < \cdots < \tau_M,$$

for example the first trading day of each month or quarter. At each refit time τ_m , estimate HMM parameters using only data up to τ_m :

$$\hat{\Theta}_{\tau_m} = \text{FitHMM}(x_{1:\tau_m}; K, \text{emissions, constraints, init policy}).$$

Then, for each day t in the subsequent interval $\tau_m \leq t < \tau_{m+1}$, compute filtered beliefs

$$\gamma_t(k) = p(s_t = k \mid x_{1:t}; \hat{\Theta}_{\tau_m}),$$

and map them into trading actions applied at the next execution time (e.g., $t + 1$ open or close). This produces a simulated sequence of decisions that is causally valid: at time t you use only past data and parameters estimated from past data.

Two design choices require particular discipline.

First, the refit cadence is not a cosmetic hyperparameter; it is part of the system definition. A daily refit may overreact to noise and increase instability, while a quarterly refit may fail to adapt to changing conditions. The cadence interacts with the transition matrix (persistence) and with feature stability. A proper evaluation therefore treats cadence as a design variable that is tested over a small set of plausible values and chosen based on robustness, not based on the best backtest outcome.

Second, the training window rule matters. An expanding window uses all data since inception, increasing sample size but mixing different market eras. A rolling window uses only recent history, improving relevance but reducing sample size and increasing estimation error. For regimes, rolling windows can be attractive because regime definitions can drift, but rolling windows demand stronger regularization. The evaluation must therefore record the window rule explicitly and examine sensitivity.

This brings us to audit trails. A regime model evaluation is not just a performance report; it is a governed experiment. For each refit, you should record at minimum:

- the exact training span (start/end dates, missingness summary),
- the model spec (K, emission family, constraints, regularization, variance floors),
- initialization policy and seeds (deterministic, plus any controlled restarts),
- EM convergence traces (log-likelihood by iteration, stop reason),
- fitted parameters ($\hat{A}$, $\hat{\pi}$, emission parameters),
- state occupancies and implied durations,
- filtered posterior time series produced in the subsequent test interval.

This audit trail is what turns a regime claim into a reproducible result. It also enables failure analysis: when the model behaves strangely, you can inspect whether the issue came from parameter estimation instability, feature drift, or posterior inference.

Finally, the evaluation must enforce timing discipline. If features use close data, then the regime belief γ_t becomes available at the close. If you trade at the close using γ_t , you have assumed instantaneous execution. If you trade at $t + 1$ open, you have a realistic one-step delay. The evaluation should state and implement the chosen convention consistently. Many regime “successes” vanish when you impose even a modest delay, which is a sign that the regime signal is not truly predictive but rather contemporaneous with the move.

14.11.2 Metrics: likelihood, regime hit-rate, trading metrics, drawdown behavior

What should you measure? A regime model has two roles: it is a statistical model of observations, and it is a decision aid for trading. Therefore, evaluation metrics must address both roles. Relying on only statistical fit (likelihood) risks building an economically irrelevant segmentation; relying on only trading performance risks overfitting a noisy backtest. A disciplined evaluation reports a small, interpretable set of metrics across four categories.

(1) Statistical fit: log-likelihood and predictive likelihood. The in-sample log-likelihood $\ell = \log p(x_{1:T}; \hat{\Theta})$ is useful as a sanity check and for EM convergence, but it is not evidence of out-of-sample usefulness. A stronger metric is *predictive* log-likelihood on held-out periods under the walk-forward protocol. For each test day $t + 1$, compute the one-step predictive density under the filtered belief and model parameters at time t :

$$p(x_{t+1} \mid x_{1:t}; \hat{\Theta}_{\tau_m}) = \sum_{k=1}^K p(x_{t+1} \mid s_{t+1} = k; \hat{\theta}_k) p(s_{t+1} = k \mid x_{1:t}; \hat{\Theta}_{\tau_m}),$$

and sum log densities over the test window. This is a proper scoring rule: it rewards calibrated predictive distributions, not just nice segmentations. If a regime model cannot improve predictive likelihood relative to a simpler baseline (e.g., single-state model) on held-out data, that is a warning sign.

(2) Regime “hit-rate” metrics (with caution). Practitioners often want a hit-rate: did the model “correctly” identify stress? The problem is that regimes are latent; there is no ground truth. Therefore, hit-rate is only meaningful if you define an external proxy for “stress” such as realized volatility exceeding a threshold, large drawdowns, or a known crisis label set. You can then measure alignment between high stress probability and these proxies. For example, define a stress indicator $y_t \in \{0, 1\}$ and compare it to $p_t^{\text{stress}} = \gamma_t(\text{stress})$ via ROC-AUC, precision/recall at thresholds, or calibration plots.

These metrics can be useful, but they must be interpreted cautiously. A model can score well by merely tracking volatility, which may be fine if volatility is your target. But if your regime concept is broader (liquidity, correlation), volatility-based hit-rate is incomplete. Therefore, regime hit-rate metrics should be treated as diagnostic, not definitive.

(3) Trading metrics: returns, Sharpe, turnover, costs. Ultimately, regimes are evaluated by their effect on a trading system. This does not mean maximizing Sharpe at all costs; it means measuring whether the regime overlay or switching policy improves outcomes robustly. The standard set includes:

- annualized return and volatility,
- Sharpe ratio (and possibly Sortino, but with consistent definitions),
- maximum drawdown and drawdown duration,
- turnover and an explicit cost model (even simple),
- tail metrics such as worst 1% day, expected shortfall proxy, or crash-period performance.

The key is to report these metrics for the baseline strategy and for the regime-enhanced strategy under the same execution and cost assumptions. A regime model that improves Sharpe slightly but doubles turnover may not be worthwhile. A regime model that reduces drawdowns materially with modest return impact may be valuable even if Sharpe changes little.

(4) Drawdown behavior and “when it helps.” Regimes are often justified as risk controls. Therefore, evaluation should not only report maximum drawdown; it should show drawdown behavior across time. For example, compute performance conditional on stress probability quantiles: how does the strategy behave when p_t^{stress} is high versus low? Or compute performance during externally defined crisis periods. The goal is to verify that the regime overlay behaves as intended: reducing exposure during stress should reduce drawdown contribution during those periods. If the overlay reduces risk during calm periods but fails to de-risk in stress, it is misaligned.

A useful way to summarize drawdown behavior is to attribute drawdowns to times when the model was confident versus uncertain. If drawdowns occur primarily when posterior entropy is high, that may indicate that uncertainty itself should trigger more conservative actions, or that the model is encountering environments not represented in training.

14.11.3 Ablations: with/without regime overlay; filtered vs smoothed

Ablations are the most honest tool in regime evaluation. They answer: what component is actually responsible for the claimed improvement? In regime systems, ablations are particularly important because it is easy to introduce hidden leakage or to attribute improvements to regimes when they are actually due to volatility targeting, smoothing, or preprocessing.

At minimum, you should run the following ablations.

Ablation 1: baseline strategy with no regime logic. This is the control. If the baseline is not strong and well-specified, you cannot interpret regime improvements. The baseline should be exactly the same strategy signals and execution assumptions, just without any regime-based scaling or switching.

Ablation 2: with regime overlay using filtered beliefs (the only tradable version). This is the main treatment condition. It uses $\gamma_t(k) = p(s_t = k \mid x_{1:t})$ and respects refit boundaries. This is what you can actually deploy.

Ablation 3: with regime overlay using smoothed beliefs (diagnostic upper bound). Compute the same overlay or switching policy but using $\tilde{\gamma}_t(k) = p(s_t = k \mid x_{1:T})$. This is not tradable. Its purpose is to quantify how much hindsight helps. If the smoothed version is dramatically better than the filtered version, that indicates that most of the apparent benefit comes from hindsight and that the live benefit may be modest. Reporting this gap is a sign of discipline, not weakness.

Ablation 4: regimes versus plain volatility targeting. Many regime overlays are, in effect, volatility targeting. Therefore, include a baseline that uses a continuous volatility estimate (Chapter 10 style) without discrete regimes. If the regime overlay does not outperform a simpler volatility targeting rule, it may not be worth the added complexity. Conversely, if regimes outperform volatility targeting, you have evidence that the regime model is capturing more than just scale.

Ablation 5: feature leak check. This is not a trading ablation but a pipeline ablation: recompute features using only prefix data and verify identity with full-sample features at each time t . Then rerun the model. If results change when you enforce causality-safe feature computation, your earlier results were contaminated.

A disciplined report presents these ablations side by side. The point is not to produce the best curve; the point is to show that improvements survive the removal of hindsight and survive comparison to simpler alternatives.

14.11.4 Failure analysis: regime churn, delayed detection, false switches

Even if an evaluation shows improvement on average, a regime model is only credible if it comes with a failure analysis. Markets punish models not for their average behavior but for their

behavior in bad states. Therefore, you should explicitly diagnose failure modes and quantify their impact. Four failure modes are especially common.

Failure mode 1: regime churn (excess switching). Churn occurs when the filtered posterior changes rapidly between states or when the MAP state flips frequently. Churn is costly because it induces turnover in any switching or scaling policy. It is also a sign that either K is too large, emissions are not well separated, or the transition matrix is not persistent enough. Churn should be quantified: number of switches per year, average duration in each state, and distribution of posterior entropy. If churn is high, you may need stronger persistence priors, fewer states, or a decision rule with inertia (e.g., hysteresis).

Failure mode 2: delayed detection (regime lag). Filtering is necessarily lagging because it uses past data. The question is whether the lag is acceptable relative to the decision objective. Delayed detection is especially important for stress regimes: if the model increases stress probability only after the major drawdown has occurred, it may still help reduce prolonged stress exposure but it will not prevent the initial shock. Evaluate detection lag by comparing stress probability to an external stress proxy (volatility spike, drawdown onset) and measuring days-to-threshold. If lag is too large, the remedy may be feature redesign (use more responsive volatility proxies), a higher-frequency model, or acceptance that the model is a risk overlay rather than a crash predictor.

Failure mode 3: false switches (false positives). A regime model that frequently signals stress when the market remains calm will cause unnecessary de-risking and lost opportunity. False switches are often caused by transient volatility bursts, data errors, or features that are too sensitive. Quantify false positives using an external proxy or by measuring the opportunity cost: how much return is lost during periods where the model de-risked but no adverse outcomes followed? The remedy can include requiring persistence before switching (hysteresis), using smoother posterior-to-action mappings, or regularizing emissions so that single-day spikes do not dominate.

Failure mode 4: structural breaks and state meaning drift. Over time, regime meanings can drift: the “high-vol” state in one era may look different in another. This is not necessarily wrong, but it complicates decision rules calibrated on past regime behavior. Diagnose drift by tracking regime anchors across refits: estimated σ_k , tail risk measures, or other state summaries. If anchors drift significantly, you may need shorter training windows, stronger constraints, or a redefinition of features. Drift can also be a signal to increase model uncertainty measures in decisions, because the model is operating out of its training distribution.

A final and often underappreciated failure source is implementation leakage. If your regime-enhanced backtest is dramatically better than baseline and the improvement seems to come from perfect timing, assume leakage until proven otherwise. The most likely sources are smoothing (using $p(s_t | x_{1:T})$), full-sample normalization, or misaligned signal-to-trade timing. A disciplined failure analysis therefore includes explicit leakage diagnostics: filtered versus smoothed comparison, prefix-feature recomputation checks, and timing assertions.

To summarize, evidence that a regime model helps requires more than a nice regime plot. It requires a walk-forward protocol with refits, filtered beliefs, and correct timing; a metrics suite that includes predictive scoring and trading outcomes with costs; ablations that separate real improvements from hindsight and from simpler baselines; and failure analysis that quantifies churn, lag, false switches, and drift. When these elements are present, a regime model claim becomes scientifically meaningful and operationally trustworthy, which is exactly the standard we need before we allow regimes to influence real capital.

14.12 Implementation Notes (Within Chapter 14 Scope)

14.12.1 Numerical issues: underflow, scaling, and log-sum-exp

Hidden Markov Models are conceptually clean and operationally treacherous. The treachery is not theoretical; it is numerical. If you implement the forward-backward recursions in naive probability space, the algorithm will quietly collapse into zeros long before you finish a real dataset. If you implement Gaussian emissions without guarding covariance stability, you can produce negative variances, singular matrices, or absurd likelihood spikes. If you normalize features inconsistently, you can create pseudo-regimes that are nothing but scaling artifacts. This subsection gathers the numerical issues that matter most in practice, with the goal of making the HMM computations stable enough to support reproducible trading research.

The primary numerical failure is *underflow* in the forward recursion. Recall the forward variables

$$\alpha_t(k) = p(x_{1:t}, s_t = k).$$

These are joint probabilities. As t grows, $p(x_{1:t}, s_t = k)$ becomes exponentially small because it is a product of many probabilities. Even if each term is moderate, the product across thousands of time points becomes smaller than the smallest representable positive floating-point number, and the computer stores it as zero. Once any $\alpha_t(k)$ becomes zero, it can remain zero and contaminate subsequent steps; once all $\alpha_t(k)$ become zero, your posterior is undefined.

There are two standard remedies: *scaling* and *log-domain computation*. In practice, you should be comfortable with both. Scaling is often simpler and is well suited to EM because it yields an easy log-likelihood computation. Log-domain computation is often more robust when emission likelihoods become extreme.

Scaling approach. In scaling, you compute the unnormalized forward update

$$\tilde{\alpha}_t(k) = f_k(x_t) \sum_{i=1}^K \alpha_{t-1}(i) A_{ik},$$

then compute a scaling constant

$$c_t = \sum_{k=1}^K \tilde{\alpha}_t(k),$$

and define the scaled forward variable

$$\alpha_t(k) = \frac{\tilde{\alpha}_t(k)}{c_t}.$$

After this scaling, $\sum_k \alpha_t(k) = 1$, so the scaled α_t is essentially the filtered posterior. The numerical benefit is immediate: the values remain in a representable range.

Scaling also provides a stable way to compute the log-likelihood of the observation sequence:

$$\log p(x_{1:T}) = \sum_{t=1}^T \log c_t.$$

This quantity is the backbone of EM convergence monitoring. In a governance-native workflow, you record the sequence $\{\log c_t\}$ as an artifact because it can reveal surprising observations: unusually small c_t implies the model assigns low probability to x_t under all states, which can correspond to data errors, structural breaks, or out-of-model events.

Scaling does not remove all numerical issues. If all $\tilde{\alpha}_t(k)$ become extremely small before normalization, c_t can underflow as well. This usually happens when emission likelihoods are extremely small due to scaling errors or ill-conditioned covariance matrices. Therefore, scaling must be paired with sensible feature normalization and with emission stability controls.

Log-domain approach and log-sum-exp. The log-domain approach avoids underflow by working with log probabilities. Define

$$\ell_t(k) = \log \alpha_t(k).$$

Then the forward recursion becomes

$$\ell_t(k) = \log f_k(x_t) + \log \left(\sum_{i=1}^K \exp(\ell_{t-1}(i) + \log A_{ik}) \right).$$

The numerical challenge shifts to computing $\log \sum_i \exp(z_i)$ stably. The standard technique is log-sum-exp:

$$\log \left(\sum_i e^{z_i} \right) = m + \log \left(\sum_i e^{z_i - m} \right), \quad m = \max_i z_i.$$

Subtracting m ensures that the exponentials $e^{z_i - m}$ are not too large and not too small. In any implementation that uses log-domain calculations, log-sum-exp is non-negotiable.

The backward recursion also has a log-domain form, but in many applications you can compute β_t with scaling consistent with the forward pass. The practical lesson is not that one method is universally superior; it is that you must pick a method and implement it consistently, with explicit numerical checks.

Emission stability: Gaussian likelihoods and covariance issues. Many trading regime models use Gaussian emissions because they are interpretable and computationally simple. But Gaussian likelihoods involve determinants and inverses of covariance matrices. In multivariate cases, covariance estimates can become nearly singular, producing unstable inverses. Even in

univariate cases, estimated variances can become too small if a state has low occupancy, creating enormous likelihood spikes that dominate EM updates. These are not rare edge cases; they are typical failure modes if you do not regularize.

Within Chapter 14 scope, the stabilizing tools are:

- variance floors: enforce $\sigma_k^2 \geq \sigma_{\min}^2$ for all k ,
- covariance shrinkage: shrink Σ_k toward a diagonal or identity target,
- occupancy floors: if $N_k^{\text{eff}} = \sum_t \gamma_t(k)$ is too small, do not update that state aggressively (or merge it, or restart).

Implementation-wise, you must also compute Gaussian log-likelihoods stably. In multivariate settings, the log density uses $\log \det \Sigma_k$ and $(x - \mu_k)^\top \Sigma_k^{-1} (x - \mu_k)$. Computing these via Cholesky decomposition is more stable than using explicit inverses. Even if you are writing minimal NumPy code, the mental model should be: avoid explicit matrix inversion; use stable decompositions.

Numerical checks as hard gates. A professional implementation includes checks that abort or flag runs when numerical pathologies occur. Examples include:

- any NaN or Inf in log-likelihood increments,
- negative variances or non-positive definite covariances,
- transition rows not summing to one within tolerance,
- posterior sums deviating from one beyond tolerance.

These checks should be treated as causality gates in the numerical domain: do not proceed with a run that is numerically broken, because downstream results will be meaningless.

14.12.2 Deterministic runs: seeds, initialization registries, and reproducible EM

Chapter 12 emphasized the research discipline required to prevent selection bias and irreproducible claims in supervised learning. Regime learning has the same requirement, amplified by the fact that EM is sensitive to initialization and can produce multiple local maxima. If you cannot reproduce a regime model run exactly, you cannot trust any story or trading improvement derived from it.

Determinism begins with explicit seeds. If you use any randomized step—even something as simple as selecting initial cluster centers or random perturbations of parameters—you must set and log a random seed. In a NumPy-only environment, you explicitly create a random number generator and use it consistently, rather than relying on implicit global state.

But deterministic seeds are not enough. You also need an *initialization registry*: a structured record of how initial parameters were generated. Two runs can share the same seed and still differ if you change the initialization procedure or if data ordering differs. Therefore, the registry should include:

- the initialization method (quantile split, k-means-like clustering, random assignment, etc.),
- any hyperparameters (e.g., persistence level a for diagonal-dominant A),
- the exact seed used for each restart,
- the mapping used for state ordering (e.g., order by volatility).

In regime models, label switching is a real nuisance: the same fitted model can appear different because state labels are permuted. A deterministic label-ordering rule (e.g., sort states by estimated volatility) is therefore part of reproducibility. It ensures that “state 1” means the same thing run-to-run, at least within the model family.

Reproducible EM also requires deterministic stopping rules. If you stop EM when improvement is below a tolerance, small floating-point differences can change the iteration count. This is usually not a problem if your run is deterministic, but it can become one if you compare runs across hardware or software versions. A robust governance practice is to log the full convergence trace (log-likelihood by iteration) and the stop reason (max iterations or tolerance). If a run is later questioned, you can show the trace.

Finally, reproducibility includes restart policies. If you run multiple restarts and choose the best likelihood solution, you must:

1. record all restart seeds and outcomes,
2. define a selection rule in advance (e.g., choose highest held-out predictive likelihood, or highest training likelihood subject to stability constraints),
3. store the winning parameters and the runner-up(s).

This prevents a subtle form of p-hacking: rerunning EM until the regimes “look right.” Without a registry and a fixed selection rule, EM becomes an aesthetic tool rather than a scientific one.

14.12.3 Complexity and performance: $O(K^2T)$ and practical constraints

HMM inference is efficient relative to brute-force enumeration, but it is not free. The forward update for each time step involves a matrix-vector product: for each target state k , you sum over all previous states i . That is $O(K^2)$ per time step, so filtering is $O(K^2T)$. The backward pass is also $O(K^2T)$. EM adds iterations: if you run M EM iterations, learning becomes $O(MK^2T)$ plus the cost of evaluating emission likelihoods.

This complexity has two practical implications.

First, K is not only a modeling choice; it is a computational choice. Doubling K roughly quadruples the cost of inference and learning. In daily data with T in the thousands, this is manageable for K up to perhaps 5–10 in a research setting. In intraday data with T in the hundreds of thousands, it becomes expensive quickly. Therefore, the default regime modeling practice in many trading systems is small K with strong constraints, because that yields interpretable and computationally stable models.

Second, emission likelihood evaluation can dominate compute when observations are multivariate and emissions are complex (e.g., Student- t with full covariance). Even if the forward recursion is $O(K^2T)$, computing $f_k(x_t)$ for each k and t may be expensive. A practical engineering tactic is to precompute emission log-likelihoods in a matrix of shape $T \times K$ (if memory permits) so that forward-backward passes reuse them. This can accelerate EM significantly. But it also introduces a governance point: precomputed likelihoods must correspond to the correct parameters, which change each EM iteration. Therefore, you precompute likelihoods per iteration, not once globally.

There are also numerical performance constraints in the sense of stability. Scaling and log-sum-exp cost additional operations. But the cost is worth it: an unstable fast algorithm is slower than a stable one because you will waste time debugging nonsense outputs.

Within Chapter 14 scope, the most important performance constraint is not speed; it is avoiding leakage and ensuring reproducibility. Still, it is helpful to state a realistic computational posture:

- keep K small unless you have strong evidence that larger K is stable and useful,
- prefer simpler emissions (e.g., univariate or diagonal covariance) when data are limited,
- monitor EM iteration count and stop early if improvement is negligible,
- use deterministic restarts sparingly and log them.

These points align with the book's general principle: complexity is a budget, and markets charge interest on borrowed complexity.

14.12.4 Engineering boundaries: what belongs to data pipelines (Ch. 05) vs HMM logic (Ch. 14)

A recurring cause of failure in trading research is boundary confusion: mixing data engineering with model logic until neither is auditable. Regime models are especially vulnerable because they sit at the interface between time-series preprocessing and sequential inference. Within the book's architecture, Chapter 05 is responsible for data pipelines and feature engineering. Chapter 14 is responsible for regime modeling logic: specifying the HMM, performing inference, learning parameters, and mapping beliefs to decisions. Maintaining this boundary is not merely pedagogical; it is essential for governance and reproducibility.

We can state the boundary explicitly.

What belongs to Chapter 05 (data pipelines and features). The following components should be implemented and tested in the pipeline layer:

- data ingestion and cleaning: timestamps, missing data handling, corporate actions adjustments (if applicable),
- resampling and alignment: daily bars, intraday bars, cross-asset timestamp conventions,
- feature computation: rolling means/volatilities, drawdowns, spreads, volume proxies,

- causal validation of features: prefix-vs-full-sample checks, no look-ahead assertions,
- scaling and transforms: rolling standardization, winsorization rules, volatility normalization,
- feature lineage artifacts: window lengths, decay parameters, update schedules.

The pipeline layer outputs a time-indexed feature array $x_{1:T}$ along with metadata proving causality and describing how each feature was constructed.

What belongs to Chapter 14 (HMM logic). The regime modeling layer should assume $x_{1:T}$ is already causality-safe and focus on:

- HMM specification: K , emission family, constraints/regularization, priors for A ,
- inference: filtering (forward algorithm) for signals; smoothing (forward–backward) for diagnostics and EM,
- learning: EM (Baum–Welch) with deterministic initialization and logged convergence,
- stability tools: scaling, log-sum-exp, variance floors, occupancy checks,
- decision mapping: turning filtered beliefs into overlays and switching weights without hard labels,
- evaluation artifacts specific to regime logic: posterior entropy, switching rates, implied durations.

The HMM layer should not silently recompute features, re-standardize with global statistics, or change data alignment. If it does, you have hidden leakage risk and you have destroyed the audit boundary.

The interface contract. To keep this boundary enforceable, define an interface contract between layers. The pipeline provides:

- $x_{1:T}$ as a numeric array (or arrays) with explicit ordering,
- a timestamp index or equivalent ordering proof,
- a feature manifest: names, definitions, windows/decays, scaling rules,
- a causality report: tests passed, alignment checks passed.

The HMM layer consumes those and produces:

- fitted parameters at each refit time,
- filtered posterior time series γ_t ,
- (optional) smoothed posterior time series for diagnostics,
- decision outputs (weights, overlays) with time stamps,

- evaluation diagnostics (entropy, churn, likelihood traces).

Everything crossing the boundary is recorded. This is how you prevent the most insidious errors: the ones where the model layer quietly “fixes” something by peeking at the future or by applying a full-sample transform.

A final point: respecting boundaries does not mean isolating components so completely that they cannot communicate. It means making communication explicit and testable. If the HMM needs standardized features, the standardization rule belongs in the pipeline manifest. If the pipeline changes, you rerun the regime layer, and your artifacts reflect the change. This is governance-native engineering: not rigid separation, but auditable separation.

In sum, a Chapter 14 implementation that is numerically stable, deterministic, computationally realistic, and boundary-disciplined is not merely a clean codebase. It is a research instrument. It produces regime beliefs you can trust because you can reproduce them, you can explain them, you can stress them, and you can prove they were computed without hindsight. That is the minimum standard for letting a latent-state model influence trading decisions.

14.13 Governance-Native Checklist for Regime Detection Pipelines

Regime detection is the kind of modeling that invites confident narratives and produces beautiful plots. It is also the kind of modeling that can fail silently, because the difference between a causal regime belief and a hindsight regime label is a single line of code, and because preprocessing choices can leak the future long before you ever run the HMM. For that reason, regime detection is not merely a statistical exercise. It is a governance exercise. The goal of a governance-native checklist is to make regime claims auditable: an independent reader should be able to reconstruct what data were used, how features were computed, which model was fit, how inference was done, and whether any form of smoothing or full-sample preprocessing contaminated signals.

This checklist is written with the discipline of earlier chapters in mind. Chapter 05 provided the pipeline foundations; Chapters 06 and 12 provided evaluation discipline; Chapter 14 adds latent-state inference with an especially sharp causality boundary: *filtered* beliefs are tradable, *smoothed* beliefs are not. Governance, here, is the set of controls that ensures this boundary is never violated, intentionally or accidentally.

14.13.1 Determinism: seeds, configs, and run manifests

Determinism is the first gate. If you cannot rerun the same experiment and obtain the same regime posteriors, you do not have a regime detector; you have a regime generator. Determinism is not merely “set a seed.” It is the combination of seeds, explicit configurations, deterministic initialization policies, and recorded software environment choices.

A governance-native regime pipeline starts each run by generating a *run manifest*. This manifest is a structured record (conceptually a JSON-like object, though the storage format is an engineering detail) containing:

- **Run identity.** A unique run ID (hash of the config, timestamp, and code version) so that outputs can be traced back to inputs.

- **Randomness controls.** Seeds for every source of randomness used in initialization, restarts, and any stochastic components. In a NumPy-only implementation, this means instantiating a local RNG and never relying on implicit global state.
- **Config snapshot.** All parameters that affect outcomes: data source IDs, feature windows, scaling rules, model spec, EM tolerances, refit cadence, and decision mapping rules.
- **Code and environment fingerprint.** Version identifiers for the code base and key libraries, plus (at minimum) Python version and NumPy version. Regime results can change subtly across numerical backends; recording versions is not paranoia, it is reproducibility.

The key requirement is that the manifest is written *before* any computation, and that all outputs carry the run ID. This makes it impossible to “forget” what produced a result.

Determinism also requires deterministic data ordering. Time series must be strictly ordered, without duplicate timestamps, and with explicit tie-breaking rules if multiple observations share a timestamp. You do not want two runs to differ because of a different ordering of same-time events. Therefore, the manifest should include an ordering proof: the first and last timestamps, the number of observations, and a monotonicity assertion.

Initialization is where determinism often breaks down. If you use multiple EM restarts, the restart policy must be deterministic: a fixed list of seeds, a fixed number of restarts, and a fixed selection rule. If you choose “the best looking” regime segmentation, you have left science. The selection rule must be pre-specified, such as “choose the run with highest training log-likelihood subject to minimum state occupancy” or “choose highest predictive likelihood on a validation prefix.” The rule, and all candidate outcomes, belong in the manifest.

Finally, determinism applies to evaluation. Walk-forward refit times must be generated deterministically from a calendar rule (e.g., first trading day of each month) and recorded. A common failure is to refit “whenever you feel like it” in research. That might be fine for exploration, but it must be fixed before you claim evidence.

14.13.2 Parameter registry: K , emission family, constraints, refit cadence

A regime model is defined as much by its constraints and operating schedule as by its mathematical form. Therefore, governance requires a *parameter registry*: a canonical list of all model-defining choices with explicit values. The registry is not only for hyperparameters; it is for everything that influences inference and decisions.

The registry begins with the structural choices:

- **Number of regimes K .** Candidate values considered (if any), selection method used, and the final chosen K .
- **Emission family.** Gaussian, Student- t , or another specified family; univariate vs multivariate; diagonal vs full covariance; any parameter tying (shared mean, scaled covariance, etc.).

- **State ordering rule.** If labels are sorted by volatility or another anchor, record the rule so that “state 1” is comparable across refits.

Then come the constraints and regularization choices, which are often the difference between stable regimes and nonsense:

- **Transition regularization.** Dirichlet pseudocounts or other persistence priors; whether diagonal dominance is encouraged; any minimum transition probabilities enforced.
- **Emission regularization.** Variance floors, covariance shrinkage targets and shrinkage intensity, degrees-of-freedom constraints for Student- t , and any bounds on means or covariances.
- **Occupancy safeguards.** Minimum effective sample size per state; policies for collapsed states (restart, merge, freeze updates).

The registry must also include EM and inference controls:

- **EM settings.** Maximum iterations, tolerance definition, stop criteria, restart count, and selection rule.
- **Numerical method.** Scaling versus log-domain computation; log-sum-exp implementation details; tolerances for normalization checks.

Finally, the registry must include the refit cadence and training window rule:

- **Refit cadence.** Monthly, quarterly, or another rule; exact calendar definition; whether refits occur at open/close.
- **Training window.** Expanding versus rolling; window length if rolling; warm-up rules.
- **Update schedule for feature scaling.** If features are standardized or winsorized using rolling estimates, record whether the scaling parameters are updated daily, weekly, monthly, etc., and how those updates align with refits.

Why does refit cadence belong in the parameter registry? Because it changes the meaning of the model. An HMM fit once on the full sample is not the same object as an HMM refit monthly. In trading, the latter is the only meaningful object.

The registry is the basis for comparability across runs. Without it, two regime results cannot be meaningfully compared because you do not know whether differences came from K , emissions, scaling, or refit policy.

14.13.3 Causality gates: filtered-only signal invariants

The sharpest governance requirement in this chapter is the filtered-only invariant. The tradable regime belief is

$$\gamma_t(k) = p(s_t = k \mid x_{1:t}),$$

or its one-step-ahead projected version for $t + 1$. Anything that uses $x_{t+1:T}$ to infer s_t is smoothing:

$$p(s_t \mid x_{1:T}),$$

and that is not a signal. The causality gate is therefore not a philosophical statement; it is a testable invariant embedded as assertions in code and as checks in the audit.

We can state the invariants explicitly:

Invariant 1: no future observations in signal computation. For every time t , the signal-relevant belief must be computed using only $x_{1:t}$ and parameters fit using only data up to the refit cutoff. If you can change observations after t and the computed belief at t changes, you have violated this invariant.

Invariant 2: smoothing is allowed only in research diagnostics and EM internals. Smoothing may appear in two places: (i) as part of the EM algorithm (E-step) when fitting on a fixed training window, and (ii) as a retrospective diagnostic plot. In both cases, smoothing outputs must be stored separately and must never enter the decision mapping or backtest action path. This separation must be enforced by design: different variable names, different artifact directories, and ideally explicit “forbidden use” assertions.

Invariant 3: refit boundaries are hard cutoffs. At refit time τ_m , parameters $\hat{\Theta}_{\tau_m}$ may depend only on data up to τ_m . For t in the next interval, the filtering must use $\hat{\Theta}_{\tau_m}$ (until the next refit). If parameters drift continuously using future data, you have broken the walk-forward simulation.

Invariant 4: preprocessing is causality-safe. The regime model consumes features. If features are standardized using full-sample statistics or winsorized using full-sample quantiles, then even filtered inference is contaminated. Therefore, causality gates must extend to preprocessing: features at time t must be identical whether computed on the prefix $1 : t$ or the full sample $1 : T$ (under the same pipeline settings). This check belongs in the feature layer, but the regime layer must refuse to run unless it is passed.

The governance lesson is that causality is not a single gate. It is a chain of gates: data ordering, feature causality, refit boundaries, filtered-only inference, and decision timing. The chain fails at its weakest link.

14.13.4 Artifact set: regime posteriors, transition matrices, convergence traces

A regime run is auditable only if it produces artifacts that reflect both what the model believed and how it reached that belief. At minimum, the artifact set must capture inputs, parameters, inference outputs, and diagnostics.

The table included in the chapter already lists minimum content. Here we expand what each artifact should include and why it matters.

Data fingerprint. This artifact identifies the raw inputs. At minimum:

- instrument identifiers (tickers, contracts, or internal IDs),
- frequency (daily close, intraday bar size),
- time span (start/end timestamps),
- missingness summary (counts of missing points, gaps),
- corporate action or adjustment flags (if applicable).

The purpose is to prevent a common failure: two runs cannot be compared because the dataset changed silently (different vendor version, different adjustment method, different calendar). The fingerprint makes the data selection explicit.

Feature manifest. This artifact defines x_t precisely:

- feature names and exact formulas,
- rolling windows and decays,
- scaling rules (rolling standardization, winsorization thresholds),
- update schedules for scaling parameters,
- alignment convention (what timestamp each feature is associated with).

The feature manifest is central because most leakage occurs here. If you cannot reproduce features, you cannot reproduce regimes.

Model spec and parameter registry. This includes:

- K , emission family, and constraints,
- initialization method, seeds, and restart policy,
- numerical method (scaling/log-domain), tolerances, variance floors,
- refit cadence and training window rule,
- state ordering rule for interpretability.

The model spec is what turns a regime plot into a defined model.

EM trace. The EM trace should contain:

- number of iterations executed,
- log-likelihood (or per-observation log-likelihood) at each iteration,
- stop criteria and stop reason,

- parameter snapshots at the end (and optionally at checkpoints),
- warnings triggered (state collapse, variance floor hits, etc.).

This artifact answers: did the model converge, and to what kind of solution? It also helps detect overfitting: if the log-likelihood continues improving while state occupancy collapses, you have learned a degenerate model.

Transition matrices and emission parameters. For each refit time, store:

- $\hat{A}$ and $\hat{\pi}$,
- emission parameters for each state (means, variances/covariances, degrees of freedom if relevant),
- implied durations $1/(1 - \hat{A}_{ii})$ for interpretability,
- state occupancy summary N_k^{eff} .

In trading, these artifacts are essential because they allow you to detect drift: do transition probabilities change materially across refits? Do state volatilities drift? If so, your decision mapping may need to adapt.

Regime posteriors. This is the main output:

- filtered posteriors $\gamma_t(k)$ for all t in evaluation windows,
- one-step-ahead state predictions $w_{t+1}(k)$ if used in decisions,
- posterior entropy time series as a measure of uncertainty,
- MAP state sequence (optional) derived from filtered posteriors, labeled clearly as derived, not fundamental.

If you also store smoothed posteriors, they must be explicitly labeled “diagnostic only” and kept separate.

Decision and performance traces (if regimes drive trading). Within Chapter 14 scope, the regime pipeline may output decision weights (risk multipliers, strategy weights) produced from γ_t . Store:

- exposure multipliers over time,
- switching weights over time,
- any thresholds triggered (stress probability gates),
- turnover induced by regime logic (incremental turnover relative to baseline).

These artifacts connect beliefs to actions, which is the core of explainability.

The philosophy is simple: every regime run should leave a trail that tells a complete story from inputs to decisions. Without that trail, you can neither reproduce results nor diagnose failures.

14.13.5 Audit plots: posterior timelines, switch frequency, drawdown by regime

Governance is not only text logs; it is also visual evidence that the model behaves plausibly. Audit plots are not “marketing charts.” They are diagnostics that reveal whether the model is stable, causal, and aligned with intended use. A minimal set of audit plots should be produced for every run, not only for successful ones.

Posterior timelines. Plot $\gamma_t(k)$ over time for each state k , ideally stacked or in separate panels. The purpose is to see:

- whether regimes persist (posterior stays high for stretches),
- whether posterior mass is diffuse (high uncertainty),
- whether transitions coincide with known volatility or drawdown episodes (as a plausibility check),
- whether early-sample behavior is dominated by priors (common in short samples).

If the posterior is chaotic day to day, you likely have too many states, too little persistence, or features that are too noisy.

Switch frequency and duration diagnostics. Plot the number of MAP state switches in rolling windows and the distribution of inferred regime durations. Also plot posterior entropy H_t as an uncertainty measure. These plots diagnose churn: high switch counts and high entropy indicate that the regime model is not providing a stable environment representation. In trading, churn translates into turnover and cost.

Transition matrix heatmaps. For each refit time, visualize A . A diagonally dominant A indicates persistence; a matrix with strong off-diagonal mass indicates frequent switching. Comparing A across refits reveals drift: if persistence collapses in certain periods, the model is reacting to noise or structural change. This is a crucial audit because it turns an abstract parameter matrix into an interpretable object.

Drawdown by regime (and by stress probability). Regimes are often justified as drawdown controls. Therefore, you should attribute drawdown and performance to regimes. A governance-friendly plot is: cumulative P&L of the baseline strategy and the regime-enhanced strategy, annotated with stress probability or regime occupancy. Another plot is the distribution of daily returns conditional on high stress probability versus low stress probability. These plots answer: does the overlay reduce exposure when bad things happen, or does it merely change behavior without benefit?

Filtered vs smoothed comparison (leakage diagnostic). Produce a plot comparing filtered and smoothed regime probabilities. The purpose is not to trade on smoothing; it is to reveal the gap between hindsight and live inference. If your “signal” looks identical to the smoothed posterior, you are likely leaking. This plot is one of the most effective leakage detectors because it makes the temptation visible.

The audit plot suite should be standardized and produced automatically. The worst governance posture is to generate plots only after you see performance, because then plots become part of selection bias.

14.13.6 Minimal test suite: ordering, no-look-ahead, and refit boundaries

A governance-native pipeline treats certain failures as non-negotiable: if tests fail, the run fails. These tests are the analog of unit tests in software engineering, but they target time causality, ordering, and boundary integrity.

A minimal test suite for regime detection includes:

Test 1: strict time ordering. Assert that timestamps are strictly increasing (or non-decreasing with an explicit tie-break rule) and that the feature array $x_{1:T}$ corresponds exactly to that order. If ordering is wrong, everything else is meaningless.

Test 2: feature causality (prefix invariance). For a selection of times t (or for all times if feasible), recompute features using only data up to t and compare to the feature values at t computed using the full sample. They must match within tolerance. This test detects full-sample normalization, centered windows, and other leaks that are otherwise invisible.

Test 3: filtered-only invariance (future perturbation test). For a selection of times t , perturb data after t (shuffle, replace with noise) and rerun inference. The filtered posterior at time t must not change. If it changes, you are using smoothing or you have a hidden future dependence. This test is powerful because it targets the regime layer directly.

Test 4: refit boundary integrity. For each refit time τ_m , assert that parameters used in the interval $\tau_m \leq t < \tau_{m+1}$ were estimated using only data up to τ_m . This can be implemented by storing parameter timestamps and checking that no parameter update occurs inside the interval. The test prevents accidental “online EM” or other leakage where parameters drift using future data in a backtest.

Test 5: signal-to-trade timing consistency. Assert that actions at time $t + 1$ depend only on beliefs computed at time t (or earlier), consistent with the stated execution convention. Misalignment here is a frequent source of hidden look-ahead: using same-day close information to trade at the same close without realistic assumptions. The test should enforce your chosen convention.

Test 6: numerical sanity. Assert that all posteriors sum to 1 within tolerance, that no NaNs or Infs appear, that transition rows sum to 1, and that variances/covariances remain valid. Numerical failures can masquerade as regime changes; you must detect them early.

These tests are minimal because they focus on failure modes that invalidate conclusions. There are many additional tests one could add (stability across seeds, sensitivity to window lengths), but the minimal suite ensures that what you are evaluating is at least causal and well-defined.

The table below summarizes the minimum artifacts required for an auditable regime run. It encodes the governance philosophy of this chapter: if you cannot produce these artifacts, you should not trust the regime output.

Artifact	Minimum content
Data fingerprint	instruments, frequency, span, missingness summary
Feature manifest	exact feature definitions, windows, scaling rules
Model spec	K , emission family, constraints, initialization method
EM trace	iterations, log-likelihood path, stop criteria, final params
Causality proof	filtered-only assertions; refit cutoffs; time indices
Outputs	posterior $p(s_t x_{1:t})$, transition matrix, diagnostics plots

Table 14.1: Minimum artifacts for an auditable HMM regime detection run.

A regime detector that meets this checklist is not guaranteed to be profitable, but it is guaranteed to be honest in the sense that matters: it can be reproduced, it respects time, and it can be audited. That is the minimum standard before regimes are allowed to influence trading decisions.

“`latex`

14.14 Conclusion

14.14.1 What regime detection can reliably contribute

Regime detection is best understood as an instrument for disciplined adaptation. Markets do not hold still: volatility clusters, correlations reconfigure, liquidity conditions tighten and loosen, and the effectiveness of a given signal can wax and wane. A Hidden Markov Model does not eliminate that non-stationarity, and it certainly does not turn a market into a predictable machine. What it can reliably contribute is a structured, auditable way to represent changing conditions and to route that representation into decisions.

The first reliable contribution is *interpretable state summaries of risk environment*. Even a simple two-state model can provide a stable decomposition into “calm” versus “stress” under an explicitly defined observation vector. When implemented with causality-safe features and filtered-only inference, the posterior $p(s_t | x_{1:t})$ becomes a time-indexed measure of environmental risk that can be logged, plotted, and explained. This is valuable in its own right: it converts the vague feeling that “markets feel different” into a quantified belief that can be monitored and compared across time.

The second reliable contribution is *risk overlays that are more robust than alpha claims*. In the trading hierarchy, regime detection is often most useful when it scales exposure, caps

leverage, or adjusts risk targeting as a function of stress probability or regime-conditioned volatility. This is where regimes earn their keep: reducing drawdown contributions during extended high-risk periods, improving the stability of strategy behavior, and producing a system that is less dependent on a single static parameter setting. The mixture view of predictive risk makes the logic explicit: uncertainty about the regime widens the predictive distribution, and a disciplined risk manager should respond by reducing exposure.

The third reliable contribution is *a principled framework for strategy conditioning*. Many strategies have known failure modes that are environment-dependent. Trend systems often suffer in choppy, mean-reverting markets; mean-reversion systems often fail in persistent trends and in volatility expansions. Regime beliefs provide a coherent mechanism to blend or gate strategies without pretending that we can forecast returns precisely. When strategy weights are defined as smooth functions of filtered regime probabilities, regime switching becomes a controlled policy rather than a brittle on/off switch. Importantly, this conditioning is explainable: the system can say not only what it did, but why it did it in terms of inferred environment.

The fourth reliable contribution is *diagnostic clarity about instability*. Even when regimes do not improve a trading system, they can reveal why a single model fails. Posterior entropy spikes, switching bursts, and inconsistent regime anchors across time folds are visible signals that the environment is changing in ways your features or model cannot capture. In that sense, regime detection can function as a monitoring layer: it alerts you when the system is operating outside its learned patterns, which is often when you want tighter risk controls and more conservative assumptions.

Taken together, the reliable value proposition of regime detection is not that it predicts the market. It is that it improves the *governance and adaptability* of a trading system: it creates an interpretable belief state, it enables risk-aware policies, and it produces artifacts that can be audited and stress-tested.

14.14.2 What regime detection cannot guarantee

Because regime models produce crisp-looking segmentations, it is easy to over-interpret them. The first thing regime detection cannot guarantee is that regimes are “real” in the ontological sense. Regimes are a modeling choice: an HMM imposes a discrete latent-state structure on continuous, noisy markets. The model will always produce states, even if the world does not contain clear state boundaries. Therefore, a regime plot is not proof that regimes exist; it is evidence that the model can compress the data into K clusters with Markov persistence. That distinction matters, because it prevents you from treating regimes as an oracle.

Second, regime detection cannot guarantee stable labels over time. Label switching is not only a mathematical nuisance; it is a practical reality: the state that was “high volatility” in one era may not have the same volatility level in another era, and a state that represented “liquidity stress” in one decade may not appear the same after microstructure changes. Even with deterministic ordering rules (e.g., order states by volatility), the economic meaning of the states can drift. This is why refit cadence, training window rules, and stability diagnostics belong at the center of evaluation. A regime model that is stable only in-sample is not stable enough for trading.

Third, regime detection cannot guarantee timely detection of shocks. Filtering is causal and therefore lagging by design. If you want a model that warns you before a sudden crash day, an HMM on daily data is structurally limited. At best, it can detect the onset of sustained stress after the shock begins. That can still be valuable for reducing exposure during the remainder of a crisis, but it is not a crash predictor. Promising otherwise is not a modeling error; it is a category error.

Fourth, regime detection cannot guarantee improved returns. In many applications, the primary benefit is drawdown control or risk stabilization. Even then, improvements depend on the decision mapping and on trading frictions. A regime overlay that reduces exposure in stress can also reduce exposure in recoveries, producing opportunity cost. Strategy switching can reduce one failure mode and introduce another, especially if regimes churn and create turnover. A regime model that improves in-sample performance can fail out-of-sample because it captured era-specific quirks. Therefore, any claim of improved returns must be conditioned on rigorous out-of-sample evaluation with realistic costs.

Fifth, regime detection cannot guarantee immunity from leakage and selection bias. In fact, regime modeling is unusually vulnerable to these failures because smoothing is so tempting and because preprocessing often uses global statistics by accident. A smoothed posterior can make a strategy look clairvoyant. Full-sample standardization can quietly leak future information into features. Choosing K based on how “clean” the regimes look is an aesthetic selection bias. These are not minor issues; they are the most common reasons regime results collapse when moved from research to production.

In short, regime detection cannot guarantee that markets are discrete, that labels are stable, that shocks are predicted, that returns improve, or that research is automatically honest. Those guarantees require discipline, not models.

14.14.3 A minimum standard for claiming regimes improved a trading system

Because regime detection is as much governance as it is modeling, the minimum standard for a credible claim must be governance-native. The standard is not “a good backtest”; it is a set of conditions that make the backtest a valid piece of evidence.

First, the evaluation must be genuinely out-of-sample under a walk-forward protocol. The regime model must be refit on a schedule using only past data, and the beliefs used for trading must be filtered-only: $p(s_t | x_{1:t})$. Any use of smoothing in the signal path disqualifies the result. The refit cadence and training window rule must be fixed and recorded.

Second, the pipeline must pass causality gates end-to-end. Features must be computed causally (no centered windows, no full-sample normalization), signal-to-trade timing must be explicit and respected, and refit boundaries must be hard cutoffs. A credible claim includes proofs or tests: prefix invariance checks for features, future perturbation tests for filtered beliefs, and assertions that prevent accidental smoothing.

Third, the claim must be supported by ablations. At minimum, report (i) baseline strategy without regimes, (ii) strategy with filtered regime overlay or switching, and (iii) the smoothed-regime version as a diagnostic upper bound clearly labeled non-tradable. If filtered improvements disappear while smoothed improvements are large, the system’s apparent benefit is primarily

hindsight. A robust claim survives the filtered constraint and remains favorable relative to simpler alternatives such as continuous volatility targeting.

Fourth, the evidence must include trading-relevant metrics with costs and drawdown behavior, not just headline Sharpe. A regime model is often justified as a drawdown control; therefore, reductions in drawdown depth and duration, improvements in tail outcomes, and stable behavior through stress periods are central. Turnover and a cost model must be included because regime switching can easily create hidden fragility.

Fifth, the run must be reproducible and fully auditable. The minimum artifact set includes the data fingerprint, feature manifest, full model spec and parameter registry, EM convergence trace, transition matrices and emission parameters per refit, filtered posterior time series, and decision traces. Without these artifacts, a regime claim cannot be independently verified, and therefore it is not a scientific claim.

Finally, the claim must come with failure analysis. Regime churn, delayed detection, false switches, and state meaning drift must be measured and discussed. A system that improves average metrics but fails catastrophically under specific conditions is not improved in the only sense that matters for real capital.

These requirements may sound strict, but they are proportionate to the risks. Regime models are powerful narrative generators. The minimum standard exists to ensure we are not mistaking narrative clarity for predictive value.

14.14.4 Transition to Chapter 15: Event-Driven & News/Sentiment

Chapter 14 introduced latent-state structure inferred from market variables and disciplined the difference between what can be used online (filtering) and what belongs only to retrospective analysis (smoothing). Chapter 15 shifts the lens from endogenous dynamics to exogenous information: events, text, and sentiment that can change beliefs about returns and regimes. Conceptually, this is a natural extension. If regimes represent market “weather,” then news and events are among the forces that change that weather, sometimes abruptly.

The core discipline, however, does not change. Event-driven data are even more vulnerable to leakage because timestamps are ambiguous, publication times can differ from ingestion times, and text corpora can be revised after the fact. Therefore, the governance lessons of Chapter 14 become prerequisites for Chapter 15: time-aware data construction, explicit information sets, filtered-only beliefs for live decisions, and evaluation protocols that survive walk-forward constraints. Chapter 15 will build event-driven features and sentiment signals in a way that respects those constraints, and it will show how exogenous information can be integrated into forecasting and regime-aware decision policies without quietly importing the future into the past.

Chapter 15

Event-Driven & News/Sentiment

Abstract

Event-driven trading is the disciplined attempt to translate discrete information arrivals into timed decisions under uncertainty. Unlike purely price-based systems, event-driven pipelines must reason about *what happened*, *when it became knowable*, and *how quickly any edge decays*. This chapter formalizes the core objects: events, clocks, observation delays, and the mapping from raw text or structured disclosures into numerically stable features that remain causal. We treat news and sentiment as a data modality with unique failure modes: leakage via timestamp confusion, survivorship via vendor backfills, non-stationarity of language, and hidden selection bias in which events are recorded. The chapter provides an end-to-end framework for building and evaluating event-driven signals without violating the book’s time-aware constraints: immutable raw logs, feature lineage, and testable “no-look-ahead” invariants. We also connect event features to risk controls and portfolio usage, emphasizing that a sentiment score is not a trading rule until the decision-time logic, execution assumptions, and robustness checks are specified. The goal is not to promise alpha, but to define a minimum standard of evidence and governance for claiming that exogenous information improved a trading system.

15.1 Position in the Book

Chapter 15 sits at a structural turning point in the book: we move from markets as a self-contained dynamical system (where all information is presumed to be encoded in prices and volumes) to markets as a system that is repeatedly *shocked* by exogenous information arrivals. In Chapters 01–14 we learned to treat time as sacred, to build features that respect causality, and to evaluate strategies under the humility that backtests can only ever be conditional evidence. Chapter 15 preserves that discipline but applies it to a data modality whose biggest dangers are not mathematical subtlety, but operational ambiguity: the meaning of timestamps, the provenance of content, the selection bias of what gets recorded, and the temptation to use “clean” hindsight labels (“earnings beat”, “downgrade”, “CEO scandal”) that silently import the future into the past.

This chapter is therefore not “about sentiment” in the casual sense. It is about the *event-driven research contract*: if you claim that text or events improved a trading decision, you must be able to prove (a) the information was available at the decision time, (b) your feature construction is reproducible from immutable raw logs, and (c) your evaluation separates the effect of the event signal from coincident market dynamics. In the arc of the 25 chapters, Chapter 15 is the first chapter where the engineering and governance aspects are not supporting characters; they are the plot.

15.1.1 Prerequisites from earlier chapters

Chapter 15 assumes that the reader already treats algorithmic trading as an end-to-end pipeline rather than a single “model.” From Chapter 01 we inherit the market logic that data is not truth; it is a record, with microstructure frictions and institutional artifacts. From Chapter 02 we inherit the minimal Python/NumPy toolbox and the preference for transparent implementations over opaque convenience layers. This matters immediately for event data because the most dangerous errors are silent: a one-line merge can accidentally align tomorrow’s headline with today’s trade, and the code will still run beautifully. You need to be able to inspect and assert each alignment step.

From Chapters 03 and 04 we carry the risk and time-series foundations: returns definitions, drawdown thinking, and the anatomy of time series as indexed objects. Event-driven research is fundamentally about indexing: which clock are you using, what counts as “known”, and how you align discrete arrivals to continuous price paths. If Chapters 03–04 taught you to fear improper compounding and mistaken scaling, Chapter 15 teaches you to fear mistaken timestamps even more.

Chapter 05 contributed the pipeline mindset: data lineage, feature engineering discipline, and artifact registries. That is the backbone here. News and sentiment are not just numbers; they are derived objects with versions (lexicon versions, tokenization rules, vendor feed revisions). Without the Chapter 05 discipline of deterministic manifests and reproducible transformations, your event research becomes un-auditable storytelling.

Chapters 06–10 provided the first full simulation discipline and the first family of strategies. Even though Chapter 15 is later in the book, it still relies heavily on Chapter 06’s backtesting

mechanics: decision-time simulation, no-leakage invariants, and baseline comparisons. The difference is that here the “signal” is not a rolling average on a price series, but a constructed object from a stream of documents. The backtester must therefore simulate not just trading rules, but *information availability*. Chapters 07–10 add practical context: trend and mean reversion remind us that markets have endogenous structure, and volatility/risk targeting remind us that event effects are not only about mean returns but also about volatility bursts and tail behavior. Finally, Chapters 11–14 established the model-evaluation discipline for richer feature sets and, crucially, introduced regimes (Chapter 14) as a way to explain non-stationary performance. Chapter 15 uses that regime lens rather than discarding it.

15.1.2 What this chapter adds

Chapter 15 adds three core capabilities that did not previously exist in the book’s toolkit.

First, it adds a rigorous *event formalism*. We introduce the idea that events live on their own clock: there is an underlying “true” occurrence time, but the trader only sees a *first-seen* time through a feed, and sometimes also a vendor “published” or “updated” time. The trader’s decision is admissible only with respect to what was knowable at the decision time. This reframes feature engineering: sentiment is not a scalar; it is a function of a document that arrived at a particular time through a particular channel with a particular delay distribution. By making these objects explicit, we can define causal joins and build tests that fail loudly when a timestamp is misused.

Second, Chapter 15 adds *text-to-feature pipelines* that are intentionally simple, transparent, and governance-friendly. In later chapters we will allow more sophisticated learning machinery, but here we emphasize baselines: lexicon-based sentiment, counts and novelty measures, decayed aggregations, and metadata-based features (source, repetition, length, instrument mapping confidence). The point is not that these are the best predictors; the point is that these are the cleanest substrates for verifying causality. If you cannot build and validate a causal lexicon baseline, you have no business trusting a transformer.

Third, Chapter 15 adds a specialized *evaluation vocabulary* for event-driven claims. We introduce event studies as a trading tool rather than an academic ritual: impact curves aligned to decision time, pre/post windows, and the notion of edge half-life. We also introduce placebo tests that are particularly important for text: randomizing timestamps (within realistic constraints), shuffling documents, or using source splits to detect vendor-specific artifacts. This chapter therefore upgrades the research workflow: it becomes harder to fool yourself with a narrative, because you now have dedicated tests for narrative-like data.

15.1.3 What this chapter postpones (and why)

Even though event-driven trading often evokes high-frequency execution, deep learning, and alternative data scale, Chapter 15 deliberately postpones several topics to preserve the book’s staircase.

We postpone *microstructure*, *latency modeling*, and *execution realism* to Chapter 18. In Chapter 15 we will acknowledge that delays and fills matter, but we do not yet build a full limit-order-book simulator or a slippage model that depends on spread and depth. The reason

is pedagogical and methodological: if you conflate information-edge and execution-edge too early, you cannot diagnose which one is responsible for performance. Chapter 15 isolates the information problem: can we define a causal signal at all?

We postpone *large-scale NLP and deep representation learning* to Chapter 13 (and, for event-specific modeling decisions, Chapters 11–13 generally). Even if the reader is eager to deploy BERT-like models, the chapter insists that complex models amplify leakage risk and reduce interpretability, especially when the data source itself is messy. The book’s contract is to earn complexity. Chapter 15 lays the ground truth: clean timestamps, immutable logs, and feature lineage. Then deep learning can be introduced without the entire pipeline becoming a black box.

We postpone *portfolio construction and optimization* to Chapter 16. Event signals often imply cross-asset decisions (sector spillovers, pair trades around earnings, hedges during macro releases), but the translation from signals to weights requires an explicit objective and constraints. If Chapter 15 tried to do optimization, we would blur the boundary between “signal validity” and “portfolio engineering.” Instead, we produce outputs that Chapter 16 can consume cleanly: time-aligned forecasts or scores, uncertainty proxies, and regime-conditional performance summaries.

We also postpone *news/sentiment as a full event-driven trading system with multi-source fusion* to Chapter 20 (multi-strategy systems) and, conceptually, to the governance and deployment chapters (21–23). Chapter 15 is where we define the admissible building blocks and minimum evidence, not where we claim to have “solved” news trading.

15.1.4 How Chapter 15 connects back to regimes (Ch. 14) and forward to portfolio construction (Ch. 16)

The backward connection to Chapter 14 is conceptually tight: regimes explain why a fixed mapping from features to returns can degrade, and events are one of the most plausible mechanisms that *cause* regime transitions or, at minimum, coincide with them. In Chapter 14 we modeled latent states inferred from market variables; in Chapter 15 we add exogenous variables that may shift the latent state probabilities. This does not require us to build a full joint model yet. It simply means that event features should be examined through a regime lens: the same headline sentiment may have different implications in a “risk-on” regime versus a “risk-off” regime, and the same macro release may produce a larger volatility shock when the market is fragile. Thus, Chapter 15 operationalizes a practical insight: regimes are not just a diagnostic after the fact; they are a conditioning set for interpreting event signals.

The forward connection to Chapter 16 is equally structural. Chapter 15 outputs *signals with timestamps and provenance*. Chapter 16 requires *expected returns, risk estimates, and constraints* to produce weights. Event features become forecasts or tilts; event clusters become risk flags; event half-life estimates become holding-period suggestions; and provenance becomes a governance layer for investment committees. When Chapter 16 asks, “what is the objective function and what is the constraint set?”, Chapter 15 supplies the inputs in a form that is both numerically usable and auditable. In other words, Chapter 15 turns narratives into time-indexed, causally admissible numbers, and Chapter 16 turns those numbers into portfolios.

Seen in the full 25-chapter arc, Chapter 15 is where the book stops pretending that trading is

only about clever transforms of prices. It is where we admit that information enters the market through messy channels, and we respond not with mysticism, but with discipline: explicit clocks, immutable logs, causal features, and evaluation tests designed to puncture stories that are too neat. That discipline is what allows the later chapters—optimization, execution, multi-strategy design, and governance—to be built on a foundation that can survive contact with reality.

15.2 Motivation: Why Events and Text Are Different From Prices

Price-based trading invites a comforting illusion: because prices arrive in a clean time series, we can pretend the world is also clean. A bar closes at a well-defined time, the close is recorded, returns can be computed, and most of the research challenge looks like signal extraction under noise. Even when the noise is brutal, the object you are working with is at least disciplined. Events and text break that illusion. They are not simply another column to join onto a price table. They are a different data modality with different physics: discrete arrivals rather than continuous sampling; ambiguous timestamps rather than canonical clocks; meaning that depends on context rather than measurement; and selection bias that is often stronger than any statistical signal. If you do not acknowledge those differences explicitly, you will either (i) build a strategy that is accidentally non-causal, or (ii) build a strategy that is causal but fragile, because the pipeline assumptions you never wrote down are the true source of performance.

This section motivates why the chapter treats event-driven trading as a research discipline in its own right rather than as an accessory to machine learning. The point is not to diminish prices; it is to respect the fact that events and language live upstream of prices and are filtered through institutions, media incentives, and asynchronous dissemination. In prices, the market itself has already aggregated information. In text, you are attempting to front-run (or interpret) that aggregation. That is a fundamentally different game, and it imposes different obligations on your data design, your backtest logic, and your governance artifacts.

15.2.1 The “information arrival” view of markets

A useful mental model for event-driven trading is that markets are not merely stochastic processes; they are *information processing machines*. In this view, price changes are not the primitive object. The primitive object is the arrival of information, and prices are the market’s evolving consensus response to that arrival. The reason this framing matters is that it changes what you call “signal.” In a purely price-based system, the signal is a function of past prices and volumes: momentum, mean reversion, volatility, or some engineered feature derived from those. In an information-arrival system, the signal is a function of *what has been learned* about the world since the last decision time.

Events, announcements, headlines, filings, and even social media posts are candidate carriers of that “what has been learned.” But the word *carrier* is doing work here. A headline is not an event in the philosophical sense; it is a representation of an underlying fact, filtered through a channel. The same underlying fact can generate multiple headlines, multiple updates, corrections, and commentary. The market may react not only to the underlying fact but also to the framing,

credibility, and dissemination speed of the carrier. That is already enough to show that events are not like prices. A price is a market-clearing number at a time index. A headline is an object with content, source, and potentially multiple timestamps: when the fact occurred, when the text was written, when it was first disseminated, and when it was archived in your dataset. In the information-arrival view, your job is not to “add sentiment to a model.” Your job is to reconstruct, as faithfully as possible, the trader’s information set $\mathcal{F}_t$ at each decision time t .

This also clarifies why event-driven trading can never be only about prediction. Even if you could predict the average direction of the reaction to a class of events, the trading problem also asks: *who learns first, who can act first, and what is the market microstructure environment at that moment?* The market reaction is often compressed into minutes or seconds for widely disseminated scheduled news. For less widely covered events, the reaction may unfold over hours or days, but then the risk shifts: you face confounding from other information arrivals, narrative evolution, and changing risk sentiment. Therefore the information-arrival view brings an uncomfortable implication: the edge you seek is rarely a static mapping from text to returns. It is a mapping from text to returns *conditional on dissemination speed, credibility, and the market’s current state*. This is one of the reasons Chapter 15 explicitly connects backward to regimes (Chapter 14) and forward to portfolio construction (Chapter 16): information arrives into a market whose receptiveness depends on state, and any actionable signal must ultimately be translated into a position with risk.

A second implication of the information-arrival view is that *selection bias is intrinsic*. Prices exist for every bar, even if nothing interesting happened. News and events do not. The very presence of an event record implies that something was notable enough to be recorded by someone. That introduces a selection mechanism that can correlate strongly with future volatility and returns, even if the content of the event is irrelevant. For example, “more news” can simply mean “more uncertainty,” which can change risk premia. If your backtest shows that days with more headlines have different returns, you must ask whether you have discovered a content-based signal or merely a proxy for attention, volatility, or market stress. The information-arrival view forces you to treat the *existence* of a record as a feature with its own biases, not as a neutral sampling process.

15.2.2 Edge half-life: why timing dominates

If prices are the market’s response to information, then an event-driven edge is, almost by definition, an edge about *timing*. The phrase “edge half-life” is a way to keep this point operational. It means: after an event becomes knowable, how quickly does any predictable component of the price reaction decay? In many event-driven settings, this decay is faster than the cadence of your research intuition. The market can incorporate obvious information almost immediately, especially for liquid assets and widely followed events. In such cases, the half-life may be measured in seconds or minutes, and the edge is largely an execution problem (which this book postpones to Chapter 18). In other cases, the market reaction can be slower: complex disclosures, gradual repricing of fundamentals, or diffusion of information across less liquid names. Then the half-life may be hours or days, and the edge becomes a research and portfolio problem. But in both cases, half-life is the central quantity: it tells you whether your strategy is

plausible given your decision frequency, your latency, and your transaction costs.

This focus on timing also changes how you interpret backtests. In a price-based strategy, you can often move the signal by one bar and still get similar results, because the signal is designed to be low-frequency and persistent. In event-driven strategies, moving the alignment by one bar can invert the result. If you accidentally treat a headline timestamp as earlier than it truly was, you may show positive performance that vanishes when corrected. Conversely, if you align events too conservatively (e.g., always using the next day's open), you may erase a real but short-lived effect. Therefore timing is not a detail to be tidied up at the end. It is the first design decision.

Half-life thinking also protects you from a common research trap: confusing *reaction magnitude* with *tradable edge*. An event can cause a large move without offering a profitable strategy. If the move is instantaneous relative to your ability to trade, it is not an edge; it is a risk. In contrast, a small but slowly unfolding drift can be tradable if it persists long enough to overcome costs and noise. This is why event studies in this chapter are framed as trading diagnostics: they help estimate not only whether something moves prices, but when it moves prices relative to your feasible execution window. A credible event-driven claim must therefore specify: (i) the decision time (when you place trades), (ii) the event observation time (what you knew), (iii) the holding period (how long you exploit), and (iv) the robustness of this timing relationship across regimes and sources.

The half-life lens also exposes a subtle but important asymmetry: *bad news often travels faster than good news*, not as a moral statement but as a structural one. Negative events can trigger mechanical de-risking, margin calls, and risk limits, which compress reaction times. Positive events may diffuse more slowly, especially if they are complex or if risk appetite is constrained. Whether or not this stylized fact holds in a particular market is an empirical question, but the chapter's point is methodological: you should expect half-lives to depend on the sign, the category, and the market state. A single "sentiment score" cannot capture that reality unless the pipeline and evaluation explicitly condition on these dimensions.

15.2.3 Event-driven strategies as a pipeline problem, not a model problem

It is tempting to think of event-driven trading as "apply NLP, get alpha." This temptation is partly driven by the broader machine-learning culture, where model capacity is often treated as the lever that matters most. In practice, the dominant determinant of success in event-driven research is usually the pipeline: what you ingest, how you timestamp it, how you deduplicate it, how you map it to instruments, how you build features without leakage, and how you simulate decision-time knowledge. If these steps are wrong, the best model in the world will simply learn the structure of your mistakes.

Calling event-driven strategies a pipeline problem means several specific things.

First, the raw object is not a single dataset but a *stream*. The pipeline must handle ordering, late arrivals, updates, and duplicates. A news story often appears multiple times: an initial headline, a longer article, an updated version, a follow-up, a commentary piece, and then a summary. If your dataset stores only one of these, you need to know which one. If it stores all of them, you need to decide whether your strategy reacts to the first mention, the most complete mention, or some aggregation. Each choice implies a different information set.

Second, instrument mapping is itself a research decision. A piece of text may mention a company name, a subsidiary, a product, or an executive. Mapping that mention to a tradable ticker is nontrivial, especially across markets and corporate structures. Errors in mapping can create spurious results: you might attribute a sector-wide story to one ticker and then “discover” an edge that is actually just the market moving. Therefore, pipeline design must include entity resolution rules and confidence scores, and these rules must be logged and versioned. This is a governance issue as much as a technical one.

Third, feature engineering for text is uniquely vulnerable to hindsight contamination. In price-based features, leakage often comes from misalignment or using future data. In text features, leakage also comes from *curation*. Many datasets include fields that are convenient but dangerous: categories assigned after the fact, normalized event types inferred with knowledge of outcomes, or revised content that includes corrections. A pipeline-first mindset treats every field as suspect until you can prove it existed at the decision time. This is why the chapter insists on “as-of” snapshots and immutable raw logs: they are the only credible defense against the dataset silently drifting.

Fourth, evaluation is inseparable from pipeline choices. Consider the simplest pipeline question: do you trade at the close, at the next open, or immediately after the event? Each choice changes not only execution assumptions but also which events are admissible. For example, an article published after the close is admissible for next-open trading but not for same-day close trading. If your backtest logic does not enforce this distinction, it will produce nonsense that looks like skill. Therefore, the pipeline is the model: it defines the information set, the decision rule, and the test.

Finally, the pipeline-first view leads to a different research workflow. Instead of immediately maximizing performance, you begin by constructing *auditable artifacts*: event ledgers, timestamp distributions, deduplication clusters, mapping confidence summaries, and causality test reports. Only after these artifacts pass do you attempt to build trading rules. This workflow is slower, but it is the only one that scales to production credibility. In event-driven work, the fastest way to waste time is to jump to modeling.

15.2.4 Common myths: “sentiment is a factor” and “news is just another feature”

Two myths repeatedly sabotage event-driven research, especially when practitioners import habits from cross-sectional equity factor work or from generic machine-learning pipelines.

Myth 1: “Sentiment is a factor.” In factor language, a factor is typically a persistent characteristic that explains cross-sectional differences in expected returns: value, momentum, quality, size, carry, and so on. These factors are not instantaneous information arrivals; they are exposures that persist and can be held through time. Sentiment is often not like that. Sentiment is frequently an *impulse*: it arrives, it moves beliefs, and then it decays as the market incorporates it. Even when there are longer-lived sentiment effects (such as attention cycles or narrative regimes), they are rarely stable exposures in the same sense as classical factors. Treating sentiment as a factor encourages the wrong behavior: you begin to optimize it as if it

were a slow-moving characteristic, rather than as a timed piece of information whose tradability depends on half-life and dissemination.

Moreover, sentiment is not a single object. “Sentiment” depends on whose sentiment, measured from which text, in which language, with what framing, and for which audience. Market impact depends not only on polarity (positive/negative) but on surprise, credibility, novelty, and the market’s prior expectations. A headline that is mildly negative but entirely anticipated may have no effect; a neutrally worded but surprising disclosure can move the market sharply. Therefore the chapter treats sentiment as a *feature family* that must be defined operationally, with versioned lexicons, normalization rules, and explicit timing. Only then can it be evaluated as an input to a decision system.

Myth 2: “News is just another feature.” This myth is even more dangerous. It suggests that you can take a standard supervised learning pipeline, add some news-derived columns, and proceed as if nothing changed. But news violates the assumptions that make “features” behave nicely. A numeric feature derived from past prices has a clear timestamp, a deterministic construction, and an unambiguous relationship to the market clock. News does not. News has multiple timestamps, revisions, duplicates, and missingness that is itself informative. The process that generates news is not stationary: editorial practices change, coverage expands or contracts, and language evolves. If you treat news as “just another feature,” you will under-specify the pipeline, and the model will happily exploit the under-specification.

In particular, models are extremely good at learning proxy variables for leakage. If your dataset contains any field that

15.3 A Minimal Formalism for Event-Driven Trading

Event-driven trading becomes much easier to reason about once we stop describing it as “using news” and start describing it as a disciplined mapping from *an event stream* to a *trading decision stream*, under explicit constraints about what was knowable when. The purpose of this section is to introduce a minimal formalism that is strong enough to prevent the classic failures (timestamp leakage, hindsight labels, and misaligned execution) while remaining simple enough to be implemented transparently with the book’s toolbox. We will not build a full probabilistic model of news generation or a full microstructure simulator here. Instead, we will define the objects precisely, name the clocks, express event arrivals as random variables with observation delays, and define what it means for a feature to be admissible under a filtration. The payoff is practical: if you can write down the formal object your code is implementing, you can test it, audit it, and defend it.

The guiding idea is that in event-driven trading, the *indexing* is the strategy. When your signal is computed from past prices, most of the complexity is in the statistical relationship. When your signal is computed from events and text, much of the complexity is in the timing relationship. If you get the timing wrong, you will get an edge that never existed. Therefore we begin with indices.

15.3.1 Objects and indices: time, instruments, and event identities

We work with a set of tradable instruments (assets) denoted by $\mathcal{A}$. A single asset is indexed by $a \in \mathcal{A}$. We use t to denote time, but in event-driven trading we must immediately distinguish which time scale and which clock we are referencing. We introduce a *market clock* and an *information clock*. The distinction sounds philosophical until you try to run a backtest; then it becomes operational.

Let $\{t_k\}_{k \geq 0}$ be an increasing sequence of market decision timestamps. For daily strategies this might be close-to-close, for intraday it could be every minute. We will call these *decision times* and denote them by $t \in \mathcal{T}_D := \{t_k\}$. At each decision time t_k the strategy chooses an action (position or order) based on the information available up to that time. We will formalize “information available” later using a filtration $\{\mathcal{F}_t\}$.

For each asset a , define a price process $P_a(t)$ observed on the market clock, and define a return process on decision times. A convenient generic return definition is

$$r_a(t_k) := \log P_a(t_k) - \log P_a(t_{k-1}), \quad (15.1)$$

or, if one prefers simple returns, $r_a(t_k) := P_a(t_k)/P_a(t_{k-1}) - 1$. The book’s earlier chapters discuss this choice; here what matters is that returns are indexed on the decision grid.

Now define an *event universe*. Let $\mathcal{E}$ be a set of event identities. Each event $e \in \mathcal{E}$ is not merely a time stamp; it is a record with content and metadata. We assume an event can be linked to one or more assets; we denote the set of linked assets by $\mathcal{A}(e) \subseteq \mathcal{A}$. In the simplest case, $\mathcal{A}(e) = \{a\}$, but in practice events can be market-wide, sector-wide, or ambiguous.

An event identity must be defined so that it is stable under duplication and updates. In practice, the raw feed might include multiple items referring to the same underlying story. Therefore it is useful to define:

- a *raw item id* (unique to a feed record),
- a *canonical event id* (deduplicated cluster id),
- and possibly a *story thread id* (linking updates).

In formal terms, we can treat e as a canonical id, and treat raw items as realizations that map into e via a deduplication function. The crucial point for causality is that this mapping itself must be defined using only information available at the time of deduplication. If you cluster using future text (e.g., a later update that clarifies the story), then your cluster identity can leak. The minimal formalism does not require a sophisticated clustering model; it requires that you recognize event identity as a stateful object that must be versioned.

Market clock vs information clock

The market clock is the timeline on which prices and trades are recorded, typically aligned to exchange sessions, holidays, and bar definitions. The information clock is the timeline on which the strategy learns about events. These two clocks are not the same. Even if both are expressed in UTC timestamps, their semantics differ.

We formalize this by introducing, for each raw event item i , an *information arrival time* $\tau(i)$: the time at which the item becomes observable to the strategy. We also have the market decision times t_k . The essential question is: for a decision at time t_k , which items i are known? Those satisfying $\tau(i) \leq t_k$. That is the first admissibility rule.

The information arrival time is not necessarily the time the event occurred in the world. It is also not necessarily the time your dataset assigns as “published” or “updated.” It is the earliest time at which you can honestly claim you would have had the information in a live system, given the data source and operational assumptions. For a public news wire with millisecond timestamps, $\tau(i)$ might be close to the wire timestamp plus a small latency buffer. For a daily scraped dataset, $\tau(i)$ might be end-of-day. If you cannot define $\tau(i)$ defensibly, you cannot claim causality.

This distinction is why event-driven backtests so often lie: researchers inadvertently use a timestamp that is downstream of market reaction (e.g., a “vendor normalized” time), then treat it as upstream. The minimal formalism insists: define the information clock as the observable arrival of items, and build everything off that.

Decision time, trade time, and settlement time (conceptual)

Even once we have a market decision grid and an information arrival clock, we still need to distinguish decision time from trade time. In earlier backtesting chapters we introduced the idea that a signal computed at time t_k cannot be traded at the same price if execution occurs later. In event-driven trading, this becomes sharper.

Let the strategy compute a signal $s_a(t_k)$ at decision time t_k , then choose an action $u_a(t_k)$, for example a target position. The trade that implements this action occurs at some trade time $\tilde{t}_k \geq t_k$ (possibly equal if you assume immediate execution at the decision time, which is rarely defensible intraday). Settlement time matters for financing and cash management, but in this chapter we treat it conceptually: the key is that the price used for execution is associated with $\tilde{t}_k$, not t_k . Therefore, in a backtest, we must define a function mapping decision times to execution prices.

We can summarize the conceptual timeline:

$$\text{Information known up to } t_k \Rightarrow \text{compute } s_a(t_k) \Rightarrow \text{choose } u_a(t_k) \Rightarrow \text{execute at } \tilde{t}_k \Rightarrow \text{realize P\&L over } [\tilde{t}_k, \dots] \quad (15.2)$$

The book postpones detailed microstructure to Chapter 18, but the formalism here forces you to declare at least a minimal execution rule. Otherwise your event signal might be “predicting” a price move that already occurred before you could trade.

The point is not to burden the chapter with execution engineering. The point is to show that a valid event-driven strategy must specify: (i) when the signal is computed relative to information arrival, (ii) when the order is placed, and (iii) how the fill price is determined. Without these, you can always produce a paper edge that disappears in reality.

15.3.2 Events as random variables with observation delays

Having defined indices, we now treat events as random variables. The core complication is that we do not observe events at the moment they occur in the world; we observe a representation of them after a delay. This delay is itself random and can be state-dependent (big stories spread faster; obscure stories spread slower; vendor outages cause missingness; after-hours events may be recorded differently). Therefore, in formal terms, each event is associated with at least two times: a latent “true” time and an observed time.

Let e denote a canonical event (the underlying fact/story). Let T_e be the latent time at which the underlying event occurs (or becomes true). We may never observe T_e precisely. Let τ_e denote the time at which the event becomes observable to the strategy (the information arrival time for the first relevant item associated with e). Define the observation delay

$$D_e := \tau_e - T_e \geq 0. \quad (15.3)$$

In many settings we cannot identify T_e and thus cannot measure D_e directly. But we can reason about D_e and its distribution qualitatively, and we can model conservative bounds: for example, treat vendor published time as an upper bound on first-seen time, or add a buffer.

We also attach content to the event. Let X_e denote the content object: the text, metadata, or structured fields associated with the event. In many datasets X_e evolves through time via updates. Therefore it is often more accurate to represent the content as a time-indexed process $X_e(t)$, where $X_e(t)$ is the content available as of time t (e.g., the headline at first publication, then later the full article, then corrections). The minimal formalism treats this evolution explicitly because it is a major leakage source: if you use the final version of the article when trading at the initial publication time, you are using future information.

Thus, an event is not a single random variable; it is a bundle:

$$e \equiv (T_e, \tau_e, X_e(\cdot), \mathcal{A}(e)). \quad (15.4)$$

This representation makes it obvious why events do not behave like price features. A price $P_a(t)$ is observed at time t on the market clock. An event has a latent time, an observed time, and content that evolves.

True event time vs first-seen time vs vendor-published time

In practice, datasets provide timestamps that may correspond to different notions of time. It is crucial to name them and, where possible, store multiple fields.

True event time (T_e). This is the time the underlying fact occurs (e.g., a company announces internally; a disaster happens; an earnings report is released). Often this is not observable, and even if it is observable, it may not be tradable until dissemination occurs. For scheduled announcements (macro releases, earnings), T_e might be close to the official release time. For unscheduled events, it can be unknowable.

First-seen time (τ_e). This is the earliest time the strategy could have observed the event through its data sources. It is the most important time for causality. In a live system, this would be recorded by your ingestion pipeline as the time you received the item (or the time you scraped it). In historical datasets, this may be approximated by the earliest timestamp among items in the feed, but beware: feeds can backfill. The only reliable way is to have stored “as-of” snapshots historically, which is why governance-native data capture matters.

Vendor-published time. Many vendors provide a “published” time. Sometimes it equals first-seen time, but sometimes it reflects an editorial notion (e.g., the time the story is officially posted), not necessarily the time it was first distributed. Vendors also provide “updated” times. Using an updated time as if it were first-seen is a classic leakage error. The minimal formalism insists that you treat vendor times as candidate fields and explicitly choose which one represents τ_e .

In code and artifacts, this means: store all provided times, then define a derived field `known_time` that is the time your strategy uses as the admissibility boundary. Then test that `known_time` never falls after the market reaction in ways that indicate you used an updated field accidentally.

Delay distributions and censoring

Once we acknowledge observation delays, we must acknowledge their distribution. For a given event category c (e.g., earnings, macro release, analyst downgrade), the delay D_e may have a typical scale. For widely distributed scheduled releases, D_e can be very small and tightly distributed. For obscure unscheduled events, D_e can be large and heavy-tailed.

We can represent the delay distribution conditionally:

$$D_e \sim F_c(\cdot \mid \text{source, market state, time of day, asset liquidity, } \dots). \quad (15.5)$$

The minimal formalism does not require you to estimate F_c precisely. It requires you to recognize that such conditioning exists and that your backtest must be robust to it. For example, if your event dataset only includes events that were later deemed important, then small-delay events may be overrepresented. Or if your dataset is scraped once per day, then the delay distribution is dominated by your scraping cadence, not by market dissemination.

Censoring occurs when events exist but are not observed by your dataset. This is pervasive: you do not have a complete record of all information that influenced prices. Therefore, event-driven research is always conditional on your observation mechanism. You must treat your dataset as a sample from the true event process. This is not a trivial philosophical note. It means that a “no news” period in your data does not imply “no information arrived.” It implies “no information was recorded by your feed.” If you treat missingness as literal absence, you may build spurious signals.

In formal terms, you can introduce an observation indicator $O_e \in \{0, 1\}$ where $O_e = 1$ if event e is observed in your dataset. Then your inference is conditional on $O_e = 1$. If the probability of observation depends on the event content or impact (which it usually does), then your sample

is biased. This is one reason why event-driven backtests can look good: the dataset includes mostly impactful events, and the model learns to trade on “attention” rather than on content. A governance-native workflow mitigates this by including placebo tests and by measuring coverage statistics.

15.3.3 Event impact mapping: from event stream to return stream

We now define the object we ultimately care about: how events map to returns. The naive approach would be to regress returns on sentiment. The formal approach is to define a mapping from an event stream to a time-indexed feature stream, then define how that feature stream enters a decision rule.

Let $\{e_j\}$ denote the sequence of observed events, each with a first-seen time τ_{e_j} . Define the event stream as a point process on the information clock. For each asset a , define the subset of events relevant to a :

$$\mathcal{E}_a := \{e \in \mathcal{E} : a \in \mathcal{A}(e)\}. \quad (15.6)$$

The strategy constructs features $f_a(t_k)$ at decision times t_k using only events with $\tau_e \leq t_k$. This is the fundamental mapping:

$$f_a(t_k) = \Phi_a(\{(e, \tau_e, X_e(\cdot)) : e \in \mathcal{E}_a, \tau_e \leq t_k\}), \quad (15.7)$$

for some deterministic feature map Φ_a . The mapping can include windowing (only recent events), decay (downweight older events), and aggregation (counts, novelty, sentiment sums). The important point is that Φ_a is defined on the information set, not on future events.

Returns, meanwhile, are defined on the market clock and realized after trades. The relationship between $f_a(t_k)$ and future returns is what we evaluate. For example, we might study

$$\mathbb{E}[r_a(t_{k+1}) \mid f_a(t_k)], \quad (15.8)$$

or conditional distributions. But we must be careful: the relevant horizon depends on half-life. Therefore the mapping is often not one-step. Event impacts can be distributed over multiple future bars. This leads naturally to impulse-response intuition.

Impulse-response intuition

A minimal mental model is that events act like impulses injected into the market’s belief system, and returns respond with a dynamic adjustment. For a single event e relevant to asset a , suppose the event carries a scalar “shock” value z_e (which could be derived from sentiment, surprise, or a category indicator). Then a linear impulse-response model would say:

$$r_a(t_k) = \epsilon_a(t_k) + \sum_{e \in \mathcal{E}_a} z_e \cdot h_a(t_k - \tau_e), \quad (15.9)$$

where $h_a(\Delta)$ is an impulse-response kernel that maps elapsed time since observation into expected return contribution, and $\epsilon_a(t_k)$ is residual noise. This is not proposed as a realistic full model; it is a conceptual tool. It highlights what matters:

- Impact depends on elapsed time since the event became observable, not since it occurred latently.
- The response is distributed in time (a path), not a single jump, unless h_a is a delta spike.
- Multiple events can overlap and sum, creating clusters.

The shape of h_a encodes half-life. If h_a decays rapidly, the tradable horizon is short. If it decays slowly, a slower strategy might capture drift.

In practice, you will not estimate h_a perfectly. But you can use the impulse-response idea to design diagnostics: event studies that plot average cumulative returns in windows after events, aligned to first-seen time, stratified by event score. These diagnostics are part of the chapter because they give you a way to test whether your pipeline’s timing is plausible. If you see “impact” before the event time, you have leakage. If impact occurs entirely within a time interval you cannot trade, your strategy is not plausible. If impact persists, you can consider longer holding periods.

Windowed aggregation and decay kernels

In code, most event-driven features are not built by modeling each event separately, but by aggregating recent events into summary statistics. The minimal formalism expresses this as a windowed, decayed sum.

Let z_e be a scalar feature extracted from event content (e.g., sentiment score, novelty score, category indicator). Define an aggregation feature at decision time t_k :

$$f_a(t_k) := \sum_{e \in \mathcal{E}_a: \tau_e \leq t_k} z_e \cdot K(t_k - \tau_e), \quad (15.10)$$

where $K(\Delta)$ is a kernel that downweights events as they age. A simple choice is an exponential decay:

$$K(\Delta) = \exp(-\lambda\Delta) \mathbf{1}\{\Delta \geq 0\}, \quad (15.11)$$

where $\lambda > 0$ controls half-life. Another choice is a hard window:

$$K(\Delta) = \mathbf{1}\{0 \leq \Delta \leq W\}, \quad (15.12)$$

meaning you only count events in the last W units. Hybrid kernels are possible. The formalism matters because it makes the assumptions explicit and tunable. It also makes causality explicit: $K(\Delta)$ is defined only for $\Delta \geq 0$.

One can build multiple features by choosing different z_e and different kernels: counts (take $z_e = 1$), sentiment sums (take z_e as sentiment), novelty-weighted sums (take z_e as novelty), source-weighted sums (multiply by a source credibility weight), and so on. The key is that the feature at t_k is a deterministic function of events with $\tau_e \leq t_k$.

Importantly, this aggregation view clarifies why event-driven trading is often about pipeline design rather than model design. The function z_e depends on text processing. The function K depends on timing assumptions. Both are fragile if not audited. A high-capacity model can hide these fragilities by overfitting to them, but it cannot remove them.

15.3.4 Causality constraint: filtration and admissible features

All of the above is meaningless unless we enforce causality. The correct mathematical language for causality in time-indexed decision problems is a filtration: a growing family of sigma-algebras representing what is known as time progresses.

Let $(\Omega, \mathcal{G}, \mathbb{P})$ be a probability space representing the underlying randomness of markets and events. Let $\{\mathcal{F}_t\}_{t \geq 0}$ be a filtration representing the information available to the trader by time t . For a decision time t_k , the strategy must choose an action $u(t_k)$ that is $\mathcal{F}_{t_k}$ -measurable. This is the formal statement of “no look-ahead”: you cannot choose based on future information that is not in $\mathcal{F}_{t_k}$.

In a price-only setting, $\mathcal{F}_{t_k}$ might be generated by past prices and volumes up to t_k . In an event-driven setting, $\mathcal{F}_{t_k}$ includes both past market data and all event items whose information arrival times are $\leq t_k$, including their content as known at those times. If your pipeline uses revised content, it is using an object not measurable with respect to $\mathcal{F}_{t_k}$.

The practical challenge is that we do not have direct access to $\mathcal{F}_t$; we have datasets. Therefore we must construct an approximation to $\mathcal{F}_t$ that is defensible. This is why the chapter insists on immutable logs and as-of snapshots: they allow you to reconstruct what would have been in $\mathcal{F}_t$.

Information sets $\mathcal{F}_t$ and causal feature maps

We can now define admissible features precisely. A feature process $f_a(t)$ is admissible if for each decision time t_k , $f_a(t_k)$ is $\mathcal{F}_{t_k}$ -measurable. In operational terms: it is computable using only data that would have been available by t_k .

We can represent feature construction as a causal map:

$$f(t_k) = \Psi(\text{MarketData}_{\leq t_k}, \text{EventData}_{\leq t_k}), \quad (15.13)$$

where Ψ is deterministic given a parameter registry. The map is causal if it only depends on inputs at times $\leq t_k$. This sounds obvious, but it must be enforced because of subtle forms of leakage:

- **Timestamp leakage:** using a vendor “updated” time that is after the decision time.
- **Content leakage:** using the final revised text rather than the text as of t_k .
- **Label leakage:** using curated categories or tags assigned after market reaction.
- **Cross-sectional leakage:** ranking assets using events that arrive at slightly different times within the same bar.

The filtration language tells you what to do: ensure that every input used to compute $f(t_k)$ is measurable at t_k . Practically, you do this by building and testing a “known-at” function that filters the raw logs by arrival time and selects the correct content version.

A particularly important operational object is the *known-at operator*. For each event e with content versions, define $X_e^{(t)}$ to be the content available as of time t . Then feature extraction must use $X_e^{(t_k)}$, not $X_e^{(\infty)}$. If you cannot produce $X_e^{(t)}$, you must adopt conservative rules (e.g., use only fields that are known not to change, or delay trading until you can be sure the content

was stable). This is a key bridge between mathematics and governance: the filtration demands an as-of content representation, and the governance artifacts ensure you can reconstruct it.

Hard “no-look-ahead” rules for text pipelines

To make the filtration constraint actionable, we state hard rules that a text/event pipeline must satisfy to be considered causal in this book.

Rule 1: Explicit `known_time`. Every event record used in trading must include an explicit `known_time` field that represents the earliest time the strategy could have observed that record. This field must be derived from raw ingestion logs or conservative approximations, not from convenience vendor fields whose semantics are unclear. All feature construction must filter on $\text{known_time} \leq t_k$.

Rule 2: As-of content only. If text can be revised, the pipeline must either (i) store snapshots and use the as-of version at t_k , or (ii) avoid revised fields altogether and trade on stable fields only. Using final text is prohibited unless you can prove it was identical at decision time. In practice, this means storing raw items and their update history, not just a cleaned final table.

Rule 3: No hindsight labels or curated tags unless time-stamped. Any categorical label (“earnings beat”, “downgrade”, “merger rumor confirmed”) is forbidden unless you can prove the label existed in the feed at or before `known_time`. Many datasets assign such tags later. If you cannot time-stamp the label, you cannot use it. You may use raw text and derive your own label causally.

Rule 4: Session-aware admissibility. Market sessions matter. An article published after the close cannot be used for same-close decisions. The pipeline must map `known_time` to the market session and enforce admissibility accordingly. This includes time zones and daylight saving issues. A causal test suite must include checks that prevent an after-hours item from being used in pre-close decisions.

Rule 5: Bar-resolution conservatism. If your trading decisions are evaluated on bars (e.g., 1-minute bars), then within-bar ordering is ambiguous unless you have tick-level alignment. Therefore, if an event arrives within a bar, you must adopt a conservative convention: either trade in the next bar, or assume worst-case latency within the bar. Otherwise you risk using within-bar information that was not actually available before the price you trade on. This rule is often where event-driven backtests gain false edge; enforcing it typically reduces performance, which is precisely why it is necessary.

Rule 6: Placebo tests as causality probes. A causal pipeline should fail obvious falsification tests. If you randomize event times (within the same day) and performance remains, you likely learned something spurious like attention/volatility. If you shift event times forward slightly and

performance improves, you likely had leakage. Therefore the pipeline must include placebo and perturbation tests that act as unit tests for causality.

Rule 7: Artifact completeness. For any reported result, you must be able to produce: (i) the raw event ledger, (ii) the mapping from raw items to canonical events, (iii) the feature values at each decision time, and (iv) the decision trace showing which events contributed to which trades. Without these artifacts, the filtration constraint cannot be audited. In this book’s governance-native approach, the artifacts are not bureaucratic overhead; they are what makes the formalism real.

The reason to state these rules in a “minimal formalism” section is that they are not implementation details. They are the concrete operational translation of the mathematical requirement $u(t_k)$ must be $\mathcal{F}_{t_k}$ -measurable. In event-driven trading, many apparent “innovations” are simply violations of this requirement, dressed up as modeling. The formalism and the rules give you a way to distinguish genuine signal from accidental hindsight.

Summary: what the formalism buys us

With these definitions, an event-driven trading system can be described in a compact, auditable way:

1. A market decision grid $\{t_k\}$ and an execution rule mapping t_k to $\tilde{t}_k$ and prices.
2. An event ledger with explicit `known_time` (information clock) and as-of content.
3. A causal feature map Φ producing $f_a(t_k)$ from events with `known_time` $\leq t_k$.
4. A decision rule $u_a(t_k) = g(f_a(t_k), \dots)$ that is $\mathcal{F}_{t_k}$ -measurable.
5. An evaluation that respects half-life, timing, and robustness under placebo tests.

This description is deliberately minimal, but it is already enough to prevent the most common errors and to connect directly to the rest of the book. Chapter 14’s regimes can be interpreted as structure in the return process conditional on $\mathcal{F}_t$; events are part of $\mathcal{F}_t$ and may change regime probabilities. Chapter 16 will consume outputs from g (forecasts or scores) and translate them into weights under objectives and constraints. Chapter 18 will enrich the execution rule from a conceptual mapping to a microstructure-aware simulator. The formalism introduced here is the spine that lets those later additions remain coherent rather than becoming an un-auditable pile of heuristics.

15.4 Taxonomy of Events (Within Book Scope)

If event-driven trading is, at its core, the disciplined use of exogenous information arrivals, then the first practical question is embarrassingly simple: *what counts as an event?* In practice, “event” is a catch-all label that can refer to everything from a scheduled CPI release to a CEO resignation rumor, from an earnings call transcript to a sudden exchange outage. But different event types behave differently along the dimensions that matter most for a tradable

edge: predictability of timing, quality and completeness of the information carrier, speed of dissemination, typical half-life of market impact, and susceptibility to bias in historical datasets. A taxonomy is therefore not a classification exercise for its own sake. It is a risk-control device. It prevents you from building one generic pipeline that implicitly assumes all events are alike, then discovering later that your backtest was powered by one narrow class of events with favorable data semantics.

This chapter’s scope imposes a deliberate constraint: we focus on event types that can be handled with the book’s time-aware, governance-native methodology without yet requiring the full microstructure machinery of Chapter 18. That means we allow ourselves to discuss intraday timing and latency conceptually, but we do not pretend we can capture tick-level execution realism. It also means we prioritize event categories where we can plausibly obtain timestamped records and where a causal “known-at” notion can be defended. Within those constraints, the taxonomy below gives a structured way to think about which event families belong to which research approach, which failure modes are dominant, and how outputs from Chapter 15 will later integrate into portfolio construction in Chapter 16.

15.4.1 Scheduled macro events: releases and central bank decisions

Scheduled macro events are the cleanest entry point into event-driven trading because they are explicitly calendared. They include economic releases (inflation prints, employment reports, GDP releases), auctions, policy decisions (central bank rate decisions, statements, press conferences), and sometimes major fiscal events (budgets, debt ceiling deadlines) when their timing is known in advance. Their defining feature is that the market can anticipate *the time* of information arrival even if it cannot anticipate *the content*. This single fact changes the entire structure of the trading problem.

Because timing is known, scheduled macro events are often markets of “surprise” rather than “news.” What moves prices is not the level of the release but the deviation from expectations, and those expectations can be proxied by consensus forecasts, swaps/implicit rates, or the market’s pre-event positioning. In a minimal Chapter 15 framework, we can represent this by separating the event into (i) the scheduled timestamp, (ii) the realized value (content), and (iii) an expectations proxy and its uncertainty. The tradable signal is then some function of surprise and of the market state (risk-on/off, liquidity conditions, recent volatility).

Scheduled macro events also illustrate why event-driven trading is often about half-life. The immediate reaction to a major release can occur in seconds. For many participants, capturing that reaction is a microstructure and execution contest. However, there is frequently a slower second phase: repricing along the yield curve, spillovers into FX and equities, and post-event drift as the market digests the implications. For Chapter 15, we acknowledge both phases but emphasize that the researcher must declare which phase is being targeted. If you backtest on daily bars, you are not capturing the first phase; your pipeline must therefore be designed to test whether the second phase is persistent enough to be tradable after costs and slippage.

In terms of data semantics, macro events have two major pitfalls. The first is the temptation to use revised data. Many macro releases have later revisions, and vendors often provide both first prints and revised prints. A causal event-driven pipeline must use the first print as the

content known at the time. Using revised values is pure look-ahead. The second pitfall is timestamp ambiguity in press conferences. A rate decision might be released at a known time, but the press conference can introduce new information in a continuous stream of language. If your dataset stores a single timestamp for the whole conference transcript, you must be careful: the market reaction depends on when specific phrases were uttered, not when the transcript was archived. Within this chapter's scope, we treat that as a conceptual caution and reserve fine-grained handling of continuous intraday text streams for later chapters and more specialized data.

Finally, scheduled macro events are the natural place to introduce cross-asset propagation. A U.S. inflation surprise can reprice global rates, strengthen or weaken the dollar, and transmit into commodities and EM FX. But to handle propagation responsibly, you need a mapping from event to instrument sets and a declared horizon. This chapter's taxonomy therefore treats macro events as both clean (because calendared) and dangerous (because speed compresses tradability), and it encourages a two-tier approach: immediate reaction belongs mostly to Chapter 18 execution; slower repricing belongs to Chapter 15 event features feeding Chapter 16 portfolio decisions.

15.4.2 Corporate events: earnings, guidance, actions, filings

Corporate events are the most familiar territory for many practitioners because they connect naturally to equities and credit and because they combine scheduled and unscheduled elements. We group them into several subfamilies: earnings and guidance, corporate actions, and regulatory filings. What unifies them is that they are firm-specific (or at least firm-anchored) and are often disseminated through multiple channels: press releases, filings, conference calls, and media interpretation. This multi-channel nature is precisely why corporate events require a careful event identity and deduplication design.

Earnings and guidance. Earnings releases are scheduled in the sense that a date and time window is typically known, but they are not as clean as macro releases because firms can shift timing, pre-announce, or release after hours in ways that interact with session boundaries. Moreover, earnings are not a single number. They include a headline EPS/revenue, but also guidance, margin commentary, segment details, and the tone of management on a call. The first market reaction may reflect headline numbers; subsequent drift may reflect narrative interpretation. Therefore, in a minimal Chapter 15 pipeline, it is often wise to treat earnings as a layered event:

- a structured numeric surprise layer (headline vs consensus),
- a guidance revision layer (direction and magnitude),
- and a language layer (call sentiment, uncertainty, forward-looking statements).

Even if later chapters allow sophisticated NLP, Chapter 15 insists that the pipeline be explicit about *what was known when*. Earnings transcripts are often posted with delays; edited versions can appear later. If you backtest using the final transcript text at the release time, you are likely leaking.

Corporate actions. Stock splits, dividends, buybacks, mergers and acquisitions, and capital raises are often treated as events with clear announcements. But their market impact depends on context: whether the action was expected, how it affects balance-sheet constraints, and how the market interprets signaling. Corporate actions also illustrate the difference between an announcement event and an implementation event. A dividend is announced at time τ_{announce} but has ex-dividend and payment dates that matter for returns and arbitrage. A split is announced but implemented later. A merger rumor may be announced as a confirmed deal later. If you do not separate these stages, your event dataset will collapse a multi-stage process into one record, and your backtest will misattribute impact.

Regulatory filings. Filings (10-K, 10-Q, 8-K, prospectuses, credit agreements) are often timestamped and thus seem clean. But they also introduce a unique problem: they are long and complex, and market interpretation may lag. Filings can therefore produce slower half-life effects, which is good for tradability at daily frequency, but they also produce more confounding: in the days after a filing, many other things happen. The pipeline must therefore distinguish immediate reaction from slow drift and must include robustness checks for overlapping events.

Corporate events also raise the instrument mapping challenge in a sharp form. A filing might refer to a subsidiary; a rumor might refer to an acquirer and a target. An earnings event can affect suppliers, competitors, and sector ETFs. Therefore, even within a “corporate events” category, you need sub-mapping rules: primary ticker mapping, secondary spillover mapping, and confidence measures. The taxonomy is not complete unless it includes these mapping considerations, because the mapping is often where non-causal errors occur (e.g., attributing sector movement to a firm-level story).

From the perspective of the book’s arc, corporate events are the canonical case where Chapter 15 outputs become inputs to Chapter 16. An earnings surprise feature can be turned into a forecast; a filing-derived uncertainty score can be turned into a risk throttle; a corporate action announcement can be turned into a conditional position sizing rule. But none of these are credible unless the event identity and timestamps are governed. Hence the taxonomy emphasizes that corporate events are rich but pipeline-intensive.

15.4.3 Unscheduled events: shocks, incidents, rumors (and why they are hard)

Unscheduled events are the romanticized domain of event-driven trading: accidents, lawsuits, geopolitical shocks, executive scandals, cyber incidents, surprise resignations, and rumors that may or may not be true. They are also the domain where researchers most often fool themselves, for reasons that can be stated plainly: *the historical record is not a faithful record of what was knowable at the time.*

There are several layers to why unscheduled events are hard.

First, the true event time is often ambiguous. When did the incident occur? When did someone notice? When did it become public? When did the market become aware? These times can differ by hours or days. A dataset might include only the time of a later article that summarizes the incident, not the moment of first dissemination. Trading on that summary

time can easily look predictive because the market may have moved earlier, and you might inadvertently align the summary with later drift or reversal.

Second, rumor datasets are especially vulnerable to survivorship and hindsight. Many rumor records exist because the rumor later proved to be important. Rumors that were false and died quietly are underrepresented. This means the dataset is conditioned on significance, which creates spurious predictability: the presence of a record is correlated with impact, regardless of content. A naive model learns to trade on “attention” rather than on truth. A rigorous pipeline must therefore include placebo tests and explicit coverage statistics, and it must treat the observation mechanism as part of the model.

Third, unscheduled events have extreme heterogeneity in dissemination speed. Some shocks hit every terminal simultaneously; others diffuse slowly through niche channels. If you are not explicit about your data source, you cannot make a tradability claim. A retail-accessible news API is not equivalent to a low-latency wire feed. The same underlying event can have a half-life of seconds for one participant and days for another. Therefore, in this book’s scope, unscheduled events are handled with extra conservatism: you must define a `known_time` that is defensible, and you must accept that the most aggressive forms of rumor trading are execution contests beyond Chapter 15.

Fourth, unscheduled events often come in clusters and cause regime-like behavior. A geopolitical crisis is not a single event but a sequence of headlines that evolve. The market response is not a clean impulse; it is a path dependent on updates. This is where Chapter 14’s regime perspective becomes invaluable. In a crisis regime, the same kind of headline can have a different impact than in calm markets. Thus, the taxonomy treats shocks and incidents as events that are likely to be both cross-asset and state-dependent. The correct output of a Chapter 15 pipeline might therefore be less about directional forecasts and more about risk throttles, volatility expectations, and exposure caps during elevated news intensity.

In short, unscheduled events are hard because the dataset semantics are hard. They are not excluded from the chapter, but they are treated as high-risk research objects that demand stronger governance artifacts and more skeptical evaluation.

15.4.4 Market micro-events (deferred to Ch. 18: microstructure)

Market micro-events are events that occur inside the market mechanism itself: order-book imbalances, large trades, quote changes, auction imbalances, halts, limit up/limit down triggers, exchange outages, and microstructure-driven phenomena such as liquidity sweeps. They are unquestionably “events,” and they are tradable in some contexts, but they are out of scope for Chapter 15 for a principled reason: micro-events live on the same clock as execution, and their tradability is inseparable from microstructure and transaction costs.

If you attempt to treat micro-events with the same machinery as macro or corporate events, you will produce misleading results. The half-life is often measured in milliseconds to seconds. The decision time and trade time separation becomes the entire strategy. Furthermore, micro-event data is large, noisy, and structurally dependent on venue rules and feed latencies. This is precisely why the book postpones microstructure to Chapter 18: it requires dedicated modeling of costs, slippage, and order execution, and it introduces additional governance concerns (synchronization

across feeds, clock drift, and venue-specific semantics).

That said, it is important to include micro-events in the taxonomy for two reasons. First, many practitioners mistakenly conflate “event-driven” with “high frequency.” By explicitly placing micro-events in Chapter 18, the arc clarifies that event-driven trading can be high frequency or low frequency depending on event class and half-life. Second, micro-events often interact with macro and corporate events. For example, a CPI release can induce microstructure stresses, widening spreads and changing fill probabilities. Even if Chapter 15 does not model micro-events directly, it must remind the reader that execution conditions around events are not constant. This reminder becomes a bridge to Chapter 18 later.

Thus, micro-events are recognized here as a distinct category whose correct treatment requires the later machinery of execution modeling. The taxonomy marks them as a boundary: if your edge relies on micro-events, you have left Chapter 15 territory and entered the microstructure domain.

15.4.5 Cross-asset propagation: event spillovers and mapping instruments

A taxonomy of events is incomplete unless it addresses propagation. Most events do not confine themselves neatly to one ticker. Macro events propagate across rates, FX, equities, and commodities. Corporate events propagate through supply chains, sector peers, indices, and credit spreads. Shocks propagate through risk sentiment and funding markets. Therefore, event-driven trading often requires an explicit mapping from an event to a set of instruments, and an explicit model of how impact decays across that set.

Within this book’s scope, we treat cross-asset propagation as a disciplined mapping problem rather than a full structural model. The minimal ingredients are:

1. **Primary linkage:** the instrument(s) explicitly referenced by the event.
2. **Secondary linkage:** a rule-based expansion to related instruments (sector ETF, index future, peer group).
3. **Confidence:** a score indicating mapping certainty.
4. **Horizon:** an assumed half-life or decay parameter for spillover impact.

For example, an earnings surprise for a large semiconductor firm may have primary linkage to that firm’s stock, secondary linkage to a sector ETF, and tertiary linkage to a broad index future if the firm is index-heavy. A central bank surprise has primary linkage to the domestic rates complex, secondary linkage to FX, and then to equities via discount rates and risk appetite.

The key methodological point is that cross-asset propagation can create subtle leakage if not handled carefully. Suppose you build a cross-sectional strategy that ranks stocks based on event features. If events arrive at slightly different times for different stocks, and you compute a ranking at time t using all available events “today,” you might inadvertently include events that arrived after t for some stocks, because you aggregated at daily frequency. This is a form of cross-sectional timing leakage. Therefore, the mapping and aggregation must be time-aware per instrument. Each instrument’s feature vector at time t_k must be built only from events known by t_k relevant to that instrument, not from a global set built with looser timing.

Cross-asset propagation also raises a conceptual caution: sometimes the correct trading response to an event is not to trade the referenced instrument but to trade a proxy. Liquidity, borrow constraints, and costs might make it preferable to express a view via an ETF, an index future, or an option. Chapter 15 does not develop full execution or derivatives machinery, but the taxonomy must acknowledge this reality because it affects what counts as “the strategy.” An event may be mapped to a set of tradable exposures, and later portfolio construction will decide the optimal allocation across them under constraints.

Finally, propagation is where the chapter connects most naturally to Chapter 16. An event feature might produce a forecast on one asset, but portfolio construction must consider correlation structure and concentration risk. If a macro event produces coherent signals across many assets, the naive response is to take many aligned positions and become inadvertently concentrated. A disciplined pipeline will therefore output not only directional scores but also event intensity measures and uncertainty measures that can be used for risk budgeting. In other words, propagation is not only about adding more tickers; it is about recognizing that events change the joint distribution of returns.

In summary, this taxonomy is designed to be actionable within the book’s constraints. Scheduled macro events are clean in timing but often too fast for naive execution. Corporate events are rich but demand careful identity and as-of content handling. Unscheduled shocks and rumors are seductive but dataset-biased, requiring conservatism and skepticism. Market micro-events are acknowledged but deferred to Chapter 18 because they are inseparable from execution. Cross-asset propagation is treated as a mapping and governance problem that prepares the outputs of Chapter 15 to become disciplined inputs for Chapter 16. The taxonomy is therefore not a list; it is a map of which research discipline you must apply to each event family if you want your “event-driven” claim to survive contact with reality.

15.5 Data Sources and Their Hidden Biases

Event-driven trading research begins long before any model is trained or any sentiment score is computed. It begins with a more uncomfortable question: *what exactly is the dataset a record of?* In price-based work, the answer is usually straightforward. A closing price is a market-clearing number recorded by an exchange or a vendor at a known time. There are still issues—corporate actions, bad ticks, holidays, stale quotes—but the semantics of the object are relatively stable. News and event data do not enjoy that stability. They are mediated by vendors, editorial practices, distribution technologies, and the incentives of whoever decided that a particular piece of information was worth recording. The biggest risks in event-driven research are therefore not merely statistical. They are epistemic: you may think you are testing whether an event signal predicts returns, when you are actually testing whether your dataset contains subtle hints of the future, or whether your coverage mechanism selects only the events that mattered.

This section is a field guide to those hidden biases. The goal is not cynicism; the goal is defensible design. We want a pipeline that can survive an audit by an internal risk committee or a skeptical coauthor who asks, “Could this be look-ahead? Could this be survivorship? Could this be vendor drift?” If you cannot answer those questions concretely, you do not have research;

you have a story. The five subsections below emphasize the failure modes that are common precisely because they are easy to miss: differences between vendors and feeds, the peculiar hazards of social media and alternative data, the dangers of backfills and revisions, the fragility of entity resolution, and the pervasive problem of label leakage from curated tags.

15.5.1 News vendors, press wires, and public feeds

The first bias to understand is that “news” is not a single thing. It is an ecosystem of distribution channels with radically different latency, coverage, and curation. A press wire aimed at professional terminals is not the same as a public RSS feed. A vendor that sells structured events is not the same as a newspaper archive. Your event-driven strategy is therefore inseparable from your data source. In practice, the same underlying story can appear on different channels at different times, with different wording, and sometimes with different emphasis. The tradability of an event signal depends on where in that ecosystem your dataset sits.

Latency and the illusion of simultaneity. Many retail-accessible datasets implicitly assume that what is published is what was first seen. In reality, professional channels often disseminate a headline first, then publish a fuller article later, and public reprints may follow later still. If your dataset uses the later publication time but your backtest treats it as the first-seen time, you will misalign the information clock. This misalignment can create false predictability in either direction: you might assign a story too early and capture a move that had already occurred; or you might assign it too late and miss the first move, incorrectly concluding there is no effect. The important point is that you must map the dataset’s timestamp semantics to the strategy’s `known_time`. If you cannot, the correct approach is conservative: delay the use of the information until you can be confident it was available.

Coverage bias and the “attention” confound. Different vendors have different coverage policies. Some cover large-cap equities heavily, some cover certain geographies, some focus on macro, and some focus on specific sectors. This creates a selection mechanism: the dataset will contain more events for some assets than others, and the *density of coverage* itself is informative about liquidity, volatility, and institutional attention. A naive model can exploit this without truly learning anything about content. For example, it can learn that stocks with more headlines tend to be more volatile, and it can appear to “predict” returns simply by timing volatility risk premia. In research, you must separate content effects from coverage effects. Practically, this means you should explicitly measure and log coverage statistics as artifacts: event counts per asset per period, source distribution, and changes in coverage over time. If your strategy performance tracks changes in coverage, you may have learned the vendor’s behavior rather than the market’s.

Editorial curation and post hoc structure. Vendors add value by cleaning, categorizing, and summarizing. That value is also a danger. A vendor may tag a story as “M&A” or “downgrade” based on an editorial decision that occurs after reading the full context, or after later updates clarify the story. A vendor may generate a “summary” that uses hindsight language.

These curated fields often embed information that was not available at the initial dissemination. Therefore, the more structured and convenient the dataset looks, the more skeptical you should be. Convenience is not free; it is often paid for with hidden look-ahead. The governance-native rule is to treat vendor-added structure as a separate data layer with its own `known_time`, not as an intrinsic part of the raw event.

Duplicate and thread handling. Press wires and vendor feeds frequently emit multiple items for the same underlying story: initial headline, correction, update, follow-up. Public feeds may reprint or paraphrase. If your pipeline counts each item as a separate event, you can accidentally create a spurious signal: repeated items are correlated with significance and with volatility. Alternatively, if you deduplicate aggressively using information from later updates, you can leak. Therefore deduplication must be treated as a time-aware process: at time t , you can only deduplicate using content observed up to time t . In practice, for Chapter 15, it is often safer to adopt conservative deduplication rules (hash identical headlines, cluster within short windows by simple string similarity) and to log the clustering decisions as artifacts.

Vendor drift and dataset non-stationarity. Vendors change formats, taxonomies, and distribution policies. A feed in 2015 is not the same as the same feed in 2025. If you train a model across long spans, you may be capturing vendor drift rather than market structure. Worse, drift can create apparent regime shifts that are purely data artifacts. This is why the book emphasizes time splits and source splits: you should test whether a model trained on one period generalizes to later periods where the vendor may have changed behavior. If the model fails, that may be a market regime shift, but it may also be a data pipeline regime shift. The only way to tell is to have coverage and format artifacts logged through time.

15.5.2 Social media and alternative data (conceptual cautions)

Social media and alternative data are appealing because they seem closer to the source of sentiment and because they can be fast. They are also perilous because they are governed by platforms, incentives, and adversarial behavior. Within this chapter’s scope, we treat them conceptually and emphasize the methodological hazards rather than offering a promise of alpha.

Non-representative sampling and platform incentives. Social media is not “the market’s mood.” It is the mood of the platform’s participants, filtered by recommendation algorithms and influenced by engagement incentives. This creates a deep sampling bias: the distribution of content is shaped by what gets clicks, not by what is true. Moreover, the platform can change its algorithms, user base, and moderation policies. Therefore, even if you can compute a clean sentiment score, you cannot assume its relationship to markets is stable. Any claim must be conditioned on the platform and the sampling window, and must be robust to platform drift.

Bots, manipulation, and adversarial content. Unlike press wires, social media is adversarial. Actors can create content to move prices, especially in smaller names. If your pipeline treats social sentiment as exogenous information, you may be treating an adversarial signal as if

it were a genuine belief. This creates a unique evaluation problem: a successful trading signal might simply be a predictor of manipulation campaigns, which may not be stable or ethically acceptable. Governance must therefore include provenance and integrity checks: bot detection proxies, account age distributions, and anomaly flags. In this chapter we do not solve these problems; we insist they exist and that ignoring them is a research failure.

Timestamp ambiguity and deletion. Social platforms allow posts to be edited, deleted, or shadowed. Historical datasets may contain content that was visible for a short time but later removed, or may miss content that was visible but not captured. Furthermore, APIs often provide timestamps that reflect server time rather than user time, and time zones can be inconsistent. This creates a severe “as-of” problem: you cannot assume your downloaded dataset reflects what a trader would have observed in real time. The correct posture is conservative and governance-heavy: if you use social media data, you must store raw captures with capture times and accept that your historical record is an approximation.

Alternative data: non-market clocks and hidden revisions. Alternative data includes web traffic, satellite imagery, app downloads, credit card aggregates, and other proxies for economic activity. These data often arrive on non-market clocks (weekly, monthly, irregular), are revised, and are subject to vendor methodology changes. They can be powerful precisely because they are not directly priced, but they also introduce huge leakage risks: many vendors “clean” the data ex post, fill missing values, adjust for holidays, or rebaseline series. If you trade on the cleaned series without as-of snapshots, you can inadvertently use future revisions. Therefore, even though alternative data is not the central focus of Chapter 15, the taxonomy and governance logic apply: define `known_time`, demand as-of snapshots, and treat vendor methodology as a parameter registry.

15.5.3 Backfills, revisions, and “as-of” snapshots

Backfills and revisions are the single most important hidden bias in event-driven datasets, because they create a dataset that looks internally consistent while embedding future information. Many researchers who are careful about look-ahead in price data become careless with event data because they assume timestamps are trustworthy. They often are not.

Backfill mechanisms. Backfill can occur for benign reasons: a vendor ingests a source late, fixes missing records, or corrects an outage. It can also occur because the vendor improves coverage historically (e.g., adds a new source and retroactively indexes it). From a research perspective, both are dangerous. A backfilled item appears in your historical dataset with a timestamp that makes it look like it was available at that time, but it may not have been available to you in real time. If your strategy trades on it, you are implicitly assuming you had the data then. In live trading, you would not. This is a subtle form of survivorship: your historical dataset is richer than any real-time dataset would have been. The cure is conceptually simple but operationally demanding: maintain *as-of snapshots*, meaning datasets captured at specific points in time and never retroactively altered.

Revisions of content and metadata. Text content can be revised (corrections, additional details). Metadata can be revised (categories, tickers, relevance scores). Even a headline can be updated for clarity. If your dataset stores only the final version, you cannot reconstruct what was known at the first release. A naive sentiment model might then use information that only appeared after the market moved. Again, the cure is as-of storage: keep the raw versions and their update history. At minimum, store the earliest version and the final version and record update times.

The “as-of” snapshot principle. An as-of snapshot is a dataset view that reflects what was available up to a certain capture time T_{snap} . Formally, if we denote the full vendor dataset at time T as $\mathcal{D}(T)$, then the as-of snapshot is $\mathcal{D}_{\leq T_{\text{snap}}}$, frozen. In event-driven work, the relevant object for backtesting is not the vendor dataset today; it is the vendor dataset as it existed then. If you cannot obtain that, you must either (i) restrict yourself to sources with stable historical semantics, or (ii) accept conservative trading rules that reduce the chance of leakage, such as delaying use of the event by a day.

The book’s governance-native approach makes this practical by treating snapshots as artifacts. Each snapshot gets a hash, a manifest (source list, ingestion time, missingness), and an explicit mapping into feature construction. Without this, your event features are built on sand.

A diagnostic mindset. Even if you cannot build perfect snapshots, you can still diagnose backfill risk. For example, compare the distribution of event counts over time and look for sudden jumps that coincide with vendor changes. Compare early periods to later periods in terms of source mix. Examine whether certain event categories only appear after a certain date. These diagnostics do not eliminate backfill, but they help you avoid naive conclusions and can motivate time splits that separate pre- and post-vendor changes.

15.5.4 Entity resolution: tickers, names, subsidiaries, and aliases

Entity resolution is the problem of mapping text mentions to tradable instruments. It is simultaneously mundane and dangerous. Many event-driven failures are not about sentiment or modeling at all; they are about mapping the wrong story to the wrong ticker or mapping too many tickers to the same story.

Why entity resolution is hard. Company names are ambiguous. Subsidiaries have different names. Firms rebrand. Tickers change. The same string can refer to different entities in different contexts (consider common words that are also company names). News often references products, executives, or locations rather than tickers. Even when a ticker is provided, it may be wrong or stale. Therefore, a robust event-driven pipeline must treat mapping as probabilistic, not deterministic, and must log mapping confidence.

Primary vs secondary mapping. A useful structure is to separate:

- **primary mapping:** the entity that the story is explicitly about,

- **secondary mapping:** related entities (suppliers, competitors, sector ETFs),
- **market mapping:** broad indices or macro instruments potentially affected.

This separation matters because it prevents a common mistake: attributing a market-wide story to a single ticker and then concluding that ticker is predictive. If a macro story drives the whole sector, the correct exposure might be sector-level, not firm-level. The mapping layer should therefore include rules for when an event is treated as market-wide versus firm-specific.

Temporal consistency and corporate actions. Entity resolution must be time-aware. If a ticker changes, the mapping must reflect the ticker valid at the event time, not today’s ticker. Corporate actions can alter symbol histories, and vendors can normalize these histories in ways that again introduce hidden revisions. Therefore, entity resolution must rely on a versioned entity dictionary that includes effective dates. This is not glamorous, but it is the difference between a credible and a fragile system.

Evaluation leakage through mapping. A subtle leakage occurs when entity resolution uses information that is only clear after the fact. For example, clustering a set of articles into a story thread might be easier once you have later articles. Mapping a rumor to a target might be easier once the deal is confirmed. If your resolution algorithm uses the full corpus, you may be using future disambiguation. A causal entity resolution pipeline must therefore operate as-of time, using only information available then. Practically, this is hard. The conservative approach is to restrict to high-confidence explicit mentions and to accept lower coverage rather than to accept high coverage with hidden look-ahead.

15.5.5 Label leakage from curated tags and hindsight summaries

Label leakage is the most insidious bias in event-driven datasets because it looks like convenience. Vendors and platforms often provide tags: event type, relevance, sentiment, impact scores, and summaries. These are extremely tempting because they allow you to bypass hard work. They are also often contaminated by hindsight.

Curated tags as future information. Consider a tag like “earnings beat.” To assign it correctly, you need the reported number and the consensus expectation. But which expectation? The one that existed at the time, or a later consolidated expectation? What about after-hours revisions? Similarly, a tag like “merger confirmed” can only be assigned after confirmation. If your dataset includes such tags attached to the initial rumor article, it may reflect later knowledge. Even a tag like “high impact” may be assigned after observing market reaction. These tags are therefore frequently not $\mathcal{F}_t$ -measurable at the time you trade. Using them gives you a model that is partly reading the answer key.

Hindsight summaries. Many datasets include a “summary” field that is not the original text but a later synopsis. Such summaries often incorporate knowledge of outcomes (“shares fell after...”) or include details that were revealed later. If you compute sentiment or features from

these summaries, you are leaking. The safe approach is to compute features from raw text as it was disseminated, not from vendor summaries.

The illusion of “ground truth” event classification. Researchers often want ground truth labels: which articles correspond to which event types, which are positive/negative, which are material. Vendors provide such labels, and it feels like progress. But the existence of a label is not evidence of causal availability. In a governance-native approach, labels are treated as suspect unless they have explicit time-of-assignment metadata. If a vendor can provide the timestamp when the tag was attached, then it can be used as-of that time. If not, it should be excluded from trading features and used only for retrospective analysis, with clear disclaimers.

A disciplined alternative: derive your own labels causally. The book’s preferred alternative is humble but robust: derive simple labels from raw text using transparent rules, and treat them as features rather than as ground truth. For example, rather than using a vendor’s “downgrade” tag, detect downgrade-related phrases in the headline and compute a confidence score. This is less accurate in an absolute sense, but it is more defensible, because you can prove the label existed at decision time (it is derived from the text you saw then). Later chapters can introduce richer models, but Chapter 15 emphasizes that causal validity is more important than label convenience.

Auditing labels and proving admissibility. When labels must be used (e.g., for evaluation), the pipeline should produce label-audit artifacts: distributions over time, correlations with contemporaneous returns, and perturbation tests. If a label is suspiciously predictive at horizons that precede the event time, or if it predicts returns even when event times are randomized, it is likely contaminated. These audits are not merely academic; they are the practical tools that prevent you from shipping a strategy that is powered by dataset hindsight.

Taken together, these hidden biases explain why event-driven trading cannot be treated as a simple “add text features” exercise. The data source determines what was knowable and when. Social and alternative data introduce adversarial and platform-driven distortions. Backfills and revisions can turn a historical dataset into a future-informed archive unless you enforce as-of snapshots. Entity resolution can inject subtle misattribution or even causal leakage through hindsight disambiguation. Curated tags and summaries can smuggle in outcomes. The discipline required in Chapter 15 is therefore, at bottom, a discipline of epistemic honesty: define the information clock, store immutable artifacts, and treat every convenient field as guilty until proven innocent. Only then do event features become legitimate inputs to trading decisions and, later, to portfolio construction.

15.6 Building an Event Log: The Governance-Native Starting Point

Every event-driven trading system is only as credible as its event log. This is not a rhetorical flourish; it is the central engineering and governance fact of the modality. You can survive sloppy

modeling with sufficiently conservative risk controls. You cannot survive a sloppy event log, because the event log defines what the strategy *knew* and *when* it knew it. If the log is not immutable, you cannot prove that a backtest is causal. If the log does not preserve the raw records, you cannot audit how meaning was derived. If the log conflates timestamps, you cannot distinguish an edge from hindsight. In the logic of this book, building the event log is therefore not “data preparation” that happens before the real work. It *is* the real work, because it is the only way to make event-driven claims defensible.

The goal of this section is to specify a governance-native event logging architecture that can be implemented with transparent tools, can scale to multiple sources, and can produce audit-grade artifacts. The design principle is simple: treat event ingestion as an append-only stream, preserve raw content with minimal transformation, then produce deterministic derived layers (canonical schema, normalization, deduplication clusters, instrument mapping) with explicit versioning and manifests. This mirrors the approach used in Chapter 05 for market data pipelines, but the constraints are stricter because event data is more revision-prone and because vendor convenience fields are often contaminated.

We will describe the event log as a set of artifacts rather than as an abstract concept. At minimum, you want: (i) immutable raw ingestion logs, (ii) a canonical event schema for normalized records, (iii) a deduplication/clustering layer that links related items without leaking future information, (iv) an instrument mapping layer with confidence, (v) a market-session normalization layer, and (vi) a provenance manifest that binds the whole run together. When these exist, every subsequent step in Chapter 15 becomes simpler: feature engineering becomes a deterministic transform; evaluation becomes a traceable decision process; and when results are questioned, you can answer with evidence rather than narrative.

15.6.1 Raw ingestion: immutable append-only logs

Raw ingestion is the moment where your event pipeline touches reality. It is the only moment where you can honestly say, “this is what arrived, at this time, from this source.” Everything after ingestion is a transformation, and transformations can be wrong. Therefore governance starts by making ingestion immutable.

Append-only means no rewrites. An append-only ingestion log is a sequence of records where new records can be added but existing records cannot be changed. If a source later corrects an article or updates a story, that update is appended as a new record with its own arrival time; the old record remains. This property is essential because it preserves the historical information state. If you overwrite the old record with the corrected version, you have destroyed evidence about what was knowable when. In event-driven research, overwriting is equivalent to quietly allowing hindsight to seep in.

Capture both “source time” and “ingestion time.” Raw ingestion should record at least two times per record:

- **source-provided time(s):** whatever timestamps the source provides (published time, updated time, etc.);

- **ingestion/capture time:** the time your system observed the record.

The ingestion time is often the closest thing you have to `first_seen_time` in a live system. If you do not have ingestion time, you will later be forced to guess first-seen time from source fields, which is precisely where many studies fail.

Store raw content as raw bytes or raw text. Raw ingestion must store the actual content payload as received: headline, body, metadata JSON, identifiers. Do not normalize or clean at ingestion beyond minimal encoding fixes. Do not strip punctuation, remove stopwords, or rewrite fields. Cleaning is a derived transformation and should be reproducible; raw storage is the anchor.

Include a source registry. Ingestion is meaningless without a stable definition of sources. Each record should reference a source identifier that is defined in a source registry: vendor name, feed name, API endpoint, authentication context, and any relevant versioning (e.g., API schema version). This enables later audits: if vendor drift occurs, you can locate which runs were affected.

Partitioning and file format (conceptual). Implementation details vary, but the principles remain: partition raw logs by ingestion date (and possibly by source) to avoid rewriting large files; use a format that preserves exact content and metadata; and compute content hashes so you can detect accidental modification. Even if you store data as line-delimited JSON, the governance aspect is the same: immutable files, checksums, and run manifests.

The critical governance insight: ingestion is the evidence. If a research result is challenged, your defense is not your model. Your defense is your ingestion evidence: “Here is the raw record. Here is when we saw it. Here are the transformations we applied. Here is the feature value we computed at decision time.” That chain is only possible if ingestion is append-only and timestamped.

15.6.2 Canonical event schema: minimum fields

Raw logs are necessary but not sufficient. They are messy and source-specific. To do research and to build features, you need a canonical schema: a normalized representation of event records that is consistent across sources. The canonical schema is a derived layer; it must be deterministic given the raw logs and a schema configuration. It should be minimal but complete enough to support causality and auditing.

A useful design is to distinguish:

- **raw item:** a single ingested record from a source at ingestion time;
- **canonical event record:** a normalized representation of that item with standardized fields;
- **canonical event id:** a stable identity for a deduplicated event concept (handled later).

In this subsection we define the minimum fields for the canonical event record. “Minimum” means: if you cannot provide these fields, you should not trust the event record for trading features.

Event identifiers, source identifiers, and dedup keys

Raw item identifiers. Each ingested record should have a `raw_item_id` that is unique within a source. This may be provided by the source (article id) or generated at ingestion (e.g., a hash of source+payload+ingestion time). The important requirement is uniqueness and stability.

Source identifiers. Each record must have `source_id` and ideally `source_item_id` (the source’s own id for the item). The `source_id` references the source registry. This allows you to group records by source and detect source-specific anomalies (e.g., duplicate rates, missing days, schema changes).

Canonical record identifiers. The canonical schema layer may assign a `canonical_record_id` that is unique across the entire dataset. Often this is simply the pair (`source_id`, `raw_item_id`) normalized into a string. This is useful for joins and tracing.

Dedup keys (pre-cluster keys). Before full clustering, it is useful to compute a `dedup_key` or set of keys that help identify likely duplicates. These keys must be computable from information available in the record at the time of ingestion and canonicalization. Typical candidates include:

- normalized headline hash (lowercase, stripped whitespace);
- normalized URL or canonical URL (if present);
- (source, title, published time rounded to minute) tuples;
- (entity mentions, title hash) tuples.

The key governance requirement is that dedup keys must not depend on future records or on later knowledge. They are functions of the record itself.

Why IDs and keys matter. In event-driven work, you will often need to answer questions like: “Which raw items contributed to this trade?” or “Did we count the same story three times?” Without stable IDs and dedup keys, you cannot trace and you cannot fix. Event research without traceability is narrative.

Timestamps: `event_time`, `first_seen_time`, `published_time`

Timestamps are the most sensitive fields in the entire event log. The canonical schema should store multiple timestamps, with explicit semantics, rather than collapsing to one ambiguous time.

published_time. This is the time provided by the source indicating publication. It is not necessarily the time you first saw the record. It is valuable as source metadata, but it is not sufficient for causality.

first_seen_time. This is the earliest time your system observed the record. In a live system, this is the ingestion timestamp. In historical datasets, you may have to approximate it. Governance demands you store both the raw ingestion time and your derived **first_seen_time** and record how it was derived. In backtests, **first_seen_time** is the primary admissibility time.

event_time. This is the time the underlying event occurred in the world, if known. For scheduled events, this can be the scheduled release time. For a corporate earnings release, it might equal published time. For a rumor, it might be unknown. The key point is that **event_time** is conceptually different from **first_seen_time**. Trading decisions should be based on when information became known, not necessarily when it occurred. However, **event_time** can be useful for classification and for aligning to economic narratives.

Time fields must include time zone semantics. All time fields should be stored in a canonical timezone (typically UTC) with explicit conversion artifacts. When sources provide local times, record the original timezone and the conversion method. In event logs, time zone errors are catastrophic because they can shift records across market sessions.

The derived known_time. The canonical schema should include a derived **known_time** that is the time your strategy treats as the earliest admissible time. Often **known_time** is set equal to **first_seen_time** plus a conservative buffer. The buffer accounts for processing latency and bar-resolution ambiguity. This field becomes the basis for causality tests and feature filtering.

Audit: timestamp distributions and anomalies. A governance-native pipeline produces a timestamp audit report: distributions of delays between published and first-seen, frequency of negative delays (should not happen), and counts of events near session boundaries. Such audits catch systematic misinterpretations of vendor fields early.

Instrument mapping and confidence

Once you have stable IDs and times, you must map events to instruments. The canonical schema should store mapping results explicitly, with confidence and method metadata.

Primary instrument mapping. At minimum, store a **primary_instrument** field: the ticker or instrument id the record is most directly about. This mapping can be derived from explicit ticker fields in the source, from entity dictionaries, or from simple string matching. Whatever method is used must be recorded.

Secondary and multi-instrument mapping. Many events reference multiple instruments. Store a list field **instruments** that contains all mapped instruments with weights or confidence

scores. Distinguish primary vs secondary. For example, an M&A article may mention acquirer and target; a macro article may map to rates, FX, and equity indices.

Confidence and provenance. Mapping confidence must be explicit. A binary mapping without confidence invites overreach. Confidence can be coarse (high/medium/low) or numerical. Alongside confidence, store a `mapping_method` (explicit ticker, alias match, dictionary match) and a `entity_dictionary_version`. This ensures that if mapping rules change, you can reproduce past results.

Why mapping belongs in the event log. Mapping is not a modeling step; it is part of the data definition. If you change mapping rules, you have changed the dataset. Therefore mapping outputs belong in the canonical event ledger as a versioned derived layer, not in ephemeral code state.

15.6.3 Normalization: time zones, calendars, and market sessions

Normalization is where you reconcile the information clock with the market clock. This step is conceptually simple but operationally dangerous: the most common hidden bug is a record that is actually after-hours being treated as during-session, or vice versa.

Time zone normalization. Convert all times to UTC (or a chosen canonical timezone) and store original time zone metadata. Maintain a conversion table and a test suite. Daylight saving time transitions are notorious sources of off-by-one-hour errors. In governance terms: if the conversion rule changes, version it.

Market calendars. A strategy must know which days are trading days, which are holidays, and what the session hours are. For global assets, there may be multiple sessions. The event log should therefore include session classification fields:

- `session_date`: the market date the event is associated with for trading decisions;
- `session_flag`: pre-market, regular session, after-hours, closed.

These fields are derived from `known_time` and the instrument's exchange calendar.

Session mapping rules are strategy-dependent. An after-hours earnings release might be tradable at the next day's open. In that case, `session_date` should map the event to the next trading day. Alternatively, for an intraday strategy, you may map it differently. The event log should not hard-code one mapping unless the strategy is fixed. A governance-native approach is to record raw times and then produce a session-mapped view per strategy configuration. The mapping function is part of the run manifest.

Bar alignment and conservative conventions. If your backtest operates on bars (e.g., minute bars), you must decide how to handle events that arrive within a bar. The conservative convention is to only allow the event to influence decisions in the next bar. This avoids within-bar look-ahead. The normalization layer can compute a `bar_time` field (the first decision time at which the event is admissible) given the bar grid and the chosen convention. This field is extremely useful because it turns ambiguous real-time timestamps into discrete admissibility indices.

Normalization audits. As part of governance, produce audits: counts of events by session flag, distributions of events near open/close boundaries, and cross-checks that a record labeled after-hours does not influence same-day close decisions. These are the kinds of tests that catch the most damaging bugs.

15.6.4 Deduplication and clustering: when “same story” repeats

Deduplication is inevitable: the same story will appear multiple times. But deduplication is also dangerous because it tempts you to use future information to decide what counts as the same story. A governance-native event log treats deduplication as a derived, versioned, time-aware process.

Why duplicates matter. If you count duplicates as separate events, you create a signal that is partly “story repetition.” Repetition is correlated with significance and with volatility. Your model might appear to predict returns by trading on repetition intensity rather than content. That might even be tradable, but you must then admit what you are trading: attention and urgency, not sentiment. More often, duplication simply inflates feature values and creates unstable behavior when vendor duplication patterns change.

Levels of deduplication. It is useful to distinguish:

- **exact duplicates:** identical headlines or identical URLs;
- **near duplicates:** same story with small edits;
- **threaded updates:** same story evolving with new details.

Each level requires different methods. Exact duplicates can be removed with simple hashes. Near duplicates might use string similarity. Threads might require link metadata if provided.

Time-aware deduplication. The crucial constraint is that deduplication should not use information that would not have been available at the time. For example, you cannot cluster a morning headline with an afternoon update using the afternoon content if you are building a feature at noon. In a live system, you would not yet know the afternoon update exists. Therefore deduplication must be defined in one of two ways:

- **offline clustering for analysis, online filtering for trading:** build rich clusters offline, but for trading features, only collapse duplicates that are identifiable in real time (e.g., exact headline duplicates within a short window);

- **as-of clustering:** cluster using only records up to a cutoff time, producing clusters that evolve over time.

The second is more faithful but more complex. The first is often adequate for Chapter 15 if documented carefully.

Clustering keys and stability. A practical approach is to compute multiple dedup keys and use them in a priority order: exact URL, exact normalized headline, then near-duplicate similarity within a time window. The parameters (window size, similarity threshold) must be in a parameter registry. Each cluster assignment must be logged: which records were grouped, what key triggered grouping, and what the resulting canonical id is. Without this, a change in dedup rules can silently change your features.

Choosing a canonical representative. Once you cluster, you may choose a canonical representative record for the event: the earliest record in the cluster (first-seen) is often the right choice for trading because it is the first observable carrier. Later updates can be treated as new information arrivals within the same thread, which can be represented as additional records linked to the same canonical id. This preserves the temporal nature of information: a story can evolve, and each evolution is itself an event for trading purposes.

Deduplication artifacts. A governance-native pipeline produces artifacts: cluster size distributions, duplicate rates by source, and examples of large clusters. Large clusters often indicate either major stories (expected) or vendor duplication bugs (problem). These artifacts make deduplication auditable.

15.6.5 Artifact: event ledger and provenance manifest

All of the above steps produce the core artifact: the event ledger. The event ledger is the canonical, normalized, time-aware, instrument-mapped, deduplicated record of events that the strategy will use. But the ledger alone is not enough; it must be accompanied by a provenance manifest that binds it to a specific run.

The event ledger. The ledger is typically a table-like artifact with one row per canonical record (or per raw record, depending on design) and the canonical fields. It includes:

- identifiers: `canonical_record_id`, `source_id`, `raw_item_id`, `canonical_event_id` (if clustered);
- timestamps: `published_time`, `first_seen_time`, `known_time`, `event_time` (if available);
- text fields: raw headline, raw body (or references to raw storage), and stable metadata;
- mapping fields: instrument list, primary instrument, confidence scores;
- session fields: session flag, session date, bar admissibility index (if applicable);

- dedup fields: dedup keys, cluster id, cluster method.

The ledger should either store text directly or store pointers to raw storage (file paths, content hashes) so that the raw content can be retrieved. Storing pointers is often better for large text.

The provenance manifest. The manifest is a small but critical artifact that answers: “How was this ledger produced?” It should include:

- run id, timestamp, and deterministic seed (if any stochastic steps exist);
- source registry versions and endpoints used;
- ingestion window (start/end), snapshot times, and any backfill policies;
- schema version and canonicalization rules;
- deduplication parameters and methods;
- entity dictionary version and mapping rules;
- time zone conversion rules and market calendar versions;
- hashes of input raw files and output ledger files.

The manifest makes the ledger reproducible. Without it, you can still run a backtest today, but you cannot prove what you ran yesterday.

The ledger-manifest pair as the unit of research. In governance-native research, you do not say “I ran a sentiment strategy.” You say: “I ran a strategy on ledger L, produced by manifest M, with feature spec F and backtest config B.” That is the level of specificity required for institutional credibility. It is also the level required for scientific learning, because it allows you to isolate changes. If performance changes, you can ask: did the ledger change, did the feature spec change, or did the backtest change?

Minimal tests as gates. Finally, because the event ledger is the starting point, it must be protected by hard gates. A minimal gate suite includes:

- monotonicity: `first_seen_time` and `known_time` not after ingestion time;
- non-negativity: no negative delays between known time and first-seen;
- session consistency: after-hours events do not map to same-session decisions;
- dedup sanity: duplicate rates within expected bounds; no pathological clusters;
- mapping sanity: instruments exist in the tradable universe; confidence distributions stable;
- reproducibility: repeated runs on the same raw snapshot produce identical ledgers (hash match).

These gates are the operational form of the filtration constraint introduced earlier. They ensure that the event log is not merely a dataset but a defensible representation of information available to the strategy.

To summarize, building an event log is the governance-native starting point because it transforms messy, revision-prone, biased information sources into a time-aware artifact that can be used without self-deception. Raw ingestion must be immutable and append-only to preserve evidence. A canonical schema must standardize identifiers, timestamps, and mappings while preserving provenance. Normalization must reconcile information time with market sessions and bar grids. Deduplication must reduce repetition without importing future knowledge. The final event ledger must be accompanied by a provenance manifest that makes the entire process reproducible and auditable. With these artifacts in place, the rest of Chapter 15 becomes what it should be: feature engineering, evaluation, and careful claims about whether events and text add incremental predictive power beyond prices, under a standard that can survive scrutiny.

15.7 From Text to Numbers: Feature Engineering Without Magic

The phrase “text features” often signals a dangerous turn in a research conversation. It can mean anything from a careful, causal transformation of raw language into time-indexed variables, to a vague promise that an NLP model will “understand the news” and therefore forecast returns. This chapter rejects the magical version. In trading, text is not useful because it is poetic; it is useful because it is an information carrier, and information can be operationalized into measurable objects that enter a decision rule. The entire purpose of this section is to show how to do that operationalization transparently: start with minimal tokenization and normalization; build a lexicon baseline that can be audited; add a small number of conceptually clear feature families that capture metadata and attention; aggregate those features over time using causal kernels; and, throughout, treat leakage as the central adversary.

Two themes guide the design. First, the baseline should be implementable with the same discipline as any other feature engineering in the book: deterministic, parameterized, and logged. Second, the baseline should be intelligible to a risk committee. You should be able to explain what each number means and how it was produced from raw records and timestamps. If you cannot explain it, you cannot govern it, and you should not trade it.

The result is not intended to be the “best” NLP pipeline. It is intended to be the *minimum credible* pipeline that can serve as a foundation for later chapters where model capacity increases. If you cannot beat simple baselines, you should not escalate complexity. And if you cannot prove that your baselines are causal, you should not believe any performance at all.

15.7.1 Tokenization and normalization choices (minimal, transparent)

Tokenization is the act of turning raw text into a sequence of units (tokens) that can be counted or scored. In modern NLP, tokenization can be sophisticated (subword units, byte-pair encodings). In Chapter 15 we keep it deliberately simple because sophistication is a leakage amplifier: it

increases the number of moving parts, which increases the number of silent assumptions.

Define the text field and its “as-of” version. Before any tokenization, you must define which text you are tokenizing. Is it the headline, the body, or both? Is it the raw wire text or a vendor summary? In this book, the default is: *tokenize the headline first*, and only add body text if you can store as-of versions and defend timing. Headlines are short, often the earliest disseminated carrier, and less prone to later edits than full articles. The more content you include, the more revision risk you inherit.

The second required definition is the “as-of” version. You must ensure that the text you tokenize at decision time t is the text that existed at or before `known_time` $\leq t$. If your dataset only stores the final version, you should assume revision risk and adopt conservative lags or restrict to fields known to be stable.

Minimal normalization: lowercase, whitespace, punctuation policy. A simple normalization pipeline typically includes:

- convert to lowercase;
- normalize whitespace (collapse multiple spaces);
- handle punctuation in a consistent rule (either keep punctuation tokens or remove punctuation except apostrophes).

Each of these choices should be explicit and versioned. The most common mistake is to let a library’s default preprocessing quietly change behavior when updated. In governance-native design, preprocessing is part of your model specification.

Stopwords and stemming: default to no. In trading, stopword removal and stemming are not obviously beneficial. They can remove informative negations (“not”) or change the meaning of terms (“raise” vs “raised”). They also introduce additional choices. The default baseline is to not remove stopwords, and to not stem, unless you can demonstrate improved stability without leakage. Instead, focus on domain-specific lexicons and explicit handling of negation.

Numbers, tickers, and percentages. Financial text contains structured tokens: numbers (“10%”), tickers, and money amounts. A minimal tokenizer should preserve these as tokens rather than discarding them. You may optionally normalize numbers into buckets (e.g., replace all integers with `<NUM>`) to reduce sparsity, but you should keep percentage signs and magnitude words (“million”, “billion”) because they often indicate scale. Again, the baseline principle is transparency: if you bucket numbers, record the rule and accept the loss of precision as a deliberate choice.

Multiword expressions: conservative handling. Many finance-relevant phrases are multiword: “beats estimates”, “cuts guidance”, “raises outlook”, “share buyback”. A minimal approach is to treat tokens individually but include a small curated list of bigrams/trigrams that you detect with deterministic rules. Avoid automatic phrase mining at this stage, because it can incorporate future corpus statistics and drift.

Language and encoding. If you have multilingual sources, you must either restrict the pipeline to one language or maintain language-specific lexicons. Mixing languages without detection will produce nonsense features. A governance-native event ledger should include a language field (even if inferred crudely) so that feature extraction can be conditioned on it.

The message is: tokenization is not where you create alpha. Tokenization is where you prevent fragility. Keep it simple, deterministic, and logged.

15.7.2 Dictionary / lexicon sentiment (baseline)

Lexicon sentiment is the appropriate baseline in this chapter for two reasons. First, it is transparent: you can inspect the dictionary and understand why a score is positive or negative. Second, it is robust to small dataset changes: unlike a trained model, it does not re-fit and therefore cannot accidentally learn leakage. This does not mean lexicon sentiment is accurate in the human sense. It means it is a clean research instrument.

Let $\mathcal{V}$ be a vocabulary of tokens, and let a lexicon be a function $w : \mathcal{V} \rightarrow \mathbb{R}$ assigning a polarity weight to certain tokens. Tokens not in the lexicon have weight zero. Given a tokenized document d with tokens $(x_1, \dots, x_n)$, a basic sentiment score is:

$$S(d) := \sum_{i=1}^n w(x_i). \quad (15.14)$$

We can then normalize:

$$\tilde{S}(d) := \frac{1}{n} \sum_{i=1}^n w(x_i), \quad (15.15)$$

to reduce dependence on length. Or use other scaling such as $\sqrt{n}$.

In finance, generic sentiment lexicons often fail because words like “liability”, “risk”, or “debt” are not negative in all contexts, and because domain-specific constructs matter (“beats”, “misses”, “downgrade”). Therefore the baseline should either use a finance-specific lexicon or a curated lexicon built for the strategy context. The crucial governance point is that the lexicon itself is an artifact with a version. If you change it, you have changed the model.

Score construction and calibration

A lexicon score is not yet a tradable signal. It is a raw measurement. To use it, you must decide how to calibrate it and how to map it into a feature that behaves consistently across time and sources.

Document-level score options. Beyond the raw sum, common constructions include:

- **polarity sum:** $S(d) = \sum w(x_i)$;
- **polarity difference:** $(\#pos - \#neg)$ for binary lexicons;
- **normalized polarity:** $S(d)/n$ or $S(d)/\sqrt{n}$;
- **clipped score:** $\text{clip}(S(d), -c, c)$ to prevent outliers dominating.

Clipping is especially useful because headlines can contain repeated strong words or punctuation patterns that produce extreme scores. Clipping makes the feature more robust.

Source calibration. Different sources write differently. Some use sensational language; some are dry. A raw sentiment score may therefore be source-biased. You can address this by computing, for each source s , a rolling mean and standard deviation of scores and standardizing:

$$Z(d) := \frac{S(d) - \mu_s}{\sigma_s + \epsilon}. \quad (15.16)$$

But this introduces a time-series estimation problem. To remain causal, μ_s and σ_s must be estimated using only past documents relative to the decision time. The simplest approach is an expanding window per source. You must also decide how quickly to update, and record window parameters in the registry.

Asset calibration. Similarly, some assets receive systematically different kinds of news (e.g., biotech vs utilities). If you intend to compare sentiment across assets (cross-sectional ranking), you may want asset-level calibration. But asset-level calibration can be fragile because many assets have sparse news. A safe baseline is to calibrate by source and category rather than by asset, unless you have sufficient volume.

Category calibration and the surprise dimension. Many events have categories (earnings, macro, legal). The meaning of “negative” words differs by category. A “lawsuit” story is usually negative; an “acquisition” story is ambiguous. Therefore, if you have reliable causal categories (derived from the text, not from hindsight tags), you can calibrate within category. Alternatively, you can compute multiple scores: a generic polarity score and a category-specific score.

Mapping score to signal. To trade, you typically map document scores to time-indexed features. If an event record e has text d_e , define $z_e := Z(d_e)$ and then aggregate over time (discussed later). The important point is that you should not treat $Z(d)$ as directly predictive; you treat it as a noisy proxy for surprise and directionality that must be tested.

Negation, intensifiers, and domain-specific vocabulary

Naive lexicon sentiment breaks first on negation and intensifiers. “Not good” is not positive. “Very strong” is stronger than “strong.” In finance, domain vocabulary also matters: “beats estimates” is often positive even if the headline contains neutral words.

Negation handling. A transparent baseline approach is windowed negation: if a negation token (e.g., “not”, “no”, “never”, “without”) occurs, flip or dampen the polarity of the next k tokens until a punctuation boundary. Formally, for token x_i , define a negation indicator $N_i \in \{0, 1\}$ that is 1 if within a negation scope, and then define an adjusted weight:

$$w'(x_i) := (1 - 2N_i) w(x_i), \quad (15.17)$$

which flips sign under negation. Or a dampening rule:

$$w'(x_i) := (1 - \alpha N_i) w(x_i), \quad (15.18)$$

with $\alpha \in (0, 1]$. The window size k and punctuation boundary rules are part of the parameter registry. The point is not perfect linguistics; it is to avoid the most obvious polarity mistakes.

Intensifiers and diminishers. Similarly, define a set of intensifier tokens (“very”, “strongly”, “significantly”) and diminishers (“slightly”, “somewhat”). Apply a multiplier to the polarity weight of nearby tokens. For example, if token x_i is in the scope of an intensifier, set $w''(x_i) = (1 + \beta)w'(x_i)$. Again, keep it simple and deterministic.

Domain-specific phrases. Financial headlines are full of phrases whose meaning is not the sum of individual word polarities. “Beats estimates” might include words that are not in a generic lexicon. Therefore a baseline should include explicit phrase detectors for a small set of high-value phrases:

- beats/misses expectations;
- raises/cuts guidance/outlook;
- upgrade/downgrade;
- initiates coverage with buy/sell;
- files for bankruptcy / restructuring;
- approves buyback / dividend increase.

These detectors can set additional features rather than modifying the generic sentiment score. For example, define binary flags $b_{\text{beat}}(d)$, $b_{\text{miss}}(d)$, etc. This is a way to capture domain meaning without pretending to solve language.

Ambiguity and modality. Words like “may”, “could”, “reportedly” indicate uncertainty. In finance, uncertainty can be predictive of volatility and risk premia even if direction is unclear. Therefore a baseline can include an “uncertainty score” based on a lexicon of modal words. This becomes important later when event signals are used for risk throttling rather than directional bets.

15.7.3 Simple embedding surrogates within scope (conceptual placeholders)

The book postpones deep NLP models to Chapter 13. Nonetheless, it is useful to introduce the idea of semantic representation without implementing heavy machinery in Chapter 15. The purpose is to define placeholders: feature families that capture more than polarity, but remain transparent.

Bag-of-words and hashed counts (conceptual). A simple representation is a bag-of-words vector: counts of tokens. In practice, the vocabulary can be large, and full vectors are unwieldy. One can use hashing tricks to project tokens into a fixed-dimensional vector. However, even hashing introduces parameters and potential drift. Within Chapter 15, we treat this as conceptual: the point is that you can represent text as counts and then reduce dimensionality, but you must treat the representation as a versioned artifact.

Topic proxies via curated keyword sets. Instead of unsupervised topic models, use curated keyword sets for topics: litigation, M&A, macro, guidance, liquidity, regulatory. For each set, compute counts or presence flags. This creates a low-dimensional embedding surrogate: a vector of topic intensities. It is crude, but it is interpretable and stable. It also avoids the risk that a learned topic model inadvertently uses future corpus structure in a way that is hard to audit.

Similarity to prototypes. Another simple surrogate is to define prototype texts (or prototype keyword lists) for categories and compute similarity. For example, define a prototype vector for “downgrade” by the presence of certain tokens, then compute cosine similarity between the document’s token-count vector and the prototype. This yields a continuous score that behaves like an embedding similarity, but remains transparent.

These placeholders matter for pedagogy: they show how one can move beyond polarity without jumping to black-box models. In later chapters, one can replace these surrogates with learned embeddings while keeping the pipeline and governance structure unchanged.

15.7.4 Event metadata features: source, length, novelty, and repetition

In event-driven trading, metadata often matters as much as content. This is not because metadata is magical; it is because metadata captures the mechanics of information dissemination. Many empirical “news effects” are actually attention and urgency effects. A governance-native approach does not ignore this; it measures it explicitly, so you can distinguish content-driven signals from attention-driven proxies.

Source features. Source identity is itself a feature. Some sources break news earlier; some amplify; some are low quality. At minimum, include:

- source id (one-hot or grouped into tiers);
- source credibility tier (curated, versioned);
- historical hit rate of the source (estimated causally).

Be careful: if you estimate hit rate using future outcomes, you can leak. Source quality measures should be based on past performance only.

Length features. Headline length and article length can proxy for seriousness or complexity. A short “breaking” headline may indicate an early stage of dissemination. A long article may indicate analysis rather than new information. Length features include token count, character count, and the presence of numbers. These features are cheap and often robust.

Novelty features. Novelty measures whether a story contains information that is new relative to recent history. A simple novelty proxy is: how similar is this headline to headlines in the recent past for the same asset? High similarity indicates repetition; low similarity indicates novelty. Another proxy is first-mention flags: has this keyword been seen in the last W days? Novelty is valuable because first mentions often move markets more than repeats, and because repetition can create false signals if counted as separate events.

Repetition and update intensity. Repetition is not always noise. Many repeats indicate evolving stories or increasing importance. Therefore include explicit repetition features: number of related headlines in the last hour/day, number of sources reporting the story, number of updates in a thread. But again, thread detection must be causal. Within Chapter 15, conservative repetition features might be based on exact headline duplicates and simple keyword overlap within short windows.

Timing and session features. Metadata includes time-of-day and session flags: pre-market, during session, after hours. The same headline can have different impact depending on when it arrives. Moreover, for daily strategies, after-hours news is not tradable until next open, so session features help interpret half-life.

These metadata features are also useful as controls. If a content-based signal appears predictive only because it correlates with high-intensity sources or with after-hours timing, you want to know. Therefore include metadata explicitly rather than letting the model learn it implicitly.

15.7.5 Temporal aggregation features: counts, novelty-weighted counts, decayed sums

A single headline is rarely the trading signal. The trading signal is usually a time-indexed summary of recent event flow. This is where event-driven feature engineering looks like time-series feature engineering, but on the information clock.

Let t_k be decision times. Let $\mathcal{E}_a(t_k)$ be the set of events for asset a with `known_time` $\leq t_k$. For each event e , define scalar extracted values: polarity z_e , novelty n_e , source weight q_e , uncertainty u_e , etc. Temporal aggregation builds features such as:

Counts in windows.

$$C_a(t_k; W) := \sum_{e \in \mathcal{E}_a(t_k)} \mathbf{1}\{0 \leq t_k - \tau_e \leq W\}. \quad (15.19)$$

Counts measure attention intensity.

Novelty-weighted counts.

$$NC_a(t_k; W) := \sum_{e \in \mathcal{E}_a(t_k)} n_e \cdot \mathbf{1}\{0 \leq t_k - \tau_e \leq W\}. \quad (15.20)$$

This discounts repeated stories.

Decayed sentiment sums.

$$S_a(t_k; \lambda) := \sum_{e \in \mathcal{E}_a(t_k)} z_e \exp(-\lambda(t_k - \tau_e)). \quad (15.21)$$

This encodes half-life explicitly. The parameter λ is part of the registry and can be chosen based on event studies.

Source-weighted decayed sums.

$$QS_a(t_k; \lambda) := \sum_{e \in \mathcal{E}_a(t_k)} q_e z_e \exp(-\lambda(t_k - \tau_e)). \quad (15.22)$$

This makes source credibility explicit.

Separate positive and negative flows. Sometimes net sentiment cancels, but positive and negative intensities matter. Define:

$$S_a^+(t_k) := \sum z_e^+ K(\Delta), \quad S_a^-(t_k) := \sum |z_e^-| K(\Delta), \quad (15.23)$$

where $z_e^+ = \max(z_e, 0)$ and $z_e^- = \min(z_e, 0)$. This yields an asymmetry-aware representation.

Event-type-specific flows. If you have causal event type proxies, aggregate separately per type. Earnings news should not be mixed with litigation news without at least separate features.

Aggregation as a reproducible transform. The governance-native requirement is that aggregation parameters (window sizes W , decay λ , bar alignment rules) are in a registry and logged. Aggregation must be implemented causally: for each t_k , only include events with `known_time` $\leq t_k$. This is where many pipelines fail: they aggregate events over a calendar day and then join to daily returns, inadvertently including events that arrived after the decision time.

15.7.6 Leakage traps in feature engineering

Feature engineering is where event-driven research most often collapses, because leakage can enter without obvious symptoms. The strategy still “works” in backtest and might even pass superficial checks. Therefore leakage traps must be treated explicitly, and the pipeline must include hard gates and tests.

Using future revisions, “final” headlines, and post-event summaries

The most common leakage trap is the use of revised content. If an article is corrected, the final version may include information that was not present initially. Vendor summaries often

incorporate later knowledge, including market reaction. Final headlines may be rewritten for clarity. If you compute sentiment on final text, you can unintentionally include future facts.

The governance-native rule is: only use as-of text. Practically, this means storing raw versions and update history. If you cannot, then you must adopt conservative rules such as:

- restrict to fields known to be stable (first headline only);
- delay use of text by a buffer long enough that revisions are unlikely to matter;
- exclude vendor summaries entirely from trading features.

You should also implement a revision audit: sample records where the text changed and measure how often the sentiment score changes. If changes are common, revision risk is high.

Using post-close articles for pre-close decisions

Session boundary leakage is subtle because it often looks like harmless daily aggregation. Suppose you trade at the close. If you construct a daily sentiment feature by summing all articles with date stamp equal to that calendar day, you will include after-hours articles that were not known at the close. This produces a classic “news predicts returns” illusion. It is especially dangerous because it can be strong: after-hours earnings releases can cause the next day’s move, and your feature will appear to predict it, even though you could not have known it at the close.

The cure is session-aware admissibility. Every event must be labeled with session flags and mapped to the first decision time it can influence. In daily systems, the simplest safe approach is:

- events during session influence same-day close decisions only if they arrive sufficiently before close (with a buffer);
- after-hours events influence next-day decisions.

This mapping must be explicit and tested. A simple causality gate is: no event with `known_time` after the decision time may contribute to features used at that decision time. This should be an assertion in code, not a comment in prose.

Cross-sectional leakage via contemporaneous ranking with late-arriving data

Cross-sectional strategies introduce a special leakage mode: contemporaneous ranking. Suppose at each day you compute a sentiment score per stock and rank stocks to form a long-short portfolio. If you compute sentiment scores using all articles from that day, you are again including some articles that arrived later for some stocks. Because ranking uses relative comparisons, even small timing differences can leak. The result is a portfolio that appears to anticipate which stocks will move, but it is using late-arriving information.

To prevent this, cross-sectional ranking must be done on a synchronized decision time. That means for each asset a , compute features using only events with `known_time` $\leq t_k$, where t_k is the global decision timestamp. Then rank using those features. If some assets have no news by t_k , that is part of reality; do not fill in later.

A second cross-sectional trap is universe conditioning: stocks that have news may be systematically different (more volatile, more liquid). If you form portfolios only among stocks with news, you are introducing a selection bias that can create spurious performance. Therefore you must define the trading universe independently of news presence and include explicit missingness features (e.g., zero news count) rather than filtering.

The lesson of this section is that “text to numbers” is not a glamorous modeling task; it is disciplined measurement under time. You choose minimal tokenization and normalization to reduce fragility. You build lexicon baselines because they are auditable and leakage-resistant. You add metadata features because dissemination mechanics matter. You aggregate features over time using causal windows and kernels to encode half-life and attention. And you treat leakage as the primary threat, with explicit rules against revised content, session misalignment, and cross-sectional timing errors. Once these foundations exist, more sophisticated models can be introduced later without turning the pipeline into a black box. Without these foundations, sophistication merely gives you a more expensive way to learn the answer key.

15.8 Event Study Mechanics for Trading

Event studies are often presented in finance as a quasi-ceremonial ritual: align returns around an event, average across many events, produce a plot, and declare that something “has an effect.” In trading research, that ritual is not enough, and in many cases it is actively misleading. The trading question is not whether an event moves prices in some averaged sense. The trading question is whether there exists a *tradable* response: a predictable component of the price path that remains after you respect (i) what you knew at the time, (ii) when you could trade, (iii) how fast the effect decays, and (iv) how much of the observed move is simply the market moving or the sector moving.

This section treats event studies as engineering diagnostics rather than as academic ornament. A well-designed event study should answer five practical questions:

1. Did my pipeline accidentally leak (is there “impact” before the event is known)?
2. How quickly does the market react after the event becomes observable (half-life)?
3. Does the effect exist at the horizon and frequency at which I can trade?
4. Is the effect incremental beyond broad market and sector movements?
5. Is the effect concentrated in a subset of events (heterogeneity), or is it robust?

If an event study cannot answer those questions, it is decorative. If it can, it becomes one of the most valuable tools in event-driven trading research: it tells you whether you are chasing a mirage, and it tells you what time scales and feature constructions are even worth attempting.

15.8.1 Aligning returns to the decision timestamp

The first mistake researchers make is aligning returns to the wrong timestamp. In event-driven trading, there are at least three relevant times: the underlying event time, the publication

time, and the `first_seen` or `known_time`. Trading decisions must be aligned to the time the information became observable to the strategy, and returns must be aligned to the time the strategy could act.

Let t denote a decision timestamp on the market clock (for example, the close, or a minute-bar timestamp). For an event e , let τ_e denote the *known time* (the earliest time the strategy could have observed the event, possibly with a conservative buffer). The central alignment decision is: what is the first decision time after τ_e at which the event can influence a trade? Call that decision time t_e :

$$t_e := \min\{t_k \in \mathcal{T}_D : t_k \geq \tau_e\}. \quad (15.24)$$

If your backtest uses a conservative within-bar rule (recommended), you might instead define:

$$t_e := \min\{t_k \in \mathcal{T}_D : t_k > \tau_e\}, \quad (15.25)$$

meaning the event only influences the next bar decision, not the current bar. The choice depends on your assumed latency and bar resolution, but the key is that it must be explicit.

Once t_e is defined, an event study for trading should align returns relative to t_e , not relative to the vendor's published time or the latent event time. Define event-time index $\ell \in \mathbb{Z}$ measured in decision steps. For example, $\ell = 0$ denotes the return interval immediately after decision time t_e (depending on your return definition), $\ell = -1$ denotes the interval immediately before, and so on.

If returns are defined as $r_a(t_k) = \log P_a(t_k) - \log P_a(t_{k-1})$, then the event-time returns are:

$$r_{a,e}(\ell) := r_a(t_{e,\ell}), \quad \text{where } t_{e,\ell} := t_e + \ell. \quad (15.26)$$

This notation simply re-indexes the return series around the event's decision-aligned anchor.

Why does this matter so much? Because trading action depends on this alignment. If you align incorrectly, you can produce an event study with strong “post-event” returns that in reality occurred before you could trade. Or you can produce pre-event drifts that are actually the market responding to leaks or earlier dissemination. In both cases, the event study becomes a leakage detector. A clean pipeline should show little systematic effect in pre-event windows unless there is genuine anticipation (e.g., scheduled events) or leakage in the real world. If you see strong pre-event effects for unscheduled events, your pipeline is likely non-causal.

Finally, alignment must respect sessions. If t_e falls outside trading hours, the first actionable decision time is the next session open (or whatever rule you use). Therefore, event studies should be stratified by session category: pre-market, regular session, after-hours. Combining them without stratification can smear effects and produce misleading averages.

15.8.2 Pre-event vs post-event windows

Once the anchor time is correct, the next design choice is the window structure. Trading event studies are not primarily about long-term cumulative abnormal returns; they are about the shape of impact over feasible horizons. Window choice should therefore be guided by trading frequency and half-life hypotheses.

Pre-event windows as leakage and anticipation diagnostics. Define a pre-event window of length L_{pre} decision steps:

$$\ell \in \{-L_{\text{pre}}, \dots, -1\}. \quad (15.27)$$

For truly unscheduled events with no systematic anticipation, the average pre-event return conditional on the event classification should be near zero, aside from noise and general market drift. If you see a consistent pre-event pattern, interpret it carefully:

- It could be **pipeline leakage** (using a time field that is too early).
- It could be **real information leakage** in markets (rumors disseminated earlier).
- It could be **anticipation** (for scheduled events, markets position ahead).

The key trading use of the pre-window is not to claim predictability but to test causality. If your event study shows that “positive news” is preceded by positive returns, you must ask whether you simply labeled events based on market reaction or used post hoc tags. This is why the chapter frames event studies as governance tools.

Post-event windows as half-life measurement. Define a post-event window of length L_{post} :

$$\ell \in \{0, 1, \dots, L_{\text{post}}\}. \quad (15.28)$$

Here, the goal is to measure the path of returns after the event becomes tradable. A simple diagnostic is the cumulative return curve:

$$CR_{a,e}(\ell) := \sum_{j=0}^{\ell} r_{a,e}(j). \quad (15.29)$$

Averaging $CR_{a,e}(\ell)$ across events yields an average impact curve. For trading, the shape matters more than the endpoint:

- A sharp jump at $\ell = 0$ with flat thereafter suggests immediate incorporation (edge half-life is shorter than your decision step).
- A gradual drift over many ℓ suggests slow diffusion (potentially tradable at your frequency).
- A reversal suggests overreaction and mean reversion (could imply contrarian post-event strategies).

Window length should be chosen to capture plausible half-lives. For daily strategies, post windows of 1–20 days are common. For intraday strategies, post windows may be minutes to hours. But within Chapter 15’s scope, the key is to connect window choice to decision frequency and to state clearly what is being tested.

Multiple horizons and regime conditioning. A single post window can hide state dependence. An event might cause immediate jumps in high-vol regimes and slow drift in calm regimes. Therefore, when possible, stratify event studies by regime indicators (from Chapter 14)

or by volatility states. This does not require building a full model; it is a descriptive diagnostic that prepares the ground for later integration.

15.8.3 Confounding controls: market moves and sector moves (conceptual)

A naive event study that looks at raw returns will often find “effects” that are actually market-wide or sector-wide moves coincident with the event timing. In trading, this matters because you want incremental signal. If your event strategy simply loads on the market, you will confuse beta with alpha.

Within this chapter’s scope, we treat confounding controls conceptually, focusing on the logic rather than on committing to a specific factor model (cross-sectional factor modeling is developed more fully in Chapter 09). The key idea is to construct *excess returns* relative to a benchmark that captures broad moves.

Market-adjusted returns. Let $r_m(t)$ denote the return of a market benchmark (e.g., an index future or ETF). Define market-adjusted return:

$$\tilde{r}_a(t) := r_a(t) - \beta_a r_m(t), \quad (15.30)$$

where β_a is an estimated sensitivity. Even a crude $\beta_a = 1$ can be informative as a diagnostic. The governance requirement is that β_a be estimated causally (using only past data) if used in a backtest. For an event study plot used as a diagnostic, one may use fixed betas with clear disclaimers. The purpose is to see whether the apparent event effect survives demeaning by market moves.

Sector-adjusted returns. Similarly, let $r_s(t)$ denote a sector benchmark relevant to asset a (e.g., a sector ETF). Define:

$$\tilde{r}_a(t) := r_a(t) - \beta_{a,s} r_s(t). \quad (15.31)$$

This is especially important for corporate events where sector news may coincide. If an earnings event for one firm occurs on a day when the whole sector rallies, you do not want to attribute that to the firm’s event without adjustment.

Event-day confounds and clustering. Events cluster in time. Macro releases affect many assets simultaneously. Earnings seasons create correlated corporate events. When events cluster, naive averaging can misattribute macro-driven moves to corporate sentiment. A practical control is to exclude days with major scheduled macro events when studying corporate event impacts, or to include macro event indicators as conditioning variables. Within Chapter 15, the key message is methodological: do not interpret raw average effects without asking what else was happening.

Why controls are “conceptual” here. A full confound-adjusted event study is close to a factor model and portfolio attribution problem. That is why cross-sectional factor modeling is treated explicitly in Chapter 09 and portfolio construction in Chapter 16. Chapter 15’s role is to

ensure you think like a trader: if the effect disappears after a simple market/sector adjustment, it is not a robust event signal; it is a beta proxy.

15.8.4 Heterogeneity: why average effects lie

Averaging across events is seductive because it produces smooth curves. It is also dangerous because event effects are heterogeneous. Two events that both score “positive” in sentiment can have opposite market impacts depending on context. For example, “raises guidance” might be positive, but if the stock was priced for even more, the reaction can be negative. A rumor might be positive if credible, negative if seen as desperate. A central bank hike might be positive if it signals inflation control, negative if it signals policy error. The average curve therefore often lies by implying a representative effect that applies to most events when in fact it applies to a minority.

There are several sources of heterogeneity that matter for trading:

Surprise vs polarity. A lexicon sentiment score measures polarity in language, not surprise relative to expectations. Markets move on surprise. Therefore, high-sentiment events that are fully anticipated can have little effect, while neutrally worded but surprising disclosures can have large effects. If you average by sentiment alone, you can dilute true signal. A more useful stratification is by a proxy for surprise: for scheduled events, realized minus consensus; for earnings, beat/miss magnitude; for unscheduled events, novelty measures.

Credibility and source. The same content from different sources can have different impacts. A wire headline can move markets; a blog post might not. Rumors from credible reporters can move markets; rumors from anonymous accounts can be ignored or fade. Therefore, stratify by source tier and by repetition across sources (multi-source confirmation).

Liquidity and constraints. Small, illiquid stocks can react differently because price impact and limits matter. Similarly, assets with short-sale constraints or high borrow costs can exhibit asymmetry. Event studies that mix liquid and illiquid names produce averages that reflect microstructure differences rather than informational content.

Regime dependence. Chapter 14 taught us that markets switch regimes. Event impact is often regime-dependent. In high-vol regimes, the market may overreact and reverse; in calm regimes, it may underreact and drift. Stratifying event studies by regime posteriors or volatility states often reveals that what looks like a weak average effect is actually a strong conditional effect.

Distributional effects and tails. Events often affect the tails more than the mean. A strategy might be profitable not because average post-event return is positive, but because downside tail risk is reduced when avoiding negative news, or because volatility increases in predictable ways. Therefore, event studies should examine not only mean cumulative returns but also quantiles, drawdowns, and variance changes. Even within Chapter 15, one can plot median impact curves alongside mean curves to see whether outliers dominate.

The practical trading conclusion is that event studies should not be used only to compute one number. They should be used to design stratifications and to decide which features matter. If a signal appears only in a subset, that is not necessarily bad; it may be the tradable subset. But you must know which subset and why.

15.8.5 Practical outputs: impact curves and half-life estimates

A trading-focused event study should end with outputs that directly inform strategy design. The most important outputs are (i) impact curves that visualize the shape of reaction, and (ii) half-life estimates that quantify how quickly any predictable component decays.

Impact curves. An impact curve is typically the average cumulative return around events, possibly adjusted for market/sector. The minimum set of curves that are genuinely useful includes:

- **pre/post cumulative return curve** for all events in a category;
- **curves stratified by score** (e.g., top vs bottom sentiment quantiles);
- **curves stratified by novelty or surprise proxies**;
- **curves stratified by regime** (high vs low volatility, or regime posteriors).

If the curves show pre-event movement that mirrors post-event movement, treat it as a causality warning. If curves show all impact in $\ell = 0$ with no drift, interpret it as an execution contest. If curves show drift over multiple steps, interpret it as potentially tradable at your decision frequency.

Half-life estimates (operational definition). In this chapter, half-life is not a theoretical parameter; it is an operational quantity: the time it takes for a fixed fraction of the total average impact to be realized. Suppose the average post-event cumulative curve for a class of events is $\overline{CR}(\ell)$ for $\ell \geq 0$. Define the long-horizon impact as $\overline{CR}(L_{\max})$ for some sufficiently large $L_{\max}$. Then define half-life $\ell_{1/2}$ as the smallest ℓ such that:

$$\overline{CR}(\ell_{1/2}) \geq \frac{1}{2} \overline{CR}(L_{\max}). \quad (15.32)$$

This definition is simple, transparent, and directly tied to strategy design. If $\ell_{1/2}$ is less than one decision step, your strategy cannot capture the effect at that frequency. If $\ell_{1/2}$ is several steps, you can design holding periods and decay kernels accordingly.

You can also define other quantiles (e.g., time to 80% of impact) and compare across regimes. The key is that half-life estimates should be robust to noise and should be reported with uncertainty (e.g., bootstrap confidence bands) if you are making strong claims. Within Chapter 15, the emphasis is on using half-life to choose feature aggregation kernels: if half-life is short, use short windows and strong decay; if half-life is long, use longer windows.

From diagnostics to feature specification. The event study outputs should feed directly into feature engineering decisions:

- Choose decay parameters λ for decayed sums based on half-life.
- Choose whether to separate positive and negative flows based on asymmetry in curves.
- Choose whether to condition on regime based on divergence in curves across regimes.
- Choose whether to trade directionally or use signals for risk throttling if mean effects are weak but volatility effects are strong.

This closes the loop: event studies are not a separate academic appendix; they are the mechanism by which you decide what your pipeline should compute and what your backtest should test.

In summary, event study mechanics for trading are a disciplined alignment and diagnostics problem. Align returns to the decision timestamp t_e derived from `known_time`, not to ambiguous vendor times. Use pre-event windows primarily to detect leakage and anticipation, and post-event windows to measure the shape and half-life of reaction in tradable units. Treat market and sector moves as confounds and use simple adjustments as sanity checks, recognizing that full attribution belongs to later chapters. Reject the comfort of averages by examining heterogeneity across surprise, source, liquidity, and regimes. And insist on practical outputs: impact curves and half-life estimates that directly inform feature aggregation and holding period choices. Done this way, event studies become one of the most powerful governance-native tools in event-driven trading, because they tell you not only whether a signal exists, but whether it exists in a form you can actually trade.

“`latex`

15.9 Backtesting Event-Driven Signals (Within Chapter 06 Discipline)

Backtesting event-driven signals is not a special case of backtesting. It is backtesting with an extra enemy: the temptation to treat a historical archive as if it were a real-time feed. Chapter 06 established the discipline that makes any backtest meaningful: explicit decision times, causal feature construction, deterministic simulation, and audit artifacts. Chapter 15 inherits that discipline and tightens it. In event-driven work, a “good” backtest is one that can defend the sentence: *at each decision time, the strategy used only the information that would have been available at that time, and it traded at prices that reflect what execution would have allowed.* Everything else is decoration.

The characteristic failure of event-driven backtests is not that they ignore transaction costs (though they often do). The failure is that they silently collapse the information clock into the market clock. They use a daily news feature computed over the entire day and then trade at the close as if all those articles were known. They use the final corrected headline rather than the first headline. They use curated event tags that were assigned after the fact. They join events to returns by calendar date rather than by decision timestamp. These failures are so

common because they are convenient: the code becomes simple and the backtest looks good. Chapter 06’s discipline exists precisely to prevent that kind of convenient self-deception.

This section presents a backtesting protocol for event-driven signals that stays strictly within Chapter 06 scope. That means we will not do full microstructure simulation, we will not pretend to capture millisecond latency, and we will not implement complex order types. We will, however, specify how to simulate decisions in time, how to construct signals causally from past events, how to incorporate conservative execution assumptions, and how to evaluate performance using diagnostics that are meaningful for event-driven strategies. We will also insist on baselines and placebos that are specifically designed to catch event-driven leakage. Finally, we will discuss robustness splits that reflect the unique non-stationarity of event data.

15.9.1 Decision-time simulation: “what was knowable when?”

The foundational object of the backtest is the decision-time simulator. Chapter 06 framed a backtest as a sequence of decision points $t_1, t_2, \dots$, at which the strategy observes an information set, computes a signal, chooses an action, and then realizes returns after execution. In event-driven trading, the only meaningful information set is an information set built on `known_time`. Therefore the simulator must be designed around the event ledger, not around a static feature table.

Define the decision grid and the trading calendar. Choose a decision grid $\mathcal{T}_D = \{t_k\}$ consistent with the strategy: daily close, daily open, hourly, minute bars, etc. The grid must be defined on a trading calendar that respects market holidays and sessions. If you trade multiple assets across markets, define per-asset calendars or a unified calendar with explicit session mapping rules. In Chapter 15 we usually assume a single market (e.g., U.S. equities) for pedagogical clarity, but the logic generalizes.

Define the admissible event set at each decision time. Given an event ledger with `known_time` τ_e , define the admissible set for asset a at time t_k :

$$\mathcal{E}_a(t_k) := \{e : a \in \mathcal{A}(e), \tau_e \leq t_k\}. \quad (15.33)$$

This is the operational form of the filtration constraint. The simulator should literally compute this set (or its equivalent in an efficient implementation) and ensure that no event with $\tau_e > t_k$ contributes to features at t_k .

Treat the event ledger as a stream, not as a static table. A common implementation mistake is to precompute daily features by grouping events by calendar date. That is exactly how you lose the information clock. The simulator should instead process events in order of `known_time`, updating per-asset state as time advances. Conceptually, think of a pointer that advances through the event stream as t_k advances, adding events to each asset’s “known events” buffer. At each t_k , features are computed from the buffer. This is how a live system behaves, and it is how a backtest must behave to be meaningful.

Log decisions and evidence. A governance-native simulator produces a decision trace: at each t_k , record the features used, the events that contributed, and the action chosen. For event-driven systems, it is especially important to record “why” a trade happened, because later you will need to inspect whether the strategy is trading on content or merely on attention spikes.

Causality gates as hard stops. Decision-time simulation should include assertions that fail fast if causality is violated. Examples include: for every feature computed at t_k , verify that all contributing events have $\tau_e \leq t_k$; for every trade executed at time $\tilde{t}_k$, verify that $\tilde{t}_k \geq t_k$; and verify that session mapping rules do not allow after-hours events to influence same-session trades. These are not optional checks; they are the firewall between research and illusion.

15.9.2 Signal construction at time t with only past events

Once the simulator defines what is known, the strategy must construct a signal from that known set. The key is to define signal construction as a deterministic function of the event stream history up to t_k .

Let $f_a(t_k)$ denote a vector of features for asset a at time t_k , constructed from past events (and possibly past prices). Let the signal be $s_a(t_k) = g(f_a(t_k))$, and the action be $u_a(t_k) = h(s_a(t_k), \text{risk state})$.

Event-only vs event-plus-price features. Within Chapter 15 scope, it is acceptable to include simple price context features alongside event features, such as recent volatility or trend state. This is not “cheating”; it reflects the reality that event impact depends on market state. However, to preserve interpretability, keep price context minimal and ensure it is computed causally from past prices only. The point of Chapter 15 is to evaluate whether event signals add incremental value beyond price-only baselines, so you should also run a price-only baseline strategy.

Temporal aggregation must be causal and aligned to decision times. As developed earlier, event features are often time-aggregated: counts, novelty-weighted counts, decayed sums of sentiment. In backtesting, the crucial detail is alignment. If t_k is the decision time, the feature uses events with $\tau_e \leq t_k$, and the decay kernel uses $\Delta = t_k - \tau_e$ measured in the same time units. If you use daily close decisions, it may be appropriate to measure Δ in days. If you use intraday bars, measure in minutes.

Stateful computation and rolling windows. Many event features rely on rolling windows (e.g., count of events in last 5 days). Implement them as stateful buffers that update as t_k advances. Avoid recomputing features from scratch using future knowledge. The stateful approach mirrors live operation and makes it easier to log provenance: the feature at t_k is a deterministic consequence of previous updates.

Cross-sectional signals require synchronized feature construction. If the signal is cross-sectional (ranking assets by sentiment), the backtest must compute $f_a(t_k)$ for each asset

using only events known by the common t_k , then rank. Do not rank using daily aggregated features that include late-arriving events. A robust implementation computes features on the same clock for all assets and leaves missingness as a feature rather than filtering it out.

Signal smoothing and hysteresis. Event signals can be spiky. In backtesting, smoothing (e.g., exponential smoothing of the signal) can reduce churn. But smoothing must be causal: it uses past signal values only. Hysteresis rules (e.g., do not flip position unless signal crosses a threshold) can also be useful. These choices belong to strategy specification and should be recorded in the parameter registry, because they strongly affect turnover and capacity.

Interpretability requirement. Because this is Chapter 15, we prefer signals that can be explained: “net negative sentiment flow over last 2 days exceeds threshold, scaled by source credibility.” If the signal becomes too complex, you are effectively doing machine learning without calling it that, and you will lose the pedagogical and governance advantages. Save high-capacity models for the appropriate chapters.

15.9.3 Execution assumptions: delays, fills, and latency (conceptual; full in Ch. 18)

Execution is where many event-driven edges die, because the market often reacts faster than you can trade. Chapter 18 will handle microstructure and transaction costs in detail. Here we remain within Chapter 06 discipline: we adopt explicit, conservative execution assumptions and we examine whether the strategy’s performance survives them.

Decision time vs execution time. Define decision time t_k and execution time $\tilde{t}_k \geq t_k$. For daily strategies, one might assume execution at next open or next close. For intraday bar strategies, assume execution at next bar open. The conservative choice is always to trade later rather than sooner, because earlier execution can silently assume impossible speed.

Latency buffer for event arrival. Even if an event is “known” at time τ_e , a live system needs time to ingest, parse, and compute features. The event ledger’s `known_time` should already include a buffer that represents this. In backtesting, you can test sensitivity by adding additional buffers: for example, compare performance when signals are delayed by 1 bar, 5 bars, or 1 day. If the strategy collapses with small delays, it is likely relying on the immediate reaction that belongs to microstructure competition.

Fill price conventions. Within Chapter 06 scope, you can use simple fill price conventions:

- execute at next bar open (for bar data);
- execute at volume-weighted average price over the next bar (conceptual, but still simplistic);
- execute at close with a lag (e.g., signal at close, trade at next close).

The key is to define the convention explicitly and apply it consistently.

Partial fills and slippage (conceptual). For large strategies, capacity and slippage matter. Chapter 15 does not model the full order book, but it should acknowledge that event-driven signals often concentrate trading in short windows, creating turnover spikes. A basic capacity proxy is to measure daily traded notional relative to average daily volume, and to penalize or cap trades when this ratio exceeds a threshold. Even if you do not model partial fills explicitly, you can treat a capacity cap as a governance constraint: “do not trade more than X% of ADV per day per asset.” This is conceptual here but becomes formal later.

Why explicit execution assumptions are essential. Because event-driven strategies are about timing, any backtest that does not specify execution is almost certainly overstating performance. In a governance-native workflow, you should report results under multiple execution assumptions (best-case and conservative-case) and treat the gap as an execution risk measure.

15.9.4 Evaluation metrics beyond Sharpe

Sharpe ratio is a blunt instrument. Event-driven strategies can have non-normal return distributions, clustered risks, and exposure spikes. They can also have performance that is concentrated around events, which Sharpe can obscure. Therefore evaluation must include event-aware metrics.

Hit rate near events

A fundamental event-aware metric is hit rate conditional on event activity. Define an “event active” indicator for asset a at time t_k :

$$I_a(t_k) := \mathbf{1}\{C_a(t_k; W) > 0\}, \quad (15.34)$$

where $C_a(t_k; W)$ is a count of events in a recent window. Then define hit rate near events as the fraction of times the strategy’s position sign matches the sign of subsequent return, conditional on $I_a(t_k) = 1$. For a simple directional position $u_a(t_k) \in \{-1, 0, +1\}$:

$$\text{HitRate}_{\text{event}} := \frac{\sum_{k,a} \mathbf{1}\{I_a(t_k) = 1\} \mathbf{1}\{u_a(t_k) \cdot r_a(t_{k+1}) > 0\}}{\sum_{k,a} \mathbf{1}\{I_a(t_k) = 1\}}. \quad (15.35)$$

This is not a substitute for P&L, but it reveals whether the strategy is directionally aligned when it matters. Compare it to hit rate in non-event periods. A strategy that only “works” in non-event periods is not an event strategy; it may be harvesting drift or carry.

One can also compute hit rates stratified by sentiment score quantiles or novelty quantiles. This connects directly to the pipeline: if the lexicon score has meaning, hit rate should improve in extreme-score bins.

Conditional returns and drawdowns around event clusters

Event-driven risk is clustered. Many bad outcomes occur when events cluster (earnings seasons, crisis weeks). Therefore evaluation should include conditional drawdown and tail metrics around event clusters.

Define an event intensity measure $C(t_k)$ for the portfolio (e.g., sum of per-asset event counts or the number of assets with events). Then define cluster periods where $C(t_k)$ exceeds a threshold (e.g., top decile). Compute performance metrics conditional on those periods:

- mean and median returns during clusters;
- worst drawdown during clusters;
- tail quantiles during clusters;
- variance and turnover during clusters.

This reveals whether the strategy is robust when activity is high, which is often when execution is hardest and correlations are elevated.

A related diagnostic is event-window P&L attribution: sum P&L contributions in windows around events (e.g., from t_e to $t_e + H$) and compare to P&L outside those windows. If most P&L comes from event windows, the strategy is truly event-driven. If not, event features may be incidental.

Turnover spikes and capacity concerns (conceptual)

Event-driven signals often generate rapid position changes, especially if the signal is based on news intensity or sentiment reversal. This creates turnover spikes. A backtest should measure:

$$\text{Turnover}(t_k) := \sum_a |u_a(t_k) - u_a(t_{k-1})|, \quad (15.36)$$

or a notional-weighted version. Then examine the relationship between turnover spikes and event intensity. If turnover spikes coincide with event clusters, capacity and slippage risk is elevated. Even within Chapter 15, you should plot turnover conditional on event intensity and compute the fraction of turnover occurring in high-intensity periods.

Capacity concerns can be approximated by comparing traded notional to volume. If you do not have volume, you can still use proxies (market cap tiers) to stratify performance. The goal is not perfect capacity estimation, but awareness: if the strategy's edge is concentrated in small names with low liquidity, it may not scale.

15.9.5 Baselines: naive rules and placebo events

A credible backtest requires baselines. In event-driven research, baselines serve two roles: they measure incremental value, and they detect leakage. Placebos are especially important because event data is so prone to subtle look-ahead.

Naive baselines. Examples include:

- **event count only:** trade volatility or risk exposure as a function of news intensity, ignoring sentiment;
- **polarity threshold:** go long on strongly positive headline sentiment, short on strongly negative, hold for fixed horizon;

- **contrarian reaction:** fade immediate post-event move (conceptual at daily frequency).

These baselines are simple enough that if your more complex signal does not beat them out-of-sample, you should question whether complexity is justified.

Placebo baselines. Placebos are strategies or transformations that should destroy true predictive information while preserving superficial structure. If your strategy still “works” under a placebo, it may be exploiting leakage or artifacts rather than genuine information.

Randomized timestamps placebo

The most powerful event-driven placebo is timestamp randomization. The idea is to keep the content and the asset mapping, but randomize `known_time` in a way that preserves some distributional properties while breaking causal alignment.

A simple version is to permute event times within each asset within a time bucket (e.g., within the same month). This preserves event counts and seasonal patterns but breaks alignment to actual returns. If performance remains, the strategy is likely trading on coverage or liquidity proxies, not on event timing. A more conservative version is to shift all event times forward by a fixed lag (e.g., +1 day) and see if performance improves (a sign your original pipeline used events too early) or collapses (a sign the effect is short-lived).

In governance terms, the timestamp placebo is a leakage detector: if the backtest depends on events being aligned exactly as in your dataset, and randomization kills performance, that is a good sign. If randomization does not kill performance, you must investigate what else the features capture.

Shuffled text placebo (within causal constraints)

Another placebo is text shuffling. Keep event times and asset mappings fixed, but shuffle the text content among events in a way that does not introduce future information. For example, within each day (or within each month) and within each source tier, randomly permute headlines across events. This preserves the distribution of sentiment scores over time and source, but breaks the association between a particular headline and a particular asset at that time.

If the strategy still performs, it suggests that the content itself is not the source of edge; rather, the strategy may be exploiting timing, volume of news, or other metadata. Again, this is not necessarily bad, but it changes the interpretation of the strategy. The goal is interpretability: know what you are trading.

The phrase “within causal constraints” matters because a shuffle that mixes across distant time periods can inadvertently change the availability distribution. Keep shuffles local in time so that you do not create a world where future language appears in earlier eras. Local shuffles also preserve regime and seasonal structure.

15.9.6 Robustness: time splits, event splits, and source splits

Robustness in event-driven trading is not only about time. It is also about sources and event types. Event datasets are non-stationary because vendors change, because language changes,

and because coverage shifts. Therefore, the backtest must include multiple splitting schemes.

Time splits. At minimum, use walk-forward or rolling splits as in Chapter 06: train/tune on past, evaluate on future. Avoid random shuffles. In event-driven work, it is also valuable to align splits with known vendor changes or major market regime changes, because these are stress tests for generalization.

Event splits (by type and by intensity). If you build a strategy on corporate events, test it on different corporate event subsets: earnings vs non-earnings, high-intensity vs low-intensity, high-novelty vs low-novelty. If performance is concentrated in one subset, that may be fine, but it should be reported as the true domain of the strategy. For research credibility, you should also test whether the strategy works in an earnings season and outside earnings season separately, because event density changes can drive performance and risk.

Source splits. Source dependence is one of the strongest failure modes. A strategy trained on one vendor may not generalize to another vendor’s writing style or latency. Therefore, perform source splits: build features using one subset of sources and test on another. Or train calibration on early sources and test on later sources. If the signal disappears when you remove one dominant source, you have learned source-specific artifacts. In practice, source splits can be implemented as:

- **leave-one-source-out tests:** remove each major source and rerun;
- **tier tests:** compare performance on high-credibility vs low-credibility sources;
- **vendor drift tests:** compare early vs late periods where source composition changed.

Cross-asset generalization. A final robustness axis is asset generalization: does the strategy work only on a small subset of highly covered large caps, or does it generalize? This is both a robustness and a capacity question. If the strategy only works on a handful of names, it might be a specialized trade rather than a general system. Report that honestly.

The overarching principle is that event-driven backtesting must be more skeptical than price-based backtesting, because the data source is more capable of embedding hindsight. Decision-time simulation is the anchor: at each t_k , build features from only events with $\tau_e \leq t_k$, and record the evidence. Signal construction must be a deterministic, time-aware transform of the event stream history. Execution assumptions must be explicit and conservative, with sensitivity to delays and bar alignment. Evaluation must include event-aware metrics that reveal whether the strategy performs when events happen and how it behaves under clusters and turnover spikes. Baselines and placebos must be treated as mandatory, not optional, because they detect leakage and clarify what the signal truly uses. And robustness must be assessed not only across time, but across event types and sources, because language and coverage are non-stationary. If you follow this protocol, then when a backtest looks good, you can at least believe it is not good for a trivial, embarrassing reason. That is the minimum standard required before you even begin to argue that events and text add edge. ““

15.10 Risk and Portfolio Integration of Event Signals (Preview Only)

Chapter 15 has insisted on a sober view of event-driven trading: events are exogenous information arrivals, their impact is fast and heterogeneous, and the primary technical challenge is not “understanding language” but building a causal, auditable pipeline. The natural next question is what to do with the resulting signals. If you can measure event intensity, polarity, novelty, and uncertainty, how do you translate that into positions, risk controls, and portfolio decisions?

This section offers only a preview because the full treatment belongs to Chapter 16. Portfolio construction is not an add-on; it is a discipline in its own right, involving objectives, constraints, correlation structure, turnover control, and risk budgeting. Chapter 15 is about building event signals and evaluating them under the backtesting discipline of Chapter 06. If we were to fully solve portfolio construction here, we would violate the book’s arc and blur the boundaries between “signal research” and “portfolio engineering.” Nonetheless, a preview is useful because it clarifies the intended role of event signals in a real system: often, their most reliable contribution is not pure direction, but *risk shaping*. Events change the conditional distribution of returns. They create clustering, volatility bursts, asymmetric tail risk, and correlation jumps. Event signals can therefore be used as throttles, gates, and context variables that modulate exposure rather than as standalone alpha engines.

We will make four points. First, event risk behaves like a regime process: it clusters, and it changes volatility and correlation in bursts. Second, because of that, position sizing should include explicit throttles around high-risk event windows. Third, event signals are best combined with price signals through simple gating and veto logic rather than through naive additive blending. Fourth, portfolio construction is deferred because it requires a careful treatment of constraints and risk budgets that cannot be done responsibly as an appendix.

15.10.1 Event risk is regime-like: clustering and volatility bursts

One of the most important practical lessons in event-driven trading is that event risk is not uniformly distributed through time. It clusters. Earnings seasons, macro weeks, crisis periods, and policy pivots all generate dense bursts of information arrivals. In these periods, markets do not merely become “more volatile” in a smooth way; they become volatile in a structured way: jumps are more frequent, gaps are larger, correlations rise, and liquidity conditions can deteriorate. This is why event risk is regime-like.

The connection to Chapter 14 is conceptual rather than methodological here. Chapter 14 introduced latent regimes as a way to model switching dynamics in return distributions. Event intensity is another lens that often aligns with regime behavior. In calm regimes, event flow is sparse and market reactions may diffuse slowly. In stressed regimes, event flow is dense and the market can reprice discontinuously. Even within a single market regime, event clusters can create local “micro-regimes” of elevated hazard. Therefore, event risk should be thought of as an additional state variable that conditions the distribution of returns:

$$r_{t+1} \mid \mathcal{F}_t \sim \mathcal{D}(\theta(\text{price state}_t, \text{event state}_t)), \quad (15.37)$$

where event state includes intensity measures (counts), novelty, and uncertainty proxies.

From a trading perspective, the most robust use of event state is often as a *hazard indicator*. It tells you that the probability of large moves has increased. This is valuable even when direction is unclear. In other words, event signals can be predictive of *risk* even when they are weak predictors of *mean returns*. A strategy that attempts to extract alpha from text polarity may be fragile; a strategy that uses event flow to reduce exposure ahead of known high-risk windows may be robust.

Clustering also matters because it changes the effective independence of observations. Many backtests implicitly assume that daily returns are roughly independent conditional on model features. In event clusters, they are not. Events arrive in correlated sequences, and positions may be repeatedly adjusted in response. This can produce overtrading and can amplify drawdowns if the strategy repeatedly “chases” the evolving narrative. Therefore, event clustering should be treated as a risk factor in its own right.

15.10.2 Position throttles around high-risk event windows

If event risk is regime-like, then position sizing should respond to it. The simplest response is a throttle: reduce exposure when event hazard is high, and allow higher exposure when hazard is low. This is not an exotic idea. Many discretionary traders do it intuitively: they reduce risk ahead of earnings, they avoid leverage around major macro releases, and they tighten stop policies during crisis weeks. The goal in a governance-native system is to formalize that intuition in a transparent rule.

Define event windows. A high-risk event window can be defined in several ways:

- **scheduled windows:** known ahead of time (earnings dates, central bank decisions);
- **detected windows:** inferred from event intensity (sudden spikes in counts or novelty-weighted counts);
- **hybrid:** scheduled events plus a detector for unscheduled clusters.

Within Chapter 15, we focus on the conceptual structure: define a function $H_a(t)$ that measures event hazard for asset a at time t , such as a decayed count of events or a binary flag for “earnings within 1 day.”

Throttle rule. A throttle is a multiplicative exposure factor $\alpha_a(t) \in [0, 1]$ that scales a base position:

$$u_a(t) = \alpha_a(t) u_a^{\text{base}}(t), \quad (15.38)$$

where $u_a^{\text{base}}(t)$ might be a price-signal-driven position (momentum, mean reversion, factor tilt) or a raw event-driven position. The throttle can be a simple piecewise rule:

$$\alpha_a(t) = \begin{cases} \alpha_{\text{low}}, & H_a(t) \geq h_{\text{high}}, \\ 1, & \text{otherwise.} \end{cases} \quad (15.39)$$

Or a smooth mapping, for example $\alpha_a(t) = \exp(-\gamma H_a(t))$. Smooth mappings avoid discontinuous trading and reduce turnover spikes.

Why throttles are often more reliable than directional bets. Directional event trading often requires being faster than the market, and it is sensitive to how sentiment is measured. Throttles do not require forecasting direction; they require forecasting hazard. Hazard forecasting can be easier because event clusters and scheduled events are observable, and because volatility tends to persist during information storms. Throttles therefore often have a better signal-to-noise ratio and are less sensitive to linguistic nuances.

Earnings and macro as archetypes. Earnings are a clean example: you can define a window around the announcement where gap risk is high. A throttle might reduce exposure to that stock ahead of the event, or might reduce leverage and widen risk limits for that period. Macro releases are similar but often cross-asset: reduce risk in rate-sensitive assets ahead of CPI, for example. The key is to make the rule explicit and to backtest it with the Chapter 06 discipline: when did you know the event time, and what was the earliest action time?

Throttles and asymmetric risk. Event windows often produce asymmetric tails. Negative surprises can cause larger moves than positive surprises, or vice versa depending on context and short constraints. Therefore, throttles can be asymmetric: reduce long exposure more than short exposure (or the reverse) when uncertainty is high. Such asymmetry should be justified by descriptive diagnostics, such as event study curves conditioned on polarity.

Governance and communication. Throttles also have a governance advantage: they are easy to explain. “We reduce exposure ahead of earnings to avoid gap risk” is more defensible than “our NLP model predicts the market will like this headline.” In institutional settings, explainability is not a luxury; it is part of risk acceptance.

15.10.3 Combining event signals with price signals

Event signals rarely live alone. Most practical systems combine them with price-based signals because prices embed the market’s consensus and because price signals provide continuity when news is absent. The challenge is to combine them without creating an unstable, overfit soup. Within Chapter 15, we preview three combination logics: gating, ensembling, and veto rules. These are conceptually simple and governance-friendly.

Gating, ensembling, and veto logic (conceptual)

Gating: event signals as “when to listen.” A gating rule uses event state to decide whether a price signal should be trusted. For example, a trend-following signal might work in normal conditions but fail during major news shocks when gaps dominate. A gate might reduce or turn off the trend signal when event hazard is high:

$$u_a(t) = \alpha(H_a(t)) u_a^{\text{price}}(t). \quad (15.40)$$

This is essentially the throttle discussed above, but interpreted as a gate on a price strategy. Conversely, one can gate the event signal using price context: only trade positive event sentiment if the price trend is not strongly negative, for example. This prevents fighting a strong price regime with fragile text interpretation.

Gating has a key advantage: it avoids requiring you to compare event and price signals on the same scale. It treats one as a context variable.

Ensembling: weighted combination with stability constraints. An ensemble combines signals additively:

$$u_a(t) = w_p u_a^{\text{price}}(t) + w_e u_a^{\text{event}}(t), \quad (15.41)$$

with weights w_p, w_e possibly time-varying. In a full portfolio chapter, weights would be optimized under risk constraints. Here we emphasize a governance-friendly version: fixed weights chosen by simple heuristics and stress-tested across time splits. If weights must vary, tie them to interpretable state variables such as volatility regime or event intensity, not to a high-dimensional learned function.

A stability constraint is crucial: prevent weights from changing too quickly, or you will introduce turnover spikes and overfit to short periods. For example, update weights only monthly, or smooth them with an exponential filter.

Veto logic: “do no harm” rules. One of the most effective and underappreciated integration patterns is veto logic. Instead of letting event sentiment drive positions, use it to *block* trades that violate common-sense risk rules. For example:

- do not initiate a long position if recent news sentiment is strongly negative and novel;
- do not increase leverage during a detected news storm;
- do not trade mean reversion signals immediately after major negative corporate events.

Veto logic is attractive because it is asymmetric: it reduces worst-case outcomes without demanding that event sentiment be a precise directional predictor. It also fits governance: veto rules can be documented as risk controls rather than as alpha claims.

The hierarchy principle. A useful conceptual hierarchy is:

1. price signals provide baseline exposure and continuity;
2. event hazard signals modulate risk (throttles/gates);
3. event polarity signals are used selectively (vetoes or conditional tilts) where event studies show slow diffusion.

This hierarchy matches empirical reality: hazard and session effects are often more robust than fine-grained sentiment direction.

Avoiding double counting. Finally, combining signals must avoid double counting information. If price has already moved in response to a headline, adding a sentiment signal at daily frequency may simply chase the move. Event studies help diagnose this: if impact is instantaneous, then polarity signals should not be used directionally at that frequency; use them as risk context instead.

15.10.4 Why portfolio construction is deferred to Chapter 16

At this point, it may feel like we are standing at the edge of portfolio construction and refusing to step in. That is intentional. The book’s structure is designed to prevent premature optimization and to keep each chapter honest about its scope.

There are three reasons portfolio construction is deferred.

First, portfolio construction changes the question. Up to now, the question has been: *does an event-driven signal contain incremental predictive information under causal backtesting discipline?* Portfolio construction changes that into: *given many signals, constraints, and costs, what is the best allocation and execution plan?* Those are not the same question. A weak signal can be valuable in a portfolio if it diversifies others. A strong signal can be harmful if it concentrates risk. Answering these requires a portfolio objective and a risk model, not just signal evaluation.

Second, portfolio construction is where hidden assumptions multiply. To construct a portfolio you must decide: long-only or long-short; leverage limits; risk budgets; correlation estimation; transaction cost models; turnover constraints; rebalance frequency. Each decision introduces degrees of freedom. If we introduced them here, we would create a risk of confusing event-signal value with portfolio-engineering choices. Chapter 16 treats these choices systematically, with explicit objective functions and constraints, and with governance artifacts that make decisions auditable.

Third, Chapter 16 provides the missing machinery. Portfolio integration properly belongs where we can discuss:

- translating signals into expected returns and confidence measures;
- risk models and covariance estimation;
- optimization under constraints;
- risk targeting and exposure caps;
- turnover penalties and transaction cost terms.

Those topics are out of scope here, and introducing them as “just a preview” would violate the no-fast-forward rule and degrade the clarity of the chapter arc.

Therefore the correct posture in Chapter 15 is modest: we preview integration patterns that are *structural* and *governance-friendly*—throttles, gates, vetoes—and we emphasize that event

signals are often most useful as risk context. Then we hand off to Chapter 16, where portfolio construction will formalize how signals become weights under constraints.

The core message to carry forward is simple. Event signals are not merely alpha candidates; they are state variables that describe the information environment. Because the information environment clusters and produces volatility bursts, event risk behaves like a regime process, and exposure should respond to it with explicit throttles around high-risk windows. When combining event and price signals, prefer integration patterns that preserve interpretability and stability: gating and veto logic first, cautious ensembling second, and only then more sophisticated blending. Finally, recognize that turning signals into portfolios is a full discipline that deserves its own chapter. Chapter 16 will provide the formal framework for that step. Chapter 15’s job is to ensure that when you arrive there, your event signals are causally defined, auditable, and honestly evaluated—so that portfolio construction is the enhancement of a real signal, not the optimization of a mirage.

15.11 Failure Modes and Anti-Patterns

Event-driven trading fails in ways that are both predictable and, embarrassingly often, repeatable. The patterns are so consistent that you can almost classify an event-driven backtest by how it fails: a mysterious pre-event “alpha” that vanishes live, a stunning earnings-season equity curve that flattens outside earnings season, a sentiment model that worked beautifully for two years and then quietly decays, a vendor feed update that changes category tags and breaks the strategy without throwing an error. None of these failures are exotic. They arise from the basic fact that text and events are not prices. They are mediated artifacts: curated, revised, distributed with delays, and embedded in shifting institutions (vendors, platforms, editorial policies) that evolve over time.

This section is deliberately blunt. Its purpose is not to list theoretical caveats but to inoculate you against the most common anti-patterns. In the book’s governance-native philosophy, a failure mode is not merely an implementation bug. It is a breach in your ability to defend the filtration constraint: “at decision time t , the strategy used only $\mathcal{F}_t$.” Most event-driven anti-patterns are therefore variations of one theme: you thought you were using information that was available, but you were actually using information that was *only available in hindsight*—or you were using a dataset whose coverage mechanism itself smuggled in selection effects.

We group the failure modes into six families: timestamp ambiguity, vendor selection bias, language drift, calendar overfitting, confirmation bias, and operational fragility. Each is common enough that if you build event-driven systems professionally, you will face all of them. The goal is to recognize them early and to build defenses that are structural rather than ad hoc.

15.11.1 Timestamp ambiguity and implicit look-ahead

The most dangerous anti-pattern in event-driven trading is to treat a timestamp as if it were unambiguous. In price data, timestamps usually correspond to a market mechanism: a bar close, a trade print, a quote snapshot. In news data, timestamps often correspond to human and vendor processes: publication, revision, redistribution, archival. Those processes do not align

cleanly with the market clock. If you do not explicitly define what time your strategy *knew* the information, you do not have a causal backtest.

The three clocks problem. Event-driven pipelines must distinguish at least three times: `event_time` (when the real-world event occurred), `published_time` (when a vendor says it was published), and `first_seen_time` (when your system observed it). Trading decisions should be anchored on `known_time`, usually derived from `first_seen_time` plus a conservative buffer. The failure mode is to collapse these into one “timestamp” and then join it to price bars.

Daily aggregation as a stealth leak. The classic implicit look-ahead is daily aggregation. A researcher groups all articles by calendar day, computes a sentiment score for the day, and trades at the close. This silently includes after-hours articles that were not available at the close, and it often includes late-day articles that arrived after the decision time. The backtest looks predictive because it is partially trading on tomorrow’s information. The fix is decision-time simulation: features must be computed at t using only events with `known_time` $\leq t$.

Within-bar ambiguity. Even if you move to intraday bars, you can still leak within-bar. If an article arrives at 10:05:30 and you use minute bars, the 10:05 bar may include price information that occurred after the article arrival. If you allow that bar’s return to be considered “post-event,” you can inadvertently give the strategy access to the market reaction before you could trade. A conservative rule is to allow events to affect the *next* bar’s decision. This costs you some edge in backtest, but it buys you credibility.

Revisions and corrected headlines. Timestamp ambiguity also hides in revisions. Many datasets store the final version of an article, with the original publication time. If you compute sentiment on the final text, you may be using information that was only added later. The backtest then exhibits “alpha” that is really editorial correction. The defense is as-of snapshots and raw append-only logs: preserve initial versions and update history.

Symptom and diagnosis. The cleanest diagnostic is the pre-event curve in an event study. If your event study shows systematic pre-event returns for unscheduled events, assume leakage until proven otherwise. Markets can leak, but your pipeline can leak more easily. Treat pre-event alpha as an alarm bell, not as a discovery.

15.11.2 Vendor selection bias and survivorship in coverage

Even if timestamps were perfect, event-driven datasets can still mislead because the dataset itself is a selection of what someone decided to cover. Coverage is not neutral. It is correlated with firm size, liquidity, volatility, sector, geography, and narrative popularity. This creates a survivorship-like bias: the events you see are the events that were recorded, and the events that were recorded are not a random sample of all information arrivals.

Coverage as a hidden factor. Suppose a model predicts returns better for large caps than for small caps. That might be a market truth, but it might also be a coverage truth: large caps have more headlines, more structured metadata, and cleaner mappings. A naive strategy might appear to work because it effectively trades on attention and liquidity: stocks with more news are different. If you do not explicitly model coverage, your event signal may actually be a proxy for these attributes.

Survivorship through significance. A particularly pernicious form of selection bias occurs in “event databases” that are curated after the fact: lists of important events, important headlines, or “key developments.” These datasets often include events precisely because they moved markets or because they turned out to matter. The mere presence of an event record then contains information about impact. A model trained on such data is training on a filtered universe where irrelevant events are missing. In live trading, you will face a flood of irrelevant items. Performance collapses because the training world was cleaner than reality.

Vendor drift as selection drift. Vendors change what they cover. They add new sources, change tagging policies, or expand into new geographies. Your dataset may show a dramatic increase in event counts in a period that coincides with a vendor expansion rather than with a true change in the world’s information rate. A strategy that performs in the later period may simply be benefiting from richer coverage. The defense is to log coverage statistics and to test source splits: does the strategy survive when a dominant source is removed?

The governance remedy. The governance-native remedy is twofold. First, treat coverage as a measured variable: event counts per asset per time, source mix, missingness, and changes through time are artifacts. Second, require that backtests be robust across source splits and across time periods with different coverage intensity. If performance depends on one vendor behavior, it is not a market effect; it is a vendor effect.

15.11.3 Language drift: models decay even if markets do not

A subtle but increasingly important failure mode is language drift: the mapping from words to meaning changes. This can happen because markets evolve (new products, new themes), because journalists evolve their phrasing, because corporate communications evolve their style, and because platforms change conventions (hashtags, memes, abbreviations). The result is that a model can decay even if the underlying market reaction function is stable.

Lexicons drift too. Even simple lexicon baselines drift. Words that were rare become common; words that were strongly negative become neutral (or vice versa). Consider the evolution of terms like “restructuring,” “AI,” or “quantitative easing” across eras. A static lexicon may mis-score modern headlines. If you recalibrate lexicons too aggressively, however, you risk fitting to noise or incorporating hindsight. Therefore lexicon updates must be versioned and evaluated with strict time splits.

Model retraining as a governance risk. High-capacity models (postponed to later chapters) amplify drift because they must be retrained. Retraining introduces the risk of inadvertently tuning to recent idiosyncrasies. Even in Chapter 15, if you calibrate sentiment scores (standardize by source, for example), you are already adapting to distribution changes. The adaptation must be causal and controlled: define refit cadence, use only past data, and log calibration parameters.

Semantic drift without vocabulary drift. Drift can occur even when the same words remain. “Inflation is transitory” meant one thing in early usage and later became a market joke. “Soft landing” can shift from optimism to skepticism. A model that treats phrases as static can be systematically wrong in new regimes. This is why event signals should often be used as risk context rather than as fine directional forecasts: the hazard component is more stable than the semantic nuance.

Defenses: monitoring and decay tests. A governance-native system treats drift as a measurable phenomenon. Monitor feature distributions through time, monitor the mapping from sentiment bins to subsequent returns in rolling windows, and include decay tests: does performance degrade smoothly or break abruptly? Abrupt breaks often signal feed changes; gradual decay often signals language drift or market adaptation. In either case, the system should have procedures for revalidation, not ad hoc patching.

15.11.4 Overfitting the calendar: “earnings season mirages”

Event-driven strategies are especially vulnerable to overfitting the calendar because many event types are seasonal and predictable in timing. Earnings seasons are the canonical example. If you backtest over a few years, you might find a strategy that performs brilliantly around earnings. This is seductive because it aligns with an intuitive narrative: earnings drive price moves. But the mirage is that many features and heuristics can accidentally latch onto earnings season structure without capturing true informational edge.

Seasonality masquerading as predictability. Suppose your strategy goes to cash when news intensity is high and increases exposure when intensity is low. During earnings season, intensity is high and volatility is high. Avoiding exposure might look brilliant in some years and terrible in others, depending on the direction of the market during earnings. The strategy is not predicting earnings outcomes; it is timing a seasonal volatility regime. That can be a legitimate risk strategy, but it is not the alpha claim many researchers think they are making.

Earnings as a regime shift in microstructure. Around earnings, spreads widen, gaps occur, and liquidity can change. A backtest that assumes frictionless execution at close can be unrealistically optimistic. Therefore, earnings-season performance that vanishes under conservative execution assumptions is often not a semantic failure but an execution failure. If your edge relies on being fast during earnings, it belongs in Chapter 18’s world, not in a daily event feature backtest.

Calendar feature leakage. Another anti-pattern is implicit leakage through calendar-derived features. If you include “days to earnings” or “earnings date” features, you must ensure that the earnings calendar was known at the time. Earnings dates change. Vendors update schedules. Using a final calendar retroactively can leak. The same applies to macro release calendars if revisions or reschedulings occurred. The defense is again as-of snapshots: calendars are data too.

Defenses: event splits and out-of-season tests. The simplest defense is to test performance outside earnings season. If the strategy only works during earnings, then it is an earnings strategy and should be evaluated as such, with execution realism appropriate to that environment. Also use event splits: compare performance on earnings-related events vs other corporate events. If “sentiment” works only on earnings, you may have a special-case system rather than a general event model. That is not a failure, but it is a different claim.

15.11.5 Confirmation bias: reading narratives into noise

Event-driven research is uniquely vulnerable to confirmation bias because the data is inherently narrative. A price series is abstract; a headline is interpretable. Humans are wired to make stories, and stories are addictive. This creates an anti-pattern where researchers look at a backtest result and then retrofit a narrative explanation: “the market overreacts to negative news,” “investors underprice uncertainty,” “optimism drifts.” Sometimes these narratives are true. Often they are just post hoc rationalizations.

The narrative trap. A common pattern is: (i) run many variants of a sentiment feature, (ii) find one with good performance, (iii) look at a few example trades with plausible headlines, (iv) declare understanding. The examples are not evidence; they are anecdotes selected by your own attention. The strategy may be driven by something else entirely (coverage, timing, sector exposure). Narratives are especially dangerous when you cannot articulate a falsifiable mechanism.

The illusion of interpretability. Lexicon models feel interpretable because you can read the words. But interpretability can be an illusion if the strategy is actually reacting to intensity spikes or source identity. The correct approach is to use placebos and controls. If shuffling text preserves performance, the narrative that “sentiment drives returns” is false, regardless of how plausible the examples look. If randomizing timestamps preserves performance, the narrative that “timing of news matters” is suspect. Placebos are the antidote to narrative seduction.

Over-claiming and selective reporting. Confirmation bias also appears in reporting: showing the best period, omitting drawdowns, ignoring failed splits. In governance-native research, this is addressed structurally: you define evaluation splits before looking at results, you log all runs, and you treat negative results as information. The artifact registry is not bureaucracy; it is a defense against your own cognitive bias.

A disciplined attitude. The disciplined stance is to treat event-driven signals as hypotheses that must survive adversarial testing. When you find an apparently good result, the immediate next step is not to celebrate; it is to try to kill it. If it survives timestamp placebos, text shuffles, source splits, and conservative execution assumptions, then you can begin to believe it. Narratives come last, and only as explanations consistent with survived tests.

15.11.6 Operational fragility: outages, duplicates, and feed changes

Finally, even a conceptually sound event-driven strategy can fail operationally. Event systems are pipelines. Pipelines break. And because event signals often trigger trading in short windows, operational failures can be amplified: missing an event feed for an hour can mean missing the only tradeable opportunities of the day, or worse, trading on incomplete information.

Outages and silent degradation. Feeds can go down. APIs can rate-limit. Authentication can expire. Vendors can deliver partial data without flagging it as partial. The most dangerous case is silent degradation: the system continues running but with missing events. If your strategy uses event absence as a signal (“no news”), then missing data is interpreted as “calm,” and the system may increase exposure precisely when it should not. Therefore, operational governance must include completeness checks: expected event counts by source, heartbeat signals, and alerts when volumes fall outside normal ranges.

Duplicates and burstiness. Conversely, feeds can produce duplicates or bursts due to vendor errors or reconnections. If your pipeline counts duplicates as separate events, it may generate extreme signals and overtrade. Deduplication must therefore be robust and conservative. Moreover, deduplication must be stable across feed changes: if the vendor changes URL formats or headline templates, naive hashing can break. A governance-native approach includes duplicate-rate monitoring and versioned dedup rules.

Schema and taxonomy changes. Vendors change schemas and taxonomies. A field that once meant “published time” might later mean “last updated.” A category label might be renamed. A source id might change. If your pipeline assumes fixed schema, it can misinterpret fields without raising errors, especially in dynamically typed environments. Therefore, schema validation is essential. At ingestion, validate that required fields exist and that their types are consistent. Record schema versions. When the vendor changes, treat it as a new source version, not as a silent continuation.

Entity dictionary drift and mapping failures. Entity resolution can break when tickers change, companies spin off subsidiaries, or new aliases appear. If mapping fails, events may be dropped or misassigned. This can bias backtests (dropping hard cases) and can destabilize live trading. Therefore, mapping coverage must be monitored: fraction of events mapped with high confidence, distribution of unmapped events, and changes through time. Updating dictionaries must be a governed process with versioning and rollback capability.

Runbooks and fail-safe modes. Operational robustness ultimately requires runbooks: what does the system do when the feed is down? The safest default is often to reduce risk, not to continue trading as if nothing happened. Event-driven systems can adopt a fail-safe mode: if ingestion completeness falls below a threshold, freeze new trades or reduce exposure. This is the operational analogue of the filtration constraint: if you cannot trust what you know, you should not pretend you know it.

These failure modes and anti-patterns share a common moral: event-driven trading is not primarily an NLP problem. It is an information-governance problem. Timestamp ambiguity creates implicit look-ahead unless you anchor decisions on `known_time` and simulate decision-time causality. Vendor selection bias and coverage survivorship can produce spurious predictability unless you measure coverage and test source robustness. Language drift causes model decay unless you monitor feature distributions and refit under strict governance. Calendar overfitting creates earnings-season mirages unless you test out-of-season and respect execution realism. Confirmation bias seduces researchers into narratives unless they use placebos and adversarial tests. Operational fragility breaks strategies in the real world unless ingestion is monitored, deduplication is robust, schemas are validated, and fail-safe modes exist. If you internalize these anti-patterns early, you do not merely avoid mistakes; you build systems that can be trusted. In event-driven trading, trust is the rarest alpha.

15.12 Governance-Native Checklist for Event-Driven & Sentiment Pipelines

Governance in event-driven trading is not a compliance afterthought; it is the only way to make the work intellectually honest. If you build features from text without being able to prove what was known when, you are not doing predictive modeling—you are doing historical reenactment with access to the script. If you cannot reproduce a run byte-for-byte, you cannot distinguish a real signal from accidental parameter drift. If you cannot trace a trade decision back to specific event records and feature values, you cannot debug failures or defend the strategy to an investment committee. And if you cannot show that your pipeline is robust to vendor drift, calendar artifacts, and placebo tests, you cannot tell whether you have learned the market or learned the dataset.

This checklist operationalizes the book’s governance-native stance for Chapter 15. It is intentionally concrete: it specifies what must be logged, what must be versioned, what must be tested, and what artifacts must exist at the end of a research run. The checklist is not meant to be aspirational. It is meant to define a minimum standard under which event-driven claims become credible enough to be discussed. Without these controls, the most common failure mode is not mild overfitting; it is catastrophic leakage that looks like alpha.

A useful way to interpret this checklist is as a chain of custody for information. In a price-based strategy, the chain of custody is relatively simple: a price series, a feature transform, a backtest, a report. In event-driven strategies, the chain is longer and more fragile: raw ingestion, as-of snapshots, canonical event ledger, tokenization/lexicon transforms, temporal aggregation, decision-time simulation, and execution assumptions. Each step can introduce

subtle hindsight if it is not governed. Therefore, governance-native event research treats each step as an artifact-producing stage with explicit versioning and hard gates.

The subsections below correspond to the core governance dimensions: determinism, data lineage, causality gates, parameter registry, audit artifacts, and a minimal test suite. They are written as prose rather than as a bullet-only compliance list because the goal is not to box-tick but to explain why each item exists and how it prevents the most damaging errors.

15.12.1 Determinism: seeds, configs, and run manifests

Determinism is the ability to re-run the same pipeline on the same inputs and obtain the same outputs. In event-driven research, determinism is more than convenience. It is a defense against self-deception. If results change from run to run, you can unconsciously select the run that “looks right” and then rationalize it. Determinism is therefore a governance control against confirmation bias as much as a software engineering best practice.

Seed everything that can be seeded. Even if Chapter 15 avoids high-capacity models, there are still stochastic elements that can appear: random splits for placebo tests, randomized timestamp shuffles, randomized text shuffles, bootstrap confidence bands for event studies, and even nondeterministic iteration orders in some processing environments. A governance-native pipeline defines a single run seed and uses it to seed every randomized component. This includes any pseudo-random operations in Python, NumPy, or other libraries. The seed is part of the run manifest and is printed in logs, not hidden in code.

One config to rule the run. Every run should be driven by a structured configuration object that includes:

- source selection (which feeds, which vendors, which tiers);
- time span, calendar, and decision grid (daily close, intraday bars, etc.);
- event ledger schema version and normalization rules;
- tokenization and lexicon rules;
- temporal aggregation windows and decay parameters;
- trading rule parameters (thresholds, holding periods, throttles);
- execution assumptions (delays, fill conventions).

The config is not just a convenience; it is the formal definition of the experiment. Without it, you cannot claim that two backtests are comparable.

Run manifests as immutable metadata. A run manifest binds together the identity of a run. It should include: run id, timestamp, git commit hash (or equivalent code fingerprint), config hash, input snapshot hashes, and output artifact hashes. The manifest is ideally stored

alongside outputs and is treated as immutable. If you change anything and rerun, you get a new manifest.

In governance terms, the manifest is the bridge between research and audit. When someone asks “how was this result produced?” the manifest is the first answer. It should make it possible to reproduce the entire run without guesswork.

Deterministic ordering and stable joins. Many event pipelines accidentally become nondeterministic because of ordering issues: iterating over dictionaries, merging sets of events without sorting, or depending on filesystem ordering. Governance requires that the pipeline impose explicit ordering: events sorted by `known_time` then by stable identifiers, instruments sorted deterministically, and joins implemented with clear tie-breaking rules. Stable ordering matters because it can change deduplication outcomes and therefore feature values, especially when events share identical timestamps.

Why determinism is non-negotiable. In event-driven pipelines, small differences can cascade. A slightly different deduplication cluster assignment can change novelty scores; novelty scores can change decayed sums; decayed sums can change threshold crossings; threshold crossings can change trades. If the pipeline is nondeterministic, you cannot even be sure that you are evaluating the same strategy twice. Therefore determinism is foundational.

15.12.2 Data lineage: immutable event logs, as-of snapshots, and hashes

Data lineage is the chain of custody for the dataset itself. In event-driven trading, the data is not a static truth. It is a moving target: vendors backfill, revise, correct, expand coverage, and change schemas. If you pull a dataset today and treat it as if it reflects what you would have known two years ago, you are likely leaking. Therefore data lineage must be built into the pipeline architecture.

Immutable raw ingestion logs. Raw ingestion is the only trustworthy record of what arrived and when your system saw it. Governance-native ingestion is append-only: new records are appended; old records are never overwritten. If a story is corrected, that correction is a new record. This preserves the historical information state and supports causal reconstruction.

Immutable logs should store the raw payload, source ids, ingestion timestamps, and content hashes. Content hashes are critical: they allow you to detect accidental corruption or silent modifications.

As-of snapshots: freezing the world. An as-of snapshot is a frozen view of the dataset as it existed at a specific time. In principle, live trading naturally produces as-of snapshots because you store what you saw. In research, especially when you rely on vendor archives, you must simulate this by either:

- storing periodic snapshots of vendor data and never retroactively changing them; or
- constructing a snapshot by filtering raw ingestion logs up to a snapshot time.

The snapshot is accompanied by a snapshot manifest that records snapshot time, included sources, backfill policy, and hashes of the included raw files.

Why is this so important? Because many vendor archives are not historical; they are edited histories. Without as-of snapshots, you cannot prove that your training set did not include items that were only added later.

Hashing and fingerprints as integrity proofs. Hashes serve two roles: integrity and identity. Integrity means you can verify that a file has not changed. Identity means you can refer to a dataset by its hash rather than by an ambiguous description. A governance-native pipeline computes hashes for:

- raw ingestion partitions (e.g., per day per source);
- snapshot bundles;
- canonical ledgers;
- feature dumps and decision traces.

These hashes are stored in manifests. The result is a lineage chain: outputs can be traced to inputs, and changes can be detected and attributed.

Entity dictionaries and calendars are also data. Event pipelines depend on auxiliary data: ticker mappings, corporate action histories, exchange calendars, timezone conversion rules, and lexicons. These must be versioned and hashed as well. If your entity dictionary changes, you have changed the dataset. If your calendar logic changes, you have changed which events are admissible at which decision times. Treating these as “just code” is an anti-pattern; they are data dependencies.

Lineage as a prerequisite for claims. A practical institutional standard is: you cannot publish a result internally without a lineage chain. If you cannot identify exactly which snapshot and which auxiliary versions produced the reported performance, the result is not decision-grade. This is not bureaucracy; it is a minimum standard for avoiding expensive mistakes.

15.12.3 Causality gates: admissible timestamps and “known-at” assertions

Causality gates are the operational enforcement of the filtration constraint. They are not descriptive statistics. They are hard stops. The pipeline should fail if it cannot guarantee that features and trades are built from admissible information.

Define `known_time` and treat it as law. Every event record must have a `known_time`—the earliest time at which the strategy is allowed to use that event. In a live system, `known_time` is typically `first_seen_time` plus a processing buffer. In research, `known_time` must be derived conservatively, and the derivation method must be recorded.

The causality gate is then simple: at decision time t , only events with `known_time` $\leq t$ are admissible. This must be enforced by code assertions, not by narrative.

Session-boundary admissibility. A special case of admissibility is session boundaries. Many strategies decide at the close. If an article arrives after the close, it is not admissible for the close decision, even if its calendar date is the same. Therefore the pipeline must classify events by market session (pre-market, regular, after-hours, closed) and map them to decision timestamps accordingly. A causality gate should verify that after-hours events never contribute to features for same-session close decisions.

Time zone checks as causality checks. Timezone errors are causality errors. If you misinterpret a local time as UTC, you can shift events earlier and leak. Therefore a causality test report must include timezone sanity checks: distributions of delays, frequency of events at implausible times, and cross-validation against known market session boundaries.

No-look-ahead rules for text-specific fields. Event data often includes vendor-added fields: categories, impact scores, sentiment tags, summaries. Many of these are assigned after the fact or updated over time. A governance-native pipeline must classify fields into:

- **raw fields** (available at ingestion time);
- **derived causal fields** (computed from raw fields using deterministic rules);
- **non-causal convenience fields** (potentially hindsight-contaminated).

Only the first two may be used for trading features unless you have explicit time-of-assignment metadata proving causality. This classification is itself an artifact, because it defines what your strategy is allowed to know.

Causality gates for cross-sectional ranking. Cross-sectional strategies introduce an additional leakage channel: contemporaneous ranking with late-arriving data. A causality gate should verify that the ranking at time t uses features computed as-of t for every asset. If some assets have no events by t , they must be treated as such. Filling in later news or using daily aggregated features violates causality and should fail tests.

Why gates must be hard. Soft governance (warnings) is not enough because leakage benefits performance. A warning can be ignored; a hard stop cannot. If your pipeline cannot pass causality gates, the correct outcome is not to continue and interpret results cautiously. The correct outcome is to fix the pipeline.

15.12.4 Parameter registry: windows, decay, lexicons, thresholds, delays

A parameter registry is a structured record of every knob in the pipeline that affects outputs. In event-driven trading, there are many knobs, and small changes can have large effects. Without a registry, you cannot reproduce runs, and you cannot know whether performance is robust or a single-point miracle.

Why parameters are especially dangerous in event pipelines. Event feature engineering involves window lengths, decay rates, novelty definitions, deduplication thresholds, tokenization rules, negation scopes, and more. Each parameter can be tuned to increase backtest performance. If you tune many parameters without proper discipline, you can easily overfit to calendar structure or vendor quirks. Therefore, the first governance step is to enumerate all parameters explicitly. You cannot govern what you do not name.

Minimum registry categories. Within Chapter 15 scope, the registry should include at least:

- **tokenization and normalization:** lowercase rules, punctuation policy, number normalization, phrase detectors;
- **lexicon sentiment:** lexicon version, word weights, negation window size, intensifier weights;
- **aggregation:** window sizes W , decay λ , clipping constants, novelty weighting rules;
- **deduplication:** dedup keys, similarity thresholds, clustering windows;
- **signal rules:** thresholds, smoothing parameters, hysteresis rules, holding periods;
- **execution assumptions:** decision-to-trade delay, fill convention, conservative buffers;
- **placebo tests:** shuffle scope (time buckets), randomization seeds, number of repetitions.

The registry is ideally machine-readable (e.g., JSON) and hashed, so that a run can be identified by a registry hash.

Parameter provenance and justification. In a mature workflow, each parameter has a provenance: why was this value chosen? Some are fixed by operational constraints (e.g., decision frequency). Some are set by prior evidence (event study half-life). Some are tuned, but tuning must be governed: define the search space, the validation scheme, and the selection criterion. Even if Chapter 15 does not implement full hyperparameter optimization, it should enforce that any parameter changes are recorded and that sensitivity analysis is run.

Sensitivity grids as robustness evidence. A strong governance practice is to define a sensitivity grid for key parameters (e.g., decay half-life, window length, thresholds) and run the backtest across that grid. If performance exists only at one precise setting, be skeptical. If performance is stable across a region, confidence increases. The grid itself becomes an artifact, stored in the robustness suite log.

15.12.5 Audit artifacts: event ledgers, feature dumps, decision traces

Audit artifacts are the tangible outputs that make the pipeline inspectable. They turn an abstract claim (“the strategy uses sentiment”) into a chain of evidence (“these events produced these features which produced these trades”).

Event ledger as the canonical dataset. The event ledger is the normalized, time-aware, instrument-mapped representation of events. It is the dataset that feature extraction operates on. At minimum it must include schema version, source ids, dedup keys, time fields (`event_time`, `published_time`, `first_seen_time`, `known_time`), and instrument mapping with confidence. The ledger should be hashable and reproducible from raw logs and configs.

The ledger is an audit artifact because it allows reviewers to inspect event distributions, spot anomalies, and verify time semantics. Without it, you are working with an implicit dataset that cannot be questioned.

Feature dumps: numbers with provenance. A feature dump is a snapshot of computed features at decision times. In price-only strategies, feature dumps are often optional. In event strategies, they are essential because features are derived from streams and can be difficult to debug. A feature dump should include:

- decision timestamps t ;
- asset ids;
- feature values (counts, decayed sentiment, novelty scores, etc.);
- references to contributing events (e.g., event ids or a hash of the contributing set).

Including contributing event references is critical. If a feature looks wrong, you need to know which events produced it.

Decision traces: explaining trades. A decision trace links features to signals to positions to trades. A trace sample (not necessarily the full trace for all times) should show, for representative periods:

- event inputs (ids and key fields);
- computed feature values at t ;
- signal value and any gating/throttle decisions;
- resulting position and trade time (including execution delay).

Decision traces are invaluable for two reasons. First, they detect logic bugs (e.g., trades happening before features exist). Second, they prevent narrative self-deception by forcing you to see what the system actually traded on.

Robustness suite log. A robustness suite log records all stress tests: time splits, source splits, placebo tests, sensitivity grids. It should store metrics, parameters, and run ids for each test. The goal is to make robustness a first-class artifact, not a paragraph in a report.

Why artifacts are part of the research result. In a governance-native approach, the “result” of a research run is not only the P&L curve. It is the set of artifacts that allow others (and your future self) to trust the curve. A backtest without artifacts is not research; it is a screenshot.

15.12.6 Minimal test suite: monotonic time, no leakage, reproducible joins

Finally, governance-native pipelines require tests. Not a massive unit test suite, but a minimal set of invariants that catch the highest-impact failures. For event-driven work, the minimal suite is centered on time, joins, and leakage.

Monotonic time invariants. Events must respect monotonicity in their time relationships. For example:

- `first_seen_time` should be $\geq$ `published_time` in most cases, unless the vendor semantics differ (and then it must be documented);
- `known_time` should be $\geq$ `first_seen_time` if a processing buffer is applied;
- decision-time admissibility: for any feature computed at t , all contributing events must satisfy `known_time` $\leq t$.

When these invariants fail, the pipeline should stop.

No leakage tests as explicit assertions. Leakage tests should include:

- **pre-event effect check:** event study pre-window returns should not show systematic “alpha” for unscheduled events; if they do, flag and investigate;
- **session-boundary checks:** after-hours events must not influence same-day close decisions;
- **field admissibility checks:** ensure that non-causal vendor fields are not used in features.

Some of these are statistical diagnostics rather than strict asserts, but the pipeline should at least produce a causality test report artifact and require human sign-off before proceeding.

Reproducible joins. Event-driven pipelines involve joins: events to instruments, events to sessions, features to decision times, signals to trades, and sometimes events to prices. Joins are a common source of silent leakage, especially when performed by calendar date or when duplicates create many-to-one merges.

A minimal join test suite includes:

- verify that event-to-decision-time mapping uses `known_time`, not calendar date;
- verify that joins are one-to-one where expected (e.g., each decision time per asset has one feature row);
- verify tie-breaking rules for duplicates (deterministic selection).

If a join produces unexpectedly many rows, that is often a sign of duplicate events being counted multiple times or of an instrument mapping ambiguity.

Placebo tests as governance tests. Placebos are not just research tools; they are governance tests. A minimal suite should include:

- randomized timestamp placebo within time buckets;
- shuffled text placebo within causal constraints;
- leave-one-source-out robustness checks.

The expectation is that true content-driven signals should degrade under text shuffles, and true timing-driven signals should degrade under timestamp shuffles. If they do not, interpretation must change.

Regression tests on artifacts. A practical governance technique is to treat artifact hashes as regression tests. If you rerun the pipeline with the same inputs and config and the output hashes differ, something changed. That may be acceptable if code changed, but it must be explainable. Artifact hash regression tests catch accidental nondeterminism and environment drift.

Taken together, this governance-native checklist defines what it means to do event-driven research within the discipline of this book. Determinism requires seeded randomness, centralized configs, stable ordering, and run manifests that bind code, data, and outputs. Data lineage requires immutable ingestion logs, as-of snapshots, and hashing of both primary and auxiliary datasets. Causality gates enforce admissible timestamps and session-aware decision rules through hard assertions, including special protections for cross-sectional ranking. Parameter registries enumerate and version every knob that could be tuned, enabling reproducibility and sensitivity analysis. Audit artifacts—event ledgers, feature dumps, decision traces, and robustness logs—turn backtests into inspectable evidence rather than opaque curves. And a minimal test suite enforces monotonic time, prevents leakage through joins and timing, and uses placebos as structural sanity checks.

The table below summarizes the minimum artifacts that should exist for any Chapter 15 research run. The deeper point is not the table itself but the philosophy behind it: event-driven trading is only credible when it is governable. If you can prove what was known when, reproduce your results, and audit your decisions, then you can begin to ask the interesting economic questions. Without that proof, any apparent “edge” is guilty until proven innocent.

15.13 Conclusion

Event-driven and sentiment-based trading sits at an awkward intersection of intuition and rigor. Intuition says that information moves markets, that headlines matter, and that those who interpret information faster should win. Rigor says that most of the market reaction to public news is fast, that archives are contaminated by revisions and selection, and that a backtest can accidentally “predict” the past by using timestamps or fields that were not knowable in real time. Chapter 15 has tried to resolve that tension in the only responsible way: by treating event-driven trading as a pipeline and governance problem first, and only then as a modeling

problem. The core deliverable is not a clever NLP trick; it is a causal event ledger, transparent feature engineering, time-aware evaluation, and a discipline for backtesting that can survive an adversarial audit.

The practical conclusion is therefore nuanced. There are things event features can reliably contribute. There are also things they cannot promise, no matter how elegant the narrative. And there is a minimum standard you must meet before you claim an “edge” rather than a dataset artifact.

15.13.1 What event-driven and sentiment features can reliably contribute

Event-driven features can reliably contribute *state information* about the market’s information environment. Even when directional prediction is weak, event flow measures often capture changes in risk conditions: clusters of news, bursts of novelty, spikes in uncertainty language, and periods where many assets experience simultaneous information arrivals. These states tend to correlate with volatility, gap risk, correlation regimes, and liquidity stress. In practice, this means event features are often most robust as risk-context variables: throttling exposure ahead of high-risk windows, gating price signals during information storms, and shaping position sizing to reflect elevated hazard. This is a powerful contribution because it improves the survivability of a trading system, not merely its average return.

Event features can also reliably provide *structure* for evaluation and research. By forcing you to define `known_time`, session mapping, and decision timestamps, event-driven work raises the bar for causal thinking. Even if a particular sentiment score does not predict returns, the event-study diagnostics and placebo tests can reveal whether a system is accidentally trading on look-ahead, calendar artifacts, or vendor quirks. In that sense, event-driven pipelines teach a discipline that generalizes: if you can build a causal event ledger, you can build more credible market data and feature pipelines in general.

Where event features sometimes contribute *directional* information is in narrow, well-defined contexts with measurable surprise and adequate latency slack. Scheduled events with known times (macro releases) and structured corporate events (earnings, guidance) can exhibit post-event drift or reversal patterns at horizons where you can plausibly trade. In these cases, the most reliable directional signals are usually not generic “sentiment” in the human sense, but specific, interpretable constructs: beat/miss language, guidance up/down phrases, downgrade/upgrade terminology, and novelty-weighted attention measures. When an effect is slow enough to survive conservative execution assumptions, it can be exploited; when it is immediate, it belongs to execution competition and microstructure.

Finally, event pipelines can reliably contribute *auditability*. If you store an immutable event log, produce canonical ledgers, dump features, and record decision traces, you gain a forensic ability that is rare in discretionary narratives. When performance changes, you can diagnose whether the change came from sources, calendars, lexicons, deduplication, or true market adaptation. This is operational alpha in the broader sense: the ability to maintain and evolve a system without drifting into unprovable story-telling.

15.13.2 What news and sentiment cannot guarantee in markets

News and sentiment cannot guarantee speed. The market often incorporates public headlines extremely quickly, especially in liquid instruments and around widely watched events. If your edge requires reacting within seconds or minutes, a research backtest built on bar data and vendor timestamps is likely overstating feasibility. This is not pessimism; it is a structural fact of modern markets. Chapter 18 will formalize this, but Chapter 15’s lesson is already clear: if your impact curve is mostly realized at the first tradable bar, your strategy is an execution problem, not a signal problem.

News and sentiment also cannot guarantee stability. Language drifts, vendor taxonomies change, and editorial styles evolve. Even if markets behave similarly, the mapping from text to numeric features can change enough to degrade performance. Retraining and recalibration are possible, but they introduce governance risk: you can accidentally chase noise or embed hindsight. Therefore, any claim of persistent sentiment edge must include evidence of robustness across time splits, source splits, and language eras, not just a single in-sample fit.

Most importantly, news and sentiment cannot guarantee *interpretability as truth*. It is easy to read a headline and tell a compelling story about why the market moved. But a backtest can “work” for reasons unrelated to semantic meaning: coverage intensity, source tier, time-of-day effects, or selection bias in the dataset. Without placebo tests (timestamp randomization, text shuffles) and confound checks (market/sector moves), semantic narratives are often post hoc. In markets, the human instinct to explain can be an adversary: it makes weak signals feel real.

Finally, news and sentiment cannot guarantee that an observed effect is tradable at scale. Event-driven signals tend to concentrate trading around bursts, creating turnover spikes and capacity constraints. If performance depends on trading aggressively during these bursts, real-world slippage can dominate. Therefore, even when a signal appears predictive, the economic value depends on execution realism and capacity, which must be tested conservatively.

15.13.3 A minimum standard for claiming an event-driven “edge”

Within this book’s discipline, claiming an event-driven “edge” requires more than a good equity curve. At minimum, you must meet three standards: causality, robustness, and economic plausibility.

Causality means you can prove “what was knowable when.” Concretely, you must have an immutable event log or an as-of snapshot with a manifest, a canonical event ledger with explicit `known_time`, and hard causality gates that prevent any post-decision events from entering features. Your event study must show no systematic pre-event effects for unscheduled events beyond what is plausibly attributable to real-world anticipation. If the strategy depends on vendor tags, summaries, or revised text without time-of-assignment proofs, it fails the causality standard.

Robustness means the edge survives splits that reflect real non-stationarity. At minimum, it should survive walk-forward time splits, leave-one-source-out tests, and placebo tests designed to destroy real information while preserving superficial structure. If shuffling text does not degrade a sentiment strategy, it is not a sentiment strategy. If randomizing timestamps does not degrade an event-timing strategy, it is not using event timing in a meaningful way. Robustness also includes

parameter sensitivity: the edge should exist across a region of reasonable window/decay/threshold settings, not only at a single finely tuned point.

Economic plausibility means the edge survives conservative execution assumptions and is consistent with measured half-life. If the half-life is shorter than your decision latency, the edge is not accessible at that frequency. You must show performance under delayed execution and realistic turnover constraints, and you must present metrics beyond Sharpe that reflect event clustering risk: conditional drawdowns during high-intensity periods, turnover spikes, and hit rates around events. Only after these conditions are met does it become reasonable to use the word “edge.”

15.13.4 Transition to Chapter 16: Portfolio Construction & Optimization

Chapter 15 has treated event-driven and sentiment features as causal objects: information arrivals with explicit timestamps, traceable provenance, and measurable half-lives. It has also treated evaluation as a decision-time simulation problem, with disciplined backtesting and governance-native artifacts. But signals are not portfolios. A signal is a noisy statement about expected return, risk, or hazard; a portfolio is a constrained allocation problem under uncertainty, correlation, transaction costs, and practical limits on leverage and turnover.

Chapter 16 therefore turns the pipeline outward: from features to forecasts, from forecasts to weights, and from weights to a portfolio that can be defended operationally and economically. It will formalize objective functions, constraints, and the translation rules that map multiple signals into a coherent allocation. The continuity with Chapter 15 is governance: reproducible inputs, explicit decision timing, and audit-ready artifacts that extend from event ingestion all the way to portfolio weights and trades. The promise is not that portfolio construction will magically rescue weak signals, but that it will provide the correct framework for using genuinely incremental signals—including event-driven ones—in a system that is stable, scalable, and honest about what it can and cannot know.